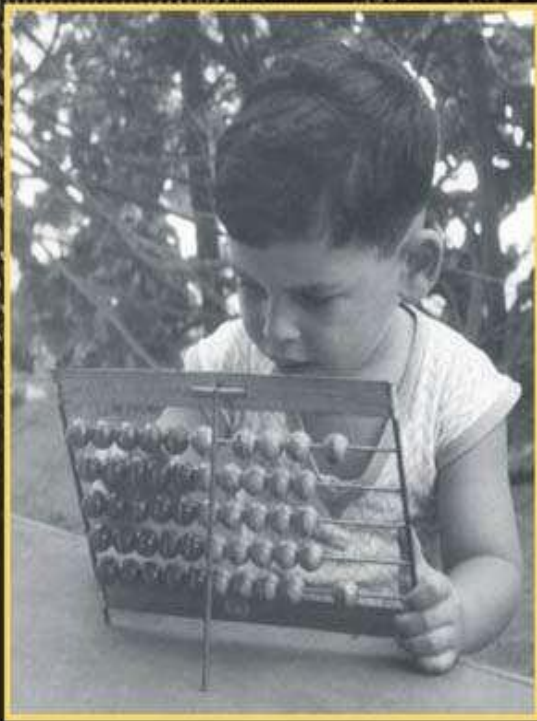


# THE ELEMENTS OF COMPUTING SYSTEMS

second edition



BUILDING A MODERN  
COMPUTER FROM  
FIRST PRINCIPLES

NOAM NISAN AND SHIMON SCHOCKEN

---

Noam Nisan y Shimon Schocken

---

# **Los elementos de los sistemas informáticos** Construir una computadora moderna a partir de los primeros principios

Segunda edición

La prensa del MIT  
Cambridge, Massachusetts  
Londres, Inglaterra

© Instituto Tecnológico de Massachusetts 2021

Todos los derechos reservados. Ninguna parte de este libro puede ser reproducida en ninguna forma por ningún medio electrónico o mecánico (incluyendo fotocopias, grabaciones o almacenamiento y recuperación de información) sin el permiso por escrito del editor.

Datos de catalogación en publicación de la Biblioteca del

Congreso Nombres: Nisan, Noam, autor. | Schocken,

Shimon, autor.

Título: Los elementos de los sistemas informáticos: construir un ordenador moderno a partir de los primeros principios / Noam

Nisan y Shimon Schocken.

Descripción: Segunda edición. | Cambridge, Massachusetts: The MIT Press, [2021] | Incluye índice.

Identificadores: LCCN 2020002671 | ISBN 9780262539807 (rústica)

Materias: LCSH: Ordenadores digitales electrónicos.

Clasificación: LCC TK7888.3. N57 2020 | DDC 004.16—dc23

Registro LC disponible en <https://lcn.loc.gov/2020002671>

d\_r0

*A nuestros padres,  
por enseñarnos que menos es más.*



# Contenido

## Prefacio

---

## **I    HARDWARE**

- 1    Lógica booleana**
- 2    Aritmética booleana**
- 3    Memoria**
- 4    Lenguaje de máquina**
- 5    Arquitectura de Computadores**
- 6    Ensamblador**

## **II   SOFTWARE**

- 7    Máquina virtual I: Procesamiento**
- 8    Máquina virtual II: Control**
- 9    Lenguaje de alto nivel**
- 10    Compilador I: Análisis de sintaxis**
- 11    Compilador II: Generación de código**
- 12    Sistema operativo**
- 13    Más diversión para llevar**

---

## Apéndices

- 1    Síntesis de funciones booleanas**
- 2    Lenguaje de descripción de hardware**
- 3    Lenguaje de descripción de pruebas**
- 4    El conjunto de chips Hack**

## 5 El conjunto de caracteres Hack

## 6 El índice de API

de Jack OS

# Lista de figuras

### Figura 1.1

Módulos principales de un sistema informático típico, que consta de una plataforma de hardware y una jerarquía de software. Cada módulo tiene una *vista abstracta* (también llamada interfaz del módulo) y una *implementación*. Las flechas que apuntan hacia la derecha significan que cada módulo se implementa utilizando bloques de construcción abstractos del nivel inferior. Cada círculo representa un proyecto y un capítulo de Nand a Tetris: doce proyectos y capítulos en total.

### Figura 1.1

Tres funciones booleanas elementales.

### Figura 1.2

Todas las funciones booleanas de dos variables binarias. En general, el número de funciones booleanas abarcadas por  $n$  variables binarias (aquí  $n = 2$ ) es  $2^{2^n}$  (eso es un montón de funciones booleanas).

### Figura 1.3

Tabla de verdad y definiciones funcionales de una función booleana (ejemplo).

### Figura 1.5

Implementación compuesta de una puerta Y de tres vías. El contorno discontinuo rectangular define el límite de la interfaz de puerta.

### Figura 1.6

Interfaz de puerta Xor (izquierda) y una posible implementación (derecha).

### Figura 1.7

Diagrama de puerta e implementación HDL de la función booleana Xor ( $a, b = \text{Or}(\text{Y}(a, \text{No}(b)), \text{Y}(\text{No}(a), b))$ ), utilizada como ejemplo. También se muestra un script de prueba y un archivo de salida generado por la prueba. Las descripciones detalladas de HDL y el lenguaje de prueba se dan en los apéndices 2 y 3, respectivamente.

### Figura 1.8

Una captura de pantalla de la simulación de un chip Xor en el simulador de hardware suministrado (otras versiones de este simulador pueden tener una GUI ligeramente diferente). El estado del simulador se muestra justo después de que el script de prueba haya terminado de ejecutarse. Los valores de los pines corresponden al último paso de la simulación ( $a=b=1$ ). No se muestra en esta captura de pantalla un *archivo de comparación* que enumera la salida esperada de la simulación especificada por este script de prueba en particular. Al igual que el script de prueba, el archivo de comparación generalmente lo proporciona el cliente que desea construir el chip. En este ejemplo concreto, el archivo de salida generado por la simulación (abajo a la derecha de la figura) es idéntico al archivo de comparación proporcionado.

### **Figura 1.9**

Multiplexor. La tabla en la parte superior derecha es una versión abreviada de la tabla de verdad.

### **Figura 1.10**

Demultiplexor.

### **Figura 2.1**

Representación complementaria de dos de números con signo, en un sistema binario de 4 bits.

### **Figura 2.2**

Medio vóbor, diseñado para agregar 2 bits.

### **Figura 2.3**

Sumador completo, diseñado para agregar 3 bits.

### **Figura 2.4**

Sumador de 16 bits, diseñado para sumar dos números de 16 bits, con un ejemplo de acción de suma (a la izquierda).

### **Figura 2.5a**

El Hack ALU, diseñado para calcular las dieciocho funciones aritmético-lógicas que se muestran a la derecha (los símbolos `!`, `&`, y `|` representan, respectivamente, las operaciones de 16 bits Not, And y Or). Por ahora, ignore los bits de salida `zr` y `ng`.

### **Figura 2.5b**

En conjunto, los valores de los seis bits de control `zx`, `nx`, `zy`, `ny`, `f` y `no` hacen que la ALU calcule una de las funciones enumeradas en la columna más a la derecha.

### **Figura 2.5c**

La API Hack ALU.

### **Figura 3.1**

La jerarquía de memoria construida en este capítulo.

### **Figura 3.2**

Representación de tiempo discreta: los cambios de estado (valores de entrada y salida) se observan solo durante las transiciones de ciclo. Dentro de los ciclos, los cambios se ignoran.

### **Figura 3.3**

El cambio de datos (arriba) y el ejemplo de comportamiento (abajo). En el primer ciclo, se desconoce la entrada anterior, por lo que la salida del DFF no está definida. En cada unidad de tiempo posterior, el DFF genera la entrada de la unidad de tiempo anterior. Siguiendo las convenciones de diagramación de puertas, la entrada del reloj está marcada por un pequeño triángulo, dibujado en la parte inferior del icono de la puerta.

### **Figura 3.4**

El diseño de lógica secuencial generalmente involucra puertas DFF que se alimentan y se conectan a chips combinados. Esto le da a los chips secuenciales la capacidad de responder a la corriente, así como a las entradas y salidas anteriores.

### **Figura 3.5**

Registro de 1 bit. Almacena y emite un valor de 1 bit hasta que se le indica que cargue un nuevo valor.

### **Figura 3.6**

Registro de 16 bits. Almacena y emite un valor de 16 bits hasta que se le indica que cargue un nuevo valor.

### **Figura 3.7**

Un chip RAM, que consta de  $n$  chips de registro de 16 bits que se pueden seleccionar y manipular por separado.

Las direcciones de registro (0 to  $n - 1$ ) no forman parte del hardware del chip. Más bien, se realizan mediante una implementación de lógica de puerta que se discutirá en la siguiente sección.

#### **Figura 3.8**

Contador de programa (PC): Para usarlo correctamente, como máximo se debe afirmar uno de los bits de carga, inc o reinicio.

#### **Figura 3.9**

La implementación de Bit (registro de 1 bit): soluciones no válidas (izquierda) y correctas (derecha).

#### **Figura 4.2**

Modelo conceptual del sistema de memoria Hack. Aunque la arquitectura real está cableada de manera algo diferente (como se describe en el capítulo 5), este modelo ayuda a comprender la semántica de los programas Hack.

#### **Figura 4.3**

Ejemplos de código ensamblador de hackeo.

#### **Figura 4.4**

Un programa de ensamblaje Hack (ejemplo). Tenga en cuenta que RAM[0] y RAM[1] se pueden denominar R0 y R1.

#### **Figura 4.5**

El conjunto de instrucciones Hack, que muestra mnemotécnicos simbólicos y sus correspondientes códigos binarios.

#### **Figura 4.6**

Un programa ensamblador de Hack que calcula una expresión aritmética simple.

#### **Figura 4.7**

Ejemplo de procesamiento de matrices, utilizando el acceso basado en punteros a los elementos de matriz.

#### **Figura 4.8**

El emulador de CPU, con un programa cargado en la memoria de instrucciones (ROM) y algunos datos en la memoria de datos (RAM). La figura muestra una instantánea tomada durante la ejecución del programa.

#### **Figura 5.1**

Una arquitectura de computadora genérica de von Neumann.

#### **Figura 5.2**

La interfaz de la unidad central de procesamiento (CPU) de Hack.

#### **Figura 5.3**

La interfaz de memoria de instrucciones Hack.

#### **Figura 5.4**

La interfaz del chip Hack Screen.

#### **Figura 5.5**

La interfaz del chip Hack Keyboard.

#### **Figura 5.6**

La interfaz de memoria de datos Hack. Tenga en cuenta que los valores decimales 16384 y 24576 son 4000 y 6000 en hexadecimal.

#### **Figura 5.7**

Interface of Computer, el chip más alto de la plataforma de hardware Hack.

### Figura 5.8

Implementación propuesta de la CPU Hack, que muestra una instrucción entrante de 16 bits. Usamos la notación de instrucciones *cccccccccccccc* para enfatizar que en el caso de una *instrucción C*, la instrucción se trata como una cápsula de bits de control, diseñada para controlar diferentes partes del chip de la CPU. En este diagrama, cada *símbolo c* que ingresa a una parte del chip representa algún bit de control extraído de la instrucción (en el caso de la ALU, la entrada de *c* representa los seis bits de control que le indican a la ALU qué calcular). En conjunto, el comportamiento distribuido inducido por estos bits de control termina ejecutando la instrucción. No especificamos qué bits van a dónde, ya que queremos que los lectores respondan estas preguntas por sí mismos.

### Figura 5.9

Propuesta de implementación de **Computer**, el chip más alto de la plataforma Hack.

### Figura 5.10

Probar el chip de computadora en el simulador de hardware suministrado. El programa almacenado es **Rect**, que dibuja un rectángulo de filas de RAM [0] de 16 píxeles cada una, todas negras, en la parte superior izquierda de la pantalla.

### Figura 6.1

Código ensamblador, traducido a código binario mediante una tabla de símbolos. Los números de línea, que no forman parte del código, se enumeran como referencia.

### Figura 6.2

El conjunto de instrucciones Hack, que muestra mnemotécnicos simbólicos y sus correspondientes códigos binarios.

### Figura 6.3

Probar la salida del ensamblador con el ensamblador suministrado.

### Figura II.1

Manipulación de puntos en un plano: ejemplo y código Jack.

### Figura II.2

Jack implementación de la abstracción de punto.

### Figura II.3

Mapa de carreteras de la parte II (el ensamblador pertenece a la parte I y se muestra aquí para completarlo). La hoja de ruta describe una jerarquía de traducción, desde un programa multiclase de alto nivel, basado en objetos hasta código de máquina virtual, código ensamblador y código binario ejecutable. Los círculos numerados representan los proyectos que implementan el compilador, el traductor de VM, el ensamblador y el sistema operativo. El Proyecto 9 se centra en escribir una aplicación de Jack para familiarizarse con el lenguaje.

### Figura 7.1

El marco de la máquina virtual, usando Java como ejemplo. Los programas de alto nivel se compilan en código de máquina virtual intermedio. El mismo código de VM se puede enviar y ejecutar en cualquier plataforma de hardware equipada con una implementación de *JVM adecuada*. Estas implementaciones de VM generalmente se realizan como programas del lado del cliente que traducen el código de VM a los lenguajes de máquina de los dispositivos de destino.

### Figura 7.2

Ejemplo de procesamiento de pila, que ilustra las dos operaciones elementales **push** y **pop**. La configuración consta de dos estructuras de datos: un segmento de memoria similar a la RAM y una pila. Siguiendo la convención, la pila se dibuja como si creciera hacia abajo. La ubicación justo después del valor superior de la pila se denomina un puntero llamado *sp* o *puntero de pila*. Los símbolos *x* e *y* se refieren a dos ubicaciones de memoria arbitrarias.

### **Figura 7.3a**

Evaluación basada en pila de expresiones aritméticas.

### **Figura 7.3b**

Evaluación basada en pila de expresiones lógicas.

### **Figura 7.4**

Segmentos de memoria virtual.

### **Figura 7.5**

Los comandos aritmético-lógicos del lenguaje VM.

### **Figura 7.6**

El emulador de VM suministrado con el paquete de software Nand to Tetris.

### **Figura 8.1**

Acción de comandos de bifurcación. (El código de la máquina virtual de la derecha usa nombres de variables simbólicas en lugar de segmentos de memoria virtual para que sea más legible).

### **Figura 8.2**

Instantáneas en tiempo de ejecución de los estados de pila y segmento seleccionados durante la ejecución de un programa de tres funciones. Los números de línea no forman parte del código y se dan solo como referencia.

### **Figura 8.3**

La pila global, que se muestra cuando se ejecuta el destinatario. Antes de que finalice el destinatario, inserta un valor devuelto en la pila (no se muestra). Cuando la implementación de la máquina virtual controla el comando `return`, copia el valor devuelto en el argumento 0 y establece `SP` para que apunte a la dirección que le sigue. Esto libera efectivamente el área de pila global por debajo del nuevo valor de `SP`. Por lo tanto, cuando el autor de la llamada reanuda su ejecución, ve el valor devuelto en la parte superior de su pila de trabajo.

### **Figura 8.4**

Varias instantáneas de la pila *global*, tomadas durante la ejecución de la función principal, que llama a `factorial` para calcular  $3!$ . La función en ejecución solo ve su pila de trabajo, que es el área sin sombreado en la punta de la pila global; Las otras áreas sin sombrear en la pila global son las pilas de trabajo de funciones en la cadena de llamadas, esperando que regrese la función que se está ejecutando actualmente. Tenga en cuenta que las áreas sombreadas no están "dibujadas a escala", ya que cada cuadro consta de cinco palabras, como se muestra en [la figura 8.3](#).

### **Figura 8.5**

Implementación de los comandos de función del lenguaje VM. Todas las acciones descritas a la derecha se realizan mediante instrucciones de ensamblaje Hack generadas.

### **Figura 8.6**

Las convenciones de nomenclatura descritas anteriormente están diseñadas para admitir la traducción de varios archivos `.vm` y funciones en un único archivo `.asm`, lo que garantiza que los símbolos de ensamblado generados sean únicos dentro del archivo.

### **Figura 9.1**

*Hello World*, escrito en el idioma Jack.

### **Figura 9.2**

Programación procedimental típica y manejo simple de matrices. Utiliza los servicios de las clases del sistema operativo

Matriz, teclado y salida.

### Figura 9.3a

Fraction (arriba) y clase Jack de muestra que lo usa para crear y manipular objetos Fraction.

### Figura 9.3b

Una implementación de Jack de la **abstracción** de fracciones.

### Figura 9.4

Implementación de lista enlazada en Jack (izquierda y arriba a la derecha) y uso de muestra (abajo a la derecha).

### Figura 9.5

Servicios del sistema operativo (resumen). La API completa del sistema operativo se proporciona en el apéndice 6.

### Figura 9.6

Los elementos de sintaxis del lenguaje Jack.

### Figura 9.7

Tipos de variables en el lenguaje Jack. A lo largo de la tabla, la *subrutina* se refiere a una *función*, una *o* un *constructor*.

### Figura 9.8

Declaraciones en el idioma de Jack.

### Figura 9.9

Capturas de pantalla de las aplicaciones Jack ejecutándose en la computadora Hack.

### Figura 10.1

Plan de desarrollo por etapas del compilador Jack.

### Figura 10.2

Definición del léxico de Jack y análisis léxico de una entrada de muestra.

### Figura 10.3

Un subconjunto de la gramática del lenguaje Jack y segmentos de código Jack que son aceptados o rechazados por la gramática.

### Figura 10.4a

Árbol de análisis de un segmento de código típico. El proceso de análisis está impulsado por las reglas gramaticales.

### Figura 10.4b

Mismo árbol de análisis, en XML.

### Figura 10.5

La gramática de Jack.

### Figura 11.1

La clase Point. Esta clase presenta todos los tipos de variables posibles (**campo**, **estático**, **local** y **argumento**) y tipos de subrutinas (**constructor**, **método** y **función**), así como subrutinas que devuelven tipos primitivos, tipos de objeto y subrutinas void. También ilustra las llamadas a funciones, las llamadas a constructores y las llamadas a métodos en el objeto actual (**this**) y en otros objetos.

### Figura 11.2

Ejemplos de tablas de símbolos. Esta fila en la tabla de nivel de subrutina se analiza más adelante en el capítulo.

### Figura 11.3

Representaciones infijas y postfijas de la misma semántica.



#### **Figura 11.4**

Un algoritmo de generación de código de máquina virtual para expresiones y un ejemplo de compilación. El algoritmo asume que la expresión de entrada es válida. La implementación final de este algoritmo debe reemplazar las variables simbólicas emitidas con sus correspondientes asignaciones de tabla de símbolos.

#### **Figura 11.5**

Expresiones en el idioma Jack.

#### **Figura 11.6**

Compilar declaraciones `if` y `while`. Las etiquetas L1 y L2 son generadas por el compilador.

#### **Figura 11.7**

Construcción de objetos desde la perspectiva de la persona que llama. En este ejemplo, el autor de la llamada declara dos variables de objeto y, a continuación, llama a un constructor de clase para construir los dos objetos. El constructor hace su magia, asignando bloques de memoria para representar los dos objetos. El código de llamada hace que las dos variables de objeto se refieran a estos bloques de memoria.

#### **Figura 11.8**

Construcción de objetos: la perspectiva del constructor.

#### **Figura 11.9**

Compilación de llamadas a métodos: la perspectiva del autor de la llamada.

#### **Figura 11.10**

Métodos de compilación: la perspectiva del destinatario.

#### **Figura 11.11**

Acceso a matrices mediante comandos de VM.

#### **Figura 11.12**

Estrategia básica de compilación para arrays, y un ejemplo de los bugs que puede generar. En este caso particular, se anula el valor almacenado en el puntero 1 y se pierde la dirección de `a[i]`.

#### **Figura 12.1**

Algoritmo de multiplicación.

#### **Figura 12.2**

Algoritmo de división.

#### **Figura 12.3**

Algoritmo de raíz cuadrada.

#### **Figura 12.4**

Conversiones de enteros de cadena. (`appendChar`, `length` y `charAt` son métodos de clase `String`).

#### **Figura 12.5a**

Algoritmo de asignación de memoria (básico).

#### **Figura 12.5b**

Algoritmo de asignación de memoria (mejorado).

#### **Figura 12.6**

Dibujar un píxel.

#### **Figura 12.7**

Algoritmo de dibujo lineal: versión básica (abajo, izquierda) y versión mejorada (abajo, derecha).

**Figura 12.8**

Algoritmo de dibujo de círculos.

**Figura 12.9**

Ejemplo de un mapa de bits de caracteres.

**Figura 12.10**

Manejo de la entrada desde el teclado.

**Figura 12.11**

Una trampilla que permite el control completo de la RAM del host desde Jack.

**Figura 12.12**

Vista lógica (izquierda) e implementación física (derecha) de una lista vinculada que admite la asignación dinámica de memoria.

**Figura A1.1**

Sintetizar una función booleana a partir de una tabla de verdad (ejemplo).

**Figura A2.1**

Ejemplo de programa HDL.

**Figura A2.2**

Autobuses en acción (ejemplo).

**Figura A2.3**

Ejemplo de definición de chip incorporado.

**Figura A2.4**

Definición de DFF .

**Figura A2.5**

Un chip que activa piezas de chip potenciadas por GUI.

**Figura A2.6**

Demostración de chips potenciados por GUI. Dado que el programa HDL cargado utiliza piezas de chip habilitadas para GUI (paso 1), el simulador renderiza sus respectivas imágenes de GUI (paso 2). Cuando el usuario cambia los valores de los pines de entrada del chip (paso 3), el simulador refleja estos cambios en las respectivas GUI (paso 4).

**Figura A3.1**

Script de prueba y archivo de comparación (ejemplo).

**Figura A3.2**

Variables y métodos de chips clave incorporados en Nand a Tetris.

**Figura A3.3**

Probando el chip de computadora superior.

**Figura A3.4**

Probar un programa de lenguaje de máquina en el emulador de CPU.

**Figura A3.5**

Prueba de un programa de VM en el emulador de VM.

# Prefacio

Lo que oigo, lo olvido; Lo que veo, lo recuerdo; Lo que hago, lo entiendo.

—Confucio (551-479 a.C.)

Se argumenta comúnmente que las personas ilustradas del siglo XXI deberían familiarizarse con las ideas clave que subyacen a BANG: bits, átomos, neuronas y genes. Aunque la ciencia ha tenido un éxito notable en el descubrimiento de sus sistemas operativos básicos, es muy posible que nunca comprendamos completamente cómo funcionan realmente los átomos, las neuronas y los genes. Sin embargo, los bits y los sistemas informáticos en general implican una excepción consoladora: a pesar de su fantástica complejidad, uno puede comprender completamente cómo funcionan las computadoras modernas y cómo se construyen. Entonces, mientras miramos con asombro el BANG que nos rodea, es agradable pensar que al menos un campo de este cuarteto puede quedar completamente al descubierto para la comprensión humana.

De hecho, en los primeros días de las computadoras, cualquier persona curiosa que se preocupara por hacerlo podía obtener una comprensión gestáltica de cómo funciona la máquina. Las interacciones entre el hardware y el software eran lo suficientemente simples y transparentes como para producir una imagen coherente de las operaciones de la computadora. Por desgracia, a medida que las tecnologías digitales se han vuelto cada vez más complejas, esta claridad casi se pierde: las ideas y técnicas más fundamentales de la informática

—la esencia misma del campo— ahora están ocultos bajo muchas capas de interfaces oscuras e implementaciones propietarias. Una consecuencia inevitable de esta complejidad ha sido la especialización: el estudio de la informática aplicada se convirtió en una búsqueda de muchos cursos de nicho, cada uno de los cuales cubría un solo aspecto del campo.

Escribimos este libro porque sentimos que muchos estudiantes de informática están perdiendo el bosque por los árboles. El alumno típico es ordenado a través de

una serie de cursos de programación, teoría e ingeniería, sin detenerse a apreciar la belleza de la imagen en general. Y la imagen en general es tal que el hardware, el software y los sistemas de aplicaciones están estrechamente interrelacionados a través de una red oculta de abstracciones, interfaces e implementaciones basadas en contratos.

No ver esta intrincada empresa en carne y hueso deja a muchos estudiantes y profesionales con la incómoda sensación de que, bueno, no entienden completamente lo que sucede dentro de las computadoras. Esto es lamentable, ya que las computadoras son las máquinas más importantes del siglo XXI.

Creemos que la mejor manera de entender cómo funcionan las computadoras es construir una desde cero. Con eso en mente, se nos ocurrió la siguiente idea: especifiquemos un sistema informático simple pero suficientemente poderoso e invitemos a los alumnos a construir su plataforma de hardware y jerarquía de software desde cero. Y mientras estamos en eso, hagámoslo bien. Decimos esto porque construir una computadora de propósito general a partir de los primeros principios es una empresa enorme.

Por lo tanto, identificamos una oportunidad educativa única no solo para construir la cosa, sino también para ilustrar, de manera práctica, cómo planificar y administrar de manera efectiva proyectos de desarrollo de hardware y software a gran escala. Además, buscamos demostrar la emoción de construir, a través de un razonamiento cuidadoso y una planificación modular, sistemas fantásticamente complejos y útiles desde los primeros principios.

El resultado de este esfuerzo se convirtió en lo que ahora se conoce coloquialmente como *Nand to Tetris*: un viaje práctico que comienza con la puerta lógica más elemental, llamada Nand, y termina, doce proyectos después, con un sistema informático de propósito general capaz de ejecutar Tetris, así como cualquier otro programa que se le ocurra. Después de diseñar, construir, rediseñar y reconstruir el sistema informático varias veces nosotros mismos, escribimos este libro, explicando cómo cualquier alumno puede hacer lo mismo. También lanzamos el [sitio web de www.nand2tetris.org](http://www.nand2tetris.org), poniendo todos nuestros materiales de proyecto y herramientas de software a disposición de cualquier persona que quiera aprender o enseñar cursos de Nand a Tetris.

Nos complace decir que la respuesta ha sido abrumadora. Hoy en día, los cursos de Nand to Tetris se imparten en numerosas universidades, escuelas secundarias, campamentos de entrenamiento de codificación, plataformas en línea y clubes de hackers de todo el mundo. El libro y nuestros cursos en

línea se hicieron muy populares, y miles de

los estudiantes, que van desde estudiantes de secundaria hasta ingenieros de Google, publican rutinariamente reseñas que describen Nand a Tetris como su mejor experiencia educativa. Como dijo Richard Feynman: "Lo que no puedo crear, no lo entiendo". Nand to Tetris tiene que ver con la comprensión a través de la creación. Aparentemente, las personas se conectan apasionadamente con esta mentalidad de creador.

Desde la publicación de la primera edición del libro, recibimos numerosas preguntas, comentarios y sugerencias. A medida que abordamos estos problemas modificando nuestros materiales en línea, se desarrolló una brecha entre las versiones basadas en la web y en libros de Nand a Tetris. Además, sentimos que muchas secciones de libros podrían beneficiarse de una mayor claridad y una mejor organización. Entonces, después de retrasar esta cirugía tanto como pudimos, decidimos arremangarnos y escribir una segunda edición, lo que nos llevó al presente libro. El resto de este prefacio describe esta nueva edición, terminando con una sección que la compara con la anterior.

---

## Alcance

El libro expone a los alumnos a un cuerpo significativo de conocimientos informáticos, adquiridos a través de una serie de tareas de construcción de hardware y software. En particular, los siguientes temas se ilustran en el contexto de proyectos prácticos:

- *Hardware*: Aritmética booleana, lógica combinacional, lógica secuencial, diseño e implementación de puertas lógicas, multiplexores, flip-flops, registros, unidades RAM, contadores, lenguaje de descripción de hardware (HDL), simulación de chips, verificación y pruebas.
- *Arquitectura*: diseño e implementación de ALU / CPU, relojes y ciclos, modos de direccionamiento, lógica de recuperación y ejecución, conjunto de instrucciones, entrada / salida mapeada en memoria.
- *Lenguajes de bajo nivel*: Diseño e implementación de un lenguaje de máquina simple (binario y simbólico), conjuntos de instrucciones, programación ensambladora, ensambladores.

- *Máquinas virtuales*: Autómatas basados en pilas, aritmética de pilas, llamada y devolución de funciones, manejo de recursividad, diseño e implementación de un lenguaje de VM simple.
- *Lenguajes de alto nivel*: Diseño e implementación de un lenguaje simple basado en objetos, similar a Java: tipos de datos abstractos, clases, constructores, métodos, reglas de alcance, sintaxis y semántica, referencias.
- *Compiladores*: Análisis léxico, análisis sintáctico, tablas de símbolos, generación de código, implementación de matrices y objetos, compilación de dos niveles.
- *Programación*: implementación de un ensamblador, una máquina virtual y un compilador, siguiendo las API proporcionadas. Se puede hacer en cualquier lenguaje de programación.
- *Sistemas operativos*: Diseño e implementación de gestión de memoria, biblioteca matemática, controladores de entrada/salida, procesamiento de cadenas, salida textual, salida gráfica, soporte de lenguaje de alto nivel.
- *Estructuras de datos y algoritmos*: pilas, tablas hash, listas, árboles, algoritmos aritméticos, algoritmos geométricos, consideraciones de tiempo de ejecución.
- *Ingeniería de software*: diseño modular, paradigma de interfaz/implementación, diseño y documentación de API, pruebas unitarias, planificación proactiva de pruebas, control de calidad, programación en general.

Una característica única de Nand to Tetris es que todos estos temas se presentan de manera cohesiva, con un propósito claro y general: construir un sistema informático moderno desde cero. De hecho, este ha sido nuestro criterio de selección de temas: el libro se centra en el conjunto mínimo de temas necesarios para construir un sistema informático de propósito general, capaz de ejecutar programas escritos en un lenguaje de alto nivel basado en objetos. Resulta que este conjunto crítico incluye la mayoría de los conceptos y técnicas fundamentales, así como algunas de las ideas más hermosas, en informática aplicada.

---

## Cursos

Los cursos de Nand a Tetris generalmente se enumeran para estudiantes de pregrado y posgrado, y son muy populares entre los autodidactas. Los

cursos basados en este libro son "perpendiculares" a la típica informática



y se puede tomar en casi cualquier momento durante el programa. Dos espacios naturales son CS-2, un curso introductorio pero posterior a la programación, y CS-99, un curso de síntesis que llega al final del programa. El primer curso implica una introducción orientada al futuro y orientada a los sistemas a la informática aplicada, mientras que el segundo es un curso integrador basado en proyectos que llena los vacíos dejados por cursos anteriores.

Otro espacio cada vez más popular es un curso que combina, en un marco, temas clave de los cursos tradicionales de arquitectura de computadoras y cursos de compilación. Cualquiera que sea el propósito para el que están hechos, los cursos de Nand a Tetris tienen muchos nombres, incluidos Elementos de sistemas informáticos, Construcción de sistemas digitales, Organización de computadoras, Construyamos una computadora y, por supuesto, Nand a Tetris.

El libro y los proyectos son altamente modulares, comenzando desde la división superior en la Parte I: Hardware y la Parte II: Software, cada uno de los cuales comprende seis capítulos y seis proyectos. Aunque recomendamos pasar por la experiencia completa, es completamente posible aprender cada una de las dos partes por separado. El libro y los proyectos pueden admitir dos cursos independientes, cada uno de seis a siete semanas de duración, un curso semestral típico o dos cursos semestrales, según la selección del tema y el ritmo de estudio.

El libro es completamente autónomo: todos los conocimientos necesarios para construir los sistemas de hardware y software descritos en el libro se dan en sus capítulos y proyectos. Parte I: El hardware no requiere conocimientos previos, lo que hace que los proyectos 1-6 sean accesibles para cualquier estudiante y autodidacta. Parte II: Software y proyectos 7-12 requieren programación (en cualquier lenguaje de alto nivel) como requisito previo.

Los cursos de Nand a Tetris no se limitan a las carreras de informática. Más bien, se prestan a estudiantes de cualquier disciplina que buscan obtener una comprensión práctica de arquitecturas de hardware, sistemas operativos, compilación e ingeniería de software, todo en un solo curso. Una vez más, el único requisito previo (para la parte II) es la programación. De hecho, muchos estudiantes de Nand to Tetris no son estudiantes que tomaron un curso de introducción a la informática y ahora desean aprender más ciencias de la computación sin comprometerse con un programa de varios cursos. Muchos otros estudiantes son desarrolladores de software que desean "ir por debajo", comprender cómo funcionan las tecnologías habilitadoras y convertirse en mejores programadores de alto nivel.

Tras la aguda escasez de desarrolladores en las industrias de hardware y software, existe una creciente demanda de programas compactos y enfocados en informática aplicada. Estos a menudo toman la forma de campamentos de entrenamiento de codificación y grupos de cursos en línea diseñados para preparar a los estudiantes para el mercado laboral sin pasar por toda la gama de un título académico. Cualquier programa sólido de este tipo debe ofrecer, como mínimo, conocimientos prácticos de programación, algoritmos y sistemas. Nand to Tetris está en una posición única para cubrir el elemento de sistemas de dichos programas, en el marco de un curso. Además, los proyectos Nand to Tetris proporcionan un medio atractivo para sintetizar y poner en práctica gran parte del conocimiento algorítmico y programático aprendido en otros cursos.

---

## Recursos

Todas las herramientas necesarias para construir los sistemas de hardware y software descritos en el libro se suministran gratuitamente en el paquete de software Nand to Tetris. Estos incluyen un simulador de hardware, un emulador de CPU, un emulador de VM (todo en código abierto), tutoriales y versiones ejecutables del ensamblador, la máquina virtual, el compilador y el sistema operativo descritos en el libro. Además, la [web de `www.nand2tetris.org`](http://www.nand2tetris.org) incluye todos los materiales del proyecto —alrededor de doscientos programas de prueba y scripts de prueba— que permiten el desarrollo incremental y las pruebas unitarias de cada uno de los doce proyectos. Las herramientas de software y los materiales del proyecto se pueden usar tal cual en cualquier computadora con Windows, Linux o macOS.

---

## Estructura

Parte I: Hardware consta de los capítulos 1 a 6. Después de una introducción al álgebra booleana, el capítulo 1 comienza con la puerta Nand elemental y construye un conjunto de puertas lógicas elementales sobre ella. El capítulo 2 presenta la lógica combinacional y construye un conjunto de sumadores, que conducen a una ALU. El capítulo 3 presenta la lógica secuencial y construye un conjunto de registros y dispositivos de memoria, que conducen a una RAM. El capítulo 4 analiza la programación de bajo nivel y especifica un lenguaje de máquina tanto en su forma simbólica

como binaria.

El capítulo 5 integra los chips integrados en los capítulos 1-3 en una arquitectura de hardware capaz de ejecutar programas escritos en el lenguaje de máquina presentado en el capítulo 4. El capítulo 6 analiza la traducción de programas de bajo nivel, que culmina en la construcción de un ensamblador.

Parte II: El software consta de los capítulos 7 a 12 y requiere experiencia en programación (en cualquier lenguaje) a nivel de introducción a los cursos de informática. Los capítulos 7 y 8 presentan autómatas basados en pilas y describen la construcción de una máquina virtual similar a JVM. El capítulo 9 presenta un lenguaje de alto nivel basado en objetos, similar a Java. Los capítulos 10 y 11 analizan los algoritmos de análisis y generación de código y describen la construcción de un compilador de dos niveles. El capítulo 12 presenta varios algoritmos de gestión de memoria, algebraicos y geométricos y describe la implementación de un sistema operativo que los pone en práctica. El sistema operativo está diseñado para cerrar las brechas entre el lenguaje de alto nivel implementado en la parte II y la plataforma de hardware construida en la parte I.

El libro se basa en un paradigma *de abstracción-implementación*. Cada capítulo comienza con una introducción que describe conceptos relevantes y un sistema genérico de hardware o software. La siguiente sección es siempre Especificación, describiendo la abstracción del sistema, es decir, los diversos servicios que se espera que entregue, de una forma u otra. Una vez presentado el *qué*, cada capítulo procede a discutir *cómo* se puede realizar la abstracción, lo que lleva a una sección de implementación propuesta. La siguiente sección es siempre Proyecto, que proporciona pautas paso a paso, materiales de prueba y herramientas de software para construir y probar unitariamente el sistema descrito en el capítulo. La sección final de Perspectiva destaca temas notables que se omiten en el capítulo.

---

## Proyectos

El sistema informático descrito en el libro es *real*. Está diseñado para ser construido, ¡y funciona! El libro está dirigido a lectores activos que están dispuestos a ensuciarse las manos y construir la computadora desde cero. Si se toma el tiempo y el esfuerzo para hacerlo, obtendrá una profundidad de comprensión y una sensación de logro inigualable por la mera lectura.

Los dispositivos de hardware integrados en los proyectos 1, 2, 3 y 5 se implementan utilizando un lenguaje de descripción de hardware (HDL) simple y se simulan en un simulador de hardware basado en software suministrado, que es exactamente como funcionan los arquitectos de hardware en la industria. Los proyectos 6, 7, 8, 10 y 11 (ensamblador, máquina virtual I+II y compilador) I+II se pueden escribir en cualquier lenguaje de programación. El proyecto 4 está escrito en el lenguaje ensamblador de la computadora, y los proyectos 9 y 12 (un simple juego de computadora y un sistema operativo básico) están escritos en Jack, el lenguaje de alto nivel similar a Java para el que construimos un compilador en los capítulos 10 y 11.

Hay doce proyectos en total. En promedio, cada proyecto implica una carga semanal de tareas en un curso típico de nivel universitario riguroso. Los proyectos son autónomos y se pueden hacer (u omitir) en cualquier orden deseado. La experiencia completa de Nand to Tetris implica hacer todos los proyectos en su orden de aparición, pero esta es solo una opción.

¿Es posible cubrir tanto terreno en un curso de un semestre? La respuesta es sí, y la prueba está en el pudín: más de 150 universidades imparten cursos semestrales de Nand a Tetris. La satisfacción de los estudiantes es excepcional, y los MOOC de Nand a Tetris se enumeran rutinariamente en la parte superior de las listas mejor calificadas de cursos en línea. Una de las razones por las que los alumnos responden a nuestra metodología es *el enfoque*. Salvo casos obvios, no prestamos atención a la optimización, dejando este importante tema a otros cursos más específicos. Además, permitimos que los estudiantes asuman entradas sin errores. Esto elimina la necesidad de escribir código para controlar las excepciones, lo que hace que los proyectos de software sean significativamente más centrados y manejables. Lidar con la entrada incorrecta es, por supuesto, de vital importancia, pero esta habilidad se puede perfeccionar en otros lugares, por ejemplo, en extensiones de proyectos y en cursos dedicados a programación y diseño de software.

---

## La segunda edición

Aunque Nand to Tetris siempre se estructuró en torno a dos temas, la segunda edición hace explícita esta estructura: el libro ahora está dividido en dos partes distintas e independientes, Parte I: Hardware y Parte II: Software. Cada parte consta de seis capítulos y seis proyectos y comienza con una introducción recién escrita que prepara el escenario para los

capítulos de la parte. Importantemente

las dos partes son independientes entre sí. Por lo tanto, la nueva estructura del libro se presta bien a cursos de un trimestre y de un semestre.

Además de los dos nuevos capítulos de introducción, la segunda edición presenta cuatro nuevos apéndices. Siguiendo las solicitudes de muchos estudiantes, estos nuevos apéndices ofrecen presentaciones enfocadas de varios temas técnicos que, en la primera edición, estaban dispersos en los capítulos. Otro nuevo apéndice proporciona una prueba formal de que cualquier función booleana se puede construir a partir de operadores Nand, agregando una perspectiva teórica a los proyectos de construcción de hardware aplicados. Se agregaron muchas secciones, figuras y ejemplos nuevos.

Todos los capítulos y materiales del proyecto se reescribieron con énfasis en separar la abstracción de la implementación, un tema importante en Nand to Tetris. Tuvimos especial cuidado en agregar ejemplos y secciones que abordan las miles de preguntas que se publicaron a lo largo de los años en los foros de preguntas y respuestas de Nand to Tetris.

---

## Reconocimientos

Las herramientas de software que acompañan al libro fueron desarrolladas por nuestros estudiantes en IDC Herzliya y en la Universidad Hebrea. Los dos arquitectos principales de software fueron Yaron Ukrainitz y Yannai Gonczarowski, y los desarrolladores incluyeron a Iftach Ian Amit, Assaf Gad, Gal Katzhendler, Hadar Rosen-Sior y Nir Rozen. Oren Baranes, Oren Cohen, Jonathan Gross, Golan Parashi y Uri Zeira trabajaron en otros aspectos de las herramientas. Trabajar con estos estudiantes-desarrolladores ha sido un gran placer y estamos orgullosos de haber tenido la oportunidad de desempeñar un papel en su educación.

También agradecemos a nuestros asistentes de enseñanza, Muawyah Akash, Philip Hendrix, Eytan Lifshitz, Ran Navok y David Rabinowitz, quienes ayudaron a ejecutar las primeras versiones del curso que condujo a este libro. Tal Achituv, Yong Bakos, Tali Gutman y Michael Schröder proporcionaron una gran ayuda con varios aspectos de los materiales del curso, y Aryeh Schnall, Tomasz Róžański y Rudolf Adamkovič dieron meticulosas sugerencias de edición. Los comentarios de Rudolf fueron particularmente esclarecedores, por lo que estamos muy agradecidos.

Muchas personas de todo el mundo se involucraron en Nand to Tetris, y no podemos agradecerles individualmente. Hacemos una excepción. Marcar

Armbrust, un ingeniero de software y firmware de Colorado, se convirtió en el ángel guardián de Nand para los estudiantes de Tetris. Como voluntario para administrar nuestro foro global de preguntas y respuestas, Mark respondió numerosas preguntas con gran paciencia y estilo elegante. Sus respuestas nunca revelaron las soluciones; más bien, guió a los alumnos sobre cómo aplicarse y ver la luz por sí mismos. Al hacerlo, Mark se ha ganado el respeto y la admiración de numerosos estudiantes de todo el mundo. Sirviendo a la vanguardia de Nand to Tetris durante más de diez años, Mark escribió 2,607 publicaciones, descubrió docenas de errores y escribió scripts correctivos y correcciones. Haciendo todo esto además de su trabajo diario, Mark se convirtió en un pilar de la comunidad de Nand to Tetris, y la comunidad se convirtió en su segundo hogar. Mark murió en marzo de 2019, luego de una lucha contra una enfermedad cardíaca que duró varios meses. Durante su hospitalización, Mark recibió un flujo diario de cientos de correos electrónicos de Nand a los estudiantes de Tetris. Hombres y mujeres jóvenes de todo el mundo agradecieron a Mark por su generosidad ilimitada y compartieron el impacto que ha tenido en sus vidas.

En los últimos años, la educación en ciencias de la computación se convirtió en un poderoso impulsor del crecimiento personal y la movilidad económica. Mirando hacia atrás, nos sentimos afortunados de haber decidido, desde el principio, hacer que todos nuestros recursos didácticos estén disponibles gratuitamente, en código abierto. Simplemente, cualquier persona que quiera hacerlo no solo puede aprender sino también enseñar cursos de Nand a Tetris, sin ninguna restricción. Todo lo que tiene que hacer es ir a nuestro sitio web y tomar lo que necesita, siempre que opere en un entorno sin fines de lucro. Esto convirtió a Nand to Tetris en un vehículo fácilmente disponible para difundir educación en ciencias de la computación de alta calidad, de manera libre y equitativa. El resultado se convirtió en un vasto ecosistema educativo, alimentado por un sinfín de suministros de buena voluntad. Agradecemos a las muchas personas de todo el mundo que nos ayudaron a hacerlo realidad.



---

# I Hardware

El verdadero viaje de descubrimiento no consiste en ir a nuevos lugares, sino en tener un nuevo par de ojos.

—Marcel Proust (1871-1922)

Este libro es un viaje de descubrimiento. Está a punto de aprender tres cosas: cómo funcionan los sistemas informáticos, cómo dividir problemas complejos en módulos manejables y cómo construir sistemas de hardware y software a gran escala. Este será un viaje práctico, ya que creará un sistema informático completo y funcional desde cero. Las lecciones que aprenderá, que son mucho más importantes que la computadora en sí, se obtendrán como efectos secundarios de estas construcciones. Según el psicólogo Carl Rogers, "El único tipo de aprendizaje que influye significativamente en el comportamiento es el autodescubierto o la auto-apropiada, verdad que ha sido asimilada en la experiencia". Este capítulo de introducción esboza algunos de los descubrimientos, verdades y experiencias que se avecinan.

---

## Hola, mundo de abajo

Si tiene algo de experiencia en programación, probablemente se haya encontrado con algo como el programa a continuación al principio de su capacitación. Y si no lo ha hecho, aún puede adivinar lo que está haciendo el programa: muestra el texto `Hello World` y termina. Este programa en particular está escrito en Jack, un lenguaje de alto nivel simple, similar a Java:

```
// First example in Programming 101
class Main {
    function void main() {
        do Output.println("Hello World");
        return;
    }
}
```

Los programas triviales como Hello World son engañosamente simples. ¿Alguna vez te detuviste a pensar en lo que se necesita para *ejecutar* un programa de este tipo en una computadora? Miremos debajo del capó. Para empezar, tenga en cuenta que el programa no es más que una secuencia de caracteres simples, almacenados en un archivo de texto. Esta abstracción es un completo misterio para la computadora, que solo entiende instrucciones escritas en lenguaje de máquina. Por lo tanto, si queremos ejecutar este programa, lo primero que debemos hacer es analizar la cadena de caracteres de la que está hecho el código de alto nivel, descubrir su semántica (averiguar qué busca hacer el programa) y luego generar código de bajo nivel que reexpresa esta semántica utilizando el lenguaje de máquina de la computadora de destino. El resultado de este elaborado proceso de traducción, conocido como *compilación*, será una secuencia ejecutable de instrucciones en lenguaje de máquina.

Por supuesto, el lenguaje de máquina también es una abstracción, un conjunto acordado de códigos binarios. Para concretar esta abstracción, debe ser realizada por alguna *arquitectura de hardware*. Y esta arquitectura, a su vez, es implementada por un cierto conjunto de chips: registros, unidades de memoria, sumadores, etc. Ahora, cada uno de estos dispositivos de hardware está construido a partir de *puertas lógicas elementales de nivel inferior*. Y estas puertas, a su vez, se pueden construir a partir de puertas primitivas como *Nand* y *Nor*. Estas puertas primitivas están muy abajo en la jerarquía, pero también están hechas de varios dispositivos de *conmutación*, típicamente implementados por transistores. Y cada transistor está hecho de... Bueno, no iremos más allá de eso, porque ahí es donde termina la informática y comienza la física.

Quizás esté pensando: "En *mi* computadora, compilar y ejecutar programas es mucho más fácil, ¡todo lo que tengo que hacer es hacer clic en este icono o escribir ese comando!" De hecho, un sistema informático moderno es como un iceberg sumergido: la mayoría de la gente solo puede ver la parte superior, y su conocimiento de los sistemas informáticos es incompleto y superficial. Sin embargo, si desea explorar debajo de la superficie, ¡Lucky You! Hay un mundo fascinante ahí abajo, hecho

de algunas de las cosas más hermosas de la informática. Una comprensión íntima de este submundo es lo que separa a los programadores ingenuos de los desarrolladores sofisticados, personas que pueden crear tecnologías complejas de hardware y software. Y la mejor manera de entender cómo funcionan estas tecnologías, y nos referimos a entenderlas en la médula de sus huesos, es construir un sistema informático completo desde cero.

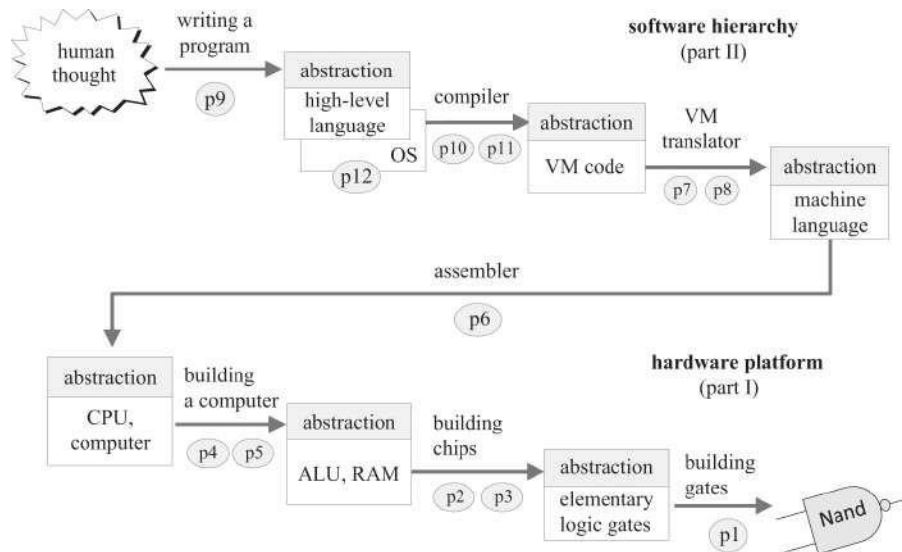
---

## Nand a Tetris

Suponiendo que queremos construir un sistema informático desde cero, ¿qué computadora específica deberíamos construir? Resulta que cada computadora de propósito general, cada PC, teléfono inteligente o servidor, es una máquina de Nand a Tetris. Primero, todas las computadoras se basan, en el fondo, en puertas lógicas elementales, de las cuales Nand es la más utilizada en la industria (explicaremos qué es exactamente una puerta Nand en el capítulo 1). En segundo lugar, cada computadora de propósito general se puede programar para ejecutar un juego de Tetris, así como cualquier otro programa que le guste. Por lo tanto, no hay nada único en Nand o Tetris. Es la palabra *to* en *Nand to Tetris* lo que convierte este libro en el viaje mágico que está a punto de emprender: pasar de un montón de dispositivos de conmutación básicos a una máquina que involucra la mente con texto, gráficos, animación, música, video, análisis, simulación, inteligencia artificial y todas las capacidades que esperamos de las computadoras de propósito general. Por lo tanto, realmente no importa qué plataforma de hardware específica y jerarquía de software construiremos, siempre que se basen en las mismas ideas y técnicas que caracterizan a *todos los* sistemas informáticos que existen.

La Figura I.1 describe los hitos clave en la hoja de ruta de Nand a Tetris. Comenzando en el nivel inferior de la figura, cualquier computadora de propósito general tiene una arquitectura que incluye una ALU (Unidad de lógica aritmética) y una RAM (Memoria de acceso aleatorio). Todos los dispositivos ALU y RAM están hechos de puertas lógicas elementales. Y, sorprendentemente y afortunadamente, como pronto veremos, todas las puertas lógicas se pueden hacer solo a partir de puertas Nand. Centrándose en la jerarquía del software, todos los lenguajes de alto nivel se basan en un conjunto de traductores (compilador/intérprete, máquina virtual, ensamblador) para reducir el código de alto nivel hasta las instrucciones a nivel de máquina. Algunos

Los lenguajes se interpretan en lugar de compilarse, y algunos no usan una máquina virtual, pero el panorama general es esencialmente el mismo. Esta observación es una manifestación de un principio fundamental de la informática, conocido como la *conjetura de Church-Turing*: en el fondo, todas las computadoras son esencialmente equivalentes.



**Figura I.1** Módulos principales de un sistema informático típico, que consisten en una plataforma de hardware y una jerarquía de software. Cada módulo tiene una *vista abstracta* (también llamada interfaz del módulo) y una *implementación*. Las flechas que apuntan hacia la derecha significan que cada módulo se implementa utilizando bloques de construcción abstractos del nivel inferior. Cada círculo representa un proyecto y un capítulo de Nand a Tetris: doce proyectos y capítulos en total.

Hacemos estas observaciones para enfatizar la generalidad de nuestro enfoque: los desafíos, ideas, consejos, trucos, técnicas y terminología que encontrará en este libro son exactamente los mismos que encuentran los ingenieros de hardware y software en ejercicio. En ese sentido, Nand to Tetris es una forma de iniciación: si logras completar el viaje, obtendrás una base excelente para convertirte en un profesional de la informática incondicional.

Entonces, ¿qué plataforma de hardware específica y qué lenguaje específico de alto nivel debemos construir en Nand to Tetris? Una posibilidad es construir un modelo informático de fuerza industrial y ampliamente utilizado y escribir un compilador para un lenguaje popular de alto nivel. Optamos por no tomar estas decisiones, por tres razones. Primero, los modelos de computadora van y vienen, y la programación en caliente

los idiomas dan paso a otros nuevos. Por lo tanto, no queríamos comprometernos con ninguna configuración de hardware/software en particular. En segundo lugar, las computadoras y los lenguajes que se utilizan en la práctica presentan numerosos detalles que tienen poco valor instructivo, pero que tardan años en implementarse. Finalmente, buscamos una plataforma de hardware y una jerarquía de software que pudiera controlarse, entenderse y ampliarse fácilmente. Estas consideraciones llevaron a la creación de Hack, la plataforma informática construida en la parte I del libro, y Jack, el lenguaje de alto nivel implementado en la parte II.

Por lo general, los sistemas informáticos se describen *de arriba hacia abajo*, mostrando cómo las abstracciones de alto nivel pueden reducirse o realizarse mediante otras más simples. Por ejemplo, podemos describir cómo las instrucciones binarias de la máquina que se ejecutan en la arquitectura de la computadora se dividen en microcódigos que viajan a través de los cables de la arquitectura y terminan manipulando los chips ALU y RAM de nivel inferior. Alternativamente, podemos ir *de abajo hacia arriba*, describiendo cómo los chips ALU y RAM están juiciosamente diseñados para ejecutar microcódigos que, en conjunto, forman instrucciones binarias de máquina. Tanto el enfoque de arriba hacia abajo como el de abajo hacia arriba son esclarecedores, cada uno de los cuales brinda una perspectiva diferente sobre el sistema que estamos a punto de construir.

En la [figura I.1](#), la dirección de las flechas sugiere una orientación de arriba hacia abajo. Para cualquier par de módulos, hay una flecha que apunta hacia la derecha que conecta el módulo superior con el inferior. El significado de esta flecha es preciso: implica que el módulo de nivel superior se implementa utilizando bloques de construcción abstractos del nivel inferior. Por ejemplo, un programa de alto nivel se implementa traduciendo cada instrucción de alto nivel en un conjunto de comandos abstractos de máquina virtual. Y cada comando de VM, a su vez, se traduce en un conjunto de instrucciones abstractas en lenguaje de máquina. Y así sigue. La distinción entre *abstracción* e *implementación* juega un papel importante en el diseño de sistemas, como ahora vamos a discutir.

---

## Abstracción e implementación

Quizás se pregunte cómo es humanamente posible construir un sistema informático completo desde cero, comenzando con nada más que puertas

lógicas elementales. ¡Esta debe ser una empresa enorme! Nos ocupamos de esta complejidad dividiendo el sistema en *módulos*. Cada módulo es

descrito por separado, en un capítulo dedicado, y construido por separado, en un proyecto independiente. Entonces podría preguntarse, ¿cómo es posible describir y construir estos módulos de forma aislada? ¡Seguramente están interrelacionados! Como demostraremos a lo largo del libro, un buen diseño modular implica precisamente eso: puede trabajar en los módulos individuales de forma independiente, mientras ignora por completo el resto del sistema. De hecho, si el sistema está bien diseñado, puede construir estos módulos en el orden que desee, e incluso en paralelo, si trabaja en equipo.

La capacidad cognitiva de "dividir y conquistar" un sistema complejo en módulos manejables se ve potenciada por otro don cognitivo: nuestra capacidad de discernir entre la *abstracción* y la *implementación* de cada módulo. En informática, tomamos estas palabras concretamente: la abstracción describe lo que hace el módulo y la implementación describe cómo lo hace. Con esta distinción en mente, esta es la regla más importante en la ingeniería de sistemas: cuando se usa un módulo como bloque de construcción, *cualquier módulo*, debe centrarse exclusivamente en la abstracción del módulo, ignorando por completo sus detalles de implementación.

Por ejemplo, centrémonos en el nivel inferior de la [figura I.1](#), comenzando en el nivel de "arquitectura de computadoras". Como se ve en la figura, la implementación de esta arquitectura utiliza varios bloques de construcción del nivel inferior, incluida una memoria de acceso aleatorio. La RAM es un dispositivo notable. Puede contener miles de millones de registros, pero se puede acceder a cualquiera de ellos directamente y casi instantáneamente. [La Figura I.1](#) nos informa que el arquitecto informático debe usar este dispositivo de acceso directo de manera abstracta, sin prestar atención a cómo se realiza realmente. Todo el trabajo, la inteligencia y el drama que se invirtieron en la implementación de la magia de la RAM de acceso directo, el *cómo*, deben ignorarse por completo, ya que esta información es irrelevante en el contexto del *uso* de la RAM para su efecto.

Bajando un nivel en la [figura I.1](#), ahora nos encontramos en la posición de tener que construir el chip RAM. ¿Cómo debemos hacerlo? Siguiendo la flecha que apunta hacia la derecha, vemos que la implementación de RAM se basará en puertas lógicas elementales y chips del nivel inferior. En particular, el almacenamiento RAM y las capacidades de acceso directo se realizarán utilizando *registros* y *multiplexores*, respectivamente. Y una vez más, entra en juego el mismo principio de implementación de abstracción: usaremos estos chips como bloques de construcción abstractos, centrándonos en sus interfaces y sin importarnos nada

sobre *sus* implementaciones. Y así sigue, hasta el nivel de Nand.

En resumen, cada vez que su implementación utilice un módulo de hardware o software de nivel inferior, debe tratar este módulo como una abstracción de caja negra lista para usar: todo lo que necesita es la documentación de la interfaz del módulo, describiendo *lo* que puede hacer, y listo. No debe prestar atención alguna a *cómo* funciona el módulo y lo que anuncia su interfaz. Este paradigma de abstracción-implementación ayuda a los desarrolladores a gestionar la complejidad y mantener la cordura: al dividir un sistema abrumador en módulos bien definidos, creamos fragmentos manejables de trabajo de implementación y localizamos la detección y corrección de errores. Este es el principio de diseño más importante en los proyectos de construcción de hardware y software.

No hace falta decir que todo en esta historia depende del intrincado arte del *diseño modular*: la capacidad humana de separar el problema en cuestión en una elegante colección de módulos bien definidos, cada uno con una interfaz clara, cada uno de los cuales representa una parte razonable del trabajo de implementación independiente, cada uno de los cuales se presta a un programa independiente de pruebas unitarias. De hecho, el diseño modular es el pan y la mantequilla de la informática aplicada: cada arquitecto de sistemas define rutinariamente abstracciones, a veces denominadas *módulos* o *interfaces*, y luego las implementa, o pide a otras personas que las implementen. Las abstracciones a menudo se construyen capa sobre capa, lo que da como resultado niveles de funcionalidad cada vez más altos. Si el arquitecto del sistema diseña un buen conjunto de módulos, el trabajo de implementación fluirá como agua clara; Si el diseño es descuidado, la implementación estará condenada al fracaso.

El diseño modular es un arte adquirido, perfeccionado al ver e implementar muchas abstracciones bien diseñadas. Eso es exactamente lo que estás a punto de experimentar en Nand to Tetris: aprenderás a apreciar la elegancia y la funcionalidad de cientos de abstracciones de hardware y software. Luego se le guiará sobre cómo implementar cada una de estas abstracciones, paso a paso, creando trozos de funcionalidad cada vez más grandes. A medida que avanzas en este viaje, pasando de un capítulo a otro, será emocionante mirar hacia atrás y apreciar el sistema informático que está tomando forma gradualmente a raíz de tus esfuerzos.



---

## Metodología

El viaje de Nand a Tetris implica la construcción de una plataforma de hardware y una jerarquía de software. La plataforma de hardware se basa en un conjunto de unas treinta puertas lógicas y chips, construidos en la parte I del libro. Cada una de estas puertas y chips, incluida la arquitectura informática más alta, se construirá utilizando un lenguaje de *descripción de hardware*. El HDL que usaremos está documentado en el apéndice 2 y se puede aprender en aproximadamente una hora. Probará la corrección de sus programas HDL utilizando un simulador de hardware basado en software que se ejecuta en su PC. Así es exactamente como trabajan los ingenieros de hardware en la práctica: construyen y prueban chips utilizando simuladores basados en software. Cuando están satisfechos con el rendimiento simulado de los chips, envían sus especificaciones (programas HDL) a una empresa de fabricación. Después de la optimización, los programas HDL se convierten en la entrada de brazos robóticos que construyen el hardware en silicio.

Avanzando en el viaje de Nand a Tetris, en la parte II del libro construiremos una pila de software que incluye un ensamblador, una máquina virtual y un compilador. Estos programas se pueden implementar en cualquier lenguaje de programación de alto nivel. Además, construiremos un sistema operativo básico, escrito en Jack.

Quizás se pregunte cómo es posible desarrollar estos ambiciosos proyectos en el ámbito de un curso o un libro. Bueno, además del diseño modular, nuestra salsa secreta es reducir la incertidumbre del diseño al mínimo absoluto. Proporcionaremos andamios elaborados para cada proyecto, incluidas API detalladas, programas esqueléticos, scripts de prueba y pautas de implementación por etapas.

Todas las herramientas de software necesarias para completar los proyectos 1-12 están disponibles en el paquete de software Nand to Tetris, que se puede descargar gratuitamente desde [www.nand2tetris.org](http://www.nand2tetris.org). Estos incluyen un simulador de hardware, un emulador de CPU, un emulador de VM y versiones ejecutables de los chips de hardware, ensamblador, compilador y sistema operativo. Una vez que descargue el paquete de software en su PC, todas estas herramientas estarán al alcance de su mano.

---

## El camino por delante

El viaje de Nand a Tetris implica doce proyectos de construcción de hardware y software. La dirección general del desarrollo en estos proyectos, así como la tabla de contenido del libro, implican un viaje de abajo hacia arriba: comenzamos con puertas lógicas elementales y avanzamos hacia arriba, lo que lleva a un lenguaje de programación basado en objetos de alto nivel. Al mismo tiempo, la dirección del desarrollo *dentro de* cada proyecto es de arriba hacia abajo. En particular, cada vez que presentamos un módulo de hardware o software, siempre comenzaremos con una descripción abstracta de *para qué* está diseñado el módulo y por qué es necesario. Una vez que comprenda la abstracción del módulo (un mundo rico por derecho propio), procederá a implementarlo, utilizando bloques de construcción abstractos del nivel inferior.

Así que aquí, finalmente, está el gran plan de la parte I de nuestro tour de force. En el capítulo 1 comenzamos con una sola puerta lógica, Nand, y construimos a partir de ella un conjunto de puertas lógicas elementales y de uso común como And, Or, Xor, etc. En los capítulos 2 y 3 usamos estos bloques de construcción para construir una unidad lógica aritmética y dispositivos de memoria, respectivamente. En el capítulo 4 hacemos una pausa en nuestro viaje de construcción de hardware e introducimos un lenguaje de máquina de bajo nivel tanto en su forma simbólica como binaria. En el capítulo 5 usamos la ALU y las unidades de memoria previamente construidas para construir una Unidad Central de Procesamiento (CPU) y una Memoria de Acceso Aleatorio (RAM). Estos dispositivos se integrarán en una plataforma de hardware capaz de ejecutar programas escritos en el lenguaje de máquina presentado en el capítulo 4. En el capítulo 6 describimos y construimos un *ensamblador*, que es un programa que traduce programas de bajo nivel escritos en lenguaje de máquina simbólico en código binario ejecutable. Esto completará la construcción de la plataforma de hardware. Esta plataforma se convertirá en el punto de partida de la parte II del libro, en la que ampliaremos el hardware básico con una jerarquía de software moderna que consta de una máquina virtual, un compilador y un sistema operativo.

Esperamos haber logrado transmitir lo que se avecina y que esté ansioso por comenzar este gran viaje de descubrimiento. Entonces, suponiendo que esté listo y listo, deje que comience la cuenta regresiva: 1, 0, ¡Vamos!

---

# 1 Lógica booleana

Cosas tan simples, y hacemos de ellas algo tan complejo que casi nos derrota.

—John Ashbery (1927–2017)

Cada dispositivo digital, ya sea una computadora personal, un teléfono celular o un enrutador de red, se basa en un conjunto de chips diseñados para almacenar y procesar información binaria. Aunque estos chips vienen en diferentes formas y formas, todos están hechos de los mismos bloques de construcción: *puertas lógicas elementales*. Las puertas se pueden realizar físicamente utilizando muchas tecnologías de hardware diferentes, pero su comportamiento lógico, o *abstracción*, es consistente en todas las implementaciones.

En este capítulo comenzamos con una puerta lógica primitiva, Nand, y construimos todas las demás puertas lógicas que necesitaremos de ella. En particular, construiremos puertas Not, And, Or y Xor, así como dos puertas llamadas *multiplexor* y *demultiplexor* (la función de todas estas puertas se describe a continuación). Dado que nuestra computadora de destino estará diseñada para operar con valores de 16 bits, también construiremos versiones de 16 bits de las puertas básicas, como Not16, And16, etc. El resultado será un conjunto bastante estándar de puertas lógicas, que luego se utilizarán para construir los chips de procesamiento y memoria de nuestra computadora. Esto se hará en los capítulos 2 y 3, respectivamente.

El capítulo comienza con el conjunto mínimo de conceptos teóricos y herramientas prácticas necesarias para diseñar e implementar puertas lógicas. En particular, presentamos el álgebra booleana y las funciones booleanas y mostramos cómo las funciones booleanas pueden realizarse mediante puertas lógicas. Luego describimos cómo se pueden implementar las puertas lógicas utilizando un lenguaje de descripción de hardware (HDL) y cómo se pueden probar estos diseños utilizando simuladores de hardware. Esta introducción tendrá su peso a lo largo de la parte I de la

, ya que el álgebra booleana y el HDL entrarán en juego en cada uno de los próximos capítulos y proyectos de hardware.

## 1.1 Álgebra booleana

El álgebra booleana manipula valores binarios de dos estados que normalmente se etiquetan como verdadero/falso, 1/0, sí/no, activado/desactivado, etc. Usaremos 1 y 0. Una función booleana es una función que opera en entradas binarias y devuelve salidas binarias. Dado que el hardware de la computadora se basa en la representación y manipulación de valores binarios, las funciones booleanas juegan un papel central en la especificación, el análisis y la optimización de las arquitecturas de hardware.

**Operadores booleanos:** La Figura 1.1 presenta tres funciones booleanas de uso común, también conocidas como *operadores booleanos*. Estas funciones se denominan Y, O y No, también escritas con la notación  $x \cdot y$ ,  $x + y$ , y  $\bar{x}$ , o  $x \wedge y$ ,  $x \vee y$ , y  $\neg x$ , respectivamente. La Figura 1.2 da la definición de *todas las* posibles funciones booleanas que se pueden definir sobre dos variables, junto con sus nombres comunes. Estas funciones se construyeron sistemáticamente enumerando todas las combinaciones posibles de valores abarcadas por dos variables binarias. Cada operador tiene un nombre convencional que busca describir su semántica subyacente. Por ejemplo, el nombre del operador Nand es la abreviatura de *Not-Y*, proveniente de la observación de que Nand ( $x, y$ ) es equivalente a Not (And ( $x, y$ )). El operador Xor, abreviatura de *o exclusivo*, se evalúa como 1 cuando exactamente una de sus dos variables es 1. La puerta Nor deriva su nombre de *Not-Or*. Todos estos nombres de puertas no son terriblemente importantes.

| $x$ | $y$ | $x \text{ And } y$ | $x$ | $y$ | $x \text{ Or } y$ | $x$ | Not $x$ |
|-----|-----|--------------------|-----|-----|-------------------|-----|---------|
| 0   | 0   | 0                  | 0   | 0   | 0                 | 0   | 1       |
| 0   | 1   | 0                  | 0   | 1   | 1                 | 1   | 0       |
| 1   | 0   | 0                  | 1   | 0   | 1                 |     |         |
| 1   | 1   | 1                  | 1   | 1   | 1                 |     |         |

**Figura 1.1** Tres funciones booleanas elementales.

|                 |                                     |     |   |   |   |   |
|-----------------|-------------------------------------|-----|---|---|---|---|
|                 |                                     | $x$ | 0 | 0 | 1 | 1 |
|                 |                                     | $y$ | 0 | 1 | 0 | 1 |
| Constant 0      | 0                                   |     | 0 | 0 | 0 | 0 |
| And             | $x \cdot y$                         |     | 0 | 0 | 0 | 1 |
| $x$ And Not $y$ | $x \cdot \bar{y}$                   |     | 0 | 0 | 1 | 0 |
| $x$             | $x$                                 |     | 0 | 0 | 1 | 1 |
| Not $x$ And $y$ | $\bar{x} \cdot y$                   |     | 0 | 1 | 0 | 0 |
| $y$             | $y$                                 |     | 0 | 1 | 0 | 1 |
| Xor             | $x \cdot \bar{y} + \bar{x} \cdot y$ |     | 0 | 1 | 1 | 0 |
| Or              | $x + y$                             |     | 0 | 1 | 1 | 1 |
| Nor             | $\overline{x + y}$                  |     | 1 | 0 | 0 | 0 |
| Equivalence     | $x \cdot y + \bar{x} \cdot \bar{y}$ |     | 1 | 0 | 0 | 1 |
| Not $y$         | $\bar{y}$                           |     | 1 | 0 | 1 | 0 |
| If $y$ then $x$ | $x + \bar{y}$                       |     | 1 | 0 | 1 | 1 |
| Not $x$         | $\bar{x}$                           |     | 1 | 1 | 0 | 0 |
| If $x$ then $y$ | $\bar{x} + y$                       |     | 1 | 1 | 0 | 1 |
| Nand            | $\overline{x \cdot y}$              |     | 1 | 1 | 1 | 0 |
| Constant 1      | 1                                   |     | 1 | 1 | 1 | 1 |

**Figura 1.2** Todas las funciones booleanas de dos variables binarias. En general, el número de funciones booleanas abarcadas por  $n$  variables binarias (aquí  $n = 2$ ) es  $2^{2^n}$  (eso es un montón de funciones booleanas).

La Figura 1.2 plantea la pregunta: ¿Qué hace que And, Or y Not sea más interesante o privilegiado que cualquier otro subconjunto de operadores booleanos? La respuesta corta es que, de hecho, no hay nada especial en And, Or, and Not. Una respuesta más profunda es que se pueden usar varios subconjuntos de operadores lógicos para expresar *cualquier* función booleana, y {And, Or, Not} es uno de esos subconjuntos. Si encuentra esta afirmación impresionante, considere esto: cualquiera de estos tres operadores básicos se puede expresar usando otro operador: Nand. ¡Eso *es* impresionante! De ello se deduce que cualquier función booleana se puede realizar usando Nand

solo puertas. El Apéndice 1, que es una lectura opcional, proporciona una prueba de esta notable afirmación.

## Funciones booleanas

Cada función booleana se puede definir utilizando dos representaciones alternativas. Primero, podemos definir la función usando una *tabla de verdad*, como lo hacemos en [la figura 1.3](#). Para cada una de las  $2^n$  tuplas posibles de valores de variables  $v_1, \dots, v_n$  (aquí  $n=3$ ), la tabla enumera el valor de  $f(v_1, \dots, v_n)$ . Además de esta definición basada en datos, también podemos definir funciones booleanas usando expresiones booleanas, por ejemplo,  $f(x, y, z) = (x \text{ Or } y) \text{ And Not } (z)$ .

| $x$ | $y$ | $z$ | $f(x, y, z) = (x \text{ Or } y) \text{ And Not}(z)$ |
|-----|-----|-----|-----------------------------------------------------|
| 0   | 0   | 0   | 0                                                   |
| 0   | 0   | 1   | 0                                                   |
| 0   | 1   | 0   | 1                                                   |
| 0   | 1   | 1   | 0                                                   |
| 1   | 0   | 0   | 1                                                   |
| 1   | 0   | 1   | 0                                                   |
| 1   | 1   | 0   | 1                                                   |
| 1   | 1   | 1   | 0                                                   |

**Figura 1.3** Tabla de verdad y definiciones funcionales de una función booleana (ejemplo).

¿Cómo podemos verificar que una expresión booleana dada es equivalente a una tabla de verdad dada? Usemos [la figura 1.3](#) como ejemplo. Comenzando con la primera fila, calculamos  $f(0, 0, 0)$ , que es  $(0 \text{ o } 0) \text{ y no } (0)$ . Esta expresión se evalúa como 0, el mismo valor que aparece en la tabla de verdad. Hasta ahora, bien. Se puede aplicar una prueba de equivalencia similar a cada fila de la tabla, un asunto bastante tedioso. En lugar de usar esta laboriosa técnica de prueba de abajo hacia arriba, podemos probar la equivalencia de arriba hacia abajo, analizando la expresión booleana  $(x \text{ o } y) \text{ y no } (z)$ . Centrándonos en el lado izquierdo del operador Y, observamos que la expresión general se evalúa como 1 solo cuando  $((x \text{ es } 1) \text{ O } (y$

es 1)). Girando hacia el lado derecho del operador Y, observamos que la expresión general se evalúa como 1 solo cuando ( $z$  es 0). Juntando estas dos observaciones, concluimos que la expresión se evalúa como 1 solo cuando  $((x \text{ es } 1) \text{ O } (y \text{ es } 1)) \text{ Y } (z \text{ es } 0)$ . Este patrón de 0 y 1 ocurre solo en las filas 3, 5 y 7 de la tabla de verdad y, de hecho, estas son las únicas filas en las que la columna más a la derecha de la tabla contiene un 1.

## Tablas de verdad y expresiones booleanas

Dada una función booleana de  $n$  variables representadas por una expresión booleana, siempre podemos construir a partir de ella la tabla de verdad de la función. Simplemente calculamos la función para cada conjunto de valores (fila) en la tabla. Esta construcción es laboriosa y obvia. Al mismo tiempo, la construcción dual no es obvia en absoluto: dada una representación de tabla de verdad de una función booleana, ¿podemos siempre sintetizar a partir de ella una expresión booleana para la función subyacente? La respuesta a esta intrigante pregunta es *sí*. Se puede encontrar una prueba en el apéndice 1.

Cuando se trata de construir computadoras, la representación de la tabla de verdad, la expresión booleana y la capacidad de construir una a partir de la otra son muy relevantes. Por ejemplo, supongamos que se nos pide que construyamos algún hardware para secuenciar datos de ADN y que nuestro biólogo experto en el dominio quiere describir la lógica de secuenciación utilizando una tabla de verdad. Nuestro trabajo es realizar esta lógica en hardware. Tomando los datos de la tabla de verdad dados como punto de partida, podemos sintetizar a partir de ellos una expresión booleana que representa la función subyacente. Después de simplificar la expresión usando álgebra booleana, podemos proceder a implementarla usando puertas lógicas, como lo haremos más adelante en el capítulo. En resumen, una tabla de verdad es a menudo un medio conveniente para describir algunos estados de la naturaleza, mientras que una expresión booleana es un formalismo conveniente para realizar esta descripción en silicio. La capacidad de pasar de una representación a otra es una de las prácticas más importantes del diseño de hardware.

Observamos de paso que, aunque la representación de la tabla de verdad de una función booleana es única, cada función booleana se puede representar mediante muchas expresiones booleanas diferentes pero equivalentes, y algunas serán más cortas y fáciles de trabajar. Por ejemplo, la expresión  $(\text{Not } (x \text{ And } y) \text{ And } (\text{Not } (x) \text{ Or } y) \text{ And } (\text{Not } (y) \text{ Or } y))$  es equivalente a la expresión  $\text{Not}$

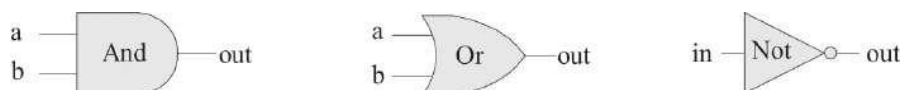
(x). Vemos que la capacidad de simplificar una expresión booleana es el primer paso hacia la optimización del hardware. Esto se hace utilizando álgebra booleana y sentido común, como se ilustra en el apéndice 1.

---

## 1.2 Puertas lógicas

Una *puerta* es un dispositivo físico que implementa una función booleana simple. Aunque la mayoría de las computadoras digitales de hoy en día usan electricidad para realizar puertas y representar datos binarios, se puede emplear cualquier tecnología alternativa que permita capacidades de conmutación y conducción. De hecho, a lo largo de los años, se crearon muchas implementaciones de hardware de funciones booleanas, incluidos mecanismos magnéticos, ópticos, biológicos, hidráulicos, neumáticos, cuánticos e incluso basados en dominó (muchas de estas implementaciones se propusieron como hazañas caprichosas de "puedo hacer"). Hoy en día, las puertas se implementan típicamente como transistores grabados en silicio, empaquetados como *chips*. En Nand to Tetris usamos las palabras *chip* y *gate* indistintamente, tendiendo a usar esta última para instancias simples de la primera.

La disponibilidad de tecnologías de conmutación alternativas, por un lado, y la observación de que el álgebra booleana se puede usar para abstraer el comportamiento de las puertas lógicas, por el otro, es extremadamente importante. Básicamente, implica que los informáticos no tienen que preocuparse por artefactos físicos como electricidad, circuitos, interruptores, relés y fuentes de energía. En cambio, los científicos informáticos se contentan con las nociones abstractas del álgebra booleana y la lógica de puertas, confiando felizmente en que alguien más, físicos e ingenieros eléctricos, descubrirá cómo realizarlos realmente en hardware. Por lo tanto, las puertas primitivas como las que se muestran en la [figura 1.4](#) pueden verse como dispositivos de caja negra que implementan operaciones lógicas elementales de una forma u otra, no nos importa cómo. El uso del álgebra booleana para analizar el comportamiento abstracto de las puertas lógicas fue articulado en 1937 por Claude Shannon, lo que llevó a lo que a veces se describe como la tesis de M.Sc. más importante en ciencias de la computación.

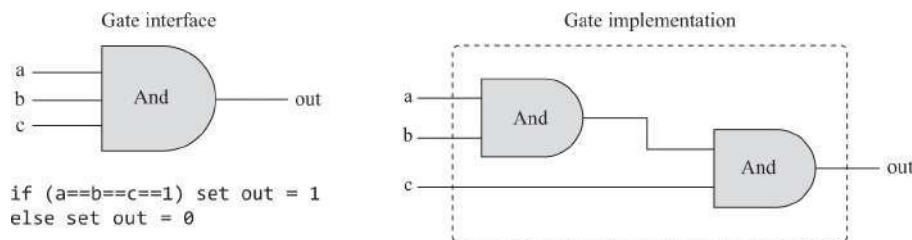




**Figura 1.4** Diagramas de puertas estándar de tres puertas lógicas elementales.

## Puertas primitivas y compuestas

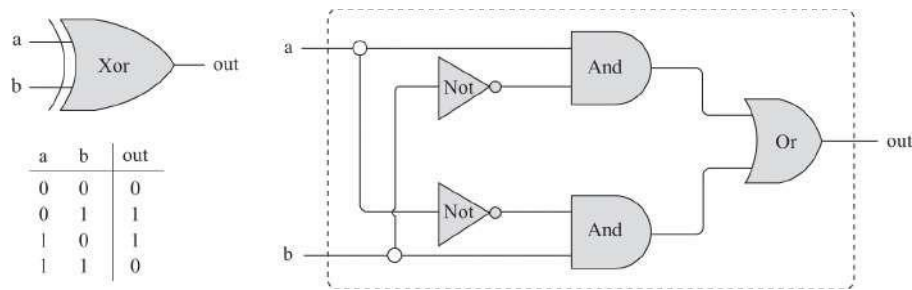
Dado que todas las puertas lógicas tienen los mismos tipos de datos de entrada y salida (0 y 1), se pueden combinar, creando *puertas compuestas* de complejidad arbitraria. Por ejemplo, supongamos que se nos pide que implementemos la función booleana de tres vías And ( $a, b, c$ ), que devuelve 1 cuando cada una de sus entradas es 1 y 0 en caso contrario. Usando álgebra booleana, podemos comenzar observando que  $a \cdot b \cdot c = (a \cdot b) \cdot c$ , o, usando notación de prefijo,  $Y(a, b, c) = \text{And}(Y(a, b), c)$ . A continuación, podemos usar este resultado para construir la puerta compuesta representada en la [figura 1.5](#).



**Figura 1.5** Implementación compuesta de una puerta Y de tres vías. El contorno discontinuo rectangular define el límite de la interfaz de puerta.

Vemos que cualquier puerta lógica dada se puede ver desde dos perspectivas diferentes: interna y externa. El lado derecho de [la figura 1.5](#) muestra la arquitectura interna de la puerta, o *implementación*, mientras que el lado izquierdo muestra la interfaz de la puerta, es decir, sus pines de entrada y salida y el comportamiento que expone al mundo exterior. La vista interna es relevante solo para el constructor de puertas, mientras que la vista externa es el nivel de detalle adecuado para los diseñadores que desean utilizar la puerta como un componente abstracto y listo para usar, sin prestar atención a su implementación.

Consideremos otro ejemplo de diseño lógico: Xor. Por definición, Xor ( $a, b$ ) es 1 exactamente cuando  $a$  es 1 y  $b$  es 0 o  $a$  es 0 y  $b$  es 1. Dicho de otra manera,  $Xor(a, b) = Or(\text{And}(a, \text{Not}(b)), Y(\text{No}(a), b))$ . Esta definición se implementa en el diseño lógico que se muestra en la [figura 1.6](#).



**Figura 1.6** Interfaz de puerta Xor (izquierda) y una posible implementación (derecha).

Tenga en cuenta que la *interfaz* de cualquier puerta dada es única: solo hay una forma de especificarla, y esto normalmente se hace usando una tabla de verdad, una expresión booleana o una especificación verbal. Esta interfaz, sin embargo, se puede realizar de muchas maneras diferentes, y algunas serán más elegantes y eficientes que otras. Por ejemplo, la implementación de Xor que se muestra en la [figura 1.6](#) es una posibilidad; hay formas más eficientes de realizar Xor, utilizando menos puertas lógicas y menos conexiones entre puertas. Por lo tanto, desde un punto de vista funcional, el requisito fundamental del diseño lógico es que *la implementación de la puerta realice su interfaz establecida, de una forma u otra*. Desde el punto de vista de la eficiencia, la regla general es tratar de usar la menor cantidad de puertas posible, ya que menos puertas implican menos costo, menos energía y un cálculo más rápido.

En resumen, el arte del diseño lógico se puede describir de la siguiente manera: Dada una abstracción de puerta (también conocida como *especificación* o *interfaz*), encuentre una manera eficiente de implementarla utilizando otras puertas que ya se implementaron.

---

### 1.3 Construcción de hardware

Ahora estamos en condiciones de discutir cómo se construyen realmente las puertas. Comencemos con un ejemplo intencionalmente ingenuo. Supongamos que abrimos un taller de fabricación de chips en el garaje de nuestra casa. Nuestro primer contrato es construir cien puertas Xor. Usando el pago inicial del pedido, compramos una pistola de soldar, un rollo de alambre de cobre y tres contenedores etiquetados como "Puertas Y", "Puertas O" y "Puertas No", cada una con muchas copias idénticas de estas puertas lógicas elementales. Cada una de estas puertas está sellada en una carcasa de plástico que expone algunos

pinos de entrada y salida, así como un puerto de fuente de alimentación. Nuestro objetivo es realizar el diagrama de puerta que se muestra en la [figura 1.6](#) utilizando este hardware.

Comenzamos tomando dos puertas Y, dos puertas Not y una puerta Or y montándolas en un tablero, de acuerdo con el diseño de la figura. A continuación, conectamos los chips entre sí pasando cables entre ellos y soldando los extremos de los cables a los respectivos pines de entrada/salida.

Ahora, si seguimos cuidadosamente el diagrama de la puerta, terminaremos teniendo tres extremos de cable expuestos. Luego soldamos un pin a cada uno de estos extremos de cable, sellamos todo el dispositivo (excepto los tres pines) en una carcasa de plástico y lo etiquetamos como "Xor". Podemos repetir este proceso de ensamblaje muchas veces. Al final del día, podemos almacenar todos los chips que hemos construido en un nuevo contenedor y etiquetarlo como "puertas Xor". Si deseamos construir otros chips en el futuro, podremos usar estas puertas Xor como bloques de construcción de caja negra, tal como usamos las puertas Y, Or y Not antes.

Como probablemente haya sentido, el enfoque de garaje para la producción de chips deja mucho que desear. Para empezar, no hay garantía de que el diagrama de chip dado sea correcto. Aunque podemos probar la corrección en casos simples como Xor, no podemos hacerlo en muchos chips realistas y complejos. Por lo tanto, debemos conformarnos con pruebas empíricas: construir el chip, conectarlo a una fuente de alimentación, activar y desactivar los pines de entrada en varias configuraciones y esperar que el comportamiento de entrada / salida del chip cumpla con la especificación deseada. Si el chip no lo hace, tendremos que jugar con su estructura física, un asunto bastante complicado. Además, incluso si se nos ocurre un diseño correcto y eficiente, replicar el proceso de ensamblaje de chips muchas veces será un asunto que llevará mucho tiempo y será propenso a errores. ¡Debe haber una mejor manera!

### **1.3.1 Lenguaje de descripción de hardware**

Hoy en día, los diseñadores de hardware ya no construyen nada con sus propias manos. En cambio, diseñan la arquitectura del chip utilizando un formalismo llamado *Lenguaje de descripción de hardware* o *HDL*. El diseñador especifica la lógica del chip escribiendo un *programa HDL*, que luego se somete a una rigurosa batería de pruebas. Las pruebas se llevan a cabo virtualmente, utilizando simulación por computadora: una herramienta de software especial, llamada *simulador de hardware*, toma el programa HDL como entrada y crea una representación de software de la lógica del

chip. A continuación, el diseñador puede indicar al simulador que pruebe el chip virtual en

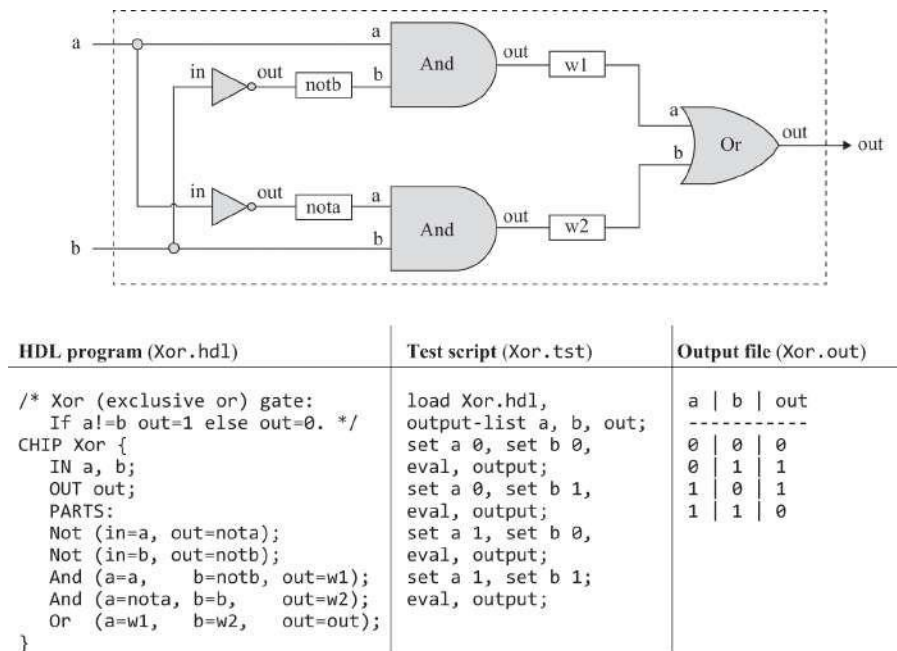
varios conjuntos de entradas. El simulador calcula las salidas del chip, que luego se comparan con las salidas deseadas, según lo dispuesto por el cliente que ordenó la construcción del chip.

Además de probar la corrección del chip, el diseñador de hardware generalmente estará interesado en una variedad de parámetros, como la velocidad de cálculo, el consumo de energía y el costo total implícito en la implementación del chip propuesta. Todos estos parámetros pueden ser simulados y cuantificados por el simulador de hardware, lo que ayuda al diseñador a optimizar el diseño hasta que el chip simulado ofrezca los niveles deseados de costo/rendimiento.

Por lo tanto, con HDL, se puede planificar, depurar y optimizar completamente un chip completo antes de gastar un solo centavo en la producción física. Cuando el rendimiento del chip simulado satisface al cliente que lo solicitó, una versión optimizada del programa HDL puede convertirse en el modelo a partir del cual se pueden estampar muchas copias del chip físico en silicio. Este paso final en el proceso de diseño de chips, desde un programa HDL optimizado hasta la producción en masa, generalmente se subcontrata a empresas que se especializan en la fabricación de chips robóticos, utilizando una tecnología de conmutación u otra.

**Ejemplo: Construyendo una puerta Xor:** El resto de esta sección ofrece una breve introducción a HDL, utilizando un ejemplo de puerta Xor; se puede encontrar una especificación detallada de HDL en el apéndice 2.

Centrémonos en la parte inferior izquierda de la [figura 1.7](#). Una definición HDL de un chip consta de una sección de *encabezado* y una sección de *piezas*. La sección de encabezado especifica la interfaz del chip, enumerando el nombre del chip y los nombres de sus pines de entrada y salida. La sección PARTS describe las partes del chip a partir de las cuales se fabrica la arquitectura del chip. Cada pieza de chip se representa mediante una sola *instrucción* que especifica el nombre de la pieza, seguida de una expresión entre paréntesis que especifica cómo se conecta a otras piezas del diseño. Tenga en cuenta que para escribir tales declaraciones, el programador HDL debe tener acceso a las interfaces de todas las partes del chip subyacentes: los nombres de sus pines de entrada y salida, así como su operación prevista. Por ejemplo, el programador que escribió el programa HDL enumerado en la [figura 1.7](#) debe haber sabido que los pines de entrada y salida de la puerta Not se nombran in y out y que los de las puertas And y Or se denominan a, b y out. (Las API de todos los chips utilizados en Nand to Tetris se enumeran en el apéndice 4).



**Figura 1.7** Diagrama de puertas e implementación HDL de la función booleana Xor ( $a, b$ ) = Or (And ( $a$ , Not ( $b$ )), And (Not ( $a$ ),  $b$ )), utilizada como ejemplo. También se muestra un script de prueba y un archivo de salida generado por la prueba. Las descripciones detalladas de HDL y el lenguaje de prueba se dan en los apéndices 2 y 3, respectivamente.

Las conexiones entre partes se especifican creando y conectando *pinos internos*, según sea necesario. Por ejemplo, considere la parte inferior del diagrama de puertas, donde la salida de una puerta Not se canaliza a la entrada de una puerta And posterior. El código HDL describe esta conexión mediante el par de instrucciones Not(..., out=nota) y And(a=nota, ...). La primera instrucción crea un pino interno (conexión saliente) denominado nota y canaliza el valor del pino de salida en él. La segunda instrucción canaliza el valor de nota en la entrada a de una puerta Y. Aquí hay que hacer dos comentarios. Primero, los pines internos se crean "automáticamente" la primera vez que aparecen en un programa HDL. En segundo lugar, los pines pueden tener una distribución ilimitada. Por ejemplo, en la figura 1.7, cada entrada se alimenta simultáneamente en dos puertas. En los diagramas de puertas, se describen múltiples conexiones dibujándolas, creando patrones bifurcados. En los programas HDL, la existencia de bifurcaciones se infiere del código.

El HDL que usamos en Nand to Tetris tiene una apariencia similar a los HDL de fuerza industrial, pero es mucho más simple. Nuestra sintaxis HDL se explica principalmente por sí misma y se puede aprender viendo algunos ejemplos y consultando el apéndice 2, según sea necesario.

## Ensayo

La rigurosa garantía de calidad exige que los chips se prueben de manera específica, replicable y bien documentada. Con eso en mente, los simuladores de hardware generalmente están diseñados para ejecutar *scripts de prueba*, escritos en un lenguaje de scripting. El script de prueba que se muestra en [la figura 1.7](#) está escrito en el lenguaje de scripting que entiende el simulador de hardware de Nand a Tetris.

Vamos a dar una breve descripción de este script de prueba. Las dos primeras líneas indican al simulador que cargue el programa `Xor.hdl` y se prepare para imprimir los valores de las variables seleccionadas. A continuación, el script enumera una serie de escenarios de prueba. En cada escenario, el script indica al simulador que vincule las entradas del chip a los valores de datos seleccionados, calcule la salida resultante y registre los resultados de la prueba en un archivo de salida designado. En el caso de puertas simples como Xor, se puede escribir un script de prueba exhaustivo que enumere todos los valores de entrada que la puerta puede obtener. En este caso, el archivo de salida resultante (lado derecho de [la figura 1.7](#)) proporciona una prueba empírica completa de que el chip se comporta bien. El lujo de tal certeza no es factible en chips más complejos, como veremos más adelante.

Los lectores que planean construir la computadora Hack estarán encantados de saber que todos los chips que aparecen en el libro van acompañados de programas HDL esqueléticos y scripts de prueba suministrados, disponibles en el paquete de software Nand to Tetris. A diferencia de HDL, que debe aprenderse para completar las especificaciones del chip, no es necesario aprender nuestro lenguaje de prueba. Al mismo tiempo, debe poder leer y comprender los scripts de prueba suministrados. El lenguaje de secuencias de comandos se describe en el apéndice 3, que se puede consultar según sea necesario.

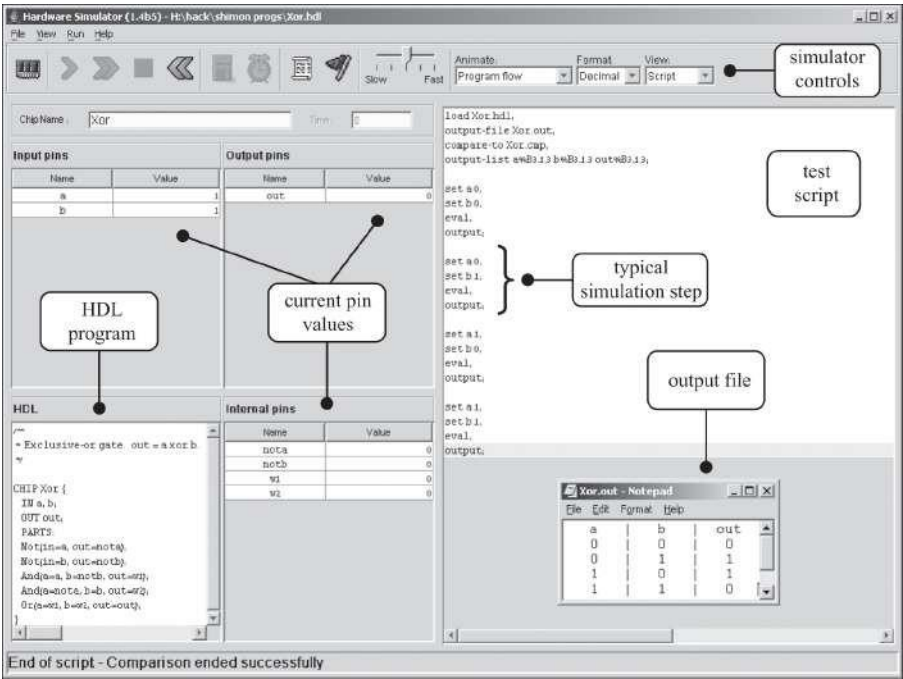
### 1.3.2 Simulación de hardware

La escritura y depuración de programas HDL es similar al desarrollo de software convencional. La principal diferencia es que en lugar de escribir código en un lenguaje de alto nivel, lo escribimos en HDL, y en lugar de compilar y ejecutar el código, usamos un simulador de *hardware* para probarlo. El simulador de hardware es un programa informático que sabe cómo analizar e interpretar el código HDL, convertirlo en una representación ejecutable y probarlo de acuerdo con los scripts de prueba suministrados. Existen muchos simuladores de hardware comerciales de

este tipo en el



mercado. El paquete de software Nand to Tetris incluye un simulador de hardware simple que proporciona todas las herramientas necesarias para construir, probar e integrar todos los chips presentados en el libro, lo que lleva a la construcción de una computadora de propósito general. La Figura 1.8 ilustra una sesión típica de simulación de chips.



**Figura 1.8** Una captura de pantalla de la simulación de un chip XOR en el simulador de hardware suministrado (otras versiones de este simulador pueden tener una GUI ligeramente diferente). El estado del simulador se muestra justo después de que el script de prueba haya terminado de ejecutarse. Los valores de los pines corresponden al último paso de la simulación ( $a=b=1$ ). No se muestra en esta captura de pantalla un *archivo de comparación* que enumera la salida esperada de la simulación especificada por este script de prueba en particular. Al igual que el script de prueba, el archivo de comparación generalmente lo proporciona el cliente que desea construir el chip. En este ejemplo concreto, el archivo de salida generado por la simulación (abajo a la derecha de la figura) es idéntico al archivo de comparación proporcionado.

## 1.4 Especificación

Ahora pasamos a especificar un conjunto de puertas lógicas que serán necesarias para construir los chips de nuestro sistema informático. Estas puertas son ordinarias, cada una diseñada para llevar a cabo una operación booleana común. Para cada puerta, nos centraremos en la interfaz de la puerta (*lo que se supone que debe hacer la puerta*), retrasando los detalles de implementación (*cómo construir la funcionalidad de la puerta*) para una sección posterior.

### 1.4.1 Nand

El punto de partida de nuestra arquitectura de computadoras es la puerta Nand, a partir de la cual se construirán todas las demás puertas y chips. La puerta Nand realiza la siguiente función booleana:

| <i>a</i> | <i>b</i> | Nand( <i>a</i> , <i>b</i> ) |
|----------|----------|-----------------------------|
| 0        | 0        | 1                           |
| 0        | 1        | 1                           |
| 1        | 0        | 1                           |
| 1        | 1        | 0                           |

O bien, usando el estilo API:

```
Chip name: Nand
Input:      a, b
Output:     out
Function:   if ((a==1) and (b==1)) then out = 0, else out = 1
```

A lo largo del libro, los chips se especifican utilizando el estilo API que se muestra arriba. Para cada chip, la API especifica el nombre del chip, los nombres de sus pines de entrada y salida, la función u operación prevista del chip y los comentarios opcionales.

### 1.4.2 Puertas lógicas básicas

Las puertas lógicas que presentamos aquí generalmente se denominan *básicas*, ya que entran en juego en la construcción de chips más complejos. Las puertas Not, And, Or y Xor implementan operadores lógicos clásicos, y las puertas multiplexor y demultiplexor proporcionan medios para controlar los flujos de información.

**No:** También conocido como *inversor*, esta puerta emite el valor opuesto al valor de su entrada. Aquí está la API:

```
Chip name: Not
Input:    in
Output:    out
Function:  if (in==0) then out = 1, else out = 0
```

**Y:** Devuelve 1 cuando sus dos entradas son 1 y 0 en caso contrario:

```
Chip name: And
Input:    a, b
Output:    out
Function:  if ((a==1) and (b==1)) then out = 1, else out = 0
```

**O:** Devuelve 1 cuando al menos una de sus entradas es 1 y 0 en caso contrario:

```
Chip name: Or
Input:    a, b
Output:    out
Function:  if ((a==0) and (b==0)) then out = 0, else out = 1
```

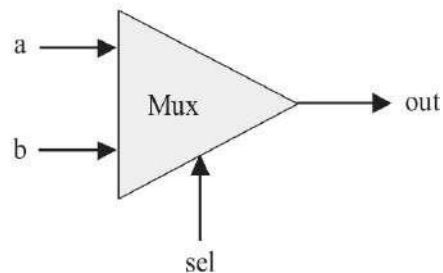
**Xor:** También conocido como *o exclusivo*, esta puerta devuelve 1 cuando exactamente una de sus entradas es 1, y 0 en caso contrario:

```
Chip name: Xor
Input:    a, b
Output:    out
Function:  if (a!=b) then out = 1, else out = 0
```

**Multiplexor:** Un multiplexor es una puerta de tres entradas (ver [figura 1.9](#)). Dos bits de entrada, llamados a y b, se interpretan como *bits de datos*, y un tercer bit de entrada, llamado sel, se interpreta como un bit de *selección*. El multiplexor utiliza sel para seleccionar y generar el valor de a o b. Por lo tanto, un nombre sensato para este dispositivo podría haber sido *selector*. El nombre *multiplexor* se adoptó de los sistemas de comunicaciones, donde se utilizan versiones extendidas de este dispositivo para serializar (multiplexar) varias señales de entrada a través de un solo canal de comunicaciones.

| a | b | sel | out |
|---|---|-----|-----|
| 0 | 0 | 0   | 0   |
| 0 | 1 | 0   | 0   |
| 1 | 0 | 0   | 1   |
| 1 | 1 | 0   | 1   |
| 0 | 0 | 1   | 0   |
| 0 | 1 | 1   | 1   |
| 1 | 0 | 1   | 0   |
| 1 | 1 | 1   | 1   |

| sel | out |
|-----|-----|
| 0   | a   |
| 1   | b   |



Chip name: Mux

Input: a, b, sel

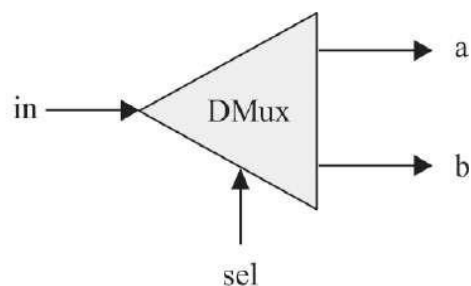
Output: out

Function: if (sel==0) then out = a, else out = b

**Figura 1.9** Multiplexor. La tabla en la parte superior derecha es una versión abreviada de la tabla de verdad.

**Demultiplexor:** Un demultiplexor realiza la función opuesta a un multiplexor: toma un solo valor de entrada y lo enruta a una de las dos salidas posibles, de acuerdo con un bit selector que selecciona la salida de destino. La otra salida se establece en 0. [La Figura 1.10](#) muestra la API.

| sel | a  | b  |
|-----|----|----|
| 0   | in | 0  |
| 1   | 0  | in |



Chip name: DMux

Input: in, sel

Output: a, b

Function: if (sel==0) then {a, b} = {in, 0},  
else {a, b} = {0, in}

**Figura 1.10** Demultiplexor.

### 1.4.3 Versiones de varios bits de puertas básicas

El hardware de la computadora a menudo está diseñado para procesar valores de varios bits, por ejemplo, calcular una función And bit a bit en dos entradas de 16 bits dadas. Esta sección describe varias puertas lógicas de 16 bits que serán necesarias para construir nuestra plataforma de computadora de destino. Observamos de paso que la arquitectura lógica de estas puertas de  $n$  bits es la misma, independientemente del valor de  $n$  (por ejemplo, 16, 32 o 64 bits). Los programas HDL tratan los valores de varios bits como valores de un solo bit, excepto que los valores se pueden indexar para acceder a bits individuales. Por ejemplo, si `in` y `out` representan valores de 16 bits, `out[3]=in[5]` establece el 3er bit de `out` en el valor del 5º bit de `in`. Los bits se indexan de derecha a izquierda, el bit más a la derecha es el bit 0 y el bit más a la izquierda es el bit 15 (en una configuración de 16 bits).

**Multi-bit Not:** Una puerta Not de  $n$  bits aplica la operación booleana Not a cada uno de los bits en su entrada de  $n$  bits:

```
Chip name: Not16
Input:     in[16]
Output:    out[16]
Function:  for i = 0..15 out[i] = Not(in[i])
```

**Y de varios bits:** una puerta And de  $n$  bits aplica la operación booleana And a cada par respectivo en sus dos entradas de  $n$  bits:

```
Chip name: And16
Input:     a[16], b[16]
Output:    out[16]
Function:  for i = 0..15 out[i] = And(a[i], b[i])
```

**O de varios bits:** una puerta Or de  $n$  bits aplica la operación booleana Or a cada par respectivo en sus dos entradas de  $n$  bits:

```
Chip name: Or16
Input:     a[16], b[16]
Output:    out[16]
Function:  for i = 0..15 out[i] = Or(a[i], b[i])
```

**Multiplexor de varios bits:** Un multiplexor de  $n$  bits funciona exactamente igual que un multiplexor básico, excepto que sus entradas y salidas tienen *un* ancho de  $n$  bits:

```
Chip name: Mux16
Input:     a[16], b[16], sel
Output:    out[16]
Function:  if (sel==0) then for i = 0..15 out[i] = a[i],
           else for i = 0..15 out[i] = b[i]
```

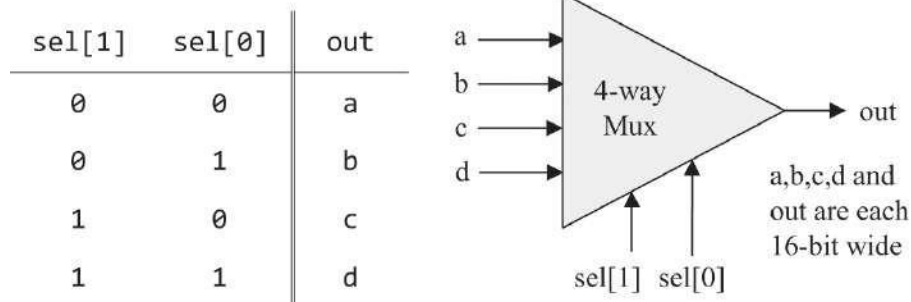
#### 1.4.4 Versiones multidireccionales de puertas básicas

Las puertas lógicas que operan en una o dos entradas tienen una generalización natural a variantes multidireccionales que operan en más de dos entradas. Esta sección describe un conjunto de puertas multidireccionales que se utilizarán posteriormente en varios chips de nuestra arquitectura de computadoras.

**O multidireccional:** Una *puerta Or de m-way emite 1 cuando al menos uno de sus  $m$  bits de entrada es 1* y 0 en caso contrario. Necesitaremos una variante de 8 vías de esta puerta:

```
Chip name: Or8Way
Input:     in[8]
Output:    out
Function:  out = Or(in[0], in[1], ..., in[7])
```

**Multiplexor multivía/multibit:** Un *multiplexor de  $n$  bits de  $m$  vías* selecciona una de sus entradas de  $n$  bits  $m$  y la envía a su salida de  $n$  bits. La selección se especifica mediante un conjunto de  $k$  bits de selección, donde  $k = \log_2 m$ . Aquí está la API de un multiplexor de 4 vías:

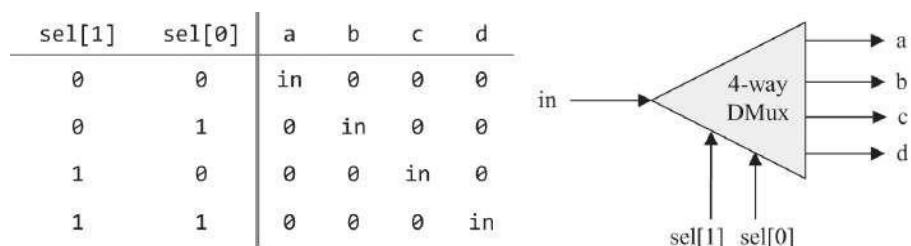


Nuestra plataforma de computadora de destino requiere dos variantes de este chip: un multiplexor de 16 bits de 4 vías y un multiplexor de 16 bits de 8 vías:

Chip name: Mux4Way16  
 Input: a[16],b[16],c[16],d[16],sel[2]  
 Output: out[16]  
 Function: if (sel==00,01,10, or 11) then out = a,b,c, or d  
 Comment: The assignment is a 16-bit operation.  
 For example, "out = a" means "for i = 0..15 out[i] = a[i]".

Chip name: Mux8Way16  
 Input: a[16],b[16],c[16],d[16],e[16],f[16],g[16],h[16], sel[3]  
 Output: out[16]  
 Function: if (sel==000,001,010, ..., or 111)  
 then out = a,b,c,d, ..., or h  
 Comment: The assignment is a 16-bit operation.  
 For example, "out = a" means "for i = 0..15 out[i] = a[i]".

**Demultiplexor multivía/multibit:** Un *demultiplexor* de  $n$  bits de  $m$  vías enruta su única *entrada* de  $n$  bits a una de sus *salidas* de  $n$  bits. Las otras salidas se establecen en 0. La selección se especifica mediante un conjunto de  $k$  bits de selección, donde  $k = \log_2 m$ . Aquí está la API de un demultiplexor de 4 vías:



Nuestra plataforma de computadora de destino requiere dos variantes de este chip: un demultiplexor de 4 vías y 1 bit y un demultiplexor de 8 vías y 1 bit:

```
Chip name: DMux4Way
Input:    in, sel[2]
Output:   a, b, c, d
Function:  if (sel==00)      then {a,b,c,d} = {1,0,0,0},
           else if (sel==01) then {a,b,c,d} = {0,1,0,0},
           else if (sel==10) then {a,b,c,d} = {0,0,1,0},
           else if (sel==11) then {a,b,c,d} = {0,0,0,1}

Chip name: Dmux8Way
Input:    in, sel[3]
Output:   a, b, c, d, e, f, g, h
Function:  if (sel==000)     then {a,b,c,..., h} = {1,0,0,0,0,0,0,0},
           else if (sel==001) then {a,b,c,..., h} = {0,1,0,0,0,0,0,0},
           else if (sel==010) then {a,b,c,..., h} = {0,0,1,0,0,0,0,0},
           ...
           else if (sel==111) then {a,b,c,..., h} = {0,0,0,0,0,0,0,1}
```

---

## 1.5 Implementación

La sección anterior describió las especificaciones, o interfaces, de una familia de puertas lógicas básicas. Habiendo descrito el *qué*, ahora pasamos a discutir el *cómo*. En particular, nos centraremos en dos enfoques generales para implementar puertas lógicas: *simulación de comportamiento* e *implementación de hardware*. Ambos enfoques juegan un papel importante en todos nuestros proyectos de construcción de hardware.

### 1.5.1 Simulación de comportamiento

Las descripciones de chips presentadas hasta ahora son estrictamente abstractas. Hubiera sido bueno si pudiéramos experimentar con estas abstracciones de forma práctica, antes de comenzar a construirlas en HDL. ¿Cómo podemos hacerlo?

Bueno, si todo lo que queremos hacer es interactuar con el comportamiento de los chips, no tenemos que pasar por la molestia de construirlas en HDL. En su lugar, podemos optar por una implementación mucho más sencilla, utilizando la programación convencional. Por ejemplo, podemos usar algún lenguaje orientado a objetos para crear un conjunto de clases, cada una implementando un chip genérico. Podemos escribir constructores de clase



para crear instancias de chip y métodos *de evaluación* para evaluar su lógica, y podemos hacer que las clases interactúen entre sí para que los chips de alto nivel se puedan definir en términos de los de nivel inferior. Luego podríamos agregar una buena interfaz gráfica de usuario que permita poner diferentes valores en las entradas del chip, evaluar su lógica y observar las salidas del chip. Esta técnica basada en software, llamada *simulación de comportamiento*, tiene mucho sentido. Permite experimentar con interfaces de chip antes de comenzar el laborioso proceso de construirlas en HDL.

El simulador de hardware de Nand a Tetris proporciona exactamente ese servicio. Además de simular el comportamiento de los programas HDL, que es su objetivo principal, el simulador cuenta con implementaciones de software integradas de todos los chips construidos en los proyectos de hardware de Nand a Tetris. La versión incorporada de cada chip se implementa como un módulo de software ejecutable, invocado por un programa HDL esquelético que proporciona la interfaz del chip. Por ejemplo, aquí está el programa HDL que implementa la versión incorporada del chip Xor:

```
/* Xor (exclusive or) gate:
   If a!=b out=1 else out=0. */
CHIP Xor {
    IN  a, b;
    OUT out;
    BUILTIN Xor;
}
```

Compare esto con el programa HDL que se muestra en la [figura 1.7](#). Primero, tenga en cuenta que los chips regulares y los chips incorporados tienen exactamente la misma interfaz. Por lo tanto, proporcionan exactamente la misma funcionalidad. Sin embargo, en la implementación incorporada, la sección PARTS se reemplaza con la instrucción única BUILTIN Xor. Esta instrucción informa al simulador de que el chip es implementado por Xor.class. Este archivo de clase, como todos los archivos de clase Java que implementan chips incorporados, se encuentra en la carpeta nand2tetris/tools/builtIn.

Señalamos de paso que realizar puertas lógicas utilizando programación de alto nivel no es difícil, y esa es otra virtud de la simulación de comportamiento: es económica y rápida. En algún momento, por supuesto, los ingenieros de hardware deben hacer lo real, que es implementar los chips no como

sino más bien como programas HDL que se pueden comprometer con silicio. Eso es lo que haremos a continuación.

### 1.5.2 Implementación de hardware

Esta sección proporciona pautas sobre cómo implementar las quince puertas lógicas descritas en este capítulo. Como regla en este libro, nuestras pautas de implementación son intencionalmente breves. Brindamos la información suficiente para comenzar, dejándole el placer de descubrir el resto de las implementaciones de la puerta usted mismo.

**Nand:** Desde que decidimos basar nuestro hardware en puertas Nand elementales, tratamos a Nand como una puerta primitiva cuya funcionalidad se da externamente. El simulador de hardware suministrado cuenta con una implementación incorporada de Nand y, por lo tanto, no es necesario implementarlo.

**No:** se puede implementar usando una sola puerta Nand. *Consejo:* Inspeccione la tabla de verdad Nand y pregúntese cómo se pueden organizar las entradas Nand para que una sola señal de entrada, 0, haga que la puerta Nand genere 1, y una sola señal de entrada, 1, haga que genere 0.

**Y:** Se puede implementar desde las dos puertas discutidas anteriormente.

**Or / Xor:** La función booleana Or se puede definir usando las funciones booleanas And and Not. La función booleana Xor se puede definir usando And, Not y Or.

**Multiplexor / Demultiplexor:** Se puede implementar utilizando puertas previamente construidas.

**Puertas de varios bits no / y / o:** Suponiendo que ya ha construido las versiones básicas de estas puertas, la implementación de sus *versiones n-arias es una cuestión de organizar matrices de n* puertas básicas y hacer que cada puerta funcione por separado en sus entradas de un solo bit. El código HDL resultante será algo aburrido y repetitivo (usando copiar y pegar), pero tendrá su peso cuando estas puertas de bits múltiples se utilicen en la construcción de chips más complejos, más adelante en el libro.

**Multiplexor multibit:** La implementación de un multiplexor  $n$ -ario es una cuestión de alimentar el mismo bit de selección a cada uno de *los  $n$*  multiplexores binarios. De nuevo, una tarea de construcción aburrida que resulta en un chip muy útil.

**Puertas multidireccionales:** Consejo de implementación: Piense en bifurcaciones.

### 1.5.3 Chips incorporados

Como señalamos cuando discutimos la simulación de comportamiento, nuestro simulador de hardware proporciona implementaciones integradas basadas en software de la mayoría de los chips descritos en el libro. En Nand to Tetris, el chip incorporado más célebre es, por supuesto, Nand: cada vez que usa una pieza de chip Nand en un programa HDL, el simulador de hardware invoca la implementación incorporada `tools/builtIn/Nand.hdl`. Esta convención es un caso especial de una estrategia de invocación de chip más general: cada vez que el simulador de hardware encuentra una parte de chip, digamos, *Xxx*, en un programa HDL, busca el archivo `Xxx.hdl` en la carpeta actual; si se encuentra el archivo, el simulador evalúa su código HDL subyacente. Si no se encuentra el archivo, el simulador lo busca en la carpeta `tools/builtIn`. Si el archivo se encuentra allí, el simulador ejecuta la implementación incorporada del chip; de lo contrario, el simulador emite un mensaje de error y finaliza la simulación.

Esta convención es útil. Por ejemplo, supongamos que comenzó a implementar un programa `Mux.hdl`, pero, por alguna razón, no lo completó. Esto podría ser un contratiempo molesto, ya que, en teoría, no puede continuar construyendo chips que usen Mux como parte del chip. Afortunadamente, y en realidad por diseño, aquí es donde los chips incorporados vienen al rescate. Todo lo que tiene que hacer es cambiar el nombre de su implementación parcial `Mux1.hdl`, por ejemplo. Cada vez que se llama al simulador de hardware para simular la funcionalidad de una pieza de chip Mux, no podrá encontrar un archivo `Mux.hdl` en la carpeta actual. Esto hará que se active la simulación de comportamiento, lo que obligará al simulador a usar la versión Mux incorporada en su lugar. ¡Exactamente lo que queremos! En una etapa posterior, es posible que desee volver a `Mux1.hdl` y reanudar el trabajo en su implementación. En este punto, puede restaurar su nombre de archivo original, `Mux.hdl`, y continuar desde donde lo dejó.

---

## 1.6 Proyecto

En esta sección se describen las herramientas y los recursos necesarios para completar el proyecto 1 y se ofrecen los pasos y sugerencias de implementación recomendados.

**Objetivo:** Implementar todas las puertas lógicas presentadas en el capítulo. Los únicos bloques de construcción que puedes usar son las puertas Nand primitivas y las puertas compuestas que construirás gradualmente sobre ellas.

**Recursos:** Suponemos que ya has descargado el archivo zip de Nand to Tetris, que contiene el paquete de software del libro, y que lo has extraído en una carpeta llamada `nand2tetris` en tu ordenador. Si ese es el caso, entonces la carpeta `nand2tetris/tools` en su computadora contiene el simulador de hardware discutido en este capítulo. Este programa, junto con un editor de texto sin formato, son las únicas herramientas necesarias para completar el proyecto 1, así como todos los demás proyectos de hardware descritos en el libro.

Los quince chips mencionados en este capítulo, excepto Nand, deben implementarse en el lenguaje HDL descrito en el apéndice 2. Para cada chip *Xxx*, *proporcionamos un programa Xxx.hdl esquelético* (a veces llamado *archivo auxiliar*) con una parte de implementación faltante. Además, para cada chip proporcionamos un *script Xxx.tst* que le dice al simulador de hardware cómo probarlo, junto con un archivo de comparación *Xxx.cmp* que enumera la salida correcta que se espera que genere la prueba suministrada. Todos estos archivos están disponibles en su carpeta `nand2tetris/projects/01`. Su trabajo es completar y probar todos los archivos *Xxx.hdl* en esta carpeta. Estos archivos se pueden escribir y editar con cualquier editor de texto sin formato.

**Contrato:** Cuando se carga en el simulador de hardware, el diseño del chip ( programa .hdl modificado), probado en el archivo .tst suministrado, debe producir los resultados enumerados en el archivo .cmp suministrado. Si las salidas reales generadas por el simulador no están de acuerdo con las salidas deseadas, el simulador detendrá la simulación y generará un mensaje de error.

**Pasos:** Recomendamos proceder en el siguiente orden:

0. El *hardware simulador* necesario para éste proyecto es disponible en [nand2tetris/herramientas](http://nand2tetris/herramientas).
1. Consulte el apéndice 2 (HDL), según sea necesario.
2. Consulte el tutorial del simulador de hardware (disponible en [www.nand2tetris.org](http://www.nand2tetris.org)), según sea necesario.
3. Construya y simule todos los chips enumerados en [nand2tetris/projects/01](http://nand2tetris/projects/01).

### Consejos generales de implementación

(Usamos los términos *puerta* y *chip* indistintamente).

- Cada puerta se puede implementar de más de una manera. Cuanto más simple sea la implementación, mejor. Como regla general, esfuércese por utilizar la menor cantidad posible de piezas de virutas.
- Aunque cada chip se puede implementar directamente solo desde puertas Nand, recomendamos usar siempre puertas compuestas que ya se implementaron. Consulte el consejo anterior.
- No es necesario construir "chips auxiliares" de su propio diseño. Sus programas HDL deben usar solo los chips mencionados en este capítulo.
- Implemente los chips en el orden en que aparecen en el capítulo. Si, por alguna razón, no completa la implementación HDL de algún chip, aún puede usarlo como una parte del chip en otros programas HDL. Simplemente cambie el nombre del archivo de chip o elimínelo de la carpeta, lo que hará que el simulador use su versión incorporada en su lugar.

**Una versión basada en la web del proyecto 1** está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

---

## 1.7 Perspectiva

Este capítulo especificó un conjunto de puertas lógicas básicas que se utilizan ampliamente en arquitecturas de computadoras. En los capítulos 2 y 3 usaremos estas puertas para construir nuestros chips de procesamiento y almacenamiento, respectivamente. Estos chips, a su vez, se utilizarán más tarde para construir la unidad central de procesamiento y los dispositivos de memoria de nuestra computadora.

Aunque hemos optado por usar Nand como nuestro bloque de construcción básico, se pueden usar otras puertas lógicas como posibles puntos de partida. Por ejemplo, puede crear una plataforma informática completa utilizando solo puertas Nor o, alternativamente, una combinación de puertas Y, Or y Not. Estos enfoques constructivos del diseño lógico son teóricamente equivalentes, al igual que la misma geometría puede basarse en conjuntos alternativos de axiomas acordados. En principio, si los ingenieros eléctricos o los físicos pueden idear implementaciones eficientes y de bajo costo de puertas lógicas utilizando cualquier tecnología que consideren adecuada, las usaremos felizmente como bloques de construcción primitivos. La realidad, sin embargo, es que la mayoría de las computadoras se construyen a partir de puertas Nand o Nor.

A lo largo del capítulo, no prestamos atención a las consideraciones de eficiencia y costo, como el consumo de energía o la cantidad de cruces de cables implícitos en nuestros programas HDL. Tales consideraciones son de vital importancia en la práctica, y una gran cantidad de experiencia en informática y tecnología se centra en optimizarlas. Otro tema que no abordamos son los aspectos físicos, por ejemplo, cómo se pueden construir puertas lógicas primitivas a partir de transistores incrustados en silicio o de otras tecnologías de conmutación. Por supuesto, existen varias opciones de implementación de este tipo, cada una con sus propias características (velocidad, consumo de energía, costo de producción, etc.). Cualquier cobertura no trivial de estos temas requiere aventurarse en áreas fuera de la informática, como la ingeniería eléctrica y la física del estado sólido.

El siguiente capítulo describe cómo se pueden usar los bits para representar números binarios y cómo se pueden usar las puertas lógicas para realizar operaciones aritméticas. Estas capacidades se basarán en las puertas lógicas elementales construidas en este capítulo.

---

## 2 Aritmética booleana

Contar es la religión de esta generación, su esperanza y salvación.

—Gertrude Stein (1874–1946)

En este capítulo construimos una familia de chips diseñados para representar números y realizar operaciones aritméticas. Nuestro punto de partida es el conjunto de puertas lógicas construidas en el capítulo 1, y nuestro punto final es una *Unidad Lógica Aritmética completamente funcional*. La ALU se convertirá más tarde en la pieza central computacional de la *Unidad Central de Procesamiento* (CPU), el chip que ejecuta todas las instrucciones manejadas por la computadora. Por lo tanto, la construcción del ALU es un hito importante en nuestro viaje de Nand a Tetris.

Como de costumbre, abordamos esta tarea gradualmente, comenzando con una sección de antecedentes que describe cómo se pueden usar los códigos binarios y la aritmética booleana, respectivamente, para representar y agregar enteros con signo. La sección Especificación presenta una sucesión de *chips sumadores* diseñados para agregar dos bits, tres bits y pares de números binarios de  $n$  bits. Esto prepara el escenario para la especificación ALU, que se basa en un diseño lógico sorprendentemente simple. Las secciones Implementación y Proyecto proporcionan consejos y directrices sobre cómo construir los chips sumadores y la ALU utilizando HDL y el simulador de hardware suministrado.

---

### 2.1 Operaciones aritméticas

Los sistemas informáticos de uso general deben realizar al menos las siguientes operaciones aritméticas en enteros con signo:

- adición
- Firma la
- conversión •
- r e s t a
- comparación
- División •d e
- m u l t i p l i c
- a c i ó n

Comenzaremos desarrollando la lógica de puerta que lleva a cabo la conversión de sumas y signos. Más adelante, mostraremos cómo se pueden implementar las otras operaciones aritméticas a partir de estos dos bloques de construcción.

Tanto en matemáticas como en informática, la *suma* es una operación simple que es profunda. Sorprendentemente, todas las funciones realizadas por las computadoras digitales, no solo las operaciones aritméticas, se pueden reducir a sumar números binarios. Por lo tanto, la comprensión constructiva de la suma binaria es la clave para comprender muchas operaciones fundamentales realizadas por el hardware de la computadora.

---

## 2.2 Números binarios

Cuando se nos dice que un cierto código, digamos, 6083, representa un número usando el *sistema decimal*, entonces, por convención, tomamos este número como:

$$(6083)_{10} = 6 \cdot 10^3 + 0 \cdot 10^2 + 8 \cdot 10^1 + 3 \cdot 10^0 = 6083$$

Cada dígito del código decimal aporta un valor que depende de la *base* 10 y de la posición del dígito en el código. Supongamos ahora que se nos dice que el código 10011 representa un número usando base 2 o *representación binaria*. Para calcular el valor de este número, seguimos exactamente el mismo procedimiento, usando base 2 en lugar de base 10:

$$(10011)_2 = 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 19$$



Dentro de las computadoras, *todo* se representa mediante códigos binarios. Por ejemplo, cuando presionamos las teclas del teclado etiquetadas como 1, 9 y Enter en respuesta a "Dar un ejemplo de un número primo", lo que termina almacenado en la memoria de la computadora es el código binario 10011. Cuando le pedimos a la computadora que muestre este valor en la pantalla, se produce el siguiente proceso. Primero, el sistema operativo de la computadora calcula el valor decimal que representa 10011, que resulta ser 19. Después de convertir este valor entero en los dos caracteres 1 y 9, el sistema operativo busca la fuente actual y obtiene las dos imágenes de mapa de bits utilizadas para representar estos caracteres en la pantalla. Luego, el sistema operativo hace que el controlador de pantalla encienda y apague los píxeles relevantes y, no contenga la respiración, todo dura una pequeña fracción de segundo, finalmente vemos aparecer la imagen 19 en la pantalla.

En el capítulo 12 desarrollaremos un sistema operativo que lleve a cabo tales operaciones de renderizado, entre muchos otros servicios de bajo nivel. Por ahora, basta con observar que la representación decimal de los números es una indulgencia humana explicada por el oscuro hecho de que, en algún momento de la historia antigua, los humanos decidieron representar cantidades usando sus diez dedos, y el hábito se mantuvo. Desde una perspectiva matemática, el número diez es completamente poco interesante y, en lo que respecta a las computadoras, es una completa molestia. Las computadoras manejan *todo* en binario y no se preocupan por el decimal. Sin embargo, dado que los humanos insisten en tratar con números usando códigos decimales, las computadoras tienen que trabajar duro detrás de escena para llevar a cabo conversiones de binario a decimal y de decimal a binario cada vez que los humanos quieren ver o proporcionar información numérica. En todos los demás momentos, las computadoras se adhieren al binario.

**Tamaño fijo de la palabra:** Los números enteros son, por supuesto, ilimitados: para cualquier número dado  $x$  hay enteros que son menores que  $x$  y enteros que son mayores que  $x$ . Sin embargo, las computadoras son máquinas finitas que usan un tamaño de palabra fijo para representar números. *Tamaño de palabra* es un término de hardware común que se usa para especificar el número de bits que usan las computadoras para representar un fragmento básico de información, en este caso, valores enteros. Normalmente, los registros de 8, 16, 32 o 64 bits se utilizan para representar números enteros.<sup>1</sup> El tamaño fijo de la palabra implica que hay un límite en el número de valores que estos registros pueden representar.

Por ejemplo, supongamos que usamos registros de 8 bits para representar números enteros. Esta representación puede codificar  $2^8 = 256$  diferentes cosas. Si deseamos representar solo enteros no negativos, podemos asignar 00000000 para representar 0, 00000001 para representar 1, 00000010 para representar 2, 00000011 para representar 3, hasta asignar 11111111 para representar 255. En general, usando  $n$  bits podemos representar todos los enteros no negativos que van de 0 a  $2^n - 1$ .

¿Qué pasa con la representación de números negativos usando códigos binarios? Más adelante en el capítulo presentaremos una técnica que cumple con este desafío de la manera más elegante y satisfactoria.

¿Y qué hay de representar números que son mayores o menores que los valores máximos y mínimos permitidos por el tamaño de registro fijo? Cada lenguaje de alto nivel proporciona abstracciones para manejar números que son tan grandes o tan pequeños como podamos desear en la práctica. Estas abstracciones generalmente se implementan uniendo tantos registros de  $n$  bits como sea necesario para representar los números. Dado que la ejecución de operaciones aritméticas y lógicas en números de varias palabras es un asunto lento, se recomienda utilizar esta práctica solo cuando la aplicación requiera procesar números extremadamente grandes o extremadamente pequeños.

---

## 2.3 Suma binaria

Se puede sumar un par de números binarios bit a bit de derecha a izquierda, utilizando el mismo algoritmo de suma decimal aprendido en la escuela primaria. Primero, agregamos los dos bits más a la derecha, también llamados *bits menos significativos* (LSB) de los dos números binarios. A continuación, agregamos el bit de transporte resultante a la suma del siguiente par de bits. Continuamos este proceso de bloqueo hasta que se agregan los dos *bits más significativos* a la izquierda (MSB). Aquí hay un ejemplo de este algoritmo en acción, asumiendo que usamos un tamaño de palabra fijo de 4 bits:

|             |   |   |   |   |         |          |   |   |   |   |
|-------------|---|---|---|---|---------|----------|---|---|---|---|
| 0           | 0 | 0 | 1 |   | (carry) | 1        | 1 | 1 | 1 |   |
|             | 1 | 0 | 0 | 1 | x       |          | 1 | 0 | 1 | 1 |
| +           | 0 | 1 | 0 | 1 | y       | +        | 0 | 1 | 1 | 1 |
| 0           | 1 | 1 | 1 | 0 | x + y   | 1        | 0 | 0 | 1 | 0 |
| no overflow |   |   |   |   |         | overflow |   |   |   |   |

Si la suma bit a bit más significativa genera un acarreo de 1, tenemos lo que se conoce como *desbordamiento*. Qué hacer con el desbordamiento es una cuestión de decisión, y la nuestra es ignorarlo. Básicamente, nos contentamos con garantizar que el resultado de sumar dos números de  $n$  bits será correcto hasta  $n$  bits. Señalamos de paso que ignorar las cosas es perfectamente aceptable siempre que uno sea claro y comunicativo al respecto.

---

## 2.4 Números binarios firmados

Un sistema binario de  $n$  bits puede codificar  $2^n$  cosas diferentes. Si tenemos que representar números con signo (positivos y negativos) en código binario, una solución natural es dividir el espacio de código disponible en dos subconjuntos: uno para representar números no negativos y el otro para representar números negativos. Idealmente, el esquema de codificación debe elegirse de tal manera que la introducción de números con signo complique lo menos posible la implementación de hardware de las operaciones aritméticas.

A lo largo de los años, este desafío ha llevado al desarrollo de varios esquemas de codificación para representar números con signo en código binario. La solución utilizada hoy en día en casi todas las computadoras se llama *método de complemento de dos*, también conocido como *complemento radix*. En un sistema binario que usa un tamaño de palabra de  $n$  bits, el código binario complementario de dos que representa  $x$  negativo se toma como el código que representa  $2^n - x$ . Por ejemplo, en un sistema binario de 4 bits,  $-7$  se representa utilizando el código binario asociado con

$2^4 - 7 = 9$ , que resulta ser 1001. Recordando que  $+7$  está representado por 0111, vemos que  $1001 + 0111 = 0000$  (ignorando el bit de desbordamiento). [La Figura 2.1](#) enumera todos los números con signo representados por un sistema de 4 bits utilizando el método de complemento de dos.

|       |    |          |
|-------|----|----------|
| 0000: | 0  |          |
| 0001: | 1  |          |
| 0010: | 2  |          |
| 0011: | 3  |          |
| 0100: | 4  |          |
| 0101: | 5  |          |
| 0110: | 6  |          |
| 0111: | 7  |          |
| 1000: | -8 | (16 - 8) |
| 1001: | -7 | (16 - 7) |
| 1010: | -6 | (16 - 6) |
| 1011: | -5 | (16 - 5) |
| 1100: | -4 | (16 - 4) |
| 1101: | -3 | (16 - 3) |
| 1110: | -2 | (16 - 2) |
| 1111: | -1 | (16 - 1) |

**Figura 2.1** Representación complementaria de dos de números con signo, en un sistema binario de 4 bits.

Una inspección de [la figura 2.1](#) sugiere que un sistema binario de  $n$  bits con representación del complemento de dos tiene las siguientes propiedades atractivas:

- El sistema codifica  $2^n$  números con signo, que van desde  $-(2^{n-1})$  a  $2^{n-1} - 1$ .
- El código de cualquier número no negativo comienza con un 0.
- El código de cualquier número negativo comienza con un 1.
- Para obtener el código binario de  $-x$  a partir del código binario de  $x$ , deje intactos todos los bits 0 menos significativos y el primer bit 1 menos significativo de  $x$ , y voltee todos los bits restantes (convierta 0 en 1 y viceversa). Alternativamente, voltea todos los bits de  $x$  y agrega 1 al

resultado.

Una característica particularmente atractiva de la representación del complemento de los dos es que la *resta* se maneja como un caso especial de suma. Para ilustrar, considere  $5 - 7$ . Teniendo en cuenta que esto es equivalente a  $5 + (-7)$ , y siguiendo la [figura 2.1](#), procedemos a calcular  $0101 + 1001$ . El resultado es  $1110$ , que de hecho es el código binario de  $-2$ . Aquí hay otro ejemplo: Para calcular  $(-2) + (-3)$ , sumamos  $1110 + 1101$ , la suma  $11011$ . Ignorando el bit de desbordamiento, obtenemos  $1011$ , que es el código binario de  $-5$ .

Vemos que el método del complemento de los dos permite sumar y restar números con signo usando nada más que el hardware requerido para sumar números no negativos. Como veremos más adelante en el libro, cada operación aritmética, desde la multiplicación hasta la división y la raíz cuadrada, se puede implementar de forma reductiva utilizando la suma binaria. Entonces, por un lado, observamos que una amplia gama de capacidades informáticas se basa en la suma binaria y, por otro lado, observamos que el método de complemento de los dos obvia la necesidad de hardware especial para sumar y restar números con signo. Tomando estas dos observaciones juntas, nos vemos obligados a concluir que el método del complemento de los dos es uno de los héroes más notables y anónimos de la informática aplicada.

---

## 2.5 Especificación

Ahora pasamos a especificar una jerarquía de fichas, comenzando con sumadores simples y culminando con una Unidad Lógica Aritmética (ALU). Como es habitual en este libro, nos centramos primero en el resumen (*para qué* están diseñados los chips), retrasando los detalles de implementación (*cómo* lo hacen) hasta la siguiente sección. No podemos resistirnos a reiterar, con placer, que gracias al método del complemento de dos no tenemos que decir nada especial sobre el manejo de números con signo. Todos los chips aritméticos que presentaremos funcionan igual de bien en números no negativos, negativos y de signos mixtos.

### 2.5.1 Serpientes

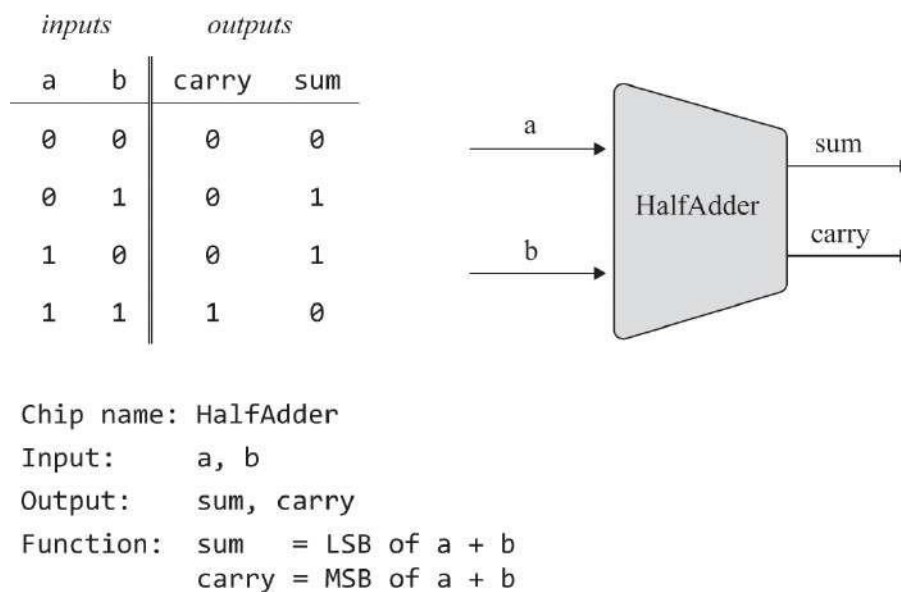
Nos centraremos en la siguiente jerarquía de

*sumadores*: • *Medio sumador*: diseñado para  
agregar dos bits

- *Sumador completo*: diseñado para agregar tres bits
- *Sumador*: diseñado para sumar dos *números de n bits*

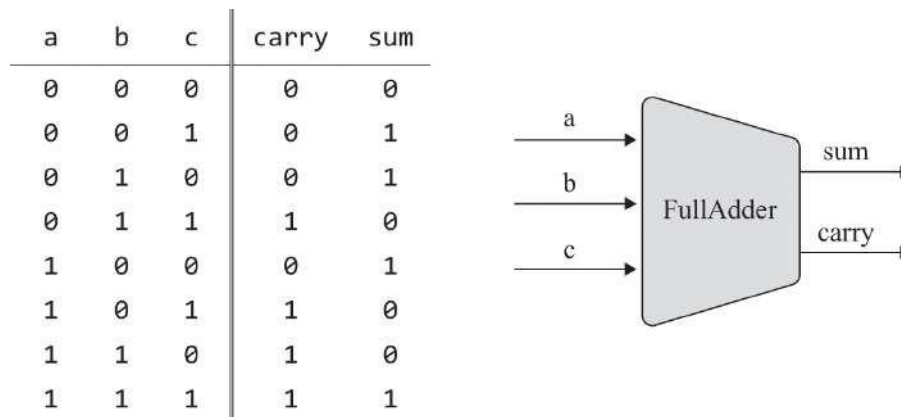
También especificaremos un sumador de propósito especial, llamado *incrementador*, diseñado para agregar 1 a un número dado. (Los nombres *medio sumador* y *sumador completo* se derivan del detalle de implementación de que se puede realizar un chip de sumador completo a partir de dos medios sumadores, como veremos más adelante en el capítulo).

**Medio sumador:** El primer paso en nuestro camino hacia la suma de números binarios es sumar dos bits. Tratemos el resultado de esta operación como un número de 2 bits, y llamemos a sus bits derecho e izquierdo sum y carry, respectivamente. [La Figura 2.2](#) presenta un chip que lleva a cabo esta operación de adición.



**Figura 2.2** Medio sumador, diseñado para agregar 2 bits.

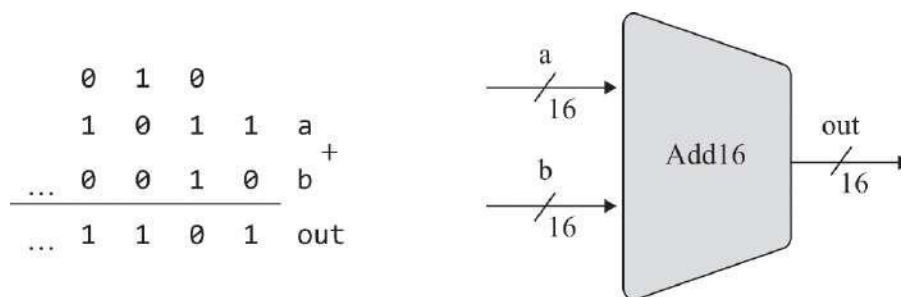
**Sumador completo:** [La Figura 2.3](#) presenta un *chip de sumador completo*, diseñado para agregar tres bits. Al igual que el medio sumador, el chip sumador completo genera dos bits que, en conjunto, representan la suma de los tres bits de entrada.



Chip name: FullAdder  
 Input: a, b, c  
 Output: sum, carry  
 Function: sum = LSB of  $a + b + c$   
 carry = MSB of  $a + b + c$

**Figura 2.3** Sumador completo, diseñado para agregar 3 bits.

**Sumador:** Las computadoras representan números enteros usando un tamaño de palabra fijo como 8, 16, 32 o 64 bits. El chip cuyo trabajo es sumar dos de estos *números de n* bits se llama *sumador*. La Figura 2.4 presenta un sumador de 16 bits.



Chip name: Add16  
 Input: a[16], b[16]  
 Output: out[16]  
 Function: Adds two 16-bit numbers.  
 The overflow bit is ignored.

**Figura 2.4** Sumador de 16 bits, diseñado para sumar dos números de 16 bits, con un ejemplo de acción de suma (a la izquierda).



Observamos de paso que el diseño lógico para agregar números de 16 bits se puede extender fácilmente para implementar cualquier chip sumador de  $n$  bits, independientemente de  $n$ .

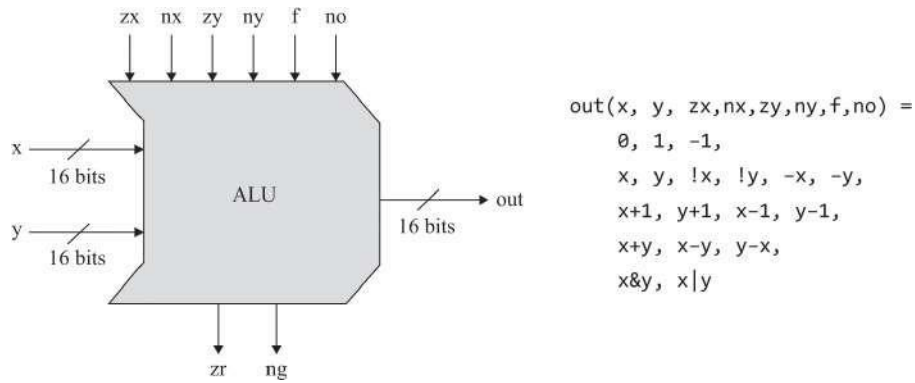
**Incrementador:** Cuando más tarde diseñemos la arquitectura de nuestra computadora, necesitaremos un chip que agregue 1 a un número dado (*Spoiler:* Esto permitirá obtener la siguiente instrucción de la memoria, después de ejecutar la actual). Aunque la  $x+1$  operación se puede realizar con nuestro chip Adder de propuesta general, un *chip incrementador dedicado* puede hacerlo de manera más eficiente. Aquí está la interfaz del chip:

```
Chip name: Inc16
Input:    in[16]
Output:   out[16]
Function:  out = in + 1
Comment:   The overflow bit is ignored.
```

## 2.5.2 La unidad lógica aritmética

Todos los chips sumadores presentados hasta ahora son genéricos: cualquier computadora que realice operaciones aritméticas usa dichos chips, de una forma u otra. Sobre la base de estos chips, ahora pasamos a describir una *Unidad Lógica Aritmética*, un chip que luego se convertirá en la pieza central computacional de nuestra CPU. A diferencia de las puertas y chips genéricos discutidos hasta ahora, el diseño de ALU es exclusivo de la computadora construida en Nand to Tetris, llamada Hack. Dicho esto, los principios de diseño que subyacen al Hack ALU son generales e instructivos. Además, nuestra arquitectura ALU logra una gran funcionalidad utilizando un conjunto mínimo de piezas internas. En ese sentido, proporciona un buen ejemplo de un diseño lógico eficiente y elegante.

Como su nombre lo indica, una unidad lógica aritmética es un chip diseñado para calcular un conjunto de operaciones aritméticas y lógicas. Exactamente *qué* operaciones debe presentar una ALU es una decisión de diseño derivada de consideraciones de rentabilidad. En el caso de la plataforma Hack, decidimos que (i) la ALU realizará solo aritmética entera (y no, por ejemplo, aritmética de coma flotante) y (ii) la ALU calculará el conjunto de dieciocho funciones aritmético-lógicas que se muestran en la [figura 2.5a](#).



**Figura 2.5a** El Hack ALU, diseñado para calcular las dieciocho funciones aritmético-lógicas que se muestran a la derecha (los símbolos !, &, y | representan, respectivamente, las operaciones de 16 bits Not, And y Or). Por ahora, ignore los bits de salida zr y ng.

Como se ve en [la figura 2.5a](#), el Hack ALU opera en dos enteros de complemento de dos de 16 bits, denotados  $x$  e  $y$ , y en seis entradas de 1 bit, llamadas bits de *control*. Estos bits de control "dicen" a la ALU qué función calcular. La especificación exacta se da en [la figura 2.5b](#).

| pre-setting<br>the x input |                    | pre-setting<br>the y input |                    | computing<br>+ or &                  | post-setting<br>the output | resulting<br>ALU output |
|----------------------------|--------------------|----------------------------|--------------------|--------------------------------------|----------------------------|-------------------------|
| if zx<br>then x=0          | if nx<br>then x=!x | if zy<br>then y=0          | if ny<br>then y=!y | if f then<br>out=x+y<br>else out=x&y | if no then<br>out=!out     | out(x,y) =              |
| zx                         | nx                 | zy                         | ny                 | f                                    | no                         | out                     |
| 1                          | 0                  | 1                          | 0                  | 1                                    | 0                          | 0                       |
| 1                          | 1                  | 1                          | 1                  | 1                                    | 1                          | 1                       |
| 1                          | 1                  | 1                          | 0                  | 1                                    | 0                          | -1                      |
| 0                          | 0                  | 1                          | 1                  | 0                                    | 0                          | x                       |
| 1                          | 1                  | 0                          | 0                  | 0                                    | 0                          | y                       |
| 0                          | 0                  | 1                          | 1                  | 0                                    | 1                          | !x                      |
| 1                          | 1                  | 0                          | 0                  | 0                                    | 1                          | !y                      |
| 0                          | 0                  | 1                          | 1                  | 1                                    | 1                          | -x                      |
| 1                          | 1                  | 0                          | 0                  | 1                                    | 1                          | -y                      |
| 0                          | 1                  | 1                          | 1                  | 1                                    | 1                          | x+1                     |
| 1                          | 1                  | 0                          | 1                  | 1                                    | 1                          | y+1                     |
| 0                          | 0                  | 1                          | 1                  | 1                                    | 0                          | x-1                     |
| 1                          | 1                  | 0                          | 0                  | 1                                    | 0                          | y-1                     |
| 0                          | 0                  | 0                          | 0                  | 1                                    | 0                          | x+y                     |
| 0                          | 1                  | 0                          | 0                  | 1                                    | 1                          | x-y                     |
| 0                          | 0                  | 0                          | 1                  | 1                                    | 1                          | y-x                     |
| 0                          | 0                  | 0                          | 0                  | 0                                    | 0                          | x&y                     |
| 0                          | 1                  | 0                          | 1                  | 0                                    | 1                          | x y                     |

**Figura 2.5b** En conjunto, los valores de los seis bits de control zx, nx, zy, Ny, fy No hacer que la ALU calcule una de las funciones enumeradas en la columna situada más a la derecha.

Para ilustrar la lógica de ALU, supongamos que deseamos calcular la función  $x - 1$ , para  $x = 27$ . Para comenzar, alimentamos el código binario de 16 bits de 27 en la entrada x. En este ejemplo en particular, no nos importa el valor de y, ya que no tiene ningún impacto en el cálculo requerido. Ahora, mirando hacia arriba  $x - 1$  en la [figura 2.5b](#), establecemos los bits de control de la ALU en 001110. De acuerdo con la especificación, esta configuración debería hacer que la ALU genere el código binario que representa 26.

¿Es así? Para averiguarlo, profundicemos más y calculemos cómo el Hack ALU realiza su magia. Centrándose en la fila superior de la [figura 2.5b](#), observe que cada uno de los seis bits de control está asociado con un

microacción. Por ejemplo, el bit  $z_x$  está asociado con "if ( $z_x=1$ ) then set  $x$  to 0". Estas seis directivas deben realizarse en orden: primero, establecemos las entradas  $x$  e  $y$  en 0, o no; luego, negamos los valores resultantes, o no; luego, calculamos  $+$  o  $\&$  en los valores preprocesados; y finalmente, negamos el valor resultante, o no. Todas estas configuraciones, negaciones, adiciones y conjunciones son operaciones de 16 bits.

Con esta lógica en mente, revisemos la fila asociada con  $x-1$  y verifiquemos que las microoperaciones codificadas por los seis bits de control harán que la ALU calcule  $x-1$ . Yendo de izquierda a derecha, vemos que los bits  $z_x$  y  $n_x$  son 0, por lo que no ponemos cero ni negamos la entrada  $x$ , la dejamos como está. Los bits  $z_y$  y  $n_y$  son 1, por lo que primero ponemos a cero la entrada  $y$  y luego negamos el resultado, lo que produce el valor de 16 bits 1111111111111111. Dado que este código binario representa el  $-1$  complemento de dos, vemos que las dos entradas de datos del ALU ahora son el valor de  $x$  y  $-1$ . Dado que el bit  $f$  es 1, la operación seleccionada es la *suma*, lo que hace que la ALU calcule  $x + (-1)$ . Finalmente, dado que el bit  $no$  es 0, la salida no se niega. Para concluir, hemos ilustrado que si alimentamos la ALU con valores  $x$  e  $y$  y establecemos los seis bits de control en 001110, la ALU calculará  $x-1$ , según lo especificado.

¿Qué pasa con las otras diecisiete funciones enumeradas en la [figura 2.5b](#)? ¿El ALU realmente los calcula también? Para verificar que este es realmente el caso, se le invita a concentrarse en otras filas de la tabla, pasar por el mismo proceso de llevar a cabo las microacciones codificadas por los seis bits de control y averiguar por sí mismo qué generará la ALU. O bien, puede creernos que el ALU funciona como se anuncia.

Tenga en cuenta que la ALU en realidad calcula un total de sesenta y cuatro funciones, ya que seis bits de control codifican muchas posibilidades. Hemos decidido centrarnos y documentar solo dieciocho de estas posibilidades, ya que serán suficientes para respaldar el conjunto de instrucciones de nuestro sistema informático objetivo. El lector curioso puede estar intrigado al saber que algunas de las operaciones de ALU no documentadas son bastante significativas. Sin embargo, hemos optado por no explotarlos en el sistema Hack.

La interfaz Hack ALU se muestra en la [figura 2.5c](#). Tenga en cuenta que además de calcular la función especificada en sus dos entradas, la ALU también calcula los dos bits de salida  $z_r$  y  $ng$ . Estos bits, que marcan si la salida de ALU es cero o negativa, respectivamente, serán utilizados por la futura CPU de nuestro sistema informático.

```

Chip name: ALU
Input:  x[16], y[16], // Two 16-bit data inputs
        zx,          // Zero the x input
        nx,          // Negate the x input
        zy,          // Zero the y input
        ny,          // Negate the y input
        f,           // if f==1 out=add(x,y) else out=and(x,y)
        no           // Negate the out output
Output: out[16],      // 16-bit output
        zr,          // if out==0 zr=1 else zr=0
        ng           // if out<0  ng=1 else ng=0

Function:
        if zx x=0      // 16-bit zero constant
        if nx x=!x     // Bit-wise negation
        if zy y=0      // 16-bit zero constant
        if ny y=!y     // Bit-wise negation
        if f out=x+y   // Integer two's complement addition
        else out=x&y   // Bit-wise And
        if no out=!out // Bit-wise negation
        if out==0 zr=1 else zr=0 // 16-bit equality comparison
        if out<0 ng=1  else ng=0 // two's complement comparison

Comment: The overflow bit is ignored.

```

**Figura 2.5c** La API de Hack ALU.

Puede ser instructivo describir el proceso de pensamiento que condujo al diseño de nuestra ALU. Primero, hicimos una lista tentativa de las operaciones primitivas que queríamos que realizara nuestra computadora (columna derecha de la [figura 2.5b](#)). A continuación, utilizamos el razonamiento hacia atrás para averiguar cómo  $x$ ,  $y$  y  $out$  pueden manipularse de forma binaria para llevar a cabo las operaciones deseadas. Estos requisitos de procesamiento, junto con nuestro objetivo de mantener la lógica de ALU lo más simple posible, han llevado a la decisión de diseño de utilizar seis bits de control, cada uno asociado con una operación sencilla que se puede implementar fácilmente con puertas lógicas básicas. El ALU resultante es simple y elegante. Y en el negocio de la ferretería, la simplicidad y la elegancia llevan el día.

---

## 2.6 Implementación

Nuestras pautas de implementación son intencionalmente mínimas. Ya dimos muchos consejos de implementación en el camino, y ahora es su turno de descubrir las partes que faltan en las arquitecturas de chips.

A lo largo de esta sección, cuando decimos "construir/implementar un diseño lógico que...", esperamos que (i) descubra el diseño lógico (por ejemplo, dibujando un diagrama de puerta), (ii) escriba el código HDL que realiza el diseño y (iii) pruebe y depure su diseño utilizando los scripts de prueba y el simulador de hardware suministrados. En la siguiente sección se dan más detalles, que describe el proyecto 2.

**Medio sumador:** Una inspección de la tabla de verdad en la [figura 2.2](#) revela que las salidas  $\text{sum}(a,b)$  y  $\text{carry}(a,b)$  resultan ser idénticas a las de dos funciones booleanas simples discutidas e implementadas en el proyecto 1. Por lo tanto, la implementación de medio sumador es sencilla.

**Sumador completo:** Se puede implementar un chip de sumador completo a partir de dos medios sumadores y una puerta adicional (y es por eso que estos sumadores se llaman *medio* y *completo*). Otras implementaciones son posibles, incluidos los enfoques directos que no usan medio sumadores.

**Sumador:** La suma de dos números de  $n$  bits se puede hacer bit a bit, de derecha a izquierda. En el paso 0, se agrega el par de bits menos significativo y el bit de transporte resultante se alimenta a la adición del siguiente par significativo de bits. El proceso continúa hasta que se agrega el par de los bits más significativos. Tenga en cuenta que cada paso implica la adición de tres bits, uno de los cuales se propaga a partir de la adición "anterior".

Los lectores pueden preguntarse cómo podemos agregar pares de bits "en paralelo" antes de que el bit de acarreo haya sido calculado por el par de bits anterior. La respuesta es que estos cálculos son lo suficientemente rápidos como para completarse y estabilizarse dentro de un ciclo de reloj. Discutiremos los ciclos de reloj y la sincronización en el próximo capítulo; por ahora, puede ignorar el elemento `time` por completo y escribir código HDL que calcule la operación de suma actuando sobre todos los pares de bits simultáneamente.

**Incrementador:** Un incrementador de  $n$  bits se puede implementar fácilmente de diferentes maneras.

**ALU:** Nuestra ALU fue cuidadosamente planificada para efectuar todas las operaciones ALU deseadas *de manera lógica*, utilizando las operaciones booleanas simples implícitas en las seis

bits de control. Por lo tanto, la *implementación física* de la ALU se puede reducir a la implementación de estas operaciones simples, siguiendo las especificaciones de pseudocódigo enumeradas en la parte superior de la [figura 2.5b](#). Es probable que el primer paso sea crear un diseño lógico para poner a cero y negar un valor de 16 bits. Esta lógica se puede utilizar para manejar las entradas  $x$  e  $y$ , así como la salida de salida. Los chips para And bit a bit y la adición ya se han construido en los proyectos 1 y 2, respectivamente. Por lo tanto, lo que queda es construir una lógica que seleccione entre estas dos operaciones, de acuerdo con el bit de control  $f$  (esta lógica de selección también se implementó en el proyecto 1). Una vez que esta funcionalidad principal de ALU funcione correctamente, puede proceder a implementar la funcionalidad requerida de las salidas  $zr$  y  $ng$  de un solo bit.

---

## 2.7 Proyecto

**Objetivo:** Implementar todos los chips presentados en este capítulo. Los únicos bloques de construcción que necesita son algunas de las puertas descritas en el capítulo 1 y los chips que construirá gradualmente en este proyecto.

**Chips incorporados:** Como se acaba de decir, los chips que construirá en este proyecto usan, como partes de chips, algunos de los chips descritos en el capítulo 1. Incluso si ha construido estos chips de nivel inferior con éxito en HDL, le recomendamos que utilice sus versiones integradas en su lugar. Como consejo de mejores prácticas relacionado con todos los proyectos de hardware en Nand to Tetris, siempre prefiera usar piezas de chip integradas en lugar de sus implementaciones HDL. Los chips incorporados están garantizados para funcionar según las especificaciones y están diseñados para acelerar el funcionamiento del simulador de hardware.

Hay una forma sencilla de seguir este consejo de mejores prácticas: No agregue a la carpeta del proyecto `nand2tetris/projects/02` ningún archivo `.hdl` del proyecto 1. Siempre que el simulador de hardware encuentre en su código HDL una referencia a una pieza de chip del proyecto 1, por ejemplo, `And16`, comprobará si hay un archivo `And16.hdl` en la carpeta actual. Si no lo encuentra, el simulador de hardware recurrirá por defecto a utilizar la versión incorporada de este chip, que es exactamente lo que queremos.

Las directrices restantes para este proyecto son idénticas a las del proyecto 1. En particular, recuerde que los buenos programas HDL usan tan pocas piezas de chip como



posible, y no hay necesidad de inventar e implementar ningún "chip auxiliar"; sus programas HDL deben usar solo chips que se especificaron en los capítulos 1 y 2.

**Una versión basada en la web del proyecto 2** está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

---

## 2.8 Perspectiva

La construcción del sumador de múltiples bits presentado en este capítulo era estándar, aunque no se prestó atención a la eficiencia. De hecho, nuestra implementación de sumador sugerida es ineficiente, debido a los retrasos incurridos mientras los bits de transporte se propagan a través de los sumandos de  $n$  bits. Este cálculo puede ser acelerado por circuitos lógicos que afectan a la llamada *heurística de búsqueda anticipada*. Dado que la adición es la operación más frecuente en las arquitecturas de computadoras, cualquier mejora de bajo nivel puede resultar en ganancias dramáticas de rendimiento en todo el sistema. Sin embargo, en este libro nos enfocamos principalmente en la funcionalidad, dejando la optimización de chips a libros y cursos de hardware más especializados. [número arábigo](#)

La funcionalidad general de cualquier sistema de hardware/software es entregada conjuntamente por la CPU y el sistema operativo que se ejecuta sobre la plataforma de hardware. Por lo tanto, al diseñar un nuevo sistema informático, la cuestión de cómo asignar la funcionalidad deseada entre la ALU y el sistema operativo es esencialmente un dilema de costo / rendimiento. Como regla general, las implementaciones directas de hardware de operaciones aritméticas y lógicas son más eficientes que las implementaciones de software, pero encarecen la plataforma de hardware.



La compensación que hemos elegido en Nand to Tetris es diseñar una ALU básica con una funcionalidad mínima y usar el software del sistema para implementar operaciones matemáticas adicionales según sea necesario. Por ejemplo, nuestro ALU no presenta ni multiplicación ni división. En la parte II del libro, cuando discutamos el sistema operativo (capítulo 12), implementaremos algoritmos bit a bit elegantes y eficientes para la multiplicación y la división, junto con otras operaciones matemáticas. Estas rutinas del sistema operativo pueden ser utilizadas por compiladores de lenguajes de alto nivel que operan sobre la plataforma Hack. Por lo tanto, cuando un programador de alto nivel escribe una expresión como, por ejemplo,  $x * 12 + \text{sqrt}(y)$ , después de la compilación, algunas partes de la expresión serán evaluadas directamente por la ALU y otras por el sistema operativo, pero el programador de alto nivel será completamente ajeno a esta división de trabajo de bajo nivel. De hecho, una de las funciones clave de un sistema operativo es cerrar las brechas entre las abstracciones de lenguaje de alto nivel que usan los programadores y el hardware básico en el que se realizan.

---

1. Que corresponden, respectivamente, a los tipos de datos típicos de alto nivel *byte*, *short*, *int* y *long*. Por ejemplo, cuando se reducen a instrucciones a nivel de máquina, las variables cortas pueden ser manejadas por registros de 16 bits. Dado que la aritmética de 16 bits es cuatro veces más rápida que la aritmética de 64 bits, se recomienda a los programadores que utilicen siempre el tipo de datos más compacto que satisfaga los requisitos de la aplicación.

2. Una razón técnica para no utilizar técnicas de anticipación de transporte en nuestros chips sumadores es que su implementación de hardware requiere conexiones de pines cíclicos, que no son compatibles con el simulador de hardware de Nand a Tetris.

---

## 3 Memoria

Es un tipo de memoria pobre que solo funciona hacia atrás.

—Lewis Carroll (1832–1898)

Considere la operación de alto nivel  $x = y + 17$ . En el capítulo 2 mostramos cómo se pueden utilizar las puertas lógicas para representar números y para calcular expresiones aritméticas simples como  $y + 17$ . Ahora pasamos a discutir cómo se pueden usar las puertas lógicas para *almacenar valores a lo largo del tiempo*, en particular, cómo se puede configurar una variable como  $x$  para "contener" un valor y persistirlo hasta que lo establezcamos en otro valor. Para ello, desarrollaremos una nueva familia de chips de *memoria*.

Hasta ahora, todos los chips que construimos en los capítulos 1 y 2, que culminaron con el ALU, eran independientes del tiempo. Tales chips a veces se denominan *combinacionales*: responden a diferentes combinaciones de sus entradas sin demora, excepto por el tiempo que tardan sus partes internas en completar el cálculo. En este capítulo presentamos y construimos chips secuenciales. A diferencia de los chips combinados, que son ajenos al tiempo, las salidas de los chips secuenciales dependen no solo de las entradas en el tiempo actual, sino también de las entradas y salidas que se han procesado previamente.

No hace falta decir que las nociones de *actual* y *anterior* van de la mano con la noción de *tiempo*: ahora recuerdas lo que antes se memorizaba. Por lo tanto, antes de comenzar a hablar de memoria, primero debemos descubrir cómo usar la lógica para modelar la progresión del tiempo. Esto se puede hacer usando un *reloj* que genera un tren continuo de señales binarias que llamamos *tic-tac*. El tiempo entre el comienzo de un tic y el final del siguiente toque se llama *ciclo*, y estos ciclos se utilizarán para regular las operaciones de todos los chips de memoria utilizados por la computadora.

Después de una breve introducción orientada al usuario a los dispositivos de memoria, presentaremos el arte de la lógica secuencial, que usaremos para construir chips dependientes del tiempo. Luego nos dispondremos a construir registros, dispositivos RAM y contadores. Estos dispositivos de memoria, junto con los dispositivos aritméticos construidos en el capítulo 2, comprenden todos los chips necesarios para construir un sistema informático completo de propósito general, un desafío que abordaremos en el capítulo 5.

---

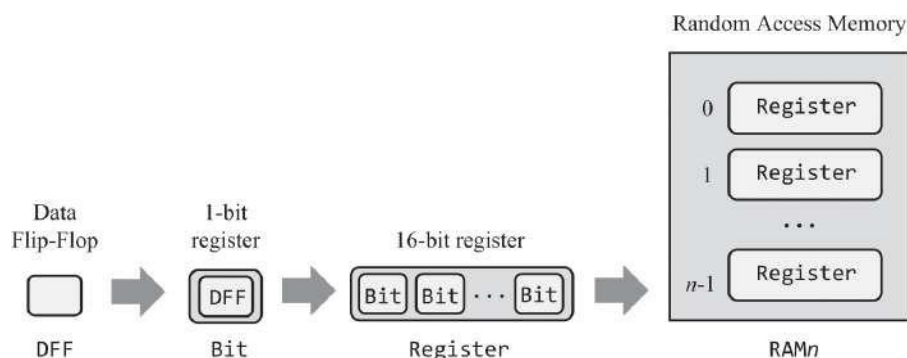
### 3.1 Dispositivos de memoria

Los programas informáticos utilizan variables, matrices y objetos, abstracciones que conservan los datos a lo largo del tiempo. Las plataformas de hardware admiten esta capacidad al ofrecer dispositivos de memoria que saben cómo mantener el *estado*. Debido a que la evolución le dio a los humanos un sistema de memoria electroquímica fenomenal, tendemos a dar por sentada la capacidad de recordar cosas a lo largo del tiempo. Sin embargo, esta capacidad es difícil de implementar en la lógica clásica, que no es consciente ni del tiempo ni del estado. Por lo tanto, para comenzar, primero debemos encontrar una manera de modelar la progresión del tiempo y dotar a las puertas lógicas con la capacidad de mantener el estado y responder a los cambios de tiempo.

Abordaremos este desafío introduciendo un reloj y una puerta lógica elemental dependiente del tiempo que puede cambiar y cambiar entre dos estados estables: representar 0 y representar 1. Esta puerta, llamada *flip-flop de datos* (DFF), es el bloque de construcción fundamental a partir del cual se construirán todos los dispositivos de memoria. Sin embargo, a pesar de su papel central, el DFF es una puerta discreta y de bajo perfil: a diferencia de los registros, los dispositivos RAM y los contadores, que juegan un papel destacado en las arquitecturas de computadoras, los DFF se usan implícitamente, como partes de chips de bajo nivel incrustadas en lo profundo de otros dispositivos de memoria.

El papel fundamental del DFF se ve claramente en la [figura 3.1](#), donde sirve como base de la jerarquía de memoria que estamos a punto de construir. Mostraremos cómo se pueden usar los DFF para crear registros de 1 bit y cómo *se* pueden unir  $n$  estos registros para crear un registro de  $n$  bits. A continuación, construiremos un dispositivo RAM que contenga un número arbitrario de tales registros. Entre otras cosas, desarrollaremos un medio para *direccionar*, es decir, acceder por dirección, a cualquier registro

elegido aleatoriamente desde la RAM de forma directa e instantánea.



**Figura 3.1** La jerarquía de memoria construida en este capítulo.

Sin embargo, antes de comenzar a construir estos chips, presentaremos una metodología y herramientas que permiten modelar la progresión del tiempo y mantener el estado a lo largo del tiempo.

## 3.2 Lógica secuencial

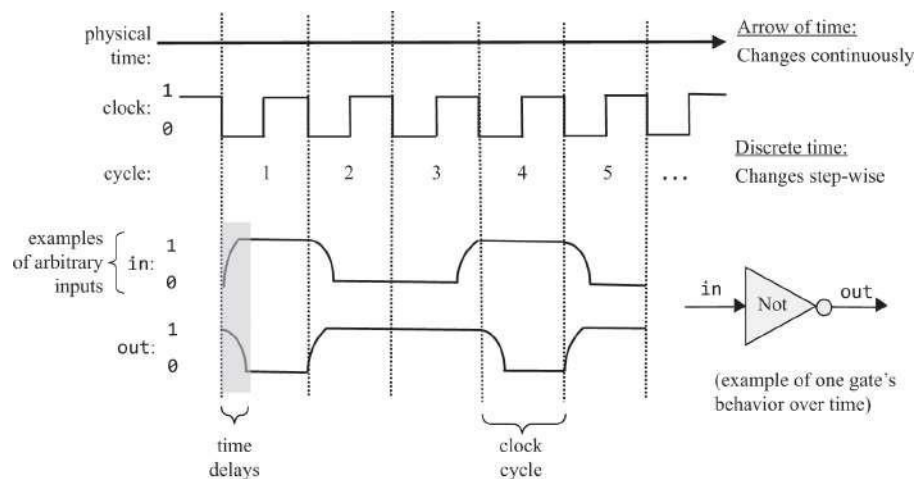
Todos los chips discutidos en los capítulos 1 y 2 se basaron en la lógica clásica, que es independiente del tiempo. Para desarrollar dispositivos de memoria, necesitamos extender nuestra lógica de puerta con la capacidad de responder no solo a los cambios de entrada sino también al tictac de un reloj: recordamos el significado de la palabra *perro* en el tiempo  $t$  ya que la recordamos en el tiempo  $t - 1$ , hasta el momento en que la memorizamos por primera vez. Para desarrollar esta capacidad temporal de mantener el *estado*, debemos ampliar nuestra arquitectura de computadora con una dimensión de tiempo y construir herramientas que manejen el tiempo usando funciones booleanas.

### 3.2.1 El tiempo importa

Hasta ahora en nuestro viaje de Nand a Tetris, hemos asumido que los chips responden a sus entradas instantáneamente: ingresa 7, 2 y "resta" en la ALU, y ... ¡puf! la salida de ALU se convierte en 5. En realidad, las salidas siempre se retrasan, debido al menos a dos razones. Primero, las entradas de los chips no aparecen de la nada; más bien, las señales que los representan viajan desde las salidas de otros chips, y este viaje lleva tiempo. En segundo lugar, los cálculos que realizan los chips también llevan tiempo; cuantas más partes de chip tenga el chip, el

cuanto más elaborada sea su lógica, más tiempo tardarán las salidas del chip en emerger completamente formadas de los circuitos del chip.

Por lo tanto, *el tiempo* es un tema que debe tratarse. Como se ve en la parte superior de la [figura 3.2](#), el tiempo generalmente se ve como una flecha metafórica que avanza implacablemente. Esta progresión se considera continua: entre cada dos puntos de tiempo hay otro punto de tiempo, y los cambios en el mundo pueden ser infinitesimalmente pequeños. Esta noción del tiempo, que es popular entre filósofos y físicos, es demasiado profunda y misteriosa para los informáticos. Por lo tanto, en lugar de ver el tiempo como una progresión continua, preferimos dividirlo en intervalos de duración fija, llamados *ciclos*. Esta representación es discreta, lo que da como resultado el ciclo 1, el ciclo 2, el ciclo 3, etc. A diferencia de la flecha continua del tiempo, que tiene una granularidad infinita, los ciclos son atómicos e indivisibles: los cambios en el mundo ocurren solo durante las transiciones del ciclo; Dentro de los ciclos, el mundo se detiene.



**Figura 3.2** Representación de tiempo discreta: los cambios de estado (valores de entrada y salida) se observan solo durante las transiciones de ciclo. Dentro de los ciclos, los cambios se ignoran.

Por supuesto, el mundo nunca se detiene. Sin embargo, al tratar el tiempo discretamente, tomamos la decisión consciente de ignorar el cambio continuo. Nos contentamos con conocer el estado del mundo en el ciclo  $n$ , y luego en el ciclo  $n+1$ , pero *durante* cada ciclo se supone que el estado es, bueno, no nos importa. Cuando se trata de construir arquitecturas informáticas, esta visión discreta del tiempo sirve a dos objetivos de diseño importantes. En primer lugar, se puede utilizar para neutralizar la aleatoriedad asociada con las comunicaciones y

retrasos en el tiempo de cálculo. En segundo lugar, se puede utilizar para sincronizar las operaciones de diferentes chips en todo el sistema, como veremos más adelante.

Para ilustrarlo, centrémonos en la parte inferior de la [figura 3.2](#), que rastrea cómo una puerta Not (utilizada como ejemplo) responde a entradas elegidas arbitrariamente. Cuando alimentamos la puerta con 1, pasa un tiempo antes de que la salida de la puerta se estabilice en 0. Sin embargo, dado que la duración del ciclo es, *por diseño*, más larga que el retardo de tiempo, cuando llegamos al final del ciclo, la salida de la puerta ya se ha estabilizado en 0. Dado que sondeamos el estado del mundo solo al final del ciclo, no podemos ver los retrasos de tiempo intermedios; más bien, parece como si hubiéramos alimentado la puerta con 0, ¡y *puf!* La puerta respondió con 1. Si hacemos las mismas observaciones al final de cada ciclo, podemos generalizar que cuando una puerta Not se alimenta con alguna entrada binaria  $x$ , instantáneamente genera  $\text{Not}(x)$ .

Los lectores reflexivos probablemente hayan notado que para que este esquema funcione, la *duración del ciclo* debe ser más larga que los retrasos máximos que pueden ocurrir en el sistema. De hecho, la longitud del ciclo es uno de los parámetros de diseño más importantes de cualquier plataforma de hardware: al planificar una computadora, el ingeniero de hardware elige una longitud de ciclo que cumpla con dos objetivos de diseño. Por un lado, el ciclo debe ser lo suficientemente largo como para contener y neutralizar cualquier posible retraso de tiempo; Por otro lado, cuanto más corto es el ciclo, más rápido es el ordenador: si las cosas suceden sólo durante las transiciones del ciclo, entonces obviamente las cosas suceden más rápido cuando los ciclos son más cortos. En resumen, la duración del ciclo se elige para que sea ligeramente más larga que el retardo de tiempo máximo en cualquier chip del sistema. Tras el tremendo progreso en las tecnologías de conmutación, ahora podemos crear ciclos tan pequeños como una milmillonésima de segundo, logrando una velocidad de computadora notable.

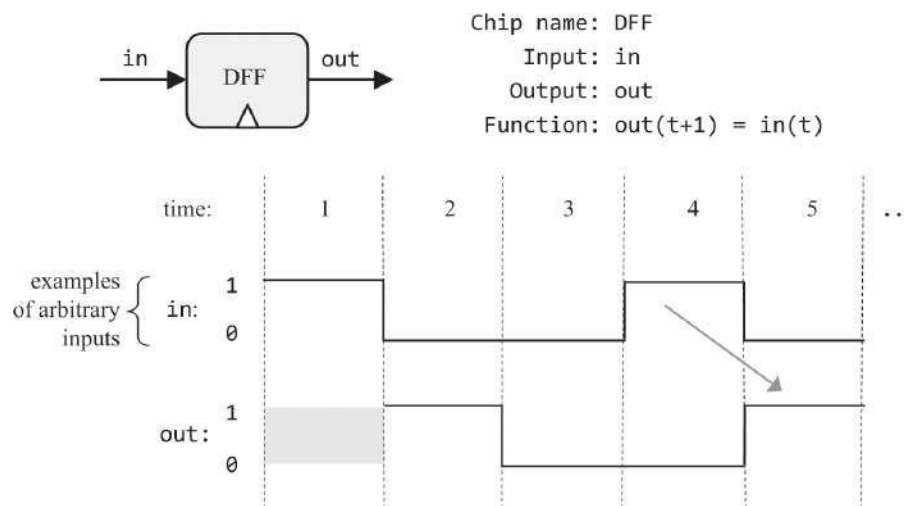
Por lo general, los ciclos se realizan mediante un oscilador que alterna continuamente entre dos fases etiquetadas como 0-1, *bajo-alto* o *tictac* (como se ve en [la figura 3.2](#)). El tiempo transcurrido entre el comienzo de un tic y el final del tock posterior se denomina *ciclo*, y cada ciclo se toma para modelar una unidad de tiempo discreta. La fase del reloj actual (*tic-tac*) está representada por una señal binaria. Usando los circuitos del hardware, la misma señal de reloj maestro se transmite simultáneamente a cada chip de memoria en el sistema. En cada uno de estos chips, la entrada del reloj se canaliza a las puertas DFF de nivel inferior, donde sirve para garantizar que

el chip se comprometa con un nuevo estado y lo emita, solo al final del ciclo del reloj.



### 3.2.2 Chancas

Los chips de memoria están diseñados para "recordar" o almacenar información a lo largo del tiempo. Los dispositivos de bajo nivel que facilitan esta abstracción de almacenamiento se denominan *puertas flip-flop*, de las cuales existen varias variantes. En Nand to Tetris usamos una variante llamada *data flip-flop*, o DFF, cuya interfaz incluye una entrada de datos de un solo bit y una salida de datos de un solo bit (ver parte superior de [la figura 3.3](#)). Además, el DFF tiene una entrada de reloj que se alimenta de la señal del reloj maestro. En conjunto, la entrada de datos y la entrada de reloj permiten al DFF implementar el comportamiento simple basado en el tiempo  $out(t) = in(t-1)$ , donde  $in$  y  $out$  son los valores de entrada y salida de la puerta, y  $t$  es la unidad de tiempo actual (de ahora en adelante, usaremos los términos "unidad de tiempo" y "ciclo" indistintamente). No nos preocupemos por cómo se implementa realmente este comportamiento. Por ahora, simplemente observamos que al final de cada unidad de tiempo, el DFF genera el valor de entrada de la unidad de tiempo anterior.



**Figura 3.3** El cambio de datos (arriba) y el ejemplo de comportamiento (abajo). En el primer ciclo, se desconoce la entrada anterior, por lo que la salida del DFF no está definida. En cada unidad de tiempo posterior, el DFF genera la entrada de la unidad de tiempo anterior. Siguiendo las convenciones de diagramación de puertas, la entrada del reloj está marcada por un pequeño triángulo, dibujado en la parte inferior del icono de la puerta.

Al igual que las puertas Nand, las puertas DFF se encuentran en lo profundo de la jerarquía de hardware. Como se muestra en [la figura 3.1](#), todos los chips de memoria en la computadora (registros, unidades de RAM y contadores) se basan, en la parte inferior, en puertas DFF. Todos estos DFF están conectados al mismo reloj maestro, formando un enorme

"chorus" distribuido

línea". Al final de cada ciclo de reloj, las salidas de *todos los* DFF en la computadora se confirman en sus entradas del ciclo anterior. En todos los demás momentos, los DFF están *bloqueados*, lo que significa que los cambios en sus entradas no tienen un efecto inmediato en sus salidas. Esta operación de conducción afecta a cualquiera de las numerosas puertas DFF del sistema muchas veces por segundo (dependiendo de la frecuencia de reloj de la computadora).

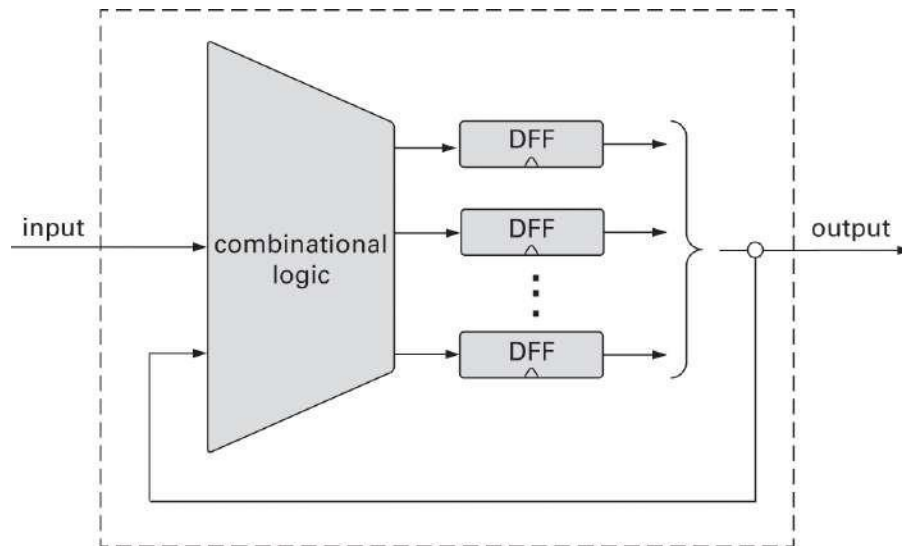
Las implementaciones de hardware realizan la dependencia del tiempo utilizando un bus de reloj dedicado que alimenta la señal de reloj maestro simultáneamente a todas las puertas DFF del sistema. Los simuladores de hardware emulan el mismo efecto en el software. En particular, el simulador de hardware de Nand a Tetris cuenta con un icono de reloj, lo que permite al usuario avanzar el reloj de forma interactiva, así como comandos de tic-tac que se pueden usar mediante programación, en scripts de prueba.

### 3.2.3 Lógica combinacional y secuencial

Todos los chips que se desarrollaron en los capítulos 1 y 2, comenzando con las puertas lógicas elementales y culminando con la ALU, fueron diseñados para responder solo a los cambios que ocurren durante el ciclo de reloj actual. Tales chips se denominan *chips independientes del tiempo* o chips *combinados*. Este último nombre alude al hecho de que estos chips responden solo a diferentes combinaciones de sus valores de entrada, sin prestar atención a la progresión del tiempo.

Por el contrario, los chips que están diseñados para responder a los cambios que ocurrieron durante las unidades de tiempo anteriores (y posiblemente también durante la unidad de tiempo actual) se denominan *secuenciales* o *sincronizados*. La puerta secuencial más fundamental es el DFF, y cualquier chip que lo incluya, ya sea directa o indirectamente, también se dice que es secuencial. [La Figura 3.4](#) muestra una configuración lógica secuencial típica. El elemento principal en esta configuración es un conjunto de uno o más chips que incluyen partes de chip DFF, ya sea directa o indirectamente. Como se muestra en la figura, estos chips secuenciales también pueden interactuar con chips combinados. El bucle de retroalimentación permite que el chip secuencial responda a las entradas y salidas de la unidad de tiempo anterior. En los chips combinados, donde el tiempo no se modela ni se reconoce, la introducción de bucles de retroalimentación es problemática: la salida del chip dependería de su entrada, que a su vez dependería de la salida, y por lo tanto la salida dependería de sí misma. Sin embargo, tenga en cuenta que no hay dificultad para alimentarse

devuelve a las entradas, siempre que el bucle de retroalimentación pase por una puerta DFF: el DFF introduce un retardo de tiempo inherente para que la salida en el tiempo  $t$  no dependa de sí misma, sino de la salida en el tiempo  $t-1$ .



**Figura 3.4** El diseño de lógica secuencial generalmente involucra puertas DFF que se alimentan y se conectan a chips combinados. Esto le da a los chips secuenciales la capacidad de responder a la corriente, así como a las entradas y salidas anteriores.

La dependencia del tiempo de los chips secuenciales tiene un efecto secundario importante que sirve para sincronizar la arquitectura general de la computadora. Para ilustrar, supongamos que instruimos a la ALU para que calcule  $x + y$ , donde  $x$  es la salida de un registro cercano e  $y$  es la salida de un registro RAM remoto. Debido a limitaciones físicas como la distancia, la resistencia y la interferencia, las señales eléctricas que representan  $x$  e  $y$  probablemente llegarán a la ALU en diferentes momentos. Sin embargo, al ser un chip combinatorio, el ALU es insensible al concepto de tiempo: suma continua y felizmente los valores de datos que se alojan en sus entradas. Por lo tanto, pasará algún tiempo antes de que la salida de la ALU se establezca en el resultado correcto  $x + y$ . Hasta entonces, el ALU generará basura.

¿Cómo podemos superar esta dificultad? Bueno, si usamos una representación discreta del tiempo, *simplemente no nos importa*. Todo lo que tenemos que hacer es asegurarnos, cuando construimos el reloj de la computadora, de que la duración del ciclo del reloj será un poco más larga que el tiempo que tarda un poco en recorrer la distancia más larga de un chip a otro, más el tiempo que lleva completar el ciclo de reloj.

cálculo dentro del chip que consume más tiempo. De esta manera, tenemos la garantía de que al final del ciclo de reloj, la salida de la ALU será válida. Esto, en pocas palabras, es el truco que convierte un conjunto de componentes de hardware independientes en un sistema bien sincronizado. Tendremos más que decir sobre esta orquestación maestra cuando construyamos la arquitectura de la computadora en el capítulo 5.

---

### 3.3 Especificación

Ahora pasamos a especificar los chips de memoria que se utilizan normalmente en las arquitecturas de computadoras:

- flip-flops de datos (DFF)
- registros (basados en DFF)
- Dispositivos RAM (basados en registros)
- c o n t a d o r e s ( b a s a d o s e n r e g i s t r o s )

Como de costumbre, describimos estos chips *de forma abstracta*. En particular, nos enfocamos en la interfaz de cada chip: entradas, salidas y función. La forma en que los chips ofrecen esta funcionalidad se discutirá en la sección Implementación.

#### 3.3.1 Chancía de fecha

El dispositivo secuencial más elemental que usaremos, el componente básico a partir del cual se construirán todos los demás chips de memoria, es el *flip-flop de datos*. Una puerta DFF tiene una entrada de datos de un solo bit, una salida de datos de un solo bit, una entrada de reloj y un comportamiento simple dependiente del tiempo:  $out(t) = in(t-1)$ .

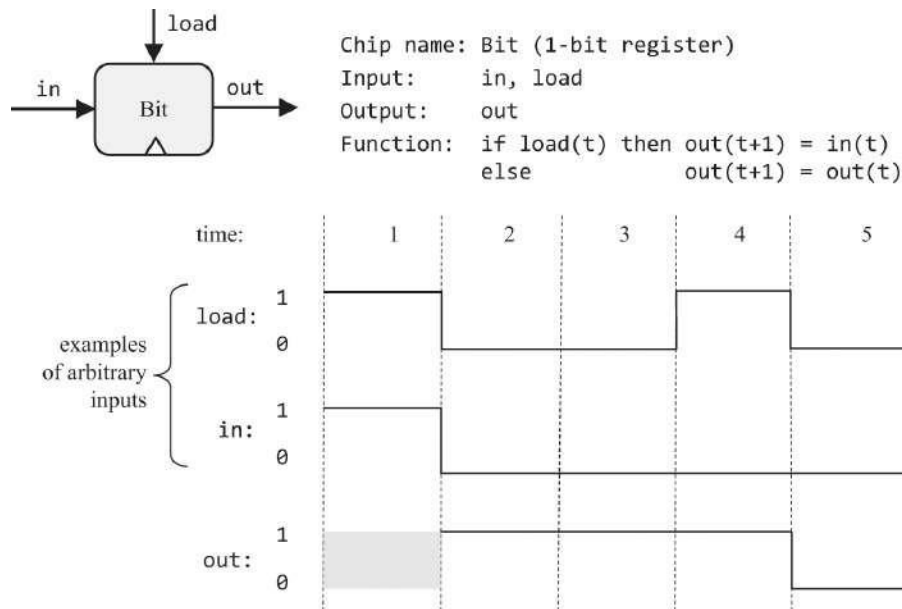
**Uso:** Si ponemos un valor de un bit en la entrada del DFF, el estado del DFF se establecerá en este valor y la salida del DFF lo emitirá en la siguiente unidad de tiempo (ver [figura 3.3](#)). Esta humilde operación resultará muy útil en la implementación de registros, que se describe a continuación.

#### 3.3.2 Registros

Presentamos un registro de un solo bit, llamado Bit, y un registro de 16 bits, llamado

Regístrese. El chip Bit está diseñado para almacenar un solo bit de información: 0 o 1

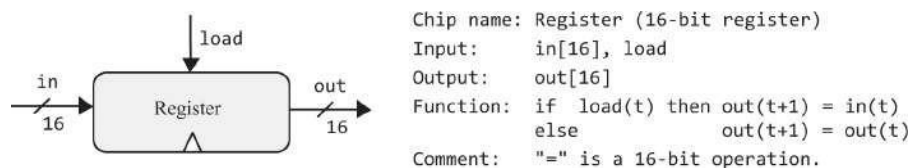
—con el tiempo. La interfaz del chip consta de una entrada que transporta un bit de datos, una entrada de carga que habilita el registro para escrituras y una salida de salida que emite el estado actual del registro. La API de bits y el comportamiento de entrada/salida se describen en [la figura 3.5](#).



**Figura 3.5** Registro de 1 bit. Almacena y emite un valor de 1 bit hasta que se le indica que cargue un nuevo valor.

La [Figura 3.5](#) ilustra cómo se comporta el registro de un solo bit a lo largo del tiempo, respondiendo a ejemplos arbitrarios de entradas de entrada y carga. Tenga en cuenta que, independientemente del valor de entrada, mientras no se afirme el bit de carga, el registro se bloquea, manteniendo su estado actual.

El chip de registro de 16 bits se comporta exactamente igual que el chip de bits, excepto que está diseñado para manejar valores de 16 bits. [La Figura 3.6](#) da los detalles.

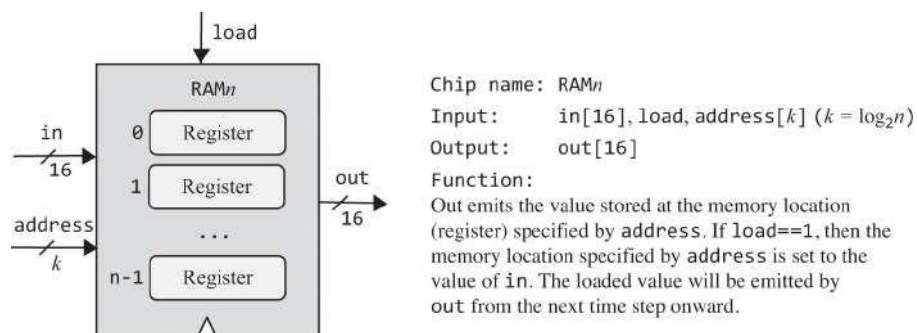


**Figura 3.6** Registro de 16 bits. Almacena y emite un valor de 16 bits hasta que se le indica que cargue un nuevo valor.

**Uso:** El registro de bits y el registro de 16 bits se utilizan de forma idéntica. Para leer el estado del registro, sondee el valor de out. Para establecer el estado del registro en  $v$ , coloque  $v$  en la entrada de entrada y afirme (ponga 1 en) el bit de carga. Esto establecerá el estado del registro en  $v$  y, a partir de la siguiente unidad de tiempo en adelante, el registro se comprometerá con el nuevo valor y su salida de salida comenzará a emitirlo. Vemos que el chip Register cumple la función clásica de un dispositivo de memoria: recuerda y emite el último valor que se escribió en él, hasta que lo establecemos en otro valor.

### 3.3.3 Memoria de acceso aleatorio

Una unidad de memoria de acceso directo, también llamada *memoria de acceso aleatorio* o RAM, es un agregado de  $n$  chips de registro. Al especificar una dirección particular (un número entre 0 y  $n-1$ ), se puede seleccionar cada registro en la RAM y ponerlo a disposición para operaciones de lectura / escritura. Es importante destacar que el tiempo de acceso a cualquier registro de memoria seleccionado al azar es instantáneo e independiente de la dirección del registro y del tamaño de la RAM. Eso es lo que hace que los dispositivos RAM sean tan notablemente útiles: incluso si contienen miles de millones de registros, aún podemos acceder y manipular cada registro seleccionado directamente, en el mismo tiempo de acceso instantáneo. La API de RAM se muestra en la [figura 3.7](#).



**Figura 3.7** Un chip RAM, que consta de  $n$  chips de registro de 16 bits que se pueden seleccionar y manipular por separado. Las direcciones de registro (0 to  $n-1$ ) no forman parte del hardware del chip. Más bien, se realizan mediante una implementación de lógica de puerta que se discutirá en la siguiente sección.

**Uso:** Para leer el contenido del número de registro  $m$ , establezca la entrada de dirección en  $m$ . Esta acción seleccionará el número de registro  $m$  y la salida de la RAM emitirá su valor. Para escribir un nuevo valor  $v$  en el número de registro  $m$ , establezca la dirección



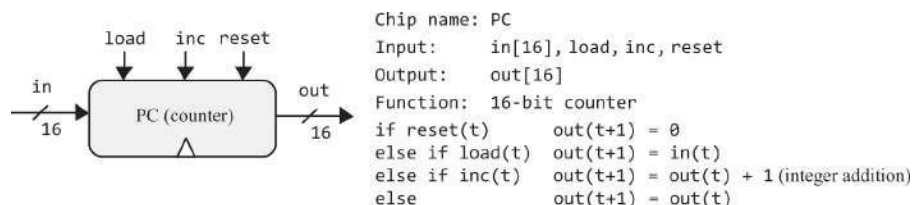
input en  $m$ , establezca la entrada en  $v$  y afirme el bit de carga (establézcalo en 1). Esta acción seleccionará el número de registro  $m$ , lo habilitará para escribir y establecerá su valor en  $v$ . En la próxima unidad de tiempo, la salida de la RAM comenzará a emitir  $v$ .

El resultado neto es que el dispositivo RAM se comporta exactamente como se requiere: un banco de registros direccionables, a cada uno de los cuales se puede acceder y operar de forma independiente. En el caso de una operación de lectura ( $\text{load}=0$ ), la salida de la RAM emite inmediatamente el valor del registro seleccionado. En el caso de una operación de escritura ( $\text{load}=1$ ), el registro de memoria seleccionado se establece en el valor de entrada y la salida de la RAM comenzará a emitirlo a partir de la siguiente unidad de tiempo en adelante.

Es importante destacar que la implementación de RAM debe garantizar que el tiempo de acceso a *cualquier* registro en la RAM sea casi instantáneo. Si este no fuera el caso, no podríamos obtener instrucciones y manipular variables en un tiempo razonable, lo que haría que las computadoras fueran poco prácticas. La magia del tiempo de acceso instantáneo se desplegará en breve, en la sección Implementación.

### 3.3.4 Mostrador

El contador es un chip que sabe cómo incrementar su valor en 1 cada unidad de tiempo. Cuando construyamos nuestra arquitectura de computadora en el capítulo 5, llamaremos a este chip Program Counter, o PC, por lo que ese es el nombre que usaremos aquí también. La interfaz de nuestro chip de PC es idéntica a la de un registro, excepto que también tiene bits de control etiquetados como *inc* y *reset*. Cuando  $\text{inc}=1$ , el contador incrementa su estado en cada ciclo de reloj, efectuando la operación  $\text{PC}++$ . Si queremos restablecer el contador a 0, afirmamos el bit de reinicio; si queremos establecer el contador en el valor  $v$ , ponemos  $v$  en la entrada de entrada y afirmamos el bit de carga, ya que normalmente lo hacen con registros. Los detalles se dan en [la figura 3.8](#).



**Figura 3.8** Contador de programa (PC): Para usarlo correctamente, como máximo uno de los carga, inco restablecimiento bits deben afirmarse.

**Uso:** Para leer el contenido actual de la PC, sondee el pin de salida. Para restablecer el

PC, afirme el bit de reinicio y establezca los otros bits de control en 0. Para tener la PC

incremente en 1 en cada unidad de tiempo hasta nuevo aviso, afirme el bit inc y establezca los otros bits de control en 0. Para establecer el PC en el valor  $v$ , establezca la entrada en  $v$ , afirme el bit de carga y establezca los otros bits de control en 0.

---

## 3.4 Implementación

La sección anterior presentó una familia de abstracciones de chips de memoria, centrándose en su interfaz y funcionalidad. Esta sección se centra en cómo se pueden realizar estos chips, utilizando chips más simples que ya se construyeron. Como de costumbre, nuestras pautas de implementación son intencionalmente sugerentes; queremos darle suficientes pistas para completar la implementación usted mismo, utilizando HDL y el simulador de hardware suministrado.

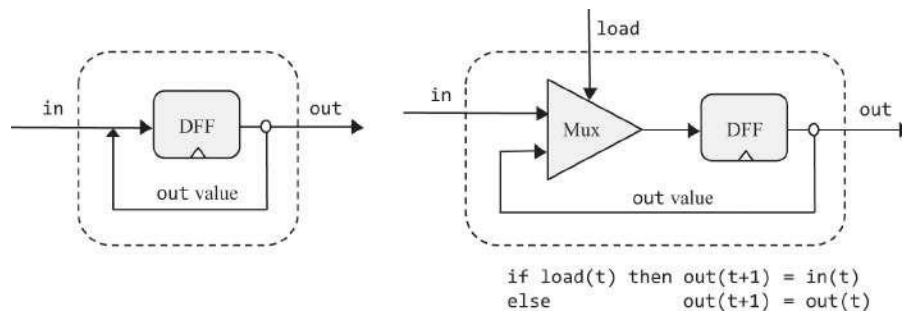
### 3.4.1 Chancía de fecha

Una puerta DFF está diseñada para poder "cambiar" entre dos estados estables, que representan 0 y 1. Esta funcionalidad se puede implementar de varias maneras diferentes, incluidas las que solo usan puertas Nand. Las implementaciones de DFF basadas en Nand son elegantes, pero intrincadas e imposibles de modelar en nuestro simulador de hardware, ya que requieren bucles de retroalimentación entre puertas combinadas. Deseando abstraer esta complejidad, trataremos el DFF como un bloque de construcción primitivo. En particular, el simulador de hardware de Nand a Tetris proporciona una implementación DFF incorporada que puede ser utilizada fácilmente por otros chips, como ahora describimos.

### 3.4.2 Registros

Los chips de registro son dispositivos de memoria: se espera que implementen el comportamiento básico  $out(t+1) = out(t)$ , recordando y emitiendo su estado a lo largo del tiempo. Esto se parece al comportamiento de DFF, que está fuera  $in(t+1) = in(t)$ . Si solo pudiéramos devolver la salida DFF a su entrada, este podría ser un buen punto de partida para implementar el registro de bits de un bit. Esta solución se muestra a la izquierda de la [figura](#)

3.9.



**Figura 3.9** El Bit (registro de 1 bit): soluciones no válidas (izquierda) y correctas (derecha).

Resulta que la implementación que se muestra a la izquierda de la [figura 3.9](#) no es válida, por varias razones relacionadas. En primer lugar, la implementación no expone un bit de carga, como lo requiere la interfaz del registro. En segundo lugar, no hay forma de decirle a la parte del chip DFF cuándo extraer su entrada del cable de entrada y cuándo del valor de salida entrante. De hecho, las reglas de programación HDL prohíben alimentar un pin desde más de una fuente.

Lo bueno de este diseño inválido es que nos lleva a la implementación correcta, que se muestra a la derecha de la [figura 3.9](#). Como muestra el diagrama de chips, una forma natural de resolver la ambigüedad de entrada es introducir un multiplexor en el diseño. El *bit de carga* del chip de registro general se puede canalizar al *bit de selección* del multiplexor interno: Si establecemos este bit en 1, el multiplexor alimentará el valor de entrada en el DFF; si establecemos el bit de carga en 0, el multiplexor alimentará la salida anterior del DFF. Esto producirá el comportamiento "si carga, establezca el registro en un nuevo valor, de lo contrario configúrelo en el valor almacenado anteriormente", exactamente cómo queremos que se comporte un registro.

Tenga en cuenta que el bucle de retroalimentación que acabamos de describir no implica problemas cíclicos *de carrera de datos*: el bucle pasa por una puerta DFF, lo que introduce un retraso de tiempo. De hecho, el diseño de bits que se muestra en [la figura 3.9](#) es un caso especial del diseño lógico secuencial general que se muestra en la [figura 3.4](#).

Una vez que hayamos completado la implementación del registro de bits de un solo bit, podemos pasar a construir un registro *de bits*  $w$ . Esto se puede lograr formando una matriz de  $w$  chips de bits (ver [figura 3.1](#)). El parámetro de diseño básico de dicho registro es  $w$ , el número de bits que se supone que contiene, por ejemplo, 16, 32 o 64. Dado que la computadora Hack se basará en una plataforma de hardware de 16 bits, nuestro chip Register se basará en partes de chip de dieciséis bits.

El registro de bits es el único chip en la arquitectura Hack que usa una puerta DFF directamente; todos los dispositivos de memoria de nivel superior en la computadora usan chips DFF indirectamente, en virtud del uso de chips de registro hechos de chips de bits. Tenga en cuenta que la inclusión de una puerta DFF en el diseño de cualquier chip, directa o indirectamente —convierte este último chip, así como todos los chips de nivel superior que lo usan como parte del chip, en chips dependientes del tiempo.

### 3.4.3 CARNERO

La plataforma de hardware Hack requiere un dispositivo RAM de 16K (16384) registros de 16 bits, así que eso es lo que tenemos que implementar. Proponemos la siguiente hoja de ruta de implementación gradual:

| chip   | $n$   | $k$ | built from:      |
|--------|-------|-----|------------------|
| RAM8   | 8     | 3   | 8 Register chips |
| RAM64  | 64    | 6   | 8 RAM8 chips     |
| RAM512 | 512   | 9   | 8 RAM64 chips    |
| RAM4K  | 4096  | 12  | 8 RAM512 chips   |
| RAM16K | 16384 | 14  | 4 RAM4K chips    |

Todos estos chips de memoria tienen exactamente la misma API de RAM $n$  que se muestra en [la figura](#)

**3.7.** Cada chip RAM tiene  $n$  registros, y el ancho de su entrada de dirección es bits  $k = \log_2 n$ . Ahora describimos cómo se pueden implementar estos chips, comenzando con RAM8.

Un chip RAM8 cuenta con 8 registros, como se muestra en [la figura 3.7](#), para  $n = 8$ . Cada registro se puede seleccionar configurando la entrada de dirección de 3 bits de la RAM8 en un valor entre 0 y 7. El acto de *leer* el valor de un registro seleccionado se puede describir de la siguiente manera: Dada alguna dirección (un valor entre 0 y 7), ¿cómo podemos "seleccionar" la dirección del número de registro y canalizar su salida a la salida de RAM8? *Sugerencia:* Podemos hacerlo usando uno de los chips combinados incorporados en el proyecto 1. Es por eso que la lectura del valor de un registro de RAM seleccionado se logra casi instantáneamente, independientemente del reloj y del número de registros en la RAM. De manera similar, el acto de *escribir* un valor en un registro seleccionado se puede describir de la siguiente manera: Dado un valor de dirección, un valor de carga (1) y un valor de 16 bits, ¿cómo podemos establecer el valor de la dirección del número de registro en  $in$ ? *Sugerencia:* Los datos de 16 bits se pueden alimentar simultáneamente a

las entradas de los ocho chips de registro. Usando otro chip combinacional desarrollado en el proyecto 1, junto con las entradas de dirección y carga, puede asegurarse de que solo uno de los registros aceptará el valor entrante, mientras que los otros siete registros lo ignorarán.

Observamos de paso que los registros de RAM no están marcados con direcciones en ningún sentido físico. Más bien, la lógica descrita anteriormente es capaz y suficiente para seleccionar registros individuales de acuerdo con su dirección, y esto se hace en virtud del uso de chips combinados. Ahora bien, aquí hay una observación de importancia crucial: dado que la lógica combinacional es independiente del tiempo, el tiempo de acceso a cualquier registro individual será casi instantáneo.

Una vez que hayamos implementado el chip RAM8, podemos pasar a implementar un chip RAM64. La implementación se puede basar en ocho partes de chip RAM8. Para seleccionar un registro en particular de la memoria RAM64, usamos una dirección de 6 bits, digamos xxxyyy. Los bits xxx se pueden usar para seleccionar uno de los chips RAM8, y los bits yyy se pueden usar para seleccionar uno de los registros dentro del RAM8 seleccionado. Este esquema de direccionamiento jerárquico se puede efectuar mediante la lógica de puerta. La misma idea de implementación puede guiar la implementación de los chips RAM512, RAM4K y RAM16K restantes.

Para recapitular, tomamos un agregado de un número ilimitado de registros y le imponemos una superestructura combinatoria que permite el acceso directo a cualquier registro individual. Esperamos que la belleza de esta solución no escape a la atención del lector.

#### **3.4.4 Mostrador**

Un contador es un dispositivo de memoria que puede incrementar su valor en cada unidad de tiempo. Además, el contador se puede establecer en 0 o en algún otro valor. Las funcionalidades básicas de almacenamiento y conteo del contador se pueden implementar, respectivamente, mediante un chip de registro y mediante el chip incrementador integrado en el proyecto 2. La lógica que selecciona entre los modos inc, load y reset del contador se puede implementar utilizando algunos de los multiplexores integrados en el proyecto 1.

---

### **3.5 Proyecto**

**Objetivo:** Construir todos los chips descritos en el capítulo. Los bloques de construcción que puedes usar son puertas DFF primitivas , chips que construirás encima de ellas y las puertas construidas en capítulos anteriores.

**Recursos:** La única herramienta que necesita para este proyecto es el simulador de hardware de Nand a Tetrís. Todos los chips deben implementarse en el lenguaje HDL especificado en el apéndice 2. Como de costumbre, para cada chip suministramos un programa .hdl esquelético con una parte de implementación faltante, un archivo de script .tst que le dice al simulador de hardware cómo probarlo y un archivo de comparación .cmp que define los resultados esperados. Su trabajo consiste en completar las partes de implementación que faltan de los programas .hdl suministrados.

**Contrato:** Cuando se carga en el simulador de hardware, el diseño del chip ( programa .hdl modificado), probado en el archivo .tst suministrado, debe producir los resultados enumerados en el archivo .cmp suministrado. Si ese no es el caso, el simulador te lo hará saber.

**Consejo:** La puerta de cambio de datos (DFF) se considera primitiva; por lo tanto, no hay necesidad de construirla. Cuando el simulador encuentra una pieza de chip DFF en un programa HDL, invoca automáticamente la implementación tools/builtIn/DFF.hdl.

**Estructura de carpetas de este proyecto:** Al construir chips de RAM a partir de piezas de chips de RAM de nivel inferior, recomendamos utilizar versiones integradas de estas últimas. De lo contrario, el simulador generará recursivamente numerosos objetos de software residentes en la memoria, uno para cada una de las muchas partes del chip que componen una unidad RAM típica. Esto puede hacer que el simulador se ejecute lentamente o, peor aún, que se quede sin memoria del equipo host en el que se ejecuta el simulador.

Para evitar este problema, hemos dividido los chips de RAM integrados en este proyecto en dos subcarpetas. Los programas RAM8.hdl y RAM64.hdl se almacenan en projects/03/a, y los otros chips RAM de nivel superior se almacenan en projects/03/b. Esta partición se realiza con un solo propósito: al evaluar los chips de RAM almacenados en la carpeta b, el simulador se verá obligado a usar implementaciones integradas de las partes del chip RAM64, porque RAM64.hdl no se puede encontrar en la carpeta b.

**Pasos:** Recomendamos proceder en el siguiente orden:

1. El hardware simulador necesario para éste proyecto es disponible en `nand2tetris/herramientas`.
2. Consulte el apéndice 2 y el Tutorial del simulador de hardware, según sea necesario.
3. Construya y simule todos los chips especificados en la carpeta `projects/03`.

**Una versión basada en la web del proyecto 3** está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

---

## 3.6 Perspectiva

La piedra angular de todos los sistemas de memoria descritos en este capítulo es el flip-flop, que tratamos de manera abstracta como una puerta primitiva e incorporada. El enfoque habitual es construir flip-flops a partir de puertas combinacionales elementales (por ejemplo, puertas Nand) conectadas en bucles de retroalimentación. La construcción estándar comienza construyendo un flip-flop no sincronizado que es biestable, es decir, se puede configurar para que esté en uno de dos estados (almacenamiento 0 y almacenamiento 1). Luego, se obtiene un flip-flop sincronizado mediante la conexión en cascada de dos de estos flip-flops no sincronizados, el primero se establece cuando el reloj *marca* y el segundo cuando el reloj *toca*. Este diseño maestro-esclavo dota al flip-flop general de la funcionalidad de sincronización sincronizada deseada.

Tales implementaciones de flip-flop son elegantes e intrincadas. En este libro hemos optado por abstraer estas consideraciones de bajo nivel tratando el flip-flop como una puerta primitiva. Los lectores que deseen explorar la estructura interna de las puertas flip-flop pueden encontrar descripciones detalladas en la mayoría de los libros de texto de diseño lógico y arquitectura de computadoras.

Una razón para no detenerse en la esotérica flip-flop es que el nivel más bajo de los dispositivos de memoria utilizados en las computadoras modernas no se construye necesariamente a partir de puertas flip-flop. En cambio, los chips de memoria modernos se optimizan cuidadosamente, explotando las propiedades físicas únicas de la tecnología de almacenamiento subyacente. Muchas de estas tecnologías alternativas están disponibles hoy en día para los diseñadores de computadoras; Como de costumbre, qué tecnología utilizar es una cuestión de costo-rendimiento.



Del mismo modo, el método de ascenso recursivo que utilizamos para construir los chips de RAM es elegante pero no necesariamente eficiente. Son posibles implementaciones más eficientes.

Aparte de estas consideraciones físicas, todas las construcciones de chips descritas en este capítulo (los registros, el contador y los chips RAM) son estándar, y se pueden encontrar versiones de ellos en todos los sistemas informáticos.

En el capítulo 5, usaremos los chips de registro contruidos en este capítulo, junto con el ALU construido en el capítulo 2, para construir una Unidad Central de Procesamiento. Luego, la CPU se aumentará con un dispositivo RAM, lo que conducirá a una arquitectura de computadora de propósito general capaz de ejecutar programas escritos en un lenguaje de máquina. Este lenguaje de máquina se analiza en el siguiente capítulo.

---

## 4 Lenguaje de máquina

Las obras de imaginación deben escribirse en un lenguaje muy sencillo; cuanto más puramente imaginativos son, más necesario es ser sencillo.

—Samuel Taylor Coleridge (1772–1834)

En los capítulos 1 a 3 construimos chips de procesamiento y memoria que se pueden integrar en la plataforma de hardware de una computadora de propósito general. Antes de comenzar a completar esta construcción, hagamos una pausa y preguntemos: ¿Cuál es exactamente el *propósito* de esta computadora? Como observó el arquitecto Louis Sullivan, "La forma sigue a la función". Si desea comprender un sistema o construir uno, comience por estudiar la función que se supone que debe cumplir el sistema. Con eso en mente, antes de comenzar a completar la construcción de nuestra plataforma de hardware, dedicaremos este capítulo a estudiar el *lenguaje de máquina* que esta plataforma está diseñada para realizar. Después de todo, ejecutar programas escritos en lenguaje de máquina de manera eficiente es la función final de cualquier computadora de propósito general.

Un lenguaje de máquina es un formalismo acordado diseñado para codificar instrucciones de máquina. Usando estas instrucciones, podemos instruir al procesador de la computadora para que realice operaciones aritméticas y lógicas, lea y escriba valores desde y hacia la memoria de la computadora, pruebe las condiciones booleanas y decida qué instrucción buscar y ejecutar a continuación. A diferencia de los lenguajes de alto nivel, cuyos objetivos de diseño son la compatibilidad multiplataforma y el poder de expresión, los lenguajes de máquina están diseñados para efectuar la ejecución directa y el control total de una plataforma de hardware específica. Por supuesto, la generalidad, la elegancia y el poder de expresión todavía son deseados, pero solo en la medida en que respalden el requisito básico de ejecución directa y eficiente en hardware.

El lenguaje de máquina es la interfaz más profunda en la empresa informática, la delgada línea donde el hardware se encuentra con el software. Este es el punto en el que los diseños abstractos de los humanos, tal como se manifiestan en programas de alto nivel, finalmente se reducen a operaciones físicas realizadas en silicio. Por lo tanto, un lenguaje de máquina puede interpretarse como un artefacto de programación y una parte integral de la plataforma de hardware. De hecho, así como decimos que el lenguaje de máquina está diseñado para controlar una plataforma de hardware en particular, podemos decir que la plataforma de hardware está diseñada para ejecutar instrucciones escritas en un lenguaje de máquina en particular.

El capítulo comienza con una introducción general a la programación de bajo nivel en lenguaje de máquina. A continuación, damos una especificación detallada del *lenguaje de máquina Hack*, que cubre tanto su versión binaria como simbólica. El proyecto que cierra el capítulo se centra en escribir algunos programas de lenguaje de máquina. Esto proporciona una apreciación práctica de la programación de bajo nivel y prepara el escenario para completar la construcción del hardware de la computadora en el próximo capítulo.

Aunque los programadores rara vez escriben programas directamente en lenguaje de máquina, el estudio de la programación de bajo nivel es un requisito previo para una comprensión completa y rigurosa de cómo funcionan las computadoras. Además, una comprensión íntima de la programación de bajo nivel ayuda al programador a escribir programas de alto nivel mejores y más eficientes. Finalmente, es bastante fascinante observar, prácticamente, cómo los sistemas de software más sofisticados son, en el fondo, flujos de instrucciones simples, cada una de las cuales especifica una operación primitiva bit a bit en el hardware subyacente.

---

## 4.1 Lenguaje de máquina: descripción general

Este capítulo no se centra en la máquina, sino en el *lenguaje* utilizado para controlar la máquina. Por lo tanto, abstraeremos la plataforma de hardware, centrándonos en el subconjunto mínimo de elementos de hardware que se mencionan explícitamente en las instrucciones del lenguaje de máquina.

### 4.1.1 Elementos de hardware

Un lenguaje de máquina puede verse como un formalismo acordado diseñado para manipular una *memoria* utilizando un *procesador* y un conjunto de *registros*.

**Memoria:** El término *memoria* se refiere vagamente a la colección de dispositivos de hardware que almacenan datos e instrucciones en una computadora. Funcionalmente hablando, una memoria es una secuencia continua de celdas, también conocidas como *ubicaciones* o *registros de memoria*, cada una con una *dirección única*. Se accede a un registro de memoria individual proporcionando su dirección.

**Procesador:** El procesador, normalmente llamado *Unidad Central de Procesamiento*, o *CPU*, es un dispositivo capaz de realizar un conjunto fijo de operaciones primitivas. Estos incluyen operaciones aritméticas y lógicas, operaciones de acceso a memoria y operaciones de control (también llamadas *bifurcación*). El procesador extrae sus entradas de registros y ubicaciones de memoria seleccionados y escribe sus salidas en registros y ubicaciones de memoria seleccionados. Consiste en una ALU, un conjunto de registros y una lógica de puerta que le permite analizar y ejecutar instrucciones binarias.

**Registros:** El procesador y la memoria se implementan como dos chips independientes e independientes, y mover datos de uno a otro es un asunto relativamente lento. Por esta razón, los procesadores normalmente están equipados con varios registros integrados, cada uno capaz de contener un solo valor. Ubicados dentro del chip del procesador, los registros sirven como una memoria local de alta velocidad, lo que permite al procesador manipular datos e instrucciones sin tener que aventurarse fuera del chip.

Los registros residentes en la CPU se dividen en dos categorías: registros de *datos*, diseñados para contener valores de datos, y *registros de direcciones*, diseñados para contener valores que pueden interpretarse como valores de datos o como direcciones de memoria. La arquitectura de la computadora está configurada de tal manera que colocar un valor particular, digamos  $n$ , en un registro de direcciones, hace que la ubicación de la memoria cuya dirección es  $n$  se *seleccione* instantáneamente.<sup>1</sup> Esto prepara el escenario para una operación posterior en la ubicación de memoria seleccionada.

#### 4.1.2 Idiomas

Los programas de lenguaje de máquina se pueden escribir de dos maneras alternativas, pero equivalentes: *binarias* y *simbólicas*. Por ejemplo, considere la operación abstracta "establecer R1 en el valor de  $R1+R2$ ". Como diseñadores de lenguajes, podemos decidir representar la operación de suma usando el código de 6 bits 101011 y los registros R1 y R2 usando los códigos 00001 y 00010, respectivamente. Ensamblando estos códigos de izquierda a derecha, podemos decidir usar la instrucción de 16 bits 1010110001000001 como la versión binaria de "establecer R1 en el valor de  $R1+R2$ ".

En los primeros días de los sistemas informáticos, las computadoras se programaban manualmente: cuando los protoprogramadores querían emitir la instrucción "establecer R1 en el valor de  $R1+R2$ ", empujaban hacia arriba y hacia abajo interruptores mecánicos que almacenaban un código binario como 1010110001000001 en la memoria de instrucciones de la computadora. Y si el programa tenía cien instrucciones, tenían que pasar por esta prueba cien veces. Por supuesto, depurar tales programas era una pesadilla perfecta. Esto llevó a los programadores a inventar y usar códigos simbólicos como una forma conveniente de documentar y depurar programas *en papel*, antes de ingresarlos en la computadora. Por ejemplo, se podría elegir el formato simbólico `add R2,R1` para representar la semántica "establecer R1 en el valor de  $R1+R2$ " y la instrucción binaria 1010110001000001.

No pasó mucho tiempo antes de que a varias personas se les ocurriera la misma idea: Símbolos

como R, 1, 2 y + también se pueden representar mediante códigos binarios acordados. ¿Por qué no usar instrucciones simbólicas para escribir programas y luego usar otro programa, un *traductor*, para traducir las instrucciones simbólicas en código binario ejecutable? Esta innovación liberó a los programadores del tedio de escribir código binario, allanando el camino para la posterior avalancha de lenguajes de programación de alto nivel. Por razones que se aclararán en el capítulo 6, los lenguajes de máquina simbólicos se denominan *lenguajes ensambladores*, y los programas que los traducen a código binario se denominan *ensambladores*.

A diferencia de la sintaxis de los lenguajes de alto nivel, que es portátil e independiente del hardware, la sintaxis de un lenguaje ensamblador está estrechamente relacionada con los detalles de bajo nivel del hardware de destino: las operaciones ALU disponibles, el número y tipo de registros, el tamaño de la memoria, etc. Dado que las diferentes computadoras varían mucho en términos de cualquiera de estos parámetros, existe una Torre de Babel de lenguajes de máquina, cada uno con su sintaxis oscura, cada uno diseñado para controlar una familia particular de CPU. Independientemente

de

Sin embargo, todos los lenguajes de máquina son teóricamente equivalentes y todos admiten conjuntos similares de tareas genéricas, como ahora vamos a describir.

### 4.1.3 Instrucciones

En lo que sigue, asumimos que el procesador de la computadora está equipado con un conjunto de registros denominados R0, R1, R2, ... El número exacto y el tipo de estos registros son irrelevantes para nuestra discusión actual.

**Operaciones aritméticas y lógicas:** Cada lenguaje de máquina presenta instrucciones para realizar operaciones aritméticas básicas como suma y resta, así como operaciones lógicas básicas como Y, O, No. Por ejemplo, considere los siguientes segmentos de código:

```
// Adds up two numbers:
load R1,17      // R1 ← 17
load R2,4       // R2 ← 4
add  R1,R1,R2   // R1 ← R1 + R2

// Computes a logical operation:
load R1,true    // R1 ← binary representation of true
load R2,false   // R2 ← binary representation of false
and  R1,R1,R2   // R1 ← R1 And R2 (bit-wise And)
```

Para que tales instrucciones simbólicas se ejecuten en una computadora, primero deben traducirse a código binario. La traducción se realiza mediante un programa llamado *ensamblador*, que desarrollaremos en el capítulo 6. Por ahora, asumimos que tenemos acceso a dicho ensamblador y que podemos usarlo según sea necesario.

**Acceso a la memoria:** todos los medios de lenguaje de máquina cuentan con medios para acceder y luego manipular ubicaciones de memoria seleccionadas. Esto generalmente se hace usando un *registro* de direcciones, llamémoslo A. Por ejemplo, supongamos que deseamos establecer la ubicación de memoria 17 en el valor 1. Podemos decidir hacerlo usando las dos instrucciones `load A,17` seguidas de `load M,1`, donde, por convención, M representa el registro de memoria seleccionado por A (es decir, el registro de memoria cuya dirección es el valor actual de A). Con eso en mente, supongamos que deseamos establecer las cincuenta



ubicaciones de memoria 200, 201, 202, ..., 249 en 1. Esto se puede hacer de la siguiente manera:

ejecutando la instrucción `load A,200` y luego ingresando a un bucle que ejecuta las instrucciones `load M,1` y suma `A,A,1` cincuenta veces.

**Control de flujo:** Si bien los programas de computadora se ejecutan de forma predeterminada de forma secuencial, una instrucción tras otra, también incluyen *saltos ocasionales* a ubicaciones distintas a la siguiente instrucción. Para facilitar estas acciones de bifurcación, los lenguajes de máquina presentan varias variantes de instrucciones goto condicionales e incondicionales, así como declaraciones de declaración de etiquetas que marcan los destinos goto. La Figura 4.1 ilustra una acción de ramificación simple utilizando lenguaje de máquina.

| Using physical addresses                                                                  | Using symbolic addresses                                                                             |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <pre>... // Sets R1 to 0+1+2, ... 12: load R1,0 13: add R1,R1,1 ... 27: goto 13 ...</pre> | <pre>... // Sets R1 to 0+1+2, ...     load R1,0 (LABEL)     add R1,R1,1 ...     goto LABEL ...</pre> |

**Figura 4.1** Dos versiones del mismo código de bajo nivel (se supone que el código incluye alguna lógica de terminación de bucle, que no se muestra aquí).

**Símbolos:** Ambas versiones de código en la figura 4.1 están escritas en lenguaje ensamblador; por lo tanto, ambas deben traducirse a código binario antes de que puedan ejecutarse. Además, ambas versiones realizan exactamente la misma lógica. Sin embargo, la versión de código que usa referencias simbólicas es mucho más fácil de escribir, depurar y mantener.

Además, a diferencia del código que usa direcciones físicas, la versión binaria traducida del código que usa referencias simbólicas se puede cargar y ejecutar desde cualquier segmento de memoria que esté disponible en la memoria de la computadora. Por lo tanto, se dice que el código de bajo nivel que no menciona direcciones físicas es *reubicable*. Claramente, el código reubicable es esencial en sistemas informáticos como PC y teléfonos celulares, que rutinariamente cargan y ejecutan múltiples aplicaciones de forma dinámica y simultánea. Por lo tanto, vemos que

Las referencias simbólicas no son solo una cuestión de cosméticos, se utilizan para liberar el código de archivos físicos adjuntos innecesarios a la memoria del host.

Esto termina nuestra breve introducción a algunos elementos esenciales del lenguaje de máquina. La siguiente sección ofrece una descripción formal de un lenguaje de máquina específico: el código nativo de la computadora Hack.

---

## 4.2 El lenguaje de la máquina de hackeo

Los programadores que escriben código de bajo nivel (o programadores que escriben compiladores e intérpretes que generan código de bajo nivel) interactúan con la computadora de manera abstracta, a través de su *interfaz*, que es el lenguaje de máquina de la computadora. Aunque los programadores no tienen que conocer todos los detalles de la arquitectura informática subyacente, deben estar familiarizados con los elementos de hardware que entran en juego en sus programas de bajo nivel.

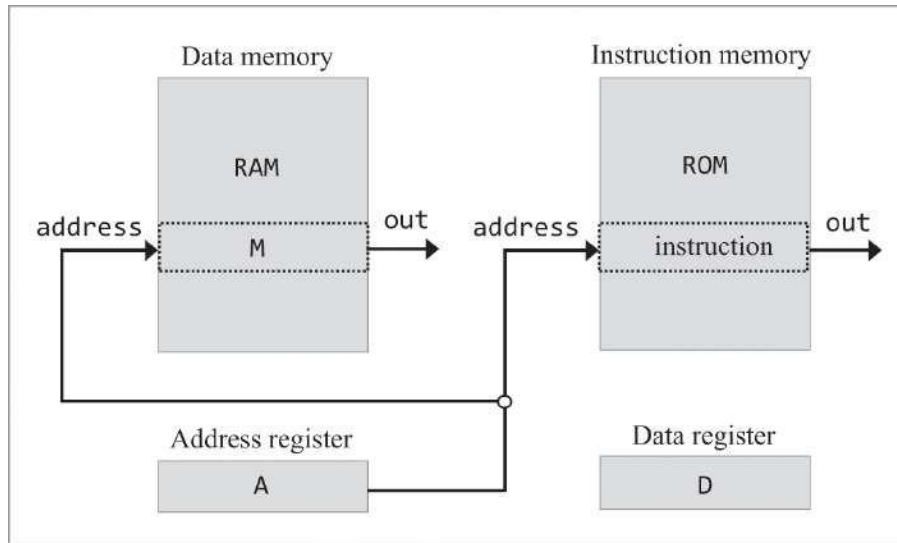
Con eso en mente, comenzamos la discusión del lenguaje de máquina Hack con una descripción conceptual de la computadora Hack. A continuación, damos un ejemplo de un programa completo escrito en el lenguaje ensamblador Hack. Esto prepara el escenario para el resto de esta sección, en la que damos una especificación formal de las instrucciones del lenguaje Hack.

### 4.2.1 Fondo

El diseño de la computadora Hack, que se presentará en el próximo capítulo, sigue un paradigma de hardware ampliamente utilizado conocido como la *arquitectura von Neumann*, que lleva el nombre del pionero de la computación John von Neumann. Hack es una computadora de 16 bits, lo que significa que la CPU y las unidades de memoria están diseñadas para procesar, mover y almacenar fragmentos de valores de 16 bits.

**Memoria:** Como se ve en [la figura 4.2](#), la plataforma Hack utiliza dos unidades de memoria distintas: una memoria de *datos* y una memoria de *instrucciones*. La memoria de datos almacena los valores binarios que manipulan los programas. La memoria de instrucciones almacena las instrucciones del programa, también representadas como valores binarios. Ambas memorias tienen un ancho de 16 bits y cada una tiene un espacio de direcciones de 15 bits. Por lo tanto, el

el tamaño máximo direccionable de cada unidad de memoria es de 215 o 32K palabras de 16 bits (el símbolo *K*, abreviado de *kilo*, la palabra griega para *mil*, se usa comúnmente para representar el número  $2^{10} = 1024$ ). Es conveniente pensar en cada unidad de memoria como una secuencia lineal de registros de memoria direccionables, con direcciones que van de 0 a 32K-1.



**Figura 4.2** Modelo conceptual del sistema de memoria Hack. Aunque la arquitectura real está cableada de manera algo diferente (como se describe en el capítulo 5), este modelo ayuda a comprender la semántica de los programas Hack.

La *memoria de datos* (que también llamamos RAM) es un dispositivo de lectura/escritura. Las instrucciones de hackeo pueden leer y escribir datos en registros de RAM seleccionados. Se selecciona un registro individual proporcionando su dirección. Dado que la entrada de direcciones de la memoria siempre contiene algún valor, siempre hay un registro seleccionado, y este registro se denomina en las instrucciones de Hack como *M*. Por ejemplo, la instrucción Hack  $M=0$  establece el registro de RAM seleccionado en 0.

La *memoria de instrucciones* (que también llamamos ROM) es un dispositivo de solo lectura, y los programas se cargan en ella utilizando algunos medios exógenos (más sobre esto en el capítulo 5). Al igual que con la RAM, la entrada de direcciones de la memoria de instrucciones siempre contiene algún valor; por lo tanto, siempre hay un registro de memoria de instrucciones seleccionado. El valor de este registro se conoce como la *instrucción actual*.

**Registros:** Las instrucciones de hackeo están diseñadas para manipular tres registros de 16 bits: un registro de *datos*, denotado D, un *registro de direcciones*, denotado A, y un registro de memoria de datos seleccionado, denotado M. Las instrucciones de hackeo tienen una sintaxis que se explica por sí misma. Por ejemplo:  $D=M$ ,  $M=D+1$ ,  $D=0$ ,  $D=M-1$ , y así sucesivamente.

El papel del registro de datos D es sencillo: sirve para almacenar un valor de 16 bits. El segundo registro, A, sirve como registro de direcciones y registro de datos. Si queremos almacenar el valor 17 en el registro A, usamos la instrucción Hack @17 (la razón de esta sintaxis se aclarará pronto). De hecho, esta es la única forma de obtener una constante en la computadora Hack. Por ejemplo, si deseamos establecer el registro D en 17, usamos las dos instrucciones @17, seguidas de  $D=A$ . Además de servir como un segundo registro de datos, el registro A también se usa para abordar la memoria de datos y la memoria de instrucciones, como ahora vamos a discutir.

**Direccionamiento:** La instrucción Hack @ xxx establece el registro A en el valor xxx. Además, la instrucción @ xxx tiene dos efectos secundarios. Primero, hace que el registro RAM cuya dirección es xxx sea el registro de memoria seleccionado, denotado

M. En segundo lugar, hace que el valor del registro ROM cuya dirección es xxx sea la instrucción seleccionada. Por lo tanto, establecer A en algún valor tiene el efecto simultáneo de preparar la etapa, potencialmente, para una de dos acciones posteriores muy diferentes: manipular el registro de memoria de datos seleccionado o hacer algo con la instrucción seleccionada. Qué acción perseguir (y cuál ignorar) está determinada por la instrucción de Hackeo posterior.

Para ilustrar, supongamos que deseamos establecer el valor de RAM[100] en 17. Esto se puede hacer usando las instrucciones de Hack @17,  $D=A$ , @100,  $M=D$ . Tenga en cuenta que en el primer par de instrucciones, A sirve como registro de datos; en el segundo par de instrucciones, sirve como registro de direcciones. Aquí hay otro ejemplo: Para establecer RAM[100] al valor de RAM[200], podemos usar las instrucciones de Hack @200,  $D=M$ , @100,  $M=D$ .

En ambos escenarios, el registro A también seleccionó registros en la memoria de instrucciones, una acción que los dos escenarios ignoraron. En la siguiente sección se analiza el escenario opuesto: usar A para seleccionar instrucciones mientras se ignora su efecto en la memoria de datos.

**Ramificación:** Los ejemplos de código hasta ahora implican que un programa Hack es una secuencia de instrucciones, que se ejecutan una tras otra. Esto es realmente

el flujo de control predeterminado, pero ¿qué sucede si deseamos ramificar para ejecutar no la siguiente instrucción sino, digamos, la instrucción número 29 en el programa? En el lenguaje Hack, esto se puede hacer usando la instrucción Hack @29, seguida de la instrucción Hack 0; JMP. La primera instrucción selecciona el registro ROM[29] (también selecciona RAM[29], pero no nos importa). El 0 subsiguiente; La instrucción JMP realiza la versión Hack de *la bifurcación incondicional*: go para ejecutar la instrucción direccionada por el registro A (explicaremos el ; 0 más tarde). Dado que se supone que la ROM contiene el programa que estamos ejecutando actualmente, a partir de la dirección 0, las dos instrucciones @29 y 0; JMP termina haciendo que el valor de ROM[29] sea la siguiente instrucción a ejecutar.

El lenguaje Hack también presenta *bifurcación condicional*. Por ejemplo, la lógica si D==0 goto 52 se puede implementar usando la instrucción @52, seguida de la instrucción D;JEQ. La semántica de la segunda instrucción es "evaluar D; si el valor es igual a cero, salte para ejecutar la instrucción almacenada en la dirección seleccionada por A". El lenguaje Hack presenta varios de estos comandos *de ramificación condicional*, como explicaremos más adelante en el capítulo.

Para recapitular: el registro A cumple dos funciones de direccionamiento simultáneas, pero muy diferentes. Después de una instrucción @xxx, nos enfocamos en el registro de memoria de datos seleccionado (M) e ignoramos la instrucción seleccionada, o nos enfocamos en la instrucción seleccionada e ignoramos el registro de memoria de datos seleccionado. Esta dualidad es un poco confusa, pero tenga en cuenta que nos salimos con la nuestra usando un registro de direcciones para controlar dos dispositivos de memoria separados (ver [figura 4.2](#)). El resultado es una arquitectura de computadora más simple y un lenguaje de máquina compacto. Como es habitual en nuestro negocio, la simplicidad y el ahorro reinan supremos.

**Variables:** El xxx en la instrucción Hack @xxx puede ser una constante o un símbolo. Si la instrucción es @23, el registro A se establece en el valor 23. Si la instrucción es @x, donde x es un símbolo que está vinculado a algún valor, digamos 513, U regístrese en

El uso de símbolos dota a los programas de ensamblaje Hack de la capacidad de usar *variables* en lugar de direcciones de memoria física. Por ejemplo, la típica declaración de asignación de alto nivel let x=17 se puede implementar en el lenguaje Hack como @17, D=A, @x,

M=D. La semántica de este código es "seleccionar el registro RAM cuya dirección es el valor que está vinculado al símbolo x, y establecer este

registro en 17". Aquí

Supongamos que hay un agente que sabe cómo vincular los símbolos que se encuentran en lenguajes de alto nivel, como *x*, a direcciones sensibles y consistentes en la memoria de datos. Este agente es el ensamblador.

Gracias al ensamblador, variables como *x* se pueden nombrar y usar en programas Hack a voluntad y según sea necesario. Por ejemplo, supongamos que deseamos escribir código que incremente algún contador. Una opción es mantener este contador en, digamos, *RAM[30]*, e incrementarlo usando las instrucciones *@30, M=M+1*. Un enfoque más sensato es usar *@count M=M+1*, y dejar que el ensamblador se preocupe por dónde colocar esta variable en la memoria. No nos importa la dirección específica siempre que el ensamblador siempre resuelva el símbolo en esa dirección. En el capítulo 6 aprenderemos cómo desarrollar un ensamblador que implemente esta útil operación de mapeo.

Además de los símbolos que se pueden introducir en los programas de ensamblaje de Hack según sea necesario, el lenguaje Hack presenta dieciséis símbolos incorporados llamados *R0*, *R1*, *R2*, ..., *R15*. Estos símbolos siempre están vinculados por el ensamblador a los valores 0, 1, 2, ..., 15. Así, por ejemplo, las dos instrucciones de Hack *@R3, M=0* terminarán poniendo *RAM[3]* en 0. En lo que sigue, a veces nos referimos a *R0*, *R1*, *R2*, ..., *R15* como *registros virtuales*.

Antes de continuar, le sugerimos que revise y se asegure de comprender completamente los ejemplos de código que se muestran en la [figura 4.3](#) (algunos de los cuales ya se discutieron).

#### Memory access examples

```
// D = 17
@17
D=A

// RAM[100] = 17
@17
D=A
@100
M=D

// RAM[100] = RAM[200]
@200
D=M
@100
M=D
```

#### Branching examples

```
// goto 29
@29
0;JMP

// if D>0 goto 63
@63
D;JGT
```

#### Variables use examples

```
// x = -1
@x
M=-1

// count = count - 1
@count
M=M-1

// sum = sum + x
@sum
D=M
@x
D=D+M
@sum
M=D
```

**Figura 4.3** Ejemplos de código ensamblador de hackeo.



### 4.2.2 Ejemplo de programa

Saltando al agua fría, revisemos un programa completo de ensamblaje de Hack, aplazando una descripción formal del lenguaje Hack para la siguiente sección. Antes de hacerlo, una advertencia: la mayoría de los lectores probablemente estarán desconcertados por el estilo oscuro de este programa. A lo que decimos: Bienvenido a la programación en lenguaje de máquina. A diferencia de los lenguajes de alto nivel, los lenguajes de máquina no están diseñados para complacer a los programadores. Más bien, están diseñados para controlar una plataforma de hardware, de manera eficiente y sencilla.

Supongamos que deseamos calcular la suma para  $1 + 2 + 3 + \dots + n$ , un valor dado  $n$ . Para poner las cosas en funcionamiento, pondremos la entrada  $n$  en `RAM[0]` y la suma de salida en `RAM[1]`. El programa que calcula esta suma se enumera en la [figura 4.4](#). Comenzando con el pseudocódigo, tenga en cuenta que en lugar de utilizar la fórmula bien conocida para calcular la suma de una serie aritmética, usamos la suma de fuerza bruta. Esto se hace para ilustrar el procesamiento condicional e iterativo en el lenguaje de máquina Hack.

#### Pseudocode

```
// Program: Sum1ToN
// Computes RAM[1]=1+2+3+...+RAM[0]
// Usage: put a value>=1 in RAM[0]
i = 1
sum = 0
LOOP:
  if (i > R0) goto STOP
  sum = sum + i
  i = i + 1
  goto LOOP
STOP:
  R1 = sum
```

#### Hack assembly code

```
// File: Sum1ToN.asm
// Computes RAM[1]=1+2+3+...+RAM[0]
// Usage: put a value>=1 in RAM[0]
// i = 1
@i
M=1
// sum = 0
@sum
M=0
(LLOOP)
// if (i > R0) goto STOP
@i
D=M
@R0
D=D-M
@STOP
D;JGT
// sum = sum + i
@sum
D=M
@i
D=D+M
@sum
M=D
// i = i + 1
@i
M=M+1
// goto LOOP
@LLOOP
0;JMP
(STOP)
// R1 = sum
@sum
D=M
@R1
M=D
(END)
@END
0;JMP
```

**Figura 4.4** Un programa de ensamblaje Hack (ejemplo). Tenga en cuenta que RAM[0] y RAM[1] puede denominarse R0 y R1.

Más adelante en el capítulo comprenderá este programa por completo. Por ahora, sugerimos ignorar los detalles y observar en su lugar el siguiente patrón: En el lenguaje Hack, cada operación que involucra una ubicación de memoria implica dos instrucciones. La primera instrucción, *@addr*, se usa para seleccionar una dirección de memoria de destino; la instrucción siguiente especifica qué hacer en esta dirección. Para apoyar esta lógica, el lenguaje Hack presenta dos instrucciones genéricas, varios ejemplos de los cuales ya hemos visto: una instrucción de *dirección*, también llamada *instrucción A* (las instrucciones que comienzan con @), y una *instrucción de cálculo*, también llamada *instrucción C* (todas las demás instrucciones).



En segundo lugar, prepara el escenario para una *instrucción C* posterior que manipula un registro RAM seleccionado, denominado M, configurando primero A en la dirección de ese registro. En tercer lugar, prepara el escenario para una *instrucción C* posterior que especifica un salto configurando primero A en la dirección del destino del salto.

## La *instrucción C*

La *instrucción C* responde a tres preguntas: qué calcular (una operación ALU, denotada *comp*), dónde almacenar el valor calculado (*dest*) y qué hacer a continuación (*salto*). Junto con la *instrucción A*, la *instrucción C* especifica todas las operaciones posibles de la computadora.

En la versión binaria, el bit más a la izquierda es el código de operación de la *instrucción C*, que es 1. Los dos bits siguientes no se usan y se establecen por convención en 1. Los siguientes siete bits son la representación binaria del *campo comp*. Los siguientes tres bits son la representación binaria del *campo dest*. Los tres bits más a la derecha son la representación binaria del *campo de salto*. Ahora describimos la sintaxis y la semántica de estos tres campos.

**Especificación de cálculo (*comp*):** El Hack ALU está diseñado para calcular una de un conjunto fijo de funciones en dos entradas de 16 bits dadas. En la computadora Hack, las dos entradas de datos ALU están conectadas de la siguiente manera. La primera entrada ALU se alimenta desde el registro D. La segunda entrada ALU se alimenta desde el registro A (cuando el bit *a* es 0) o desde M, el registro de memoria de datos seleccionado (cuando el bit *a* es 1). En conjunto, la función calculada se especifica mediante el bit *a* y los seis bits *c* que componen el campo de composición de la *instrucción*. Este patrón de 7 bits puede codificar potencialmente 128 cálculos diferentes, de los cuales solo los veintiocho enumerados en [la figura 4.5](#) están documentados en la especificación del lenguaje.

Recuerde que el formato de la *instrucción C* es 111acccccddjjj. Supongamos que queremos calcular el valor del registro D, menos 1. De acuerdo con [la figura 4.5](#), esto se puede hacer usando la *instrucción* simbólica D-1, que se 1110001110000000 en binario (el campo de composición de 7 bits relevante está subrayado para enfatizar). Para calcular un Or bit a bit entre los valores de los registros D y M, usamos la *instrucción* D|M (en binario: 1111010101000000). Para calcular la constante -1, usamos la *instrucción* -1 (en binario: 1110111010000000), y así sucesivamente.

**Especificación de destino (*dest*):** La salida de ALU se puede almacenar en cero, uno, dos o tres destinos posibles, simultáneamente. El primer y segundo *d*-bits codifican si se debe almacenar el valor calculado en el registro A y en el registro D, respectivamente. El tercer bit *d* codifica si se debe almacenar el valor calculado en M, el registro de memoria seleccionado actualmente. Uno, más de uno o ninguno de estos tres bits puede afirmarse.

Recuerde que el formato de la instrucción *C* es 111acccccddjjj. Supongamos que deseamos incrementar el valor del registro de memoria cuya dirección es 7 y también almacenar el nuevo valor en el registro D. De acuerdo con la [figura 4.5](#), esto se puede lograr usando las dos instrucciones:

```
0000000000000111 // @7
1111110111011000 // DM=M+1
```

**Directiva de salto (*salto*):** El campo de *salto* de la instrucción *C* especifica qué hacer a continuación. Hay dos posibilidades: obtener y ejecutar la siguiente instrucción en el programa, que es la predeterminada, o buscar y ejecutar alguna otra instrucción designada. En este último caso, asumimos que el registro A ya estaba configurado en la dirección de la instrucción de destino.

Durante el tiempo de ejecución, el salto o no está determinado conjuntamente por los tres bits *j* del campo de salto *de la instrucción* y por la salida ALU. El primer, segundo y tercer *j*-bits especifican si saltar en caso de que la salida de ALU sea negativa, cero o positiva, respectivamente. Esto da ocho posibles condiciones de salto, enumeradas en la parte inferior derecha de la [figura 4.5](#). La convención para especificar una instrucción goto incondicional es 0; JMP (dado que el *campo comp* es obligatorio, la convención es calcular 0, una operación ALU elegida arbitrariamente, que se ignora).

**Prevención de usos conflictivos del registro A:** La computadora Hack usa un registro de direcciones para direccionar tanto la RAM como la ROM. Por lo tanto, cuando ejecutamos la instrucción @*n*, seleccionamos tanto RAM [*n*] como ROM [*n*]. Esto se hace para preparar el escenario para una instrucción *C* posterior que opera en el registro de memoria de datos seleccionado, M, o una instrucción *C posterior* que especifica un salto. Para asegurarnos de que realizamos exactamente una de estas dos operaciones, emitimos los siguientes consejos de mejores prácticas: A *C*-

instrucción que contiene una referencia a M debe especificar que no hay salto, y viceversa: una instrucción C que especifica un salto no debe hacer referencia a M.

#### 4.2.4 Símbolos

Las instrucciones de ensamblado pueden especificar ubicaciones de memoria (direcciones) utilizando constantes o símbolos. Los símbolos se dividen en tres categorías funcionales: *símbolos predefinidos*, que representan direcciones de memoria especiales; *símbolos de etiqueta*, que representan destinos de instrucciones GoTo; y *símbolos de variables*, que representan variables.

**Símbolos predefinidos:** Hay varios tipos de símbolos predefinidos, diseñados para promover la consistencia y la legibilidad de los programas Hack de bajo nivel.

**R0, R1, ..., R15:** Estos símbolos están vinculados a los valores de 0 a 15. Este enlace predefinido ayuda a que los programas Hack sean más legibles. Para ilustrarlo, considere el siguiente segmento de código:

```
// Sets RAM[3] to 7:
@7
D=A
@R3
M=D
```

La instrucción @7 establece el registro A en 7 y @R3 establece el registro A en 3. ¿Por qué usamos R en este último y no en el primero? Porque hace que el código se explique por sí mismo. En la instrucción @7, la sintaxis sugiere que A se usa como *registro de datos*, ignorando el efecto secundario de seleccionar también RAM[7]. En la instrucción @R3, la sintaxis sugiere que A se usa *para seleccionar una dirección de memoria de datos*. En general, los símbolos predefinidos R0, R1, ..., R15 pueden considerarse variables de trabajo listas para usar, a veces denominadas *registros virtuales*.

**SP, LCL, ARG, THIS, THAT:** estos símbolos están vinculados a los valores 0, 1, 2, 3 y 4, respectivamente. Por ejemplo, la dirección 2 se puede seleccionar usando @2, @R2 o @ARG. Los símbolos SP, LCL, ARG, THIS y THAT se usarán en la parte II del libro, cuando implementemos el compilador y el

máquina que se ejecuta sobre la plataforma Hack. Estos símbolos pueden ignorarse por completo por ahora; los especificamos para completarlos.

**SCREEN, KBD:** Los programas de piratería pueden leer datos de un teclado y mostrar datos en una pantalla. La pantalla y el teclado interactúan con la computadora a través de dos bloques de memoria designados conocidos como *mapas de memoria*. Los símbolos SCREEN y KBD están vinculados, respectivamente, a los valores 16384 y 24576 (en hexadecimal: 4000 y 6000), que son las direcciones base acordadas del mapa de memoria de *pantalla* y el mapa de memoria de *teclado*, respectivamente. Estos símbolos son utilizados por los programas Hack que manipulan la pantalla y el teclado, como veremos en la siguiente sección.

**Símbolos de etiqueta:** Las etiquetas pueden aparecer en cualquier lugar de un programa de ensamblado Hack y se declaran usando la sintaxis (*xxx*). Esta directiva vincula el símbolo *xxx* a la dirección de la siguiente instrucción del programa. Las instrucciones Goto que utilizan símbolos de etiqueta pueden aparecer en cualquier parte del programa, incluso antes de que se haya declarado la etiqueta. Por convención, los símbolos de etiqueta se escriben con letras mayúsculas. El programa enumerado en [la figura 4.4](#) utiliza tres símbolos de etiqueta: LOOP, STOP y END.

**Símbolos variables:** Cualquier símbolo *xxx* que aparezca en un programa de ensamblado Hack que no esté predefinido y no se declare en otro lugar usando (*xxx*) se trata como una variable y está enlazado a un número único que comienza en 16. Por convención, los símbolos variables se escriben con letras minúsculas. Por ejemplo, el programa enumerado en [la figura 4.4](#) usa dos variables: *i* y *suma*. Estos símbolos están vinculados por el ensamblador a 16 y 17, respectivamente. Por lo tanto, después de la traducción, instrucciones como *@i* y *@sum* terminan seleccionando las direcciones de memoria 16 y 17, respectivamente. La belleza de este contrato es que el programa de ensamblaje es completamente ajeno a las direcciones físicas. El programa de ensamblado usa solo símbolos, confiando en que el ensamblador sabrá cómo resolverlos en direcciones reales.

#### 4.2.5 Manejo de entrada/salida

La plataforma de hardware Hack se puede conectar a dos dispositivos periféricos de E/S: una pantalla y un teclado. Ambos dispositivos interactúan con la computadora

plataforma a través de *mapas de memoria*.

El dibujo de píxeles en la pantalla se realiza escribiendo valores binarios en un segmento de memoria designado asociado con la pantalla, y la escucha del teclado se realiza leyendo una ubicación de memoria designada asociada con el teclado. Los dispositivos de E/S físicos y sus mapas de memoria se sincronizan a través de bucles de actualización continua que son externos a la plataforma de hardware principal.

**Pantalla:** La computadora Hack interactúa con una pantalla en blanco y negro organizada como 256 filas de 512 píxeles por fila. El contenido de la pantalla está representado por un mapa de memoria, almacenado en un bloque de memoria de 8K de palabras de 16 bits, a partir de la dirección RAM 16384 (en hexadecimal: 4000), también denominada por el símbolo predefinido SCREEN. Cada fila de la pantalla física, comenzando en la esquina superior izquierda de la pantalla, está representada en la RAM por treinta y dos palabras consecutivas de 16 bits. Siguiendo la convención, el origen de la pantalla es la esquina superior izquierda, que se considera fila 0 y columna 0. Con eso en mente, el píxel en la fila de fila y columna *col* se asigna al % de *columna* de 16 bits (contando de LSB a MSB) de la palabra ubicada en RAM[SCREEN

+ *row* · 32 + *col* / 16]. Este píxel puede leerse (sondeando si es blanco o negro), volverse negro configurándolo en 1 o volverse blanco configurándolo en 0. Por ejemplo, considere el siguiente segmento de código, que ennegrece los primeros 16 píxeles en la parte superior izquierda de la pantalla:

```
// Sets the A register to the address of the RAM register that represents
// the 16 left-most pixels at the screen's top row:
@SCREEN
// Sets this RAM register to 1111111111111111:
M=-1
```

Tenga en cuenta que las instrucciones de Hack no pueden acceder directamente a píxeles/bits individuales. En su lugar, debemos obtener una palabra completa de 16 bits del mapa de memoria, averiguar qué bit o bits deseamos manipular, llevar a cabo la manipulación usando operaciones aritméticas / lógicas (sin tocar los otros bits) y luego escribir la palabra modificada de 16 bits en la memoria. En el ejemplo anterior, nos salimos con la nuestra sin hacer bits específicos



manipulaciones, ya que la tarea podría implementarse utilizando una manipulación masiva.

**Teclado:** La computadora Hack puede interactuar con un teclado físico estándar a través de un mapa de memoria de una sola palabra ubicado en la dirección RAM 24576 (en hexadecimal: 6000), también conocida por el símbolo predefinido KBD. El contrato es el siguiente: Cuando se presiona una tecla en el teclado físico, su código de caracteres de 16 bits aparece en RAM [KBD]. Cuando no se presiona ninguna tecla, aparece el código 0. El conjunto de caracteres Hack se enumera en el apéndice 5.

A estas alturas, los lectores con experiencia en programación probablemente hayan notado que manipular dispositivos de entrada / salida usando lenguaje ensamblador es un asunto tedioso. Esto se debe a que están acostumbrados a usar declaraciones de alto nivel como `write ("hola")` o `draw Circle (x, y, radius)`. Como puede apreciar ahora, existe una brecha considerable entre estas declaraciones de E/S abstractas y de alto nivel y las instrucciones de máquina bit a bit que terminan realizándolas en silicio. Uno de los agentes que cierra esta brecha es el *sistema operativo*, un programa que sabe, entre muchas otras cosas, cómo renderizar texto y dibujar gráficos mediante manipulaciones de píxeles. Discutiremos y escribiremos uno de esos sistemas operativos en la parte II del libro.

#### 4.2.7 Convenciones de sintaxis y formatos de archivo

**Archivos de código binario:** Por convención, los programas escritos en el lenguaje binario Hack se almacenan en archivos de texto con una extensión `hack`, por ejemplo, `Prog.hack`. Cada línea del archivo codifica una sola instrucción binaria, utilizando una secuencia de dieciséis caracteres 0 y 1. En conjunto, todas las líneas del archivo representan un programa de lenguaje de máquina. El contrato es el siguiente: Cuando se carga un programa de lenguaje de máquina en la memoria de instrucciones de la computadora, el código binario que aparece en la *enésima línea del archivo se almacena en la dirección  $n$*  de la memoria de instrucciones. Los recuentos de líneas de programa, instrucciones y direcciones de memoria comienzan en 0.

**Archivos de lenguaje ensamblador:** Por convención, los programas escritos en el lenguaje ensamblador simbólico Hack se almacenan en archivos de texto con una extensión `asm`, por ejemplo, `Prog.asm`. Un archivo de lenguaje ensamblador se compone de líneas de texto,

cada uno de los cuales es una *instrucción A*, una *instrucción C*, una declaración de etiqueta o un comentario.

Una declaración de etiqueta es una línea de texto con la forma (*símbolo*). El ensamblador controla dicha declaración enlazando el *símbolo* a la dirección de la siguiente instrucción del programa. Esta es la única acción que realiza el ensamblador al controlar una declaración de etiqueta; no se genera ningún código binario. Es por eso que las declaraciones de etiquetas a veces se denominan *pseudoinstrucciones*: existen solo a nivel simbólico, sin generar código.

**Constantes y símbolos:** Estos son los xxx en las instrucciones A de la forma @xxx. Las constantes son valores no negativos de 0 a  $2^{15}-1$ , y se escriben en notación decimal. Un símbolo es cualquier secuencia de letras, dígitos, guiones bajos (\_), punto (.), el signo de dólar (\$) y los dos puntos (:) que no comienzan con un dígito.

**Comentarios:** Una línea de texto que comienza con dos barras diagonales () y termina al final de la línea se considera un comentario y se ignora.

**Espacio en blanco:** se ignoran los caracteres del espacio inicial y las líneas vacías.

**Convenciones de mayúsculas y minúsculas:** Todos los mnemotécnicos de ensamblaje ([figura 4.5](#)) deben escribirse en mayúsculas. Por convención, los símbolos de etiqueta se escriben en mayúsculas y los símbolos de variables en minúsculas. Consulte [la figura 4.4](#) para ver ejemplos.

---

## 4.3 Hackear la programación

Ahora pasamos a presentar tres ejemplos de programación de bajo nivel, utilizando el lenguaje ensamblador Hack. Dado que el proyecto 4 se centra en escribir programas de ensamblaje Hack, le será útil leer y comprender cuidadosamente estos ejemplos.

**Ejemplo 1:** [La Figura 4.6](#) muestra un programa que suma los valores de los dos primeros registros de RAM, agrega 17 a la suma y almacena el resultado en el tercer registro de RAM. Antes de ejecutar el programa, se espera que el usuario (o un script de prueba) ponga algunos valores en RAM[0] y RAM[1].

```

// Program: Add.asm
// Computes: RAM[2] = RAM[0] + RAM[1] + 17
// Usage: put values in RAM[0] and in RAM[1]
// D = RAM[0]
@R0
D=M
// D = D + RAM[1]
@R1
D=D+M
// D = D + 17
@17
D=D+A
// RAM[2] = D
@R2
M=D
(END)
@END
0;JMP

```

**Figura 4.6** Un programa ensamblador de Hack que calcula una expresión aritmética simple.

Entre otras cosas, el programa ilustra cómo los llamados registros virtuales R0, R1, R2, ... se pueden utilizar como variables de trabajo. El programa también ilustra la forma recomendada de terminar los programas Hack, que es la puesta en escena y la entrada en un bucle infinito. En ausencia de este bucle infinito, la lógica de búsqueda-ejecución de la CPU (explicada en el siguiente capítulo) se deslizará alegremente hacia adelante, tratando de ejecutar las instrucciones almacenadas en la memoria de la computadora siguiendo la última instrucción del programa actual. Esto puede tener consecuencias impredecibles y potencialmente peligrosas. El bucle infinito deliberado sirve para controlar y contener el funcionamiento de la CPU después de completar la ejecución del programa.

**Ejemplo 2:** El segundo ejemplo calcula la suma  $1 + 2 + 3 + \dots + n$ , donde  $n$  es el valor del primer registro de RAM y coloca la suma en la segunda RAM

registro. Este programa se muestra en [la figura 4.4](#), y ahora tenemos lo que se necesita para entenderlo completamente.

Entre otras cosas, este programa ilustra el uso de variables simbólicas, en este caso  $i$  y  $\text{suma}$ . El ejemplo también ilustra nuestra práctica recomendada para el desarrollo de programas de bajo nivel: en lugar de escribir código ensamblador directamente, comience escribiendo pseudocódigo orientado a goto. A continuación, pruebe su pseudocódigo en papel, rastreando los valores de las variables clave. Cuando esté convencido de que la lógica del programa es correcta y de que hace lo que se supone que debe hacer, proceda a expresar cada pseudoinstrucción como una o más instrucciones de ensamblaje.

Las virtudes de escribir y depurar instrucciones simbólicas (en lugar de físicas) fueron observadas por la talentosa matemática y escritora Augusta Ada King-Noel, condesa de Lovelace, en 1843. Esta importante idea ha contribuido a su fama duradera como la primera programadora de la historia. Antes de Ada Lovelace, los protoprogramadores que trabajaban con las primeras computadoras mecánicas se redujeron a jugar directamente con las operaciones de las máquinas, y la codificación era difícil y propensa a errores. Lo que era cierto en 1843 sobre la programación simbólica y física es igualmente cierto hoy sobre la pseudoprogramación y la programación ensambladora: cuando se trata de programas no triviales, escribir y probar pseudocódigo y luego traducirlo en instrucciones de ensamblaje es más fácil y seguro que escribir código ensamblador directamente.

**Ejemplo 3:** Considere el lenguaje de procesamiento de matrices de alto nivel para  $i = 0 \dots n$  {do something with  $\text{arr}[i]$ }. Si deseamos expresar esta lógica en ensamblador, entonces nuestro primer desafío es que la abstracción de la matriz no existe en el lenguaje de máquina. Sin embargo, si conocemos la dirección base de la matriz en la RAM, podemos implementar fácilmente esta lógica en el ensamblador, utilizando el acceso basado en punteros a los elementos de la matriz.

Para ilustrar la noción de puntero, supongamos que la variable  $x$  contiene el valor 523, y considere las dos posibles pseudo-instrucciones  $x=17$  y  $*x=17$  (de las cuales ejecutamos solo una). La primera instrucción establece el valor de  $x$  en

17. La segunda instrucción informa que  $x$  debe tratarse como un *puntero*, es decir, una variable cuyo valor se interpreta como una dirección de memoria. Por lo tanto, la instrucción termina configurando  $\text{RAM}[523]$  en 17, dejando intacto el valor de  $x$ .

El programa de [la figura 4.7](#) ilustra el procesamiento de matrices basado en punteros en el lenguaje de máquina Hack. Las instrucciones clave de interés son  $A=D+M$ ,

seguido de  $M=-1$ . En el lenguaje Hack, el lenguaje básico de procesamiento de punteros se implementa mediante una instrucción de la forma  $A = \dots$ , seguida de una *instrucción C-* que opera en  $M$  (que significa  $RAM[A]$ , la ubicación de memoria seleccionada por  $A$ ). Como veremos cuando escribamos el compilador en la segunda parte del libro, este humilde lenguaje de programación de bajo nivel permite implementar, en el ensamblador Hack, cualquier acceso a matrices u operación get/set basada en objetos expresada en cualquier lenguaje de alto nivel.

| Pseudocode                                                                                                                                                                            | Hack assembly code                                                                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// Program: PointerDemo // Starting at base address R0, // sets the first R1 words to -1 n = 0 LOOP:   if (n == R1) goto END   *(R0 + n) = -1   n = n + 1   goto LOOP END:</pre> | <pre>// Program: PointerDemo.asm // Starting at base address R0, // sets the first R1 words to -1 // n = 0 @n M=0 (LLOOP) // if (n == R1) goto END @n D=M @R1 D=D-M @END D;JEQ // *(R0 + n) = -1 @R0 D=M @n A=D+M M=-1 // n = n + 1 @n M=M+1 // goto LOOP @LOOP 0;JMP (END) @END 0;JMP</pre> |

**Figura 4.7** Ejemplo de procesamiento de matrices, utilizando el acceso basado en punteros a los elementos de la matriz.

## 4.4 Proyecto

**Objetivo:** Probar la programación de bajo nivel y familiarizarse con el sistema informático Hack. Esto se hará escribiendo y ejecutando dos programas de bajo nivel, escritos en el lenguaje ensamblador Hack.

**Recursos:** Los únicos recursos necesarios para completar el proyecto son el emulador de *CPU Hack*, disponible en [nand2tetris/tools](http://nand2tetris/tools), y los scripts de prueba

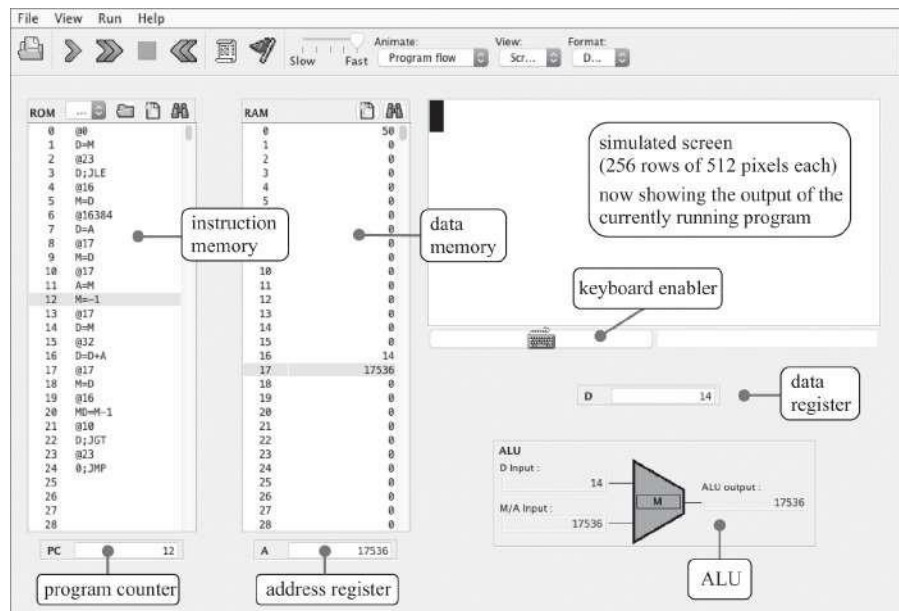
que se describe a continuación, disponible en la carpeta `projects/04`.

**Contrato:** Escriba y pruebe los dos programas que se describen a continuación. Cuando se ejecutan en el emulador de CPU suministrado, los programas deben realizar los comportamientos descritos.

**Multiplicación (Mult.asm):** Las entradas de este programa son los valores almacenados en R0 y R1 (RAM[0] y RAM[1]). El programa calcula el producto  $R0 * R1$  y almacena el resultado en R2. Suponga que  $R0 \geq 0$ ,  $R1 \geq 0$ , and  $R0 * R1 < 32768$  (su programa no necesita probar estas afirmaciones). Los scripts Mult.tst y Mult.cmp suministrados están diseñados para probar el programa en valores de datos representativos.

**Control de E/S (Fill.asm):** Este programa ejecuta un bucle infinito que escucha el teclado. Cuando se presiona una tecla (cualquier tecla), el programa ennegrece la pantalla escribiendo *negro* en cada píxel. Cuando no se presiona ninguna tecla, el programa borra la pantalla escribiendo *blanco* en cada píxel. Puede optar por ennegrecer y borrar la pantalla en cualquier patrón espacial, siempre que presionar una tecla continuamente durante el tiempo suficiente resulte en una pantalla completamente ennegrecida, y no presionar ninguna tecla durante el tiempo suficiente dará como resultado una pantalla despejada. Este programa tiene un script de prueba (Fill.tst) pero no un archivo de comparación: debe verificarse inspeccionando visiblemente la pantalla simulada en el emulador de CPU.

**Emulador de CPU:** Este programa, disponible en `nand2tetris/tools`, proporciona una simulación visual del ordenador Hack (ver [figura 4.8](#)). La GUI del programa muestra los estados actuales de la memoria de instrucciones (ROM) de la computadora, la memoria de datos (RAM), los dos registros A y D, la PC del contador de programas y la ALU. También muestra el estado actual de la pantalla de la computadora y permite ingresar entradas a través del teclado.



**Figura 4.8** El emulador de CPU, con un programa cargado en la memoria de instrucciones (ROM) y algunos datos en la memoria de datos (RAM). La figura muestra una instantánea tomada durante la ejecución del programa.

La forma típica de usar el emulador de CPU es cargar un programa de lenguaje de máquina en la ROM, ejecutar el código y observar su impacto en los elementos de hardware simulados. Es importante destacar que el emulador de CPU permite cargar archivos `binary.hack`, así como archivos `.asm` simbólicos, escritos en el lenguaje ensamblador Hack. En este último caso, el emulador traduce el programa ensamblador en código binario sobre la marcha. Convenientemente, el código cargado se puede ver tanto en sus representaciones binarias como simbólicas.

Dado que el emulador de CPU suministrado cuenta con un ensamblador incorporado, no es necesario usar un ensamblador Hack independiente en este proyecto.

**Pasos:** Recomendamos proceder de la siguiente manera:

0. El emulador de CPU suministrado está disponible en la carpeta `nand2tetris/tools`. Si necesitas ayuda, consulta el tutorial disponible en [www.nand2tetris.org](http://www.nand2tetris.org).
1. Escriba / edite el programa `Mult.asm` usando un editor de texto sin formato. Comience con el programa esquelético almacenado en `projects/04/mult/Mult.asm`.
2. Carga `Mult.asm` en el CPU emulador. Éste enlazar ser hecho ya sea de forma interactiva o cargando y ejecutando el `Mult.tst` Guión.



3. Ejecute el script. Si recibe algún error de traducción o de tiempo de ejecución, vaya al paso 1.

Siga los pasos 1 a 3 para escribir el segundo programa, usando los proyectos / 04 / fill carpeta.

**Consejo de depuración:** El lenguaje Hack distingue entre mayúsculas y minúsculas. Un error común de programación ensambladora ocurre cuando uno escribe, digamos, @foo y @Foo en diferentes partes del programa, pensando que ambas instrucciones se refieren al mismo símbolo. De hecho, el ensamblador generará y administrará dos variables que no tienen nada en común.

**Una versión basada en la web del proyecto 4** está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

---

## 4.5 Perspectiva

El lenguaje de máquina Hack es básico. Los lenguajes de máquina típicos presentan más operaciones, más tipos de datos, más registros y más formatos de instrucciones. En términos de sintaxis, hemos optado por darle a Hack una apariencia más ligera que la de los lenguajes ensambladores convencionales. En particular, hemos elegido una sintaxis amigable para la instrucción C, por ejemplo,  $D = D + M$ , en lugar de la sintaxis de prefijo más común `add M,D` utilizada en muchos lenguajes de máquina. El lector debe tener en cuenta, sin embargo, que esto es solo un efecto de sintaxis. Por ejemplo, el carácter `+` en el código de operación  $D + M$  no juega ningún papel algebraico. Más bien, la cadena de tres caracteres  $D + M$ , tomada en su conjunto, se trata como un único mnemotécnico de ensamblaje, diseñado para codificar una sola operación ALU.

Una de las principales características que le da a los lenguajes de máquina su sabor particular es la cantidad de direcciones de memoria que se pueden comprimir en una sola instrucción. A este respecto, el austero lenguaje Hack puede describirse como un *lenguaje de máquina de 1/2 dirección*: Dado que no hay espacio para empaquetar tanto un código de instrucción como una dirección de 15 bits en una sola instrucción de 16 bits, las operaciones que implican el acceso a la memoria requieren dos instrucciones Hack: una para especificar la dirección en la que deseamos operar, y otro para especificar la operación. En comparación, muchos lenguajes de máquina pueden especificar una operación y al menos una dirección en cada instrucción de máquina.

De hecho, el código ensamblador de Hack generalmente termina siendo principalmente una secuencia alterna de *instrucciones A* y *C*: @sum seguida de M=0, @LOOP seguida de 0; JMP, y así sucesivamente. Si encuentra este estilo de codificación tedioso o peculiar, debe tener en cuenta que *las macroinstrucciones* más amigables como sum=0 y goto LOOP se pueden introducir fácilmente en el lenguaje, lo que hace que el código ensamblador de Hack sea más corto y legible. El truco consiste en extender el ensamblador para traducir cada macroinstrucción en las dos instrucciones de Hack que conlleva, un ajuste relativamente simple.

El *ensamblador*, mencionado muchas veces en este capítulo, es el programa responsable de traducir los programas ensambladores simbólicos en programas ejecutables escritos en código binario. Además, el ensamblador es responsable de administrar todos los símbolos definidos por el sistema y el usuario que se encuentran en el programa de ensamblado y de resolverlos en direcciones de memoria física que se insertan en el código binario generado. Volveremos a esta tarea de traducción en el capítulo 6, que está dedicado a comprender y construir ensambladores.

---

1. Por *instantáneo* nos referimos al mismo ciclo de reloj, o unidad de tiempo.

---

## 5 Arquitectura de Computadores

Haz que todo sea lo más simple posible, pero no más simple.

—Albert Einstein (1879–1955)

Este capítulo es el pináculo de la parte de hardware de nuestro viaje. Ahora estamos listos para tomar los chips que construimos en los capítulos 1-3 e integrarlos en un sistema informático de propósito general, capaz de ejecutar programas escritos en el lenguaje de máquina presentado en el capítulo 4. El ordenador específico que construiremos, llamado Hack, tiene dos virtudes importantes. Por un lado, Hack es una máquina sencilla que se puede construir en pocas horas, utilizando chips previamente contruidos y el simulador de hardware suministrado. Por otro lado, Hack es lo suficientemente poderoso como para ilustrar los principios operativos clave y los elementos de hardware de cualquier computadora de propósito general. Por lo tanto, construirlo le dará una comprensión práctica de cómo funcionan las computadoras modernas y cómo se construyen.

La sección 5.1 comienza con una descripción general de la arquitectura de *von Neumann*, un dogma central en la informática que subyace al diseño de casi todas las computadoras modernas. La plataforma Hack es una variante de la máquina von Neumann, y la sección 5.2 da su especificación exacta de hardware. La sección 5.3 describe cómo se puede implementar la plataforma Hack a partir de chips contruidos previamente, en particular el ALU incorporado en el proyecto 2 y los registros y dispositivos de memoria incorporados en el proyecto 3. La Sección 5.4 describe el proyecto en el que construirá la computadora. La sección 5.5 proporciona perspectiva. En particular, comparamos la máquina Hack con las computadoras de potencia industrial y enfatizamos el papel crítico que juega la optimización en estas últimas.

La computadora que surgirá de este esfuerzo será lo más simple posible, pero no más simple. Por un lado, el ordenador se basará en un

configuración de hardware minimalista y elegante. Por otro lado, la configuración resultante será lo suficientemente potente como para ejecutar programas escritos en un lenguaje de programación similar a Java, presentado en la parte II del libro. Este lenguaje permitirá desarrollar juegos de computadora interactivos y aplicaciones que involucren gráficos y animación, brindando un rendimiento sólido y una experiencia de usuario satisfactoria. Para realizar estas aplicaciones de alto nivel en la plataforma de hardware barebone, necesitaremos construir un compilador, una máquina virtual y un sistema operativo. Esto se hará en la parte II. Por ahora, completemos la parte I integrando los chips que hemos construido hasta ahora en una plataforma de hardware completa y de propósito general.

---

## **5.1 Fundamentos de la arquitectura de computadoras**

### **5.1.1 El concepto de programa almacenado**

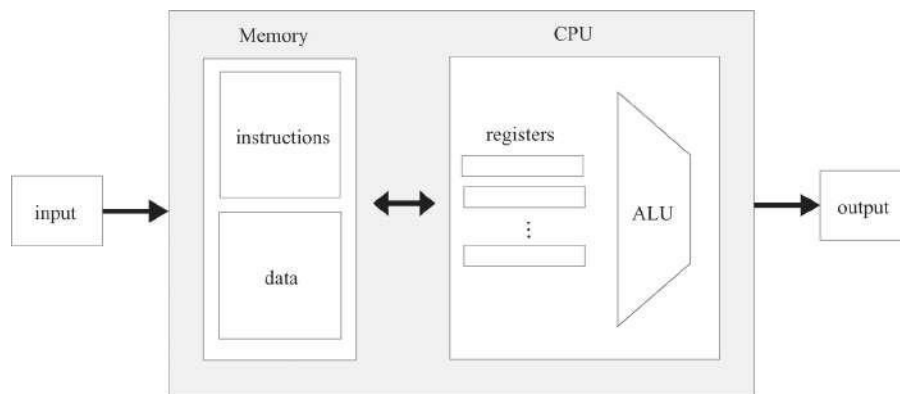
En comparación con todas las máquinas que nos rodean, la característica más notable de la computadora digital es su asombrosa versatilidad. Aquí hay una máquina con un hardware finito y fijo que puede realizar un número infinito de tareas, desde jugar juegos hasta componer libros y conducir automóviles. Esta notable versatilidad, una bendición que hemos llegado a dar por sentada, es el fruto de una brillante idea temprana llamada concepto de *programa almacenado*. Formulado de forma independiente por varios científicos e ingenieros en la década de 1930, el concepto de programa almacenado todavía se considera la invención más profunda, si no la base misma, de la informática moderna.

Como muchos avances científicos, la idea básica es simple. La computadora se basa en una plataforma de hardware fija capaz de ejecutar un repertorio fijo de instrucciones simples. Al mismo tiempo, estas instrucciones se pueden combinar como bloques de construcción, produciendo programas arbitrariamente sofisticados. Además, la lógica de estos programas no está integrada en el hardware, como era habitual en las computadoras mecánicas anteriores a 1930. En cambio, el código del programa se almacena temporalmente en la memoria de la computadora, *como datos*, convirtiéndose en lo que se conoce como *software*. Dado que el funcionamiento de la computadora se manifiesta al usuario a través del software que se está ejecutando actualmente, se puede hacer que la misma plataforma de hardware se comporte de manera completamente diferente cada vez que se carga con un programa diferente.

### 5.1.2 La arquitectura de von Neumann

El concepto de programa almacenado es el elemento clave de los modelos informáticos abstractos y prácticos, sobre todo la máquina de *Turing* (1936) y la *máquina de von Neumann* (1945). La máquina de Turing, un artefacto abstracto que describe una computadora engañosamente simple, se usa principalmente en informática teórica para analizar los fundamentos lógicos de la computación. Por el contrario, la máquina de von Neumann es un modelo práctico que informa la construcción de casi todas las plataformas informáticas actuales.

La arquitectura de von Neumann, que se muestra en [la figura 5.1](#), se basa en una *Unidad Central de Procesamiento* (CPU), que interactúa con un dispositivo de memoria, recibe datos de algún dispositivo de *entrada* y emite datos a algún dispositivo de *salida*. En el corazón de esta arquitectura se encuentra el concepto de *programa almacenado*: la memoria de la computadora almacena no solo los datos que la computadora manipula, sino también las instrucciones que le dicen a la computadora qué hacer. Exploremos esta arquitectura con cierto detalle.



**Figura 5.1** Una arquitectura de computadora genérica de von Neumann.

### 5.1.3 Memoria

La memoria de la computadora se puede discutir desde perspectivas físicas y lógicas. Físicamente, la memoria es una secuencia lineal de registros direccionables de tamaño fijo, cada uno con una dirección única y un valor. Lógicamente, este espacio de direcciones tiene dos propósitos: almacenar datos y almacenar instrucciones. Tanto las "palabras de instrucción" como las "palabras de datos" se implementan exactamente de la misma manera, como secuencias de bits.

Todos los registros de memoria, independientemente de sus funciones, se manejan de la misma manera: para acceder a un registro de memoria en particular, proporcionamos la dirección del registro. Esta acción, también conocida como *direccionamiento*, proporciona un acceso inmediato a los datos del registro. El término *memoria de acceso aleatorio* deriva del importante requisito de que cada registro de memoria seleccionado al azar se pueda alcanzar instantáneamente, es decir, dentro del mismo ciclo (o paso de tiempo), independientemente del tamaño de la memoria y la ubicación del registro. Este requisito claramente tiene su peso en las unidades de memoria que tienen miles de millones de registros. Los lectores que construyeron los dispositivos RAM en el proyecto 3 saben que ya hemos cumplido con este requisito.

A continuación, nos referiremos al área de memoria dedicada a los datos como memoria de *datos* y al área de memoria dedicada a las instrucciones como *memoria de instrucciones*. En algunas variantes de la arquitectura de von Neumann, la memoria de datos y la memoria de instrucciones se asignan y administran dinámicamente, según sea necesario, dentro del mismo espacio de direcciones físicas. En otras variantes, la memoria de datos y la memoria de instrucciones se mantienen en dos unidades de memoria físicamente separadas, cada una con su propio espacio de direcciones distinto. Ambas variantes tienen pros y contras, como discutiremos más adelante.

**Memoria de datos:** los programas de alto nivel están diseñados para manipular artefactos abstractos como variables, matrices y objetos. Sin embargo, a nivel de hardware, estas abstracciones de datos se realizan mediante valores binarios almacenados en registros de memoria. En particular, después de la traducción al lenguaje de máquina, el procesamiento de matrices abstractas y las operaciones de obtención / establecimiento en objetos se reducen a *leer* y *escribir* registros de memoria seleccionados. Para leer un registro, proporcionamos una dirección y sondeamos el valor del registro seleccionado. Para escribir en un registro, proporcionamos una dirección y almacenamos un nuevo valor en el registro seleccionado, anulando su valor anterior.

**Memoria de instrucciones:** antes de que se pueda ejecutar un programa de alto nivel en una computadora de destino, primero debe traducirse al lenguaje de máquina de la computadora de destino. Cada instrucción de alto nivel se traduce en una o más instrucciones de bajo nivel, que luego se escriben como valores binarios en un archivo llamado *versión binaria* o *ejecutable* del programa. Antes de ejecutar un programa, primero debemos

cargar su versión binaria desde un dispositivo de almacenamiento masivo y serializar sus instrucciones en la memoria de instrucciones de la computadora.



Desde el enfoque puro de la arquitectura de computadoras, *la forma* en que se carga un programa en la memoria de la computadora se considera un problema externo. Lo importante es que cuando se llama a la CPU para ejecutar un programa, el código del programa ya residirá en la memoria de la computadora.

#### 5.1.4 Unidad central de procesamiento

La Unidad Central de Procesamiento (CPU), la pieza central de la arquitectura de la computadora, está a cargo de ejecutar las instrucciones del programa que se está ejecutando actualmente. Cada instrucción le dice a la CPU qué cálculo realizar, a qué registros acceder y qué instrucción buscar y ejecutar a continuación. La CPU ejecuta estas tareas utilizando tres elementos principales: una unidad lógica aritmética (ALU), un conjunto de registros y una unidad de control.

**Unidad lógica aritmética:** El chip ALU está diseñado para realizar todas las operaciones aritméticas y lógicas de bajo nivel que presenta la computadora. Una ALU típica puede sumar dos valores dados, calcular su And bit a bit, compararlos para la igualdad, etc. La cantidad de funcionalidad que debe presentar la ALU es una decisión de diseño. En general, cualquier función no compatible con la ALU se puede realizar más tarde, utilizando el software del sistema que se ejecuta sobre la plataforma de hardware. La compensación es simple: las implementaciones de hardware suelen ser más eficientes pero dan como resultado un hardware más costoso, mientras que las implementaciones de software son económicas y menos eficientes.

**Registros:** En el curso de la realización de cálculos, a menudo se requiere que la CPU almacene valores provisionales temporalmente. En teoría, podríamos haber almacenado estos valores en registros de memoria, pero esto implicaría viajes de larga distancia entre la CPU y la RAM, que son dos chips separados. Estos retrasos frustrarían la ALU residente de la CPU, que es una calculadora combinatoria ultrarrápida. El resultado será una condición conocida como *inanición*, que es lo que sucede cuando un procesador rápido depende de un almacén de datos lento para suministrar sus entradas y consumir sus salidas.

Para evitar el hambre y aumentar el rendimiento, normalmente equipamos la CPU con un pequeño conjunto de registros de alta velocidad (y relativamente caros), que actúan como memoria inmediata del procesador. Estos registros sirven para varios propósitos: *los registros de*

*datos* almacenan valores provisionales, *los registros de direcciones*  
almacenan valores

que se utilizan para direccionar la RAM, el *contador del programa* almacena la dirección de la instrucción que debe obtenerse y ejecutarse a continuación, y el *registro de instrucciones* almacena la instrucción actual. Una CPU típica usa unas pocas docenas de registros de este tipo, pero nuestra frugal computadora Hack solo necesitará tres.

**Control:** Una instrucción informática es un paquete estructurado de microcódigos acordados, es decir, secuencias de uno o más bits diseñados para indicar a diferentes dispositivos qué hacer. Por lo tanto, antes de que se pueda ejecutar una instrucción, primero debe decodificarse en sus microcódigos. A continuación, cada microcódigo se enruta a su dispositivo de hardware designado (ALU, registros, memoria) dentro de la CPU, donde le dice al dispositivo cómo participar en la ejecución general de la instrucción.

**Fetch-Execute:** En cada paso (ciclo) de la ejecución del programa, la CPU obtiene una instrucción de máquina binaria de la memoria de instrucciones, la decodifica y la ejecuta. Como efecto secundario de la ejecución de la instrucción, la CPU también averigua qué instrucción buscar y ejecutar a continuación. Este proceso repetitivo a veces se denomina *ciclo de recuperación-ejecución*.

### 5.1.5 Entrada y salida

Las computadoras interactúan con sus entornos externos utilizando una gran variedad de dispositivos de entrada y salida (E / S): pantallas, teclados, dispositivos de almacenamiento, impresoras, micrófonos, altavoces, tarjetas de interfaz de red, etc., sin mencionar la desconcertante variedad de sensores y activadores integrados en automóviles, cámaras, audífonos, sistemas de alarma y todos los dispositivos que nos rodean. Hay dos razones por las que no nos preocupamos por estos dispositivos de E/S. En primer lugar, cada uno de ellos representa una pieza única de maquinaria, que requiere un conocimiento único de ingeniería. En segundo lugar, por esa misma razón, los científicos informáticos han ideado esquemas inteligentes para abstraer esta complejidad y hacer que todos los dispositivos de E / S se vean exactamente iguales para la computadora. El elemento clave de esta abstracción se denomina *E/S mapeada en memoria*.

La idea básica es crear una emulación binaria del dispositivo de E / S, haciendo que parezca a la CPU como si fuera un segmento de memoria lineal normal. Esto se hace asignando, para cada dispositivo de E/S, un área designada en el

memoria que actúa como su mapa de memoria. En el caso de un dispositivo de entrada como un teclado, el mapa de memoria está hecho para reflejar continuamente el estado físico del dispositivo: cuando el usuario presiona una tecla en el teclado, aparece un código binario que representa esa tecla en el mapa de memoria del teclado. En el caso de un dispositivo de salida como una pantalla, la pantalla está hecha para reflejar continuamente el estado de su mapa de memoria designado: cuando escribimos un bit en el mapa de memoria de la pantalla, un píxel respectivo se enciende o apaga en la pantalla.

Los dispositivos de E/S y los mapas de memoria se actualizan o sincronizan muchas veces por segundo, por lo que el tiempo de respuesta desde la perspectiva del usuario parece ser instantáneo. Programáticamente, la implicación clave es que los programas informáticos de bajo nivel pueden acceder a cualquier dispositivo de E / S manipulando su mapa de memoria designado.

La convención del mapa de memoria se basa en varios contratos acordados. Primero, los datos que controlan cada dispositivo de E/S deben serializarse o asignarse a la memoria de la computadora, de ahí el nombre *de mapa de memoria*. Por ejemplo, la pantalla, que es una cuadrícula bidimensional de píxeles, se asigna a un bloque unidimensional de registros de memoria de tamaño fijo. En segundo lugar, se requiere que cada dispositivo de E / S admita un protocolo de interacción acordado para que los programas puedan acceder a él de manera predecible. Por ejemplo, se debe decidir qué códigos binarios deben representar qué teclas en el teclado. Dada la multitud de plataformas informáticas, dispositivos de E / S y diferentes proveedores de hardware y software, se puede apreciar el papel crucial que desempeñan los estándares acordados en toda la industria en la realización de estos contratos de interacción de bajo nivel.

Las implicaciones prácticas de la E/S mapeada en memoria son significativas: el sistema informático es totalmente independiente del número, la naturaleza o la marca de los dispositivos de E/S que interactúan, o pueden interactuar, con él. Siempre que queramos conectar un nuevo dispositivo de E/S al ordenador, todo lo que tenemos que hacer es asignarle un nuevo mapa de memoria y tomar nota de la dirección base del mapa (estas configuraciones únicas las llevan a cabo los llamados programas de *instalación*). Otro elemento necesario es un programa *controlador de dispositivo*, que se agrega al sistema operativo de la computadora. Este programa cierra la brecha entre los datos del mapa de memoria del dispositivo de E/S y la forma en que estos datos se representan realmente en el dispositivo de E/S físico o se generan en él.

---

## 5.2 La plataforma de hardware Hack: Especificación

El marco arquitectónico descrito hasta ahora es característico de cualquier sistema informático de propósito general. Ahora pasamos a describir una variante específica de esta arquitectura: la computadora Hack. Como es habitual en Nand to Tetris, comenzamos con la abstracción, centrándonos en *lo que* la computadora está diseñada para hacer. La implementación de la computadora, *cómo* lo hace, se describe más adelante.

### 5.2.1 Visión general

La plataforma Hack es una máquina von Neumann de 16 bits diseñada para ejecutar programas escritos en el lenguaje de máquina Hack. Para ello, la plataforma Hack consta de una *CPU*, dos módulos de memoria separados que sirven como *memoria de instrucciones* y *memoria de datos*, y dos dispositivos de E/S mapeados en memoria: una *pantalla* y un *teclado*.

La computadora Hack ejecuta programas que residen en una memoria de instrucciones. En las implementaciones físicas de la plataforma Hack, la memoria de instrucciones se puede implementar como un chip ROM (memoria de solo lectura) que está precargado con el programa requerido. Los emuladores basados en software de la computadora Hack admiten esta funcionalidad al proporcionar medios para cargar la memoria de instrucciones desde un archivo de texto que contiene un programa escrito en el lenguaje de máquina Hack.

La CPU Hack consta del proyecto 2 integrado en ALU y tres registros llamados *Registro de datos* (D), *Registro de direcciones* (A) y *Contador de programas* (PC). El registro D y el registro A son idénticos al chip de registro incorporado en el proyecto 3, y el contador de programa es idéntico al chip de PC integrado en el proyecto

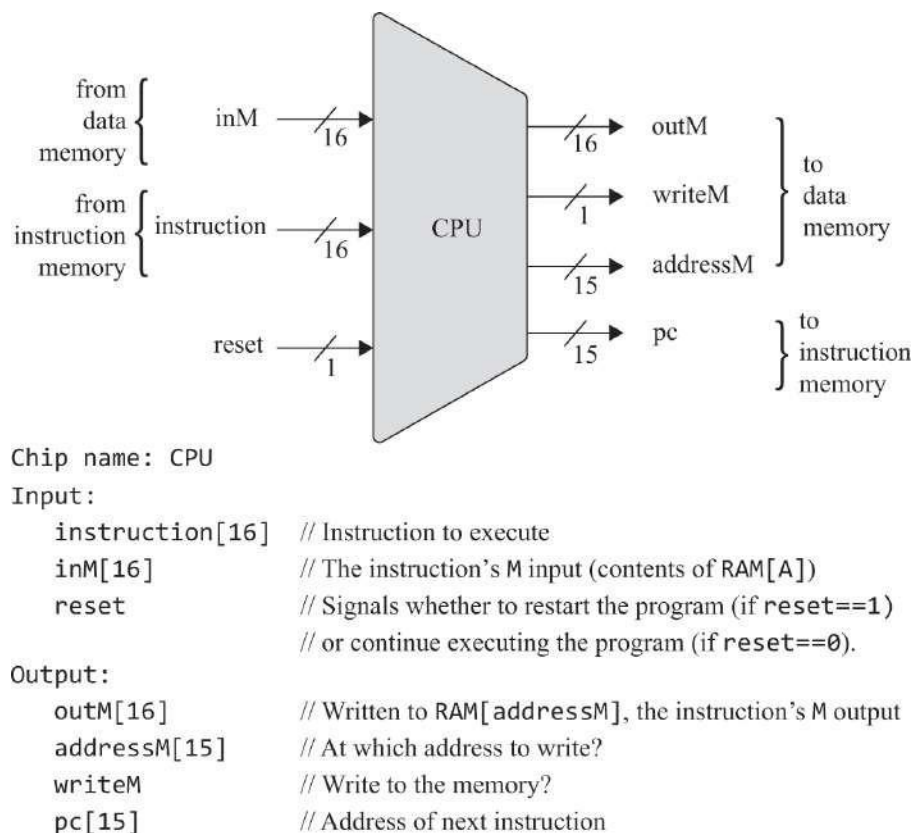
3. Mientras que el registro D se utiliza únicamente para almacenar valores de datos, el registro A sirve para uno de tres propósitos diferentes, dependiendo del contexto en el que se utilice: almacenar un valor de datos (como el registro D), seleccionar una dirección en la memoria de instrucciones o seleccionar una dirección en la memoria de datos.

La CPU Hack está diseñada para ejecutar instrucciones escritas en el lenguaje de máquina Hack. En el caso de una *instrucción A*, los 16 bits de la instrucción se tratan como un valor binario que se carga tal cual en el registro A. En el caso de una *instrucción C*, la instrucción se trata como una cápsula de bits de control que especifican varias microoperaciones que deben realizar varias partes del chip

dentro de la CPU. Ahora pasamos a describir cómo la CPU materializa estos microcódigos en acciones concretas.

### 5.2.2 Unidad central de procesamiento

La interfaz de Hack CPU se muestra en la [figura 5.2](#). La CPU está diseñada para ejecutar instrucciones de 16 bits de acuerdo con la especificación de lenguaje de máquina Hack presentada en el capítulo 4. La CPU consta de una ALU, dos registros llamados A y D, y un contador de programas llamado PC (estas partes internas del chip no se ven fuera de la CPU). La CPU espera estar conectada a una memoria de instrucciones, de la que obtiene instrucciones para su ejecución, y a una memoria de datos, desde la que puede leer y escribir valores de datos. La entrada `inM` y la salida `outM` contienen los valores denominados `M` en la sintaxis de la instrucción `C`. La salida `addressM` contiene la dirección en la que se debe escribir `outM`.



**Figura 5.2** La interfaz de la unidad central de procesamiento (CPU) de Hack.

Si la entrada de instrucción es una instrucción  $A$ , la CPU carga el valor de instrucción de 16 bits en el registro A. Si la instrucción es una *instrucción C*, entonces (i) la CPU hace que la ALU realice el cálculo especificado por la instrucción y (ii) la CPU hace que este valor se almacene en el subconjunto de

$\{A,D,M\}$  registros de destino especificados por la instrucción. Si uno de los registros de destino es M, la salida outM de la CPU se establece en la salida ALU y la salida writeM de la CPU se establece en 1. De lo contrario, writeM se establece en 0 y cualquier valor puede aparecer en outM.

Siempre que la entrada de reinicio sea 0, la CPU usa la salida ALU y los bits de salto de la instrucción actual para decidir qué instrucción buscar a continuación. Si el reinicio es 1, la CPU pone pc en 0. Más adelante en el capítulo conectaremos la salida de la PC de la CPU a la entrada de dirección del chip de memoria de instrucciones, lo que hará que este último emita la siguiente instrucción. Esta configuración realizará el paso de recuperación del ciclo de captura y ejecución.

Las salidas outM y writeM de la CPU se realizan mediante *lógica combinatorial*; por lo tanto, se ven afectadas instantáneamente por la ejecución de la instrucción. Las salidas addressM y pc están *sincronizadas*: aunque se ven afectadas por la ejecución de la instrucción, se comprometen con sus nuevos valores solo en el siguiente paso de tiempo.

### 5.2.3 Memoria de instrucciones

La *memoria de instrucciones Hack*, llamada ROM32K, se especifica en la [figura 5.3](#).



Chip name: ROM32K  
 Input: address[15]  
 Output: out[16]  
 Function: Emits the 16-bit value stored in the address selected by the address input. It is assumed that the chip is preloaded with a program written in the Hack machine language.

**Figura 5.3** La interfaz de memoria de instrucciones Hack.

#### 5.2.4 Entrada/Salida

El acceso a los dispositivos de entrada/salida de la computadora Hack es posible gracias a la memoria de datos de la computadora, un dispositivo RAM de lectura/escritura que consta de registros direccionables de 16 bits de 32K. Además de servir como almacén de datos de propósito general de la computadora, la memoria de datos también interactúa entre la CPU y los dispositivos de entrada / salida de la computadora, como ahora vamos a describir.

La plataforma Hack se puede conectar a dos dispositivos periféricos: una *pantalla* y un *teclado*. Ambos dispositivos interactúan con la plataforma informática a través de áreas de memoria designadas llamadas *mapas de memoria*. Específicamente, las imágenes se pueden dibujar en la pantalla escribiendo valores de 16 bits en un segmento de memoria designado llamado *mapa de memoria de pantalla*. Del mismo modo, la tecla que se presiona actualmente en el teclado se puede determinar sondeando un registro de memoria de 16 bits designado llamado *mapa de memoria del teclado*.

El mapa de memoria de la pantalla y el mapa de memoria del teclado se actualizan continuamente, muchas veces por segundo, mediante la lógica de actualización periférica externa al equipo. Por lo tanto, cuando se cambian uno o más bits en el mapa de memoria de la pantalla, el cambio se refleja inmediatamente en la pantalla física. Del mismo modo, cuando se presiona una tecla en el teclado físico, el código de caracteres de la tecla presionada aparece inmediatamente en el mapa de memoria del teclado. Con eso en mente, cuando un programa de bajo nivel quiere leer algo desde el teclado o escribir algo en la pantalla, el programa manipula los respectivos mapas de memoria de estos dispositivos de E / S.

En la plataforma informática Hack, el mapa de memoria de pantalla y el mapa de memoria de teclado se realizan mediante dos chips incorporados llamados Screen y Keyboard. Estos chips se comportan como dispositivos de memoria estándar, con los efectos secundarios adicionales de la sincronización continua entre los dispositivos de E/S y sus respectivos mapas de memoria. Ahora pasamos a especificar estos chips en detalle.

**Pantalla:** La computadora Hack puede interactuar con una pantalla física que consta de 256 filas de 512 píxeles en blanco y negro cada una, que abarcan una cuadrícula de 131,072 píxeles. La computadora interactúa con la pantalla física a través de un mapa de memoria, implementado por un chip de memoria 8K de registros de 16 bits. Este chip, llamado Screen, se comporta como un chip de memoria normal, lo que significa que se puede



leer y escribir utilizando la interfaz RAM normal. Además, el chip de pantalla

presenta el efecto secundario de que el estado de cualquiera de sus bits se refleja continuamente en un píxel respectivo en la pantalla física (1 = black, 0 = white).

La pantalla física es un espacio de direcciones bidimensional, donde cada píxel se identifica mediante una fila y una columna. Los lenguajes de programación de alto nivel suelen contar con una biblioteca de gráficos que permite acceder a píxeles individuales proporcionando coordenadas (*fila*, *columna*). Sin embargo, el mapa de memoria que representa esta pantalla bidimensional en el nivel bajo es una secuencia unidimensional de palabras de 16 bits, cada una identificada proporcionando una dirección. Por lo tanto, no se puede acceder directamente a los píxeles individuales. Más bien, tenemos que averiguar en qué palabra se encuentra el bit de destino y luego acceder y manipular toda la palabra de 16 bits a la que pertenece este píxel. La asignación exacta entre estos dos espacios de direcciones se especifica en la [figura 5.4](#). Este mapeo se realizará mediante el controlador de pantalla del sistema operativo que desarrollaremos en la parte II del libro.

```

Chip name: Screen      // Screen memory map
Input:      in[16]      // What to write
            address[13] // Where to read/write
            load         // Write-enable bit
Output:     out[16]     // Screen value at the given address
Function:    Exactly like a 16-bit, 8K RAM, plus a screen refresh side effect.

    Emits the value stored at the memory location specified by address.
    If load==1, then the memory location specified by address is set to the value of in.
    The loaded value will be emitted by out from the next time step onward.
    In addition, the chip continuously refreshes a physical screen, consisting of 256 rows
    and 512 columns of black and white pixels.

    The pixel at row r from the top and column c from the left ( $0 \leq r \leq 255$ ,  $0 \leq c \leq 511$ )
    is mapped onto the  $c\%16$  bit (counting from LSB to MSB) of the 16-bit word stored
    in Screen[r * 32 + c / 16].

    (Simulators of the Hack computer are expected to simulate the physical screen,
    the mapping, and the refresh contract).
```

**Figura 5.4** La interfaz del chip Hack Screen .

**Teclado:** La computadora Hack puede interactuar con un teclado físico, como el de una computadora personal. La computadora interactúa con el teclado físico a través de un mapa de memoria, implementado por el chip del teclado, cuya interfaz se muestra en la [figura 5.5](#). La interfaz del chip es idéntica a la de un registro de 16 bits de solo lectura. Además, el chip del teclado tiene el efecto secundario de reflejar el estado del teclado físico: cuando se presiona una tecla en el

teclado físico, el código de 16 bits del carácter respectivo es emitido por la salida del chip del teclado . Cuando no se presiona ninguna tecla, el chip emite 0. El conjunto de caracteres admitido por la computadora Hack se proporciona en el apéndice 5, junto con el código de cada carácter.

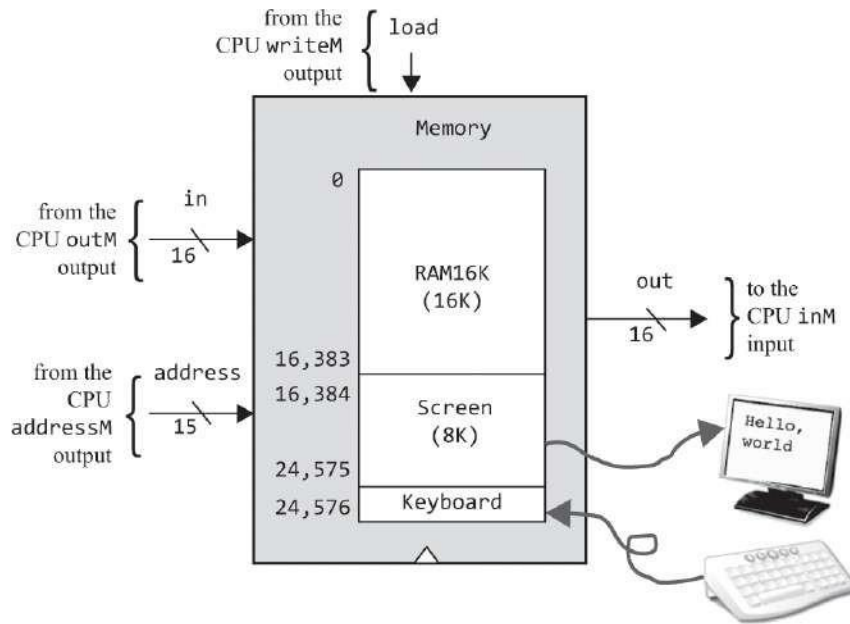
```
Chip name: Keyboard    // Keyboard memory map
Output:    out[16]
Function:   Emits the 16-bit character code of the currently pressed
            key on the physical keyboard or 0 if no key is pressed.

(Simulators of the Hack computer are expected to simulate this refresh contract).
```

**Figura 5.5** La interfaz del chip Hack Keyboard .

### 5.2.5 Memoria de datos

El espacio de direcciones general de la memoria de datos de Hack se realiza mediante un chip llamado Memory. Este chip es esencialmente un paquete de tres partes de chip de 16 bits: RAM16K (un chip RAM de registros de 16K, que sirve como almacén de datos de propósito general), Screen (un chip RAM incorporado de registros 8K, que actúa como mapa de memoria de la pantalla) y Keyboard (un chip de registro incorporado, que actúa como mapa de memoria del teclado). La especificación completa se da en la [figura 5.6](#).



```

Chip name: Memory           // Data memory
Input:      in[16]           // What to write
            address[15]      // Where to read/write
            load              // Write-enable bit
Output:     out[16]          // Value at the given address
Function:

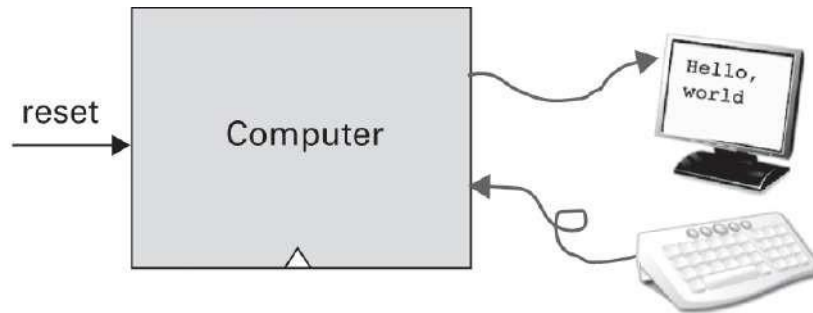
```

The complete address space of the Hack computer's data memory.  
 Only the top 16K+8K+1 words of the address space are used.  
 Accessing an address in the range 0 - 16383 results in accessing RAM16K;  
 Accessing an address in the range 16384 - 24575 results in accessing Screen;  
 Accessing the address 24576 results in accessing Keyboard;  
 Accessing any other address is invalid.

**Figura 5.6** La interfaz de memoria de datos Hack. Tenga en cuenta que los valores decimales 16384 y 24576 son 4000 y 6000 en hexadecimal.

## 5.2.6 Ordenador

El elemento más alto en la jerarquía de hardware de Hack es una computadora en un chip llamada Computadora (figura 5.7). El chip de la computadora se puede conectar a una pantalla y un teclado. El usuario ve la pantalla, el teclado y una entrada de un solo bit llamada reset. Cuando el usuario establece este bit en 1 y luego en 0, la computadora comienza a ejecutar el programa cargado actualmente. A partir de este momento, el usuario está a merced del software.



Chip name: Computer

Input: reset

Function:

When `reset==0`, the program stored in the computer executes.

When `reset==1`, the execution of the program restarts.

To start the program's execution, set `reset` to 1, and then to 0.

(It is assumed that the computer's instruction memory is loaded with a program written in the Hack machine language).

**Figura 5.7** Interfaz de Ordenador, el chip superior de la plataforma de hardware Hack.

Esta lógica de inicio realiza lo que a veces se conoce como "*arrancar la computadora*". Por ejemplo, cuando inicia una PC o un teléfono celular, el dispositivo está configurado para ejecutar un programa residente en ROM. Este programa, a su vez, carga el kernel del sistema operativo (también un programa) en la RAM y comienza a ejecutarlo. Luego, el kernel ejecuta un proceso (otro programa más) que escucha los dispositivos de entrada de la computadora, es decir, teclado, mouse, pantalla táctil, micrófono, etc. En algún momento, el usuario hará algo y el sistema operativo responderá ejecutando otro proceso o invocando algún programa.

En la computadora Hack, el software consiste en una secuencia binaria de instrucciones de 16 bits, escritas en el lenguaje de máquina Hack y almacenadas en la memoria de instrucciones de la computadora. Por lo general, este código binario será la versión de bajo nivel de un programa escrito en algún lenguaje de alto nivel y traducido por un *compilador* al lenguaje de máquina Hack. El proceso de compilación se discutirá e implementará en la parte II del libro.

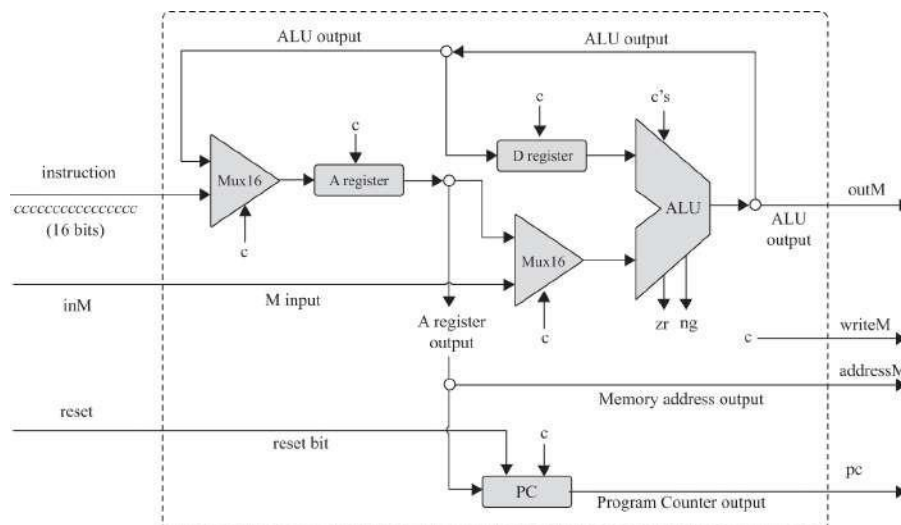
---

## 5.3 Implementación

En esta sección se describe una implementación de hardware que realiza el equipo Hack especificado en la sección anterior. Como de costumbre, no damos instrucciones de construcción exactas. Más bien, esperamos que los lectores descubran y completen los detalles de implementación por su cuenta. Todos los chips descritos a continuación se pueden construir en HDL y simular en una computadora personal utilizando el simulador de hardware suministrado.

### 5.3.1 La Unidad Central de Procesamiento

La implementación de la CPU Hack implica la construcción de una arquitectura de puerta lógica capaz de (i) ejecutar una instrucción Hack dada y (ii) determinar qué instrucción debe obtenerse y ejecutarse a continuación. Para ello, utilizaremos la lógica de puerta para decodificar la instrucción actual, una unidad lógica aritmética (ALU) para calcular la función especificada por la instrucción, un conjunto de registros para almacenar el valor resultante, según lo especificado por la instrucción, y un contador de programas para realizar un seguimiento de qué instrucción debe obtenerse y ejecutarse a continuación. Dado que todos los bloques de construcción subyacentes (ALU, registros, PC y puertas lógicas elementales) ya se construyeron en capítulos anteriores, la pregunta clave a la que nos enfrentamos ahora es cómo conectar estas partes del chip juiciosamente de una manera que realice la operación deseada de la CPU. Una posible configuración se ilustra en [la figura 5.8](#) y se explica en las páginas siguientes.



**Figura 5.8** Implementación propuesta de la CPU Hack, que muestra una instrucción entrante de 16 bits. Usamos la notación de instrucciones *cccccccccccccccc* para enfatizar que en el caso de una *instrucción C*, la instrucción se trata como una cápsula de bits de control, diseñada para controlar diferentes partes del chip de la CPU. En este diagrama, cada *símbolo c* que ingresa a una parte del chip representa algún bit de control extraído de la instrucción (en el caso de la ALU, la entrada de *c* representa los seis bits de control que le indican a la ALU qué calcular). En conjunto, el comportamiento distribuido inducido por estos bits de control termina ejecutando la instrucción. No especificamos qué bits van a dónde, ya que queremos que los lectores respondan estas preguntas por sí mismos.

**Decodificación de instrucciones:** Comencemos centrándonos en la entrada de instrucciones de la CPU. Este valor de 16 bits representa una *instrucción A* (cuando el bit más a la izquierda es 0) o una *instrucción C* (cuando el bit más a la izquierda es 1). En el caso de una *instrucción A*, los bits de instrucción se interpretan como un valor binario que debe cargarse en el registro A. En el caso de una *instrucción C*, la instrucción se trata como una cápsula de bits de control *1xxacccccddjjj*, como se indica a continuación. Los bits *a* y *cccccc* codifican la parte *comp* de la instrucción; los bits *ddd* codifican la parte *dest* de la instrucción; los bits *jjj* codifican la parte *jump* de la instrucción. Los *xx* bits se ignoran.

**Ejecución de instrucciones:** En el caso de una *instrucción A*, los 16 bits de la instrucción se cargan tal cual en el registro A (en realidad, este es un valor de 15 bits, ya que el MSB es el código de operación 0). En el caso de una *instrucción C*, el bit A determina si la entrada ALU se alimentará desde el valor del registro A o desde el valor M entrante. Los bits *cccccc* determinan qué función calculará la ALU. Los bits *ddd* determinan qué registros deben aceptar la salida ALU. Los bits *jjj* se utilizan para determinar qué instrucción buscar a continuación.

La arquitectura de la CPU debe extraer los bits de control descritos anteriormente de la entrada de instrucciones y enrutarlos a sus destinos de partes del chip, donde instruyen a las partes del chip qué hacer para participar en la ejecución de la instrucción. Tenga en cuenta que cada una de estas piezas de chip ya está diseñada para llevar a cabo su función prevista. Por lo tanto, el diseño de la CPU es principalmente una cuestión de conectar los chips existentes de una manera que realice este modelo de ejecución.

**Obtención de instrucciones:** Como efecto secundario de la ejecución de la instrucción actual, la CPU determina y genera la dirección de la instrucción que debe



se recuperará y ejecutará a continuación. El elemento clave en esta subtarea es el contador de *programas*, una parte del chip de la CPU cuya función es almacenar siempre la dirección de la siguiente instrucción.

De acuerdo con la especificación de la computadora Hack, el programa actual se almacena en la memoria de instrucciones, comenzando en la dirección 0. Por lo tanto, si deseamos iniciar (o reiniciar) la ejecución de un programa, debemos establecer el contador de programa en 0. Por eso, en [la figura 5.8](#), la entrada de reinicio de la CPU se introduce directamente en la entrada de reinicio de la parte del chip de PC. Si afirmamos este bit, haremos  $PC=0$ , que la computadora obtenga y ejecute la primera instrucción en el programa.

¿Qué debemos hacer a continuación? Normalmente, nos gustaría ejecutar la siguiente instrucción del programa. Por lo tanto, y suponiendo que la entrada de reinicio se haya vuelto a poner en 0, el funcionamiento por defecto del contador de programa es  $PC++$ .

Pero, ¿qué pasa si la instrucción actual incluye una directiva de salto? De acuerdo con la especificación del lenguaje, la ejecución siempre se ramifica a la instrucción cuya dirección es el valor actual de A. Por lo tanto, la implementación de la CPU debe realizar el siguiente comportamiento del contador de programas: si *jump then*

$PC=A$  else  $PC++$ .

¿Cómo podemos efectuar este comportamiento utilizando la lógica de la puerta? La respuesta se insinúa en [la figura 5.8](#). Tenga en cuenta que la salida del registro A alimenta la entrada del registro de PC. Por lo tanto, si afirmamos el bit de carga de la PC, habilitaremos la operación  $PC=A$  en lugar de la operación predeterminada  $PC++$ . Debemos hacer esto solo si tenemos que efectuar un salto. Lo que lleva a la siguiente pregunta: ¿Cómo sabemos si tenemos que efectuar un salto? La respuesta depende de los tres bits *j* de la instrucción actual y los dos bits de salida ALU *zr* y *ng*. En conjunto, estos bits se pueden usar para determinar si la condición de salto se cumple o no.

Nos detendremos aquí, para no robar a los lectores el placer de completar la implementación de la CPU ellos mismos. Esperamos que mientras lo hacen, saboreen la elegancia mecánica de la CPU Hack.

### 5.3.2 Memoria

El chip de memoria de la computadora Hack es un agregado de tres partes del chip:

RAM16K, pantalla y teclado. Esta modularidad, sin embargo, está implícita: Hack

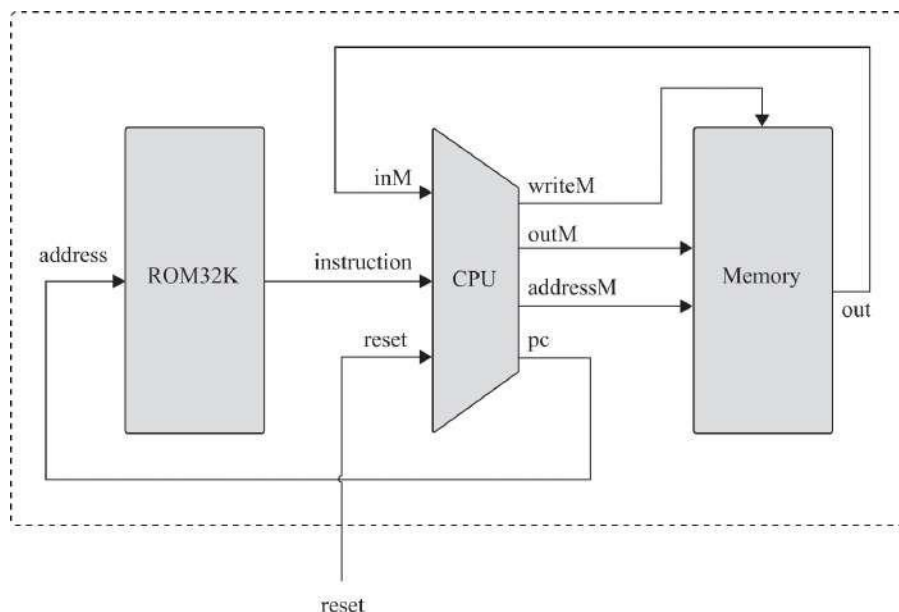


Los programas de lenguaje de máquina ven un *solo espacio de direcciones*, que va desde la dirección 0 hasta la dirección 24576 (en hexadecimal: 6000).

La interfaz del chip de memoria se muestra en [la figura 5.6](#). La implementación de esta interfaz debe realizar el efecto continuo que acabamos de describir. Por ejemplo, si la entrada de dirección del chip de memoria es 16384, la implementación debe acceder a la dirección 0 en el chip de pantalla, y así sucesivamente. Una vez más, preferimos no proporcionar demasiados detalles y dejar que usted mismo descubra el resto de la implementación.

### 5.3.3 Ordenador

Hemos llegado al final de nuestro viaje de hardware. El chip de computadora superior se puede realizar utilizando tres partes del chip: la CPU, el chip de memoria de datos y el chip de memoria de instrucciones, ROM32K. [La Figura 5.9](#) da los detalles.



**Figura 5.9** Implementación propuesta de Ordenador, el chip más alto de la plataforma Hack.

La implementación de la computadora está diseñada para realizar el siguiente ciclo de búsqueda-ejecución: Cuando el usuario afirma la entrada de reinicio, la salida de la PC de la CPU emite 0, lo que hace que la memoria de instrucciones (ROM32K) emita la primera instrucción del programa. La instrucción será ejecutada por la CPU, y esta ejecución puede implicar la lectura o escritura de un registro de memoria de datos.

En el proceso de ejecución de la instrucción, la CPU averigua qué instrucción buscar a continuación y emite esta dirección a través de la salida de su PC. La salida de la PC de la CPU alimenta la entrada de dirección de la memoria de instrucciones, lo que hace que esta última emita la instrucción que debe ejecutarse a continuación. Esta salida, a su vez, alimenta la entrada de instrucciones de la CPU, cerrando el ciclo de búsqueda-ejecución.

---

## 5.4 Proyecto

**Objetivo:** Construir el ordenador Hack, culminando en el ordenador superior chip.

**Recursos:** Todos los chips descritos en este capítulo deben escribirse en HDL y probarse en el simulador de hardware suministrado, utilizando los programas de prueba que se describen a continuación.

**Contrato:** Construir una plataforma de hardware capaz de ejecutar programas escritos en el lenguaje de máquina Hack. Demuestre las operaciones de la plataforma haciendo que su chip de computadora ejecute los tres programas de prueba suministrados.

**Programas de prueba:** Una forma natural de probar la implementación general del chip de computadora es hacer que ejecute programas de muestra escritos en el lenguaje de máquina Hack. Para ejecutar una prueba de este tipo, se puede escribir un script de prueba que cargue el chip de la computadora en el simulador de hardware, cargue un programa de un archivo de texto externo en la parte del chip ROM32K (la memoria de instrucciones) y luego ejecute el reloj suficientes ciclos para ejecutar el programa. Proporcionamos tres de estos programas de prueba, junto con los correspondientes scripts de prueba y archivos de comparación:

- **Add.hack:** Suma las dos constantes 2 y 3, y escribe el resultado en RAM[0].
- **Max.hack:** calcula el máximo de RAM[0] y RAM[1] y escribe el resultado en RAM[2].
- **Rect.hack:** Dibuja en la pantalla un rectángulo de filas RAM [0] de 16 píxeles cada una. La esquina superior izquierda del rectángulo se encuentra en la esquina superior izquierda de la pantalla.

Antes de probar su chip de computadora en cualquiera de estos programas, revise el script de prueba asociado con el programa y asegúrese de comprender las instrucciones dadas al simulador. Si es necesario, consulte el apéndice 3 ("Lenguaje de descripción de pruebas").

**Pasos:** Implemente el equipo en el siguiente orden:

**Memoria:** Este chip se puede construir a lo largo del esquema general que se da en la [figura 5.6](#), utilizando tres partes del chip: RAM16K, Pantalla y Teclado. La pantalla y el teclado están disponibles como chips incorporados; no hay necesidad de construirlos. Aunque el chip RAM16K se construyó en el proyecto 3, recomendamos usar su versión incorporada en su lugar.

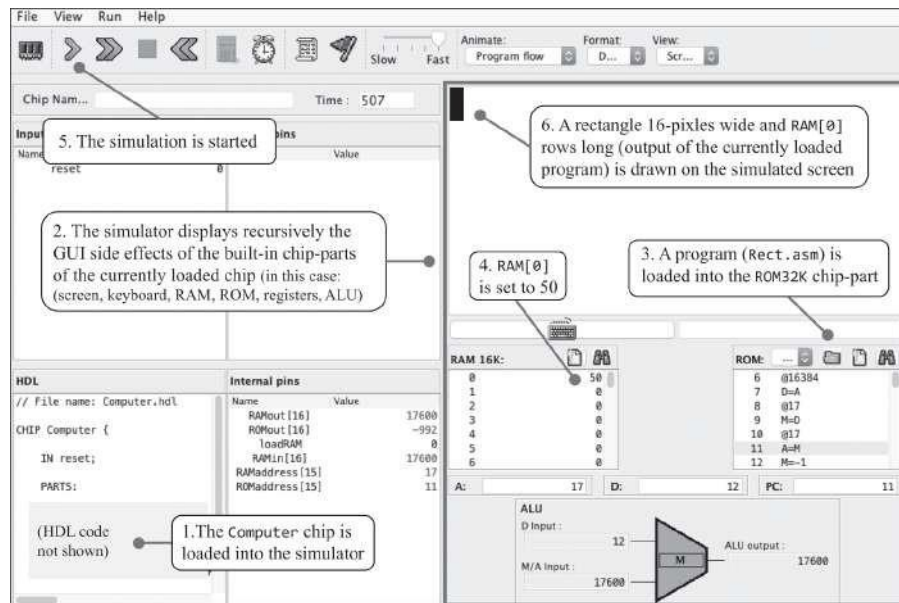
**CPU:** La unidad central de procesamiento se puede construir de acuerdo con la implementación propuesta que se muestra en la [figura 5.8](#). En principio, la CPU puede usar la ALU incorporada en el proyecto 2, los chips Register y PC integrados en el proyecto 3 y las puertas lógicas del proyecto 1, según sea necesario. Sin embargo, recomendamos utilizar las versiones integradas de todos estos chips (en particular, utilizar los registros integrados ARegister, DRegister y PC). Los chips incorporados tienen exactamente la misma funcionalidad que los chips de memoria construidos en proyectos anteriores, pero cuentan con efectos secundarios de GUI que facilitan las pruebas y la simulación de su trabajo.

En el curso de la implementación de la CPU, puede tener la tentación de especificar y construir chips internos ("auxiliares") propios. Tenga en cuenta que no es necesario hacerlo; la CPU Hack se puede implementar de manera elegante y eficiente usando solo las partes del chip que aparecen en la [figura 5.8](#), además de algunas puertas lógicas elementales construidas en el proyecto 1 (de las cuales es mejor usar sus versiones incorporadas).

**Memoria de instrucciones:** Utilice el chip ROM32K incorporado.

**Computadora:** La computadora se puede construir de acuerdo con la implementación propuesta que se da en la [figura 5.9](#).

**Simulador de hardware:** Todos los chips de este proyecto, incluido el chip de computadora superior, se pueden implementar y probar utilizando el simulador de hardware suministrado. La [Figura 5.10](#) es una captura de pantalla de la prueba del programa Rect.hack en una implementación de chip de computadora.



**Figura 5.10** Prueba del chip de computadora en el simulador de hardware suministrado. El programa almacenado es Rect, que dibuja un rectángulo de filas de RAM [0] de 16 píxeles cada una, todas negras, en la parte superior izquierda de la pantalla.

Una versión basada en la web del proyecto 5 está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

## 5.5 Perspectiva

Siguiendo el espíritu de Nand to Tetris, la arquitectura de la computadora Hack es mínima. Las plataformas informáticas típicas cuentan con más registros, más tipos de datos, ALU más potentes y conjuntos de instrucciones más ricos. La mayoría de estas diferencias, sin embargo, son cuantitativas. Desde un punto de vista cualitativo, casi todas las computadoras digitales, incluido Hack, se basan en la misma arquitectura conceptual: la máquina de von Neumann.

En términos de *propósito*, los sistemas informáticos se pueden clasificar en dos categorías: *computadoras de propósito general* y *computadoras de un solo propósito*. Las computadoras de propósito general, como PC y teléfonos celulares, generalmente interactúan con un usuario. Están diseñados para ejecutar muchos programas y cambiar fácilmente de un programa a otro. *Las computadoras de un solo propósito* generalmente están integradas en otros sistemas como automóviles, cámaras, transmisores de medios, dispositivos médicos, controladores industriales, etc. Para cualquier aplicación en particular, se graba un solo programa en la ROM de la computadora dedicada

(Memoria de solo lectura). Por ejemplo, en algunas consolas de juegos, el software del juego reside en un cartucho externo, que es un módulo ROM reemplazable encerrado en un paquete elegante. Aunque las computadoras de propósito general suelen ser más complejas y versátiles que las computadoras dedicadas, todas comparten las mismas ideas arquitectónicas básicas: programas almacenados, lógica de búsqueda-decodificación-ejecución, CPU, registros y contadores.

La mayoría de las computadoras de propósito general usan un solo espacio de direcciones para almacenar programas y datos. Otras computadoras, como Hack, usan dos espacios de direcciones separados. Esta última configuración, que por razones históricas se denomina *arquitectura Harvard*, es menos flexible en términos de utilización de memoria ad hoc, pero tiene claras ventajas. Primero, es más fácil y más barato de construir. En segundo lugar, a menudo es más rápido que la configuración de un solo espacio de direcciones. Finalmente, si se conoce de antemano el tamaño del programa que debe ejecutar la computadora, el tamaño de la memoria de instrucciones se puede optimizar y corregir en consecuencia. Por estas razones, la arquitectura de Harvard es la arquitectura elegida en muchas computadoras integradas, dedicadas y de un solo propósito.

Las computadoras que usan el mismo espacio de direcciones para almacenar instrucciones y datos enfrentan el siguiente desafío: ¿Cómo podemos alimentar la dirección de la instrucción y la dirección del registro de datos en el que debe operar la instrucción en la misma entrada de direcciones del dispositivo de memoria compartida? Claramente, no podemos hacerlo al mismo tiempo. La solución estándar es basar el funcionamiento de la computadora en una lógica de dos ciclos. Durante el *ciclo de recuperación*, la dirección de la instrucción se alimenta a la entrada de direcciones de la memoria, lo que hace que emita inmediatamente la instrucción actual, que luego se almacena en un registro de *instrucciones*. En el ciclo de *ejecución posterior*, la instrucción se decodifica y la dirección de datos en la que tiene que operar se alimenta a la misma entrada de dirección. Por el contrario, las computadoras que usan memorias de datos e instrucciones separadas, como Hack, se benefician de una lógica de búsqueda y ejecución de un solo ciclo, que es más rápida y fácil de manejar. El precio es tener que usar unidades de memoria de instrucciones y datos separadas, aunque no es necesario usar un registro de instrucciones.

La computadora Hack interactúa con una pantalla y un teclado. Las computadoras de uso general generalmente están conectadas a múltiples dispositivos de E / S como impresoras, dispositivos de almacenamiento,

conexiones de red, etc. Además, los dispositivos de visualización típicos son mucho más elegantes que la pantalla Hack, con más píxeles, más colores y un rendimiento de renderizado más rápido. Aún así, lo básico

Se mantiene el principio de que cada píxel está impulsado por un valor binario residente en la memoria: en lugar de un solo bit que controla el color blanco o negro del píxel, normalmente se dedican 8 bits a controlar el nivel de brillo de cada uno de los varios colores primarios que, en conjunto, producen el color final del píxel. El resultado son millones de colores posibles, más de los que el ojo humano puede discernir.

El mapeo de la pantalla Hack en la memoria principal de la computadora es simplista. En lugar de tener bits de memoria que impulsan píxeles directamente, muchas computadoras permiten que la CPU envíe instrucciones gráficas de alto nivel como "dibujar una línea" o "dibujar un círculo" a un chip gráfico dedicado o una unidad de procesamiento de gráficos independiente, también conocida como GPU. El hardware y el software de bajo nivel de estos procesadores gráficos dedicados están especialmente optimizados para renderizar gráficos, animaciones y videos, descargando de la CPU y la computadora principal la carga de manejar estas tareas voraces directamente.

Finalmente, cabe destacar que gran parte del esfuerzo y la creatividad en el diseño de hardware informático se invierte en lograr un mejor rendimiento. Muchos arquitectos de hardware dedican su trabajo a acelerar el acceso a la memoria, utilizando algoritmos inteligentes de almacenamiento en caché y estructuras de datos, optimizando el acceso a dispositivos de E/S y aplicando canalización, paralelismo, precarga de instrucciones y otras técnicas de optimización que se eludieron por completo en este capítulo.

Históricamente, los intentos de acelerar el rendimiento del procesamiento han llevado a dos campos principales de diseño de CPU. Los defensores del enfoque de *la computación de conjuntos de instrucciones complejas* (CISC) argumentaron a favor de lograr un mejor rendimiento mediante la construcción de procesadores más potentes con conjuntos de instrucciones más elaborados. Por el contrario, el campo de la *Computación de Conjunto de Instrucciones Reducido* (RISC) construyó procesadores más simples y conjuntos de instrucciones más ajustados, argumentando que estos en realidad ofrecen un rendimiento más rápido en las pruebas de referencia. La computadora Hack no entra en este debate, ya que no presenta un conjunto de instrucciones sólido ni técnicas especiales de aceleración de hardware.

---

## 6 Ensamblador

¿Qué hay en un nombre? Lo que llamamos rosa con cualquier otro nombre olería igual de dulce.

—Shakespeare, *Romeo y Julieta*

En los capítulos anteriores, completamos el desarrollo de una plataforma de hardware diseñada para ejecutar programas en el lenguaje de máquina Hack. Presentamos dos versiones de este lenguaje, simbólica y binaria, y explicamos que los programas simbólicos se pueden traducir a código binario utilizando un programa llamado *ensamblador*. En este capítulo describimos cómo funcionan los ensambladores y cómo se construyen. Esto conducirá a la construcción de un *ensamblador Hack*, un programa que traduce programas escritos en el lenguaje simbólico Hack en código binario que puede ejecutarse en el hardware Hack básico.

Dado que la relación entre las instrucciones simbólicas y sus correspondientes códigos binarios es sencilla, implementar un ensamblador utilizando un lenguaje de programación de alto nivel no es una tarea difícil. Una complicación surge al permitir que los programas ensambladores usen referencias simbólicas a direcciones de memoria. Se espera que el ensamblador administre estos símbolos y los resuelva en direcciones de memoria física. Esta tarea normalmente se realiza mediante una *tabla de símbolos*, una estructura de datos de uso común.

La implementación del ensamblador es el primero de una serie de siete proyectos de desarrollo de software que acompañan a la parte II del libro. El desarrollo del ensamblador lo equipará con un conjunto básico de habilidades generales que le servirán bien a lo largo de todos estos proyectos y más allá: manejo de argumentos de línea de comandos, manejo de archivos de texto de entrada y salida, análisis de instrucciones, manejo de espacios en blanco, manejo de símbolos, generación de código y muchas otras técnicas que entran en juego en muchos proyectos de desarrollo de software.



Si no tiene experiencia en programación, puede desarrollar un ensamblador basado en papel. Esta opción se describe en la versión basada en web del proyecto 6, disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

---

## 6.1 Fondo

Los lenguajes de máquina generalmente se especifican en dos tipos: *binario* y *simbólico*. Un binario instrucción para ejemplo 1100001000000011000000000000111, es un paquete acordado de microcódigos diseñado para ser decodificado y ejecutado por alguna plataforma de hardware de destino. Por ejemplo, los 8 bits más a la izquierda de la instrucción, 11000010, puede representar una operación como "load". Los siguientes 8 bits, 00000011, puede representar un registro, por ejemplo, R3. Los 16 bits restantes, 00000000000000111, puede representar un valor, digamos 7. Cuando nos proponemos construir una arquitectura de hardware y un lenguaje de máquina, podemos decidir que esta instrucción particular de 32 bits hará que el hardware efectúe la operación "cargar la constante 7 en el registro R3." Las plataformas informáticas modernas admiten cientos de estas posibles operaciones. Por lo tanto, los lenguajes de máquina pueden ser complejos e involucrar muchos códigos de operación, modos de direccionamiento de memoria y formatos de instrucciones.

Claramente, especificar estas operaciones en código binario es una molestia. Una solución natural es usar una sintaxis simbólica equivalente acordada, digamos, "cargar R3,7". El código de operación de carga a veces se denomina *mnemotécnico*, que en latín significa un patrón de letras diseñado para ayudar a recordar algo. Dado que la traducción de mnemotécnicos y símbolos a código binario es sencilla, tiene sentido escribir programas de bajo nivel directamente en notación simbólica y hacer que un programa de computadora los traduzca a código binario. El lenguaje simbólico se llama *ensamblador* y el programa traductor *ensamblador*. El ensamblador analiza cada instrucción de ensamblado en sus campos subyacentes, por ejemplo, load, R3 y 7, traduce cada campo en su código binario equivalente y, finalmente, ensambla los bits generados en una instrucción binaria que puede ser ejecutada por el hardware. De ahí el nombre *ensamblador*.

**Símbolos:** Considere la instrucción simbólica goto 312. Después de la traducción, esta instrucción hace que la computadora obtenga y ejecute la instrucción

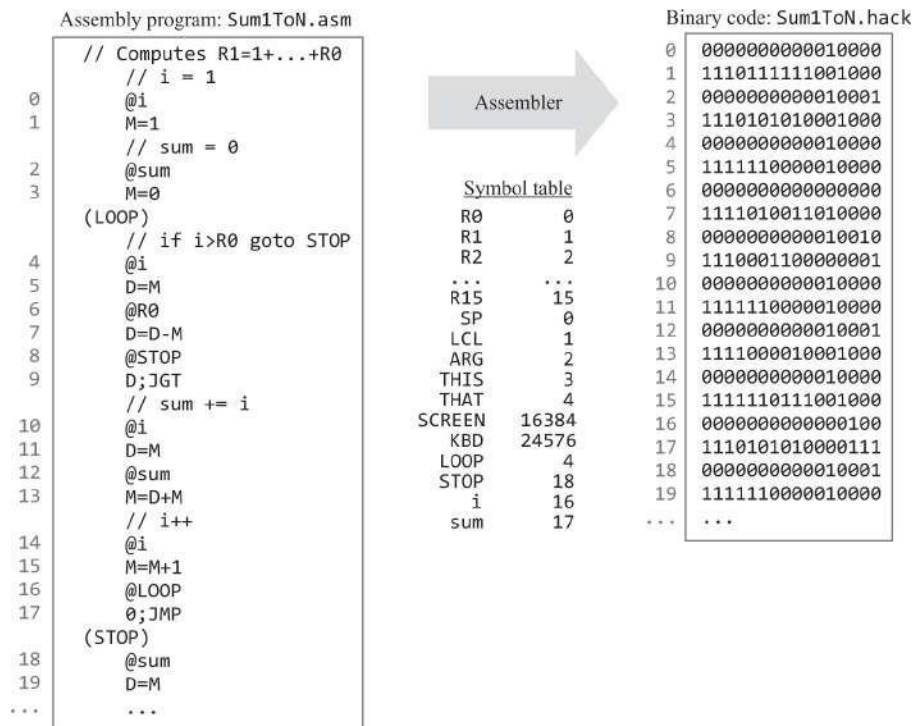
almacenado en la dirección 312, que puede ser el comienzo de algún bucle. Bueno, si es el comienzo de un bucle, ¿por qué no marcar este punto en el programa ensamblador con una etiqueta descriptiva, digamos LOOP, y usar el comando goto LOOP en lugar de goto 312? Todo lo que tenemos que hacer es registrar en algún lugar que LOOP significa 312. Cuando traducimos el programa a código binario, reemplazamos cada aparición de LOOP con 312. Ese es un pequeño precio a pagar por la ganancia en legibilidad y portabilidad del programa.

En general, los lenguajes ensambladores usan símbolos para tres propósitos:

- *Etiquetas*: los programas de ensamblado pueden declarar y usar símbolos que marcan varias ubicaciones en el código, por ejemplo, LOOP y END.
- *Variables*: Los programas ensambladores pueden declarar y usar variables simbólicas, por ejemplo, i y sum.
- *Símbolos predefinidos*: Los programas de ensamblaje pueden hacer referencia a direcciones especiales en la memoria de la computadora utilizando símbolos acordados, por ejemplo, SCREEN y KBD.

Por supuesto, no hay almuerzo gratis. Alguien debe ser responsable de administrar todos estos símbolos. En particular, alguien debe recordar que SCREEN significa 16384, que LOOP significa 312, que sum representa alguna otra dirección, y así sucesivamente. Esta tarea de manejo de símbolos es una de las funciones más importantes del ensamblador.

**Ejemplo:** La Figura 6.1 enumera dos versiones del mismo programa escritas en el lenguaje de máquina Hack. La versión simbólica incluye todo tipo de cosas que a los humanos les gusta ver en los programas de computadora: comentarios, espacios en blanco, sangría, instrucciones simbólicas y referencias simbólicas. Ninguno de estos adornos concierne a las computadoras, que solo entienden una cosa: bits. El agente que cierra la brecha entre el código simbólico conveniente para los humanos y el código binario entendido por la computadora es el ensamblador.



**Figura 6.1** Código ensamblador, traducido a código binario mediante una tabla de símbolos. Los números de línea, que no forman parte del código, se enumeran como referencia.

Ignoremos por ahora todos los detalles de la [figura 6.1](#), así como la tabla de símbolos, y hagamos algunas observaciones generales. Primero, tenga en cuenta que, aunque los números de línea no son parte del código, juegan un papel importante, aunque implícito, en el proceso de traducción. Si el código binario se cargará en la memoria de instrucciones a partir de la dirección 0, entonces el número de línea de cada instrucción coincidirá con su dirección de memoria. Claramente, esta observación debería ser de interés para el ensamblador. En segundo lugar, tenga en cuenta que los comentarios y las declaraciones de etiquetas no generan código, y es por eso que estas últimas a veces se *denominan pseudoinstrucciones*. Finalmente, y afirmando lo obvio, tenga en cuenta que para escribir un ensamblador para algún lenguaje de máquina, el desarrollador del ensamblador debe obtener una especificación completa de la sintaxis simbólica y binaria del lenguaje.

Con eso en mente, ahora pasamos a especificar el lenguaje de máquina Hack.

## 6.2 La especificación del lenguaje de máquina Hack

El lenguaje ensamblador Hack y su representación binaria equivalente se describieron en el capítulo 4. La especificación del lenguaje se repite aquí para facilitar la referencia. Esta especificación es el contrato que los ensambladores de Hack deben implementar, de una forma u otra.

### 6.2.1 Programas

**Programa de hackeo binario:** Un programa de hackeo binario es una secuencia de líneas de texto, cada una de las cuales consta de dieciséis caracteres 0 y 1. Si la línea comienza con un 0, representa una instrucción A binaria. De lo contrario, representa una instrucción C- binaria.

**Programa de ensamblaje Hack:** Un programa de ensamblaje Hack es una secuencia de líneas de texto, cada una de las cuales es una instrucción de *ensamblaje*, una *declaración de etiqueta* o un *comentario*:

- *Instrucciones de montaje* : Un simbólico  
Un-instrucción o un simbólico C-  
instrucción (ver [Figura 6.2](#)).
- *Declaración de etiqueta*: una línea con el formato (xxx), donde xxx es un símbolo.
- *Comentario*: Una línea que comienza con dos barras diagonales (//) se considera un comentario y se ignora.

|                                |                                           |                                                                                                   |                                |
|--------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------|
| (symbolic): @xxx               |                                           | (xxx is a decimal value ranging from 0 to 32767, or a symbol bound to such a decimal value)       |                                |
| A-instruction                  | (binary): 0 v v v v v v v v v v v v v v v | (v v... v = 15-bit value of xxx)                                                                  |                                |
| (symbolic): dest = comp ; jump |                                           | (comp is mandatory.<br>If dest is empty, the = is omitted;<br>If jump is empty, the ; is omitted) |                                |
| C-instruction                  | (binary): 111aeeeeeedddjjj                |                                                                                                   |                                |
| comp                           | c c c c c c                               | dest                                                                                              | d d d Effect: store comp in:   |
| 0                              | 1 0 1 0 1 0                               | null                                                                                              | 0 0 0 the value is not stored  |
| 1                              | 1 1 1 1 1 1                               | M                                                                                                 | 0 0 1 RAM[A]                   |
| -1                             | 1 1 1 0 1 0                               | D                                                                                                 | 0 1 0 D register (reg)         |
| D                              | 0 0 1 1 0 0                               | DM                                                                                                | 0 1 1 D reg and RAM[A]         |
| A                              | 1 1 0 0 0 0                               | A                                                                                                 | 1 0 0 A reg                    |
| !D                             | 0 0 1 1 0 1                               | AM                                                                                                | 1 0 1 A reg and RAM[A]         |
| !A                             | !M 1 1 0 0 0 1                            | AD                                                                                                | 1 1 0 A reg and D reg          |
| -D                             | 0 0 1 1 1 1                               | ADM                                                                                               | 1 1 1 A reg, D reg, and RAM[A] |
| -A                             | -M 1 1 0 0 1 1                            | jump                                                                                              | j j j Effect:                  |
| D+1                            | 0 1 1 1 1 1                               | null                                                                                              | 0 0 0 no jump                  |
| A+1                            | M+1 1 1 0 1 1 1                           | JGT                                                                                               | 0 0 1 if comp > 0 jump         |
| D-1                            | 0 0 1 1 1 0                               | JEQ                                                                                               | 0 1 0 if comp = 0 jump         |
| A-1                            | M-1 1 1 0 0 1 0                           | JGE                                                                                               | 0 1 1 if comp ≥ 0 jump         |
| D+A                            | D+M 0 0 0 0 1 0                           | JLT                                                                                               | 1 0 0 if comp < 0 jump         |
| D-A                            | D-M 0 1 0 0 1 1                           | JNE                                                                                               | 1 0 1 if comp ≠ 0 jump         |
| A-D                            | M-D 0 0 0 1 1 1                           | JLE                                                                                               | 1 1 0 if comp ≤ 0 jump         |
| D&A                            | D&M 0 0 0 0 0 0                           | JMP                                                                                               | 1 1 1 unconditional jump       |
| D A                            | D M 0 1 0 1 0 1                           |                                                                                                   |                                |

a == 0 a == 1

**Figura 6.2** El conjunto de instrucciones Hack, que muestra tanto la mnemotecnia simbólica como sus correspondientes códigos binarios.

## 6.2.2 Símbolos

Los símbolos en los programas de ensamblaje de Hack se dividen en tres categorías: símbolos predefinidos, símbolos de etiqueta y símbolos variables.

**Símbolos predefinidos:** Cualquier programa de ensamblaje de Hack puede usar símbolos predefinidos, como se indica a continuación. R0, R1, ..., R15 significan 0, 1, ... 15, respectivamente. SP, LCL, ARG, THIS, THAT significan 0, 1, 2, 3, 4, respectivamente. SCREEN y KBD significan 16384 y 24576, respectivamente. Los valores de estos símbolos se interpretan como direcciones en la RAM de Hack.

**Símbolos de etiqueta:** La pseudo-instrucción (xxx) define el símbolo xxx para referirse a la ubicación en la Hack ROM que contiene la siguiente instrucción en el programa. Un símbolo de etiqueta se puede definir una vez y se puede utilizar en cualquier parte del programa de ensamblaje, incluso antes de la línea en la que se define.

**Símbolos de variable:** cualquier símbolo *xxx* que aparezca en un programa de ensamblaje que no esté predefinido y no esté definido en otra parte por una declaración de etiqueta (*xxx*) se trata como una variable. Las variables se asignan a ubicaciones de RAM consecutivas a medida que se encuentran por primera vez, comenzando en la dirección de RAM 16. Por lo tanto, la primera variable encontrada en un programa se asigna a RAM[16], la segunda a RAM[17], y así sucesivamente.

### 6.2.3 Convenciones de sintaxis

**Símbolos:** Un *símbolo* puede ser cualquier secuencia de letras, dígitos, guiones bajos (*\_*), punto (*.*), el signo de dólar (*\$*) y los dos puntos (*:*) que no comienzan con un dígito.

**Constantes:** Puede aparecer solo en las instrucciones A de la forma *@xxx*. La constante *xxx* es un valor en el rango de 0 a 32767 y se escribe en notación decimal.

**Espacio en blanco:** se ignoran los caracteres del espacio inicial y las líneas vacías.

**Convenciones de mayúsculas y minúsculas:** Todos los mnemotécnicos de ensamblaje (como *A+1*, *JEQ*, etc.) deben escribirse en mayúsculas. Los símbolos restantes (etiquetas y nombres de variables) distinguen entre mayúsculas y minúsculas. La convención recomendada es usar mayúsculas para las etiquetas y minúsculas para las variables.

Esto completa la especificación del lenguaje de máquina Hack.

---

## 6.3 Traducción de ensamblado a binario

Esta sección describe cómo traducir programas de ensamblado Hack en código binario. Aunque nos enfocamos en desarrollar un ensamblador para el lenguaje Hack, las técnicas que presentamos son aplicables a cualquier ensamblador.

El ensamblador toma como entrada un flujo de instrucciones de ensamblado y genera como salida un flujo de instrucciones binarias traducidas. El código resultante se puede cargar tal cual en la memoria de la computadora y ejecutarse. Para llevar a cabo el proceso de traducción, el ensamblador debe manejar instrucciones y símbolos.

### 6.3.1 Instrucciones de manejo

Para cada instrucción de ensamblado, el ensamblador

- analiza la instrucción en sus campos subyacentes;
- para cada campo, genera el código de bits correspondiente, como se especifica en la [figura 6.2](#);
- si la instrucción contiene una referencia simbólica, resuelve el símbolo en su valor numérico;
- ensambla los códigos binarios resultantes en una cadena de dieciséis 0 y 1 Caracteres; y
- escribe la cadena ensamblada en el archivo de salida.

### 6.3.2 Manejo de símbolos

Los programas de ensamblaje pueden usar etiquetas simbólicas (destinos de instrucciones goto) antes de definir los símbolos. Esta convención facilita la vida de los escritores de código ensamblador y la de los desarrolladores ensambladores más difícil. Una solución común es desarrollar un *ensamblador de dos pasos* que lea el código dos veces, de principio a fin. En el primer paso, el ensamblador crea una *tabla de símbolos*, agrega todos los símbolos de etiqueta a la tabla y no genera ningún código. En el segundo paso, el ensamblador maneja los símbolos variables y genera código binario, utilizando la tabla de símbolos. Aquí están los detalles.

**Inicialización:** el ensamblador crea una tabla de símbolos y la inicializa con todos los símbolos predefinidos y sus valores preasignados. En [la figura 6.1](#), el resultado de la etapa de inicialización es la tabla de símbolos con todos los símbolos hasta KBD inclusive.

**Primera pasada:** El ensamblador recorre todo el programa de ensamblaje, línea por línea, realizando un seguimiento del número de línea. Este número comienza en 0 y se incrementa en 1 cada vez que *se encuentra una instrucción A* o una *instrucción C*, pero no cambia cuando se encuentra un comentario o una declaración de etiqueta. Cada vez que se encuentra una declaración de etiqueta (xxx), el ensamblador agrega una nueva entrada a la tabla de símbolos, asociando el símbolo xxx



con el número de línea actual más 1 (esta será la dirección ROM de la siguiente instrucción en el programa).

Este paso da como resultado la adición a la tabla de símbolos de todos los símbolos de etiqueta del programa, junto con sus valores correspondientes. En [la figura 6.1](#), la primera pasada da como resultado la adición de los símbolos LOOP y STOP a la tabla de símbolos. No se genera ningún código durante la primera pasada.

**Segunda pasada:** El ensamblador vuelve a recorrer todo el programa y analiza cada línea de la siguiente manera. Cada vez que se encuentra una instrucción A con una referencia simbólica, a saber, @xxx, donde xxx es un símbolo y no un número, el ensamblador busca xxx en la tabla de símbolos. Si se encuentra el símbolo, el ensamblador lo reemplaza con su valor numérico y completa la traducción de la instrucción. Si no se encuentra el símbolo, debe representar una nueva variable. Para manejarlo, el ensamblador (i) agrega la entrada <xxx, value> a la tabla de símbolos, donde value es la siguiente dirección disponible en el espacio RAM designado para variables, y (ii) completa la traducción de la instrucción, usando esta dirección. En la plataforma Hack, el espacio de RAM designado para almacenar variables comienza en 16 y se incrementa en 1 después de cada vez que se encuentra una nueva variable en el código. En [la figura 6.1](#), la segunda pasada da como resultado la adición de los símbolos i y sum a la tabla de símbolos.

---

## 6.4 Implementación

**Uso:** El ensamblador Hack acepta un único argumento de línea de comandos, como se indica a continuación:

```
prompt> HackAssembler Prog.asm
```

donde el archivo de entrada *Prog.asm* contiene instrucciones de ensamblaje (la extensión .asm es obligatoria). El nombre del archivo puede contener una ruta de archivo. Si no se especifica ninguna ruta de acceso, el ensamblador opera en la carpeta actual. El ensamblador crea un archivo de salida llamado *Prog.hack* y escribe las instrucciones binarias traducidas en él. El archivo de salida se crea en la misma carpeta que el archivo de entrada. Si hay un archivo con este nombre en la carpeta, se sobrescribirá.



Proponemos dividir la implementación del ensamblador en dos etapas. En la primera etapa, desarrolle un ensamblador básico para programas Hack que no contengan referencias simbólicas. En la segunda fase, extienda el ensamblador básico para controlar las referencias simbólicas.

### 6.4.1 Desarrollo de un ensamblador básico

El ensamblador básico supone que el código fuente no contiene referencias simbólicas. Por lo tanto, excepto para manejar comentarios y espacios en blanco, el ensamblador tiene que traducir *instrucciones C* o *instrucciones A* de la forma @xxx, donde xxx es un valor decimal (y no un símbolo). Esta tarea de traducción es sencilla: cada componente mnemotécnico (campo) de una instrucción C simbólica se traduce a su código de bits correspondiente, de acuerdo con la [figura 6.2](#), y cada constante decimal xxx en una instrucción A simbólica se traduce a su código binario equivalente.

Proponemos basar el ensamblador en una arquitectura de software que consiste en un *módulo Parser* para analizar la entrada en instrucciones y las instrucciones en campos, un *módulo Code* para traducir los campos (mnemotecnica simbólica) en códigos binarios y un programa *ensamblador Hack* que impulsa todo el proceso de traducción. Antes de proceder a especificar los tres módulos, deseamos hacer una nota sobre el estilo que usamos para describir estas especificaciones.

**Documentación** de la API: El desarrollo del ensamblador Hack es el primero de una serie de siete proyectos de construcción de software que siguen en la parte II del libro. Cada uno de estos proyectos se puede desarrollar de forma independiente, utilizando cualquier lenguaje de programación de alto nivel. Por lo tanto, nuestro estilo de documentación de API no hace suposiciones sobre el lenguaje de implementación.

En cada proyecto, comenzando con este, proponemos una API que consta de varios *módulos*. Cada módulo documenta una o más *rutinas*. En un lenguaje típico orientado a objetos, un módulo corresponde a una *clase* y una rutina corresponde a un *método*. En otros lenguajes, un módulo puede corresponder a un *archivo* y una rutina a una *función*. Cualquiera que sea el lenguaje que utilice para implementar los proyectos de software, comenzando con el ensamblador, no debería haber problemas para mapear los *módulos* y *rutinas* de nuestras API propuestas en los elementos de programación de su lenguaje de implementación.

## El analizador

El analizador encapsula el acceso al código ensamblador de entrada. En particular, proporciona un medio conveniente para avanzar a través del código fuente, omitir comentarios y espacios en blanco, y dividir cada instrucción simbólica en sus componentes subyacentes.

Aunque la versión básica del ensamblador no es necesaria para manejar referencias simbólicas, el analizador que especificamos a continuación sí lo hace. En otras palabras, el analizador especificado aquí sirve tanto al ensamblador básico como al ensamblador completo.

El analizador ignora los comentarios y los espacios en blanco en el flujo de entrada, permite acceder a la entrada una línea a la vez y analiza las instrucciones simbólicas en sus componentes subyacentes.

La API del analizador se muestra en la página siguiente. Estos son algunos ejemplos de cómo se pueden usar los servicios de analizador. Si la instrucción actual es @17 o @sum, una llamada a symbol() devolvería la cadena "17" o "suma", respectivamente. Si la instrucción actual es (LOOP), una llamada a symbol() devolvería la cadena "LOOP". Si la instrucción actual es D=D+1; JLE, una llamada a dest (), comp () y jump () devolverían las cadenas "D", "D+1", y "JLE", respectivamente.

En el proyecto 6 tienes que implementar esta API utilizando algún lenguaje de programación de alto nivel. Para ello, debe estar familiarizado con la forma en que este lenguaje maneja los archivos de texto y las cadenas.

| <i>Routine</i>            | <i>Arguments</i>    | <i>Returns</i>                                          | <i>Function</i>                                                                                                                                                                                                                                     |
|---------------------------|---------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Constructor / initializer | Input file / stream | —                                                       | Opens the input file / stream and gets ready to parse it.                                                                                                                                                                                           |
| hasMoreLines              | —                   | boolean                                                 | Are there more lines in the input?                                                                                                                                                                                                                  |
| advance                   | —                   | —                                                       | Skips over white space and comments, if necessary.<br>Reads the next instruction from the input, and makes it the current instruction.<br>This routine should be called only if hasMoreLines is true.<br>Initially there is no current instruction. |
| instructionType           | —                   | A_INSTRUCTION, C_INSTRUCTION, L_INSTRUCTION (constants) | Returns the type of the current instruction:<br>A_INSTRUCTION for @xxx, where xxx is either a decimal number or a symbol.<br>C_INSTRUCTION for <i>dest=comp;jump</i><br>L_INSTRUCTION for (xxx), where xxx is a symbol.                             |
| symbol                    | —                   | string                                                  | If the current instruction is (xxx), returns the symbol xxx. If the current instruction is @xxx, returns the symbol or decimal xxx (as a string).<br>Should be called only if instructionType is A_INSTRUCTION or L_INSTRUCTION.                    |
| dest                      | —                   | string                                                  | Returns the symbolic <i>dest</i> part of the current C-instruction (8 possibilities).<br>Should be called only if instructionType is C_INSTRUCTION.                                                                                                 |
| comp                      | —                   | string                                                  | Returns the symbolic <i>comp</i> part of the current C-instruction (28 possibilities).<br>Should be called only if instructionType is C_INSTRUCTION.                                                                                                |
| jump                      | —                   | string                                                  | Returns the symbolic <i>jump</i> part of the current C-instruction (8 possibilities).<br>Should be called only if instructionType is C_INSTRUCTION.                                                                                                 |

## El módulo de código

Este módulo proporciona servicios para traducir mnemotécnicos simbólicos de Hack a sus códigos binarios. Específicamente, traduce la mnemotecnia simbólica de Hack

en sus códigos binarios de acuerdo con las especificaciones del lenguaje (ver [Figura 6.2](#)). Aquí está la API:

| <i>Routine</i> | <i>Arguments</i> | <i>Returns</i>      | <i>Function</i>                                      |
|----------------|------------------|---------------------|------------------------------------------------------|
| dest           | string           | 3 bits, as a string | Returns the binary code of the <i>dest</i> mnemonic. |
| comp           | string           | 7 bits, as a string | Returns the binary code of the <i>comp</i> mnemonic. |
| jump           | string           | 3 bits, as a string | Returns the binary code of the <i>jump</i> mnemonic. |

Todos los códigos de  $n$  bits se devuelven como cadenas de caracteres '0' y '1'. Por ejemplo, una llamada a `dest` ("DM") devuelve la cadena "011", una llamada a `comp` ("A+1") devuelve la cadena "0110111", una llamada a `comp` ("M+1") devuelve la cadena "1110111", una llamada a `jump` ("JNE") devuelve la cadena "101", y así sucesivamente. Todas estas asignaciones mnemotécnicas-binarias se especifican en la [figura 6.2](#).

## El ensamblador de hacks

Este es el programa principal que impulsa todo el proceso de ensamblaje, utilizando los servicios de los módulos Parser y Code. La versión básica del ensamblador (que describimos ahora) asume que el código fuente del ensamblador no contiene referencias simbólicas. Esto significa que (i) en todas las instrucciones de tipo `@xxx`, las constantes `xxx` son números decimales y no símbolos y (ii) el archivo de entrada no contiene instrucciones de etiqueta, es decir, no hay instrucciones de la forma `(xxx)`.

El programa ensamblador básico ahora se puede describir de la siguiente manera. El programa obtiene el nombre del archivo fuente de entrada, por ejemplo, *Prog*, del argumento de la línea de comandos. Construye un analizador para analizar el archivo de entrada *Prog.asm* y crea un archivo de salida, *Prog.hack*, en el que escribirá las instrucciones binarias traducidas. A continuación, el programa entra en un bucle que itera a través de las líneas (instrucciones de montaje) en el archivo de entrada y las procesa de la siguiente manera.

Para cada *instrucción C*, el programa utiliza los servicios Parser y Code para analizar la instrucción en sus campos y traducir cada campo a su código binario correspondiente. A continuación, el programa ensambla (concatena) el

códigos binarios traducidos en una cadena que consta de dieciséis '0' y '1' y escribe esta cadena como la siguiente línea en el archivo `.hack` de salida.

Para cada instrucción `A` de tipo `@xxx`, el programa traduce `xxx` en su representación binaria, crea una cadena de dieciséis caracteres '0' y '1' y la escribe como la siguiente línea en el archivo `.hack` de salida.

No proporcionamos ninguna API para este módulo, invitándolo a implementarlo como mejor le parezca.

## 6.4.2 Completar el ensamblador La tabla de símbolos

Dado que las instrucciones de Hack pueden contener referencias simbólicas, el proceso de ensamblaje debe resolverlas en direcciones reales. El ensamblador se ocupa de esta tarea utilizando una *tabla de símbolos*, diseñada para crear y mantener la correspondencia entre los símbolos y su significado (en el caso de Hack, direcciones RAM y ROM).

Un medio natural para representar esta asignación <símbolo, dirección> es cualquier estructura de datos diseñada para manejar pares <clave, valor>. Todos los lenguajes de programación modernos de alto nivel cuentan con una abstracción lista para usar, generalmente llamada *tabla hash*, *mapa*, *diccionario*, entre otros nombres. Puede implementar la tabla de símbolos desde cero o personalizar una de estas estructuras de datos. Aquí está la API de `SymbolTable`:

| <i>Routine</i>            | <i>Arguments</i>                                             | <i>Returns</i> | <i>Function</i>                                                   |
|---------------------------|--------------------------------------------------------------|----------------|-------------------------------------------------------------------|
| Constructor / initializer | —                                                            | —              | Creates a new empty symbol table.                                 |
| <code>addEntry</code>     | <code>symbol (string)</code> ,<br><code>address (int)</code> | —              | Adds < <code>symbol</code> , <code>address</code> > to the table. |
| <code>contains</code>     | <code>symbol (string)</code>                                 | boolean        | Does the symbol table contain the given <code>symbol</code> ?     |
| <code>getAddress</code>   | <code>symbol (string)</code>                                 | int            | Returns the address associated with the <code>symbol</code> .     |

## 6.5 Proyecto

**Objetivo:** Desarrollar un ensamblador que traduzca programas escritos en lenguaje ensamblador Hack en código binario Hack.

Esta versión del ensamblador supone que el código de ensamblado de origen está libre de errores. La comprobación de errores, la generación de informes y el control se pueden agregar a versiones posteriores del ensamblador, pero no forman parte del proyecto 6.

**Recursos:** La principal herramienta que necesitas para completar este proyecto es el lenguaje de programación en el que implementarás tu ensamblador. El ensamblador y el emulador de CPU suministrados en `nand2tetris/tools` también pueden ser útiles. Estas herramientas permiten experimentar con un ensamblador que funcione antes de comenzar a construir uno usted mismo. Es importante destacar que el ensamblador suministrado permite comparar su salida con las salidas generadas por su ensamblador. Para obtener más información sobre estas capacidades, consulte el tutorial del ensamblador en [www.nand2tetris.org](http://www.nand2tetris.org).

**Contrato:** Cuando se le da a su ensamblador como un argumento de línea de comandos, un *archivo Prog.asm* que contiene un programa válido de lenguaje ensamblador Hack debe traducirse al código binario Hack correcto y almacenarse en un archivo llamado *Prog.hack*, ubicado en la misma carpeta que el archivo fuente (si existe un archivo con este nombre, se sobrescribe). La salida producida por su ensamblador debe ser idéntica a la salida producida por el ensamblador suministrado.

**Plan de desarrollo:** Sugerimos construir y probar el ensamblador en dos etapas. Primero, escriba un ensamblador básico diseñado para traducir programas que no contengan referencias simbólicas. A continuación, amplíe el ensamblador con capacidades de manejo de símbolos.

**Programas de prueba:** El primer programa de prueba no tiene referencias simbólicas. Los programas de prueba restantes vienen en dos versiones: *Prog.asm* y *ProgL.asm*, que son con y sin referencias simbólicas, respectivamente.

**Add.asm:** Suma las constantes 2 y 3 y pone el resultado en R0.

**Max.asm:** calcula  $\max(R0, R1)$  y pone el resultado en R2.

**Rect.asm:** dibuja un rectángulo en la esquina superior izquierda de la

pantalla. El rectángulo tiene 16 píxeles de ancho y  $R0$  píxeles de alto. Antes de ejecutar este programa,

ponga un valor no negativo en R0.

**Pong.asm:** Un juego arcade clásico para un solo jugador. Una pelota rebota repetidamente en los bordes de la pantalla. El jugador intenta golpear la pelota con una paleta, moviendo la paleta presionando las teclas de flecha izquierda y derecha. Por cada golpe exitoso, el jugador gana un punto y la paleta se encoge un poco para hacer el juego más difícil. Si el jugador falla la pelota, el juego termina. Para salir del juego, presiona ESC.

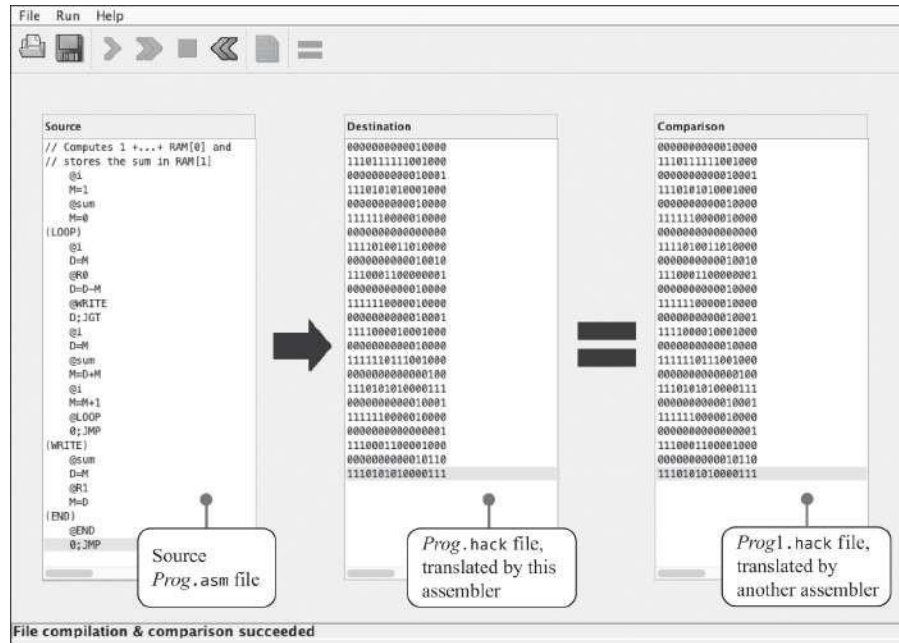
El programa Pong suministrado se desarrolló utilizando herramientas que se presentarán en la parte II del libro. En particular, el software del juego se escribió en el lenguaje de programación Jack de alto nivel y se tradujo al Pong.asm del *compilador Jack*. Aunque el Pong.jack de alto nivel

El programa es solo unas trescientas líneas de código, la aplicación ejecutable Pong tiene aproximadamente veinte mil líneas de código binario, la mayoría de las cuales es el sistema operativo Jack. Ejecutar este programa interactivo en el emulador de CPU suministrado es un asunto lento, así que no esperes un juego de Pong de alta potencia. Esta lentitud es en realidad una virtud, ya que permite rastrear el comportamiento gráfico del programa. A medida que desarrolles la jerarquía de software en la parte II, este juego se ejecutará mucho más rápido.

**Testing:** Sea *Prog.asm* un programa de ensamblaje Hack, por ejemplo, uno de los programas de prueba dados. Básicamente, hay dos formas de probar si su ensamblador traduce *Prog.asm* correctamente. Primero, puede cargar el archivo *Prog.hack* generado por su ensamblador en el emulador de CPU suministrado, ejecutarlo y verificar que esté haciendo lo que se supone que debe hacer.

La segunda técnica de prueba consiste en comparar el código generado por el ensamblador con el código generado por el ensamblador proporcionado. Para empezar, cambie el nombre del archivo generado por su ensamblador a *Prog1.hack*. A continuación, cargue *Prog.asm* en el ensamblador suministrado y tradúzcalo. Si su ensamblador funciona correctamente, se deduce que *Prog1.hack* debe ser idéntico al archivo *Prog.hack* producido por el ensamblador suministrado. Esta comparación se puede realizar cargando *Prog1.asm* como un archivo de comparación: consulte [la figura 6.3](#) para obtener más detalles.





**Figura 6.3** Probar la salida del ensamblador usando el ensamblador suministrado.

Una versión basada en la web del proyecto 6 está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

## 6.6 Perspectiva

Como la mayoría de los ensambladores, el ensamblador Hack es un traductor relativamente simple, que se ocupa principalmente del procesamiento de texto. Naturalmente, los ensambladores para lenguajes de máquina más ricos son más elaborados. Además, algunos ensambladores cuentan con capacidades de manejo de símbolos más sofisticadas que no se encuentran en Hack. Por ejemplo, algunos ensambladores admiten *aritmética constante* en símbolos, como `usarbase+5` para referirse a la quinta ubicación de memoria después de la dirección a la que hace referencia la base.

Muchos ensambladores se extienden para manejar *macroinstrucciones*. Una macroinstrucción es una secuencia de instrucciones de máquina que tiene un nombre. Por ejemplo, nuestro ensamblador se puede extender para traducir las instrucciones de macro acordadas, por ejemplo, `D=M[addr]`, en las dos instrucciones primitivas de Hack `@addr`, seguidas de `D=M`. Del mismo modo, la macroinstrucción `goto addr` se puede traducir en `@addr`, seguida de `0; JMP`, y así sucesivamente. Tales macro-instrucciones pueden simplificar considerablemente la escritura de programas ensambladores, a un bajo costo de traducción.

Cabe señalar que los programas de lenguaje de máquina rara vez son escritos por humanos. Más bien, generalmente son escritos por compiladores. Y un compilador, al ser un autómata, puede opcionalmente omitir las instrucciones simbólicas y generar código de máquina binario directamente. Dicho esto, un ensamblador sigue siendo un programa útil, especialmente para los desarrolladores de programas C / C ++ que se preocupan por la eficiencia y la optimización. Al inspeccionar el código simbólico generado por el compilador, el programador puede mejorar el código de alto nivel para lograr un mejor rendimiento en el hardware host. Cuando el código ensamblador generado se considera eficaz, el ensamblador puede traducirlo aún más en el código ejecutable binario final.

\* \* \*

¡Felicidades! Has llegado al final de la parte I del viaje de Nand a Tetris. Si completó los proyectos 1 a 6, ha construido un sistema informático de propósito general a partir de los primeros principios. Este es un logro fantástico, y debes sentirte orgulloso y realizado.

Por desgracia, la computadora es capaz de ejecutar solo programas escritos en lenguaje de máquina. En la parte II del libro utilizaremos esta plataforma de hardware básica como punto de partida y construiremos sobre ella una jerarquía de software moderna. El software consistirá en una máquina virtual, un compilador para un lenguaje de programación basado en objetos de alto nivel y un sistema operativo básico.

Entonces, cuando esté listo para más aventuras, pasemos a la parte II de nuestro gran viaje de Nand a Tetris.

---

## II Software

Cualquier tecnología suficientemente avanzada es indistinguible de la magia.

—Arthur C. Clarke (1962)

A lo que agregamos: "y cualquier magia suficientemente avanzada es indistinguible del trabajo duro, detrás de escena". En la parte I del libro construimos la plataforma de hardware de un sistema informático llamado Hack, capaz de ejecutar programas escritos en el lenguaje de máquina Hack. En la parte II transformaremos esta máquina barebone en una tecnología avanzada, indistinguible de la magia: una caja negra que puede metamorfosearse en un jugador de ajedrez, un motor de búsqueda, un simulador de vuelo, un transmisor de medios o cualquier otra cosa que te guste. Para hacerlo, desarrollaremos la elaborada jerarquía de software detrás de escena que dota a las computadoras de la capacidad de ejecutar programas escritos en lenguajes de programación de alto nivel. En particular, nos centraremos en Jack, un lenguaje de programación simple, similar a Java, basado en objetos, descrito formalmente en el capítulo 9. A lo largo de los años, los lectores y estudiantes de Nand to Tetris han utilizado Jack para desarrollar Tetris, Pong, Snake, Space Invaders y muchos otros juegos y aplicaciones interactivas. Al ser una computadora de propósito general, Hack puede ejecutar todos estos programas y cualquier otro programa que se le ocurra.

Claramente, la brecha entre la sintaxis expresiva de los lenguajes de programación de alto nivel, por un lado, y las torpes instrucciones del lenguaje de máquina de bajo nivel, por el otro, es enorme. Si no está convencido, intente desarrollar un juego de Tetris usando instrucciones como `@ 17 y M=M+1`. Cerrar esta brecha es de lo que trata la parte II de este libro. Construiremos este puente desarrollando gradualmente algunos de los más poderosos y ambiciosos

Programas en informática aplicada: un *compilador*, una *máquina virtual* y un *sistema operativo* básico.

Nuestro compilador Jack estará diseñado para tomar un programa Jack, digamos Tetris, y producir a partir de él un flujo de instrucciones en lenguaje de máquina que, cuando se ejecuta, hace que la plataforma Hack ofrezca una experiencia de juego de Tetris. Por supuesto, Tetris es solo un ejemplo: el compilador que construyas será capaz de traducir *cualquier* programa Jack en código máquina que se pueda ejecutar en la computadora Hack. El compilador, cuyas tareas principales consisten en *el análisis de sintaxis* y *la generación de código*, se construirá en los capítulos 10 y 11.

Al igual que con lenguajes de programación como Java y C #, el compilador Jack tendrá *dos niveles*: el compilador generará *código de VM* provisional, diseñado para ejecutarse en una *máquina virtual* abstracta. Luego, un traductor separado compilará el código de VM al lenguaje de máquina Hack. *La virtualización*, una de las ideas más importantes en informática aplicada, entra en juego en numerosos entornos, incluida la compilación de programas, la computación en la nube, el almacenamiento distribuido, el procesamiento distribuido y los sistemas operativos. Dedicaremos los capítulos 7 y 8 a motivar, diseñar y construir nuestra máquina virtual.

Como muchos otros lenguajes de alto nivel, el lenguaje básico de Jack es sorprendentemente simple. Lo que convierte a los lenguajes modernos en potentes sistemas de programación son *las bibliotecas estándar* que proporcionan funciones matemáticas, procesamiento de cadenas, gestión de memoria, dibujo de gráficos, manejo de la interacción del usuario y más. En conjunto, estas bibliotecas estándar forman un *sistema operativo* (SO) básico que, en el marco de Jack, se empaqueta como la biblioteca de *clases estándar de Jack*. Este sistema operativo básico, diseñado para cerrar muchas brechas entre el lenguaje Jack de alto nivel y la plataforma Hack de bajo nivel, se desarrollará en el propio Jack. Quizás se pregunte cómo se puede desarrollar un software que se supone que habilita un lenguaje de programación en este mismo lenguaje. Nos enfrentaremos a este desafío siguiendo una estrategia de desarrollo conocida como *bootstrapping*, similar a cómo se desarrolló el sistema operativo Unix utilizando el lenguaje C.

La construcción del sistema operativo nos dará la oportunidad de presentar algoritmos elegantes y estructuras de datos clásicas que normalmente se utilizan para administrar recursos de hardware y dispositivos periféricos. Luego implementaremos estos algoritmos en Jack, ampliando las capacidades del lenguaje paso a paso. A medida que avanza

por los capítulos de la parte II, se ocupará del sistema operativo de

varias perspectivas diferentes. En el capítulo 9, actuando como programador de *aplicaciones*, desarrollará una aplicación Jack y utilizará los servicios del sistema operativo de forma abstracta, desde una perspectiva de cliente de alto nivel. En los capítulos 10 y 11, al compilar el compilador Jack, utilizará los servicios del sistema operativo como un cliente de bajo nivel, por ejemplo, para varios servicios de administración de memoria requeridos por el compilador. En el capítulo 12, finalmente se pondrá el sombrero del desarrollador del sistema operativo e implementará todos estos servicios del sistema usted mismo.

---

## II.1 Una muestra de la programación de Jack

Antes de profundizar en todos estos emocionantes proyectos, daremos una breve e informal introducción al lenguaje Jack. Esto se hará usando dos ejemplos, comenzando con Hello World. Usaremos este ejemplo para demostrar que incluso el programa de alto nivel más trivial tiene mucho más de lo que parece. Luego presentaremos un programa simple que ilustra las capacidades basadas en objetos del lenguaje Jack. Una vez que probemos el lenguaje Jack de alto nivel orientado al programador, estaremos preparados para comenzar el viaje de realización del lenguaje mediante la construcción de una máquina virtual, un compilador y un sistema operativo.

**Hello World, de nuevo:** Comenzamos este libro con el icónico programa Hello World que los estudiantes a menudo encuentran como lo primero en los cursos introductorios de programación. Aquí está este programa trivial una vez más, escrito en el lenguaje de programación Jack:

```
Primer ejemplo en Programación 101 class Main {
    function void main () {
        do Output.println ("Hola mundo"); devolución;
    }
}
```

Analicemos algunas de las suposiciones implícitas que normalmente hacemos cuando se nos presentan tales programas. La primera magia que damos por sentada es que una secuencia de caracteres, por ejemplo, `println ("Hello World")`, puede hacer que la computadora muestre algo en la pantalla. ¿Cómo averigua la computadora *qué* hacer? Y aunque el ordenador supiera *qué* hacer,

*cómo* ¿realmente lo hará? Como vimos en la parte I del libro, la pantalla es una cuadrícula de píxeles. Si queremos mostrar H En la pantalla, tenemos que activar y desactivar un subconjunto de píxeles cuidadosamente seleccionado que, en conjunto, representan la imagen de la letra deseada en la pantalla. Por supuesto, esto es solo el comienzo. ¿Qué tal mostrar esto? H ¿Legible en pantallas que tienen diferentes tamaños y resoluciones? ¿Y qué hay de lidiar con *mientras* y *para* Bucles *Matrices*, *Objetos*, *métodos*, *Clases*, y todo el Otro Golosinas ese ¿Los programadores de alto nivel están capacitados para usar sin pensar nunca en cómo funcionan? De hecho, la belleza de los lenguajes de programación de alto nivel, y la de las abstracciones bien diseñadas en general, es que permiten usarlos en un estado de feliz ignorancia. De hecho, se alienta a los programadores de aplicaciones a ver el lenguaje como una abstracción de caja negra, sin prestar atención a cómo se implementa realmente. Todo lo que necesitas es un buen tutorial, algunos ejemplos de código, y listo.

Sin embargo, claramente, en un momento u otro, *alguien* debe implementar esta abstracción del lenguaje. Alguien debe desarrollar, de una vez por todas, la capacidad de calcular eficientemente las raíces cuadradas cuando el programador de aplicaciones dice felizmente `sqrt(1764)`, para obtener un número del usuario cuando el programador dice felizmente `x=readInt()`, que encuentre y cree un bloque de memoria disponible cuando el programador crea despreocupadamente un objeto usando `nuevos`, y realizar de forma transparente todos los demás servicios abstractos que los programadores esperan obtener sin tener que pensar en ellos. Entonces, ¿quiénes son las almas buenas que convierten la programación de alto nivel en una tecnología avanzada indistinguible de la magia? Son los magos del software que desarrollan *compiladores*, *máquinas virtuales* y *sistemas operativos*. Y eso es precisamente lo que harás en los próximos capítulos.

Quizás se pregunte por qué tiene que preocuparse por esta escurridiza escena detrás de escena. ¿No acabamos de decir que puedes usar lenguajes de alto nivel sin preocuparte por cómo funcionan? Hay al menos dos razones para ello. Primero, cuanto más profundice en los componentes internos del sistema de bajo nivel, más sofisticado se convertirá en un programador de alto nivel. En particular, aprenderá a escribir código de alto nivel que explote el hardware y el sistema operativo de manera inteligente y eficiente y cómo evitar errores desconcertantes como fugas de memoria.

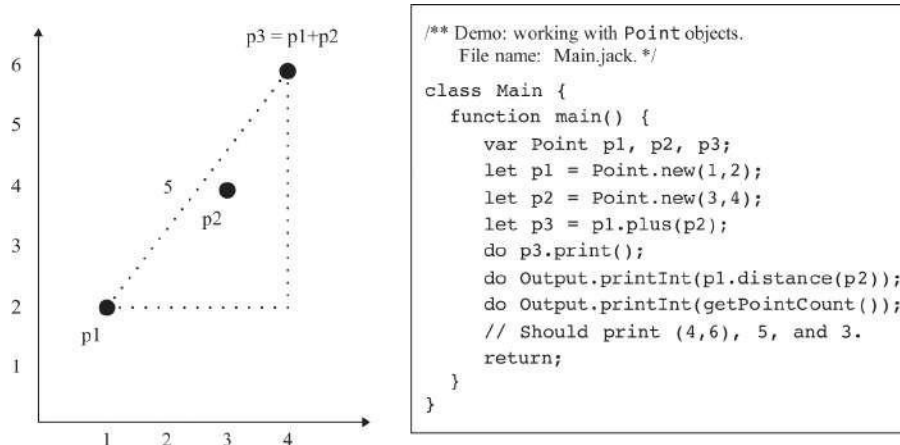
En segundo lugar, al ensuciarse las manos y desarrollar las partes internas del sistema usted mismo, descubrirá algunos de los algoritmos y

estructuras de datos más bellos y poderosos de la informática aplicada. Es importante destacar que el



las ideas y técnicas que se desarrollarán en la parte II no se limitan a los compiladores y sistemas operativos. Más bien, son los componentes básicos de numerosos sistemas y aplicaciones de software que lo acompañarán a lo largo de su carrera.

**El programa PointDemo:** Supongamos que queremos representar y manipular *puntos* en un plano. La Figura II.1 muestra dos de estos puntos,  $p_1$  y  $p_2$ , y un tercer punto,  $p_3$ , resultante de la suma vectorial  $p_3 = p_1 + p_2 = (1, 2) + (3, 4) = (4, 6)$ . La figura también muestra la *distancia euclidiana* entre  $p_1$  y  $p_3$ , que se puede calcular utilizando el teorema de Pitágoras. El código de la clase Main ilustra cómo se pueden realizar tales manipulaciones algebraicas utilizando el lenguaje Jack basado en objetos.



**Figura II.1** Manipulación de puntos en un plano: ejemplo y código Jack.

Quizás se pregunte por qué Jack usa palabras clave como `var`, `let` y `do`. Por ahora, le recomendamos que no se detenga en los detalles sintácticos. En su lugar, centrémonos en el panorama general y procedamos a revisar cómo se puede usar el lenguaje Jack para implementar el tipo de datos abstracto `Point` (figura II.2).

```

/** Represents a two-dimensional point.
File name: Point.jack. */
class Point {
    // The coordinates of this point:
    field int x, y

    // The number of Point objects constructed so far:
    static int pointCount;

    /** Constructs a two-dimensional point and
    initializes it with the given coordinates. */
    constructor Point new(int ax, int ay) {
        let x = ax;
        let y = ay;
        let pointCount = pointCount + 1;
        return this;
    }

    /** Returns the x coordinate of this point. */
    method int getX() {return x;}

    /** Returns the y coordinate of this point. */
    method int getY() {return y;}

    /** Returns the number of Points constructed so far. */
    function int getPointCount() {
        return pointCount;
    }

    // Class declaration continues on top right.
}

/** Returns a point which is this point plus
the other point. */
method Point plus(Point other) {
    return Point.new(x + other.getX(),
                    y + other.getY());
}

/** Returns the Euclidean distance between
this and the other point. */
method int distance(Point other) {
    var int dx, dy;
    let dx = x - other.getX();
    let dy = y - other.getY();
    return Math.sqrt((dx*dx) + (dy*dy));
}

/** Prints this point, as "(x,y)" */
method void print() {
    do Output.printString("(");
    do Output.printInt(x);
    do Output.printString(",");
    do Output.printInt(y);
    do Output.printString(")");
    return;
}

} // End of Point class declaration.

```

**Figura II.2** Jack de la Punto abstracción.

El código que se muestra en la figura II.2 ilustra que una clase Jack (de la cual Main y Point son dos ejemplos) es una colección de una o más *subrutinas*, cada una de las cuales es un *constructor*, *método* o *función*. Los *constructores* son subrutinas que crean nuevos objetos, los *métodos* son subrutinas que operan en el objeto actual y las *funciones* son subrutinas que no operan en ningún objeto en particular. (Los puristas del diseño orientado a objetos pueden fruncir el ceño al mezclar métodos y funciones en la misma clase; lo estamos haciendo aquí con fines ilustrativos).

El resto de esta sección es una descripción general informal de las clases Main y Point. Nuestro objetivo es dar una idea de la programación de Jack, aplazando una descripción completa del lenguaje hasta el capítulo 9. Entonces, permitiéndonos el lujo de centrarnos solo en la esencia, comencemos. La función Main.main comienza declarando tres variables de *objeto* (también conocidas como *referencias* o *punteros*), diseñadas para hacer referencia a instancias de la clase Point. Luego pasa a construir dos objetos Point y les asigna las variables p1 y p2. A continuación, llama al método plus y asigna p3 al objeto Point devuelto por ese método. El resto de la función Main.main imprime algunos resultados.

La clase Point comienza declarando que cada objeto Point se caracteriza por dos variables *de campo* (también conocidas como *propiedades* o *variables de instancia*). A continuación, declara una *variable estática*, es decir, una variable de nivel de clase asociada a ningún objeto en particular. El constructor de la clase configura los valores de campo de la clase

recién creado e incrementa el número de instancias derivadas de esta clase hasta el momento. Tenga en cuenta que un constructor Jack debe devolver explícitamente la dirección de memoria del objeto recién creado, que, de acuerdo con las reglas del lenguaje, se denota así.

Quizás se pregunte por qué el resultado de la raíz cuadrada calculada por el método de distancia se almacena en una variable `int`: claramente, un tipo de datos de valor real como `float` tendría más sentido. La razón de esta peculiaridad es simple: el lenguaje Jack presenta solo tres tipos de datos primitivos: `int`, booleano y `char`. Otros tipos de datos se pueden implementar a voluntad usando clases, como lo haremos en los capítulos 9 y 12.

**El sistema operativo:** las clases `Main` y `Point` usan tres funciones del sistema operativo: `Output.println`, `Output.printString` y `Math.sqrt`. Al igual que otros lenguajes modernos de alto nivel, el lenguaje Jack se ve aumentado por un conjunto de *clases estándar* que proporcionan servicios de sistema operativo de uso común (la API completa del sistema operativo se proporciona en el apéndice 6). Tendremos mucho más que decir sobre los servicios del sistema operativo en el capítulo 9, donde los usaremos en el contexto de la programación de Jack, así como en el capítulo 12, donde construiremos el sistema operativo.

Además de llamar a los servicios del sistema operativo para sus efectos directamente desde los programas Jack, el sistema operativo entra en juego de otras maneras menos obvias. Por ejemplo, considere la nueva operación, utilizada para construir objetos en lenguajes orientados a objetos. ¿Cómo sabe el compilador en qué parte de la RAM del host colocar el objeto recién construido? Bueno, no es así. Se llama a una rutina del sistema operativo para resolverlo. Cuando construyamos el sistema operativo en el capítulo 12, implementará, entre muchas otras cosas, un sistema típico de administración de memoria en tiempo de ejecución. A continuación, aprenderá, de forma práctica, cómo interactúa este sistema con el hardware, desde un extremo, y con los compiladores, desde el otro, para asignar y recuperar espacio de RAM de forma inteligente y eficiente. Este es solo un ejemplo que ilustra cómo el sistema operativo cierra las brechas entre las aplicaciones de alto nivel y la plataforma de hardware del host.

---

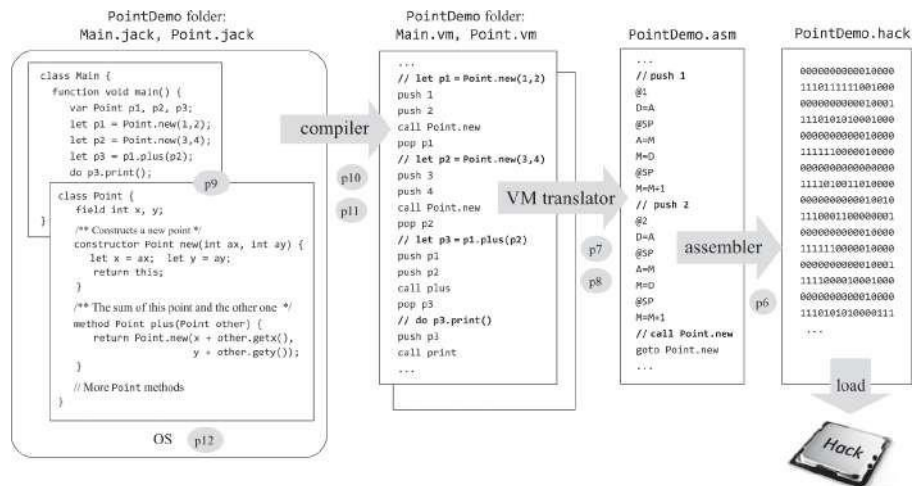
## II.2 Compilación de programas

Un programa de alto nivel es una abstracción simbólica que no significa nada para el hardware subyacente. Antes de ejecutar un programa, el código de alto nivel debe

traducirse al lenguaje de máquina. Este proceso de traducción se llama *compilación*, y el programa que lo lleva a cabo se llama *compilador*. Escribir un compilador que traduzca programas de alto nivel en instrucciones de máquina de bajo nivel es un desafío digno. Algunos lenguajes, por ejemplo, Java y C#, se enfrentan a este desafío empleando un elegante *modelo de compilación de dos niveles*. Primero, el programa fuente se traduce en un código VM abstracto e provisional (llamado *código de bytes* en Java y Python y *lenguaje intermedio* en C # / .NET). A continuación, utilizando un proceso completamente separado e independiente, el código de la máquina virtual se puede traducir aún más al lenguaje de máquina de cualquier plataforma de hardware de destino.

Esta modularidad es al menos una de las razones por las que Java se convirtió en un lenguaje de programación tan dominante. Desde una perspectiva histórica, Java puede verse como un poderoso lenguaje orientado a objetos cuyo modelo de compilación de dos niveles fue lo correcto en el momento adecuado, justo cuando las computadoras comenzaron a evolucionar de unas pocas plataformas predecibles de procesador / sistema operativo a una desconcertante mezcla de numerosas PC, teléfonos celulares, dispositivos móviles y dispositivos de Internet de las cosas, todos conectados por una red global. Escribir programas de alto nivel que puedan ejecutarse en cualquiera de estas plataformas host es un desafío abrumador. Una forma de optimizar este ecosistema distribuido de múltiples proveedores (desde una perspectiva de compilación) es basarlo en una arquitectura de máquina virtual general y acordada. Actuando como un entorno de tiempo de ejecución intermedio común, el enfoque de VM permite a los desarrolladores escribir programas de alto nivel que se ejecutan casi tal cual en muchas plataformas de hardware diferentes, cada una equipada con su propia implementación de VM. Tendremos mucho más que decir sobre el poder habilitador de esta modularidad a medida que se desarrolle la parte II.

**El camino por delante:** En el resto del libro nos aplicaremos al desarrollo de todas las emocionantes tecnologías de software mencionadas anteriormente. Nuestro objetivo final es crear una infraestructura para convertir programas de alto nivel, *cualquier* programa, en código ejecutable. La hoja de ruta se muestra en [el gráfico II.3](#).



**Figura II.3** Mapa de carreteras de la parte II (el ensamblador pertenece a la parte I y se muestra aquí para completarlo). La hoja de ruta describe una jerarquía de traducción, desde un programa multiclase de alto nivel, basado en objetos hasta código de máquina virtual, código ensamblador y código binario ejecutable. Los círculos numerados representan los proyectos que implementan el compilador, el traductor de VM, el ensamblador y el sistema operativo. El Proyecto 9 se centra en escribir una aplicación de Jack para familiarizarse con el lenguaje.

Siguiendo el espíritu de Nand to Tetris, seguiremos la hoja de ruta de la parte II de abajo hacia arriba. Para empezar, asumimos que tenemos una plataforma de hardware equipada con un lenguaje ensamblador. En los capítulos 7 y 8 presentaremos una arquitectura de máquina virtual y un lenguaje de máquina virtual, e implementaremos esta abstracción mediante el desarrollo de un *traductor de máquinas virtuales* que traduce los programas de máquinas virtuales en programas de ensamblaje Hack. En el capítulo 9 presentaremos el lenguaje de alto nivel de Jack y lo usaremos para desarrollar un juego de computadora simple. De esta manera, se familiarizará con el lenguaje y el sistema operativo Jack antes de comenzar a construirlos. En los capítulos 10 y 11 desarrollaremos el compilador Jack, y en el capítulo 12 construiremos el sistema operativo.

Entonces, ¡jarremanguémonos y pongámonos manos a la obra!

---

## 7 Máquina virtual I: Procesamiento

Los programadores son creadores de universos de los que solo ellos son responsables. Se pueden crear universos de complejidad prácticamente ilimitada en forma de programas informáticos.

—Joseph Weizenbaum, *El poder de la computadora y la razón humana* (1974)

En este capítulo se describen los primeros pasos para crear un compilador para un lenguaje típico de alto nivel basado en objetos. Abordamos este desafío en dos etapas principales, cada una de las cuales abarca dos capítulos. En los capítulos 10 y 11 describiremos la construcción de un *compilador*, diseñado para traducir programas de alto nivel en *código intermedio*; en los capítulos 7 y 8 describiremos la construcción de un traductor de seguimiento, diseñado para traducir el código intermedio al lenguaje de máquina de una plataforma de hardware de destino. Como sugieren los números de los capítulos, continuaremos este desarrollo sustancial de abajo hacia arriba, comenzando con el traductor.

El código intermedio que se encuentra en el núcleo de este modelo de compilación está diseñado para ejecutarse en una computadora abstracta, llamada *máquina virtual* o VM. Hay varias razones por las que este modelo de compilación de dos niveles tiene sentido, en comparación con los compiladores tradicionales que traducen programas de alto nivel directamente al lenguaje de máquina. Un beneficio es la compatibilidad multiplataforma: dado que la máquina virtual se puede realizar con relativa facilidad en muchas plataformas de hardware, se puede ejecutar el mismo código de VM que en cualquier dispositivo equipado con dicha implementación de VM. Esa es una de las razones por las que Java se convirtió en un lenguaje dominante para desarrollar aplicaciones para dispositivos móviles, que se caracterizan por muchas combinaciones diferentes de procesador / sistema operativo. La máquina virtual se puede implementar en los dispositivos de destino mediante intérpretes de software o hardware de propósito especial, o traduciendo los programas de máquina virtual al lenguaje de máquina del dispositivo. Este último enfoque de

aplicación es adoptado por

Java, Scala, C# y Python, así como por el lenguaje Jack desarrollado en Nand to Tetris.

Este capítulo presenta una arquitectura de máquina virtual típica y un lenguaje de máquina virtual, conceptualmente similar a la máquina virtual Java (JVM) y al código de bytes, respectivamente. Como es habitual en Nand to Tetris, la máquina virtual se presentará desde dos perspectivas. Primero, motivaremos y especificaremos la abstracción de la máquina virtual, describiendo para qué está diseñada la máquina virtual. A continuación, describiremos una implementación propuesta de la VM sobre la plataforma Hack. Nuestra implementación implica escribir un programa llamado *traductor de VM* que traduce el código de VM en código ensamblador Hack.

El lenguaje de máquina virtual que presentaremos consta de comandos aritmético-lógicos, comandos de acceso a memoria llamados *push* y *pop*, comandos de ramificación y comandos de llamada y retorno de funciones. Dividimos la discusión e implementación de este lenguaje en dos partes, cada una cubierta en un capítulo y proyecto separados. En este capítulo creamos un traductor de VM básico que implementa los comandos aritmético-lógicos y push/pop de la VM. En el siguiente capítulo ampliamos el traductor básico para manejar comandos de ramificación y función. El resultado será una implementación de máquina virtual a gran escala que servirá como back-end del compilador que construiremos en los capítulos 10 y 11.

La máquina virtual que surgirá de este esfuerzo ilustra varias ideas y técnicas importantes. Primero, la noción de que un marco informático emule a otro es una idea fundamental en la informática, que se remonta a Alan Turing en la década de 1930. Hoy en día, el modelo de máquina virtual es la pieza central de varios entornos de programación convencionales, incluidos Java, .NET y Python. La mejor manera de obtener una visión interna íntima de cómo funcionan estos entornos de programación es crear una versión simple de sus núcleos de VM, como lo hacemos aquí.

Otro tema importante en este capítulo es el *procesamiento de pilas*. La *pila* es una estructura de datos fundamental y elegante que entra en juego en numerosos sistemas informáticos, algoritmos y aplicaciones. Dado que la máquina virtual presentada en este capítulo está basada en pilas, proporciona un ejemplo práctico de esta estructura de datos notablemente versátil y poderosa.

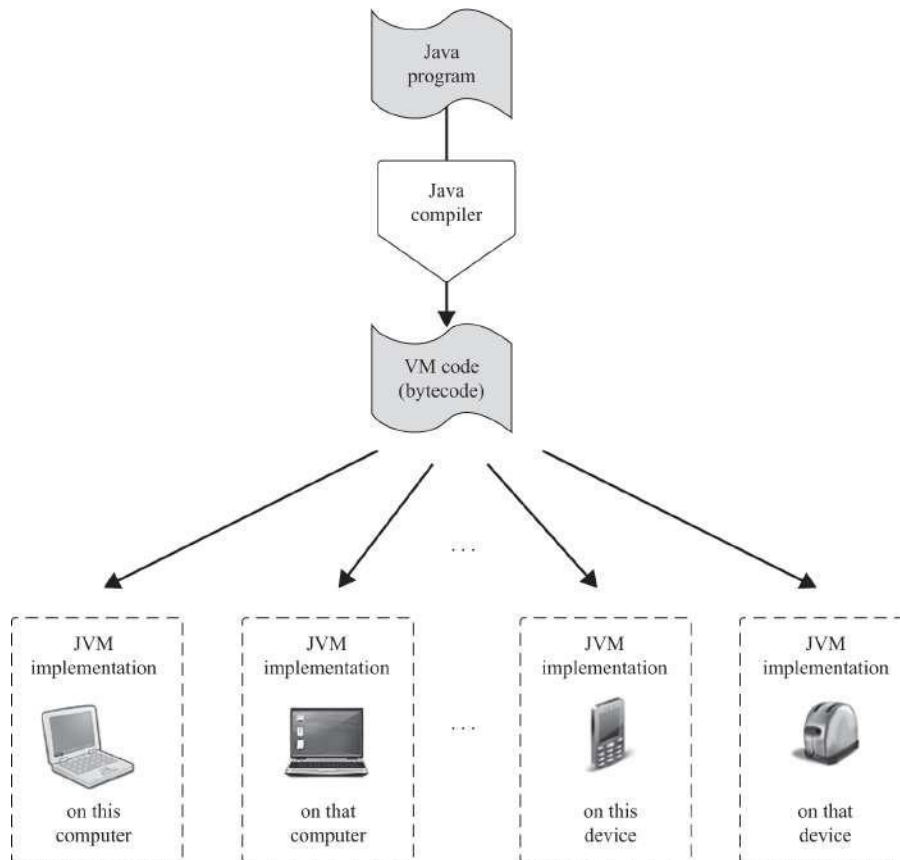


---

## 7.1 El paradigma de la máquina virtual

Antes de que un programa de alto nivel pueda ejecutarse en un equipo de destino, debe traducirse al lenguaje de máquina del equipo. Tradicionalmente, se desarrollaba un compilador independiente específicamente para cualquier par de lenguaje de alto nivel y lenguaje de máquina de bajo nivel. A lo largo de los años, la realidad de muchos lenguajes de alto nivel, por un lado, y muchos procesadores y conjuntos de instrucciones, por el otro, ha llevado a una proliferación de muchos compiladores diferentes, cada uno dependiendo de cada detalle de sus lenguajes de origen y destino. Una forma de desacoplar esta dependencia es dividir el proceso de compilación general en dos etapas casi separadas. En la primera etapa, el código de alto nivel se analiza y se traduce en pasos de procesamiento intermedios y abstractos, pasos que no son ni altos ni bajos. En la segunda etapa, los pasos intermedios se traducen al lenguaje de máquina de bajo nivel del hardware de destino.

Esta descomposición es muy atractiva desde la perspectiva de la ingeniería de software. En primer lugar, tenga en cuenta que la primera etapa de traducción depende solo de los detalles del lenguaje de alto nivel de origen y la segunda etapa solo de los detalles del lenguaje de máquina de bajo nivel de destino. Por supuesto, la interfaz entre las dos etapas de traducción, la definición exacta de los pasos intermedios del procesamiento, debe diseñarse y optimizarse cuidadosamente. En algún momento de la evolución de las soluciones de traducción de programas, los desarrolladores de compiladores concluyeron que esta interfaz intermedia es lo suficientemente importante como para merecer su propia definición como un lenguaje independiente diseñado para ejecutarse en una máquina abstracta. Específicamente, se puede describir una *máquina virtual* cuyos comandos realizan los pasos intermedios de procesamiento en los que se traducen los comandos de alto nivel. El compilador que antes era un solo programa monolítico ahora se divide en dos programas separados y mucho más simples. El primer programa, todavía denominado *compilador*, traduce el código de alto nivel en comandos intermedios de VM; el segundo programa, llamado *traductor de VM*, traduce los comandos de VM aún más en las instrucciones de la máquina de la plataforma de hardware de destino. [La Figura 7.1](#) describe cómo este marco de compilación de dos niveles ha contribuido a la portabilidad multiplataforma de los programas Java.



**Figura 7.1** El marco de la máquina virtual, usando Java como ejemplo. Los programas de alto nivel se compilan en código de máquina virtual intermedio. El mismo código de VM se puede enviar y ejecutar en cualquier plataforma de hardware equipada con una implementación de *JVM adecuada*. Estas implementaciones de VM generalmente se realizan como programas del lado del cliente que traducen el código de VM a los lenguajes de máquina de los dispositivos de destino.

El marco de la máquina virtual conlleva muchos beneficios prácticos. Cuando un proveedor introduce en el mercado un nuevo dispositivo digital, por ejemplo, un teléfono celular, puede desarrollar para él una implementación de JVM, conocida como JRE (Java Runtime Environment), con relativa facilidad. Esta infraestructura habilitadora del lado del cliente dota inmediatamente al dispositivo de una enorme base de software Java disponible. Y, en un mundo como .NET, en el que varios lenguajes de alto nivel se compilan en el mismo lenguaje de VM intermedio, los compiladores de diferentes lenguajes pueden compartir el mismo back-end de VM, lo que permite el uso de bibliotecas de software comunes y la interoperabilidad de lenguajes.

El precio pagado por la elegancia y la potencia del marco de VM es una eficiencia reducida. Naturalmente, un proceso de traducción de dos niveles resulta, en última instancia, en la generación de código máquina que es más detallado y

engorroso que el código producido por la compilación directa. Sin embargo, a medida que los procesadores se vuelven más rápidos y las implementaciones de VM más optimizadas, la eficiencia degradada apenas se nota en la mayoría de las aplicaciones. Por supuesto, siempre habrá aplicaciones de alto rendimiento y sistemas integrados que continuarán exigiendo el código eficiente generado por compiladores de lenguaje de un solo nivel como C y C++. Dicho esto, las versiones modernas de C++ cuentan con compiladores clásicos de un nivel y compiladores basados en máquinas virtuales de dos niveles.

---

## 7.2 Máquina de pila

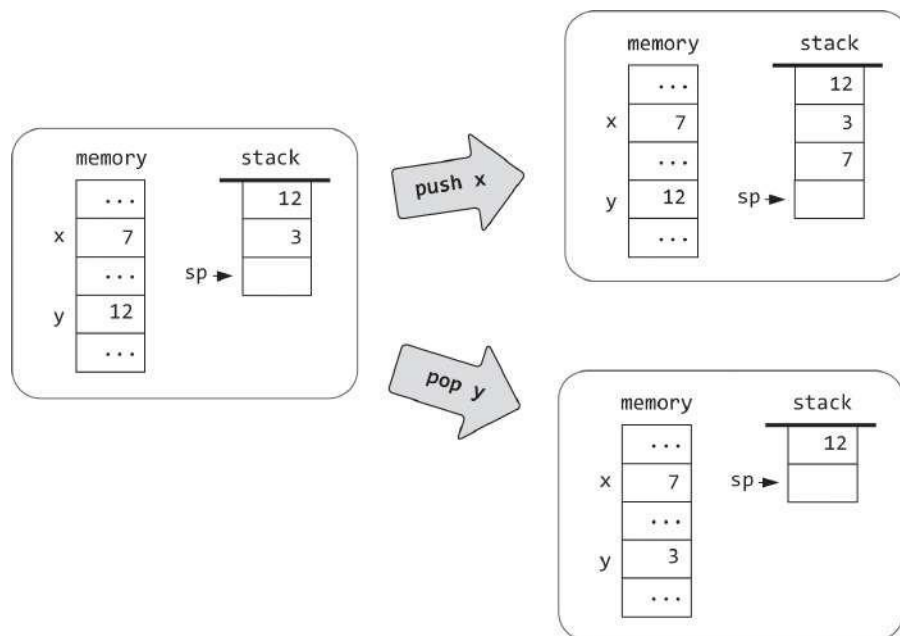
El diseño de un lenguaje de VM eficaz busca lograr un equilibrio conveniente entre los lenguajes de programación de alto nivel, por un lado, y una gran variedad de lenguajes de máquina de bajo nivel, por el otro. Por lo tanto, el lenguaje de VM deseado debe satisfacer varios requisitos provenientes tanto de arriba como de abajo. Primero, el lenguaje debe tener un poder expresivo razonable. Logramos esto mediante el diseño de un lenguaje de VM que presenta comandos aritmético-lógicos, comandos push/pop, comandos de ramificación y comandos de función. Estos comandos de VM deben ser lo suficientemente "altos" para que el código de VM generado por el compilador sea razonablemente elegante y esté bien estructurado. Al mismo tiempo, los comandos de VM deben ser lo suficientemente "bajos" para que el código de máquina generado a partir de ellos por los traductores de VM sea ajustado y eficiente. Dicho de otra manera, tenemos que asegurarnos de que las brechas de traducción entre el nivel alto y el nivel de VM, por un lado, y el nivel de VM y el nivel de máquina, por el otro, no sean amplias. Una forma de satisfacer estos requisitos algo conflictivos es basar el lenguaje de máquina virtual provisional en una arquitectura abstracta denominada *máquina de pila*.

Antes de continuar, nos gustaría hacer un llamado a la paciencia. La relación entre la máquina de pila que ahora vamos a describir y el compilador que presentaremos más adelante en el libro es sutil. Por lo tanto, aconsejamos a los lectores que se permitan saborear la belleza intrínseca de la abstracción de la máquina de pila sin preocuparse por su propósito final en cada paso del camino. Todo el poder práctico de esta notable abstracción sólo tendrá su peso hacia el final del *próximo* capítulo; por ahora, baste decir

que cualquier programa, escrito en cualquier lenguaje de programación de alto nivel, se puede traducir en una secuencia de operaciones en una pila.

### 7.2.1 Empujar y estallar

La pieza central del modelo de máquina de pila es una estructura de datos abstracta llamada *pila*. Una pila es un espacio de almacenamiento secuencial que crece y se reduce según sea necesario. La pila admite varias operaciones, las dos clave son push y pop. La operación de inserción agrega un valor a la parte superior de la pila, como agregar una placa a la parte superior de una pila de placas. La operación pop quita el valor superior de la pila; el valor que estaba justo antes de que se convierta en el elemento de la pila superior. Consulte la figura 7.2 para ver un ejemplo. Tenga en cuenta que la lógica push/pop da como resultado una *lógica de acceso* LIFO: el valor extraído es siempre el último que se insertó en la pila. Resulta que esta lógica de acceso se presta perfectamente a los propósitos de traducción y ejecución de programas, pero esta idea tardará dos capítulos en desarrollarse.

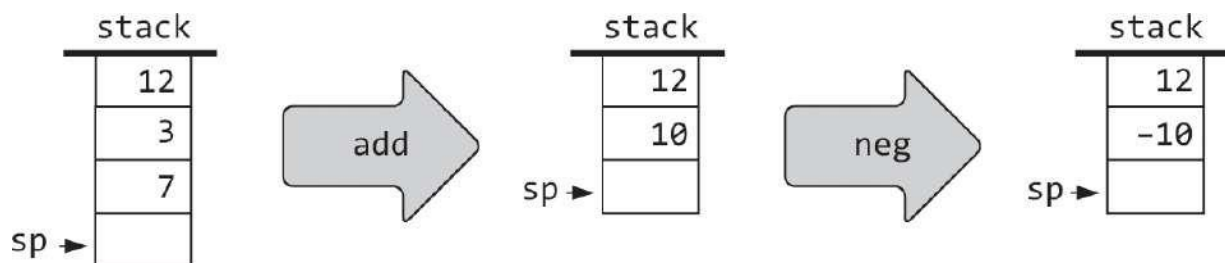


**Figura 7.2** Ejemplo de procesamiento de pila, que ilustra las dos operaciones elementales push y pop. La configuración consta de dos estructuras de datos: un segmento de memoria similar a la RAM y una pila. Siguiendo la convención, la pila se dibuja como si creciera hacia abajo. La ubicación justo después del valor superior de la pila se denomina un puntero llamado *sp* o *puntero de pila*. Los símbolos *x* e *y* se refieren a dos ubicaciones de memoria arbitrarias.

Como se muestra en la figura 7.2, nuestra abstracción de VM incluye una *pila*, así como un segmento de memoria secuencial similar a RAM. Observe que el acceso a la pila es diferente del acceso a la memoria convencional. En primer lugar, solo se puede acceder a la pila desde su parte superior, mientras que la memoria normal permite el acceso directo e indexado a cualquier valor de la memoria. En segundo lugar, *leer* un valor de la pila es una operación con pérdidas: solo se puede leer el valor superior y la única forma de acceder a él implica eliminarlo de la pila (aunque algunos modelos de pila también proporcionan una *operación de inspección*, que permite leer sin eliminar). Por el contrario, el acto de leer un valor de una memoria normal no deja ningún impacto en el estado de la memoria. Por último, *escribir* en la pila implica agregar un valor en la parte superior de la pila sin cambiar los otros valores de la pila. Por el contrario, escribir un elemento en una ubicación de memoria normal es una operación con pérdida, ya que invalida el valor anterior de la ubicación.

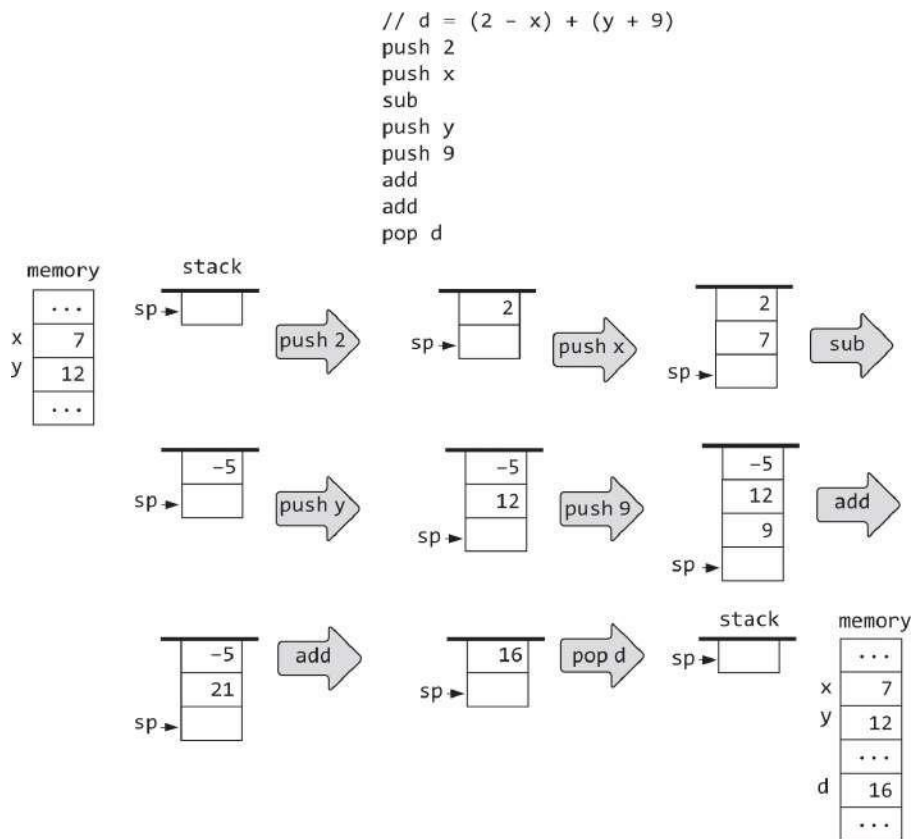
### 7.2.2 Apilar aritmética

Considere la operación genérica  $x \text{ op } y$ , donde el operador  $\text{op}$  se aplica a los operandos  $x$  e  $y$ , por ejemplo,  $7 + 5$ ,  $3 - 8$ , y así sucesivamente. En una máquina de pila, cada *operación*  $x \text{ op } y$  se lleva a cabo de la siguiente manera: primero, los operandos  $x$  e  $y$  se extraen de la parte superior de la pila; a continuación, se calcula el valor de  $x \text{ op } y$ ; finalmente, el valor calculado se empuja a la parte superior de la pila. Del mismo modo, la operación unaria  $\text{op } x$  se realiza sacando  $x$  de la parte superior de la pila, calculando el valor de  $\text{op } x$  y finalmente empujando este valor a la parte superior de la pila. Por ejemplo, así es como se manejan la suma y la negación:

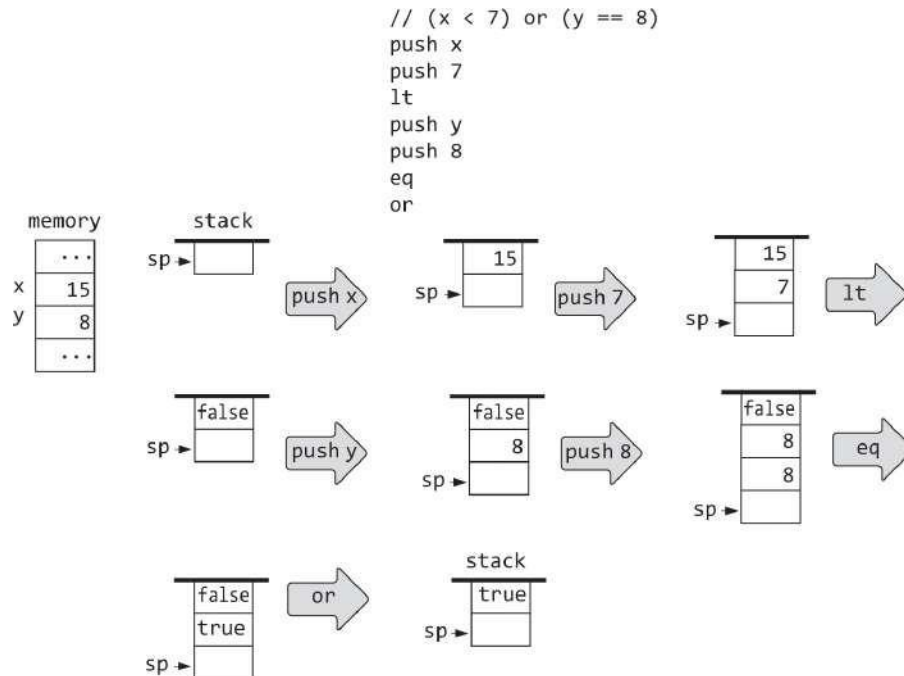


La evaluación basada en pilas de expresiones aritméticas generales es una extensión de la misma idea. Por ejemplo, considere la expresión  $d = (2 - x) + (y + 9)$ , tomada de algún programa de alto nivel. La evaluación basada en pilas de este

La expresión se muestra en [la Figura 7.3a](#). La evaluación basada en pilas de expresiones lógicas, que se muestra en [la figura 7.3b](#), sigue el mismo esquema.



**Figura 7.3a** Evaluación basada en pila de expresiones aritméticas.



**Figura 7.3b** Evaluación basada en pila de expresiones lógicas.

Tenga en cuenta que, desde la perspectiva de la pila, cada operación aritmética o lógica tiene el impacto neto de reemplazar los operandos de la operación con el resultado de la operación, sin afectar al resto de la pila. Esto es similar a cómo los humanos realizan el cálculo mental, utilizando nuestra memoria a corto plazo. Por ejemplo, cómo calcular el valor de  $3 \times (11 + 7) - 6$ ? Empezamos mentalmente sacando 11 y 7 de la expresión y calculando  $11 + 7$ . Luego conectamos el resultante de nuevo en la expresión, lo que produce  $3 \times 18 - 6$ . El efecto neto es ese  $(11 + 7)$  ha sido reemplazado por 18, y el resto de la expresión sigue siendo la misma que antes. Ahora podemos proceder a realizar operaciones mentales similares de pop-compute-and-push hasta que la expresión se reduzca a un solo valor. Estos ejemplos ilustran una virtud importante de las máquinas apiladas: cualquier aritmética y lógico expresión: no materia cómo complejo: puede convertirse sistemáticamente en una secuencia de operaciones simples en una pila y evaluarse mediante ellas. Por lo tanto, se puede escribir un *compilador* Eso traduce expresiones aritméticas y lógicas de alto nivel en secuencias de comandos de pila, como de hecho haremos en los capítulos 10 y 11. Una vez que las expresiones de alto nivel se han reducido a comandos de pila, podemos proceder a Evalúelos mediante una implementación de Stack Machine.

### 7.2.3 Segmentos de memoria virtual

Hasta ahora, en nuestros ejemplos de procesamiento de pilas, los comandos push/pop se ilustraron conceptualmente, utilizando la sintaxis push  $x$  y pop  $y$ , donde  $x$  e  $y$  se referían abstractamente a ubicaciones de memoria arbitrarias. Ahora pasamos a dar una descripción formal de nuestros comandos push y pop.

Los lenguajes de alto nivel presentan variables simbólicas como  $x$ ,  $y$ , suma, recuento, etc. Si el lenguaje está basado en objetos, cada una de estas variables puede ser una variable estática de nivel de clase, un *campo* de nivel de instancia de un objeto o una *variable local* o *de argumento a nivel de método*. En máquinas virtuales como la JVM de Java y en nuestro propio modelo de VM, no hay variables simbólicas. En su lugar, las variables se representan como entradas en segmentos de memoria virtual que tienen nombres como static, this, local y argument. En particular, como veremos en capítulos posteriores, el compilador mapea el primer, segundo, tercero, ... static que se encuentra en el programa de alto nivel en static 0, static 1, static 2, etc. Los otros tipos de variables se asignan de manera similar en los segmentos this, local y argument. Por ejemplo, si la variable local  $x$  y el campo  $y$  se han mapeado en el local 1 y este 3, respectivamente, el compilador traducirá  $x = y$  una instrucción de alto nivel como let en push this 3 seguido de pop local 1. En total, nuestro modelo de VM presenta ocho segmentos de memoria, cuyos nombres y roles se enumeran en [la figura 7.4](#).

| <i>Segment</i> | <i>Role</i>                                          |
|----------------|------------------------------------------------------|
| argument       | Represents the function's arguments                  |
| local          | Represents the function's local variables            |
| static         | Represents the static variables seen by the function |
| constant       | Represents the constant values 0,1,2,3, ..., 32767   |
| this           | Described in later chapters                          |
| that           | Described in later chapters                          |
| pointer        | Described in later chapters                          |
| temp           | Described in later chapters                          |

**Figura 7.4** Segmentos de memoria virtual.



Observamos de paso que los desarrolladores de implementaciones de VM no necesitan preocuparse por cómo el compilador asigna variables simbólicas en los segmentos de memoria virtual. Trataremos estos temas en detalle cuando desarrollemos el compilador en los capítulos 10 y 11. Por ahora, observamos que los comandos de VM acceden a todos los segmentos de memoria virtual exactamente de la misma manera: mediante el uso del *nombre del segmento* seguido de un índice no negativo.

---

## 7.3 Especificación de VM, Parte I

Nuestro modelo de VM se *basa en la pila*: todas las operaciones de VM toman sus operandos y almacenan sus resultados en la *pila*. Solo hay un tipo de datos: un entero de 16 bits con signo. Un *programa* de VM es una secuencia de comandos de VM que se dividen en cuatro categorías:

- *Los comandos push / pop* transfieren datos entre la pila y los segmentos de memoria.
- *Los comandos aritmético-lógicos* realizan operaciones aritméticas y lógicas.
- *Los comandos de bifurcación* facilitan las operaciones de bifurcación condicionales e incondicionales.
- *Los comandos de función* facilitan las operaciones de llamada y devolución de funciones.

La especificación e implementación de estos comandos abarca dos capítulos. En este capítulo nos centramos en los *comandos aritmético-lógicos* y *push/pop*. En el siguiente capítulo completamos la especificación de los comandos restantes.

**Comentarios y espacios en blanco:** Las líneas que comienzan con `;` se consideran comentarios y se ignoran. Se permiten líneas en blanco y se ignoran.

### Comandos Push / Pop

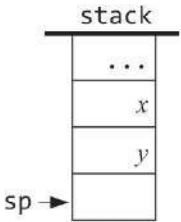
|                                 |                                                                                                                                                                                                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>push segment index</code> | Pushes the value of <code>segment[index]</code> onto the stack, where <code>segment</code> is argument, local, static, constant, this, that, pointer, or temp and <code>index</code> is a nonnegative integer. |
| <code>pop segment index</code>  | Pops the top stack value and stores it in <code>segment[index]</code> , where <code>segment</code> is argument, local, static, this, that, pointer, or temp and <code>index</code> is a nonnegative integer.   |

### Comandos aritmético-lógicos

- *Comandos aritméticos*: add, sub, neg
- *Comandos de comparación*: eq, gt, lt
- *Comandos lógicos*: y, o, no

Los comandos add, sub, eq, gt, lt y, y o tienen dos operandos implícitos. Para ejecutar cada uno de ellos, la implementación de la máquina virtual extrae dos valores de la parte superior de la pila, calcula la función indicada en ellos y devuelve el valor resultante a la pila (por *operando implícito* queremos decir que el operando no forma parte de la sintaxis del comando: dado que el comando está diseñado para operar siempre en los dos valores de la pila superior, no es necesario especificarlos). Los comandos neg y not restantes tienen un operando implícito y funcionan de la misma manera. [La Figura 7.5](#) da los detalles.

| Command | Computes           | Comment                                |
|---------|--------------------|----------------------------------------|
| add     | $x + y$            | integer addition (two's complement)    |
| sub     | $x - y$            | integer subtraction (two's complement) |
| neg     | $-y$               | arithmetic negation (two's complement) |
| eq      | $x == y$           | equality                               |
| gt      | $x > y$            | greater than                           |
| lt      | $x < y$            | less than                              |
| and     | $x \text{ And } y$ | bit-wise And                           |
| or      | $x \text{ Or } y$  | bit-wise Or                            |
| not     | $\text{Not } y$    | bit-wise Not                           |



**Figura 7.5** Los comandos aritmético-lógicos del lenguaje VM.

## 7.4 Implementación

La máquina virtual descrita hasta este punto es una abstracción. Si queremos usar esta VM de verdad, debe implementarse en alguna plataforma host real. Hay varias opciones de implementación, de las cuales describiremos una: un traductor de VM. El traductor de VM es un programa que traduce los comandos de VM en instrucciones de lenguaje de máquina. Escribir un programa de este tipo implica dos tareas principales. Primero, tenemos que decidir cómo representar la pila y los segmentos de memoria virtual en la plataforma de destino. En segundo lugar, tenemos que traducir cada comando de VM en una secuencia de instrucciones de bajo nivel que se puedan ejecutar en la plataforma de destino.

Por ejemplo, supongamos que la plataforma de destino es una máquina típica de von Neumann. En este caso, podemos representar la pila de la máquina virtual mediante un bloque de memoria designado en la RAM del host. El extremo inferior de este bloque de RAM será una dirección base fija y su extremo superior cambiará a medida que la pila crezca y se encoja. Por lo tanto, dada una dirección de pila fija, podemos administrar la pila realizando un seguimiento de una sola variable: un *puntero de pila*, o SP, que contiene la dirección de la entrada de RAM justo después del valor superior de la pila. Para inicializar la pila, establecemos SP en stackBase. A partir de este momento, cada comando `push x` puede ser implementado por las operaciones de pseudocódigo  $RAM[SP] = x$  seguidas de  $SP++$ , y cada comando `pop x` puede ser implementado por  $SP--$  seguido de  $x = RAM[SP]$ .

Supongamos que la plataforma host es la computadora Hack y que decidimos anclar la base de la pila en la dirección 256 en la RAM Hack. En ese caso, el traductor de VM puede comenzar generando código ensamblador que realiza

$SP = 256$ , es decir,  $@256, D=A, @SP, M=D$ . A partir de este momento, el traductor de VM puede controlar cada comando `push x` y `pop x` generando código ensamblador que realiza las operaciones  $RAM[SP++] = x$  y  $x = RAM[--SP]$ , respectivamente.

Con eso en mente, consideremos ahora la implementación de los comandos aritmético-lógicos de VM `add`, `sub`, `neg`, etc. Convenientemente, todos estos comandos comparten exactamente la misma lógica de acceso: sacar los operandos del comando de la pila, realizar un cálculo simple y enviar el resultado a la pila. Esto significa que una vez que descubramos cómo implementar los comandos `push` y `pop` de la máquina virtual, la implementación de los comandos aritmético-lógicos de la máquina virtual seguirá directamente.

#### 7.4.1 Mapeo estándar de VM en la plataforma de piratería, parte I

Hasta ahora en este capítulo, no hemos hecho ninguna suposición sobre la plataforma de destino en la que se implementará nuestra máquina virtual: todo se describió de forma abstracta. Cuando se trata de máquinas virtuales, esta independencia de la plataforma es el punto: no desea comprometer su máquina abstracta en ninguna plataforma de hardware en particular, precisamente porque desea que se ejecute potencialmente en *cualquier* plataforma, incluidas aquellas que aún no se construyeron o inventaron.

Por supuesto, en algún momento tenemos que implementar la abstracción de VM en una plataforma de hardware en particular (por ejemplo, en una de las plataformas de destino mencionadas en la [figura 7.1](#)). ¿Cómo debemos hacerlo? En principio, podemos hacer lo que nos plazca, siempre y cuando acabemos realizando la abstracción de VM de forma fiel y eficiente. Sin embargo, los arquitectos de VM normalmente publican pautas de implementación básicas, conocidas como *asignaciones estándar*, para diferentes plataformas de hardware. Con eso en mente, el resto de esta sección especifica el mapeo estándar de nuestra abstracción de VM en la computadora Hack. En lo que sigue, usamos los términos implementación de *VM* y *traductor de VM* indistintamente.

**Programa VM:** La definición completa de un *programa VM* se presentará en el próximo capítulo. Por ahora, vemos un programa de VM como una secuencia de comandos de VM almacenados en un archivo de texto denominado *FileName.vm* (el primer carácter del nombre de archivo debe ser una letra mayúscula y la extensión debe ser *vm*). El traductor de VM debe leer cada línea del archivo, tratarla como un comando de VM y traducirla a una o más instrucciones escritas en el lenguaje Hack. La salida resultante, una secuencia de instrucciones de ensamblaje de Hack, debe almacenarse en un archivo de texto denominado *FileName.asm* (el nombre del archivo es idéntico al del archivo de origen; la extensión debe ser *asm*). Cuando el ensamblador Hack lo traduce a código binario o se ejecuta tal cual en un emulador de CPU Hack, esto *.asm* debe realizar la semántica exigida por el programa de máquina virtual de origen.

**Tipo de datos:** la abstracción de la máquina virtual solo tiene un tipo de datos: un entero con signo. Este tipo se implementa en la plataforma Hack como un valor de 16 bits de complemento de dos. Los valores booleanos *de VM true* y *false* se representan como  $-1$  y  $0$ , respectivamente.

**Uso de RAM:** El host Hack RAM consta de 32K palabras de 16 bits. Las implementaciones de VM deben usar la parte superior de este espacio de direcciones de la siguiente manera:

| <i>RAM addresses</i> | <i>Usage</i>                                     |
|----------------------|--------------------------------------------------|
| 0 – 15               | Sixteen virtual registers, usage described below |
| 16 – 255             | Static variables                                 |
| 256 – 2047           | Stack                                            |

Recuerde que de acuerdo con la *especificación del lenguaje de máquina Hack* (capítulo 6), se puede hacer referencia a las direcciones RAM 0 a 4 usando los símbolos SP, LCL, ARG, THIS y THAT. Esta convención se introdujo en el lenguaje ensamblador con previsión, para ayudar a los desarrolladores de implementaciones de VM a escribir código legible. El uso esperado de estas direcciones en la implementación de VM es el siguiente:

| <i>Name</i>       | <i>Location</i> | <i>Usage</i>                                                                                                   |
|-------------------|-----------------|----------------------------------------------------------------------------------------------------------------|
| SP                | RAM[0]          | <i>Stack Pointer</i> : the memory address just following the memory address containing the topmost stack value |
| LCL               | RAM[1]          | Base address of the local segment                                                                              |
| ARG               | RAM[2]          | Base address of the argument segment                                                                           |
| THIS              | RAM[3]          | Base address of the this segment                                                                               |
| THAT              | RAM[4]          | Base address of the that segment                                                                               |
| TEMP              | RAM[5–12]       | Holds the temp segment                                                                                         |
| R13<br>R14<br>R15 | RAM[13–15]      | If the assembly code generated by the VM translator needs variables, it can use these registers.               |

Cuando decimos *dirección base* de un segmento, nos referimos a una dirección física en la RAM del host. Por ejemplo, si deseamos mapear el segmento local en el segmento de RAM física a partir de la dirección 1017, podemos escribir código Hack que establezca LCL en 1017. Señalamos de paso que decidir dónde ubicar los segmentos de memoria virtual en la RAM del host es un tema delicado. Por ejemplo, cada vez que una función comienza a ejecutarse, tenemos que asignar espacio de RAM para contener sus segmentos de memoria local y de argumentos. Y cuando la función llama a otra función, tenemos que poner estos segmentos en espera y asignar espacio RAM adicional para representar los segmentos de la función llamada, y así sucesivamente. ¿Cómo podemos asegurarnos de que estos segmentos de memoria abiertos no se desborden entre sí y en otras áreas reservadas de RAM? Estos desafíos de administración de memoria se abordarán en el próximo capítulo, cuando implementaremos los comandos de llamada y devolución de funciones del lenguaje VM.

Por ahora, sin embargo, ninguno de estos problemas de asignación de memoria debería molestarnos. En su lugar, debe suponer que SP, ARG, LCL, THIS y THAT ya se han inicializado en algunas direcciones sensibles en la RAM del host. Tenga en cuenta que las implementaciones de máquinas virtuales nunca ven estas direcciones de todos modos. Más bien, los manipulan simbólicamente, utilizando los nombres de puntero. Por ejemplo, supongamos que queremos empujar el valor del registro D a la pila. Esta operación se puede implementar utilizando la lógica  $RAM[SP++] = D$ , que se puede expresar en el ensamblado Hack como `@SP, A=M, M=D, @SP, M=M+1`. Este código ejecutará la operación de inserción perfectamente, sin saber dónde se encuentra la pila en la RAM del host ni cuál es el valor actual del puntero de la pila.

Sugerimos tomarse unos minutos para digerir el código ensamblador que se acaba de mostrar. Si no lo entiende, debe actualizar sus conocimientos sobre la manipulación de punteros en el lenguaje ensamblador Hack (sección 4.3, ejemplo 3). Este conocimiento es un requisito previo para desarrollar el traductor de VM, ya que la traducción de cada comando de VM implica generar código en el lenguaje ensamblador Hack.

## Asignación de segmentos de memoria

**Local, argumento, esto, aquello:** En el siguiente capítulo analizamos cómo la implementación de la máquina virtual asigna estos segmentos dinámicamente en la RAM del host. Por ahora, todo lo que tenemos que saber es que las direcciones base de estos segmentos se almacenan en los registros LCL, ARG, THIS y THAT, respectivamente. Por lo tanto, cualquier acceso a la  $i$ -ésima entrada de un segmento virtual (en el contexto de un comando VM "push / pop *segmentName i*") debe traducirse en código ensamblador que acceda a la dirección  $(base + i)$  en la RAM, donde *base* es uno de los punteros LCL, ARG, THIS o THAT.

**Puntero:** A diferencia de los segmentos virtuales descritos anteriormente, el segmento de puntero contiene exactamente dos valores y se asigna directamente a las ubicaciones de RAM 3 y 4. Recuerde que estas ubicaciones de RAM también se llaman, respectivamente, ESTO y AQUELLO. Por lo tanto, la semántica del segmento de puntero es la siguiente. Cualquier acceso al puntero 0 debe dar lugar al acceso al puntero THIS, y cualquier acceso al puntero 1 debe dar lugar al acceso al puntero THAT. Por ejemplo, el puntero pop 0 debe establecer THIS en el valor despuntado y el puntero push 1 debe empujar

en la pila el valor actual de THAT. Esta peculiar semántica tendrá mucho sentido cuando escribamos el compilador en los capítulos 10 y 11, así que estad atentos.

**Temp:** Este segmento de 8 palabras también es fijo y se asigna directamente en las ubicaciones de RAM 5 – 12. Con eso en mente, cualquier acceso a temp  $i$ , donde  $i$  varía de 0 a 7, debe traducirse en código ensamblador que acceda a la ubicación de RAM

$5 + i$ .

**Constante:** Este segmento de memoria virtual es verdaderamente virtual, ya que no ocupa ningún espacio físico de RAM. En su lugar, la implementación de la máquina virtual controla cualquier acceso a la constante  $i$  simplemente proporcionando la constante  $i$ . Por ejemplo, el comando push constant 17 debe traducirse en código ensamblador que inserte el valor 17 en la pila.

**Estático:** las variables estáticas se asignan en las direcciones 16 a 255 de la RAM del host. El traductor de VM puede realizar esta asignación automáticamente, ya que

Sigue. Cada referencia a `estático  $i$`  en un programa de VM `Foo.vm` en la se traducirá al símbolo de ensamblador almacenado en un archivo `tar`

De acuerdo con la especificación del *lenguaje de máquina Hack* (capítulo 6), el ensamblador Hack mapeará estas variables simbólicas en la RAM del host, comenzando en la dirección 16. Esta convención hará que las variables estáticas que aparecen en un programa de máquina virtual se asignen a las direcciones 16 y posteriores, *en el orden en que aparecen en el código de la máquina virtual*. Por ejemplo, supongamos que un programa de máquina virtual comienza con la constante de inserción de código 100, la constante de inserción 200, la estática pop 5, la estática pop 2. El esquema de traducción descrito anteriormente hará que el estático 5 y el estático 2 se asignen a las direcciones RAM 16 y 17, respectivamente.

Esta implementación de variables estáticas es algo tortuosa, pero funciona bien. Hace que las variables estáticas de diferentes archivos de VM coexistan sin mezclarse, ya que su *FileName* generado. Los símbolos  $i$  tienen nombres de archivo de prefijo únicos. Observamos para concluir que, dado que la pila comienza en la dirección 256, la implementación limita el número de variables estáticas en un programa Jack a

$255 - 16 + 1 = 240$ .



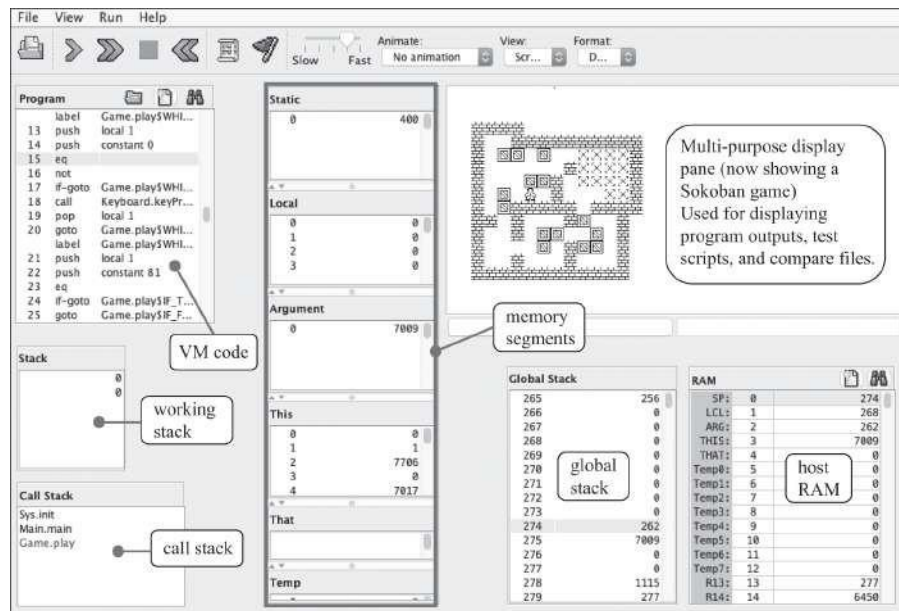
**Símbolos del lenguaje ensamblador:** Resumamos todos los símbolos especiales mencionados anteriormente. Supongamos que el programa de VM que tenemos que traducir está almacenado en un archivo llamado Foo.vm. Traductores de VM que cumplen con el estándar

*La asignación de máquinas virtuales en la plataforma Hack genera código ensamblador que usa los siguientes símbolos: SP, LCL, ARG, THIS, THAT y Foo.i, donde i es un entero no negativo. Si necesitan generar código que use variables para el almacenamiento temporal, los traductores de máquinas virtuales pueden usar los símbolos R13, R14 y R15.*

## **7.4.2 El emulador de VM**

Una forma relativamente sencilla de implementar una máquina virtual es escribir un programa de alto nivel que represente la pila y los segmentos de memoria e implemente todos los comandos de la máquina virtual mediante programación de alto nivel. Por ejemplo, si representamos la pila usando una matriz suficientemente grande llamada `stack`, las operaciones `push` y `pop` se pueden realizar directamente usando declaraciones de alto nivel como `stack[SP++] = x` y `x = stack[--SP]`, respectivamente. Los segmentos de memoria virtual también se pueden manejar mediante matrices.

Si queremos que este programa de emulación de VM sea elegante, podemos aumentarlo con una interfaz gráfica, lo que permite a los usuarios experimentar con comandos de VM e inspeccionar visualmente su impacto en las imágenes de la pila y los segmentos de memoria. El paquete de software *Nand to Tetris* incluye uno de estos emuladores, escrito en Java (ver [figura 7.6](#)). Este práctico programa permite cargar y ejecutar código de VM tal cual y observar visualmente, durante el tiempo de ejecución simulado, cómo los comandos de VM afectan los estados de la pila emulada y los segmentos de memoria. Además, el emulador muestra cómo se asignan la pila y los segmentos de memoria en la RAM del host y cómo cambia el estado de la RAM cuando se ejecutan los comandos de la máquina virtual. El emulador de VM suministrado es un programa genial, ¡pruébalo!



**Figura 7.6** El emulador de VM suministrado con el paquete de software Nand to Tetris.

### 7.4.3 Sugerencias de diseño para la implementación de VM

**Uso:** el traductor de VM acepta un único argumento de línea de comandos, como se indica a continuación:

prompt> origen de VMTranslator

donde *source* es un nombre de archivo con el formato *ProgName.vm*. El nombre del archivo puede contener una ruta de archivo. Si no se especifica ninguna ruta de acceso, el traductor de máquina virtual funciona en la carpeta actual. El primer carácter del nombre de archivo debe ser una letra mayúscula y la extensión *vm* es obligatoria. El archivo contiene una secuencia de uno o varios comandos de máquina virtual. En respuesta, el traductor crea un archivo de salida, denominado *ProgName.asm*, que contiene las instrucciones de ensamblado que realizan los comandos de máquina virtual. El archivo de salida *ProgName.asm* se almacena en la misma carpeta que la de la entrada. Si el archivo *ProgName.asm* ya existe, se sobrescribirá.

### Estructura del programa

Proponemos implementar el traductor de VM utilizando tres módulos: un programa principal llamado VMTranslator, un analizador y un codificador. El trabajo del analizador es

dar sentido a cada comando de VM, es decir, comprender lo que el comando busca hacer. El trabajo del CodeWriter es traducir el comando VM entendido en instrucciones de ensamblaje que realicen la operación deseada en la plataforma Hack. VMTranslator impulsa el proceso de traducción.

## **El analizador**

Este módulo controla el análisis de un único archivo .vm. El analizador proporciona servicios para leer un comando de máquina virtual, desempaquetar el comando en sus diversos componentes y proporcionar un acceso conveniente a estos componentes. Además, el analizador ignora todos los espacios en blanco y comentarios. El analizador está diseñado para manejar todos los comandos de VM, incluidos los *comandos de ramificación y función* que se implementarán en el capítulo 8.

| <i>Routine</i>            | <i>Arguments</i>    | <i>Returns</i>                                                                                 | <i>Function</i>                                                                                                                                                                                                |
|---------------------------|---------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Constructor / initializer | Input file / stream | —                                                                                              | Opens the input file / stream, and gets ready to parse it.                                                                                                                                                     |
| hasMoreLines              | —                   | boolean                                                                                        | Are there more lines in the input?                                                                                                                                                                             |
| advance                   | —                   | —                                                                                              | <p>Reads the next command from the input and makes it the current command.</p> <p>This routine should be called only if hasMoreLines is true.</p> <p>Initially there is no current command.</p>                |
| commandType               | —                   | C_ARITHMETIC, C_PUSH, C_POP, C_LABEL, C_GOTO, C_IF, C_FUNCTION, C_RETURN, C_CALL<br>(constant) | <p>Returns a constant representing the type of the current command.</p> <p>If the current command is an arithmetic-logical command, returns C_ARITHMETIC.</p>                                                  |
| arg1                      | —                   | string                                                                                         | <p>Returns the first argument of the current command.</p> <p>In the case of C_ARITHMETIC, the command itself (add, sub, etc.) is returned.</p> <p>Should not be called if the current command is C_RETURN.</p> |
| arg2                      | —                   | int                                                                                            | <p>Returns the second argument of the current command.</p> <p>Should be called only if the current command is C_PUSH, C_POP, C_FUNCTION, or C_CALL.</p>                                                        |

Por ejemplo, si el comando actual es push local 2, entonces llamar a arg1() y arg2() devolvería, respectivamente, "local" y 2. Si el comando actual es add, entonces llamar a arg1() devolvería "add", y arg2() no sería llamado.

## El Escritor de Código

Este módulo traduce un comando de VM analizado en código ensamblador Hack.

| <i>Routine</i>                                                        | <i>Arguments</i>                                         | <i>Returns</i> | <i>Function</i>                                                                                   |
|-----------------------------------------------------------------------|----------------------------------------------------------|----------------|---------------------------------------------------------------------------------------------------|
| Constructor / initializer                                             | Output file / stream                                     | —              | Opens an output file / stream and gets ready to write into it.                                    |
| writeArithmetic                                                       | command (string)                                         | —              | Writes to the output file the assembly code that implements the given arithmetic-logical command. |
| writePushPop                                                          | command (C_PUSH or C_POP), segment (string), index (int) | —              | Writes to the output file the assembly code that implements the given push or pop command.        |
| close                                                                 | —                                                        | —              | Closes the output file / stream.                                                                  |
| More code writing routines will be added to this module in chapter 8. |                                                          |                |                                                                                                   |

Por ejemplo, llamar a `writePushPop (C_PUSH,"local",2)` generaría instrucciones de ensamblado que implementan el comando VM `push local 2`. Otro ejemplo: Llamar a `WriteArithmetic("add")` daría como resultado la generación de instrucciones de ensamblado que extraen los dos elementos superiores de la pila, los suman y empujan el resultado a la pila.

## El traductor de VM

Este es el programa principal que impulsa el proceso de traducción, utilizando los servicios de un analizador y un codificador. El programa obtiene el nombre del archivo fuente de entrada, digamos *Prog.vm*, del argumento de la línea de comandos. Construye un analizador para analizar el archivo de entrada *Prog.vm* y crea un archivo de salida, *Prog.asm*, en el que escribirá las instrucciones de ensamblado traducidas. A continuación, el programa entra en un bucle que itera a través de los comandos de VM en el archivo de entrada. Para cada comando, el programa usa los servicios Parser y CodeWriter para analizar el comando en sus campos y luego generar a partir de ellos una secuencia de instrucciones de ensamblado. Las instrucciones se escriben en el *archivo Prog.asm* de salida.

No proporcionamos ninguna API para este módulo, invitándolo a implementarlo como mejor le parezca.

## Consejos de implementación

1. Al comenzar a traducir un comando de máquina virtual, por ejemplo, `push local 2`, considere la posibilidad de generar y emitir al flujo de código ensamblado de salida, un

- Comentario como `Push Local 2`. Estos comentarios le ayudarán a leer el código generado y a depurar su traductor si es necesario.
2. Casi todos los comandos de VM necesitan insertar datos en la pila o sacarlos de la pila. Por lo tanto, las rutinas `writeXxx` deberán generar instrucciones de ensamblado similares una y otra vez. Para evitar escribir código repetitivo, considere la posibilidad de escribir y usar rutinas privadas (a veces denominadas *métodos* auxiliares) que generen estos fragmentos de código de uso frecuente.
  3. Como se explicó en el capítulo 6, se recomienda finalizar cada programa de lenguaje de máquina con un bucle infinito. Por lo tanto, considere la posibilidad de escribir una rutina privada que escriba el código de bucle infinito en el ensamblado. Llame a esta rutina una vez, cuando haya terminado de traducir todos los comandos de VM.

---

## 7.5 Proyecto

Básicamente, debe escribir un programa que lea los comandos de la máquina virtual, un comando a la vez, y traduzca cada comando en instrucciones de Hack. Por ejemplo, ¿cómo debemos manejar el comando `VM push local 2`? *Consejo:* Deberíamos escribir varias instrucciones de ensamblaje de Hack que, entre otras cosas, manipulen los punteros `SP` y `LCL`. Crear una secuencia de instrucciones de Hack que realice cada uno de los comandos aritmético-lógicos y `push/pop` de VM es la esencia misma de este proyecto. De eso se trata la generación de código.

Le recomendamos que comience escribiendo y probando estos fragmentos de código ensamblador en papel. Dibuje un segmento de RAM, dibuje una tabla de seguimiento que registre los valores de, por ejemplo, `SP` y `LCL`, e inicialice estas variables en direcciones de memoria arbitrarias. Ahora, rastree en papel el código ensamblador que cree que realiza, por ejemplo, `empuje local 2`. ¿El código afecta correctamente a la pila y a los segmentos locales (en cuanto a RAM)? ¿Te acordaste de actualizar el puntero de pila? Y así sucesivamente. Una vez que se sienta seguro de que los fragmentos de código ensamblador hacen su trabajo correctamente, puede hacer que CodeWriter los genere, casi tal cual.

Dado que su traductor de VM tiene que escribir código ensamblador, debe flexionar sus músculos de programación de Hack de bajo nivel. La mejor manera de hacerlo es revisando los ejemplos de programas de ensamblaje en el capítulo 4 y los programas

que escribiste en el Proyecto 4. Si necesita consultar la documentación del lenguaje ensamblador Hack, consulte la sección 4.2.

**Objetivo:** Crear un traductor de VM básico diseñado para implementar el *aritmético-lógico* y *push / pop* del lenguaje VM.

Esta versión del traductor de máquina virtual supone que el código de máquina virtual de origen no tiene errores. La comprobación de errores, la generación de informes y el control se pueden agregar a versiones posteriores del traductor de VM, pero no forman parte del proyecto 7.

**Recursos:** Necesitará dos herramientas: el lenguaje de programación en el que implementará el traductor de VM y el emulador de *CPU* suministrado en su carpeta `nand2tetris/tools`. El emulador de CPU le permitirá ejecutar y probar el código ensamblador generado por su traductor. Si el código generado se ejecuta correctamente en el emulador de CPU, asumiremos que el traductor de VM funciona según lo esperado. Esta es solo una prueba parcial del traductor, pero será suficiente para nuestros propósitos.

Otra herramienta que resulta útil en este proyecto es el *emulador VM*, también suministrado en su carpeta `nand2tetris/tools`. Recomendamos usar este programa para ejecutar los programas de prueba proporcionados y observar cómo el código de la máquina virtual afecta los estados (simulados) de la pila y los segmentos de memoria virtual. Por ejemplo, supongamos que un programa de prueba inserta algunas constantes en la pila y luego las coloca en el segmento local. Puede ejecutar el programa de prueba en el emulador de máquina virtual, inspeccionar cómo crece y se reduce la pila y ver cómo se rellena el segmento local con valores. Esto puede ayudarle a comprender qué acciones se supone que debe generar el traductor de VM antes de comenzar a implementarlo.

**Contrato:** Escriba un traductor de VM a Hack que se ajuste a la especificación de VM dada en la sección 7.3 y al mapeo estándar de VM en la plataforma Hack dado en la sección 7.4.1. Utilice su traductor para traducir los programas de VM de prueba suministrados, produciendo los programas correspondientes escritos en el lenguaje ensamblador Hack. Cuando se ejecutan en el emulador de CPU suministrado, los programas de ensamblado generados por su traductor deben entregar los resultados exigidos por los scripts de prueba y los archivos de comparación proporcionados.

## Etapas de prueba e implementación



Ofrecemos cinco programas de VM de prueba. Le recomendamos que desarrolle y pruebe su traductor en evolución en los programas de prueba en el orden en que se dan. De esta manera, se le guiará implícitamente para construir las capacidades de generación de código del traductor gradualmente, de acuerdo con las demandas presentadas por cada programa de prueba.

**SimpleAdd:** Este programa inserta dos constantes en la pila y las suma. Prueba cómo su implementación maneja los comandos `, push`, `constant`, `i` y `add`.

**StackTest:** Inserta algunas constantes en la pila y prueba cómo su implementación maneja todos los comandos aritmético-lógicos.

**BasicTest:** ejecuta comandos `push`, `pop` y aritméticos utilizando los segmentos de memoria `constant`, `local`, `argument`, `this`, `that` y `temp`. Prueba cómo la implementación controla estos segmentos de memoria (ya ha controlado `constant`).

**PointerTest:** ejecuta comandos `push`, `pop` y aritméticos mediante el puntero de segmentos de memoria, `this` y `that`. Prueba cómo la implementación controla el segmento de puntero.

**StaticTest:** ejecuta comandos `push`, `pop` y aritméticos mediante constantes y el segmento de memoria `static`. Prueba cómo la implementación controla el segmento estático.

**Inicialización:** para que cualquier programa de VM traducido comience a ejecutarse, debe incluir código de inicio que obligue al código ensamblador generado a comenzar a ejecutarse en la plataforma host. Y, antes de que este código comience a ejecutarse, la implementación de la máquina virtual debe anclar las direcciones base de la pila y los segmentos de memoria virtual en ubicaciones de RAM seleccionadas. Ambos problemas (inicializaciones de código de inicio y segmentos) se describen e implementan en el capítulo siguiente. La dificultad aquí es que necesitamos estas inicializaciones para ejecutar los programas de prueba en este proyecto. La buena noticia es que no debe preocuparse por estos detalles, ya que todas las inicializaciones necesarias para este proyecto son manejadas "manualmente" por los scripts de prueba suministrados.

**Pruebas / Depuración:** Suministramos cinco conjuntos de programas de prueba, scripts de prueba y archivos de comparación. Para cada programa de prueba *Xxx.vm* recomendamos seguir estos pasos:

0. Use el script *XxxVME.tst* para ejecutar el programa de prueba *Xxx.vm* en el emulador de máquina virtual proporcionado. Esto lo familiarizará con el comportamiento previsto del programa de prueba. Inspeccione la pila simulada y los segmentos virtuales, y asegúrese de que comprende lo que está haciendo el programa de prueba.
1. Utilice su traductor parcialmente implementado para traducir el archivo *Xxx.vm* (programa de prueba). El resultado debe ser un archivo de texto llamado *Xxx.asm*, que contenga el código ensamblador Hack generado por su traductor.
2. Inspeccione el código *Xxx.asm* generado por su traductor. Si hay errores visibles, depure y corrija su traductor.
3. Utilice los archivos *Xxx.tst* y *Xxx.cmp* proporcionados para ejecutar y probar el programa *Xxx.asm* traducido en el emulador de CPU suministrado. Si hay algún error, depure y corrija su traductor.

Cuando haya terminado con este proyecto, asegúrese de guardar una copia del traductor de VM. En el siguiente capítulo se le pedirá que amplíe este programa, agregando el manejo de más comandos de VM. Si las modificaciones de tu proyecto 8 terminan rompiendo el código desarrollado en el proyecto 7, podrás recurrir a tu versión de respaldo.

**Una versión basada en la web del proyecto 7** está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

---

## 7.6 Perspectiva

En este capítulo comenzamos el proceso de desarrollo de un compilador para un lenguaje de alto nivel. Siguiendo las prácticas modernas de ingeniería de software, hemos optado por basar el compilador en un modelo de compilación de dos niveles. En el *nivel de front-end*, que se trata en los capítulos 10 y 11, el código de alto nivel se traduce en un código intermedio diseñado para ejecutarse en una máquina virtual. En el *nivel de back-end*, que se trata en este capítulo y en el siguiente, el código intermedio es

traducido al lenguaje de máquina de una plataforma de hardware de destino (ver [Figura 7.1](#)).

A lo largo de los años, este modelo de compilación en dos etapas se ha utilizado, implícita y explícitamente, en muchos proyectos de construcción de compiladores. A fines de la década de 1970, las corporaciones IBM y Apple introdujeron dos computadoras personales pioneras y fenomenalmente exitosas, conocidas como IBM PC y Apple II. Un lenguaje de alto nivel que se hizo popular en estas primeras PC fue Pascal. Por desgracia, se tuvieron que desarrollar diferentes compiladores Pascal, ya que las máquinas de IBM y Apple usaban diferentes procesadores, diferentes lenguajes de máquina y diferentes sistemas operativos. Además, IBM y Apple eran empresas rivales y no tenían interés en ayudar a los desarrolladores a portar su software a la otra máquina. Como resultado, los desarrolladores de software que querían que sus aplicaciones Pascal se ejecutaran en ambas líneas de computadoras tuvieron que usar diferentes compiladores, cada uno diseñado para generar código binario específico de la máquina. ¿No podría haber una mejor manera de manejar la compilación multiplataforma, de modo que los programas puedan escribirse una vez y ejecutarse en todas partes?

Una solución a este desafío fue un marco de máquina virtual temprano llamado *p-code*. La idea básica era compilar programas Pascal en código p intermedio (similar a nuestro lenguaje VM) y luego usar una implementación para traducir el código p abstracto en el conjunto de instrucciones x86 de Intel, utilizado por las PC de IBM, y otra implementación para traducir el mismo código p en el conjunto de instrucciones 68000 de Motorola, utilizado por las computadoras Apple. Mientras tanto, otras compañías desarrollaron compiladores Pascal altamente optimizados que generaron código p eficiente. El resultado neto fue que el mismo programa Pascal podía ejecutarse tal cual en prácticamente todas las máquinas de este nascente mercado de PC: sin importar qué computadora usaran sus clientes, podía enviarles exactamente los mismos archivos de código p, obviando la necesidad de usar múltiples compiladores. Por supuesto, todo el esquema se basaba en la suposición de que la computadora del cliente estaba equipada con una implementación de código p del lado del cliente (equivalente a nuestro traductor de VM). Para que esto sucediera, las implementaciones de código p se distribuyeron gratuitamente a través de Internet y se invitó a los clientes a descargarlas en sus computadoras. Históricamente, esta fue quizás la primera vez que la noción de un lenguaje de alto nivel multiplataforma comenzó a desarrollar todo su potencial.

Tras el crecimiento explosivo de Internet y los dispositivos móviles a mediados de la década de 1990, la compatibilidad multiplataforma se convirtió en un problema universalmente desconcertante.

Para abordar el problema, la compañía Sun Microsystems (luego adquirida por Oracle) buscó desarrollar un nuevo lenguaje de programación cuyo código compilado podría ejecutarse tal cual en cualquier computadora y dispositivo digital conectado a Internet. El lenguaje que surgió de esta iniciativa, *Java*, se fundó en un modelo de ejecución de código intermedio llamado *Java Virtual Machine*, o JVM.

La JVM es una especificación que describe un lenguaje intermedio llamado *bytecode*, el lenguaje de VM de destino de los compiladores de Java. Los archivos escritos en código de bytes se utilizan ampliamente para la distribución de código de programas Java a través de Internet. Para ejecutar estos programas portátiles, las PC cliente, tabletas y teléfonos celulares que los descargan deben estar equipados con implementaciones JVM adecuadas, conocidas como JRE (Java Runtime Environments). Estos programas están ampliamente disponibles para numerosas combinaciones de procesadores y sistemas operativos. Hoy en día, muchos propietarios de computadoras personales y teléfonos celulares utilizan estos programas de infraestructura (los JRE) de manera rutinaria e implícita, sin darse cuenta de su existencia en sus dispositivos.

El lenguaje Python, concebido a fines de la década de 1980, también se basa en un modelo de traducción de dos niveles, cuya pieza central es la PVM (Python Virtual Machine), que utiliza su propia versión de código de bytes.

A principios de la década de 2000, Microsoft lanzó su .NET Framework. La pieza central de .NET es un marco de máquina virtual llamado *Common Language Runtime* (CLR). Según la visión de Microsoft, muchos lenguajes de programación, como C # y C ++, se pueden compilar en código intermedio que se ejecuta en CLR. Esto permite que el código escrito en diferentes lenguajes interopere y comparta las bibliotecas de software de un entorno de tiempo de ejecución común. Por supuesto, los compiladores de un solo nivel para C y C++ todavía se usan ampliamente, especialmente en aplicaciones de alto rendimiento que requieren un código ajustado y optimizado.

De hecho, un tema que se eludió por completo en este capítulo es *la eficiencia*. Nuestro contrato requiere desarrollar un traductor de VM, sin requerir que el código ensamblador generado sea eficiente. Claramente, este es un descuido serio. El traductor de VM es una tecnología habilitadora de misión crítica, que se encuentra en el núcleo de su PC, tableta o teléfono celular: si genera código de bajo nivel ajustado y eficiente, las aplicaciones se ejecutarán en su máquina rápidamente, utilizando la menor cantidad de recursos de hardware posible. Por lo tanto, la optimización del traductor de

VM es una prioridad absoluta.

En general, existen numerosas oportunidades para optimizar el traductor de VM. Por ejemplo, las asignaciones como `let x = y` prevalecen en el código de alto nivel; el compilador traduce estas instrucciones en comandos de máquina virtual como, por ejemplo, `push local 3` seguido de `pop static 1`. Las implementaciones inteligentes de estos pares de comandos de VM pueden generar código ensamblador que elude la pila por completo, lo que resulta en ganancias de rendimiento dramáticas. Por supuesto, este es uno de los muchos ejemplos de posibles optimizaciones de VM. De hecho, a lo largo de los años, las implementaciones de VM de Java, Python y C# se volvieron dramáticamente más poderosas y sofisticadas.

Para terminar, señalamos que un ingrediente crucial que debe agregarse al modelo de máquina virtual antes de que se libere todo su potencial es una biblioteca de software común. De hecho, la máquina virtual Java viene con la *biblioteca de clases Java estándar*, y el .NET Framework de Microsoft viene con la *biblioteca de clases Framework*. Estas vastas bibliotecas de software se pueden ver como sistemas operativos portátiles, que brindan numerosos servicios como administración de memoria, kits de herramientas GUI, funciones de cadena, funciones matemáticas, etc. Estas extensiones se describirán y construirán en el capítulo 12.

---

## 8 Máquina virtual II: Control

Si todo parece estar bajo control, simplemente no vas lo suficientemente rápido.

—Mario Andretti (n. 1940), campeón de carreras de autos

El capítulo 7 introdujo la noción de una *máquina virtual* (VM), y el proyecto 7 comenzó a implementar nuestra máquina virtual abstracta y el lenguaje VM sobre la plataforma Hack. La implementación implicó el desarrollo de un programa que traduce los comandos de VM en código ensamblador Hack. Específicamente, en el capítulo anterior aprendimos a usar e implementar los comandos aritmético-lógicos y los comandos push/pop de la máquina virtual; en este capítulo aprenderemos a usar e implementar los comandos de bifurcación y los comandos de función de la máquina virtual. A medida que avanza el capítulo, ampliaremos el traductor básico desarrollado en el proyecto 7, terminando con un traductor de VM a gran escala sobre la plataforma Hack. Este traductor servirá como módulo back-end del compilador que construiremos en los capítulos 10 y 11.

En cualquier concurso de Great Gems in Applied Computer Science, el procesamiento de pilas será un fuerte finalista. El capítulo anterior mostró cómo las expresiones aritméticas y booleanas pueden ser representadas y evaluadas por operaciones elementales en una pila. Este capítulo continúa mostrando cómo esta estructura de datos notablemente simple también puede admitir tareas notablemente complejas como la llamada a funciones anidadas, el paso de parámetros, la recursión y las diversas tareas de asignación y reciclaje de memoria necesarias para admitir la ejecución del programa durante el tiempo de ejecución. La mayoría de los programadores tienden a dar por sentado estos servicios, esperando que el compilador y el sistema operativo los entreguen, de una forma u otra. Ahora estamos en condiciones de abrir esta caja negra y ver cómo se realizan realmente estos mecanismos de programación fundamentales.



**Sistema en tiempo de ejecución:** cada sistema informático debe especificar un modelo en tiempo de ejecución. Este modelo responde a preguntas esenciales sin las cuales los programas no pueden ejecutarse: cómo iniciar la ejecución de un programa, qué debe hacer la computadora cuando un programa termina, cómo pasar argumentos de una función a otra, cómo asignar recursos de memoria a funciones en ejecución, cómo liberar recursos de memoria cuando ya no son necesarios, etc.

En Nand to Tetris, estos problemas se abordan mediante la especificación del *lenguaje VM*, junto con la *asignación estándar en la especificación de la plataforma Hack*. Si se desarrolla un traductor de VM de acuerdo con estas directrices, terminará realizando un sistema ejecutable en tiempo de ejecución. En particular, el traductor de VM no solo traducirá los comandos de VM (push, pop, add, etc.) en instrucciones de ensamblado, sino que también generará código ensamblador que realiza un sobre en el que se ejecuta el programa. Todas las preguntas mencionadas anteriormente (cómo iniciar un programa, cómo administrar el comportamiento de llamada y devolución de funciones, etc.) se responderán generando código ensamblador habilitador que envuelva el código propiamente dicho. Veamos un ejemplo.

---

## 8.1 Magia de alto nivel

Los lenguajes de alto nivel permiten escribir programas en términos de alto nivel. Por ejemplo, una expresión como  $x = -b + \sqrt{b^2 - 4 \cdot a \cdot c}$  se puede escribir como `x = -b + sqrt(power(b,2) - 4 * a * c)`, que es casi tan descriptiva como la real. Tenga en cuenta la diferencia entre operaciones primitivas como `+` y `-` y funciones como `sqrt` y `power`. Los primeros están integrados en la sintaxis básica del lenguaje de alto nivel. Estos últimos son extensiones del lenguaje básico.

La capacidad ilimitada para ampliar el lenguaje a voluntad es una de las características más importantes de los lenguajes de programación de alto nivel. Por supuesto, en algún momento, alguien debe implementar funciones como `sqrt` y `power`. Sin embargo, la historia de la implementación de estas abstracciones está completamente separada de la historia de su uso. Por lo tanto, los programadores de aplicaciones pueden suponer que cada una de estas funciones se ejecutará, de alguna manera, y que después de su ejecución, el control volverá, de alguna manera, a la siguiente operación en el código de uno. Los comandos de ramificación dotan al lenguaje de un poder expresivo adicional, lo que permite escribir código condicional como si !

$(a == 0) \{ x = (-b + \sqrt{\text{power}(b, 2) - 4 * a * c}) / (2 * a) \}$  else  $\{ x = -c / b \}$ . Una vez

Nuevamente, vemos que el código de alto nivel permite la expresión de lógica de alto nivel

—en este caso el algoritmo para resolver ecuaciones cuadráticas— casi directamente.

De hecho, los lenguajes de programación modernos son fáciles de programar y ofrecen abstracciones útiles y poderosas. Sin embargo, es un pensamiento aleccionador que no importa cuán alto permitamos que sea nuestro lenguaje de alto nivel, al final del día debe realizarse en alguna plataforma de hardware que solo pueda ejecutar instrucciones de máquina primitivas. Por lo tanto, entre otras cosas, los arquitectos de compiladores y máquinas virtuales deben encontrar soluciones de bajo nivel para realizar comandos de llamada y devolución de funciones y ramificaciones.

*Las funciones*, el pan y la mantequilla de la programación modular, son unidades de programación independientes a las que se les permite llamarse entre sí por su efecto. Por ejemplo, *solve* puede llamar a *sqrt*, y *sqrt*, a su vez, puede llamar a *power*. Esta secuencia de llamadas puede ser tan profunda como queramos, así como recursiva. Normalmente, la función que llama (el autor de la *llamada*) pasa argumentos a la función llamada (el *destinatario*) y suspende su ejecución hasta que esta última complete *su* ejecución. El destinatario usa los argumentos pasados para ejecutar o calcular algo y, a continuación, devuelve un valor (que puede ser nulo) al autor de la llamada. Luego, la persona que llama vuelve a la acción, reanudando su ejecución.

En general, entonces, cada vez que una función (el autor de la *llamada*) llama a una función (el *destinatario*), alguien debe encargarse de los siguientes gastos generales:

- Guarde la dirección de *retorno*, que es la dirección dentro del código del autor de la llamada a la que debe volver la ejecución después de que el destinatario complete su ejecución;
- Guardar los recursos de memoria del autor de la llamada;
- Asignar los recursos de memoria requeridos por el destinatario de la llamada;
- Haga que los argumentos pasados por el autor de la llamada estén disponibles para el código del destinatario de la llamada; •

Comience a ejecutar el código del  
destinatario de la llamada.

Cuando el destinatario de la llamada finaliza y devuelve un valor, alguien debe encargarse de la siguiente sobrecarga:

- Haga que el valor devuelto *del destinatario de la llamada* esté disponible para el código del autor de la llamada; •

Reciclar los recursos de memoria  
utilizados por el destinatario de la  
llamada;

- Restablecer los recursos de memoria guardados anteriormente de la persona que llama;

- Recuperar la dirección de devolución guardada anteriormente;
- Reanuda la ejecución del código de la persona que llama, desde la dirección de retorno en adelante.

Afortunadamente, los programadores de alto nivel no tienen que pensar nunca en todas estas tareas esenciales: el código ensamblador generado por el compilador las maneja, de manera sigilosa y eficiente. Y, en un modelo de compilación de dos niveles, esta responsabilidad de limpieza recae en el back-end del compilador, que es el traductor de VM que estamos desarrollando ahora. Por lo tanto, en este capítulo, descubriremos, entre otras cosas, el marco de tiempo de ejecución que permite lo que probablemente sea la abstracción más importante en el arte de la programación: la *llamada y devolución de funciones*. Pero primero, comencemos con el desafío más fácil de manejar comandos de bifurcación.

---

## 8.2 Ramificación

El flujo predeterminado de los programas informáticos es secuencial, ejecutando un comando tras otro. Por varias razones, como embarcarse en una nueva iteración en un bucle, este flujo secuencial se puede redirigir mediante comandos de bifurcación. En la programación de bajo nivel, la bifurcación se realiza mediante *comandos goto destination*. La especificación de destino puede tomar varias formas, siendo la más primitiva la dirección de memoria física de la instrucción que debe ejecutarse a continuación. Se establece una especificación un poco más abstracta especificando una etiqueta simbólica (vinculada a una dirección de memoria física). Esta variación requiere que el lenguaje esté equipado con una directiva de etiquetado, diseñada para asignar etiquetas simbólicas a ubicaciones seleccionadas en el código. En nuestro lenguaje VM, esto se hace mediante un comando de etiquetado cuya sintaxis es símbolo de etiqueta.

Teniendo esto en cuenta, el lenguaje VM admite dos formas de bifurcación. *La bifurcación incondicional* se efectúa mediante un comando *goto symbol*, lo que significa: jump para ejecutar el comando justo después del comando label *symbol* en

|            |                                 |                |
|------------|---------------------------------|----------------|
| el código. | <i>Ramificación condicional</i> | es             |
| Efectuado  | Usando el                       | <i>símbolo</i> |

, cuya semántica es: Saca el valor superior de la pila; si no es falso, salta para ejecutar el comando justo después del comando label *symbol* ; de lo contrario, ejecuta el siguiente comando en el código. Este contrato implica que antes de especificar un comando goto condicional, el escritor de código

de máquina virtual (para

ejemplo, un compilador) debe especificar primero una condición. En nuestro lenguaje de VM, esto se hace insertando una expresión booleana en la pila. Por ejemplo, el compilador que desarrollaremos en los capítulos 10 y 11 traducirá `if (n < 100) goto LOOP` en `push n, push 100, lt, if-goto LOOP`.

**Ejemplo:** Considere una función que recibe dos argumentos,  $x$  e  $y$ , y devuelve el producto  $x \cdot y$ . Esto se puede hacer sumando  $x$  repetidamente a una variable local, digamos `suma`,  $y$  veces, y luego devolviendo el valor de `suma`. En la figura 8.1 se enumera una función que implementa este algoritmo de multiplicación ingenuo. En este ejemplo, se muestra cómo se puede expresar una lógica de bucle típica mediante los comandos de bifurcación de VM `goto`, `if-goto` y `label`.

| <u>High-level code</u>                                                                                                                                             | <u>VM code</u>                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// Returns x * y int mult(int x, int y) {     int sum = 0;     int i = 0;     while (i &lt; y) {         sum += x;         i++;     }     return sum; }</pre> | <pre>// Returns x * y function mult(x,y)     push 0     pop sum     push 0     pop i     label WHILE_LOOP     push i     push y     lt     neg     if-goto WHILE_END     push sum     push x     add     pop sum     push i     push 1     add     pop i     goto WHILE_LOOP     label WHILE_END     push sum     return</pre> |

**Figura 8.1** Acción de comandos de bifurcación. (El código de la máquina virtual de la derecha usa nombres de variables simbólicas en lugar de segmentos de memoria virtual para que sea más legible).

¡Observe cómo la condición booleana `(i < y)`, Implementado como `Push I`, `Push Y`, `LT`, `ng`, se inserta en la pila justo antes del comando `if-goto WHILE_END`. En

En el capítulo 7 vimos que los comandos de VM se pueden usar para expresar y evaluar cualquier expresión booleana. Como vemos en [la figura 8.1](#), las estructuras de control de alto nivel como `if` y `while` se pueden realizar fácilmente usando nada más que los comandos `goto` y `if-goto`. En general, cualquier flujo de estructura de control que se encuentre en los lenguajes de programación de alto nivel se puede realizar utilizando nuestro (conjunto bastante mínimo de) comandos lógicos y de ramificación de VM.

**Implementación:** La mayoría de los lenguajes de máquina de bajo nivel, incluido Hack, cuentan con medios para declarar etiquetas simbólicas y para realizar acciones condicionales e incondicionales de "ir a la etiqueta". Por lo tanto, si basamos la implementación de VM en un programa que traduce los comandos de VM en instrucciones de ensamblado, implementar los comandos de bifurcación de VM es un asunto relativamente simple.

**Sistema operativo:** Terminamos esta sección con dos comentarios al margen. Primero, los programas de VM no están escritos por humanos. Más bien, están escritos por compiladores. [La figura 8.1](#) ilustra el código fuente a la izquierda y el código de máquina virtual a la derecha. En los capítulos 10 y 11 desarrollaremos un *compilador* que traduce lo primero en lo segundo. En segundo lugar, tenga en cuenta que la implementación múltiple que se muestra en [la figura 8-1](#) es ineficiente. Más adelante en el libro presentaremos algoritmos optimizados de multiplicación y división que operan a nivel de bits. Estos algoritmos se utilizarán para realizar las funciones `Math.multiply` y `Math.divide`, que forman parte del sistema operativo que construiremos en el capítulo 12.

Nuestro sistema operativo se escribirá en el lenguaje Jack y será traducido por un compilador Jack al lenguaje VM. El resultado será una biblioteca de ocho archivos denominados `Math.vm`, `Memory.vm`, `String.vm`, `Array.vm`, `Output.vm`, `Screen.vm`, `Keyboard.vm` y `Sys.vm` (la API del sistema operativo se proporciona en el apéndice 6). Cada archivo del sistema operativo presenta una colección de funciones útiles que cualquier función de VM puede llamar para su efecto. Por ejemplo, siempre que una función de máquina virtual necesite servicios de multiplicación o división, puede llamar a la función `Math.multiply` o `Math.divide`.

---

## 8.3 Funciones

Cada lenguaje de programación se caracteriza por un conjunto fijo de operaciones integradas. Además, los lenguajes de alto nivel y algunos de



bajo nivel ofrecen el

Gran libertad para ampliar este repertorio fijo con una colección abierta de operaciones definidas por el programador. Dependiendo del lenguaje, estas operaciones predefinidas suelen denominarse *subrutinas*, *procedimientos*, *métodos* o *funciones*. En nuestro lenguaje VM, todas estas unidades de programación se denominan *funciones*.

En lenguajes bien diseñados, los comandos integrados y las funciones definidas por el programador tienen la misma apariencia. Por ejemplo, para calcular  $x + y$  en nuestra máquina de pila, empujamos  $x$ , empujamos  $y$  y sumamos. Al hacerlo, esperamos que la implementación `add` saque los dos valores superiores de la pila, los sume y empuje el resultado a la pila. Supongamos ahora que nosotros, o alguien más, hemos escrito una función *de potencia* diseñada para calcular  $x^y$ . Para usar esta función, seguimos exactamente la misma rutina: presionamos  $x$ , presionamos  $y$  y llamamos a `power`. Este protocolo de llamada consistente permite componer comandos primitivos y llamadas a funciones sin problemas. Por ejemplo, expresiones como  $(x + y)^3$  se pueden evaluar usando `push x`, `push y`, `add`, `push 3`, `call power`.

Vemos que la única diferencia entre aplicar una operación primitiva e invocar una función es la llamada de palabra clave que precede a esta última. Todo lo demás es exactamente igual: ambas operaciones requieren que el autor de la llamada prepare el escenario insertando argumentos en la pila, se espera que ambas operaciones consuman sus argumentos y se espera que ambas operaciones inserten valores devueltos en la pila. Este protocolo de llamada tiene una consistencia elegante que, esperamos, no pase desapercibida para el lector.

**Ejemplo:** La figura 8.2 muestra un programa de máquina virtual que calcula la función

$\sqrt{x^2 + y^2}$ , también conocido como *hypot*. El programa consta de tres funciones, con el siguiente comportamiento en tiempo de ejecución: `main` llama a `hypot` y, a continuación, `hypot` llama a `mult`, dos veces. También hay una llamada a una función `sqrt`, que no rastreamos, para reducir el desorden.

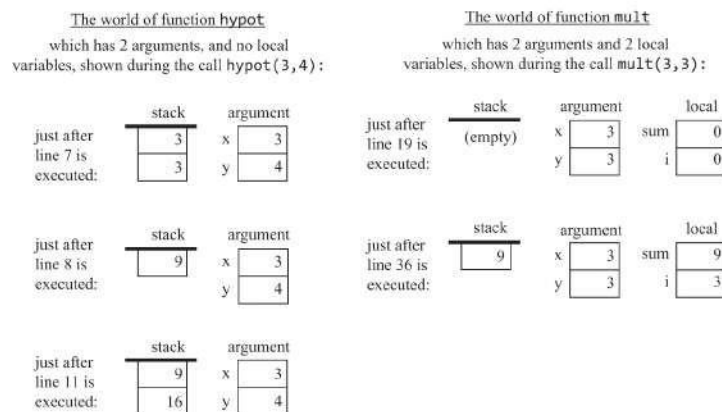
```

0 function main()
  // Computes hypot(3,4)
1  push 3
2  push 4
3  call hypot
4  return

5 function hypot(x,y)
  // Computes sqrt(x*x + y*y)
6  push x
7  push x
8  call mult
9  push y
10 push y
11 call mult
12 add
13 call sqrt
14 return

15 function mult(x,y)
  // Computes x * y (same as in figure 8.1)
16 push 0
17 pop sum
18 push 0
19 pop i
  ...
36 push sum
37 return

```



**Figura 8.2** Instantáneas en tiempo de ejecución de los estados de pila y segmento seleccionados durante la ejecución de un programa de tres funciones. Los números de línea no forman parte del código y se dan solo como referencia.

La parte inferior de la figura 8.2 muestra que durante el tiempo de ejecución, cada función ve un mundo privado, que consta de su propia pila de trabajo y segmentos de memoria. Estos mundos separados están conectados a través de dos "agujeros de gusano": cuando una función dice call mult, los argumentos que empujó a su pila antes de la llamada se pasan de alguna manera al segmento de argumentos del destinatario. Del mismo modo, cuando una función dice return, el último valor que insertó en su pila justo antes de regresar se copia de alguna manera en la pila del autor de la llamada, reemplazando los argumentos enviados anteriormente. Estas acciones de apretón de manos se llevan a cabo mediante la implementación de VM, como ahora vamos a describir.

**Implementación:** Un programa informático consta típicamente de varias y posiblemente muchas funciones. Sin embargo, en un momento dado durante el tiempo de ejecución, solo unas pocas de estas funciones están haciendo algo. Usamos el término *cadena de llamadas* para referirnos, conceptualmente, a todas las funciones que actualmente están involucradas en la ejecución del programa. Cuando un programa de VM comienza a ejecutarse, la cadena de llamadas consta de una sola función, por ejemplo, `main`. En algún momento, `main` puede llamar a otra función, digamos, `foo`, y esa función puede llamar a otra función, digamos, `bar`. En este punto, la cadena de llamada es `main→foo→bar`. Cada función de la cadena de llamada espera a que regrese la función a la que llamó. Por lo tanto, la única función que está realmente activa en la cadena de llamadas es la última, a la que llamamos la *función actual*, es decir, la función que se está ejecutando actualmente.

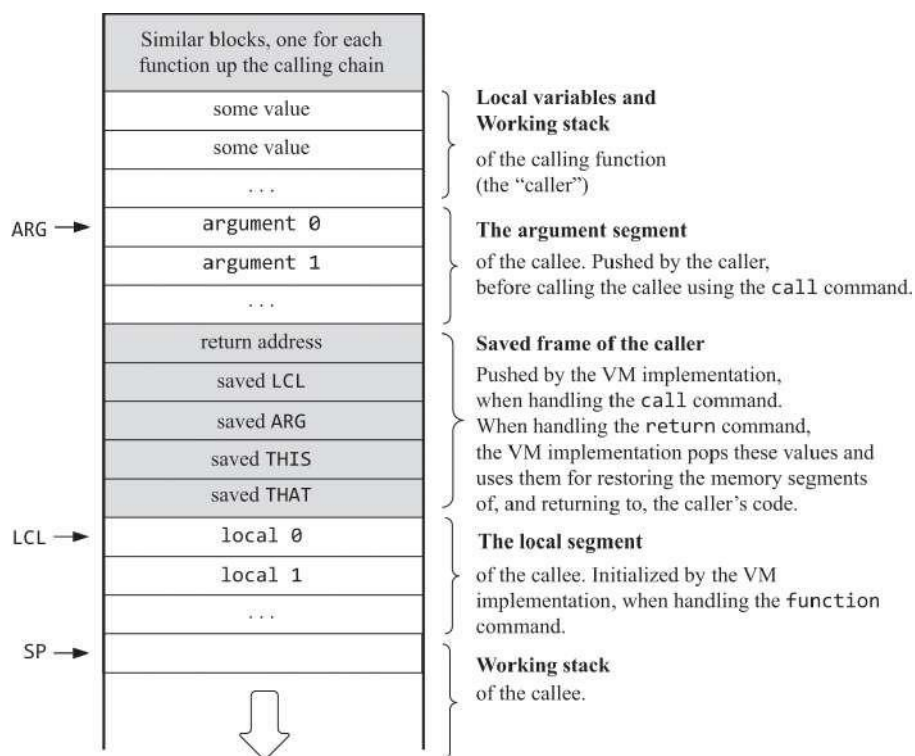
Para llevar a cabo su trabajo, las funciones normalmente usan *variables locales y argumentales*. Estas variables son temporales: los segmentos de memoria que las representan deben asignarse cuando la función comienza a ejecutarse y se pueden reciclar cuando la función regresa. Esta tarea de administración de memoria se complica por el requisito de que la llamada a funciones pueda anidarse arbitrariamente, así como recursiva. Durante el tiempo de ejecución, cada llamada de función debe ejecutarse independientemente de todas las demás llamadas y mantener su propia pila, variables locales y variables de argumento. ¿Cómo podemos implementar este mecanismo de anidamiento ilimitado y las tareas de gestión de memoria asociadas a él?

La propiedad que hace que esta tarea de mantenimiento sea manejable es la naturaleza lineal de la lógica de llamada y retorno. Aunque la cadena de llamada de funciones puede ser arbitrariamente profunda y recursiva, en un momento dado solo se ejecuta una función al final de la cadena, mientras que todas las demás funciones de la cadena de llamada están esperando que regrese. Este *modelo de procesamiento Last-In-First-Out* se presta perfectamente a la estructura de datos de la pila, que también es LIFO. Echemos un vistazo más de cerca.

Suponga que la función actual es `foo`. Supongamos que `foo` ya ha insertado algunos valores en su pila de trabajo y ha modificado algunas entradas en sus segmentos de memoria. Supongamos que en algún momento `foo` quiere llamar a otra función, `bar`, para su efecto. En este punto, tenemos que poner la ejecución de `foo` en espera hasta que `bar` termine su ejecución. Ahora, poner la pila de trabajo de `foo` en espera no es un problema: debido a que la pila crece solo en una dirección, la pila de trabajo de `bar` nunca anulará los valores empujados anteriormente. **Por lo tanto, guardar la pila de trabajo de la persona que llama es fácil:**

Lo obtenemos "gratis" gracias a la estructura de pila lineal y unidireccional. Pero, ¿cómo podemos guardar los segmentos de Recuerde que en el capítulo 7 usamos los punteros LCL, ARG, THIS y THAT para referirnos a las direcciones RAM base de los segmentos local, argument, this y that de la función actual. Si deseamos poner estos segmentos en espera, podemos empujar sus punteros a la pila y colocarlos más tarde, cuando queramos devolver la vida a foo. En lo que sigue, usamos el término *marco* para referirnos, colectivamente, al conjunto de valores de puntero necesarios para guardar y restablecer el estado de

Vemos que una vez que pasamos de una configuración de una sola función a una configuración multifunción, la humilde pila comienza a alcanzar un papel bastante formidable en nuestra historia. Específicamente, ahora usamos la misma estructura de datos para mantener tanto las pilas de trabajo como los marcos de todas las funciones en la cadena de llamadas. Para darle el respeto que se merece, a partir de ahora nos referiremos a esta estructura de datos de trabajo duro como la pila *global*. Consulte [la figura 8.3](#) para obtener más detalles.



**Figura 8.3** La pila global, que se muestra cuando se ejecuta el destinatario. Antes de que finalice el destinatario, inserta un valor devuelto en la pila (no se muestra). Cuando la implementación de la máquina virtual controla el comando `return`, copia el valor devuelto en el argumento 0 y establece SP para que apunte a la dirección solo

siguiéndolo. Esto libera efectivamente el área de pila global por debajo del nuevo valor de SP. Por lo tanto, cuando el autor de la llamada reanuda su ejecución, ve el valor devuelto en la parte superior de su pila de trabajo.

Como se muestra en [la figura 8.3](#), al controlar el comando `functionName` *de llamada*, la implementación de la máquina virtual inserta el marco del autor de la llamada en la pila. Al final de esta limpieza, estamos listos para pasar a ejecutar el código del destinatario de la llamada. Este mega salto no es difícil de implementar. Como veremos más adelante, al manejar un comando `functionName` *de función*, usamos el nombre de la función para crear e inyectar en el flujo de código ensamblador generado, una etiqueta simbólica única que marca dónde comienza la función. Por lo tanto, al manejar un comando `"function functionName"`, podemos generar código ensamblador que efectúe una operación `"goto functionName"`. Cuando se ejecuta, este comando transferirá efectivamente el control al destinatario de la llamada.

Devolver del destinatario de la llamada al autor de la llamada cuando finaliza el primero es más complicado, ya que el comando VM `return` no especifica ninguna dirección de retorno. De hecho, el anonimato de la persona que llama es inherente a la noción de una llamada a la función: funciones como `mult` o `sqrt` están diseñadas para servir a cualquier persona que llame, lo que implica que no se puede especificar una dirección de retorno a priori. En su lugar, un comando `return` se interpreta de la siguiente manera: redirigir la ejecución del programa a la ubicación de memoria que contiene el comando justo después del comando de llamada que invocó la función actual.

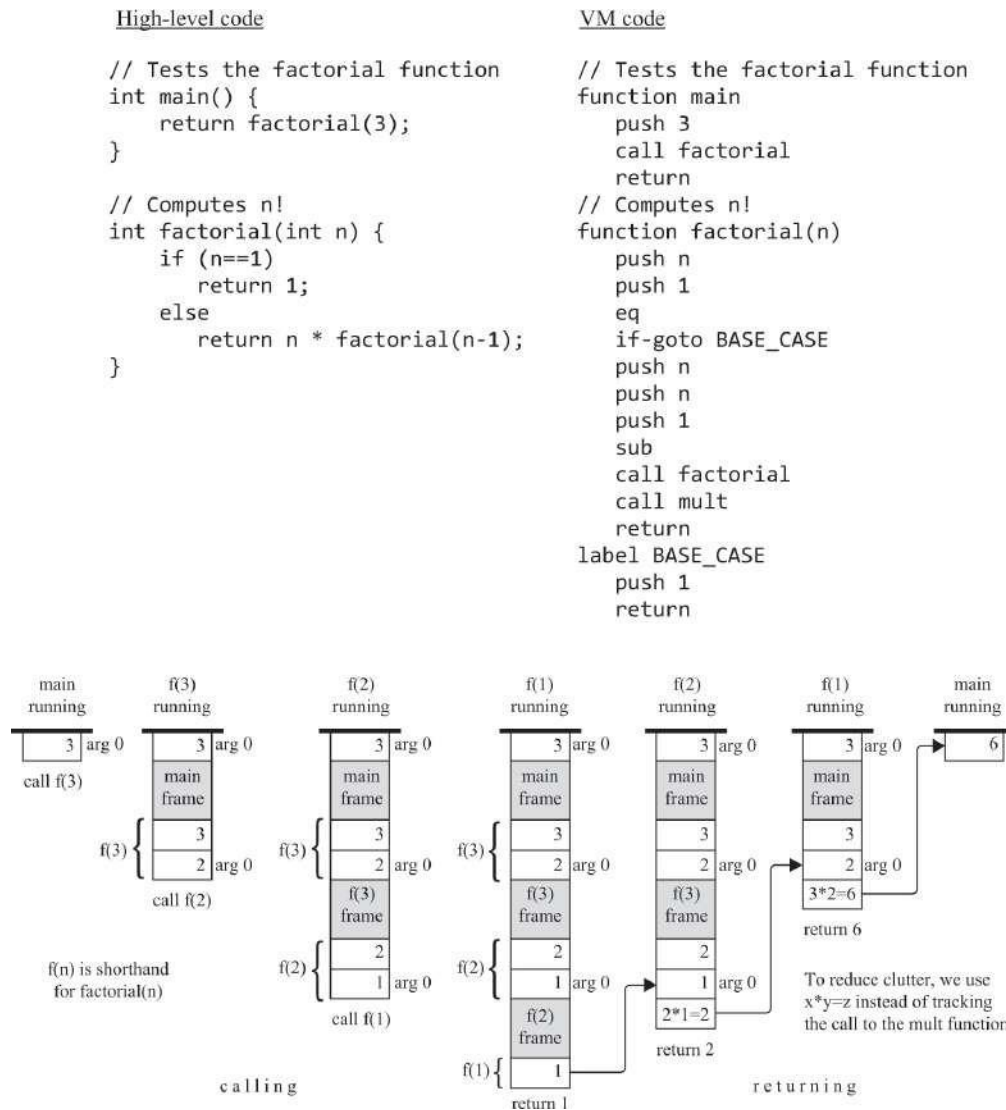
La implementación de VM puede realizar este contrato (i) guardando la dirección de devolución justo antes de que se transfiera el control a la ejecución del autor de la llamada y (ii) recuperando la dirección de retorno y saltando a ella justo después de que regrese el destinatario de la llamada. Pero, ¿dónde guardaremos la dirección del remitente? Una vez más, la pila ingeniosa viene al rescate. Para recordarle, el traductor de VM avanza de un comando de VM al siguiente, generando código ensamblador a medida que avanza. Cuando encontramos un comando `call foo` en el código de la máquina virtual, sabemos exactamente qué comando debe ejecutarse cuando termina `foo`: es el comando `assembly` justo después de los comandos `assembly` el que realiza el comando `call foo`. Por lo tanto, podemos hacer que el traductor de VM coloque una etiqueta allí mismo, en el flujo de código ensamblador generado, e inserte esta etiqueta en la pila. Cuando más adelante encontramos un comando `return` en el código de la máquina virtual, podemos sacar la dirección de retorno guardada anteriormente de la pila (llamémosla

*returnAddress*) y realizar la operación goto *returnAddress* en el ensamblado.  
Este es el

level que permite la magia en tiempo de ejecución de redirigir el control al lugar correcto en el código del autor de la llamada.

**La implementación de VM en acción:** ahora pasamos a dar una ilustración paso a paso de cómo la implementación de VM admite la acción de llamada y devolución de función. Lo haremos en el contexto de la ejecución de una función factorial, diseñada para calcular  $n!$  recursivamente. [La Figura 8.4](#) muestra el código del programa, junto con instantáneas seleccionadas de la pila global durante la ejecución de `factorial(3)`. Una simulación completa en tiempo de ejecución de este cálculo también debe incluir la acción de llamada y retorno de la función `mult`, que, en este ejemplo de tiempo de ejecución en particular, se llama dos veces: una antes de que `factorial(2)` regrese y una vez antes de que `factorial(3)` regrese.





**Figura 8.4** Varias instantáneas de la pila *global*, tomadas durante la ejecución de la función principal, que llama a **factorial** para calcular  $3!$ . La función en ejecución solo ve su pila de trabajo, que es el área sin sombreado en la punta de la pila global; Las otras áreas sin sombreado en la pila global son las pilas de trabajo de funciones en la cadena de llamadas, esperando que regrese la función que se está ejecutando actualmente. Tenga en cuenta que las áreas sombreadas no están "dibujadas a escala", ya que cada cuadro consta de cinco palabras, como se muestra en la [figura 8.3](#).

Centrándonos en las partes más a la izquierda y a la derecha de la parte inferior de la [figura 8.4](#), esto es lo que ocurrió desde la perspectiva de **main**: "Para preparar el escenario, empujé la constante 3 en la pila, y luego llamé **factorial** por su efecto (ver la instantánea de la pila más a la izquierda). En este punto me pusieron a dormir; en algún momento posterior me despertaron para descubrir que la pila ahora contiene 6 (ver la instantánea final y más a la derecha); No tengo idea de cómo esta magia

sucedió, y realmente no me importa; todo lo que sé es que me propuse calcular 3!, y obtuve exactamente lo que pedí". En otras palabras, la persona que llama es completamente ajena al elaborado mini-drama que se desató por su comando de llamada .

Como se ve en [la figura 8.4](#), el backstage en el que se desarrolla este drama es la pila global, y el coreógrafo que dirige el espectáculo es el VM implementación: Cada llamada operación se implementa guardando el marco de el llamador en la pila y saltando para ejecutar al destinatario de Cada devolución La operación se implementa (i) utilizando el marco almacenado más recientemente para obtener la dirección de retorno dentro del código de la persona que llama y restablecer sus segmentos de memoria, (ii) copiando el valor de pila superior (el valor de retorno) en la ubicación de la pila asociada con el argumento 0, y (iii) saltando para ejecutar el código de la persona que llama desde la dirección de retorno en adelante. Todas estas operaciones deben realizarse mediante código ensamblador generado.

Algunos lectores se preguntarán por qué tenemos que entrar en todos estos detalles. Hay al menos tres razones para ello. Primero, los necesitamos para implementar el traductor de VM. En segundo lugar, la implementación del protocolo de llamada y devolución de funciones es un hermoso ejemplo de ingeniería de software de bajo nivel, por lo que simplemente podemos disfrutar viéndolo en acción. En tercer lugar, una comprensión íntima de las partes internas de la máquina virtual nos ayuda a convertirnos en programadores de alto nivel mejores y más informados. Por ejemplo, jugar con la pila proporciona una comprensión profunda de los beneficios y las dificultades asociados con la recursividad. Tenga en cuenta que durante el tiempo de ejecución, cada llamada recursiva hace que la implementación de la máquina virtual agregue a la pila un bloque de memoria que consta de argumentos, marcos de función, variables locales y una pila de trabajo para el destinatario de la llamada. Por lo tanto, el uso no controlado de la recursividad puede conducir a la infame debacle en tiempo de ejecución del desbordamiento de pila. Esto, así como las consideraciones de eficiencia, lleva a los escritores de compiladores a tratar de reexpresar el código recursivo como código secuencial, siempre que sea posible. Pero esa es una historia diferente que se abordará en el capítulo 11.

---

## 8.4 Especificación de VM, Parte II

Hasta ahora en este capítulo, hemos presentado comandos generales de VM sin comprometernos con la sintaxis exacta y las convenciones de programación. Ahora pasamos a especificar formalmente los *comandos de bifurcación de VM*, los *comandos de función de VM* y la *estructura de los programas de VM*. Esto completa la especificación del lenguaje VM que comenzamos a describir en VM Specification, Parte I, en el capítulo 7.

Es importante reiterar que, normalmente, los programas de VM no son escritos por humanos. Más bien, son generados por compiladores. Por lo tanto, las especificaciones descritas aquí están dirigidas a los desarrolladores de compiladores. Es decir, si escribe un compilador que se supone que traduce programas de algún lenguaje de alto nivel a código de máquina virtual, se espera que el código que genera el compilador se ajuste a las convenciones descritas aquí.

### Comandos de bifurcación

- `label label`: etiqueta la ubicación actual en el código de la función. Solo se puede saltar a ubicaciones etiquetadas. El ámbito de la etiqueta es la función en la que se define. La *etiqueta* es una cadena compuesta por cualquier secuencia de letras, dígitos, guiones bajos (`_`), puntos (`.`) y dos puntos (`:`) que no comienzan con un dígito. El comando `label` se puede ubicar en cualquier lugar de la función, antes o después de los comandos `goto` que hacen referencia a él.
- `goto label`: Realiza una operación `goto` incondicional, lo que hace que la ejecución continúe desde la ubicación marcada por la etiqueta. El comando `goto`. El destino de salto etiquetado debe estar ubicado en la misma
- `if-goto label`: Afecta a una operación `goto` condicional. Se muestra el valor superior de la pila; Si el valor no es cero, la ejecución continúa desde la ubicación marcada por la etiqueta; de lo contrario, la ejecución continúa desde el siguiente comando en el programa. El `goto` y el salto etiquetado destino debe estar ubicado en la misma función.

### Comandos de función

- *function functionName nVars*: Marca el comienzo de una función denominada *functionName*. El comando informa que la función tiene *variables locales* *nVars*.
- *call functionName nArgs*: Llama a la función con nombre. El comando informa que *los argumentos nArgs* se han insertado en la pila antes de la llamada.
- *return*: Transfiere la ejecución al comando justo después de la llamada en el código de la función que llamó a la función actual.

## Programa VM

Los programas de VM se generan a partir de programas de alto nivel escritos en lenguajes como Jack. Como veremos en el próximo capítulo, un programa Jack de alto nivel se define vagamente como una colección de uno o más archivos de clase *jack* almacenados en la misma carpeta. Cuando se aplica a esa carpeta, el compilador Jack traduce cada archivo de clase *FileName.jack* en un archivo correspondiente denominado *FileName.vm*, que contiene comandos de VM.

Después de la compilación, cada *constructor*, *función (método estático)* y *método* denominado *bar* en un archivo Jack *FileName.jack* se traduce en una función de VM correspondiente, identificada de forma única por el nombre de la función de VM *FileName.bar*. El ámbito de los nombres de las funciones de máquina virtual es global: todas las funciones de máquina virtual de todos los archivos *.vm* de la carpeta del programa se ven entre sí y pueden llamarse entre sí mediante el nombre de función único y completo *FileName.functionName*.

**Punto de entrada del programa:** Un archivo en cualquier programa Jack debe llamarse *Main.jack*, y una función en este archivo debe llamarse *main*. Por lo tanto, después de la compilación, se espera que un archivo de cualquier programa de máquina virtual se denomine *Main.vm* y se espera que una función de VM de este archivo se denomine *Main.main*, que es el punto de entrada de la aplicación. Esta convención en tiempo de ejecución se implementa de la siguiente manera. Cuando comenzamos a ejecutar un programa de VM, la primera función que siempre se ejecuta es una función de VM sin argumentos denominada *Sys.init*, que forma parte del sistema operativo. Esta función del sistema operativo está programada para llamar a la función de punto de entrada en el programa del usuario. En el caso de los programas de Jack, *Sys.init* está programado para llamar a *Main.main*.

**Ejecución del programa:** hay varias formas de ejecutar programas de VM, una de las cuales es usar el emulador de VM suministrado que se presenta en el capítulo 7. Cuando carga una carpeta de programa que contiene uno o más archivos .vm en el emulador de VM, el emulador carga todas las funciones de VM en todos estos archivos, una tras otra (el orden de las funciones de VM cargadas es insignificante). La base de código resultante es una secuencia de todas las funciones de máquina virtual en todos los archivos .vm de la carpeta del programa. La noción de archivos VM deja de existir, aunque está implícita en los nombres de las funciones VM cargadas (*FileName.functionName*).

El emulador de VM de Nand a Tetris, que es un programa Java, presenta una implementación incorporada del sistema operativo Jack, también escrito en Java. Cuando el emulador detecta una llamada a una función del sistema operativo, por ejemplo, llamar a `Math.sqrt`, procede de la siguiente manera. Si encuentra un comando `Math.sqrt` de función correspondiente en el código de máquina virtual cargado, el emulador ejecuta el código de máquina virtual de la función. De lo contrario, el emulador vuelve a usar su implementación integrada del método `Math.sqrt`. Esto implica que, siempre que use el emulador de máquina virtual proporcionado para ejecutar programas de máquina virtual, no es necesario incluir archivos del sistema operativo en el código. El emulador de VM atenderá todas las llamadas del sistema operativo que se encuentran en el código, mediante su implementación integrada del sistema operativo.

---

## 8.5 Implementación

La sección anterior completó la especificación de nuestro lenguaje y marco de VM. En esta sección nos centramos en los problemas de implementación, que conducen a la construcción de un traductor de VM a Hack a gran escala. La Sección 8.5.1 propone cómo implementar el protocolo de llamada y devolución de funciones. Sección 8.5.2 completa el mapeo estándar de la implementación de VM sobre la plataforma Hack. La sección 8.5.3 ofrece un diseño y una API propuestos para completar el traductor de VM que comenzamos a construir en el proyecto 7.

### 8.5.1 Llamada y retorno de funciones

Los eventos de llamar a una función y regresar de una llamada a una función se pueden ver desde dos perspectivas: la de la *función que llama*,

también conocida como *llamador*, y la de la *función llamada*, también conocida como *destinatario*. Tanto el

El autor de la llamada y el destinatario de la llamada tienen ciertas expectativas y ciertas responsabilidades con respecto al control de los comandos de llamada, función y retorno. Cumplir las expectativas de uno es responsabilidad del otro. Además, la implementación de VM juega un papel importante en la ejecución de este contrato. A continuación, las responsabilidades de la implementación de VM están marcadas por [†]:

| La opinión de la persona que llama:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | La vista del destinatario de la llamada:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Antes de llamar a una función, debo insertar en la pila tantos argumentos (<i>nArgs</i>) como el destinatario espera obtener.</li> <li>• A continuación, invoco al destinatario mediante el comando <code>call fileName.functionName nArgs</code>.</li> <li>• Una vez que el destinatario de la llamada regresa, los valores de argumento que inserté antes de la llamada han desaparecido de la pila y aparece un <i>valor devuelto</i> (que siempre existe) en la parte superior de la pila. Excepto por este cambio, mi pila de trabajo es exactamente la misma que antes de la llamada [†].</li> <li>• Una vez que regresa el destinatario de la llamada, todos mis segmentos de memoria son exactamente los mismos que antes de la llamada [†], excepto que el contenido de mi <b>segmento estático</b> puede haber cambiado y el <b>segmento temporal</b> no está definido.</li> </ul> | <ul style="list-style-type: none"> <li>• Antes de comenzar a ejecutar, mi <b>argumento</b> se ha inicializado con los valores de argumento pasados por el autor de la llamada, y mi <b>local</b> variables se ha asignado e inicializado a ceros. Mi <b>estático</b> segment se ha establecido en el segmento estático del archivo de VM al que pertenezco y mi pila de trabajo está vacía. Los segmentos de memoria <b>éste, ese, puntero</b> y <b>Temp</b> no están definidos al ingresar [†].</li> <li>• Antes de regresar, debo insertar un valor de retorno en la pila.</li> </ul> |

La implementación de VM admite este contrato manteniendo y manipulando la estructura de pila global descrita en la [figura 8.3](#). En concreto, cada función, llamada y comando return del código de la máquina virtual se controla mediante la generación de código ensamblado que manipula la pila global de la siguiente manera: Un comando de llamada genera código que guarda el marco del autor de la llamada en la pila y salta para **ejecutar al destinatario de la llamada**. Un comando de función genera código que **inicializa las variables locales del destinatario** que

Finalmente, un comando return genera código que copia el valor de retorno en la parte superior del trabajo de la persona que llama

stack, restablece los punteros de segmento de la persona que llama y salta para ejecutar este último desde la dirección de retorno en adelante. Consulte la figura 8.5 para obtener más detalles.

| <i>VM command</i>                                                                                                                                     | <i>Assembly (pseudo) code, generated by the VM translator</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>call f nArgs</code><br>(calls a function <i>f</i> ,<br>informing that <i>nArgs</i><br>arguments were pushed<br>to the stack before the<br>call) | <code>push returnAddress</code> // generates a label and pushes it to the stack<br><code>push LCL</code> // saves LCL of the caller<br><code>push ARG</code> // saves ARG of the caller<br><code>push THIS</code> // saves THIS of the caller<br><code>push THAT</code> // saves THAT of the caller<br><code>ARG = SP-5-nArgs</code> // repositions ARG<br><code>LCL = SP</code> // repositions LCL<br><code>goto f</code> // transfers control to the callee<br>( <code>returnAddress</code> ) // injects the return address label into the code                                                                             |
| <code>function f nVars</code><br>(declares a function <i>f</i> ,<br>informing that the<br>function has <i>nVars</i><br>local variables)               | ( <i>f</i> ) // injects a function entry label into the code<br><code>repeat nVars times:</code> // <i>nVars</i> = number of local variables<br><code>push 0</code> // initializes the local variables to 0                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <code>return</code><br>(terminates the current<br>function and returns<br>control to the caller)                                                      | <code>frame = LCL</code> // <i>frame</i> is a temporary variable<br><code>retAddr = *(frame-5)</code> // puts the return address in a temporary variable<br><code>*ARG = pop()</code> // repositions the return value for the caller<br><code>SP = ARG+1</code> // repositions SP for the caller<br><code>THAT = *(frame-1)</code> // restores THAT for the caller<br><code>THIS = *(frame-2)</code> // restores THIS for the caller<br><code>ARG = *(frame-3)</code> // restores ARG for the caller<br><code>LCL = *(frame-4)</code> // restores LCL for the caller<br><code>goto retAddr</code> // go to the return address |

**Figura 8.5** Implementación de los comandos de función del lenguaje VM. Todas las acciones descritas a la derecha se realizan mediante instrucciones de ensamblaje Hack generadas.

### 8.5.2 Mapeo estándar de VM en la plataforma de piratería, parte II

Se recomienda a los desarrolladores de la implementación de VM en el equipo Hack que sigan las convenciones descritas aquí. Estas convenciones completan las directrices de asignación de VM estándar en la plataforma de piratería, parte I, que se proporcionan en la sección 7.4.1.

**La pila:** En la plataforma Hack, las ubicaciones de RAM 0 a 15 están reservadas para punteros y registros virtuales, y las ubicaciones de RAM 16 a 255 están reservadas para variables estáticas. La pila se asigna en la mapeo, el traductor de VM debe comenzar generando código ensamblador que



comandos como pop, push, add, etc. en el código de la máquina virtual fuente, genera código ensamblado que afecta a estas operaciones manipulando la dirección a la que apunta SP y modificando SP, según sea necesario. Estas acciones se explicaron en el capítulo 7 y se implementaron en el proyecto 7.

**Símbolos especiales:** al traducir comandos de VM en ensamblado Hack, el traductor de VM trata con dos tipos de símbolos. En primer lugar, administra símbolos predefinidos a nivel de ensamblado como SP, LCL y ARG. En segundo lugar, genera y utiliza etiquetas simbólicas para marcar direcciones de retorno y puntos de entrada de funciones. Para ilustrarlo, revisemos el programa PointDemo presentado en la introducción de la parte II. Este programa consta de dos archivos de clase Jack, Main.jack (figura II.1) y Point.jack (figura II.2), almacenados en una carpeta llamada PointDemo. Cuando se aplica a la carpeta PointDemo, el compilador Jack genera dos archivos de máquina virtual, denominados Main.vm y Point.vm. El primer archivo contiene una única función de máquina virtual, Main.main, y el segundo archivo contiene las funciones de VM Point.new, Point.getx, ..., Apuntar.imprimir.

Cuando el traductor de VM se aplica a esta misma carpeta, genera un único archivo de código de ensamblado, denominado PointDemo.asm. En el nivel de código ensamblador, las abstracciones de funciones ya no existen. En su lugar, para cada comando de función, el traductor de VM genera una etiqueta de entrada en el ensamblado; para cada comando de llamada, el traductor de VM (i) genera una instrucción goto de ensamblado, (ii) crea una etiqueta de dirección de retorno y la inserta en la pila, y (iii) inserta la etiqueta en el código generado. Para cada comando return, el traductor de VM extrae la dirección de retorno de la pila y genera una instrucción goto. Por ejemplo:

| <u>VM code</u>     | <u>Generated assembly code</u>   |
|--------------------|----------------------------------|
| function Main.main | (Main.main)                      |
| ...                | ...                              |
| call Point.new     | goto Point.new                   |
| // Next VM command | (Main.main\$ret0)                |
| ...                | // Next VM command (in assembly) |
|                    | ...                              |
| function Point.new | (Point.new)                      |
| ...                | ...                              |
| return             | goto Main.main\$ret0             |

En la figura 8.6 se especifican todos los símbolos que el traductor de máquinas virtuales controla y genera.

| <i>Symbol</i>                                                 | <i>Usage</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SP                                                            | This predefined symbol points to the memory address within the host RAM just following the address containing the topmost stack value.                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| LCL, ARG, THIS, THAT                                          | These predefined symbols point, respectively, to the base RAM addresses of the virtual segments local, argument, this, and that of the currently running VM function.                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <i>Xxx.i</i> symbols<br>(represent static variables)          | Each reference to static <i>i</i> appearing in file <i>Xxx.vm</i> is translated to the assembly symbol <i>Xxx.i</i> . In the subsequent assembly process, the Hack assembler will allocate these symbolic variables to the RAM, starting at address 16.                                                                                                                                                                                                                                                                                                                                                             |
| <i>functionName\$label</i><br>(destinations of goto commands) | Let <i>foo</i> be a function within the file <i>Xxx.vm</i> . The handling of each <code>label bar</code> command within <i>foo</i> generates, and injects into the assembly code stream, the symbol <i>Xxx.foo\$bar</i> .<br>When translating <code>goto bar</code> and <code>if-goto bar</code> commands (within <i>foo</i> ) into assembly, the label <i>Xxx.foo\$bar</i> must be used instead of <i>bar</i> .                                                                                                                                                                                                    |
| <i>functionName</i><br>(function entry point symbols)         | The handling of each function <i>foo</i> command within the file <i>Xxx.vm</i> generates, and injects into the assembly code stream, a symbol <i>Xxx.foo</i> that labels the entry-point to the function's code. In the subsequent assembly process, the assembler translates this symbol into the physical address where the function code starts.                                                                                                                                                                                                                                                                 |
| <i>functionName\$ret.i</i><br>(return address symbols)        | Let <i>foo</i> be a function within the file <i>Xxx.vm</i> .<br>The handling of each <code>call</code> command within <i>foo</i> 's code generates, and injects into the assembly code stream, a symbol <i>Xxx.foo\$ret.i</i> , where <i>i</i> is a running integer (one such symbol is generated for each <code>call</code> command within <i>foo</i> ).<br>This symbol is used to mark the return address within the caller's code. In the subsequent assembly process, the assembler translates this symbol into the physical memory address of the command immediately following the <code>call</code> command. |
| R13 - R15                                                     | These predefined symbols can be used for any purpose. For example, if the VM translator generates assembly code that needs to use low-level variables for temporary storage, R13 - R15 can come handy.                                                                                                                                                                                                                                                                                                                                                                                                              |

**Figura 8.6** Las convenciones de nomenclatura descritas anteriormente están diseñadas para admitir la traducción de varios archivos y funciones .vm en un único archivo .asm, lo que garantiza que los símbolos de ensamblado generados sean únicos dentro del archivo.

**Código de arranque:** la asignación estándar de VM en la plataforma Hack estipula que la pila se asigne en la RAM del host desde la dirección 256 en adelante, y que la primera función de VM que debe comenzar a ejecutarse es la función del sistema operativo `sys.init`. ¿Cómo podemos afectar estas convenciones en la plataforma Hack? Recuerde que cuando construimos la computadora Hack en el capítulo 5, la conectamos de tal manera que al reiniciarla, buscará y ejecutará la instrucción ubicada en la dirección ROM 0. Por lo tanto, si queremos que la computadora ejecute un segmento de código predeterminado cuando se inicie, podemos poner esto

código en la memoria de instrucciones de la computadora Hack, comenzando en la dirección 0. Aquí está el código:

El código de arranque (pseudo) debe expresarse en lenguaje de máquina SP = 256  
llamar a Sys.init

Se espera que la función Sys.init, que forma parte del sistema operativo, llame a la función principal de la aplicación y entre en un bucle infinito. Esta acción hará que el programa de máquina virtual traducido comience a ejecutarse. Tenga en cuenta que las nociones de *aplicación* y *función principal* varían de un lenguaje de alto nivel a otro. En el lenguaje de Jack, la convención es que Sys.init debe llamar a la clase

Función de máquina virtual Main.main. Esto es similar a la configuración de Java: cuando le indicamos a la JVM que ejecute una clase Java dada, digamos, Foo, busca y ejecuta el método Foo.main. En general, podemos afectar a rutinas de inicio específicas del lenguaje mediante diferentes versiones de la función Sys.init.

**Uso:** El traductor acepta un único argumento de línea de comandos, como se indica a continuación:

```
prompt> origen de VMTranslator
```

donde *source* es un nombre de archivo con el formato Xxx.vm (la extensión es obligatoria) o el nombre de una carpeta (en cuyo caso no hay extensión) que contiene uno o más archivos .vm. El nombre del archivo o carpeta puede contener una ruta de acceso de archivo. Si no se especifica ninguna ruta, el traductor opera en la carpeta actual. La salida del traductor de VM es un único archivo de ensamblado, denominado source.asm. Si *source* es un nombre de carpeta, el único archivo .asm contiene la traducción de todas las funciones de todos los archivos .vm de la carpeta, una tras otra. El archivo de salida se crea en la misma carpeta que el archivo de entrada. Si hay un archivo con este nombre en la carpeta, se sobrescribirá.

### 8.5.3 Sugerencias de diseño para la implementación de VM

En el proyecto 7 propusimos construir el traductor de VM básico utilizando tres módulos: VMTranslator, Parser y CodeWriter. Ahora describimos cómo extender esta implementación básica a un traductor de VM a gran escala. Esta extensión se puede lograr agregando la funcionalidad que se describe a

continuación al

Tres módulos ya construidos en el Proyecto 7. No es necesario desarrollar módulos adicionales.

## **El VMTranslator**

Si la entrada del traductor es un solo archivo, por ejemplo, Prog.vm, VMTranslator construye un analizador para analizar Prog.vm y un CodeWriter que comienza creando un archivo de salida denominado Prog.asm. A continuación, VMTranslator entra en un bucle que utiliza los servicios del analizador para iterar a través del archivo de entrada y analizar cada línea como un comando de máquina virtual, salvo espacios en blanco. Para cada comando analizado, VMTranslator usa CodeWriter para generar código ensamblador Hack y emitir el código generado en el archivo de salida. Todo esto ya se hizo en el proyecto 7.

Si la entrada del traductor es una carpeta, llamada, por ejemplo, Prog, VMTranslator construye un analizador para manejar cada archivo .vm en la carpeta y un solo CodeWriter para generar código ensamblador Hack en el archivo de salida único Prog.asm. Cada vez que VMTranslator comienza a traducir un nuevo archivo .vm en la carpeta, debe informar al CodeWriter de que se está procesando un nuevo archivo. Esto se hace llamando a una rutina de CodeWriter denominada `setFileName`, como ahora pasamos a describir.

## **El analizador**

Este módulo es idéntico al analizador desarrollado en el proyecto 7.

## **El Escritor de Código**

El CodeWriter desarrollado en el proyecto 7 fue diseñado para manejar los *comandos aritmético-lógicos y push / pop* de VM. Aquí está la API de un CodeWriter completo que maneja todos los comandos en el lenguaje VM:

| <i><b>Routine</b></i>                          | <i><b>Arguments</b></i>                                        | <i><b>Returns</b></i> | <i><b>Function</b></i>                                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------|----------------------------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Constructor /<br>initializer                   | Output file / stream                                           | —                     | <p>Opens an output file / stream and gets ready to write into it.</p> <p>Writes the assembly instructions that effect the bootstrap code that starts the program's execution. This code must be placed at the beginning of the generated output file / stream.</p> <p>Comment: See the <i>Implementation Tips</i> at the end of section 8.6.</p> |
| setFileName                                    | fileName (string)                                              | —                     | <p>Informs that the translation of a new VM file has started (called by the VMTranslator).</p>                                                                                                                                                                                                                                                   |
| writeArithmetic<br>(developed in<br>project 7) | command (string)                                               | —                     | <p>Writes to the output file the assembly code that implements the given arithmetic-logical command.</p>                                                                                                                                                                                                                                         |
| writePushPop<br>(developed in<br>project 7)    | command (C_PUSH or<br>C_POP), segment<br>(string), index (int) | —                     | <p>Writes to the output file the assembly code that implements the given push or pop command.</p>                                                                                                                                                                                                                                                |
| writeLabel                                     | label (string)                                                 | —                     | <p>Writes assembly code that effects the label command.</p>                                                                                                                                                                                                                                                                                      |
| writeGoto                                      | label (string)                                                 | —                     | <p>Writes assembly code that effects the goto command.</p>                                                                                                                                                                                                                                                                                       |
| writeIf                                        | label (string)                                                 | —                     | <p>Writes assembly code that effects the if-goto command.</p>                                                                                                                                                                                                                                                                                    |
| writeFunction                                  | functionName (string)<br>nVars (int)                           | —                     | <p>Writes assembly code that effects the function command.</p>                                                                                                                                                                                                                                                                                   |
| writeCall                                      | functionName (string)<br>nArgs (int)                           | —                     | <p>Writes assembly code that effects the call command.</p>                                                                                                                                                                                                                                                                                       |
| writeReturn                                    | —                                                              | —                     | <p>Writes assembly code that effects the return command.</p>                                                                                                                                                                                                                                                                                     |
| close<br>(developed in<br>project 7)           | —                                                              | —                     | <p>Closes the output file / stream.</p>                                                                                                                                                                                                                                                                                                          |

## 8.6 Proyecto

En pocas palabras, tenemos que ampliar el traductor básico desarrollado en el capítulo 7 con la capacidad de manejar múltiples archivos *.vm* y la capacidad de traducir *los comandos* de ramificación de *VM* y *los comandos de función* de VM en código ensamblador Hack. Para cada comando de VM analizado, el traductor de VM tiene que generar código ensamblador que implemente la semántica del comando en la plataforma Hack del host. La traducción de los tres *comandos de ramificación* en ensamblador no es difícil. La traducción de los tres comandos de *función* es más desafiante e implica la implementación del pseudocódigo enumerado en la [figura 8.5](#), utilizando los símbolos descritos en la [figura 8.6](#). Repetimos la sugerencia dada en el capítulo anterior: Comience escribiendo el código ensamblador requerido en papel. Dibuje imágenes de RAM y pila global, realice un seguimiento del puntero de pila y los punteros de segmento de memoria pertinentes y asegúrese de que el código ensamblado basado en papel implementa correctamente todas las acciones de bajo nivel asociadas con el control de los comandos de llamada, función y devolución.

**Objetivo:** Ampliar el traductor de VM básico integrado en el proyecto 7 en un traductor de VM a gran escala, diseñado para manejar programas de varios archivos escritos en el lenguaje de VM.

Esta versión del traductor de máquina virtual supone que el código de máquina virtual de origen no tiene errores. La comprobación de errores, la generación de informes y el control se pueden agregar a versiones posteriores del traductor de VM, pero no forman parte del proyecto 8.

**Contrato:** Completar la construcción de un traductor de VM a Hack, de acuerdo con la Especificación de VM, Parte II (sección 8.4) y con el Mapeo de VM estándar en la plataforma de hackeo, Parte II (sección 8.5.2). Utilice su traductor para traducir los programas de prueba de VM suministrados, lo que produce los programas correspondientes escritos en el lenguaje ensamblador Hack. Cuando se ejecuta en el emulador de CPU suministrado junto con los scripts de prueba suministrados, los programas de ensamblado generados por el traductor deben entregar los resultados exigidos por los archivos de comparación proporcionados.

**Recursos:** Necesitará dos herramientas: el lenguaje de programación en el que implementará su traductor de VM y el *emulador de CPU* suministrado en el paquete de software Nand to Tetris. Use el emulador de CPU para ejecutar y probar el código ensamblador generado por el traductor. Si se ejecuta el código generado



correctamente, asumiremos que el traductor de VM funciona como se espera. Esta prueba parcial del traductor será suficiente para nuestros propósitos.

Otra herramienta que resulta útil en este proyecto es el *emulador de VM suministrado*. Use este programa para ejecutar los programas de máquina virtual de prueba proporcionados y observe cómo el código de máquina virtual afecta a los estados simulados de la pila y los segmentos de memoria virtual. Esto puede ayudar a comprender las acciones que el traductor de VM debe realizar finalmente en el ensamblado.

Dado que el traductor de VM a gran escala se implementa extendiendo el traductor de VM integrado en el proyecto 7, también necesitará el código fuente de este último.

## **Etapas de prueba e implementación**

Recomendamos completar la implementación del traductor de VM en dos etapas. Primero, implemente los comandos *de bifurcación* y, a continuación, los comandos de *función*. Esto le permitirá realizar pruebas unitarias de su implementación de forma incremental, utilizando los programas de prueba proporcionados.

### **Probar el manejo de la etiqueta de comandos de máquina virtual, if, if-goto:**

- BasicLoop:  $1 + 2 + \dots + \text{argument}[0]$ , calcula e inserta el resultado en la pila. Prueba cómo el traductor de VM controla los comandos label y if-goto.
- FibonacciSeries: Calcula y almacena en la memoria los primeros  $n$  elementos de la serie de Fibonacci. Una prueba más rigurosa del manejo de los comandos label, goto y if-goto.

### **Prueba del manejo de la llamada, función y retorno de los comandos de VM :**

A diferencia de lo que hicimos en el proyecto 7, ahora esperamos que el traductor de VM maneje programas de varios archivos. Recordamos al lector que, por convención, la primera función que comienza a ejecutarse en un programa de VM es Sys.init. Normalmente, Sys.init está programado para llamar a la función Main.main del programa. Sin embargo, para el propósito de este proyecto, usamos las funciones Sys.init proporcionadas para preparar el escenario para las diversas pruebas que deseamos realizar.

- SimpleFunction: realiza un cálculo simple y devuelve el resultado. Prueba



cómo el traductor de VM controla la función y los comandos de retorno. Dado que esta prueba implica el manejo de un solo archivo que consta de una sola función, no se puede  
Sys.init función de prueba.

- **FibonacciElement:** Este programa de prueba consta de dos archivos: `Main.vm` contiene una sola función de Fibonacci que devuelve recursivamente el  $n$ -ésimo elemento de la serie de Fibonacci; `Sys.vm` contiene una única función `Sys.init` que llama a `Main.fibonacci` con  $n = 4$  y, a continuación, entra en un bucle infinito (recuerde que el traductor de VM genera código de arranque que llama a `Sys.init`). La configuración resultante proporciona una prueba rigurosa del manejo del traductor de VM de varios archivos `.vm`, los comandos de llamada y devolución de funciones de VM, el código de arranque y la mayoría de los otros comandos de VM. Dado que el programa de prueba consta de dos `.vm`, se debe traducir toda la carpeta, produciendo un único `FibonacciElement.asm`.
- **StaticsTest:** Este programa de prueba consta de tres archivos: `Class1.vm` y `Class2.vm` contienen funciones que establecen y obtienen los valores de varias variables estáticas; `Sys.vm` contiene una única función `Sys.init` que llama a estas funciones. Dado que el programa consta de varios archivos `.vm`, se debe traducir toda la carpeta, produciendo un solo archivo `StaticsTest.asm`.

## Consejos de implementación

Dado que el proyecto 8 se basa en extender el traductor VM básico desarrollado en el proyecto 7, recomendamos hacer una copia de seguridad del código fuente de este último (si aún no lo ha hecho).

Empiece por averiguar el código de ensamblado necesario para realizar la lógica de los comandos de máquina virtual `label`, `goto` e `if-goto`. A continuación, proceda a implementar los métodos `writeLabel`, `writeGoto` y `writeln` del `CodeWriter`. Pruebe su traductor de VM en evolución haciendo que traduzca los programas `BasicLoop.vm` y `FibonacciSeries.vm` suministrados.

**Código de arranque:** para que cualquier programa de VM traducido comience a ejecutarse, debe incluir código de inicio que obligue a la implementación de VM a comenzar a ejecutar el programa en la plataforma host. Además, para que cualquier código de máquina virtual funcione correctamente, la implementación de la máquina virtual debe almacenar las direcciones base de la pila y los segmentos virtuales en las ubicaciones correctas de la RAM del host. Los primeros tres programas de prueba de este proyecto (`BasicLoop`, `FibonacciSeries`, `SimpleFunction`) asumen que el código de inicio aún no se ha implementado e incluyen scripts de prueba que efectúan las inicializaciones necesarias *manualmente*, lo que significa que en esta etapa de desarrollo no tiene que preocuparse por ello.

Los dos últimos programas de prueba (FibonacciElement y StaticsTest) asumen que el código de inicio ya forma parte de la implementación de la máquina virtual.

Con eso en mente, el constructor del CodeWriter debe desarrollarse en dos etapas. La primera versión del constructor no debe generar ningún código de arranque (es decir, omitir la directriz de la API del constructor que comienza con el texto "Escribe las instrucciones de ensamblado..."). Utilice esta versión del traductor para realizar pruebas unitarias de los programas BasicLoop, FibonacciSeries y SimpleFunction. La segunda y última versión del constructor de CodeWriter debe escribir el código de arranque, como se especifica en la API del constructor. Esta versión debe usarse para pruebas unitarias

FibonacciElement y StaticsTest.

Los programas de prueba suministrados se planificaron cuidadosamente para probar las características específicas de cada etapa de la implementación de la máquina virtual. Recomendamos implementar su traductor en el orden propuesto y probarlo utilizando los programas de prueba apropiados en cada etapa. La implementación de una etapa posterior antes de una temprana puede hacer que los programas de prueba fallen.

**Una versión basada en la web del proyecto 8** está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

---

## 8.7 Perspectiva

Las nociones de *ramificación* y *llamada a funciones* son fundamentales para todos los lenguajes de alto nivel. Esto significa que en algún momento de la ruta de traducción de un lenguaje de alto nivel a un código binario, alguien debe encargarse de manejar las intrincadas tareas de limpieza relacionadas con su implementación. En Java, C#, Python y Jack, esta carga recae en el nivel de máquina virtual. Y si la arquitectura de VM está *basada en pilas*, se presta muy bien para el trabajo, como hemos visto a lo largo de este capítulo.

Para apreciar el poder expresivo de nuestro modelo de máquina virtual basado en pilas, eche un segundo vistazo a los programas presentados en este capítulo. Por ejemplo, [las cifras](#)

[8.1](#) y [8.4](#) presentan programas de alto nivel y sus traducciones de VM. Si realiza un recuento de líneas, observará que cada línea de código de alto nivel genera un promedio de aproximadamente cuatro líneas de código de máquina

virtual compilado. Resulta que esta relación de traducción de 1:4 es bastante consistente cuando se compilan programas Jack en código VM. Incluso sin saber mucho sobre el arte de

compilación, se puede apreciar la brevedad y legibilidad del código de VM generado por el compilador. Por ejemplo, como veremos cuando construyamos el compilador, una declaración de alto nivel como `let y = Math.sqrt(x)` se traduce en `push x, call Math.sqrt, pop y`. El compilador de dos niveles puede salirse con la suya con tan poco trabajo, ya que cuenta con la implementación de la máquina virtual para controlar el resto de la traducción. Si tuviéramos que traducir declaraciones de alto nivel como `let y = Math.sqrt(x)` directamente en código Hack, sin tener el beneficio de una capa intermedia de VM, el código resultante sería mucho menos elegante y más críptico.

Dicho esto, también sería más eficiente. No olvidemos que el código de la máquina virtual debe realizarse en lenguaje de máquina, de eso se tratan los proyectos 7 y 8. Por lo general, el código de máquina final resultante de un proceso de traducción de dos niveles es más largo y menos eficiente que el generado a partir de la traducción directa. Entonces, ¿qué es más deseable: un programa Java de dos niveles que eventualmente genere mil instrucciones de máquina o un programa C++ equivalente de un nivel que genere setecientas instrucciones? La respuesta pragmática es que cada lenguaje de programación tiene sus pros y sus contras, y cada aplicación tiene diferentes requisitos operativos.

Una de las grandes virtudes del modelo de dos niveles es que el código intermedio de la máquina virtual (por ejemplo, el código de bytes de Java) puede ser *administrado*, por ejemplo, por programas que prueban si contiene código malicioso, programas que monitorean el código para el modelado de procesos comerciales, etc. En general, para la mayoría de las aplicaciones, las ventajas del código administrado justifican la degradación del rendimiento causada por el nivel de máquina virtual. Sin embargo, para los programas de alto rendimiento, como los sistemas operativos y las aplicaciones integradas, la necesidad de generar código ajustado y eficiente generalmente exige el uso de C / C++, compilado directamente en lenguaje de máquina.

Para los escritores de compiladores, una ventaja obvia de usar un lenguaje de VM provisional explícito es que simplifica las tareas de escritura y mantenimiento de compiladores. Por ejemplo, la implementación de VM desarrollada en este capítulo libera al compilador de las tareas importantes de controlar la implementación de bajo nivel del protocolo de llamada y devolución de funciones. En general, la capa intermedia de VM desacopla el desalentador desafío de crear un compilador de alto nivel a bajo nivel en dos desafíos mucho más simples: crear un compilador de alto nivel a VM y crear un traductor de VM a bajo nivel. Dado que este último

traductor, también llamado *back-end del compilador*, ya se desarrolló en los proyectos 7 y 8, podemos estar contentos de que aproximadamente la mitad del total de la

ya se ha cumplido. La otra mitad, el desarrollo de la interfaz del compilador, se retomará en los capítulos 10 y 11.

Terminamos este capítulo con una observación general sobre la virtud de separar la abstracción de la implementación, un tema constante en Nand to Tetris y un principio crucial de construcción de sistemas que va mucho más allá del contexto de la compilación de programas. Recuerde que las funciones de máquina virtual pueden acceder a sus segmentos de memoria mediante comandos como `push argument 2`, `pop local 1`, etc., sin tener idea de *cómo* se representan, guardan y restablecen estos valores durante el tiempo de ejecución. La implementación de VM se encarga de todos los detalles sangrientos. Esta separación completa de la abstracción y la implementación implica que los desarrolladores de compiladores que generan código de VM no tienen que preocuparse por cómo terminará ejecutándose el código que generan; tienen suficientes problemas propios, como pronto te darás cuenta.

¡Así que ánimo! Está a la mitad de escribir un compilador de dos niveles para un lenguaje de programación de alto nivel, basado en objetos y similar a Java. El siguiente capítulo está dedicado a describir este lenguaje. Esto preparará el escenario para los capítulos 10 y 11, en los que completaremos el desarrollo del compilador. Comenzamos a ver ladrillos de Tetris cayendo al final del túnel.

---

## 9 Lenguaje de alto nivel

Los pensamientos elevados necesitan un lenguaje elevado.

—Aristófanes (427-386 a.C.)

Los lenguajes ensamblador y VM presentados hasta ahora en este libro son de bajo nivel, lo que significa que están destinados a controlar máquinas, no a desarrollar aplicaciones. En este capítulo presentamos un lenguaje de alto nivel, llamado Jack, diseñado para permitir a los programadores escribir programas de alto nivel. Jack es un lenguaje simple basado en objetos. Tiene las características básicas y el sabor de los lenguajes convencionales como Java y C ++, con una sintaxis más simple y sin soporte para herencia. A pesar de esta simplicidad, Jack es un lenguaje de propósito general que se puede utilizar para crear numerosas aplicaciones. En particular, se presta muy bien a juegos interactivos como Tetris, Snake, Pong, Space Invaders y clásicos similares.

La introducción de Jack marca el comienzo del fin de nuestro viaje. En los capítulos 10 y 11 escribiremos un compilador que traduce los programas Jack en código VM, y en el capítulo 12 desarrollaremos un sistema operativo simple para la plataforma Jack/Hack. Esto completará la construcción de la computadora. Con eso en mente, es importante decir desde el principio que el objetivo de este capítulo no es convertirte en un programador de Jack. Tampoco afirmamos que Jack sea un idioma importante fuera del contexto de Nand a Tetris. Más bien, vemos a Jack como un andamio necesario para los capítulos 10-12, en los que construiremos un compilador y un sistema operativo que hagan posible a Jack.

Si tiene alguna experiencia con un lenguaje de programación moderno orientado a objetos, inmediatamente se sentirá como en casa con Jack. Por lo tanto, comenzamos el capítulo con algunos ejemplos representativos de programas de Jack. Todos estos programas pueden ser compilados por el compilador Jack suministrado en `nand2tetris/tools`.



El código de máquina virtual generado por el compilador se puede ejecutar tal cual en cualquier implementación de máquina virtual, incluido el emulador de máquina virtual proporcionado. Como alternativa, puede traducir el código de máquina virtual compilado al lenguaje de máquina mediante el traductor de máquina virtual integrado en los capítulos 7 y 8. El código ensamblador resultante se puede ejecutar en el emulador de CPU suministrado o traducirse a código binario y ejecutarse en la plataforma de hardware construida en los capítulos 1 a 5.

Jack es un lenguaje simple, y esta simplicidad tiene un propósito. Primero, puedes aprender (y desaprender) Jack en aproximadamente una hora. En segundo lugar, el lenguaje Jack fue cuidadosamente diseñado para prestarse muy bien a las técnicas comunes de compilación. Como resultado, puede escribir un *compilador elegante de Jack* con relativa facilidad, como lo haremos en los capítulos 10 y 11. En otras palabras, la estructura deliberadamente simple de Jack está diseñada para ayudar a descubrir la infraestructura de software de lenguajes modernos como Java y C#. En lugar de separar los compiladores y los entornos de tiempo de ejecución de estos lenguajes, nos parece más instructivo construir un compilador y un entorno de tiempo de ejecución nosotros mismos, centrándonos en las ideas más importantes que subyacen a su construcción. Esto se hará más adelante, en los últimos tres capítulos del libro. Ahora, saquemos a Jack de la caja.

---

## 9.1 Ejemplos

Jack se explica por sí mismo. Por lo tanto, aplazamos la especificación del lenguaje a la siguiente sección y comenzamos con ejemplos. El primer ejemplo es el inevitable *programa Hello World*. El segundo ejemplo ilustra la programación procedimental y el procesamiento de matrices. El tercer ejemplo ilustra cómo se pueden implementar tipos de datos abstractos en el lenguaje Jack. El cuarto ejemplo ilustra una implementación de lista enlazada utilizando las capacidades de control de objetos del lenguaje.

A lo largo de los ejemplos, discutimos brevemente varios modismos orientados a objetos y estructuras de datos de uso común. Suponemos que el lector tiene una familiaridad básica con estos temas. Si no, sigue leyendo, te las arreglarás.

**Ejemplo 1: Hola mundo:** El programa que se muestra en la [figura 9.1](#) ilustra varias características básicas de Jack. Por convención, cuando ejecutamos un

Jack, la ejecución siempre comienza con la función `Main.main`. Por lo tanto, cada programa Jack debe incluir al menos una clase, llamada `Main`, y esta clase debe incluir al menos una función, llamada `Main.main`. Esta convención se ilustra en la figura 9.1.

```
/** Prints "Hello World". File name: Main.jack */
class Main {
    function void main() {
        do Output.printString("Hello World");
        do Output.println(); // New line
        return;              // The return statement is mandatory
    }
}
```

**Figura 9.1** *Hola mundo*, escrito en el lenguaje Jack.

Jack viene con una biblioteca de *clases estándar* cuya API completa se proporciona en el apéndice 6. Esta biblioteca de software, también conocida como *Jack OS*, amplía el lenguaje básico con varias abstracciones y servicios, como funciones matemáticas, procesamiento de cadenas, gestión de memoria, gráficos y funciones de entrada / salida. El programa Hello World invoca dos de estas funciones del sistema operativo, lo que afecta a la salida del programa. El programa también ilustra los formatos de comentarios admitidos por Jack.

**Ejemplo 2: Programación procedimental y manejo de matrices:** Jack presenta declaraciones típicas para manejar la asignación y la iteración. El programa que se muestra en la figura 9.2 ilustra estas capacidades en el contexto del procesamiento de matrices.

```

/** Inputs a sequence of integers, and computes their average. */
class Main {
    function void main() {
        var Array a;    // Jack arrays are not typed
        var int length;
        var int i, sum;
        let i = 0;
        let sum = 0;
        let length = Keyboard.readInt("How many numbers? ");
        let a = Array.new(length); // Constructs the array
        while (i < length) {
            let a[i] = Keyboard.readInt("Enter a number: ");
            let sum = sum + a[i];
            let i = i + 1;
        }
        do Output.printString("The average is: ");
        do Output.printInt(sum / length);
        do Output.println();
        return;
    }
}

```

**Figura 9.2** Programación de procedimientos típica y manejo simple de matrices. Utiliza los servicios de las clases de sistema operativo Array, Keyboard y Output.

La mayoría de los lenguajes de programación de alto nivel incluyen la declaración de matriz como parte de la sintaxis básica del lenguaje. En Jack, hemos optado por tratar las matrices como instancias de una clase Array, que es parte del sistema operativo que extiende el lenguaje básico. Esto se hizo por razones pragmáticas, ya que simplifica la construcción de los compiladores Jack.

**Ejemplo 3: Tipos de datos abstractos:** Cada lenguaje de programación presenta un conjunto fijo de tipos de datos primitivos, de los cuales Jack tiene tres: int, char y boolean. En los lenguajes basados en objetos, los programadores pueden introducir nuevos tipos creando clases que representen tipos de datos abstractos según sea necesario. Por ejemplo, supongamos que deseamos dotar a Jack de la capacidad de manejar números racionales como  $2/3$  y  $314159/100000$  sin pérdida de precisión. Esto se puede hacer desarrollando una clase Jack independiente diseñada para crear y manipular objetos de fracción de la forma  $x/y$ , donde  $x$  e  $y$  son enteros. Esta clase puede proporcionar una abstracción de fracciones a cualquier programa Jack que necesite representar y manipular números racionales. Ahora pasamos a describir cómo

se puede usar y desarrollar una clase Fraction. Este ejemplo ilustra la programación típica basada en objetos de varias clases en Jack.

**Uso de clases:** La Figura 9.3a enumera un esqueleto de clase (un conjunto de firmas de método) que especifica algunos de los servicios que uno puede esperar razonablemente obtener de una abstracción de fracciones. Esta especificación a menudo se denomina *interfaz de programa de aplicación*. El código de cliente en la parte inferior de la figura ilustra cómo se puede usar esta API para crear y manipular objetos de fracción.

```
/** Represents the Fraction type and related operations (class skeleton) */
class Fraction {

    /** Constructs a (reduced) fraction from x and y */
    constructor Fraction new(int x, int y)

    /** Returns the numerator of this fraction */
    method int getNumerator()

    /** Returns the denominator of this fraction */
    method int getDenominator()

    /** Returns the sum of this fraction and the other one */
    method Fraction plus(Fraction other)

    /** Prints this fraction in the format x/y */
    method void print()

    /** Disposes this fraction */
    method void dispose() {

    // More fraction-related methods:
    // minus, times, div, invert, etc.
    }

}

// Computes and prints the sum of 2/3 and 1/5
class Main {
    function void main() {
        // Creates 3 fraction variables (pointers to Fraction objects)
        var Fraction a, b, c;
        let a = Fraction.new(4,6); // a = 2/3
        let b = Fraction.new(1,5); // b = 1/5

        // Adds up the two fractions, and prints the result
        let c = a.plus(b); // c = a + b
        do c.print(); // Should print "13/15"
        return;
    }
}
```

**Figura 9.3a** Fracción API (arriba) y clase Jack de muestra que lo usa para crear y manipular Fraction (Objetos de fracción).

La Figura 9.3a ilustra un importante principio de ingeniería de software: los usuarios de una abstracción (como Fraction) no tienen que saber nada sobre su implementación. Todo lo que necesitan es la interfaz de clase, también conocida como API. La API informa qué funcionalidad ofrece la clase y cómo usar esta funcionalidad. Eso es todo lo que el cliente necesita saber.

**Implementación de clases:** Hasta ahora, solo hemos visto la perspectiva del cliente de la clase Fraction, la vista desde la cual Fraction se usa como una caja negra

abstracción. [Figura 9.3b](#) ListasUno posible implementación de esta abstracción.

```
/** Represents the Fraction type and related operations. */
class Fraction {
    // Each Fraction object has a numerator and a denominator
    field int numerator, denominator;
    /** Constructs a (reduced) fraction from x and y */
    constructor Fraction new(int x, int y) {
        let numerator = x;
        let denominator = y;
        do reduce(); // Reduces this fraction
        return this; // Returns a reference to the new object
    }
    // Reduces this fraction
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {
            let numerator = numerator / g;
            let denominator = denominator / g;
        }
        return;
    }
    // Computes the greatest common divisor of two given integers
    function int gcd(int a, int b) {
        // Applies Euclid's algorithm
        var int r;
        while (~(b = 0)) {
            let r = a - (b * (a / b)); // r = remainder
            let a = b;
            let b = r;
        }
        return a;
    }
}
// The Fraction class declaration continues on the top right

/** Accessors */
method int getNumerator() {
    return numerator;
}
method int getDenominator() {
    return denominator;
}
/** Returns the sum of this fraction and the other one */
method Fraction plus(Fraction other) {
    var int sum;
    let sum = (numerator * other.getDenominator()) +
              (other.getNumerator() * denominator);
    return Fraction.new(sum, denominator *
                        other.getDenominator());
}
/** Prints this fraction in the format x/y */
method void print() {
    do Output.printInt(numerator);
    do Output.printString("/");
    do Output.printInt(denominator);
    return;
}
/** Disposes this fraction */
method void dispose() {
    // Frees the memory held by this object
    do Memory.deAlloc(this);
    return;
}
// More fraction-related methods can come here:
// minus, times, div, invert, etc.
} // End of the Fraction class declaration.
```

**Figura 9.3b** Una implementación Jack de la abstracción de fracciones.

La clase Fraction ilustra varias características clave de la programación basada en objetos en Jack. *Los campos* especifican propiedades de objeto (también denominadas *variables miembro*). *Los constructores* son subrutinas que crean nuevos objetos y *los métodos* son subrutinas que operan en el objeto actual (al que se hace referencia con la palabra clave *this*). *Las funciones* son subrutinas de nivel de clase (también llamadas *métodos estáticos*) que no operan en ningún objeto en particular. La clase Fraction también muestra todos los tipos de instrucciones disponibles en el lenguaje Jack: *let*, *do*, *if*, *while* y *return*. La clase Fraction es, por supuesto, solo un ejemplo del número ilimitado de clases que se pueden crear en Jack para admitir cualquier objetivo de programación concebible.

**Ejemplo 4: Implementación de lista vinculada:** la lista de estructura de datos se define recursivamente como un valor, seguido de una lista. El valor null, el caso base de la definición, también se considera una lista. [La Figura 9.4](#) muestra una posible implementación Jack de una lista de números enteros. Este ejemplo ilustra cómo se puede usar Jack para realizar una estructura de datos importante, ampliamente utilizada en informática.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> /** Represents a list of integers. */ class List {     field int data;    // A list consists of an int value,     field List next;  // followed by a List      /* Creates a list whose head is car and whose tail is cdr */     // These identifiers are used in memory of the Lisp programming language     constructor List new(int car, List cdr) {         let data = car;         let next = cdr;         return this;     }      /* Accessors */     method int getData() {return data;}     method List getNext() {return next;}      /* Prints the elements of this list */     method void print() {         // Initializes a pointer to the first element of this list         var List current;         let current = this;         // Iterates through the list         while (~(current = null)) {             do Output.printInt(current.getData());             do Output.printChar(32); // Prints a space             let current = current.getNext();         }         return;     } } // The List class declaration continues on the top right </pre> | <pre> /* Disposes this List */ method void dispose() {     // Disposes the tail of this list, recursively     if (~(next = null)) {         do next.dispose();     }     // Uses an OS routine to free the memory     // held by this object.     do Memory.deAlloc(this);     return; } // More list-related methods can come here } // End of the List class declaration. </pre> |
| <pre> // Client code example: // Creates, prints, and disposes the list (2,3,5), // which is shorthand for the list (2,(3,(5,null))). // (This code can appear in any Jack class): ... var List v; let v = List.new(5,null); let v = List.new(2,List.new(3,v)); do v.print();           // Prints 2 3 5 do v.dispose();         // Disposes the list ... </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                    |

**Figura 9.4** Implementación de la lista enlazada en Jack (izquierda y arriba a la derecha) y uso de la muestra (abajo a la derecha).

**El sistema operativo:** Los programas Jack hacen un uso extensivo del sistema operativo Jack, que se discutirá y desarrollará en el capítulo 12. Por ahora, basta con decir que los programas Jack usan los servicios del sistema operativo de manera abstracta, sin prestar atención a su implementación subyacente. Los programas Jack pueden usar los servicios del sistema operativo directamente, no es necesario incluir o importar ningún código externo.

La SG consta de ocho clases, resumidas en la figura 9.5 y documentadas en el apéndice 6.



| <i>OS class</i> | <i>Services</i>                                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------------------------|
| Math            | Common mathematical operations:<br>max(int,int), sqrt(int), ...                                                       |
| String          | Represents strings and related operations:<br>length(), charAt(int) , ...                                             |
| Array           | Represents arrays and related operations:<br>new(int), dispose()                                                      |
| Output          | Facilitates text output to the screen:<br>printString(String), printInt(int), println(), ...                          |
| Screen          | Facilitates graphics output to the screen : setColor(boolean),<br>drawPixel(int,int), drawLine(int,int,int,int) , ... |
| Keyboard        | Facilitates input from the keyboard:<br>readLine(String), readInt(String) , ...                                       |
| Memory          | Facilitates access to the host RAM:<br>peek(int), poke(int,int), alloc(int), deAlloc(Array)                           |
| Sys             | Facilitates execution-related services: halt(), wait(int), ...                                                        |

**Figura 9.5** Servicios del sistema operativo (resumen). La API completa del sistema operativo se proporciona en el apéndice 6.

---

## 9.2 La especificación del lenguaje Jack

Esta sección se puede leer una vez y luego se puede utilizar como referencia técnica, para ser consultada según sea necesario.

### 9.2.1 Elementos sintácticos

Un programa Jack es una secuencia de tokens, separados por una cantidad arbitraria de espacios en blanco y comentarios. Los tokens pueden ser símbolos, palabras reservadas, constantes e identificadores, como se indica en la [figura 9.6](#).

|                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|--------------------|--------------------------|-----------------|--------------------|-----------------------|----------------------------------|------------|-------------------|-----------------|------|------------------|
| White space and comments             | Space characters, newline characters, and comments are ignored.<br>The following comment formats are supported:<br>// Comment to end of line<br>/* Comment until closing */<br>/** Aimed at software tools that extract API documentation. */                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
| Symbols                              | () Used for grouping arithmetic expressions and for enclosing argument-lists (in subroutine calls) and parameter-lists (in subroutine declarations)<br>[] Used for array indexing<br>{ } Used for grouping program units and statements<br>, Variable-list separator<br>; Statement terminator<br>= Assignment and comparison operator<br>. Class membership<br>+ - * / &   ~ < > Operators                                                                                                                                                                                                                                                                                                                                                                                                         |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
| Reserved words                       | <table> <tr> <td>class, constructor, method, function</td><td>Program components</td></tr> <tr> <td>int, boolean, char, void</td><td>Primitive types</td></tr> <tr> <td>var, static, field</td><td>Variable declarations</td></tr> <tr> <td>let, do, if, else, while, return</td><td>Statements</td></tr> <tr> <td>true, false, null</td><td>Constant values</td></tr> <tr> <td>this</td><td>Object reference</td></tr> </table>                                                                                                                                                                                                                                                                                                                                                                    | class, constructor, method, function | Program components | int, boolean, char, void | Primitive types | var, static, field | Variable declarations | let, do, if, else, while, return | Statements | true, false, null | Constant values | this | Object reference |
| class, constructor, method, function | Program components                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
| int, boolean, char, void             | Primitive types                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
| var, static, field                   | Variable declarations                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
| let, do, if, else, while, return     | Statements                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
| true, false, null                    | Constant values                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
| this                                 | Object reference                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
| Constants                            | <ul style="list-style-type: none"> <li>• <i>Integer constants</i> are values in the range 0 to 32767. Negative integers are not constants but rather expressions consisting of a unary minus operator applied to an integer constant. The resulting valid range of values is -32768 to 32767 (the former can be obtained using the expression -32767-1).</li> <li>• <i>String constants</i> are enclosed within double quote (") characters and may contain any character except newline or double quote. These characters are supplied by the OS functions <code>String.newLine()</code> and <code>String.doubleQuote()</code>.</li> <li>• <i>Boolean constants</i> are <code>true</code> and <code>false</code>.</li> <li>• The <code>null</code> constant signifies a null reference.</li> </ul> |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |
| Identifiers                          | Identifiers are composed from arbitrarily long sequences of letters (A-Z, a-z), digits (0-9), and "_". The first character must be a letter or "_". The language is case sensitive: x and X are treated as different identifiers.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                      |                    |                          |                 |                    |                       |                                  |            |                   |                 |      |                  |

**Figura 9.6** Los elementos de sintaxis del lenguaje Jack.

## 9.2.2 Estructura del programa

Un programa Jack es una colección de una o más clases almacenadas en la misma carpeta. Una clase debe llamarse `Main` y esta clase debe tener una función llamada `main`. La ejecución de un programa Jack compilado siempre comienza con la función `Main.main`.

La unidad básica de programación en Jack es una *clase*. Cada clase *Xxx* reside en un archivo separado llamado `Xxx.jack` y se compila por separado. Por convención, los nombres de clase comienzan con una letra mayúscula. El nombre del archivo debe ser idéntico



al nombre de la clase, incluidas las mayúsculas. La declaración de clase tiene la siguiente estructura:

```
class name {  
    field variable declarations    // Must precede the subroutine declarations  
    static variable declarations  // Must precede the subroutine declarations  
    subroutine declarations       // Constructor, method and function declarations, in any order  
}
```

Cada declaración de clase especifica un nombre a través del cual se puede acceder globalmente a los servicios de clase. Luego viene una secuencia de cero o más declaraciones de campo y cero o más declaraciones de variables estáticas. Luego viene una secuencia de una o más declaraciones de subrutinas, cada una de las cuales define un *método*, una *función* o un *constructor*.

*Los métodos* están diseñados para operar en el objeto actual. *Las funciones* son métodos estáticos *de nivel de clase* que no están asociados a ningún objeto determinado. *Los constructores* crean y devuelven nuevos objetos del tipo de clase. Una declaración de subrutina tiene la siguiente estructura:

```
subroutine type name (parameter-list) {  
    local variable declarations  
    statements  
}
```

donde *subrutina* es constructor, método o función. Cada subrutina tiene un *nombre* a través del cual se puede acceder a ella y un *tipo* que especifica el tipo de datos del valor devuelto por la subrutina. Si la subrutina está diseñada para no devolver ningún valor, su tipo se declara nulo. La *lista de parámetros* es una lista separada por comas de pares de <identificador de tipo>, por ejemplo, (int x, signo booleano, fracción g).

Si la subrutina es un *método* o una *función*, su tipo de retorno puede ser cualquiera de los tipos de datos primitivos admitidos por el lenguaje (int, char o booleano), cualquiera de los tipos de clase proporcionados por la biblioteca de clases estándar (String o Array) o cualquiera de los tipos realizados por otras clases del programa (por ejemplo, Fraction o List). Si la subrutina es un *constructor*, puede tener un nombre arbitrario, pero su tipo debe ser el nombre de la clase a la que pertenece. Una clase puede tener 0,

1 o más constructores. Por convención, uno de los constructores se llama `nuev`

Siguiendo su especificación de interfaz, la declaración de subrutina contiene una secuencia de cero o más declaraciones de variables locales (instrucciones `var`) y luego una secuencia de una o más instrucciones. Cada subrutina debe terminar con la expresión de retorno de la instrucción `.`. En el caso de una subrutina `void`, cuando no hay nada que devolver, la subrutina debe terminar con la declaración `return` (que puede verse como una abreviatura de `return void`, donde `void` es una constante que representa "nada"). Los constructores deben terminar con la instrucción `return this`. Esta acción devuelve la dirección de memoria del objeto recién construido, denotado así (los constructores de Java hacen lo mismo, implícitamente).

### 9.2.3 Tipos de datos

El tipo de datos de una variable es *primitivo* (`int`, `char` o `booleano`) o *ClassName*, donde *ClassName* es `String`, `Array` o el nombre de una clase que reside en la carpeta del programa.

**Tipos primitivos:** Jack presenta tres tipos de datos primitivos:

`int`: entero de 16 bits de complemento de dos

`char`: no negativo, entero de 16 bits

`Booleano`: verdadero o falso

Cada uno de los tres tipos de datos primitivos `int`, `char` y `boolean` se representa internamente como un valor de 16 bits. El lenguaje está débilmente tipado: `type` se puede asignar a una variable de cualquier tipo sin

**Arrays:** Las matrices se declaran mediante la clase de sistema operativo `Array`. Se accede a los elementos de matriz utilizando la notación `arr[i]` típica, donde el índice del primer elemento es 0. Se puede obtener una matriz multidimensional creando una matriz de matrices. Los elementos de la matriz no se escriben y los diferentes elementos de la misma matriz pueden tener diferentes tipos. La declaración de una matriz crea una referencia, mientras que la matriz propiamente dicha se construye ejecutando la llamada al constructor `Array.new (arrayLength)`. Para ver un ejemplo de cómo trabajar con matrices, consulte la [figura 9.2](#).

**Tipos de objeto:** Una clase Jack que contiene al menos un método define un tipo de objeto. Como es típico en la programación orientada a objetos, la creación de objetos es un asunto de dos pasos. Aquí hay un ejemplo:

```
// This client code example uses the Car and Employee classes, whose code is not shown here.
// The Car class has two fields: model (a String) and licensePlate (a String).
// The Employee class has two fields: name (a String) and car (a Car).
...
// Declares a Car object and two Employee objects (three pointer variables):
var Car c;
var Employee emp1, emp2;
...
// Constructs a new car:
let c = Car.new("Aston Martin", "007"); // Sets c to the base address of a memory block
                                         // containing the new car's data.
// Constructs a new employee, and assigns a car to it:
let emp1 = Employee.new("Bond", c);
...
// Creates an alias of Bond:
let emp2 = emp1; // Only the reference (address) is copied, no new object is constructed.
// We now have two Employee pointers referring to the same object.
```

**Cadenas:** Las cadenas son instancias de la clase del sistema operativo String, que implementa cadenas como matrices de valores char. El compilador Jack reconoce la sintaxis "foo" y la trata como un objeto String. Se puede acceder al contenido de un objeto String mediante `charAt(index)` y otros métodos documentados en la API de la clase String (consulte el apéndice 6). Aquí hay un ejemplo:

```
var String s; // An object variable
var char c;   // A primitive variable
...
let s = "Hello World"; // Sets s to the String object "Hello World"
let c = s.charAt(6);    // Sets c to 87, the integer character code of 'W'
```

La instrucción `let s = "Hello World"` es equivalente a la instrucción `let s = String.new(11)`, seguido de las once llamadas al método `do s.appendChar(72)`, ..., `do s.appendChar(100)`, donde el argumento de `appendChar` es el entero del carácter

código. De hecho, así es exactamente como el compilador maneja la traducción de let

sTenga en cuenta que el modismo de un solo carácter, por ejemplo, 'H', no es respaldado por el lenguaje Jack. La única forma de representar a un usar su código de caracteres enteros o . El conjunto de caracteres Hack

se documenta en el apéndice 5.

**Conversiones de tipos:** Jack tiene un tipo débil: la especificación del lenguaje no define lo que sucede cuando se asigna un valor de un tipo a una variable de un tipo diferente. Las decisiones de permitir tales operaciones de fundición, y cómo manejarlas, se dejan a los compiladores específicos de Jack. Esta especificación insuficiente es intencional, lo que permite la construcción de compiladores mínimos que ignoran los problemas de escritura. Dicho esto, se espera que todos los compiladores de Jack admitan y realicen automáticamente las siguientes asignaciones.

- Se puede asignar un valor de carácter a una variable entera, y viceversa, de acuerdo con la especificación del conjunto de caracteres Jack (apéndice 5). Ejemplo:

```
var char c;  
let c = 33; // 'A'  
  
// Equivalently:  
var String s;  
let s = "A";  
let c = s.charAt(0);
```

- Un número entero se puede asignar a una variable de referencia (de cualquier tipo de objeto), en cuyo caso se interpreta como una dirección de memoria. Ejemplo:

```
var Array arr; // Creates a pointer variable  
let arr = 5000; // Sets arr to 5000  
let arr[100] = 17; // Sets the contents of memory address 5100 to 17
```

- Una variable de objeto se puede asignar a una variable de matriz y viceversa. Esto permite acceder a los campos del objeto como elementos de la matriz, y viceversa. Ejemplo:

```
// Creates the array [2,5]:
var Array arr;
let arr = Array.new(2);
let arr[0] = 2;
let arr[1] = 5;

// Creates the Fraction 2/5:
var Fraction x;
let x = arr; // sets x to the base address of the memory block
              // representing the array [2,5]

do Output.printInt(x.getNumerator()) // prints "2"
do x.print()                        // prints "2/5"
```

## 9.2.4 Variables

Jack presenta cuatro tipos de variables. *Las variables estáticas* se definen a nivel de clase y todas las subrutinas de clase pueden acceder a ellas. *Las variables de campo*, también definidas a nivel de clase, se utilizan para representar las propiedades de objetos individuales y pueden acceder a ellas todos los constructores y métodos de clase. *Las subrutinas* usan las variables locales para los cálculos locales y *las variables de parámetro* representan los argumentos que el autor de la llamada pasó a la subrutina. Los valores locales y de parámetros se crean justo antes de que la subrutina comience a ejecutarse y se reciclan cuando la subrutina regresa. [La Figura 9.7](#) da los detalles. El *ámbito* de una variable es la región del programa en la que se reconoce la variable.

| Kind                | Description                                                                                                                                                                                                  | Declared in            | Scope                                     |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------------------------------------------|
| Class variables     | <i>static type</i> <i>varName1</i> , <i>varName2</i> , ... ;<br>One copy of each static variable exists, and this copy is shared by all the class subroutines (like <i>private static variables</i> in Java) | Class declaration      | The class in which they are declared      |
| Field variables     | <i>field type</i> <i>varName1</i> , <i>varName2</i> , ... ;<br>Every object (instance of the class) has a private copy of the field variables (like <i>member variables</i> in Java)                         | Class declaration      | The class in which they are declared      |
| Local variables     | <i>var type</i> <i>varName1</i> , <i>varName2</i> , ... ;<br>Created when the subroutine starts running and disposed when the subroutine returns.                                                            | Subroutine declaration | The subroutine in which they are declared |
| Parameter variables | Represent the arguments passed to the subroutine. Treated like local variables whose values are initialized by the subroutine caller.                                                                        | Subroutine declaration | The subroutine in which they are declared |

**Figura 9.7** Tipos de variables en el lenguaje Jack. A lo largo de la tabla, *subrutina* se refiere a un constructor, un método o un constructor.

**Inicialización de variables:** Las variables estáticas no se inicializan y depende del programador escribir código que las inicialice antes de usarlas. Las variables de campo no se inicializan; Se espera que sean inicializados por el constructor de clase o constructores. Las variables locales no se inicializan y depende del programador inicializarlas. Las variables de parámetro se inicializan en los valores de los argumentos pasados por el autor de la llamada.

**Visibilidad de variables:** no se puede acceder a las variables estáticas y de campo directamente desde fuera de la clase en la que están definidas. Solo se puede acceder a ellos a través de métodos de descriptor de acceso y mutador, según lo facilitado por el diseñador de clases.

### 9.2.5 Declaraciones

El lenguaje Jack presenta cinco declaraciones, como se ve en [la figura 9.8](#).

| <i>Statement</i> | <i>Syntax</i>                                                                                                       | <i>Description</i>                                                                                                                                               |
|------------------|---------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| let              | let <i>varName</i> = <i>expression</i> ; or:<br>let <i>varName</i> [ <i>expression1</i> ] =<br><i>expression2</i> ; | An assignment operation.<br>The variable kind may be <i>static</i> ,<br><i>local</i> , <i>field</i> , or <i>parameter</i> .                                      |
| if               | if ( <i>expression</i> ) {<br><i>statements1</i> ;<br>} else {<br><i>statements2</i> ;<br>}                         | Typical <i>if</i> statement, with an optional<br><i>else</i> clause.<br>The curly brackets are mandatory,<br>even if <i>statements</i> is a single<br>statement. |
| while            | while ( <i>expression</i> ) {<br><i>statements</i> ;<br>}                                                           | Typical <i>while</i> statement.<br>The curly brackets are mandatory,<br>even if <i>statements</i> is a single<br>statement.                                      |
| do               | do <i>function-or-method-call</i> ;                                                                                 | Used to call a function or a method<br>for its effect, ignoring the returned<br>value, if any.                                                                   |
| return           | return <i>expression</i> ; or<br>return;                                                                            | Used to return a value from a<br>subroutine.<br>The second form must be used by<br>void subroutines.<br>Constructors must return the value<br><i>this</i> .      |

**Figura 9.8** Declaraciones en el idioma de Jack.

### 9.2.6 Expresiones

Una expresión de Jack es una de las siguientes:

- Una *constante*
- Un *nombre de variable* en el ámbito. La variable puede ser *estática*, *de campo*, *local* o *parámetro*
- La palabra clave *this*, que denota el objeto actual (no se puede usar en funciones)
- Un *elemento de matriz* que usa la sintaxis *arr[expression]*, donde *arr* es un nombre de variable de tipo Array en el ámbito
- Una llamada de *subrutina* que devuelve un tipo no void
- Una expresión prefijada por uno de los operadores unarios - o ~:
  - *expresión*: negación aritmética
  - ~ *expresión*: negación booleana (bit a bit para enteros)
- Una expresión de la forma *expression op expression* donde *op* es uno de los siguientes operadores binarios:
  - + - \* /: Operadores aritméticos enteros
  - & |: Operadores booleanos And y booleanos Or (bit a bit para enteros)
  - < > =: Operadores de comparación
- (*expresión*): una expresión entre paréntesis

**Prioridad del operador y orden de evaluación:** el lenguaje no define la prioridad del operador, excepto que las expresiones entre paréntesis se evalúan primero. Por lo tanto, el valor de la expresión  $2 + 3 * 4$  es impredecible, mientras que  $2 + (3 * 4)$  se garantiza que se evalúe como 14. El compilador Jack suministrado en Nand to Tetris (así como el compilador que desarrollaremos en los capítulos 10 y 11) evalúa las expresiones de izquierda a derecha, por lo que la expresión  $2 + 3 * 4$  se evalúa como 20. Una vez más, si desea obtener el resultado algebraicamente correcto, use  $2 + (3 * 4)$ .

La necesidad de usar paréntesis para hacer cumplir la prioridad del operador hace que las expresiones de Jack sean un poco engorrosas. Sin embargo, esta falta de prioridad formal del operador es intencional, ya que simplifica la implementación de los compiladores Jack. Los diferentes compiladores Jack son bienvenidos a especificar una prioridad de operador y agregarla a la documentación del lenguaje, si así lo desea.

### **9.2.7 Llamadas de subrutina**



Una llamada a una subrutina invoca una función, un constructor o un método para su efecto, utilizando la sintaxis general *subroutineName (exp1, exp2, ..., expn)*, donde cada argumento *exp* es una expresión. El número y el tipo de los argumentos deben coincidir con los de los parámetros de la subrutina, como se especifica en la declaración de la subrutina. Los paréntesis deben aparecer incluso si la lista de argumentos está vacía.

Las subrutinas se pueden llamar desde la clase en la que están definidas, o desde otras clases, de acuerdo con las siguientes reglas de sintaxis:

### **Llamadas a funciones / Llamadas al constructor:**

- *className.functionalityName (exp1, exp2, ... , expn)*
- *className.constructorName (exp1, exp2, ... , expn)*

Siempre se debe especificar *className* , incluso si la función o el constructor están en la misma clase que el autor de la llamada.

### **Llamadas de método:**

- *varName.methodName (exp1, exp2, ... , expn)*

Aplica el método al objeto al que hace referencia

*varName.* • *methodName (exp1, exp2, ... , expn)*

Aplica el método al *objeto actual*. Igual que *esto.methodName (exp1, exp2, ... , expn)*.

A continuación, se muestran ejemplos de llamadas de subrutinas:

```

class Foo {
    ...
    method void f() {
        var Bar b;           // Declares a local variable of class type Bar
        var int i;            // Declares a local variable of primitive type int
        ...
        do Foo.g()            // Calls function g of the current class
        do Bar.h()            // Calls function h of class Bar
        do m()                // Calls method m of the current class, on the this object
        do b.q()              // Calls method q of class Bar, on object b
        let i = w(b.s(), Foo.t()) // Calls method w on the this object,
                                // Calls method s of class Bar on object b,
                                // Calls function or constructor t of class Foo.
    }
}

```

### 9.2.8 Construcción y eliminación de objetos

La construcción de objetos se realiza en dos etapas. Primero, se declara una variable de referencia (puntero a un objeto). Para completar la construcción del objeto (si así lo desea), el programa debe llamar a un constructor de la clase del objeto. Por lo tanto, una clase que implementa un tipo (por ejemplo, Fraction) debe presentar al menos un constructor. Los constructores Jack pueden tener nombres arbitrarios; Por convención, uno de ellos se llama nuevo.

Los objetos se construyen y asignan a variables usando el modismo `let varName = className.constructorName (exp1, exp2, ... , expn)`, por ejemplo, sea `c = Circle.new (x,y,50)`. Los constructores suelen incluir código que inicializa los campos del nuevo objeto en los valores de argumento pasados por el autor de la llamada.

Cuando un objeto ya no es necesario, se puede desechar para liberar la memoria que ocupa. Por ejemplo, supongamos que el objeto al que apunta `c` ya no es necesario. El objeto se puede desasignar de la memoria llamando a la función del sistema operativo `Memory.deAlloc (c)`. Como Jack no tiene basura, sejo de mejores prácticas es que cada clase que represente un objeto debe presentar un método que encapsula adecuadamente esta asignación `dispose()`.

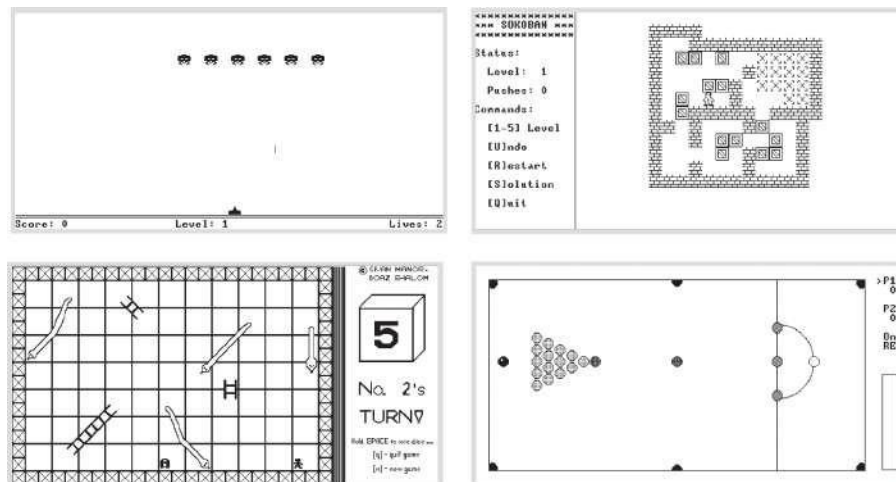
Las figuras 9.3 y 9.4 dan ejemplos. Para evitar pérdidas de memoria, se recomienda a los programadores de Jack que desechen los objetos cuando ya no sean necesarios.

---

### 9.3 Escribir aplicaciones de Jack

Jack es un lenguaje de propósito general que se puede implementar en diferentes plataformas de hardware. En Nand to Tetris desarrollamos un *compilador Jack sobre la plataforma Hack*, y por lo tanto es natural discutir las aplicaciones Jack en el contexto de Hack.

**Ejemplos:** La Figura 9.9 muestra capturas de pantalla de cuatro programas Jack de muestra. En términos generales, la plataforma Jack / Hack se presta muy bien a juegos interactivos simples como Pong, Snake, Tetris y clásicos similares. Su carpeta `projects/09/Square` incluye el código Jack completo de un programa interactivo simple que permite al usuario mover una imagen cuadrada en la pantalla usando las cuatro teclas de flecha del teclado.



**Figura 9.9** Capturas de pantalla de las aplicaciones Jack ejecutándose en el ordenador Hack.

Ejecutar este programa mientras revisa su código fuente de Jack es una buena manera de aprender a usar Jack para escribir aplicaciones gráficas interactivas. Más adelante en el capítulo describimos cómo compilar y ejecutar programas Jack utilizando las herramientas suministradas.

**Diseño e implementación de aplicaciones:** El desarrollo de software siempre debe basarse en una planificación cuidadosa, especialmente cuando se realiza sobre una plataforma de hardware espartana como la computadora Hack. En primer lugar, el diseñador del programa debe

Considere las limitaciones físicas del hardware y planifique en consecuencia. Para empezar, las dimensiones de la pantalla de la computadora limitan el tamaño de las imágenes gráficas que puede manejar el programa. Del mismo modo, se debe considerar el rango de comandos de entrada / salida del lenguaje y la velocidad de ejecución de la plataforma para obtener una expectativa realista de lo que se puede y no se puede hacer.

El proceso de diseño normalmente comienza con una descripción conceptual del comportamiento del programa deseado. En el caso de programas gráficos e interactivos, esto puede tomar la forma de dibujos manuscritos de pantallas típicas. A continuación, normalmente se diseña una arquitectura basada en objetos del programa. Esto implica la identificación de *clases*, *campos* y *subrutinas*. Por ejemplo, si se supone que el programa permite al usuario crear objetos cuadrados y moverlos por la pantalla usando las teclas de flecha del teclado, tendrá sentido diseñar una clase *Square* que encapsule estas operaciones usando métodos como *moveRight*, *moveLeft*, *moveUp* y *moveDown*, así como una subrutina constructora para crear cuadrados y una subrutina trituradora para eliminarlos. Además, tendrá sentido crear una clase *SquareGame* que lleve a cabo la interacción del usuario y una clase *Main* que ponga las cosas en marcha. Una vez que las API de estas clases se especifican cuidadosamente, se puede proceder a implementarlas, compilarlas y probarlas.

**Compilar y ejecutar programas Jack:** Todos los archivos *.jack* que componen el programa deben residir en la misma carpeta. Cuando aplique el compilador Jack a la carpeta del programa, cada archivo *.jack* de origen se traducirá en un archivo *.vm* correspondiente, almacenado en la misma carpeta del programa.

La forma más sencilla de ejecutar o depurar un programa Jack compilado es cargar la carpeta del programa en el emulador de VM. El emulador cargará todas las funciones de VM en todos los archivos *.vm* de la carpeta, uno tras otro. El resultado será una secuencia (posiblemente larga) de funciones de máquina virtual, enumeradas en el panel de código del emulador de máquina virtual con su *fileName completo.nombres de nombresdefunciones*. Cuando le indique al emulador que ejecute el programa, el emulador comenzará a ejecutar la función *OS Sys.init*, que luego llamará a la función *Main.main* en su programa Jack.

Alternativamente, puede usar un traductor de VM (como el integrado en los proyectos 7-8) para traducir el código de VM compilado, así como los ocho archivos de sistema operativo *tools/OS/\*.vm* suministrados, en un solo archivo *.asm* escrito en la máquina Hack

Idioma. A continuación, el código ensamblador se puede ejecutar en el emulador de CPU proporcionado. O bien, puede usar un ensamblador (como el integrado en el proyecto 6) para traducir el archivo .asm en un archivo .hack de código binario. A continuación, puede cargar un chip de computadora Hack (como el integrado en los proyectos 1-5) en el simulador de hardware o usar el chip de computadora incorporado, cargar el código binario en el chip ROM y ejecutarlo.

**El sistema operativo:** Los programas Jack hacen un uso extensivo de la biblioteca de *clases estándar del lenguaje*, a la que también nos referimos como *el sistema operativo*. En el proyecto 12 desarrollará la biblioteca de clases del sistema operativo en Jack (como Unix está escrito en C) y la compilará usando un compilador Jack. La compilación producirá ocho archivos .vm, que comprenden la implementación del sistema operativo. Si coloca estos ocho archivos .vm en la carpeta del programa, todas las funciones del sistema operativo serán accesibles para el código de la máquina virtual compilada, ya que pertenecen a la misma base de código (en virtud de pertenecer a la misma carpeta).

Actualmente, sin embargo, no hay necesidad de preocuparse por la implementación del sistema operativo. El emulador de VM suministrado, presenta una implementación Java incorporada del

Cuando el código de máquina virtual cargado en el emulador llama a una función del sistema operativo, por ejemplo, Math.sqrt, sucede una de dos cosas. Si la función del sistema operativo se encuentra en la base de código cargada, el emulador de VM la ejecuta, al igual que la ejecución de cualquier otra función de VM. Si la función del sistema operativo no se encuentra en la base de código cargada, el emulador ejecuta su implementación integrada.

---

## 9.4 Proyecto

A diferencia de los otros proyectos de este libro, este no requiere construir un módulo de hardware o software. Más bien, debe elegir alguna aplicación de su elección y construirla en Jack sobre la plataforma Hack.

**Objetivo:** La "agenda oculta" de este proyecto es familiarizarse con el lenguaje Jack, con dos propósitos: escribir el compilador Jack en los proyectos 10 y 11, y escribir el sistema operativo Jack en el proyecto 12.

**Contrato:** Adoptar o inventar una idea de aplicación como un simple juego de computadora o algún programa interactivo. A continuación, diseñe y compile la aplicación.

**Recursos:** Necesitará las herramientas/JackCompiler suministradas para traducir su programa en un conjunto de archivos .vm, y las herramientas/VMEulator suministradas para ejecutar y probar el código compilado.

### Compilar y ejecutar un programa Jack

0. Cree una carpeta para el programa. Llamémoslo la carpeta del *programa*.
1. Escriba su programa Jack, un conjunto de una o más clases Jack, cada una almacenada en un archivo de *texto* *ClassName.jack* separado. Coloque todos estos archivos .jack en la carpeta del programa.
2. Compile la carpeta del programa usando el compilador Jack suministrado. Esto hará que el compilador traduzca todas las clases .jack que se encuentran en la carpeta en los archivos .vm correspondientes. Si se notifica un error de compilación, depure el programa y vuelva a compilar hasta que no se emitan mensajes de error.
3. En este punto, la carpeta del programa debe contener los archivos .jack de origen junto con los archivos .vm compilados. Para probar el programa compilado, cargue la carpeta del programa en el emulador de máquina virtual proporcionado y ejecute el código cargado. En caso de errores en tiempo de ejecución o comportamiento no deseado del programa, corrija el archivo correspondiente y vuelva al paso 2.

**Ejemplos de programas:** Su carpeta `nand2tetris/project/09` incluye el código fuente de un programa Jack interactivo completo de tres clases (Square). También incluye el código fuente de los programas Jack discutidos en este capítulo.

**Editor de mapas de bits:** Si desarrolla un programa que necesita gráficos de alta velocidad, es mejor diseñar *sprites* para representar los elementos gráficos clave del programa. Por ejemplo, la salida de la aplicación Sokoban representada en la [figura 9.9](#) consiste en varios sprites repetidos. Si desea diseñar tales sprites y escribirlos directamente en el mapa de memoria de la pantalla (sin pasar por los servicios de la clase OS Screen, que puede ser demasiado lenta), encontrará útil la herramienta `projects/09/BitmapEditor`.

**Una versión basada en la web del proyecto 9** está disponible en

[www.nand2tetris.org](http://www.nand2tetris.org).

---

## 9.5 Perspectiva

Jack es un lenguaje *basado en objetos*, lo que significa que admite objetos y clases, pero no herencia. En este sentido, se sitúa en algún lugar entre lenguajes procedimentales como Pascal o C y lenguajes orientados a objetos como Java o C++. Jack es ciertamente más ingenuo que cualquiera de estos lenguajes de programación de fuerza industrial. Sin embargo, su sintaxis y semántica básicas son similares a las de los lenguajes modernos.

Algunas características del lenguaje Jack dejan mucho que desear. Por ejemplo, su sistema de tipos primitivo es, bueno, bastante primitivo. Además, es un lenguaje de tipos débiles, lo que significa que la conformidad de tipos en las asignaciones y operaciones no se aplica estrictamente. Además, puede preguntarse por qué la sintaxis de Jack incluye palabras clave torpes como *do* y *let*, por qué cada subrutina debe terminar con una declaración *return*, por qué el lenguaje no aplica la prioridad del operador, etc.: puede agregar su queja favorita a la lista.

Todas estas idiosincrasias algo tediosas se introdujeron en Jack con un propósito: permitir el desarrollo de compiladores de Jack simples y mínimos, como lo haremos en los próximos dos capítulos. Por ejemplo, el análisis de una declaración (en cualquier idioma) es significativamente más fácil si el primer token de la declaración revela en qué declaración estamos. Es por eso que Jack usa una palabra clave *let* para prefijar las declaraciones de asignación. Por lo tanto, aunque la simplicidad de Jack puede ser una molestia al escribir *aplicaciones* de Jack, agradecerá esta simplicidad al escribir el *compilador* de Jack, como lo haremos en los próximos dos capítulos.

La mayoría de los lenguajes modernos se implementan con un conjunto de *clases estándar*, al igual que Jack. En conjunto, estas clases se pueden ver como un sistema operativo portátil y orientado al lenguaje. Sin embargo, a diferencia de las bibliotecas estándar de lenguajes de fuerza industrial, que cuentan con numerosas clases, Jack OS proporciona un conjunto mínimo de servicios, que sin embargo es suficiente para desarrollar aplicaciones interactivas simples.

Claramente, sería bueno extender el sistema operativo Jack para proporcionar concurrencia para admitir subprocesos múltiples, un sistema de archivos para almacenamiento permanente, sockets para comunicaciones, etc. Aunque todos estos servicios se pueden agregar al sistema operativo, los lectores quizás quieran perfeccionar sus habilidades de programación en otro lugar. Después de todo, no esperamos que Jack sea parte de tu vida más allá



Nand a Tetris. Por lo tanto, es mejor ver la plataforma Jack/Hack como un entorno determinado y aprovecharla al máximo. Eso es precisamente lo que hacen los programadores cuando escriben software para dispositivos integrados y procesadores dedicados que operan en entornos restringidos. En lugar de ver las limitaciones impuestas por la plataforma anfitriona como un problema, los profesionales lo ven como una oportunidad para mostrar su ingenio e ingenio. Eso es lo que se espera que hagas en el proyecto 9.

---

## 10 Compilador I: Análisis de sintaxis

Tampoco se pueden encontrar adornos del lenguaje sin disposición y expresión de pensamientos, ni se puede hacer que los pensamientos brillen sin la luz del lenguaje.

—Cicerón (106-43 a.C.)

El capítulo anterior presentó Jack, un lenguaje de programación simple basado en objetos con una sintaxis similar a Java. En este capítulo comenzamos a construir un compilador para el lenguaje Jack. Un *compilador* es un programa que traduce programas de un idioma de origen a un idioma de destino. El proceso de traducción, conocido como *compilación*, se basa conceptualmente en dos tareas distintas. Primero, tenemos que entender la sintaxis del programa fuente y, a partir de ella, descubrir la semántica del programa. Por ejemplo, el análisis del código puede revelar que el programa busca declarar una matriz o manipular un objeto. Una vez que conocemos la semántica, podemos reexpresarla utilizando la sintaxis del idioma de destino. La primera tarea, normalmente llamada *análisis de sintaxis*, se describe en este capítulo; la segunda tarea, la *generación de código*, se aborda en el capítulo siguiente.

¿Cómo podemos saber que un compilador es capaz de "entender" programas? Bueno, siempre que el código generado por el compilador esté haciendo lo que se supone que debe hacer, podemos asumir con optimismo que el compilador está funcionando correctamente. Sin embargo, en este capítulo construimos solo el módulo de análisis de sintaxis del compilador, sin capacidades de generación de código. Si deseamos probar unitariamente el analizador de sintaxis de forma aislada, tenemos que idear una forma de demostrar que entiende el programa fuente. Nuestra solución es hacer que el analizador de sintaxis genere un archivo XML cuyo contenido marcado refleje la estructura sintáctica del código fuente. Inspeccionando el XML generado

, podremos determinar que el analizador está analizando los programas de entrada correctam

Escribir un compilador desde cero es una hazaña que aporta varios temas fundamentales en informática. Requiere el uso de técnicas de análisis y traducción de idiomas, la aplicación de estructuras de datos clásicas como árboles y tablas hash, y el uso de algoritmos de compilación recursiva. Por todas estas razones, escribir un compilador también es una hazaña desafiante. Sin embargo, al dividir la construcción del compilador en dos proyectos separados (en realidad *cuatro*, contando también los capítulos 7 y 8) y al permitir el desarrollo modular y las pruebas unitarias de cada parte de forma aislada, convertimos el desarrollo del compilador en una actividad manejable y autónoma.

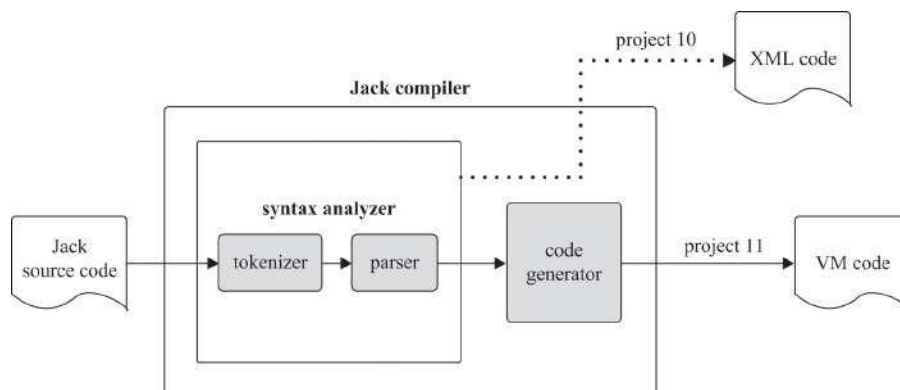
¿Por qué debería tomarse la molestia de crear un compilador? Además de los beneficios de sentirse competente y realizado, una comprensión práctica de los aspectos internos de la compilación lo convertirá en un mejor programador de alto nivel. Además, las mismas reglas y gramáticas utilizadas para describir los lenguajes de programación también se utilizan en diversos campos como gráficos por computadora, comunicaciones y redes, bioinformática, aprendizaje automático, ciencia de datos y tecnología blockchain. Y, el área vibrante del procesamiento del *lenguaje natural*, la ciencia y la práctica detrás de los chatbots inteligentes, los asistentes personales robóticos, los traductores de idiomas y muchas aplicaciones de inteligencia artificial, requiere habilidades para analizar textos y sintetizar semántica. Por lo tanto, aunque la mayoría de los programadores no desarrollan compiladores en sus trabajos habituales, muchos programadores tienen que analizar y manipular textos y conjuntos de datos de estructuras complejas y variables. Estas tareas se pueden realizar de manera eficiente y elegante utilizando los algoritmos y técnicas descritos en este capítulo.

Comenzamos con una sección de Antecedentes que examina el conjunto mínimo de conceptos necesarios para construir un analizador de sintaxis: análisis léxico, gramáticas libres de contexto, árboles de análisis y algoritmos de análisis de descenso recursivo. Esto prepara el escenario para una sección de Especificación que presenta la gramática del lenguaje Jack y la salida que se espera que genere un analizador Jack. La sección Implementación propone una arquitectura de software para construir un analizador Jack, junto con una API sugerida. Como de costumbre, la sección Proyecto proporciona instrucciones paso a paso y programas de prueba para crear un analizador de sintaxis. En el siguiente capítulo, este analizador se extenderá a un compilador a gran escala.

## 10.1 Fondo

La compilación consta de dos etapas principales: análisis de *sintaxis* y *generación de código*. La etapa de análisis de sintaxis generalmente se divide en dos subetapas: *tokenización*, la agrupación de caracteres de entrada en átomos de lenguaje llamados *tokens*, y *análisis*, la agrupación de tokens en declaraciones estructuradas que tienen un significado.

Las tareas de tokenización y análisis son completamente independientes del idioma de destino al que buscamos traducir la entrada de origen. Dado que en este capítulo no tratamos con la generación de código, hemos optado por que el analizador de sintaxis genere la estructura analizada del programa de entrada como un archivo XML. Esta decisión tiene dos beneficios. En primer lugar, el archivo de salida se puede inspeccionar fácilmente, lo que demuestra que el analizador de sintaxis está analizando correctamente los programas de origen. En segundo lugar, el requisito de generar este archivo nos obliga explícitamente a escribir el analizador de sintaxis en una arquitectura que luego se puede transformar en un compilador a gran escala. De hecho, como [muestra la figura 10.1](#), en el próximo capítulo ampliaremos el analizador de sintaxis desarrollado en este capítulo a un motor de compilación a gran escala capaz de generar código VM ejecutable en lugar de código XML pasivo.



**Figura 10.1** Plan de desarrollo por etapas del compilador Jack.

En este capítulo nos centramos solo en el módulo analizador de sintaxis del compilador, cuyo trabajo es *comprender la estructura de un programa*. Esta noción necesita explicación. Cuando los humanos leen el código fuente de un programa de computadora, pueden relacionarse inmediatamente con la estructura del programa. Pueden

hacerlo porque tienen una imagen mental de la gramática del idioma. En particular, detectan qué construcciones de programas son válidas y cuáles no. Usando esta visión gramatical, los humanos pueden identificar dónde comienzan y terminan las clases y los métodos, qué son las declaraciones, qué son las expresiones y cómo se construyen, etc. Para reconocer estas construcciones lingüísticas, que bien pueden estar anidadas, los humanos las mapean recursivamente en el rango de patrones textuales aceptados por la gramática del lenguaje.

Los analizadores de sintaxis se pueden desarrollar para que funcionen de manera similar construyéndolos de acuerdo con una *gramática determinada*, el conjunto de reglas que definen la sintaxis de un lenguaje de programación. Comprender (*analizar*) un programa determinado significa determinar la correspondencia exacta entre el texto del programa y las reglas gramaticales. Para ello, primero debemos transformar el texto del programa en una lista de *tokens*, como ahora vamos a describir.

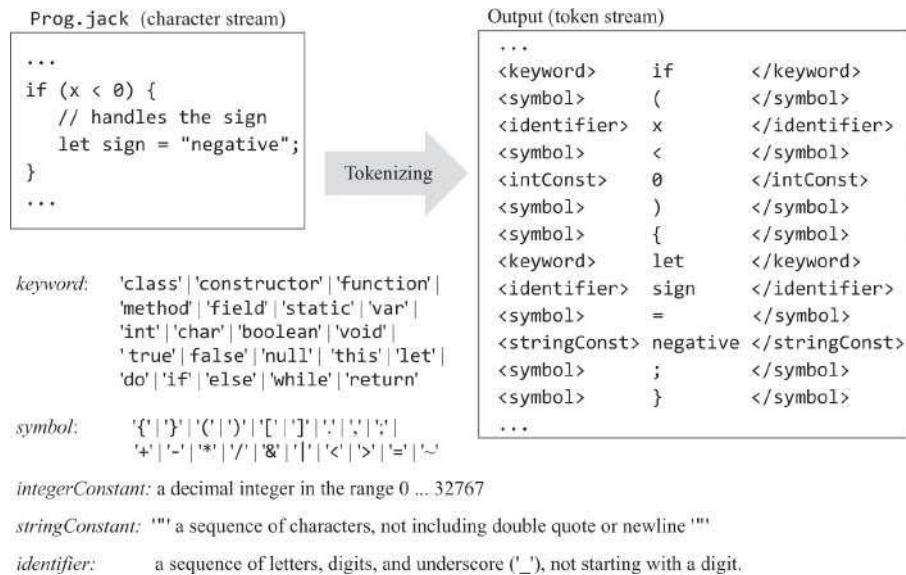
### 10.1.1 Análisis léxico

Cada especificación de lenguaje de programación incluye los tipos de *tokens*, o palabras, que el lenguaje reconoce. En el lenguaje de Jack, las fichas se dividen en cinco categorías: *palabras clave* (como `if`, `while`, `do`, `and` y `<`), *entero* *constantes* (como `314`), *constantes de punto flotante* (como `3.14`), *preguntas frecuentes* (como `"Preguntas Preguntas"`) y *identificadores*, que son las etiquetas textuales utilizadas para variables, clases y subrutinas. En conjunto, los tokens definidos por estas categorías léxicas pueden denominarse *léxico lingüístico*.

En su forma más simple, un programa de computadora es un flujo de caracteres almacenados en un archivo de texto. El primer paso para analizar la sintaxis del programa es agrupar los caracteres en tokens, según lo definido por el léxico del lenguaje, mientras se ignoran los espacios en blanco y los comentarios. Esta tarea se llama *análisis léxico*, *escaneo* o *tokenización*, todos significan exactamente lo mismo.

Una vez que un programa ha sido tokenizado, los tokens, en lugar de los caracteres, se ven como sus átomos básicos. Por lo tanto, el flujo de tokens se convierte en la entrada principal del compilador.

La Figura 10.2 presenta el léxico del lenguaje Jack e ilustra la tokenización de un segmento de código típico. Esta versión del tokenizador genera los tokens, así como sus clasificaciones léxicas.



**Figura 10.2** Definición del léxico de Jack y análisis léxico de una entrada de muestra.

La tokenización es una tarea simple e importante. Dado un léxico de lenguaje, no es difícil escribir un programa que convierta cualquier flujo de caracteres en un flujo de tokens. Esta capacidad proporciona el primer paso hacia el desarrollo de un analizador de sintaxis.

### 10.1.2 Gramáticas

Una vez que desarrollamos la capacidad de acceder a un texto dado como un flujo de tokens o palabras, podemos proceder a intentar agrupar las palabras en oraciones válidas. Por ejemplo, cuando escuchamos que "Bob consiguió el trabajo" asentimos con aprobación, mientras que entradas como "Tengo trabajo el Bob" o "Job Bob el tiene" suenan extrañas. Realizamos estas tareas de análisis sin pensar en ellas, ya que nuestros cerebros han sido entrenados para mapear secuencias de palabras en patrones que son aceptados o rechazados por la gramática inglesa. Las gramáticas de los lenguajes de programación son mucho más simples que las de los lenguajes naturales. Consulte [la figura 10.3](#) para ver un ejemplo.

Jack grammar (subset)

```

statements: statement*

statement: letStatement |
          ifStatement |
          whileStatement

letStatement: 'let' varName '=' expression ';'

ifStatement: 'if' '(' expression ')'
            '{' statements '}'

whileStatement: 'while' '(' expression ')'
              '{' statements '}'

expression: term (op term)?

varName: a string not beginning with a digit

constant: a non-negative integer

op: '+' | '-' | '=' | '>' | '<'

```

Input examples

`let x = 100;` ✓

`let x = x + 1;` ✓

`if (x = 1)
 let x = 100;
 let x = x + 1;
}` ✗

`while (lim < 100) {
 if (x = 1) {
 let z = 100;
 while (z > 0) {
 let z = z - 1;
 }
 }
 let lim = lim + 10;
}` ✓

**Figura 10.3** Un subconjunto de la gramática del lenguaje Jack y segmentos de código Jack que son aceptados o rechazados por la gramática.

Una gramática está escrita en un *metalenguaje*: un lenguaje que describe un idioma. La teoría de la compilación está plagada de formalismos para especificar y razonar sobre gramáticas, lenguajes y metalenguajes. Algunos de estos formalismos son, bueno, dolorosamente formales. Tratando de mantener las cosas simples, en Nand to Tetris vemos una gramática como un conjunto de reglas. Cada regla consta de un lado izquierdo y un lado derecho. El lado izquierdo especifica el nombre de la regla, que no forma parte del idioma. Más bien, está inventado por la persona que describe la gramática y, por lo tanto, no es terriblemente importante. Por ejemplo, si reemplazamos el nombre de una regla con otro nombre a lo largo de la gramática, la gramática será igual de válida (aunque puede ser menos legible).

El lado derecho de la regla describe el patrón lingual que especifica la regla. Este patrón es una secuencia de izquierda a derecha que consta de tres bloques de construcción: *terminales*, *no terminales* y *calificadores*. Los terminales son tokens, los no terminales son nombres de otras reglas y los calificadores están representados por los cinco símbolos |, \*, ?, (, y ). Los elementos terminales, como **if**, se especifican en negrita y entre comillas simples; los elementos no terminales, como *expression*, se especifican usando una fuente cursiva; los calificadores se especifican usando



fuente regular. Por ejemplo, la regla *ifStatement*: `if '(' expresión ')' '{ declaraciones }` estipula que cada instancia válida de una *ifStatement* debe comenzar con el token `if`, seguido del token `(`, seguido de una instancia válida de una *expresión* (definida en otra parte de la gramática), seguida del token `)`, seguida del token `{`, seguido de una instancia válida de *declaraciones* (definidas en otra parte de la gramática), seguidas del token `}`.

Cuando hay más de una forma de analizar un patrón, usamos el calificador `|` para enumerar las alternativas. Por ejemplo, la instrucción de regla: *letStatement* `| ifStatement` `| whileStatement` estipula que una *instrucción* puede ser *letStatement*, *ifStatement* o *whileStatement*.

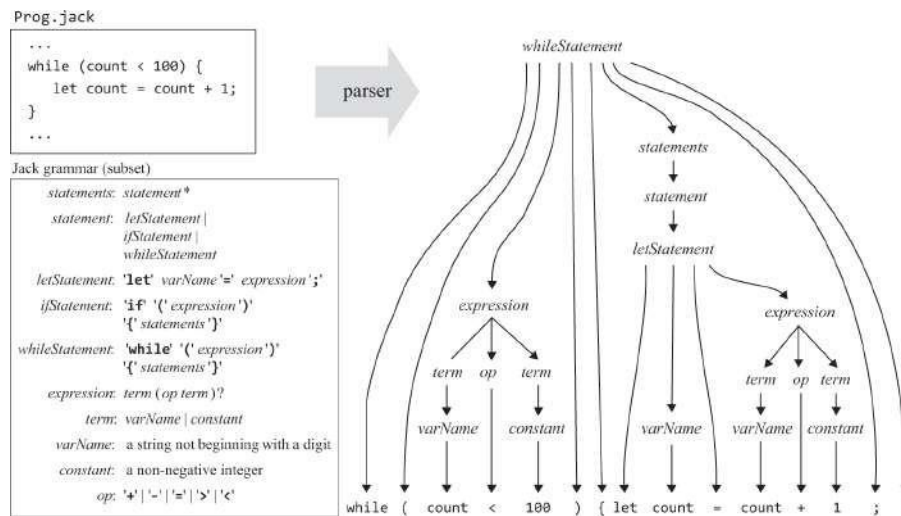
El calificador `*` se usa para denotar "0, 1 o más veces". Por ejemplo, las instrucciones de regla: *declaración*`*` estipulan que *las instrucciones* representan 0, 1 o más instancias de *declaración*. De manera similar, el calificador `?` se usa para denotar "0 o 1 veces". Por ejemplo, la expresión de la regla: *término* `(op term)?` estipula que *la expresión* es un *término* que puede o no ir seguido de la secuencia *op término*. Esto implica que, por ejemplo, `x` es una *expresión*, y también lo son `x+17`, `5*7`, y `x<y`. Los calificadores `(y)` se usan para agrupar elementos gramaticales. Por ejemplo, `(op term)` estipula que, en el contexto de esta regla, *op* seguido de *term* debe tratarse como un elemento gramatical.

### 10.1.3 Análisis sintáctico

Las gramáticas son inherentemente recursivas. Al igual que la oración "Bob consiguió el trabajo que ofreció Alice" se considera válida, también lo es la declaración `si (x<0) { if (y>0) { ... } }`. ¿Cómo podemos saber que esta entrada es aceptada por la gramática? Después de obtener el primer token y darnos cuenta de que tenemos un patrón `if`, nos enfocamos en la regla *ifStatement*: `if '(' expresión ')' '{ declaraciones }`. La regla informa que después del token `si` debería haber el token `(`, seguido de una *expresión*, seguido del token `)`. Y, de hecho, estos requisitos son satisfechos por el elemento de entrada `(x<0)`. Volviendo a la regla, vemos que ahora tenemos que anticipar el token `{`, seguido de *declaraciones*, seguido del token `}`. Ahora, *las declaraciones* se definen como 0 o más instancias de *la declaración*, y *la declaración*, a su vez, es una *letStatement*, una *ifStatement* o una *whileStatement*. Esta expectativa se cumple con el elemento de entrada interno `if(y>0) { ... }`, que es *ifStatement*.



Vemos que la gramática de un lenguaje de programación se puede usar para determinar, sin ambigüedad, si las entradas dadas son aceptadas o rechazadas.<sup>1</sup> Como efecto secundario de este acto de análisis, el analizador produce una correspondencia exacta entre la entrada dada, por un lado, y los patrones sintácticos admitidos por las reglas gramaticales, por el otro. La correspondencia se puede representar mediante una estructura de datos llamada *árbol de análisis*, también llamado *árbol de derivación*, como el que se muestra en la [figura 10.4a](#). Si se puede construir un árbol de este tipo, el analizador hace que la entrada sea válida; de lo contrario, puede informar de que la entrada contiene errores de sintaxis.



**Figura 10.4a** Árbol de análisis de un segmento de código típico. El proceso de análisis está impulsado por las reglas gramaticales.

¿Cómo podemos representar los árboles de análisis textualmente? En Nand to Tetris, decidimos que el analizador generara un archivo XML cuyo formato marcado refleje la estructura del árbol. Al inspeccionar este archivo de salida XML, podemos determinar que el analizador está analizando la entrada correctamente. Consulte [la figura 10.4b](#) para ver un ejemplo.

Prog.xml

```
...
<whileStatement>
  <keyword> while </keyword>
  <symbol> ( </symbol>
  <expression>
    <term> <varName> count </varName> </term>
    <op> <symbol> < </symbol> </op>
    <term> <constant> 100 </constant> </term>
  </expression>
  <symbol> ) </symbol>
  <symbol> { </symbol>
  <statements>
    <statement> <letStatement>
      <keyword> let </keyword>
      <varName> count </varName>
      <symbol> = </symbol>
      <expression>
        <term> <varName> count </varName> </term>
        <op> <symbol> + </symbol> </op>
        <term> <constant> 1 </constant> </term>
      </expression>
      <symbol> ; </symbol>
    </letStatement> </statement>
  </statements>
  <symbol> } </symbol>
</whileStatement>
...
```

**Figura 10.4b** Mismo árbol de análisis, en XML.

### 10.1.4 Analizador

Un analizador es un agente que opera de acuerdo con una gramática dada. El analizador acepta como entrada un flujo de tokens e intenta producir como salida el árbol de análisis asociado con la entrada dada. En nuestro caso, se espera que la entrada esté estructurada de acuerdo con la gramática de Jack y la salida esté escrita en XML. Sin embargo, tenga en cuenta que las técnicas de análisis que ahora describimos son aplicables al manejo de cualquier lenguaje de programación y formato de archivo estructurado.

Hay varios algoritmos para construir árboles de análisis. El enfoque descendente, también conocido como *análisis de descenso recursivo*, intenta analizar la entrada tokenizada de forma recursiva, utilizando las estructuras anidadas admitidas por la gramática del lenguaje. Dicho algoritmo se puede implementar de la siguiente manera. Para cada regla no trivial de la gramática, equipamos el programa analizador con una rutina diseñada para analizar la entrada de acuerdo con esa regla. Por ejemplo, la gramática enumerada en la [figura 10.3](#) se puede implementar usando un conjunto de rutinas denominadas `compileStatement`, `compileStatements`, `compileLet`, `compileIf`, ..., `compileExpression`, etc. Usamos el verbo de acción *compilar* en lugar de *analizar*, ya que en el próximo capítulo extenderemos esta lógica a un motor de compilación a gran escala.

La lógica de análisis de cada rutina `compilexxx` debe seguir el patrón sintáctico especificado por el lado derecho de la regla `xxx`. Por ejemplo, centrémonos en la regla *whileStatement*: `'while' '(' expression ')' '{ statements }'`. De acuerdo con nuestro esquema, esta regla se implementará mediante una rutina de análisis llamada `compileWhile`. Esta rutina debe realizar la lógica de derivación de izquierda a derecha especificada por el patrón `'while' '(' expresión ')' '{ declaraciones }'`. Aquí hay una forma de implementar esta lógica, usando pseudocódigo:

```
// This routine implements the rule whileStatement:
// 'while' '(' expression ')' '{ statements }'
// Should be called if the current token is 'while'.
compileWhile():
    print("<whileStatement>")
    process("while")
    process("(")
    compileExpression()
    process(")")
    process("{")
    compileStatements()
    process("}")
    print("</whileStatement>")
```

```
// A helper routine that handles
// the current token, and advances
// to get the next token.
process(str):
    if (currentToken == str)
        printXMLToken(str)
    else
        print("syntax error")
    // Gets the next token
    currentToken =
        tokenizer.advance()
```

Este proceso de análisis continuará hasta que las partes de expresión y declaraciones de la declaración *while* se hayan analizado por completo. Por supuesto, la parte de *las declaraciones* puede contener una declaración *while* de nivel inferior, en cuyo caso el análisis continuará recursivamente.

El ejemplo que acabamos de mostrar ilustra la implementación de una regla relativamente simple, cuya lógica de derivación implica un caso simple de análisis en línea recta. En general, las reglas gramaticales pueden ser más complejas. Por ejemplo, considere la siguiente regla, que especifica la definición de variables de campo estáticas y de nivel de instancia a nivel de clase en el lenguaje Jack:

```
classVarDec: ('static'|'field') type varName (',' varName)* ';'
            (where type and varName are defined elsewhere in the full Jack grammar)
```

```
Examples:  static int count;
           static char a, b, c;
           field boolean sign;
           field int up, down, left, right;
```

Esta regla presenta dos desafíos de análisis que van más allá del análisis en línea recta. Primero, la regla admite `static` o `field` como su primer token. En segundo lugar, la regla admite múltiples declaraciones de variables. Para solucionar ambos problemas, la implementación de la rutina `compileClassVarDec` correspondiente puede (i) controlar el procesamiento del primer token (estático o de campo) directamente, sin llamar a una rutina auxiliar, y (ii) usar un bucle para controlar todas las declaraciones de variables que contiene la entrada. En términos generales, diferentes reglas gramaticales implican implementaciones de análisis ligeramente diferentes. Al mismo tiempo, todos siguen el mismo contrato: cada rutina `compilexxx` debe obtener de la entrada y manejar todos los tokens que componen `xxx`, avanzar el tokenizador exactamente más allá de estos tokens y generar el árbol de análisis de `xxx`.

Los algoritmos de análisis recursivo son simples y elegantes. Si el lenguaje es simple, una sola búsqueda anticipada de tokens es todo lo que se necesita para saber qué regla de análisis invocar a continuación. Por ejemplo, si el token actual es `let`, sabemos que tenemos un *letStatement*; si el token actual es `while`, sabemos que tenemos un *whileStatement*, y así sucesivamente. De hecho, en la gramática simple que se muestra en la [figura 10.3](#), mirar hacia adelante una ficha es suficiente para resolver, sin ambigüedad, qué regla usar a continuación. Las gramáticas que tienen esta propiedad lingual se llaman *LL* (1). Estas gramáticas se pueden manejar de manera simple y elegante mediante algoritmos de descenso recursivo, sin retroceder.

El término *LL* proviene de la observación de que la gramática analiza la entrada de *izquierda* a derecha, realizando la derivación *más a la izquierda*

de la entrada. El (1)

informa que mirar hacia adelante 1 token es todo lo que se necesita para saber qué regla de análisis invocar a continuación. Si ese token no es suficiente para resolver qué regla usar, podemos mirar hacia adelante un token más. Si esta anticipación resuelve la ambigüedad, se dice que el analizador es *LL* (2). Y si no, podemos mirar hacia adelante otra ficha, y así sucesivamente. Claramente, a medida que necesitamos mirar hacia adelante cada vez más abajo en el flujo de tokens, las cosas se complican y requieren un analizador sofisticado.

La gramática completa del lenguaje Jack, que ahora pasamos a presentar, es *LL* (1), salvo una excepción que se puede manejar fácilmente. Por lo tanto, Jack se presta muy bien a un analizador de descenso recursivo, que es la pieza central del proyecto 10.

---

## 10.2 Especificación

Esta sección consta de dos partes. Primero, especificamos la gramática del idioma Jack. A continuación, especificamos un analizador de sintaxis diseñado para analizar programas de acuerdo con esta gramática.

### 10.2.1 La gramática del lenguaje Jack

La especificación funcional del lenguaje Jack presentada en el capítulo 9 estaba dirigida a los programadores de Jack; ahora damos una especificación formal del lenguaje Jack, dirigida a los desarrolladores de compiladores de Jack. La especificación del lenguaje, o *gramática*, utiliza la siguiente notación:

|            |   |                                                       |
|------------|---|-------------------------------------------------------|
| 'xxx'      | : | Represents language tokens that appear verbatim       |
| xxx        | : | Represents names of terminal and nonterminal elements |
| ()         | : | Used for grouping                                     |
| $x \mid y$ | : | Either $x$ or $y$                                     |
| $x \ y$    | : | $x$ is followed by $y$                                |
| $x?$       | : | $x$ appears 0 or 1 times                              |
| $x^*$      | : | $x$ appears 0 or more times                           |

Con esta notación en mente, la gramática completa de Jack se especifica en la [figura 10.5](#).

|                           |                                                                                                                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Lexical elements:</b>  |                                                                                                                                                                                                             |
| <i>keyword:</i>           | 'class'   'constructor'   'function'   'method'   'field'   'static'   'var'   'int'   'char'   'boolean'   'void'   'true'   'false'   'null'   'this'   'let'   'do'   'if'   'else'   'while'   'return' |
| <i>symbol:</i>            | {   }   (   )   [   ]   .   '   "   ;   :   +   -   *   /   &   '   <   >   =   ~                                                                                                                           |
| <i>integerConstant:</i>   |                                                                                                                                                                                                             |
| <i>StringConstant:</i>    | "" A sequence of characters not including double quote or newline ""                                                                                                                                        |
| <i>identifier:</i>        | A sequence of letters, digits, and underscore ( '_ ' ), not starting with a digit                                                                                                                           |
| <b>Program structure:</b> |                                                                                                                                                                                                             |
| <i>class:</i>             | 'class' className '{ classVarDec* subroutineDec* }'                                                                                                                                                         |
| <i>classVarDec:</i>       | ('static'   'field') type varName (',' varName)* ';'                                                                                                                                                        |
| <i>type:</i>              | 'int'   'char'   'boolean'   className                                                                                                                                                                      |
| <i>subroutineDec:</i>     | ('constructor'   'function'   'method') (void   type) subroutineName<br>'(' parameterList ')' subroutineBody                                                                                                |
| <i>parameterList:</i>     | ( (type varName) (',' type varName)* )?                                                                                                                                                                     |
| <i>subroutineBody:</i>    | { varDec* statements }                                                                                                                                                                                      |
| <i>varDec:</i>            | 'var' type varName (',' varName)* ';'                                                                                                                                                                       |
| <i>className:</i>         | identifier                                                                                                                                                                                                  |
| <i>subroutineName:</i>    | identifier                                                                                                                                                                                                  |
| <i>varName:</i>           | identifier                                                                                                                                                                                                  |
| <b>Statements:</b>        |                                                                                                                                                                                                             |
| <i>statements:</i>        | statement*                                                                                                                                                                                                  |
| <i>statement:</i>         | letStatement   ifStatement   whileStatement   doStatement   returnStatement                                                                                                                                 |
| <i>letStatement:</i>      | 'let' varName '(' [ expression ] ')? '=' expression ';'                                                                                                                                                     |
| <i>ifStatement:</i>       | 'if' '(' expression ')' '{ statements }' ('else' '{ statements }')?                                                                                                                                         |
| <i>whileStatement:</i>    | 'while' '(' expression ')' '{ statements }'                                                                                                                                                                 |
| <i>doStatement:</i>       | 'do' subroutineCall ';'                                                                                                                                                                                     |
| <i>returnStatement:</i>   | 'return' expression? ';'                                                                                                                                                                                    |
| <b>Expressions:</b>       |                                                                                                                                                                                                             |
| <i>expression:</i>        | term (op term)*                                                                                                                                                                                             |
| <i>term:</i>              | integerConstant   stringConstant   keywordConstant   varName  <br>varName '[' expression ']'   '(' expression ')'   (unaryOp term)   subroutineCall                                                         |
| <i>subroutineCall:</i>    | subroutineName '(' expressionList ')'   (className   varName) '.' subroutineName<br>'(' expressionList ')'                                                                                                  |
| <i>expressionList:</i>    | (expression (',' expression)*)?                                                                                                                                                                             |
| <i>op:</i>                | +'   -'   '*'   '/'   '&'   ' '   '<'   '>'   '='                                                                                                                                                           |
| <i>unaryOp:</i>           | '-'   '~'                                                                                                                                                                                                   |
| <i>keywordConstant:</i>   | 'true'   'false'   'null'   'this'                                                                                                                                                                          |

**Figura 10.5** La gramática de Jack.

## 10.2.2 Un analizador de sintaxis para el lenguaje Jack

Un analizador de sintaxis es un programa que realiza tanto la tokenización como el análisis. En Nand to Tetris, el propósito principal del analizador de sintaxis es procesar un programa Jack y comprender su estructura sintáctica de acuerdo con la gramática Jack. Por *comprensión* queremos decir que el analizador de sintaxis debe conocer, en cada punto del proceso de análisis, la identidad estructural del elemento del programa que está manejando actualmente, es decir, si es una expresión, una declaración, un nombre de variable, etc. El analizador de sintaxis debe poseer este conocimiento sintáctico en un sentido completamente recursivo. Sin él, será



imposible pasar a la generación de código, el objetivo final del proceso de compilación.

**Uso:** el analizador de sintaxis acepta un único argumento de línea de comandos, como se indica a continuación,

prompt> Fuente de JackAnalyzer

donde *source* es un nombre de archivo de la forma *Xxx jack* (la extensión es obligatoria) o el nombre de una carpeta (en cuyo caso no hay extensión) que contiene uno o más archivos *jack*. El nombre del archivo o carpeta puede contener una ruta de acceso de archivo. Si no se especifica ninguna ruta de acceso, el analizador funciona en la carpeta actual. Para *.xml* y escribe el cada *Xxx .sota*, el analizador crea un archivo de salida *Xxx* analizó la salida en él. El archivo de salida se crea en la misma carpeta que la de la entrada. Si hay un archivo con este nombre en la carpeta, se sobrescribirá.

**Entrada:** Un archivo *Xxx jack* es un flujo de caracteres. Si el archivo representa un programa válido, se puede tokenizar en un flujo de tokens válidos, según lo especificado por el léxico de Jack. Los tokens pueden estar separados por un número arbitrario de caracteres de espacio, caracteres de nueva línea y comentarios, que se ignoran. Hay tres formatos de comentarios posibles: */\** comentario hasta el cierre *\*/*, */\*\** comentario de la API hasta el cierre *\*/*, y comentario hasta el final de la línea.

**Salida:** el analizador de sintaxis emite una descripción XML del archivo de entrada, como se indica a continuación. Para cada elemento terminal (*token*) de tipo *xxx* que aparece en la entrada, el analizador de sintaxis imprime la salida marcada *<xxx> token </xxx>*, donde *xxx* es una de las etiquetas *keyword*, *symbol*, *integerConstant*, *stringConstant* o *identifier*, que representa uno de los cinco tipos de fichas reconocidas por el lenguaje Jack. Cada vez que se detecta un elemento de lenguaje que no es terminal *xxx*, el analizador de sintaxis lo controla mediante el siguiente pseudocódigo:

```
print("<xxx>")
    Recursive code for handling the body of the xxx element
print("</xxx>")
```



donde *xxx* es una de las siguientes (y solo las siguientes) etiquetas: `class`, `classVarDec`, `subroutineDec`, `parameterList`, `subroutineBody`, `varDec`, `statements`, `letStatement`, `ifStatement`, `whileStatement`, `doStatement`, `returnStatement`, `expression`, `term`, `expressionList`.

Para simplificar las cosas, las siguientes reglas gramaticales de Jack no se tienen en cuenta explícitamente en la salida XML: *type*, *className*, *subroutineName*, *varName*, *statement*, *subroutineCall*. Explicaremos esto con más detalle en la siguiente sección, cuando discutamos la arquitectura de nuestro motor de compilación.

---

## 10.3 Implementación

En la sección anterior se especificaba *lo que* debía hacer un analizador de sintaxis, con poca información sobre la implementación. En esta sección se describe *cómo* compilar un analizador de este tipo. Nuestra implementación propuesta se basa en tres módulos:

- JackAnalyzer: programa principal que configura e invoca los otros módulos
- JackTokenizer: tokenizador
- CompilationEngine: analizador recursivo de arriba hacia abajo

En el próximo capítulo ampliaremos esta arquitectura de software con dos módulos adicionales que manejan la semántica del lenguaje: una tabla de *símbolos* y un *escritor de código VM*. Esto completará la construcción de un compilador a gran escala para el lenguaje Jack. Dado que el módulo principal que impulsa el proceso de análisis en este proyecto terminará impulsando también el proceso de compilación general, lo llamamos `CompilationEngine`.

### El JackTokenizer

Este módulo ignora todos los comentarios y espacios en blanco en el flujo de entrada y permite acceder a la entrada de un token a la

Además, analiza y proporciona el *tipo* de cada token, según lo definido por la gramática de Jack.

| <i>Routine</i>            | <i>Arguments</i>    | <i>Returns</i>                                                                                                                                | <i>Function</i>                                                                                                                                                                    |
|---------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Constructor / initializer | Input file / stream | –                                                                                                                                             | Opens the input .jack file / stream and gets ready to tokenize it.                                                                                                                 |
| hasMoreTokens             | –                   | boolean                                                                                                                                       | Are there more tokens in the input?                                                                                                                                                |
| advance                   | –                   | –                                                                                                                                             | Gets the next token from the input, and makes it the current token.<br><br>This method should be called only if hasMoreTokens is true.<br><br>Initially there is no current token. |
| tokenType                 | –                   | KEYWORD, SYMBOL, IDENTIFIER, INT_CONST, STRING_CONST                                                                                          | Returns the type of the current token, as a constant.                                                                                                                              |
| keyword                   | –                   | CLASS, METHOD, FUNCTION, CONSTRUCTOR, INT, BOOLEAN, CHAR, VOID, VAR, STATIC, FIELD, LET, DO, IF, ELSE, WHILE, RETURN, TRUE, FALSE, NULL, THIS | Returns the keyword which is the current token, as a constant.<br><br>This method should be called only if tokenType is KEYWORD.                                                   |
| symbol                    | –                   | char                                                                                                                                          | Returns the character which is the current token. Should be called only if tokenType is SYMBOL.                                                                                    |
| identifier                | –                   | string                                                                                                                                        | Returns the string which is the current token. Should be called only if tokenType is IDENTIFIER.                                                                                   |
| intVal                    | –                   | int                                                                                                                                           | Returns the integer value of the current token. Should be called only if tokenType is INT_CONST.                                                                                   |
| stringVal                 | –                   | string                                                                                                                                        | Returns the string value of the current token, without the opening and closing double quotes. Should be called only if tokenType is STRING_CONST.                                  |

## El motor de compilación

CompilationEngine es el módulo troncal del analizador de sintaxis descrito en este capítulo y del compilador a escala completa descrito en el capítulo siguiente. En el analizador de sintaxis, el motor de compilación emite un

representación del código fuente de entrada envuelto en etiquetas XML. En el compilador, el motor de compilación emitirá en su lugar código de máquina virtual ejecutable. En ambas versiones, la lógica de análisis y la API que se presentan a continuación son exactamente las mismas.

El motor de compilación obtiene su entrada de un `JackTokenizer` y emite su salida a un archivo de salida. La salida es generada por una serie de rutinas `compilexxx`, cada una diseñada para manejar la compilación de una construcción específica del lenguaje Jack `xxx`. El contrato entre estas rutinas es que cada rutina `compilexxx` debe obtener de la entrada y manejar todos los tokens que componen `xxx`, avanzar el tokenizador exactamente más allá de estos tokens y generar el análisis de `xxx`. Como regla general, cada rutina `compilexxx` se llama solo si el token actual es `xxx`.

**Reglas gramaticales que no tienen rutinas `compilexxx` correspondientes:** *type*, *className*, *subroutineName*, *varName*, *statement*, *subroutineCall*. Introdujimos estas reglas para hacer que la gramática de Jack sea más estructurada. Resulta que la lógica de análisis de estas reglas se maneja mejor con las rutinas que implementan las reglas que se refieren a ellas. Por ejemplo, en lugar de escribir una rutina `compileType`, siempre que se mencione el tipo en alguna regla `xxx`, el análisis de los tipos posibles debe realizarse directamente mediante la rutina `compile xxx`.

**Búsqueda anticipada de tokens:** Jack es casi un lenguaje *LL(1)*: el token actual es suficiente para determinar a qué rutina `CompilationEngine` llamar a continuación. La única excepción ocurre cuando se analiza un *término*, lo que ocurre solo cuando se analiza una *expresión*. Para ilustrar, considere la expresión `y+arr[5]-p.get(row)*count()-Math.sqrt(dist)/2`. Esta expresión se compone de seis términos: la variable `y`, el elemento de matriz `arr[5]`, la llamada al método en el objeto `p` `p.get(fila)`, la llamada al método en el recuento de este objeto `()`, la llamada a la función (método estático) `Math.sqrt(dist)` y la constante `2`.

Supongamos que estamos analizando esta expresión y el token actual es uno de los identificadores `y`, `arr`, `p`, `count` o `Math`. En cada uno de estos casos, sabemos que tenemos un *término* que comienza con un *identificador*, pero no sabemos qué posibilidad de análisis seguir a continuación. Esa es la mala noticia; La buena noticia es que un solo anticipo del próximo token es todo lo que necesitamos para resolver el dilema.

La necesidad de esta operación de búsqueda anticipada irregular se produce en CompilationEngine dos veces: al analizar un *término*, lo que ocurre solo al analizar una *expresión*, y al analizar una *subroutineCall*. Ahora, una inspección de la gramática de Jack muestra que *subroutineCall* aparece solo en dos lugares: ya sea en una declaración de *subroutineCall* o en un *término*.

Con eso en mente, proponemos analizar las declaraciones de *subroutineCall* como si su sintaxis fuera una expresión *do*. Esta recomendación pragmática obvia la necesidad de escribir el código de búsqueda anticipada irregular dos veces. También implica que el análisis de *subroutineCall* ahora puede ser manejado directamente por la rutina `compileTerm`. En resumen, hemos localizado la necesidad de escribir el código de búsqueda anticipada de tokens irregulares en una sola rutina, `compileTerm`, y hemos eliminado la necesidad de una rutina `compileSubroutineCall`.

| <i>Routine</i>               | <i>Arguments</i>                                      | <i>Returns</i> | <i>Function</i>                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------|-------------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Constructor / initializer    | Input<br>file / stream<br><br>Output<br>file / stream | —              | Creates a new compilation engine with the given input and output.<br><br>The next routine called (by the <b>JackAnalyzer</b> module) must be <b>compileClass</b>                                                                                                                                                                                                     |
| <b>compileClass</b>          | —                                                     | —              | Compiles a complete class.                                                                                                                                                                                                                                                                                                                                           |
| <b>compileClassVarDec</b>    | —                                                     | —              | Compiles a static variable declaration, or a field declaration.                                                                                                                                                                                                                                                                                                      |
| <b>compileSubroutine</b>     | —                                                     | —              | Compiles a complete method, function, or constructor.                                                                                                                                                                                                                                                                                                                |
| <b>compileParameterList</b>  | —                                                     | —              | Compiles a (possibly empty) parameter list. Does not handle the enclosing parentheses tokens ( and ).                                                                                                                                                                                                                                                                |
| <b>compileSubroutineBody</b> | —                                                     | —              | Compiles a subroutine's body.                                                                                                                                                                                                                                                                                                                                        |
| <b>compileVarDec</b>         | —                                                     | —              | Compiles a <b>var</b> declaration.                                                                                                                                                                                                                                                                                                                                   |
| <b>compileStatements</b>     | —                                                     | —              | Compiles a sequence of statements. Does not handle the enclosing curly bracket tokens { and }.                                                                                                                                                                                                                                                                       |
| <b>compileLet</b>            | —                                                     | —              | Compiles a <b>let</b> statement.                                                                                                                                                                                                                                                                                                                                     |
| <b>compileIf</b>             | —                                                     | —              | Compiles an <b>if</b> statement, possibly with a trailing <b>else</b> clause.                                                                                                                                                                                                                                                                                        |
| <b>compileWhile</b>          | —                                                     | —              | Compiles a <b>while</b> statement.                                                                                                                                                                                                                                                                                                                                   |
| <b>compileDo</b>             | —                                                     | —              | Compiles a <b>do</b> statement.                                                                                                                                                                                                                                                                                                                                      |
| <b>compileReturn</b>         | —                                                     | —              | Compiles a <b>return</b> statement.                                                                                                                                                                                                                                                                                                                                  |
| <b>compileExpression</b>     | —                                                     | —              | Compiles an expression.                                                                                                                                                                                                                                                                                                                                              |
| <b>compileTerm</b>           | —                                                     | —              | Compiles a <i>term</i> . If the current token is an <i>identifier</i> , the routine must resolve it into a <i>variable</i> , an <i>array element</i> , or a <i>subroutine call</i> . A single lookahead token, which may be [, (, or ., suffices to distinguish between the possibilities. Any other token is not part of this term and should not be advanced over. |
| <b>compileExpressionList</b> | —                                                     | int            | Compiles a (possibly empty) comma-separated list of expressions. Returns the number of expressions in the list.                                                                                                                                                                                                                                                      |

**La rutina `compileExpressionList`:** devuelve el número de expresiones de la lista. El valor devuelto es necesario para generar código de VM, como veremos cuando completemos el desarrollo del compilador en el proyecto 11. En este proyecto no generamos código VM; por lo tanto, el valor devuelto no se usa y las rutinas que llaman a `compileExpressionList` pueden omitirlo.

## El JackAnalyzer

Este es el programa principal que impulsa el proceso general de análisis de sintaxis, utilizando los servicios de un `JackTokenizer` y un `CompilationEngine`. Para cada archivo *Xxx.jack* de origen, el analizador

1. crea un `JackTokenizer` a partir del *archivo de entrada* *Xxx.jack*;
2. crea un archivo de salida llamado *Xxx.xml*; y
3. usa `JackTokenizer` y `CompilationEngine` para analizar el archivo de entrada y escribir el código analizado en el archivo de salida.

No proporcionamos ninguna API para este módulo, invitándolo a implementarlo como mejor le parezca. Recuerde que la primera rutina a la que se debe llamar al compilar un `.jack` es `compileClass`.

---

## 10.4 Proyecto

**Objetivo:** Construir un analizador de sintaxis que analice los programas Jack de acuerdo con la gramática Jack. La salida del analizador debe escribirse en XML, como se especifica en la sección 10.2.2.

Esta versión del analizador de sintaxis asume que el código fuente de Jack está libre de errores. La comprobación de errores, la generación de informes y el control se pueden agregar a versiones posteriores del analizador, pero no forman parte del proyecto 10.

**Recursos:** La herramienta principal de este proyecto es el lenguaje de programación que usará para implementar el analizador de sintaxis. También necesitará la utilidad `TextComparer` suministrada. Este programa permite comparar archivos ignorando el espacio en blanco. Esto le ayudará a comparar los archivos de salida generados

por su analizador con los archivos de comparación suministrados. También es posible que desee inspeccionar estos archivos mediante un visor XML. Cualquier navegador web estándar debería hacer el trabajo: simplemente use la opción "abrir archivo" de su navegador para abrir el archivo XML que desea inspeccionar.

**Contrato:** Escriba un analizador de sintaxis para el lenguaje Jack y pruébelo en los archivos de prueba suministrados. Los archivos XML generados por el analizador deben ser idénticos a los archivos de comparación proporcionados, ignorando los espacios en blanco.

**Archivos de prueba:** Proporcionamos varios archivos .jack para fines de prueba. El programa `projects/10/Square` es una aplicación de tres clases que permite mover un cuadrado negro por la pantalla usando las teclas de flecha del teclado. El programa `projects/10/ArrayTest` es una aplicación de una sola clase que calcula el promedio de una secuencia de enteros proporcionada por el usuario mediante el procesamiento de matrices. Ambos programas se discutieron en el capítulo 9, por lo que deberían ser familiares. Sin embargo, tenga en cuenta que hicimos algunos cambios inofensivos en el código original para asegurarnos de que el analizador de sintaxis se pruebe completamente en todos los aspectos del lenguaje Jack. Por ejemplo, hemos agregado una variable estática a `projects/10/Square/Main.jack`, así como una función llamada `more`, que nunca se usan ni se llaman. Estos cambios permiten probar cómo el analizador maneja los elementos del lenguaje que no aparecen en los archivos `Square` y `ArrayTest` originales, como variables estáticas, `else` y operadores unarios.

**Plan de desarrollo:** Sugerimos desarrollar y probar unitariamente el analizador en cuatro etapas:

- Primero, escriba y pruebe un tokenizador de Jack.
- A continuación, escriba y pruebe un motor de compilación básico que maneje todas las características del lenguaje Jack, excepto las expresiones y las declaraciones orientadas a matrices.
- A continuación, amplíe el motor de compilación para controlar las expresiones.
- Por último, amplíe el motor de compilación para controlar las instrucciones orientadas a matrices.

Proporcionamos archivos .jack de entrada y comparamos .xml archivos para realizar pruebas unitarias en cada una de las cuatro etapas, como ahora

describimos.



### 10.4.1 Tokenizador

Implemente el módulo `JackTokenizer` especificado en la sección 10.3. Pruebe su implementación escribiendo una versión básica de `JackAnalyzer`, definida de la siguiente manera. El analizador, que es el programa principal, se invoca usando el comando `JackAnalyzer source`, donde *source* es un nombre de archivo de la forma `Xxx.jack` (la extensión es obligatoria) o un nombre de carpeta (en cuyo caso no hay extensión). En este último caso, la carpeta contiene uno o más archivos `.jack` y, posiblemente, también otros archivos. El nombre del archivo o carpeta puede incluir una ruta de acceso de archivo. Si no se especifica ninguna ruta de acceso, el analizador funciona en la carpeta actual.

El analizador maneja cada archivo por separado. En concreto, para cada *archivo* `Xxx.jack`, el analizador construye un `JackTokenizer` para controlar la entrada y un archivo de salida para escribir la salida. En esta primera versión del analizador, el archivo de salida se denomina `XxxT.xml` (donde *T* significa *salida tokenizada*). A continuación, el analizador entra en un bucle para avanzar y controlar todos los tokens del archivo de entrada, un token a la vez, mediante los métodos `JackTokenizer`. Cada token debe imprimirse en una línea independiente, como `<tokenType> token </tokenType>`, donde *tokenType* es una de las cinco etiquetas XML posibles que codifican el tipo del token. Aquí hay un ejemplo:

*Input* (e.g. `Prog.jack`)

```
...
// Comments and white space
// are ignored.
if (x < 0) {
    let quit = "yes";
}
...
```

*JackAnalyzer Output* (e.g. `ProgT.xml`)

```
<tokens>
...
<keyword> if </keyword>
<symbol> ( </symbol>
<identifier> x </identifier>
<symbol> &lt; </symbol>
<integerConstant> 0 </integerConstant>
<symbol> ) </symbol>
<symbol> { </symbol>
<keyword> let </keyword>
<identifier> quit </identifier>
<symbol> = </symbol>
<stringConstant> yes </stringConstant>
<symbol> ; </symbol>
<symbol> } </symbol>
...
</tokens>
```

Tenga en cuenta que en el caso de *las constantes de cadena*, el programa ignora las comillas dobles. Este requisito es por diseño.

La salida generada tiene dos tecnicismos triviales dictados por las convenciones XML. Primero, un archivo XML debe estar encerrado dentro de algunas etiquetas de inicio y final; Esta convención se satisface con las etiquetas `<tokens>` y `</tokens>`. En segundo lugar, cuatro de los símbolos utilizados en el lenguaje Jack (`<`, `>`, `"`, `&`) también se utilizan para el marcado XML; por lo tanto, no pueden aparecer como datos en archivos XML. Siguiendo la convención, el analizador representa estos símbolos como `<`, `>`, `"`, y `&` respectivamente. Por ejemplo, cuando el analizador encuentra el `< símbolo` en el archivo de entrada, genera la línea `< símbolo> < </símbolo>`. Esta llamada secuencia de *escape* es representada por los visores XML como `< símbolo> < </símbolo>`, que es lo que queremos.

## Pautas de prueba

- Comience aplicando su JackAnalyzer a uno de los archivos .jack suministrados y verifique que funcione correctamente en un solo archivo de entrada.
- A continuación, aplique su JackAnalyzer a la carpeta Square, que contiene los archivos Main.jack, Square.jack y SquareGame.jack, y a la carpeta TestArray, que contiene el archivo Main.jack.
- Utilice la utilidad TextComparer suministrada para comparar los archivos de salida generados por su JackAnalyzer con los archivos de comparación de .xml proporcionados. Por ejemplo, compare el archivo generado SquareT.xml con el archivo de comparación proporcionado SquareT.xml.
- Dado que los archivos generados y comparados tienen los mismos nombres, sugerimos colocarlos en carpetas separadas.

## 10.4.2 Motor de compilación

La próxima versión de su analizador de sintaxis debería ser capaz de analizar todos los elementos del lenguaje Jack, excepto las expresiones y los comandos orientados a matrices. Con ese fin, implemente el módulo CompilationEngine especificado en la sección 10.3, excepto para las rutinas que manejan expresiones y matrices. Pruebe la implementación utilizando su analizador Jack, como se indica a continuación.

Para cada archivo *Xxx.jack*, el analizador construye un JackTokenizer para controlar la entrada y un archivo de salida para escribir la salida, denominado *Xxx.xml*. El

A continuación, el analizador llama a la rutina `compileClass` de `CompilationEngine`. A partir de este momento, las rutinas `CompilationEngine` deben llamarse entre sí de forma recursiva, emitiendo una salida XML similar a la que se muestra en la [figura 10.4b](#).

Pruebe unitariamente esta versión de su `JackAnalyzer` aplicándola a la carpeta

`ExpressionlessSquare`. Esta carpeta contiene versiones de los archivos `Square.jack`, `SquareGame.jack` y `Main.jack`, en las que cada expresión del código original se ha reemplazado por un único identificador (un nombre de variable en el ámbito). Por ejemplo:

Square folder:

```
// Square.jack
...
method void incSize() {
    if (((y + size) < 254) & ((x + size) < 510)
        do erase();
        let size = size + 2;
        do draw();
    }
    return;
}
...
```

ExpressionlessSquare folder:

```
// Square.jack
...
method void incSize() {
    if (x) {
        do erase();
        let size = size;
        do draw();
    }
    return;
}
...
```

Tenga en cuenta que el reemplazo de expresiones con variables da como resultado un código sin sentido. Esto está bien, ya que la semántica del programa es irrelevante para el proyecto 10. El código sin sentido es sintácticamente correcto, y eso es todo lo que importa para probar el analizador. Tenga en cuenta también que los archivos originales y sin expresión tienen los mismos nombres pero se encuentran en carpetas separadas.

Utilice la utilidad `TextComparer` suministrada para comparar los archivos de salida generados por su `JackAnalyzer` con los archivos de comparación de `.xml` proporcionados.

A continuación, complete las rutinas `CompilationEngine` que manejan expresiones y pruébelas aplicando su `JackAnalyzer` a la carpeta `Square`. Por último, complete las rutinas que manejan matrices y pruébelas aplicando su `JackAnalyzer` a la carpeta `ArrayTest`.

**Una versión basada en la web del proyecto 10** está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

---

## 10.5 Perspectiva

Aunque es conveniente describir la estructura de los programas informáticos mediante árboles de análisis y archivos XML, es importante comprender que los compiladores no necesariamente tienen que mantener dichas estructuras de datos explícitamente. Por ejemplo, el algoritmo de análisis descrito en este capítulo analiza la entrada a medida que la lee y no mantiene todo el programa de entrada en la memoria. Básicamente, hay dos tipos de estrategias para realizar dicho análisis. La estrategia más simple funciona de arriba hacia abajo, y esa es la que se presenta en este capítulo. Los algoritmos de análisis más avanzados, que funcionan de abajo hacia arriba, no se describieron aquí ya que requieren la elaboración de más teoría de compilación.

De hecho, en este capítulo hemos eludido la teoría del lenguaje formal estudiada en los cursos típicos de compilación. Además, hemos elegido una sintaxis simple para el lenguaje Jack, una sintaxis que se puede compilar fácilmente utilizando técnicas de descenso recursivo. Por ejemplo, la gramática de Jack no exige la prioridad habitual de los operadores en la evaluación de expresiones algebraicas, como la multiplicación antes de la suma. Esto nos permitió evitar algoritmos de análisis que son más poderosos, pero también más intrincados, que las elegantes técnicas de análisis de arriba hacia abajo presentadas en este capítulo.

Todos los programadores experimentan el vergonzoso manejo de errores de compilación, que es típico de muchos compiladores. Resulta que el diagnóstico y la notificación de errores son un problema desafiante. En muchos casos, el impacto de un error se detecta varias o muchas líneas de código después de que se cometió el error. Por lo tanto, los informes de errores a veces son crípticos y poco amigables. De hecho, un aspecto en el que los compiladores varían mucho es su capacidad para diagnosticar y ayudar a depurar errores. Para ello, los compiladores conservan partes del árbol de análisis en la memoria y amplían el árbol con anotaciones que ayudan a identificar el origen de los errores y retroceder el proceso de diagnóstico, según sea necesario. En Nand to Tetris omitimos todas estas extensiones, asumiendo que los archivos fuente que maneja el compilador están libres de errores.

Otro tema que apenas mencionamos es cómo se estudia la sintaxis y la semántica de los lenguajes de programación en informática y ciencias cognitivas. Existe una rica teoría de los lenguajes formales y naturales que discute las propiedades de las clases de lenguajes, así como los metalenguajes y los formalismos para especificarlos. Este es también el lugar donde la informática se encuentra con el estudio de los lenguajes humanos, lo que lleva a las vibrantes áreas de investigación y práctica

conocidas como lingüística computacional y procesamiento del lenguaje natural.

Finalmente, vale la pena mencionar que los analizadores de sintaxis generalmente no son programas independientes y rara vez se escriben desde cero. En cambio, los programadores generalmente construyen tokenizadores y analizadores utilizando una variedad de herramientas generadoras de *compiladores* como LEX (para *análisis LEXical*) y YACC (para *Yet Another Compiler Compiler*). Estas utilidades reciben como entrada una gramática libre de contexto y producen como salida código de análisis de sintaxis capaz de tokenizar y analizar programas escritos en esa gramática. El código generado se puede personalizar para adaptarse a las necesidades específicas del escritor del compilador. Sin embargo, siguiendo el espíritu de "muéstrame" de Nand to Tetris, hemos optado por no usar tales cajas negras en la implementación de nuestro compilador, sino construir todo desde cero.

---

1. Y aquí radica una diferencia crucial entre los lenguajes de programación y los lenguajes naturales. En los lenguajes naturales, podemos decir cosas como "Quien salva una vida, salva al mundo entero". En el idioma inglés, poner el adjetivo después del sustantivo es gramaticalmente incorrecto. Sin embargo, en este caso particular, suena perfectamente aceptable. A diferencia de los lenguajes de programación, los lenguajes naturales exigen una licencia poética para romper las reglas gramaticales, siempre que el escritor sepa lo que está haciendo. Esta libertad de expresión hace que los lenguajes naturales sean infinitamente ricos.

---

# 11    **Compilador II: Generación de código**

Cuando estoy trabajando en un problema, nunca pienso en la belleza. Pero cuando he terminado, si la solución no es hermosa, sé que está mal.

—R. Buckminster Fuller (1895–1993)

La mayoría de los programadores dan por sentado a los compiladores. Pero si te detienes a pensarlo, la capacidad de traducir un programa de alto nivel en código binario es casi como magia. En Nand to Tetrís dedicamos cuatro capítulos (7-11) para desmitificar esta magia. Nuestra metodología práctica se basa en el desarrollo de un compilador para Jack, un lenguaje simple y moderno basado en objetos. Al igual que con Java y C#, el compilador Jack general se basa en dos niveles: un *back-end de máquina virtual (VM)* que traduce los comandos de VM al lenguaje de máquina y un *compilador front-end* que traduce los programas Jack en código VM. Construir un compilador es una tarea desafiante, por lo que lo dividimos en dos módulos conceptuales: un *analizador de sintaxis*, desarrollado en el capítulo 10, y un *generador de código*, el tema de este capítulo.

El analizador de sintaxis se creó para desarrollar y demostrar una capacidad para analizar programas de alto nivel en sus elementos sintácticos subyacentes. En este capítulo transformaremos el analizador en un compilador a gran escala: un programa que convierte los elementos analizados en comandos de VM diseñados para ejecutarse en la máquina virtual abstracta descrita en los capítulos 7:

8. Este enfoque sigue el paradigma modular de análisis-síntesis que informa la construcción de compiladores bien diseñados. También captura la esencia misma de traducir texto de un idioma a otro: primero, se usa la *sintaxis* del idioma de origen para analizar el texto de origen y descubrir su *semántica subyacente*, es decir, lo que el texto busca decir; luego, se reexpresa la semántica analizada utilizando la sintaxis del idioma de destino. En



el contexto de este capítulo, el origen y el destino son Jack y el lenguaje VM, respectivamente.

Los lenguajes de programación modernos de alto nivel son ricos y poderosos. Permiten definir y usar abstracciones elaboradas como funciones y objetos, expresar algoritmos usando declaraciones elegantes y construir estructuras de datos de complejidad ilimitada. Por el contrario, las plataformas de hardware en las que finalmente se ejecutan estos programas son espartanas y mínimas. Por lo general, ofrecen poco más que un conjunto de registros para el almacenamiento y un conjunto de instrucciones primitivas para el procesamiento. Por lo tanto, la traducción de programas de alto nivel a bajo nivel es una hazaña desafiante. Si la plataforma de destino es una máquina virtual y no el hardware básico, la vida es algo más fácil ya que los comandos abstractos de VM no son tan primitivos como las instrucciones concretas de la máquina. Aún así, la brecha entre la expresividad de un lenguaje de alto nivel y la de un lenguaje de VM es amplia y desafiante.

El capítulo comienza con una discusión general de *la generación de código*, dividida en seis secciones. Primero, describimos cómo los compiladores usan tablas de *símbolos* para asignar variables simbólicas a segmentos de memoria virtual. A continuación, presentamos algoritmos para compilar *expresiones* y *cadena*s de caracteres. Luego presentamos técnicas para compilar *declaraciones* como *let*, *if*, *while*, *do* y *return*. En conjunto, la capacidad de compilar *variables*, *expresiones* e *instrucciones* forma una base para construir compiladores para lenguajes simples, procedimentales y similares a C. Esto prepara el escenario para el resto de la sección 11.1, en la que discutimos la compilación de *objetos* y *matrices*.

La sección 11.2 (Especificación) proporciona pautas para mapear programas Jack en la plataforma y el lenguaje de VM, y la sección 11.3 (Implementación) propone una arquitectura de software y una API para completar el desarrollo del compilador. Como de costumbre, el capítulo termina con una sección de Proyecto, que proporciona pautas paso a paso y programas de prueba para completar la construcción del compilador, y una sección de Perspectiva que comenta varias cosas que se omitieron en el capítulo.

Entonces, ¿qué hay para ti? Aunque muchos profesionales están ansiosos por entender cómo funcionan los compiladores, pocos terminan ensuciándose las manos y construyendo un compilador desde cero. Esto se debe a que el costo de esta experiencia, al menos en el mundo académico, suele ser un curso electivo desalentador de un semestre completo. Nand to Tetris reúne los elementos clave de esta experiencia en cuatro capítulos y

proyectos, que culminan en el presente capítulo. En

, discutimos e ilustramos los algoritmos clave, las estructuras de datos y los trucos de programación que subyacen a la construcción de compiladores típicos. Ver estas ideas y técnicas inteligentes en acción lleva a uno a maravillarse, una vez más, de cómo el ingenio humano puede vestir una máquina de conmutación primitiva para que parezca algo que se acerca a la magia.

---

## 11.1 Generación de código

Los programadores de alto nivel trabajan con bloques de construcción abstractos como *variables*, *expresiones*, *declaraciones*, *subrutinas*, *objetos* y *matrices*. Los programadores usan estos bloques de construcción abstractos para describir lo que quieren que haga el programa. El trabajo del compilador es traducir esta semántica a un lenguaje que entienda una computadora de destino.

En nuestro caso, la computadora de destino es la máquina virtual descrita en los capítulos 7 y 8. Por lo tanto, tenemos que descubrir cómo traducir sistemáticamente *expresiones*, *declaraciones*, *subrutinas* y el manejo de *variables*, *objetos* y *matrices* en secuencias de comandos de VM basados en pila que ejecuten la semántica deseada en la máquina virtual de destino. No tenemos que preocuparnos por la posterior traducción de los programas de VM al lenguaje de máquina, ya que ya nos ocupamos de este dolor de cabeza real en los proyectos 7-8. Gracias a Dios por la compilación de dos niveles y por el diseño modular.

A lo largo del capítulo, presentamos ejemplos de compilación de varias partes de la clase Point presentadas anteriormente en el libro. Repetimos la declaración de clase en la [figura 11.1](#), que ilustra la mayoría de las características del lenguaje Jack. Recomendamos echar un vistazo rápido a este código Jack ahora para refrescar su memoria sobre la funcionalidad de la clase Point. A continuación, estará listo para profundizar en el esclarecedor viaje de reducir sistemáticamente esta funcionalidad de alto nivel, y cualquier otro programa similar basado en objetos, en el código de VM.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> /** Represents a two-dimensional point. File name: Point.jack. */ class Point {     // The coordinates of this point:     field int x, y      // The number of Point objects constructed so far:     static int pointCount;      /** Constructs a two-dimensional point and     initializes it with the given coordinates. */     constructor Point new(int ax, int ay) {         let x = ax;         let y = ay;         let pointCount = pointCount + 1;         return this;     }      /** Returns the x coordinate of this point. */     method int getX() { return x; }      /** Returns the y coordinate of this point. */     method int getY() { return y; }      /** Returns the number of Point constructed so far. */     function int getPointCount() {         return pointCount;     }      // Class declaration continues on top right. </pre> | <pre>     /** Returns a point which is this point plus     the other point. */     method Point plus(Point other) {         return Point.new(x + other.getX(),                         y + other.getY());     }      /** Returns the Euclidean distance between     this and the other point. */     method int distance(Point other) {         var int dx, dy;         let dx = x - other.getX();         let dy = y - other.getY();         return Math.sqrt((dx*dx) + (dy*dy));     }      /** Prints this point, as "(x,y)" */     method void print() {         do Output.printString("(");         do Output.printInt(x);         do Output.printString(",");         do Output.printInt(y);         do Output.printString(")");         return;     } } // End of Point class declaration. </pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Figura 11.1** La clase Point. Esta clase presenta todos los tipos de variables posibles (campo, estático, local y argumento) y tipos de subrutinas (constructor, método y función), así como subrutinas que devuelven tipos primitivos, tipos de objeto y subrutinas void. También ilustra las llamadas a funciones, las llamadas a constructores y las llamadas a métodos en el objeto actual (this) y en otros objetos.

### 11.1.1 Manejo de variables

Una de las tareas básicas de los compiladores es asignar las variables declaradas en el programa de alto nivel de origen a la RAM host de la plataforma de destino. Por ejemplo, considere Java: las variables int están diseñadas para representar valores de 32 bits; variables largas, valores de 64 bits; y así sucesivamente. Si la RAM del host tiene un ancho de 32 bits, el compilador asignará variables int y long en una palabra de memoria y en dos palabras de memoria consecutivas, respectivamente. En Nand to Tetris no hay complicaciones de mapeo: todos los tipos primitivos en Jack (int, char y boolean) tienen 16 bits de ancho, al igual que las direcciones y palabras de la RAM de Hack. Por lo tanto, todas las variables de Jack, incluidas las direcciones, se pueden asignar exactamente en una palabra de memoria de 16 bits.

El segundo desafío que enfrentan los compiladores es que las variables de diferentes *tipos* tienen diferentes ciclos de vida. Las variables estáticas de nivel de clase son compartidas globalmente por todas las subrutinas de la clase. Por lo tanto, una sola copia de cada variable estática sobrevive a la ejecución completa de la variable.

Las variables de campo a nivel de instancia se tratan de manera diferente: cada objeto (instancia de la clase) debe tener un conjunto privado de su campo.

variables y, cuando el objeto ya no es necesario, esta memoria debe liberarse. Las variables locales y de argumento a nivel de subrutina se crean cada vez que una subrutina comienza a ejecutarse y deben liberarse cuando finaliza la subrutina. Esa es la mala noticia. La buena noticia es que ya hemos manejado todas estas dificultades. En nuestra arquitectura de compilador de dos niveles, la asignación y desasignación de memoria se delega en el nivel de máquina virtual. Todo lo que tenemos que hacer ahora es mapear a Jack *estático* variables en *estático 0*, *estático 1*, *estático 2*, ...; *campo* variables en *este 0*, *este 1*, ...; *local* variables en *local 0*, *local 1*, ...; y *argumento* variables en *argumento 0*, *Argumento 1*, .... El mapeo posterior de los segmentos de memoria virtual en la RAM del host, así como la intrincada gestión de su tiempo de ejecución vida Ciclos son completamente Tomado cuidado de por el VM implementación.

Recuerde que esta implementación no se logró fácilmente: tuvimos que trabajar duro para generar código ensamblador que mapeara dinámicamente los segmentos de memoria virtual en la RAM del host como un efecto secundario de la realización del protocolo de llamada y retorno de funciones. Ahora podemos cosechar los beneficios de este esfuerzo: el único segmento de memoria

Todos los detalles sangrientos posteriores asociados con la administración de estos segmentos en la RAM serán manejados por la implementación de VM. Es por eso que a veces nos referimos a este último como el *back-end del compilador*.

En resumen, en un modelo de compilación de dos niveles, el control de variables se puede reducir a la asignación de variables de alto nivel en segmentos de memoria virtual y el uso de esta asignación, según sea necesario, a lo largo de la generación de código. Estas tareas se pueden administrar fácilmente utilizando una abstracción clásica conocida como tabla de *símbolos*.

**Tabla de símbolos:** cada vez que el compilador encuentre variables en una instrucción de alto nivel, por ejemplo, hágale  $y = \text{foo}(x)$ , saber qué significan las variables. ¿Es  $x$  una variable estática, un campo de un objeto, una variable local o un argumento de una subrutina? ¿Representa un número entero, un booleano, un char o algún tipo de clase? Todas estas preguntas deben responderse, para la generación de código, cada vez que aparece la variable  $x$  en el código fuente. Por supuesto, la variable  $y$  debe tratarse exactamente de la misma manera.

Las propiedades de las variables se pueden administrar cómodamente mediante una tabla de *símbolos*. Cuando se declara una variable estática, de campo, local o de argumento en el

código fuente, el compilador lo asigna a la siguiente entrada disponible en el segmento de VM estático, este, local o de argumento correspondiente y registra la asignación en la tabla de símbolos. Cada vez que se encuentra una variable en otra parte del código, el compilador busca su nombre en la tabla de símbolos, recupera sus propiedades y las usa, según sea necesario, para la generación de código.

Una característica importante de los lenguajes de alto nivel son *los espacios de nombres separados*: el mismo identificador puede representar cosas diferentes en diferentes regiones del programa. Para habilitar espacios de nombres independientes, cada identificador se asocia implícitamente con un *ámbito*, que es la región del programa en la que se reconoce. En Jack, el alcance de las variables estáticas y de campo es la clase en la que se declaran, el alcance de las variables locales es la subrutina en la que se declaran.

Los compiladores Jack pueden realizar las abstracciones de alcance administrando dos tablas de símbolos separadas, como se ve en la [figura 11.2](#).

High-level (Jack) code

```
class Point {  
    field int x, y;  
    static int pointCount;  
    ...  
    method int distance(Point other) {  
        var int dx, dy;  
        let dx = x - other.getX();  
        let dy = y - other.getY();  
        return Math.sqrt((dx*dx) + (dy*dy));  
    }  
    ...  
}
```

| name       | type | kind   | # |
|------------|------|--------|---|
| x          | int  | field  | 0 |
| y          | int  | field  | 1 |
| pointCount | int  | static | 0 |

Class-level  
symbol table

| name  | type  | kind | # |
|-------|-------|------|---|
| this  | Point | arg  | 0 |
| other | Point | arg  | 1 |
| dx    | int   | var  | 0 |
| dy    | int   | var  | 1 |

Subroutine-level  
symbol table

**Figura 11.2** Ejemplos de tablas de símbolos. El éste de la tabla de nivel de subrutina se analiza más adelante en el capítulo.

Los ámbitos están anidados, con ámbitos internos que ocultan los externos. Por ejemplo, cuando el compilador Jack encuentra la expresión,  $x + 17$ , primero verifica si  $x$  es una variable de nivel de subrutina (local o argumento). De lo contrario, el compilador comprueba si  $x$  es una variable estática o un campo. Algunos lenguajes cuentan con un alcance anidado de profundidad ilimitada, lo que permite que las variables sean locales en cualquier bloque de código en el que se declaren. Para admitir el anidamiento ilimitado, el compilador puede usar una lista vinculada de tablas de símbolos, cada una de las cuales refleja un único ámbito anidado dentro del siguiente de la lista. Cuando el compilador no encuentra la variable en la tabla asociada con el ámbito actual, la busca en

la siguiente tabla de la lista, desde los ámbitos internos hacia afuera. Si la variable no se encuentra en la lista, el compilador puede generar un error de "variable no declarada".

En el lenguaje Jack solo hay dos niveles de alcance: la subrutina que se está compilando actualmente y la clase en la que se declara la subrutina. Por lo tanto, el compilador puede salirse con la suya administrando solo dos tablas de símbolos.

**Manejo de declaraciones de variables:** Cuando el compilador Jack comienza a compilar una declaración de clase, crea una tabla de símbolos a nivel de clase y una tabla de símbolos a nivel de subrutina. Cuando el compilador analiza una declaración de variable estática o de campo, agrega una nueva fila a la tabla de símbolos de nivel de clase. La fila registra el nombre de la variable, el *tipo* (entero, booleano, char o nombre de clase), el *tipo* (estático o de campo) y el *índice* dentro del tipo.

Cuando el compilador Jack comienza a compilar una declaración de subrutina (constructor, método o función), restablece la tabla de símbolos a nivel de subrutina. Si la subrutina es un método, el compilador agrega la fila `<this, className, arg, 0>` a la tabla de símbolos a nivel de subrutina (este detalle de inicialización se explica en la sección 11.1.5.2 y se puede ignorar hasta entonces). Cuando el compilador analiza una declaración de variable local o de argumento, agrega una nueva fila a la tabla de símbolos de nivel de subrutina, registrando el nombre de la variable, el tipo (entero, booleano, char o nombre de clase), el tipo (var o arg) y el índice dentro del tipo. El índice de cada tipo (var o arg) comienza en 0 y se incrementa en 1 después de cada vez que se agrega una nueva variable de ese tipo a la tabla.

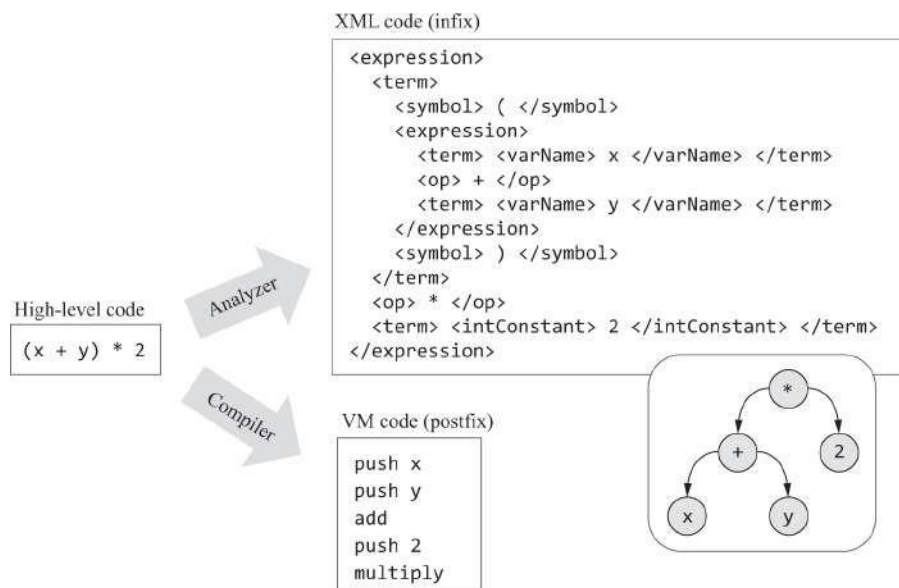
**Manejo de variables en instrucciones:** cuando el compilador encuentra una variable en una instrucción, busca el nombre de la variable en la tabla de símbolos de nivel de subrutina. Si no se encuentra la variable, el compilador la busca en la tabla de símbolos de nivel de clase. Una vez que se ha encontrado la variable, el compilador puede completar la traducción de la instrucción. Por ejemplo, considere las tablas de símbolos que se muestran en la [figura 11.2](#) y supongamos que estamos compilando la instrucción de alto nivel `let y = y + dy`. El compilador traducirá esta instrucción en los comandos de VM `push this 1, push local 1, add, pop this 1`. Aquí asumimos que el compilador sabe cómo manejar expresiones y sentencias `let`, temas que se tratan en las dos secciones siguientes.

### 11.1.2 Compilación de expresiones



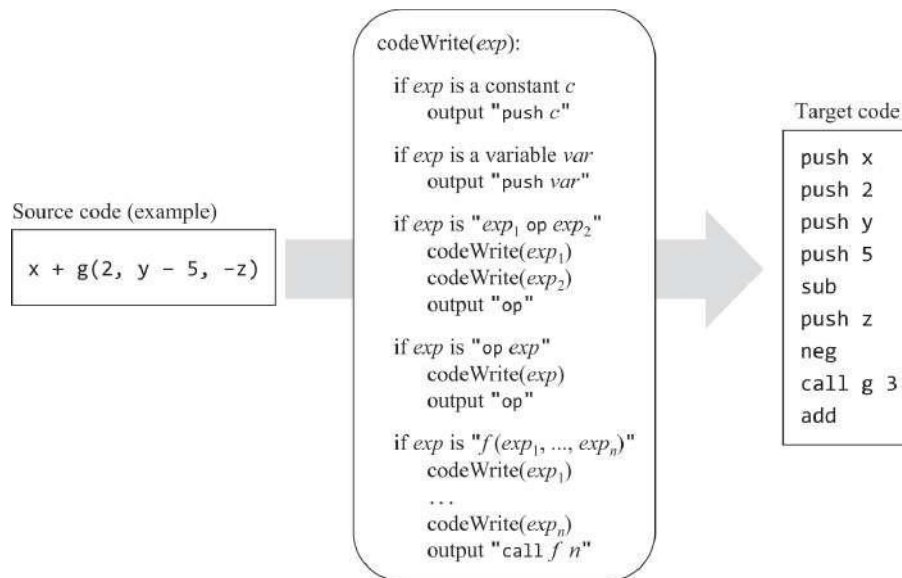
Comencemos considerando la compilación de expresiones simples como  $x + y - 7$ . Por "expresión simple" nos referimos a una secuencia de *término operador término término ...*, donde cada *término* es una variable o una constante, y cada *operador* es  $+$ ,  $-$ ,  $*$  o  $/$ .

En Jack, como en la mayoría de los lenguajes de alto nivel, las expresiones se escriben usando *notación infija*: Para agregar  $x$  e  $y$ , se escribe  $x + y$ . Por el contrario, el lenguaje de destino de nuestra compilación es *postfix*: la misma semántica de adición se expresa en el código de VM orientado a la pila como `push x, push y, add`. En el capítulo 10 presentamos un algoritmo que emite el código fuente analizado en infijo usando etiquetas XML. Aunque la lógica de análisis de este algoritmo puede seguir siendo la misma, la parte de salida del algoritmo ahora debe modificarse para generar comandos de sufijo. La Figura 11.3 ilustra esta dicotomía.



**Figura 11.3** Representaciones infijas y postfijas de la misma semántica.

Para recapitular, necesitamos un algoritmo que sepa cómo analizar una expresión de infijo y generarla como código de sufijo de salida que realice la misma semántica en una máquina de pila. La Figura 11.4 presenta uno de esos algoritmos. El algoritmo procesa la expresión de entrada de izquierda a derecha, generando código de máquina virtual a medida que avanza. Convenientemente, este algoritmo también maneja operadores unarios y llamadas a funciones.



**Figura 11.4** Un algoritmo de generación de código de máquina virtual para expresiones y un ejemplo de compilación. El algoritmo asume que la expresión de entrada es válida. La implementación final de este algoritmo debe reemplazar las variables simbólicas emitidas con sus correspondientes asignaciones de tabla de símbolos.

Si ejecutamos el código de VM basado en pila generado por el algoritmo `codeWrite` (lado derecho de la [figura 11.4](#)), la ejecución terminará consumiendo todos los términos de la expresión y colocando el valor de la expresión en la parte superior de la pila. Eso es exactamente lo que se espera del código compilado de una expresión.

Hasta ahora hemos tratado con expresiones relativamente simples. La [Figura 11.5](#) da la definición gramatical completa de las expresiones de Jack, junto con varios ejemplos de expresiones reales consistentes con esta definición.

Definition (from the Jack grammar):

```
expression: term (op term) *  
  
term: integerConstant | stringConstant | keywordConstant | varName |  
      varName '[' expression ']' | '(' expression ')' | (unaryOp term) | subroutineCall  
  
subroutineCall: subroutineName '(' expressionList ')' |  
               (className | varName) '.' subroutineName '(' expressionList ')' |  
  
expressionList: (expression '(' ',' expression ')') * ?  
  
op: '+' | '-' | '*' | '/' | '%' | '^' | '<' | '>' | '='  
  
unaryOp: '-' | '~'  
  
keywordConstant: 'true' | 'false' | 'null' | 'this'
```

The definitions of *integerConstant*, *stringConstant*, *keywordConstant*, and other elements are given in the complete Jack grammar (figure 10.6), and are self-explanatory.

Examples:

```
5  
x  
x + 5  
(-b + Math.sqrt(b*b - (4 * a * c))) / (2 * a)  
arr[i] + foo(x)  
foo(Math.abs(arr[x + foo(5)]))
```

**Figura 11.5** Expresiones en el lenguaje Jack.

La compilación de expresiones Jack será manejada por una rutina llamada `compileExpression`. El desarrollador de esta rutina debe comenzar con el algoritmo presentado en la [figura 11.4](#) y extenderlo para manejar las diversas posibilidades especificadas en la [figura 11.5](#). Tendremos más que decir sobre esta implementación más adelante en el capítulo.

### 11.1.3 Compilación de cadenas

Las cadenas (secuencias de caracteres) se utilizan ampliamente en programas informáticos. Los lenguajes orientados a objetos suelen manejar cadenas como instancias de una clase llamada `String` ( la clase `String` de Jack, que es parte del sistema operativo Jack, se documenta en el apéndice 6). Cada vez que aparece una constante de cadena en una instrucción o expresión de alto nivel, el compilador genera código que llama al constructor `String`, que crea y devuelve un nuevo objeto `String`. A continuación, el compilador inicializa el nuevo objeto con los caracteres de cadena. Esto se hace generando una secuencia de llamadas al método `String appendChar`, una para cada carácter enumerado en la constante de cadena de alto nivel.

Esta implementación de constantes de cadena puede ser un desperdicio, lo que lleva a posibles pérdidas de memoria. Para ilustrarlo, considere la instrucción `Output.printString("Cargando ... por favor espere")`. Presumiblemente, todo lo que el programador de alto nivel quiere es mostrar un mensaje; Ciertamente no le importa si el compilador crea un nuevo objeto, y puede sorprenderse al saber que el objeto persistirá en la memoria hasta que finalice el programa. Pero eso es exactamente lo que realmente sucede: se creará un nuevo objeto `String`, y este objeto seguirá acechando en segundo plano, sin hacer nada.

Java, C# y Python usan un proceso de recolección de *elementos no reutilizados* en tiempo de ejecución que se que ya no están en juego objetos que no tienen ninguna variable que haga

En general, los lenguajes modernos usan una variedad de optimizaciones y clases de cadena especializadas para promover el uso eficiente de objetos de cadena. El sistema operativo Jack presenta solo una clase `String` y no optimizaciones relacionadas con cadenas.

**Servicios del sistema operativo:** En el manejo de cadenas, mencionamos por primera vez que el compilador puede usar los servicios del sistema operativo según sea necesario. De hecho, los desarrolladores de compiladores Jack pueden suponer que todos los constructores, métodos y funciones enumerados en la API del sistema operativo (apéndice 6) están disponibles *como una función de máquina virtual compilada*. Técnicamente hablando, cualquiera de estas funciones de VM puede ser llamada por el código generado por el compilador. Esta configuración se realizará completamente en el capítulo 12, en el que implementaremos el sistema operativo en Jack y lo compilaremos en código VM.

#### 11.1.4 Compilación de instrucciones

El lenguaje de programación Jack presenta cinco declaraciones: `let`, `do`, `return`, `if` y `while`. Ahora pasamos a analizar cómo el compilador Jack genera código de máquina virtual que maneja la semántica de estas declaraciones.

**Compilación de declaraciones de retorno:** Ahora que sabemos cómo compilar expresiones, la compilación de expresiones de retorno es simple. En primer lugar, llamamos a la rutina `compileExpression`, que genera código de máquina virtual diseñado para evaluar y colocar el valor de la expresión en la pila. A continuación, generamos el comando VM `return`.

**Compilación de sentencias let:** Aquí discutimos el manejo de sentencias de la forma `let varName = expresión`. Dado que el análisis es un proceso de izquierda a derecha, comenzamos recordando `varName`. A continuación, llamamos a `compileExpression`, que coloca el valor de la expresión en la pila. Por último, generamos el comando de máquina virtual `pop varName`, donde `varName` es en realidad la asignación de tabla de símbolos de `varName` (por ejemplo, `local 3`, `static 1`, etc.).

Discutiremos la compilación de declaraciones de la forma `let varName[expression1] = expression2` más adelante en el capítulo, en una sección dedicada al manejo de matrices.

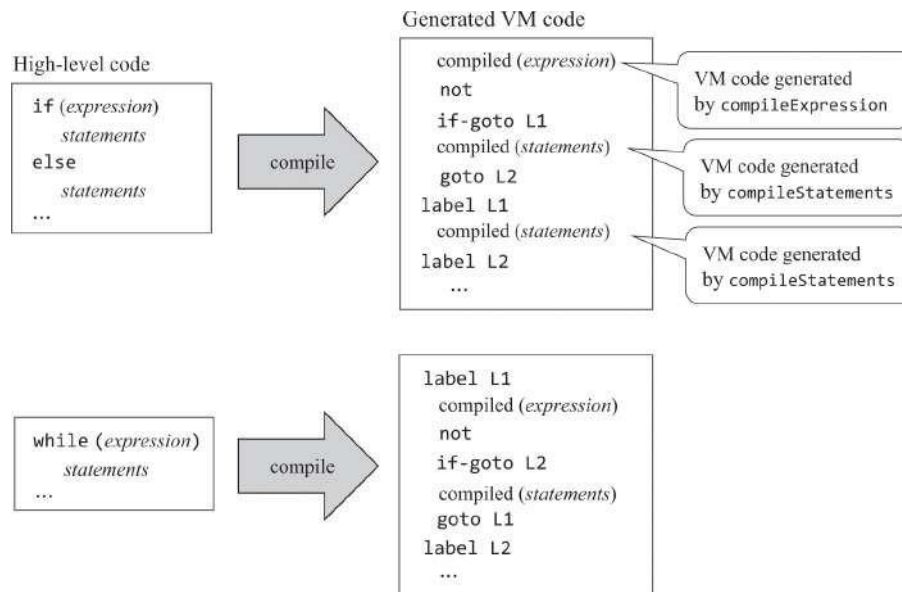
**Compilación de declaraciones do:** Aquí discutimos la compilación de llamadas a funciones de la forma `do className.functionName (exp1, exp2, ..., expn)`. La compilación está diseñada para llamar a una subrutina por su efecto,

valor. En el capítulo 10 recomendamos compilar tales declaraciones como si su sintaxis fuera una *expresión do*. Repetimos esta recomendación aquí: para compilar un `do className.functionName (...)`, llamamos a `compileExpression` y luego nos deshacemos del elemento de pila superior (el valor de la expresión) generando un comando como `pop temp 0`.

Discutiremos la compilación de *llamadas a métodos* de la forma `varName.methodName (...)` y `do methodName (...)` más adelante en el capítulo, en una sección dedicada a la compilación de llamadas a métodos.

**Compilación de instrucciones if y while:** Los lenguajes de programación de alto nivel presentan una variedad de declaraciones de *flujo de control* como `if`, `while`, `for` y `switch`, de las cuales Jack presenta `if` y `while`. Por el contrario, los lenguajes de ensamblado y VM de bajo nivel controlan el flujo de ejecución mediante dos primitivos de bifurcación: *goto condicional* y *goto incondicional*. Por lo tanto, uno de los desafíos que enfrentan los desarrolladores de compiladores es expresar la semántica de las instrucciones de flujo de control de alto nivel utilizando nada más que primitivas `goto`. [Figura](#)

[11.6](#) muestra cómo se puede cerrar esta brecha sistemáticamente.



**Figura 11.6** Compilar si y mientras Declaraciones. El L1 y L2 Las etiquetas son generadas por el compilador.

Cuando el compilador detecta una palabra clave `if`, sabe que tiene que analizar un patrón de la forma `if (expresión) {statements} else {statements}`. Por lo tanto, el compilador comienza llamando a `compileExpression`, que genera comandos de máquina virtual diseñados para calcular e insertar el valor de la expresión en la pila. A continuación, el compilador genera el comando VM `not`, diseñado para negar el valor de la expresión. A continuación, el compilador crea una etiqueta, por ejemplo, `L1`, y la usa para generar el comando de máquina virtual `if-goto L1`. A continuación, el compilador llama a `compileStatements`. Esta rutina está diseñada para compilar una secuencia de la instrucción de formulario; *declaración*; ... *declaración*;, donde cada *instrucción* es un `let`, un `do`, un `return`, un `if` o un `while`. El código de VM resultante se conoce conceptualmente en la figura 11.6 como "compiladas (*instrucciones*)". El resto de la estrategia de compilación se explica por sí misma.

Un programa de alto nivel normalmente contiene varias instancias de `if` y `while`. Para controlar esta complejidad, el compilador puede generar etiquetas que son únicas globalmente, por ejemplo, etiquetas cuyo sufijo es el valor de un contador en ejecución. Además, las instrucciones de flujo de control a menudo están anidadas, por ejemplo, un `if` dentro de un tiempo dentro de otro tiempo, y así sucesivamente. Estos anidamientos se cuidan implícitamente, ya que la rutina `compileStatements` es inherentemente recursiva.

## 11.1.5 Manejo de objetos



Hasta ahora en este capítulo, describimos técnicas para compilar *variables*, *expresiones*, *cadenas* y *declaraciones*. Esto forma la mayor parte del conocimiento necesario para construir un compilador para un lenguaje procedimental, similar a C. En Nand to Tetris, sin embargo, apuntamos más alto: construir un compilador para un lenguaje basado en objetos, similar a Java. Con eso en mente, ahora pasamos a discutir el manejo de *objetos*.

Los lenguajes orientados a objetos cuentan con medios para declarar y manipular abstracciones agregadas conocidas como *objetos*. Cada objeto se implementa como un bloque de memoria al que se puede hacer referencia local o variable de

La variable de referencia, también conocida como *variable de objeto* o *puntero*, contiene la dirección base del bloque de memoria. El sistema operativo realiza este modelo administrando un área lógica en la RAM denominada *montón*. El *montón* se usa como un grupo de memoria del que se tallan bloques de memoria, según sea necesario, para representar nuevos objetos. Cuando un objeto ya no es necesario, su bloque de memoria se puede liberar y reciclar de nuevo en el montón. El compilador organiza estas acciones de administración de memoria llamando a funciones del sistema operativo, como veremos más adelante.

En cualquier momento dado durante la ejecución de un programa, el montón puede contener numerosos objetos. Supongamos que queremos aplicar un método, digamos *foo*, a uno de estos objetos, digamos *p*. En un lenguaje orientado a objetos, esto se hace a través de la llamada al método *idiom p.foo()*. Al cambiar nuestra atención del llamador al destinatario de la llamada, observamos que *foo*, como cualquier otro método, está diseñado para operar en un marcador de posición conocido como *el objeto actual*, o *this*. En particular, cuando los comandos VM en el código de *foo* hacen referencias a este 0, este 1, este 2, etc., deben afectar los campos de *p*, el objeto en el que se llamó a *foo*. Lo que plantea la pregunta: ¿Cómo alineamos este segmento con *p*?

La máquina virtual construida en los capítulos 7 y 8 tiene un mecanismo para realizar esta alineación: el segmento de puntero de dos valores, asignado directamente a las ubicaciones de RAM 3 y 4, también conocido como *THIS* y *THAT*. De acuerdo con la especificación de VM, el puntero *THIS* (denominado puntero 0) está diseñado para contener la dirección base del segmento de memoria *this*. Por lo tanto, para alinear este segmento con el objeto *p*, podemos empujar el valor de *p* (que es una dirección) en la pila y luego colocarlo en el puntero 0. Las versiones de esta técnica de inicialización se utilizan de manera conspicua en la compilación de *constructores* y *métodos*, como ahora describimos.

### 11.1.5.1 Compilación de constructores

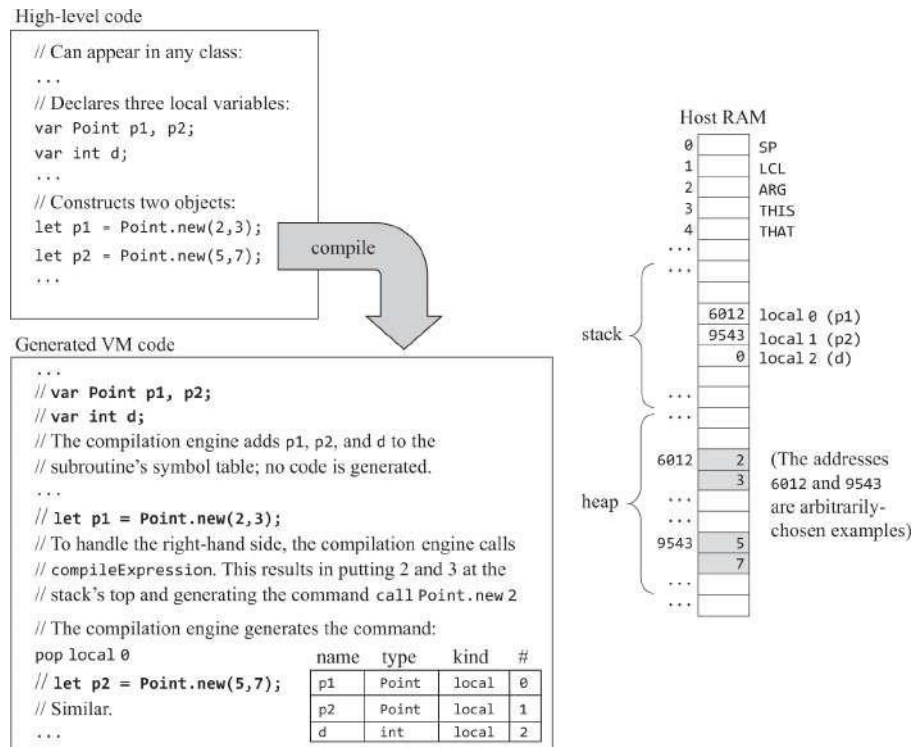
En los lenguajes orientados a objetos, los objetos son creados por subrutinas conocidas como *constructores*. En esta sección describimos cómo compilar una llamada al constructor (por ejemplo, el operador `new` de Java) desde la perspectiva del autor de la llamada y cómo compilar el código del propio constructor: el *destinatario*.

**Compilación de llamadas de constructor:** la construcción de objetos es normalmente un asunto de dos etapas. Primero, se declara una variable de algún tipo de clase, por ejemplo, `var Point`

pág. En una etapa posterior, se puede crear una instancia del objeto llamando a un constructor de clase, por ejemplo, `let p = Point.new(2,3)`. O, dependiendo del lenguaje utilizado, se pueden declarar y construir objetos utilizando una sola instrucción de alto nivel. Sin embargo, detrás de escena, esta acción siempre se divide en dos etapas separadas: declaración seguida de construcción.

Echemos un vistazo de cerca a la declaración `let p = Point.new(2,3)`. Esta abstracción se puede describir como "hacer que el constructor `Point.new` asigne un bloque de memoria de dos palabras para representar una nueva instancia de `Point`, inicializar las dos palabras de este bloque en 2 y 3, y hacer que `p` se refiera a la dirección base de este bloque". Implícito en esta semántica hay dos suposiciones: Primero, el constructor sabe cómo asignar un bloque de memoria del tamaño requerido. En segundo lugar, cuando el constructor, al ser una subrutina, termina su ejecución, devuelve al autor de la llamada la dirección base del bloque de memoria asignado. [La Figura 11.7](#) muestra cómo se puede realizar esta abstracción.



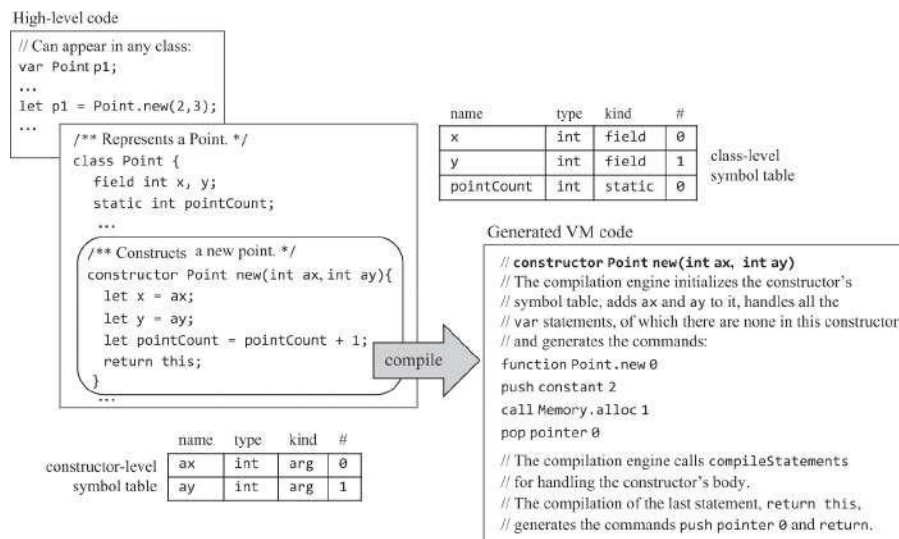


**Figura 11.7** Construcción de objetos desde la perspectiva de la persona que llama. En este ejemplo, el autor de la llamada declara dos variables de objeto y, a continuación, llama a un constructor de clase para construir los dos objetos. El constructor hace su magia, asignando bloques de memoria para representar los dos objetos. El código de llamada hace que las dos variables de objeto se refieran a estos bloques de memoria.

Tres observaciones sobre la figura 11.7 están en orden. Primero, tenga en cuenta que no hay nada especial en compilar declaraciones como `let p = Point.new(2,3)` y `let p = Punto.nuevo(5,7)`. Ya discutimos cómo compilar declaraciones `let` y llamadas a subrutinas. Lo único que hace que estas llamadas sean especiales es la suposición de que, de alguna manera, se construirán dos objetos. La implementación de esta magia se delega por completo a la compilación del *destinatario*: el constructor. Como resultado de esta magia, el constructor crea los dos objetos que se ven en el diagrama de RAM en la figura 11.7. Esto lleva a la segunda observación: las direcciones físicas 6012 y 9543 son irrelevantes; el código de alto nivel, así como el código de VM compilado, no tienen idea de dónde se almacenan los objetos en la memoria; Las referencias a estos objetos son estrictamente simbólicas, a través de `p1` y `p2` en el código de alto nivel y `Local 0` y `Local 1` en el código compilado. (Como comentario al margen, esto hace que el programa sea reubicable y más seguro). En tercer lugar, y afirmando lo obvio, nada sustancial sucede hasta que se ejecuta el código de VM generado. En particular, durante *el tiempo de compilación*,

Se actualiza la tabla de símbolos, se genera código de bajo nivel y listo. Los objetos se construirán y enlazarán a las variables solo durante el *tiempo de ejecución*, es decir, si se ejecutará el código compilado y cuándo.

**Compilación de constructores:** Hasta ahora, hemos tratado a los constructores como abstracciones de caja negra: asumimos que crean objetos, *de alguna manera*. La Figura 11.8 desentraña esta magia. Antes de inspeccionar la figura, tenga en cuenta que un constructor es, ante todo, una *subrutina*. Puede tener argumentos, variables locales y un cuerpo de declaraciones; por lo tanto, el compilador lo trata como tal. Lo que hace que la compilación de un constructor sea especial es que, además de tratarlo como una subrutina regular, el compilador también debe generar código que (i) cree un nuevo objeto y (ii) haga que el nuevo objeto sea el *objeto actual* (también conocido como *este*), es decir, el objeto sobre el que operará el código del constructor.



**Figura 11.8** Construcción de objetos: la perspectiva del constructor.

La creación de un nuevo objeto requiere encontrar un bloque de RAM libre lo suficientemente grande como para acomodar los datos del nuevo objeto y marcar el bloque como utilizado. Estas tareas se delegan al sistema operativo host. De acuerdo con la API del sistema operativo que se enumera en el apéndice 6, la función del sistema operativo `Memory.alloc` (*tamaño*) sabe cómo encontrar un bloque de RAM disponible de un *tamaño determinado* (número de palabras de 16 bits) y devolver la dirección base del bloque.

`Memory.alloc` y su función hermana `Memory.deAlloc` usan algoritmos inteligentes para asignar y liberar recursos de RAM de manera eficiente. Estos algoritmos se presentarán e implementarán en el capítulo 12, cuando construiremos el sistema operativo. Por ahora, basta con decir que los compiladores generan código de bajo nivel que usa `alloc` (en constructores) y `deAlloc` (en destructores), de forma abstracta.

Antes de llamar a `Memory.alloc`, el compilador determina el tamaño del bloque de memoria necesario. Esto se puede calcular fácilmente a partir de la tabla de símbolos de nivel de clase. Por ejemplo, la tabla de símbolos de la clase `Point` especifica que cada objeto `Point` se caracteriza por dos valores `int` (las coordenadas  $x$  e  $y$  del punto). Por lo tanto, el compilador genera los comandos `push constant 2` y llama a `Memory.alloc 1`, efectuando la llamada de función `Memory.alloc(2)`. La asignación de funciones del sistema operativo se pone a trabajar, encuentra un bloque de RAM disponible de tamaño 2 e inserta su dirección base en la pila, el equivalente de máquina virtual a devolver un valor. La siguiente instrucción de máquina virtual generada, el puntero `pop 0`, establece `THIS` en la dirección base devuelta por `alloc`. A partir de este momento, el segmento `this` del constructor se alineará con el bloque RAM que se asignó para representar el objeto recién construido.

Una vez que este segmento esté correctamente alineado, podemos proceder a generar código fácilmente. Por ejemplo, cuando se llama a la rutina `compileLet` para manejar la instrucción `let x = ax`, busca en las tablas de símbolos, resolviendo  $x$  en este 0 y  $ax$  en el argumento 0. Por lo tanto, `compileLet` genera los comandos `push argumento 0`, seguido de `pop this 0`. El último comando se basa en la suposición de que este segmento está correctamente alineado con la dirección base del nuevo objeto, como de hecho se hizo cuando establecimos el puntero 0 (en realidad, `THIS`) a la dirección base devuelta por `alloc`. Esta inicialización única asegura que todos los comandos posteriores `push / pop this i` terminarán golpeando los objetivos correctos en la RAM (más exactamente, en el *montón*). Esperamos que la intrincada belleza de este contrato no se pierda en el lector.

De acuerdo con la especificación del lenguaje Jack, cada constructor debe terminar con una declaración `return this`. Esta convención obliga al compilador a finalizar la versión compilada del constructor con el puntero `push 0` y `return`. Estos comandos insertan en la pila el valor de `THIS`, la dirección base del objeto construido. En algunos lenguajes, como Java, los constructores no tienen que terminar con una declaración de retorno explícita. No obstante, el código compilado de los constructores de Java realiza exactamente la misma acción a nivel de VM, ya que

Eso es lo que se espera que hagan los constructores: crear un objeto y devolver su identificador al autor de la llamada.

Recuerde que el elaborado drama de bajo nivel que acabamos de describir fue desatado por la instrucción del lado de la llamada `let varName = className.constructorName (...)`. Ahora vemos que, por diseño, cuando el constructor termina, *varName* termina almacenando la dirección base del nuevo objeto. Cuando decimos "por diseño", nos referimos a la sintaxis del lenguaje de construcción de objetos de alto nivel y al arduo trabajo que el compilador, el sistema operativo, el traductor de VM y el ensamblador tienen que hacer para realizar esta abstracción. El resultado neto es que los programadores de alto nivel se salvan de todos los detalles sangrientos de la construcción de objetos y pueden crear objetos de manera fácil y transparente.

### 11.1.5.2 Métodos de compilación

Como hicimos con los constructores, describiremos cómo compilar las llamadas a los métodos y luego cómo compilar los propios métodos.

**Llamadas a métodos de compilación:** Supongamos que deseamos calcular la distancia euclidiana entre dos puntos en un plano, *p1* y *p2*. En la programación de procedimientos de estilo C, esto podría haberse implementado utilizando una llamada de función como `distance(p1,p2)`, donde *p1* y *p2* representan tipos de datos compuestos. Sin embargo, en la programación orientada a objetos, *p1* y *p2* se implementarán como instancias de alguna clase *Point*, y el mismo cálculo se realizará utilizando una llamada a un método como `p1.distance(p2)`. A diferencia de las funciones, *los métodos* son subrutinas que siempre operan en un objeto determinado, y es responsabilidad del autor de la llamada especificar este objeto. (El hecho de que el `distance` toma otro objeto *Point* como argumento es una coincidencia. En general, aunque un método siempre está diseñado para operar en un objeto, el método puede tener 0, 1 o más argumentos, de cualquier tipo).

Obsérvese que la distancia puede describirse como un *procedimiento* para calcular la distancia de un punto dado a otro, y *p1* puede describirse como los *datos* sobre los que opera el procedimiento. Además, tenga en cuenta que tanto los modismos `distance(p1,p2)` como `p1.distance(p2)` están diseñados para calcular y devolver el mismo valor. Sin embargo, mientras que la sintaxis de estilo C se centra en la distancia, en la sintaxis orientada a objetos, el objeto es lo primero, literalmente hablando. Es por eso que los lenguajes similares a C a veces se denominan lenguajes *procedimentales* y orientados a objetos

se dice que se *basan en datos*. Entre otras cosas, el estilo de programación orientado a objetos se basa en la suposición de que los objetos saben cómo cuidarse a sí mismos. Por ejemplo, un objeto Point sabe cómo calcular la distancia entre él y otro objeto Point. Dicho de otra manera, la operación de distancia está *encapsulada* dentro de la definición de ser un Punto.

El agente responsable de traer todas estas abstracciones sofisticadas a la tierra es, como de costumbre, el compilador trabajador. Dado que el lenguaje de máquina virtual de destino no tiene ningún concepto de objetos o métodos, el compilador controla las llamadas a métodos orientados a objetos como `p1.distance (p2)` como si fueran llamadas de procedimiento como `distance (p1,p2)`. Específicamente, traduce `p1.distance (p2)` en `push p1, push p2, call distance`. Generalicemos: Jack presenta dos tipos de llamadas a métodos:

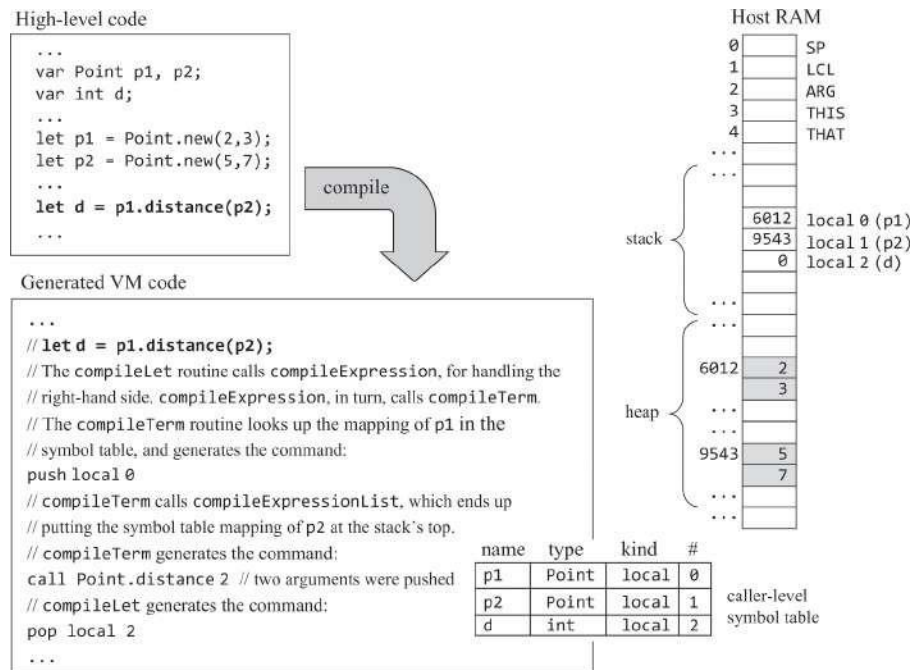
```
// Applies a method to the object referred to by varName:
```

```
varName.methodName(exp1, exp2, ..., expn)
```

```
// Applies a method to the current object:
```

```
methodName(exp1, exp2, ..., expn)      // Same as this.methodName(exp1, exp2, ..., expn)
```

Para compilar el método, llame a `varName.methodName(exp1, exp2, ..., expn)`, comenzamos generando el comando `push varName`, donde `varName` es el mapeo de la tabla de símbolos de `varName`. Si la llamada al método no menciona `varName`, insertamos el mapeo de la tabla de símbolos de `this`. A continuación, llamamos a `compileExpressionList`. Esta rutina llama a `compileExpression`  $n$  veces, una vez para cada expresión entre paréntesis. Finalmente, generamos el comando `call className.methodName`  $n+1$ , informando que  $n+1$  los argumentos se insertaron en la pila. El caso especial de llamar a un método sin argumentos se traduce en la llamada `className.methodName` 1. Tenga en cuenta que `className` es el tipo de tabla de símbolos del identificador `varName`. Consulte [la figura 11.9](#) para ver un ejemplo.



**Figura 11.9** Compilación de llamadas a métodos: la perspectiva de la persona que llama.

**Métodos de compilación:** Hasta ahora discutimos el método de distancia de forma abstracta, desde la perspectiva de la persona que llama. Considere cómo se podría implementar este método, por ejemplo, en Java:

```

/** A Point class method: returns the distance between this Point and the other one. */
int distance(Point other) {
    int dx, dy;
    dx = x - other.x;
    dy = y - other.x;
    return Math.sqrt((dx*dx) + (dy*dy));
}

```

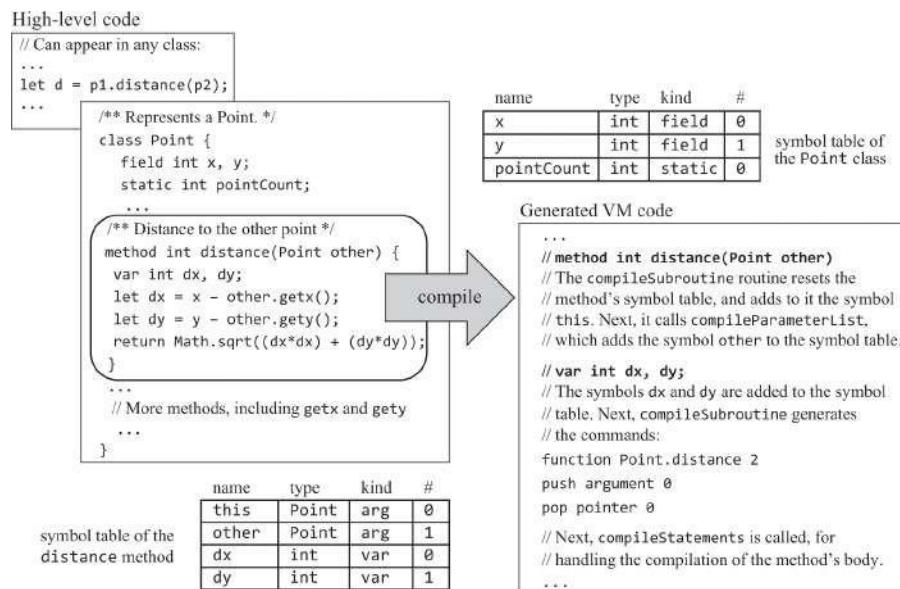
Como cualquier método, la distancia está diseñada para operar en el *objeto actual*, representado en Java (y en Jack) por el identificador incorporado `this`. Sin embargo, como ilustra el ejemplo anterior, uno puede escribir un método completo sin mencionar esto. Esto se debe a que el compilador amigable de Java maneja las declaraciones como `dx=x-other.x` si fueran `dx=this.x-other.x`. Esta convención hace que el código de alto nivel sea más legible y fácil de escribir.



Sin embargo, señalamos de pasada que en el idioma Jack, el idioma *objeto.no* es compatible. Por lo tanto, los campos de objetos distintos del objeto actual solo se pueden manipular mediante métodos de descriptor de acceso y mutador. Por ejemplo, expresiones como  $x - \text{other.x}$  se implementan en Jack como  $x - \text{other.getx}()$ , donde `getx` es un método de acceso en la clase `Point`.

Entonces, ¿cómo maneja el compilador Jack expresiones como  $x - \text{other.getx}()$ ? Al igual que el compilador de Java, busca  $x$  en las tablas de símbolos y encuentra que representa el primer campo en el objeto actual. Pero, ¿qué objeto en el grupo de tantos objetos representa el objeto actual? Bueno, de acuerdo con el contrato de llamada al método, debe ser el primer argumento que pasó el llamador del método. Por lo tanto, desde la perspectiva del destinatario de la llamada, el objeto actual debe ser el objeto cuya dirección base es el valor del argumento

0. Esto, en pocas palabras, es el truco de compilación de bajo nivel que hace posible la abstracción ubicua "aplicar un método a un objeto" en lenguajes como Java, Python y, por supuesto, Jack. Consulte la figura 11.10 para obtener más detalles.



**Figura 11.10** Métodos de compilación: la perspectiva del destinatario.

El ejemplo comienza en la parte superior izquierda de la figura 11.10, donde el código del autor de la llamada hace que el método llame a `p1.distance(p2)`. Dirigiendo nuestra atención a la versión compilada del destinatario de la llamada, observe que el código propiamente dicho comienza con el argumento `push 0`, seguido del puntero `pop 0`. Estos comandos

establecen el puntero `THIS` del método en

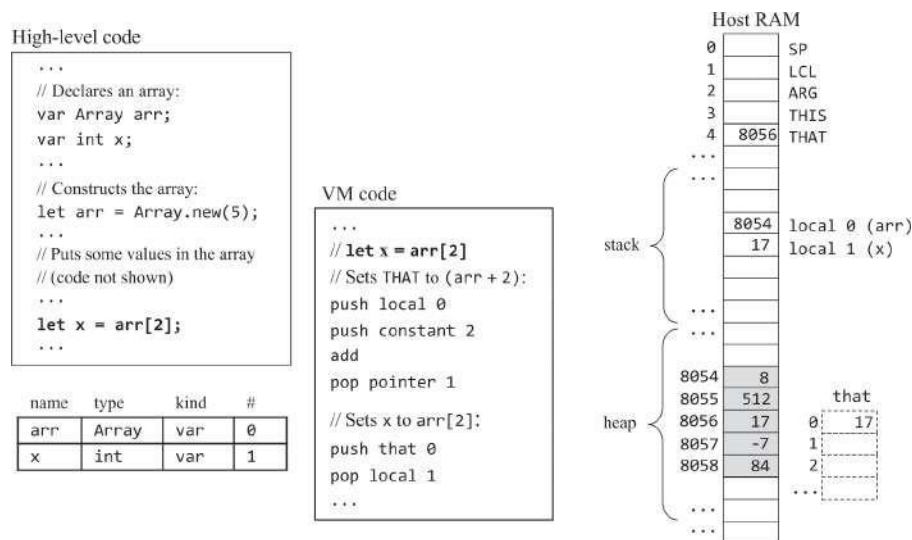


el valor del argumento 0, que, en virtud del método que llama al contrato, contiene la dirección base del objeto en el que se llamó al método para operar. Por lo tanto, a partir de este punto, este segmento del método está correctamente alineado con la dirección base del objeto de destino, lo que hace que cada comando `push / pop this i` también esté correctamente alineado. Por ejemplo, la expresión `x ← other.getx ()` se compilará en `push this 0, push argument 1, call Point.getx 1, sub`. Dado que comenzamos el código del método compilado estableciendo `THIS` en la dirección base del objeto llamado, tenemos la garantía de que este 0 (y cualquier otra referencia de este *i*) dará en el blanco, apuntando al campo derecho del objeto correcto.

### 11.1.6 Compilación de matrices

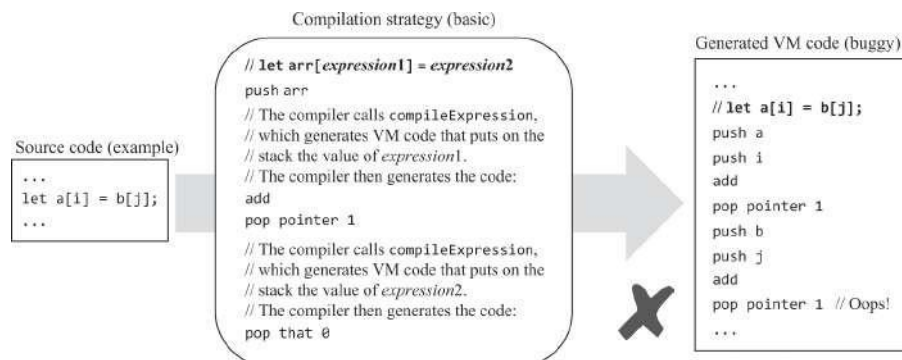
Las matrices son similares a los objetos. En Jack, las matrices se implementan como instancias de una clase `Array`, que forma parte del sistema operativo. Por lo tanto, las matrices y los objetos se declaran, implementan y almacenan exactamente de la misma manera; De hecho, las matrices *son* objetos, con la diferencia de que la abstracción de matrices permite acceder a los elementos de la matriz mediante un índice, por ejemplo, `let arr[3] = 17`. El agente que concreta esta útil abstracción es el compilador, como ahora pasamos a describir.

Usando la notación de puntero, observe que `arr[i]` se puede escribir como `*(arr + i)`, es, dirección de memoria `arr + i`. Esta información tiene la clave para compilar declaraciones como `let x = arr[i]`. Para calcular la dirección física de `arr[i]`, ejecutamos `push arr, push i, add`, lo que da como resultado empujar la dirección de destino en la pila. A continuación, ejecutamos el puntero `pop 1`. De acuerdo con la especificación de VM, esta acción almacena la dirección de destino en el puntero `THAT` del método (`RAM[4]`), lo que tiene el efecto de alinear la dirección base del segmento virtual `that` con la dirección de destino. Por lo tanto, ahora podemos ejecutar `push that 0` y `pop x`, completando la traslación de bajo nivel de `let x = arr[i]`. la [figura 11.11](#) para obtener más detalles.



**Figura 11.11** Acceso a matrices mediante comandos de VM.

Esta bonita estrategia de compilación solo tiene un problema: no funciona. Más exactamente, funciona con declaraciones como `let a=b[j]` pero falla con declaraciones en las que se indexa el lado izquierdo de la tarea, como en `let a[i]=b[j]`. Ver [figura 11.12](#).



**Figura 11.12** Estrategia básica de compilación para arrays, y un ejemplo de los bugs que puede generar. En este caso particular, el valor almacenado en puntero 1 se anula, y la dirección de `a[i]` se pierde.

La buena noticia es que esta estrategia de compilación defectuosa se puede arreglar fácilmente para compilar correctamente cualquier instancia de `let arr[expression1] = expression2`. Como antes, comenzamos generando el comando `push arr`, llamando a `compileExpression` y generando el comando `add`. Esta secuencia coloca la dirección de destino (`arr + expression1`) en la parte superior de la pila. A continuación, llamamos a

compileExpression, que terminará poniendo en la parte superior de la pila el valor de *expression2*. En este punto, guardamos este valor, podemos hacerlo usando `pop temp 0`. Esta operación tiene el agradable efecto secundario de hacer  $(arr + expression1)$  el elemento de pila superior. Por lo tanto, ahora podemos hacer estallar el puntero 1, empujar la temperatura 0 y hacer estallar ese 0. Esta pequeña corrección, junto con la naturaleza recursiva de la rutina compileExpression, hace que esta estrategia de compilación sea capaz de controlar `let arr[expression1] = expression2` instrucciones de cualquier complejidad recursiva, como, por ejemplo, `let a[b[i]+a[j+b[a[3]]]]=b[b[j]+2]`.

Para terminar, varias cosas hacen que la compilación de matrices de Jack sea relativamente simple. Primero, las matrices de Jack no están tipificadas; más bien, están diseñados para almacenar valores de 16 bits, sin restricciones. En segundo lugar, todos los tipos de datos primitivos en Jack tienen 16 bits de ancho, todas las direcciones tienen 16 bits de ancho y también lo es el ancho de palabra de la RAM. En lenguajes de programación fuertemente tipados, y en lenguajes donde no se puede garantizar esta correspondencia uno a uno, la compilación de matrices requiere más trabajo.

---

## 11.2 Especificación

Los desafíos y soluciones de compilación que hemos descrito hasta ahora se pueden generalizar para admitir la compilación de cualquier lenguaje de programación basado en objetos. Ahora pasamos de lo general a lo específico: desde aquí hasta el final del capítulo, describimos el *compilador Jack*. El compilador Jack es un programa que obtiene un programa Jack como entrada y genera código VM ejecutable como salida. El código de VM realiza la semántica del programa en la máquina virtual especificada en los capítulos 7 y 8.

**Uso:** El compilador acepta un único argumento de línea de comandos, como se indica a continuación:

```
prompt> JackCompiler fuentes
```

donde *source* es un nombre de archivo de la forma `xxx.jack` (la extensión es obligatoria) o el nombre de una carpeta (en cuyo caso no hay extensión) que contiene uno o más archivos `.jack`. El nombre del archivo o carpeta puede contener una ruta de acceso de archivo. Si no se especifica ninguna ruta de

acceso, el compilador funciona en la carpeta actual. Para cada *archivo* *Xxx.jack*, el compilador crea un archivo de salida *Xxx.vm* y escribe el archivo

VM en él. El archivo de salida se crea en la misma carpeta que el archivo de entrada. Si hay un archivo con este nombre en la carpeta, se sobrescribirá.

---

## 11.3 Implementación

Ahora pasamos a proporcionar pautas, consejos de implementación y una API propuesta para extender el analizador de sintaxis integrado en el capítulo 10 en un compilador Jack a gran escala.

### 11.3.1 Asignación estándar a través de la máquina virtual

Los compiladores Jack se pueden desarrollar para diferentes plataformas de destino. Esta sección proporciona pautas sobre cómo mapear varias construcciones del lenguaje Jack en una plataforma específica: la máquina virtual especificada en los capítulos 7 y 8.

#### Nombrar archivos y funciones

- Un archivo de clase Jack `Xxx.jack` se compila en un archivo de clase VM denominado `Xxx.vm`
- Una subrutina de Jack `yyy` en el archivo `Xxx.jack` *se compila en una función de VM llamada* `xx.yyy`

#### Mapeo de variables

- El primero, segundo, tercero, ... *static* declarada en una declaración de clase se asigna a la entrada del segmento virtual `static 0`, `static 1`, `static 2`, ...
- El primero, segundo, tercero, ... *field* declarada en una declaración de clase se asigna a `este 0`, `este 1`, `este 2`, ...
- El primero, segundo, tercero, ... *La* variable local declarada en las sentencias `var` de una subrutina se asigna en `local 0`, `local 1`, `local 2`, ...
- El primero, segundo, tercero, ... *argument* declarada en la lista de parámetros de una *función* o un *constructor* (pero no un *método*) se asigna en el argumento 0, el argumento 1, el argumento 2, ...
- El primero, segundo, tercero, ... *argument* declarada en la lista de parámetros de un *método* se asigna en el argumento 1, argumento 2, argumento 3, ...

## Asignación de campos de objetos

Para alinear el segmento virtual `this` con el objeto pasado por el llamador de un  
, use los comandos de VM `push argumento 0`, `puntero pop 0`.

## Asignación de elementos de matriz

La referencia de alto nivel `arr[expression]` se compila estableciendo el puntero 1 en (`arr`  
+ *expresión*) y accediendo a ese 0.

## Asignación de constantes

- Las referencias a las constantes de Jack `null` y `false` se compilan en la constante `push 0`.
- Las referencias a la constante de Jack verdadera se compilan en la constante de empuje 1, neg.  
Esta secuencia empuja el valor  $-1$  a la pila.
- Las referencias a la constante de Jack `this` se compilan en el puntero de inserción 0. Este comando inserta la dirección base del objeto actual en la pila.

### 11.3.2 Directrices de aplicación

A lo largo de este capítulo hemos visto muchos ejemplos de compilaciones conceptuales. A continuación hacemos un resumen conciso y formal de todas estas técnicas de recopilación.

## Manejo de identificadores

Los identificadores utilizados para nombrar variables se pueden manejar mediante tablas de símbolos. Durante la compilación de código Jack válido, cualquier identificador no encontrado poner que es un nombre de subrutina o nombre de

Dado que las reglas de sintaxis de Jack son suficientes para distinguir entre estas dos posibilidades, y dado que el compilador de Jack no realiza ningún "enlace", no hay necesidad de mantener estos identificadores en una tabla de símbolos.

## Compilación de expresiones

La rutina `compileExpression` debe procesar la entrada como el término de secuencia *op term op term ....* Para hacerlo, `compileExpression` debe implementar el algoritmo `codeWrite` (figura 11.4), extendido para manejar todos los *términos posibles* especificados en la gramática Jack (figura 11.5). De hecho, una inspección de las reglas gramaticales revela que la mayor parte de la acción en la compilación *de expresiones* ocurre en la compilación de sus *términos subyacentes*. Esto es especialmente cierto después de nuestra recomendación de que la compilación de llamadas a subrutinas sea manejada directamente por la compilación de *términos* (notas de implementación después de la API `CompilationEngine`, sección 10.3).

La *gramática* de la expresión y, por lo tanto, la rutina `compileExpression` correspondiente son inherentemente recursivas. Por ejemplo, cuando `compileExpression` detecta un paréntesis izquierdo, debe llamar recursivamente a `compileExpression` para controlar la expresión interna. Este descenso recursivo asegura que la expresión interna se evaluará primero. Excepto por esta regla de prioridad, el lenguaje Jack no admite *ninguna prioridad de operador*. Por supuesto, es posible manejar la prioridad del operador, pero en Nand to Tetris lo consideramos una extensión opcional específica del compilador, no una característica estándar del lenguaje Jack.

La expresión  $x * y$  se compila en `push x, push y, call Math.multiply 2`. La expresión  $x / y$  se compila en `push x, push y, call Math.divide 2`. La clase `Math` es parte del sistema operativo, documentado en el apéndice 6. Esta clase se desarrollará en el capítulo 12.

## Compilación de cadenas

Cada constante de cadena "*ccc ... c*" se controla mediante (i) empujar la longitud de la cadena en la pila y llamar al constructor `String.new`, y (ii) empujar el código de caracteres de *c* en la pila y llamar al método `String.appendChar`, una vez por cada carácter *c* en la cadena (el juego de caracteres Jack se documenta en el apéndice 5). Como se documenta en la API de la clase `String` en el apéndice 6, tanto el nuevo constructor como el método `appendChar` devuelven la cadena como valor devuelto (es decir, insertan el objeto de cadena en la pila). Esto simplifica la compilación, evitando la necesidad de volver a insertar la cadena cada vez que se llama a `appendChar`.

## Compilación de llamadas a funciones y llamadas a constructores

La versión compilada de llamar a una función o llamar a un constructor que tiene  $n$  argumentos debe (i) llamar a `compileExpressionList`, que llamará a `compileExpression`  $n$  veces, y (ii) realizar la llamada informando que  $n$  argumentos se insertaron en la pila antes de la llamada.

### Compilación de llamadas a métodos

La versión compilada de llamar a un método que tiene  $n$  argumentos debe (i) insertar una referencia al objeto en el que se llama al método para operar, (ii) llamar a `compileExpressionList`, que llamará a `compileExpression`  $n$  veces, y (iii) realizar la llamada, informando que  $n+1$  los argumentos se insertaron en la pila antes de la llamada.

### Compilación de declaraciones `do`

Se recomienda compilar las instrucciones `do subroutineCall` como si fueran instrucciones de *expresión do* y, a continuación, extraer el valor de pila superior mediante `pop temp 0`.

### Compilación de clases

Al comenzar a compilar una clase, el compilador crea una tabla de símbolos de nivel de clase y le agrega todas las *variables estáticas y de campo* declaradas en la declaración de clase. El compilador también crea una tabla de símbolos vacía a nivel de subrutina. No se genera ningún código.

### Compilación de subrutinas

- Al comenzar a compilar una subrutina (*constructor, función o método*), el compilador inicializa la tabla de símbolos de la subrutina. Si la subrutina es un *método*, el compilador agrega a la tabla de símbolos la asignación `<this, className, arg, 0>`.
- A continuación, el compilador agrega a la tabla de símbolos todos los parámetros, si los hay, declarados en la lista de parámetros de la subrutina. A continuación, el compilador maneja todas las declaraciones `var`, si las hay, agregando a la tabla de símbolos todas las variables locales de la subrutina.



- En esta etapa, el compilador comienza a generar código, comenzando con la función de comando *className.subroutineName nVars*, donde *nVars* es el número de variables locales en la subrutina.
- Si la subrutina es un *método*, el compilador genera el argumento de inserción de código 0, puntero pop 0. Esta secuencia alinea el segmento de memoria virtual con la dirección base del objeto en el que se llamó al método.

## Compilación de constructores

- Primero, el compilador realiza todas las acciones descritas en la sección anterior, finalizando con la generación de la función de comando *className.nVars de constructorName*.
- A continuación, el compilador genera la constante de inserción *de código nFields*, llama a *Memory.alloc 1*, puntero emergente 0, donde *nFields* es el número de campos de la clase compilada. Esto da como resultado la asignación de un bloque de memoria de *palabras nFields* de 16 bits y la alineación del segmento de memoria virtual con la dirección base del bloque recién asignado.
- El constructor compilado debe terminar con el puntero push 0, return. Esta secuencia devuelve al autor de la llamada la dirección base del nuevo objeto creado por el constructor.

## Compilación de métodos void y funciones void

Se espera que cada función de VM inserte un valor en la pila antes de devolver. Al compilar un método o función void Jack, la convención es finalizar el código generado con la constante push 0, return.

## Compilación de matrices

Las sentencias de la forma *let arr[expression1] = expression2* se compilan utilizando la técnica descrita al final de la sección 11.1.6. *Consejo de implementación:* Al manejar matrices, nunca es necesario usar las entradas cuyo índice es mayor que 0.

## El sistema operativo

Considere la expresión de alto nivel `Math.sqrt((dx * dx)+(dy * dy))`. El compilador lo compila en los comandos de máquina virtual `push dx`, `push dx`, `call Math.multiply 2`, `push dy`, `push dy`, `call Math.multiply 2`, `add`, `call Math.sqrt 1`, donde `dx` y `dy` son las asignaciones de tabla de símbolos de `dx` y `dy`. En este ejemplo se ilustran las dos formas en que los servicios del sistema operativo entran en juego durante la compilación. Primero, algunas abstracciones de alto nivel, como la expresión `x * y`, se compilan generando código que llama a subrutinas del sistema operativo como `Math.multiply`. En segundo lugar, cuando una expresión Jack incluye una llamada de alto nivel a una rutina del sistema operativo, por ejemplo, `Math.sqrt(x)`, el compilador genera código de máquina virtual que realiza exactamente la misma llamada mediante la sintaxis de sufijo de máquina virtual.

El sistema operativo presenta ocho clases, documentadas en el apéndice 6. Nand to Tetris proporciona dos implementaciones diferentes de este sistema operativo: *nativa* y *emulada*.

## Implementación nativa del sistema operativo

En el proyecto 12 desarrollará la biblioteca de clases del sistema operativo en Jack y la compilará usando un compilador Jack. La compilación producirá ocho archivos `.vm`, que comprenden la implementación nativa del sistema operativo. Si coloca estos ocho archivos `.vm` en la misma carpeta que almacena los archivos `.vm` resultantes de la compilación de *cualquier* programa Jack, todas las funciones del sistema operativo serán accesibles para el código de VM compilado, ya que pertenecen a la misma base de código.

## Implementación del sistema operativo emulado

El emulador de VM suministrado, que es un programa Java, presenta una implementación basada en Java del sistema operativo Jack. Cada vez que el código de VM cargado en el emulador llama a una función del sistema operativo, el emulador comprueba si existe una función de VM con ese nombre en la base de código cargada. Si es así, ejecuta la función VM. De lo contrario, llama a la implementación integrada de esta función del sistema operativo. La conclusión es la siguiente: si usa el emulador de VM suministrado para ejecutar el código de VM generado por su compilador, como lo hacemos en el proyecto 11, no necesita preocuparse por la configuración del sistema operativo; el emulador atenderá todas las llamadas del sistema operativo sin más preámbulos.

### **11.3.3 Arquitectura de software**

La arquitectura del compilador propuesta se basa en el analizador de sintaxis descrito en el capítulo 10. Específicamente, proponemos evolucionar gradualmente el analizador de sintaxis en un compilador a gran escala, utilizando los siguientes módulos:

- JackCompiler: programa principal, configura e invoca los otros módulos
- JackTokenizer: tokenizador para el lenguaje Jack
- SymbolTable: realiza un seguimiento de todas las variables que se encuentran en el código Jack
- VMWriter: escribe código de máquina virtual
- CompilationEngine: motor de compilación recursivo de arriba hacia abajo

## El compilador JackCompiler

Este módulo impulsa el proceso de compilación. Funciona con un nombre de archivo de la forma *Xxx.jack* o con un nombre de carpeta que contiene uno o más de estos archivos. Para cada *archivo Xxx.jack* de origen, el programa

1. crea un JackTokenizer a partir del *archivo de entrada Xxx.jack*;
2. crea un archivo de salida denominado *Xxx.vm*; y
3. usa un CompilationEngine, un SymbolTable y un VMWriter para analizar el archivo de entrada y emitir el código de máquina virtual traducido en el archivo de salida.

No proporcionamos ninguna API para este módulo, invitándolo a implementarlo como mejor le parezca. Recuerde que la primera rutina a la que se debe llamar al compilar un *.jack* es `compileClass`.

## El JackTokenizer

Este módulo es idéntico al tokenizador integrado en el proyecto 10. Consulte la API en la sección 10.3.

## La tabla de símbolos

Este módulo proporciona servicios para crear, rellenar y usar tablas de símbolos que realizan un seguimiento del *nombre*, el *tipo*, el *tipo* y un índice en ejecución *de las propiedades del símbolo* para cada tipo. Consulte [la figura 11.2](#) para ver un ejemplo.

| <i><b>Routine</b></i>     | <i><b>Arguments</b></i>                                             | <i><b>Returns</b></i>           | <i><b>Function</b></i>                                                                                                                             |
|---------------------------|---------------------------------------------------------------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Constructor / initializer | —                                                                   | —                               | Creates a new symbol table.                                                                                                                        |
| reset                     | —                                                                   | —                               | Empties the symbol table, and resets the four indexes to 0. Should be called when starting to compile a subroutine declaration.                    |
| define                    | name (string)<br>type (string)<br>kind (STATIC, FIELD, ARG, or VAR) | —                               | Defines (adds to the table) a new variable of the given name, type, and kind. Assigns to it the index value of that kind, and adds 1 to the index. |
| varCount                  | kind (STATIC, FIELD, ARG, or VAR)                                   | int                             | Returns the number of variables of the given kind already defined in the table.                                                                    |
| kindOf                    | name (string)                                                       | (STATIC, FIELD, ARG, VAR, NONE) | Returns the kind of the named identifier. If the identifier is not found, returns NONE.                                                            |
| typeOf                    | name (string)                                                       | string                          | Returns the type of the named variable.                                                                                                            |
| indexOf                   | name (string)                                                       | int                             | Returns the index of the named variable.                                                                                                           |

*Nota de implementación:* Durante la compilación de un archivo de clase Jack, el compilador Jack usa dos instancias de SymbolTable.

## El VMWriter

Este módulo presenta un conjunto de rutinas simples para escribir comandos de VM en el archivo de salida.

| <i>Routine</i>            | <i>Arguments</i>                                                                      | <i>Returns</i> | <i>Function</i>                                                      |
|---------------------------|---------------------------------------------------------------------------------------|----------------|----------------------------------------------------------------------|
| Constructor / initializer | Output file / stream                                                                  | —              | Creates a new output .vm file / stream, and prepares it for writing. |
| writePush                 | segment (CONSTANT, ARGUMENT, LOCAL, STATIC, THIS, THAT, POINTER, TEMP)<br>index (int) | —              | Writes a VM push command.                                            |
| writePop                  | segment ( ARGUMENT, LOCAL, STATIC, THIS, THAT, POINTER, TEMP)<br>index (int)          | —              | Writes a VM pop command.                                             |
| writeArithmetic           | command (ADD, SUB, NEG, EQ, GT, LT, AND, OR, NOT)                                     | —              | Writes a VM arithmetic-logical command.                              |
| writeLabel                | label (string)                                                                        | —              | Writes a VM label command.                                           |
| writeGoto                 | label (string)                                                                        | —              | Writes a VM goto command.                                            |
| writeIf                   | label (string)                                                                        | —              | Writes a VM if-goto command.                                         |
| writeCall                 | name (string)<br>nArgs (int)                                                          | —              | Writes a VM call command.                                            |
| writeFunction             | name (string)<br>nVars (int)                                                          | —              | Writes a VM function command.                                        |
| writeReturn               | —                                                                                     | —              | Writes a VM return command.                                          |
| close                     | —                                                                                     | —              | Closes the output file / stream.                                     |

## El motor de compilación

Este módulo ejecuta el proceso de compilación. Aunque la API de CompilationEngine es casi idéntica a la API presentada en el capítulo 10, la repetimos aquí para facilitar la referencia.

CompilationEngine obtiene su entrada de un JackTokenizer y usa un VMWriter para escribir la salida del código de la máquina virtual (en lugar del XML generado en el proyecto 10). La salida es generada por una serie de rutinas compilexxx, cada una diseñada para manejar la compilación de una construcción específica del lenguaje Jack xxx (por ejemplo, compileWhile genera el código de VM que realiza las declaraciones while). El contrato entre estas rutinas es el siguiente: Cada

`compilexxx` obtiene de la entrada y controla todos los tokens que componen `xxx`, avanza el tokenizador exactamente más allá de estos tokens y emite al código de máquina virtual de salida que afecta la semántica de `xxx`. Si `xxx` forma parte de una expresión y, por lo tanto, tiene un valor, el código de máquina virtual emitido debe calcular este valor y dejarlo en la parte superior de la pila. Como regla general, cada rutina `compilexxx` se llama solo si el token actual es `xxx`. Dado que el primer token de un archivo `.jack` válido debe ser la clase de palabra clave, el proceso de compilación comienza llamando a la rutina `compileClass`.

| <i>Routine</i>                     | <i>Arguments</i>                                      | <i>Returns</i> | <i>Function</i>                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------|-------------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Constructor / initializer          | Input<br>file / stream<br><br>Output<br>file / stream | —              | Creates a new compilation engine with the given input and output.<br><br>The next routine called must be <code>compileClass</code> .                                                                                                                                                                                                                                                                           |
| <code>compileClass</code>          | —                                                     | —              | Compiles a complete class.                                                                                                                                                                                                                                                                                                                                                                                     |
| <code>compileClassVarDec</code>    | —                                                     | —              | Compiles a static variable declaration, or a field declaration.                                                                                                                                                                                                                                                                                                                                                |
| <code>compileSubroutine</code>     | —                                                     | —              | Compiles a complete method, function, or constructor.                                                                                                                                                                                                                                                                                                                                                          |
| <code>compileParameterList</code>  | —                                                     | —              | Compiles a (possibly empty) parameter list. Does not handle the enclosing parenthesis tokens <code>(</code> and <code>)</code> .                                                                                                                                                                                                                                                                               |
| <code>compileSubroutineBody</code> | —                                                     | —              | Compiles a subroutine's body.                                                                                                                                                                                                                                                                                                                                                                                  |
| <code>compileVarDec</code>         | —                                                     | —              | Compiles a <code>var</code> declaration.                                                                                                                                                                                                                                                                                                                                                                       |
| <code>compileStatements</code>     | —                                                     | —              | Compiles a sequence of statements. Does not handle the enclosing curly bracket tokens <code>{</code> and <code>}</code> .                                                                                                                                                                                                                                                                                      |
| <code>compileLet</code>            | —                                                     | —              | Compiles a <code>let</code> statement.                                                                                                                                                                                                                                                                                                                                                                         |
| <code>compileIf</code>             | —                                                     | —              | Compiles an <code>if</code> statement, possibly with a trailing <code>else</code> clause.                                                                                                                                                                                                                                                                                                                      |
| <code>compileWhile</code>          | —                                                     | —              | Compiles a <code>while</code> statement.                                                                                                                                                                                                                                                                                                                                                                       |
| <code>compileDo</code>             | —                                                     | —              | Compiles a <code>do</code> statement.                                                                                                                                                                                                                                                                                                                                                                          |
| <code>compileReturn</code>         | —                                                     | —              | Compiles a <code>return</code> statement.                                                                                                                                                                                                                                                                                                                                                                      |
| <code>compileExpression</code>     | —                                                     | —              | Compiles an expression.                                                                                                                                                                                                                                                                                                                                                                                        |
| <code>compileTerm</code>           | —                                                     | —              | Compiles a <i>term</i> . If the current token is an <i>identifier</i> , the routine must resolve it into a <i>variable</i> , an <i>array element</i> , or a <i>subroutine call</i> . A single lookahead token, which may be <code>[</code> , <code>(</code> , or <code>.</code> , suffices to distinguish between the possibilities. Any other token is not part of this term and should not be advanced over. |
| <code>compileExpressionList</code> | —                                                     | int            | Compiles a (possibly empty) comma-separated list of expressions. Returns the number of expressions in the list.                                                                                                                                                                                                                                                                                                |



**Nota:** Las siguientes reglas gramaticales de Jack no tienen rutinas xxx de compilación correspondientes en `CompilationEngine`: *type*, *className*, *subroutineName*, *varName*, *statement*, *subroutineCall*.

La lógica de análisis de estas reglas debe ser manejada por las rutinas que implementan las reglas que hacen referencia a ellas. La gramática del lenguaje Jack se presenta en la sección 10.2.1.

**Búsqueda anticipada de tokens:** La necesidad de una búsqueda anticipada de tokens y la solución propuesta para manejarla se analizan en la sección 10.3, justo después de la API `CompilationEngine`.

---

## 11.4 Proyecto

**Objetivo:** Extender el analizador de sintaxis integrado en el capítulo 10 a un compilador Jack a gran escala. Aplique el compilador a todos los programas de prueba que se describen a continuación. Ejecute cada programa traducido y asegúrese de que funcione de acuerdo con la documentación proporcionada.

Esta versión del compilador asume que el código fuente de Jack está libre de errores. La comprobación de errores, la generación de informes y el control se pueden agregar a versiones posteriores del compilador, pero no forman parte del proyecto 11.

**Recursos:** La principal herramienta que necesita es el lenguaje de programación en el que implementará el compilador. También necesitará el emulador de máquina virtual suministrado para probar el código de máquina virtual generado por el compilador. Dado que el compilador se implementa extendiendo el analizador de sintaxis integrado en el proyecto 10, también necesitará el código fuente del analizador.

### Etapas de implementación

Proponemos transformar el analizador de sintaxis integrado en el proyecto 10 en el compilador final. En particular, proponemos reemplazar gradualmente las rutinas que generan salida XML pasiva con rutinas que generan código VM ejecutable. Esto se puede hacer en dos etapas principales de desarrollo.

(Fase 0: Realice una copia de seguridad del código del analizador de sintaxis desarrollado en el proyecto 10).

**Fase 1: Tabla de símbolos:** comience por compilar el módulo SymbolTable del compilador y úselo para ampliar el analizador de sintaxis integrado en Project 10, como se indica a continuación. Actualmente, cada vez que se encuentra un identificador en el código fuente, digamos foo, el analizador de sintaxis genera la línea XML <identificador> foo </identificador>. En su lugar, extienda el analizador de sintaxis para generar la siguiente información sobre cada identificador:

- *nombre*
- *Categoría* (campo, estático, var, arg, clase, subrutina)
- *Index*: si la categoría del identificador es field, static, var o arg, el índice continuo asignado al identificador por la tabla de símbolos
- *uso*: si el identificador se está *declarando actualmente* (por ejemplo, el identificador aparece en una declaración de variable estática / field / var Jack) o *se usa* (por ejemplo, el identificador aparece en una expresión Jack)

Haga que el analizador de sintaxis genere esta información como parte de su salida XML, utilizando las etiquetas de marcado de su elección.

Pruebe el nuevo módulo SymbolTable y la nueva funcionalidad que se acaba de describir ejecutando el analizador de sintaxis extendido en los programas Jack de prueba proporcionados en el proyecto 10. Si su analizador de sintaxis extendido genera la información descrita anteriormente correctamente, significa que ha desarrollado una capacidad ejecutable completa para comprender la semántica de los programas Jack. En esta fase, puede realizar el cambio para desarrollar el compilador a gran escala y empezar a generar código de máquina virtual en lugar de salida XML. Esto se puede hacer gradualmente, como ahora vamos a describir.

(Fase 1.5: Realice una copia de seguridad del código del analizador de sintaxis extendido).

**Etapla 2: Generación de código:** Proporcionamos seis programas de aplicación, diseñados para probar gradualmente las capacidades de generación de código de su compilador Jack. Recomendamos desarrollar y probar su compilador en evolución en los programas de prueba en el orden dado. De esta manera, se le guiará implícitamente a

Construya las capacidades de generación de código del compilador en etapas sensatas, de acuerdo con las demandas presentadas por cada programa de prueba.

Normalmente, cuando uno compila un programa de alto nivel y se encuentra con dificultades, concluye que el programa está. En este proyecto el escenario es exactamente el contrario. Todos los programas de prueba suministrados están libres de errores. Por lo tanto, si su compilación arroja algún error, es el compilador el que debe corregir, no los programas. Específicamente, para cada programa de prueba, recomendamos seguir la siguiente rutina:

1. Compile la carpeta del programa utilizando el compilador que está desarrollando. Esta acción debe generar un archivo `.vm` para cada archivo `.jack` de origen en la carpeta especificada.
2. Inspeccione los archivos de máquina virtual generados. Si hay problemas visibles, corrija su compilador y vaya al paso 1. Recuerde: todos los programas de prueba suministrados están libres de errores.
3. Cargue la carpeta del programa en el emulador de máquina virtual y ejecute el código cargado. Tenga en cuenta que cada uno de los seis programas de prueba suministrados contiene pautas de ejecución específicas; probar el programa compilado (código de máquina virtual traducido) de acuerdo con estas directrices.
4. Si el programa se comporta de forma inesperada o si el emulador de VM muestra un mensaje de error, corrija el compilador y vaya al paso 1.

## Programas de prueba

**Siete:** prueba cómo el compilador maneja un programa simple que contiene una expresión aritmética con constantes enteras, una instrucción `do` y una instrucción `return`. Específicamente, el programa calcula la expresión  $1 + (2 * 3)$  e imprime su valor en la parte superior izquierda de la pantalla. Para probar si el compilador ha traducido el programa correctamente, ejecute el código traducido en el emulador de máquina virtual y compruebe que muestra 7 correctamente.

**ConvertToBin:** Prueba cómo el compilador maneja todos los elementos de procedimiento del lenguaje Jack: expresiones (sin matrices ni llamadas a métodos), funciones y las declaraciones `if`, `while`, `do`, `let` y `return`. El programa no prueba el manejo de métodos, constructores, matrices, cadenas, variables estáticas y variables de campo. Específicamente, el programa obtiene un valor decimal de 16 bits de

RAM[8000], lo convierte en binario y almacena los bits individuales en RAM [8001... 8016] (cada ubicación contendrá 0 o 1). Antes de que comience la conversión, el programa inicializa RAM[8001...8016] a -1. Para probar si el compilador ha traducido el programa correctamente, cargue el código traducido en el emulador de máquina virtual y proceda de la siguiente manera:

- Ponga (interactivamente, usando la GUI del emulador) algún valor decimal en RAM[8000].
- Ejecute el programa durante unos segundos y luego detenga su ejecución.
- Compruebe (mediante inspección visual) que las ubicaciones de memoria RAM [8001...8016] contienen los bits correctos y que ninguno de ellos contiene -1.

**Cuadrado:** prueba cómo el compilador maneja las características basadas en objetos del lenguaje Jack: constructores, métodos, campos y expresiones que incluyen llamadas a métodos. No prueba el manejo de variables estáticas. Específicamente, este programa multiclase organiza un juego interactivo simple que permite mover un cuadrado negro alrededor de la pantalla usando las cuatro teclas de flecha del teclado.

Mientras se mueve, el tamaño del cuadrado se puede aumentar y disminuir presionando las teclas z y x, respectivamente. Para salir del juego, presiona la tecla q. Para probar si el compilador ha traducido el programa correctamente, ejecute el código traducido en el emulador de máquina virtual y compruebe que el juego funciona según lo previsto.

**Promedio:** prueba cómo el compilador controla las matrices y las cadenas. Esto se hace calculando el promedio de una secuencia de enteros proporcionada por el usuario. Para probar si el compilador ha traducido el programa correctamente, ejecute el código traducido en el emulador de VM y siga las instrucciones que se muestran en la pantalla.

**Pong:** Una prueba completa de cómo el compilador maneja una aplicación basada en objetos, incluido el manejo de objetos y variables estáticas. En el juego clásico de Pong, una pelota se mueve al azar, rebotando en los bordes de la pantalla. El usuario intenta golpear la pelota con una pequeña paleta que se puede mover presionando las teclas de flecha izquierda y derecha del teclado. Cada vez que la paleta golpea la pelota, el usuario anota un punto y la paleta se encoge un poco, lo que hace que el juego sea cada vez más desafiante. Si el usuario falla y el

La pelota toca fondo, el juego ha terminado. Para probar si el compilador ha traducido este programa correctamente, ejecute el código traducido en el emulador de VM y juegue el juego. Asegúrese de obtener algunos puntos para probar la parte del programa que muestra la puntuación en la pantalla.

**ComplexArrays:** prueba cómo el compilador controla las referencias y expresiones de matrices complejas. Para ello, el programa realiza cinco cálculos complejos utilizando matrices. Para cada uno de estos cálculos, el programa imprime en la pantalla el resultado esperado junto con el resultado calculado por el programa compilado. Para probar si el compilador ha traducido el programa correctamente, ejecute el código traducido en el emulador de máquina virtual y asegúrese de que los resultados esperados y reales sean idénticos.

**Una versión basada en la web del proyecto 11** está disponible en [www.nand2tetris.org](http://www.nand2tetris.org).

---

## 11.5 Perspectiva

Jack es un lenguaje de programación de propósito general basado en objetos. Por diseño, se hizo para ser un lenguaje relativamente simple. Esta simplicidad nos permitió eludir varios problemas espinosos de compilación. Por ejemplo, aunque Jack parece un lenguaje mecanografiado, ese no es el caso: todos los tipos de datos de Jack

—int, char y boolean— tienen 16 bits de ancho, lo que permite a los compiladores de Jack ignorar casi toda la información de tipo. En particular, al compilar y evaluar expresiones, los compiladores de Jack no necesitan determinar sus tipos. La única excepción es la compilación de llamadas a métodos con el formato `x.m()`, que requiere determinar el tipo de clase de `x`. Otro aspecto de la simplicidad del tipo Jack es que los elementos de matriz no se escriben.

A diferencia de Jack, la mayoría de los lenguajes de programación cuentan con sistemas de tipos ricos, que imponen demandas adicionales a sus compiladores: se deben asignar diferentes cantidades de memoria para diferentes tipos de variables; la conversión de un tipo a otro requiere operaciones de conversión implícitas y explícitas; la compilación de una expresión simple como `x+y` depende en gran medida de los tipos de `x` e `y`; y así sucesivamente.

Otra simplificación significativa es que el lenguaje Jack no admite la *herencia*. En los lenguajes que admiten la herencia, el manejo de

Las llamadas a métodos como `x.m()` dependen de la pertenencia a la clase del objeto `x`, que solo se puede determinar durante el tiempo de ejecución. Por lo tanto, los compiladores de lenguajes orientados a objetos que presentan herencia deben tratar todos los métodos como virtuales y resolver sus pertenencias a clases según el tipo de tiempo de ejecución del objeto en el que se aplica el método. Dado que Jack no admite la herencia, todas las llamadas a métodos se pueden compilar estáticamente durante el tiempo de compilación.

Otra característica común de los lenguajes orientados a objetos no admitida por Jack es la distinción entre miembros de clase privados y públicos. En Jack, todas las variables estáticas y de campo son privadas (reconocidas solo dentro de la clase en la que se declaran), y todas las subrutinas son públicas (se pueden llamar desde cualquier

La falta de tipos reales, herencia y campos públicos permite una compilación verdaderamente independiente de clases: una clase Jack se puede compilar sin acceder al código de ninguna otra clase. Los campos de otras clases nunca se mencionan directamente, y todos los enlaces a métodos de otras clases se realizan "tarde" y se hacen solo por su nombre.

Muchas otras simplificaciones del lenguaje de Jack no son significativas y se pueden relajar con poco esfuerzo. Por ejemplo, se puede ampliar fácilmente el lenguaje con declaraciones `for` y `switch`. Del mismo modo, se puede agregar la capacidad de asignar constantes de caracteres como `'c'` a variables de tipo `char`, que actualmente no es compatible con el lenguaje.

Finalmente, nuestras estrategias de generación de código no prestaron atención a la optimización. Considere la instrucción de alto nivel `c++`. Un compilador ingenuo lo traducirá en la serie de operaciones de máquina virtual de bajo nivel `push c`, `push 1`, `add`, `pop c`. A continuación, el traductor de VM traducirá cada uno de estos comandos de VM en varias instrucciones de nivel de máquina, lo que dará como resultado una parte considerable del código. Al mismo tiempo, un compilador optimizado notará que estamos tratando con un incremento simple y lo traducirá en, digamos, las dos instrucciones de la máquina `@c` seguidas `M=M+1`. Por supuesto, este es solo un ejemplo de la delicadeza que se espera de los compiladores de fuerza industrial. En general, los escritores de compiladores invierten mucho esfuerzo e ingenio para garantizar que el código generado sea eficiente en tiempo y espacio.

En Nand to Tetris, la eficiencia rara vez es un problema, con una excepción importante: el sistema operativo. El sistema operativo Jack se basa en algoritmos eficientes y estructuras de datos optimizadas, como explicaremos en el próximo capítulo.

---

## 12 Sistema operativo

La civilización progresa ampliando el número de operaciones que podemos realizar sin pensar en ellas.  
—Alfred North Whitehead, *Introducción a las matemáticas*  
(1911)

En los capítulos 1 a 6 describimos y construimos una arquitectura de hardware de propósito general. En los capítulos 7-11 desarrollamos una jerarquía de software que hace que el hardware sea utilizable, culminando en la construcción de un lenguaje moderno basado en objetos. Se pueden especificar e implementar otros lenguajes de programación de alto nivel sobre la plataforma de hardware, cada uno de los cuales requiere su propio compilador.

La última pieza importante que falta en este rompecabezas es un *sistema operativo*. El sistema operativo está diseñado para cerrar las brechas entre el hardware y el software de la computadora, haciendo que el sistema informático sea más accesible para programadores, compiladores y usuarios. Por ejemplo, para mostrar el texto Hello World en la pantalla, se deben dibujar varios cientos de píxeles en ubicaciones específicas de la pantalla. Esto se puede hacer consultando la especificación de hardware y escribiendo código que enciende y desactiva los bits en ubicaciones de RAM seleccionadas. Claramente, los programadores de alto nivel esperan una mejor interfaz. Quieren escribir letra impresa ("Hola mundo") y dejar que alguien más se preocupe por los detalles. Ahí es donde entra en escena el sistema operativo.

A lo largo de este capítulo, el término *sistema operativo* se usa de manera bastante vaga. Nuestro sistema operativo es mínimo, con el objetivo de (i) encapsular servicios específicos de hardware de bajo nivel en servicios de software de alto nivel fáciles de programar y (ii) extender lenguajes de alto nivel con funciones de uso común y tipos de datos abstractos. La línea divisoria entre un *sistema operativo* en este sentido y una *biblioteca de clases estándar* no está clara. De hecho, la programación moderna

Los idiomas incluyen muchos servicios estándar del sistema operativo como gráficos, administración de memoria, multitarea y muchas otras extensiones en lo que se conoce como la biblioteca de *clases estándar del lenguaje*. Siguiendo este modelo, el sistema operativo Jack se empaqueta como una colección de clases de soporte, cada una de las cuales proporciona un conjunto de servicios relacionados a través de llamadas de subrutina Jack. La API completa del sistema operativo se proporciona en el apéndice 6.

Los programadores de alto nivel esperan que el sistema operativo brinde sus servicios a través de interfaces bien diseñadas que ocultan los detalles sangrientos del hardware de sus programas de aplicación. Para hacerlo, el código del sistema operativo debe operar cerca del hardware, manipulando la memoria, la entrada / salida y los dispositivos de procesamiento casi directamente. Además, debido a que el sistema operativo admite la ejecución de todos los programas que se ejecutan en la computadora, debe ser altamente eficiente. Por ejemplo, los programas de aplicación crean y eliminan objetos y matrices todo el tiempo. Por lo tanto, es mejor que lo hagamos de forma rápida y económica. Cualquier ganancia en la eficiencia de tiempo y espacio de un servicio de sistema operativo habilitado puede afectar drásticamente el rendimiento de todos los programas de aplicación que dependen de él.

Los sistemas operativos generalmente se escriben en un lenguaje de alto nivel y se compilan en forma binaria. Nuestro sistema operativo no es una excepción: está escrito en Jack, al igual que Unix se escribió en C. Al igual que el lenguaje C, Jack fue diseñado con suficiente "bajeza", lo que permite una cercanía íntima con el hardware cuando es necesario.

El capítulo comienza con una sección de antecedentes relativamente larga que presenta algoritmos clave que normalmente se usan en las implementaciones del sistema operativo. Estos incluyen operaciones matemáticas, manipulaciones de cadenas, administración de memoria, salida de texto y gráficos y entrada de teclado. Esta introducción algorítmica va seguida de una sección de Especificación que describe el sistema operativo Jack y una sección de implementación que ofrece orientación sobre cómo construir el sistema operativo utilizando los algoritmos presentados. Como de costumbre, la sección Proyecto proporciona las pautas y los materiales necesarios para construir y probar gradualmente todo el sistema operativo.

El capítulo incorpora lecciones clave en ingeniería de software orientada a sistemas y en ciencias de la computación. Por un lado, describimos técnicas de programación para desarrollar servicios de sistemas de bajo



nivel, así como técnicas de "programación a lo grande" para integrar y optimizar los servicios del sistema operativo. Por otro lado, presentamos un conjunto de algoritmos elegantes y altamente eficientes, cada uno de los cuales es una joya de la informática.

---

## 12.1 Fondo

Las computadoras generalmente están conectadas a una variedad de dispositivos de entrada / salida, como un teclado, pantalla, mouse, almacenamiento masivo, tarjeta de interfaz de red, micrófono, parlantes y más. Cada uno de estos dispositivos de E/S tiene su propia idiosincrasia electromecánica; Por lo tanto, leer y escribir datos en ellos implica muchos detalles técnicos. Los lenguajes de alto nivel abstraen estos detalles al ofrecer abstracciones de alto nivel como `let n = Keyboard.readInt("Ingrese un número:").` Profundicemos en lo que se debe hacer para realizar esta operación de entrada de datos aparentemente simple.

Primero, involucramos al usuario mostrando el mensaje `Ingrese un número:.` Esto implica crear un objeto `String` e inicializarlo en la matriz de valores `char` `'E', 'n', 't', ..., etc.` A continuación, tenemos que representar esta cadena en la pantalla, un carácter a la vez, mientras actualizamos la posición del *cursor* para realizar un seguimiento de dónde debe mostrarse físicamente el siguiente carácter. Después de mostrar el mensaje `Enter a number:.`, tenemos que organizar un bucle que espera hasta que el usuario obligue a presionar algunas teclas en el teclado, con suerte teclas que representan dígitos. Esto requiere saber cómo (i) capturar una pulsación de tecla, (ii) obtener una entrada de un solo carácter, (iii) agregar estos caracteres a una cadena y (iv) convertir la cadena en un valor entero.

Si lo que se ha elaborado hasta ahora suena arduo, el lector debe saber que en realidad fuimos bastante amables, barriando muchos detalles sangrientos debajo de la alfombra. Por ejemplo, ¿qué se entiende exactamente por "crear un objeto de cadena", "mostrar un carácter en la pantalla" y "obtener una entrada de varios caracteres"? Comencemos con "crear un objeto de cadena". Los objetos de cuerda no surgen de la nada, completamente formados. Cada vez que queramos crear un objeto, debemos encontrar espacio disponible para representar el objeto en la RAM, marcar este espacio como usado y recordar liberarlo cuando el objeto ya no sea necesario. Continuando con la abstracción de "mostrar un carácter", tenga en cuenta que los caracteres no se pueden mostrar. Lo único que se puede mostrar físicamente son píxeles individuales. Por lo tanto, tenemos que averiguar cuál es la fuente del carácter, calcular dónde se pueden encontrar los bits que representan la imagen de la fuente en el mapa de memoria de la pantalla y luego activar y desactivar estos bits según sea necesario. Finalmente, para "obtener una entrada de varios caracteres", tenemos que ingresar a un bucle que no solo escucha el teclado y acumula caracteres a medida que aparecen, sino que también

Permite al usuario retroceder, eliminar y volver a escribir caracteres, sin mencionar la necesidad de hacer eco de cada una de estas acciones en la pantalla para obtener comentarios visuales.

El agente que se encarga de este elaborado trabajo detrás de escena es el sistema operativo. La ejecución de la instrucción `let n = Keyboard.readInt("Enter a number:")` implica muchas llamadas a funciones del sistema operativo, que se ocupan de diversos problemas como la asignación de memoria, la conducción de entrada, la conducción de salida y el procesamiento de cadenas. Los compiladores usan los servicios del sistema operativo de forma abstracta inyectando llamadas a funciones del sistema operativo en el código compilado, como vimos en el capítulo anterior. En este capítulo exploramos *cómo se realizan realmente estas funciones*. Por supuesto, lo que hemos examinado hasta ahora es solo un pequeño subconjunto de las responsabilidades del sistema operativo. Por ejemplo, no mencionamos operaciones matemáticas, salida gráfica y otros servicios comúnmente necesarios. La buena noticia es que un sistema operativo bien escrito puede integrar estas tareas diversas y aparentemente no relacionadas de una manera elegante y eficiente, utilizando algoritmos y estructuras de datos geniales. De eso se trata este capítulo.

### 12.1.1 Operaciones matemáticas

Las cuatro operaciones aritméticas *suma*, *resta*, *multiplicación* y *división* se encuentran en el núcleo de casi todos los programas de computadora. Si un bucle que se ejecuta un millón de veces contiene expresiones que usan algunas de estas operaciones, es mejor que se implementen de manera eficiente.

Normalmente, la suma se implementa en hardware, a nivel de ALU, y la resta se obtiene libremente, cortesía del método de complemento de los dos. Otras operaciones aritméticas se pueden manejar por hardware o por software, dependiendo de las consideraciones de costo/rendimiento. Ahora pasamos a presentar algoritmos eficientes para calcular la multiplicación, la división y las raíces cuadradas. Estos algoritmos se prestan a implementaciones de software y hardware.

---

## La eficiencia es lo primero

Los algoritmos matemáticos están hechos para operar con valores de  $n$  bits, siendo  $n$  típicamente 16, 32 o 64 bits, dependiendo de los tipos de datos de los operandos. Como regla general, de ejecución sea una función polinómica

**Tamaño de la palabra  $n$ .** Los algoritmos cuyo tiempo de ejecución depende de los *valores* de los números de  $n$  bits son inaceptables, ya que estos valores son exponenciales en  $n$ . Por ejemplo, supongamos que implementamos la operación de multiplicación  $x \times y$  ingenuamente, utilizando el algoritmo de suma repetida para  $i = 1 \dots y \{ \text{sum} = \text{sum} + x \}$ . Si  $y$  tiene 64 bits de ancho, su valor puede ser mayor que 9,000,000,000,000,000,000,000, lo que implica que el bucle puede ejecutarse durante miles de millones de años antes de terminar.

En marcado contraste, los tiempos de ejecución de los algoritmos de multiplicación, división y raíz cuadrada que presentamos a continuación no dependen de los *valores* de  $n$  bits en los que están llamados a operar, que pueden ser tan grandes como  $2^n$ , sino más bien de  $n$ , el número de sus bits. Cuando se trata de la eficiencia de las operaciones aritméticas, eso es lo mejor que podemos esperar.

Usaremos la *notación* Big-O,  $O(n)$ , para describir un tiempo de ejecución que está "en el orden de magnitud de  $n$ ". El tiempo de ejecución de todos los algoritmos aritméticos que presentamos en este capítulo es  $O(n)$ , donde  $n$  es el ancho de bits de las entradas.

## Multiplicación

Considere el método de multiplicación estándar que se enseña en la escuela primaria. Para calcular 356 por 73, alineamos los dos números uno encima del otro, justificados a la derecha. A continuación, multiplicamos 356 por 3. A continuación, desplazamos 356 a la izquierda una posición y multiplicamos 3560 por 7 (que es lo mismo que multiplicar 356 por 70). Finalmente, sumamos las columnas y obtenemos el resultado. Este procedimiento se basa en la idea  $356 \times 73 = 356 \times 70 + 356 \times 3$ . de que la versión binaria de este procedimiento se ilustra en [la figura 12.1](#), utilizando otro ejemplo.

$$\begin{array}{r}
 x = 27 = \dots 000011011 \\
 y = 9 = \dots 000001001 \quad i\text{-th bit of } y \\
 \hline
 \dots 000011011 \quad 1 \\
 \dots 000110110 \quad 0 \\
 \dots 001101100 \quad 0 \\
 \dots 011011000 \quad 1 \\
 \hline
 x * y = 243 = \dots 011110011 \quad \text{sum}
 \end{array}$$

```

// Returns x * y, where x, y ≥ 0.
multiply(x, y):
    sum = 0
    shiftedx = x
    for i = 0 ... n - 1 do
        if ((i-th bit of y) == 1)
            sum = sum + shiftedx
        shiftedx = 2 * shiftedx
    return sum

```

**Figura 12.1** Algoritmo de multiplicación.

**Nota de notación:** Los algoritmos presentados en este capítulo están escritos en una sintaxis de pseudocódigo que se explica por sí misma. Usamos sangría para marcar bloques de código, obviando la necesidad de llaves o palabras clave de inicio / final. Por ejemplo, en la figura 12.1, `sum = sum + shiftedx` pertenece a la instrucción única cuerpo de la lógica `if`, y `shiftedx = 2 * shiftedx` termina el cuerpo de dos declaraciones de la lógica `for`.

Inspeccionemos el procedimiento de multiplicación ilustrado a la izquierda de la figura 12.1. Para cada  $i$ -ésimo bit de  $y$ , desplazamos  $x$   $i$  veces a la izquierda (lo mismo que multiplicar  $x$  por  $2^i$ ). A continuación, miramos el  $i$ -ésimo bit de  $y$ : Si es 1, sumamos la  $x$  desplazada a un acumulador; de lo contrario, no hacemos nada. El algoritmo que se muestra a la derecha formaliza este procedimiento. Tenga en cuenta que  $2 * shiftedx$  se puede calcular de manera eficiente desplazando a la izquierda la representación bit a bit de `shiftedx` o agregando `shiftedx` a sí mismo. Cualquiera de las operaciones se presta a operaciones de hardware primitivas.

**Tiempo de ejecución:** El algoritmo de multiplicación realiza  $n$  iteraciones, donde  $n$  es el ancho de bits de la entrada  $y$ . En cada iteración, el algoritmo realiza algunas operaciones de suma y comparación. De ello se deduce que el tiempo total de ejecución del algoritmo es  $a + b \cdot n$ , donde  $a$  es el tiempo que se tarda en inicializar algunas variables y  $b$  es el tiempo que se tarda en realizar algunas operaciones de suma y comparación. Formalmente, el tiempo de ejecución del algoritmo es  $O(n)$ , donde  $n$  es el ancho de bits de las entradas.

Para reiterar, el tiempo de ejecución de este  $x \times y$  algoritmo no depende de los valores de las entradas  $x$  e  $y$ ; más bien, depende del ancho de bits de las entradas. En las computadoras, el ancho de bits es normalmente una pequeña constante fija como 16 (corto), 32 (int) o 64 (largo), dependiendo de los tipos de datos de las entradas. En la plataforma Hack, el ancho de bits de todos los tipos de datos es 16. Si asumimos que cada iteración del algoritmo de multiplicación implica alrededor de diez instrucciones de la máquina Hack, se deduce que cada operación de multiplicación requerirá como máximo 160 ciclos de reloj, independientemente del tamaño de las entradas. Por el contrario, los algoritmos cuyo tiempo de ejecución no es proporcional al ancho de bits sino a los valores de las entradas requerirán ciclos de  $10 \cdot 2^{16} = 655,360$  reloj.

---

## División

La forma ingenua de calcular la división de dos números *de  $n$  bits*  $x / y$  es contar cuántas veces se puede restar  $y$  de  $x$  hasta que el resto sea menor que  $y$ . El tiempo de ejecución de este algoritmo es proporcional al valor del dividendo  $x$  y, por lo tanto, es inaceptablemente exponencial en el número de bits  $n$ .

Para acelerar las cosas, podemos intentar restar grandes trozos de  $y$  de  $x$  en cada iteración. Por ejemplo, supongamos que tenemos que dividir 175 entre 3. Comenzamos preguntando: ¿Cuál es el número más grande?,  $x = (90, 80, 70, \dots, 20, 10)$ , de modo que  $3 \cdot x \leq 175$ ? la respuesta es 50. En otras palabras, logramos restar cincuenta 3 de 175, recortando cincuenta iteraciones del enfoque ingenuo. Esta resta acelerada deja un resto de  $175 - 3 \cdot 50 = 25$ . Avanzando, ahora preguntamos: ¿Cuál es el número más grande,  $x = (9, 8, 7, \dots, 2, 1)$ , de modo que  $3 \cdot x \leq 25$ ? la respuesta es 8, por lo que logramos hacer ocho restas adicionales de 3, y la respuesta, hasta ahora, es  $50 + 8 = 58$ . El resto es  $25 - 3 \cdot 8 = 1$ , que es menor que 3, por lo que detenemos el proceso y anunciamos que  $175/3 = 58$  con un resto de 1.

Esta técnica es la razón detrás del temido procedimiento escolar conocido como *división larga*. La versión binaria de este algoritmo es idéntica, excepto que en lugar de acelerar la resta usando potencias de 10, usamos potencias de 2. El algoritmo realiza  $n$  iteraciones, *siendo  $n$*  el número de dígitos en el dividendo, y cada iteración implica algunas operaciones de multiplicación (en realidad, desplazamiento), comparación y resta. Una vez más, tenemos un  $x/y$  algoritmo cuyo tiempo de ejecución no depende de los valores de  $x$  e  $y$ . Más bien, el tiempo de ejecución es  $O(n)$ , donde  $n$  es el ancho de bits de las entradas.

Escribir este algoritmo como lo hemos hecho para la multiplicación es un ejercicio fácil. Para hacer las cosas interesantes, la [figura 12.2](#) presenta otro algoritmo de división que es igual de eficiente, pero más elegante y fácil de implementar.

```
// Returns the integer division  $x / y$ ,  
// where  $x \geq 0$  and  $y > 0$ .
```

```
divide( $x, y$ ):
```

```
    if ( $y > x$ ) return 0
```

```
     $q = \text{divide}(x, 2 * y)$ 
```

```
    if ( $(x - 2 * q * y) < y$ )
```

```
        return  $2 * q$ 
```

```
    else
```

```
        return  $2 * q + 1$ 
```

**Figura 12.2** Algoritmo de división.

Supongamos que tenemos que dividir 480 entre 17. El algoritmo que se muestra en [la figura 12.2](#) es basado en el [visión](#)  $480 / 17 = 2 \cdot (240 / 17) = 2 \cdot (2 \cdot (120 / 17)) = 2 \cdot (2 \cdot (2 \cdot (60 / 17))) = \dots$ , y así sucesivamente. La profundidad de esta recursión está limitada por el número de veces *que*  $y$  se puede multiplicar por 2 antes de llegar a  $x$ . Este también es, como máximo, el número de bits necesarios para representar  $x$ . Por lo tanto, el tiempo de ejecución de este algoritmo es  $O(n)$ , donde  $n$  es el ancho de bits de las entradas.

Un inconveniente en este algoritmo es que cada operación de multiplicación también requiere *operaciones*  $O(n)$ . Sin embargo, una inspección de la lógica del algoritmo revela que el valor de la expresión  $(2 * q * y)$  se puede calcular sin multiplicación. En cambio, se puede obtener a partir de su valor en el nivel de recursividad anterior, usando la suma.

---

## Raíz cuadrada

Las raíces cuadradas se pueden calcular de manera eficiente de varias maneras diferentes, por ejemplo, utilizando el método de Newton-Raphson o una expansión de la serie de Taylor. Para nuestro propósito, sin embargo, un algoritmo más simple será suficiente. La función de raíz cuadrada  $y = \sqrt{x}$  tiene dos propiedades atractivas. Primero, está aumentando monótonamente. En segundo lugar, su función inversa  $y = x^2$ , es una función que ya sabemos cómo calcular de manera eficiente: la multiplicación. En conjunto, estas propiedades implican que tenemos todo lo que necesitamos para calcular raíces cuadradas de manera eficiente, utilizando una forma de *búsqueda binaria*. La Figura 12.3 da los detalles.

```
// Computes the integer part of  $y = \sqrt{x}$ 
// Strategy: finds an integer  $y$  such that  $y^2 \leq x < (y+1)^2$  (for  $0 \leq x < 2^n$ )
// by performing binary search in the range  $0 \dots 2^{n/2} - 1$ 

sqrt(x):
    y = 0
    for j = (n/2 - 1) ... 0 do
        if  $(y + 2^j)^2 \leq x$  then  $y = y + 2^j$ 
    return y
```

**Figura 12.3** Algoritmo de raíz cuadrada.

Dado que el número de iteraciones en la búsqueda binaria que realiza el algoritmo está limitado por  $n / 2$  donde  $n$  es el número de bits en  $x$ , el tiempo de ejecución del algoritmo es  $O(n)$ .

Para resumir esta sección sobre operaciones matemáticas, presentamos algoritmos para calcular la multiplicación, la división y la raíz cuadrada. El tiempo de ejecución de cada uno de los algoritmos es  $O(n)$ , donde  $n$  es el ancho de bits de las entradas. También observamos que en las computadoras,  $n$  es una pequeña constante como 16, 32 o 64. Por lo tanto, cada operación de suma, resta, multiplicación y división se puede llevar a cabo rápidamente, en un tiempo predecible que no se ve afectado por la magnitud de las entradas.

### 12.1.2 Instrumentos de cuerda



Además de los tipos de datos primitivos, la mayoría de los lenguajes de programación cuentan con un

*tipo de cadena* diseñado para representar secuencias de caracteres como "Cargando juego

..." y "QUIT". Normalmente, la abstracción de cadena la proporciona una clase String que forma parte de la biblioteca de clases estándar que admite el lenguaje. Este es también el enfoque adoptado por Jack.

Todas las constantes de cadena que aparecen en los programas Jack se implementan como objetos String. La clase String, cuya API se documenta en el apéndice 6, presenta varios métodos de procesamiento de cadenas, como anexar un carácter a la cadena, eliminar el último carácter, etc. Estos servicios no son difíciles de implementar, como describiremos más adelante en el capítulo. Los métodos String más desafiantes son aquellos que convierten valores enteros en cadenas y cadenas de caracteres de dígitos en valores enteros. Ahora pasamos a discutir los algoritmos que llevan a cabo estas operaciones.

**Representación de cadenas de números:** Las computadoras representan números internamente usando códigos binarios. Sin embargo, los humanos están acostumbrados a tratar con números que están escritos en notación decimal. Por lo tanto, cuando los humanos tienen que leer o ingresar números, *y solo entonces*, se debe realizar una conversión hacia o desde la notación decimal. Cuando estos números se capturan desde un dispositivo de entrada como un teclado, o se representan en un dispositivo de salida como una pantalla, se convierten en cadenas de caracteres, cada una de las cuales representa uno de los dígitos del 0 al 9. El subconjunto de caracteres relevantes es:

|                 |     |     |     |     |     |     |     |     |     |     |
|-----------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Character:      | '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9' |
| Character code: | 48  | 49  | 50  | 51  | 52  | 53  | 54  | 55  | 56  | 57  |

(El conjunto completo de caracteres de Hack se da en el apéndice 5). Vemos que los caracteres de dígitos se pueden convertir fácilmente en los números enteros que representan, y viceversa. El valor entero del carácter  $c$ , donde  $48 \leq c \leq 57$ , es  $c - 48$ . Por el contrario, el código de carácter del entero  $x$ , donde  $0 \leq x \leq 9$ , es  $x + 48$ .

Una vez que sepamos cómo manejar caracteres de un solo dígito, podemos desarrollar algoritmos para convertir cualquier número entero en una cadena y cualquier cadena de caracteres de dígitos en el número entero correspondiente. Estos algoritmos de conversión pueden basarse en lógica

iterativa o recursiva, por lo que [la figura 12.4](#) presenta uno de cada uno.

|                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> int to string:  // Returns the string representation of // a nonnegative integer. <b>int2String(val):</b>     lastDigit = val % 10     c = character representing lastDigit     if (val &lt; 10)         return c (as a string)     else         return int2String(val / 10).appendChar(c) </pre> | <pre> string to int:  // Returns the integer value of a string // of digit characters, assuming that str[0] // represents the most significant digit. <b>string2Int(str):</b>     val = 0     for (i = 0 ... str.length() ) do         d = integer value of str.charAt(i)         val = val * 10 + d     return val </pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Figura 12.4** Conversiones de enteros de cadena. (appendChar, length y charAt son métodos de clase String).

Es fácil inferir de la [figura 12.4](#) que los tiempos de ejecución de los algoritmos int2String y string2Int son  $O(n)$ , donde  $n$  es el número de caracteres de dígitos en la entrada.

### 12.1.3 Gestión de memoria

Cada vez que un programa crea una nueva matriz o un nuevo objeto, se debe asignar un bloque de memoria de cierto tamaño para representar la nueva matriz u objeto. Y cuando la matriz u objeto ya no es necesario, su espacio RAM puede reciclarse. Estas tareas se realizan mediante dos funciones clásicas del sistema operativo llamadas alloc y deAlloc. Estas funciones son utilizadas por los compiladores al generar código de bajo nivel para manejar constructores y destructores, así como por programadores de alto nivel, según sea necesario.

Los bloques de memoria para representar matrices y objetos se tallan y reciclan de nuevo en un área de RAM designada llamada *montón*. El agente responsable de administrar este recurso es el sistema operativo. Cuando el sistema operativo comienza a ejecutarse, inicializa un puntero llamado heapBase, que contiene la dirección base del montón en la RAM (en Jack, el montón comienza justo después del final de la pila, con heapBase=2048). Presentaremos dos algoritmos de administración del montón: básico y mejorado.

**Algoritmo de asignación de memoria (básico):** la estructura de datos que administra este algoritmo es un único puntero, denominado free, que apunta al principio del segmento del montón que aún no se ha asignado. Consulte [la figura 12.5a](#) para obtener más detalles.

```
init():
    free = heapBase

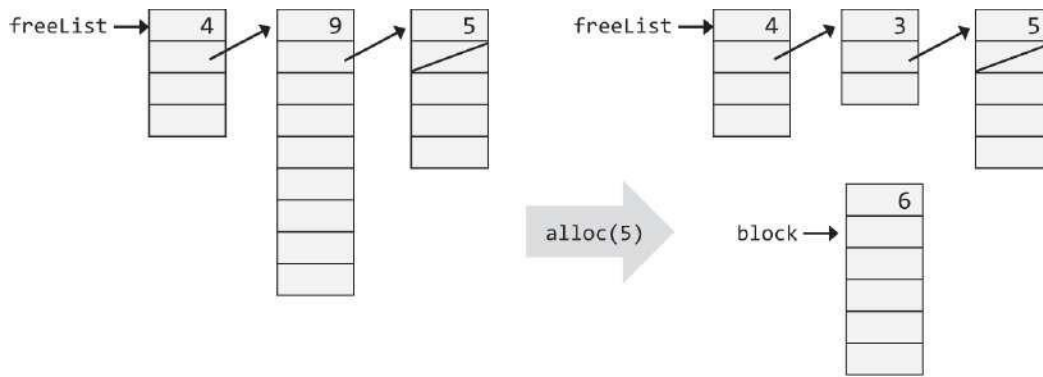
// Allocates a memory block of size words.
alloc(size):
    block = free
    free = free + size
    return block

// Frees the memory space of the given object.
deAlloc(object):
    do nothing
```

**Figura 12.5a** Algoritmo de asignación de memoria (básico).

El esquema básico de administración del montón es claramente un desperdicio, ya que nunca recupera espacio de memoria. Pero, si sus programas de aplicación usan solo unos pocos objetos pequeños y matrices, y no demasiadas cadenas, puede salirse con la suya.

**Algoritmo de asignación de memoria (mejorado):** Este algoritmo administra una lista enlazada de segmentos de memoria disponibles, llamada *freeList* (ver [figura 12.5b](#)). Cada segmento de la lista comienza con dos campos de limpieza: la longitud del segmento y un puntero al *siguiente* segmento de la lista.



```

init():
    freeList = heapBase
    freeList.size = heapSize
    freeList.next = 0

// Allocates a memory block of size words.
alloc(size):
    search freeList using best-fit or first-fit heuristics
        to obtain a segment with segment.size ≥ size + 2
    if no such segment is found, return failure
        (or attempt defragmentation)
    block = base address of the found space
    update the freeList and the fields of block
        to account for the allocation
    return block

// Frees the memory space of the given object.
deAlloc(object):
    append object to the end of the freeList

```

**Figura 12.5b** Algoritmo de asignación de memoria (mejorado).

Cuando se le pide que asigne un bloque de memoria de un tamaño determinado, el algoritmo tiene que buscar en *freeList* un segmento adecuado. Hay dos heurísticas para hacer esta búsqueda. *El mejor ajuste* encuentra el segmento más corto que es lo suficientemente largo para representar el tamaño requerido, mientras que *el primer ajuste* encuentra el primer segmento que es lo suficientemente largo. Una vez que se ha encontrado un segmento adecuado, se talla el bloque de memoria requerido a partir de él (la ubicación justo antes del comienzo del bloque devuelto,  $\text{block}[-1]$ , se reserva para mantener su longitud, para ser utilizada durante la desasignación).

A continuación, la *longitud* de este segmento se actualiza en *freeList*, reflejando la longitud de la parte que quedó después de la asignación. Si no quedaba memoria

en el segmento, o si la parte restante es prácticamente demasiado pequeña, todo el segmento se elimina de la freeList.

Cuando se le pide que recupere el bloque de memoria de un objeto no utilizado, el algoritmo agrega el bloque desasignado al final de freeList.

Algoritmos de asignación de memoria dinámica como el que se muestra en la [figura](#)

[12.5b](#) puede crear problemas de fragmentación de bloques. Por lo tanto, *se debe considerar una operación de desfragmentación*, es decir, fusionar áreas de memoria que son físicamente adyacentes en la memoria pero que se dividen lógicamente en diferentes segmentos en freeList. La desfragmentación se puede realizar cada vez que se desasigna un objeto, cuando alloc() no encuentra un bloque del tamaño solicitado, o de acuerdo con alguna otra condición ad hoc periódica.

**Echar un vistazo y empujar:** Terminamos la discusión sobre la administración de memoria con dos funciones simples del sistema operativo que no tienen nada que ver con la asignación de recursos. Memory.peek(addr) devuelve el valor de la RAM en la dirección addr, y Memory.poke(addr,value) establece la palabra en la dirección RAM addr en value. Estas funciones juegan un papel en varios servicios del sistema operativo que manipulan la memoria, incluidas las rutinas de gráficos, como ahora vamos a discutir.

#### 12.1.4 Salida gráfica

Las computadoras modernas renderizan resultados gráficos como animación y video en pantallas a color de alta resolución, utilizando controladores de gráficos optimizados y unidades de procesamiento gráfico (GPU) dedicadas. En Nand to Tetris abstraemos la mayor parte de esta complejidad, centrándonos en cambio en los algoritmos y técnicas fundamentales de dibujo de gráficos.

Suponemos que la computadora está conectada a una pantalla física en blanco y negro dispuesta como una cuadrícula de filas y columnas, y en la intersección de cada una se encuentra un píxel. Por convención, las columnas se numeran de izquierda a derecha y las filas se numeran de arriba a abajo. Por lo tanto, el píxel (0,0) se encuentra en la esquina superior izquierda de la pantalla.

Suponemos que la pantalla está conectada al sistema informático a través de un *mapa de memoria*, un área de RAM dedicada en la que cada píxel está representado por un bit. **La pantalla se actualiza a partir de este mapa de memoria muchas veces por segundo mediante un proceso externo a la computadora. Se espera que los programas que simulan las operaciones de**

la computadora emulen este proceso de actualización.

La operación más básica que se puede realizar en la pantalla es dibujar un píxel individual especificado por coordenadas  $(x,y)$ . Esto se hace activando o desactivando el bit correspondiente en el mapa de memoria. Otras operaciones como dibujar una línea y dibujar un círculo se basan en esta operación básica. El paquete de gráficos mantiene un *color actual* que se puede establecer en *blanco* o *negro*. Todas las operaciones de dibujo utilizan el color actual.

**Dibujo de píxeles (drawPixel):** El dibujo de un píxel seleccionado en la ubicación de la pantalla  $(x,y)$  se logra localizando el bit correspondiente en el mapa de memoria y configurándolo en el color actual. Dado que la RAM es un dispositivo de  $n$  bits, esta operación requiere leer y escribir un valor de  $n$  bits. Ver [figura 12.6](#).

```
// Sets pixel  $(x,y)$  to the current color.  
drawPixel( $x,y$ ):  
    Using  $x$  and  $y$ , compute the RAM address where  
        the pixel is represented;  
    Using Memory.peek, get the 16-bit value of  
        this address;  
    Using some bitwise operation, set (only) the bit  
        that corresponds to the pixel to the current color;  
    Using Memory.poke, write the modified 16-bit  
        value "back" to the RAM address.
```

**Figura 12.6** Dibujar un píxel.

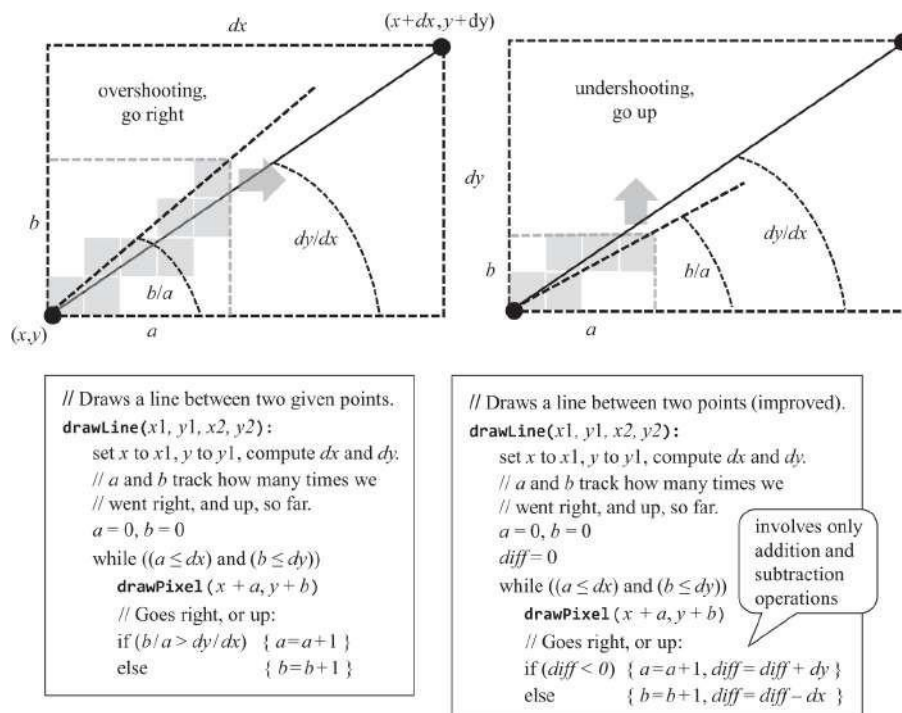
La interfaz del mapa de memoria de la pantalla Hackear se especifica en la sección  
5.2.4. Este mapeo debe usarse para realizar el `drawPixel` algoritmo.

**Dibujo lineal (drawLine):** Cuando se nos pide que representemos una "línea" continua entre dos "puntos" en una cuadrícula hecha de píxeles discretos, lo mejor que podemos hacer es aproximar la línea dibujando una serie de píxeles a lo largo de la línea imaginaria que conecta los dos puntos. El "bolígrafo" que usamos para dibujar la línea solo puede moverse en cuatro direcciones: arriba, abajo, izquierda y derecha.



Por lo tanto, la línea dibujada seguramente será irregular, y la única forma de que se vea bien es usar una pantalla de alta resolución con los píxeles más pequeños posibles. Sin embargo, tenga en cuenta que el ojo humano, al ser otra máquina, también tiene una capacidad limitada de captura de imágenes, determinada por el número y el tipo de células receptoras en la retina. Por lo tanto, las pantallas de alta resolución pueden engañar al cerebro humano para que crea que las líneas hechas de píxeles son visiblemente suaves. De hecho, siempre son irregulares.

El procedimiento para dibujar una línea de  $(x_1, y_1)$  a  $(x_2, y_2)$  comienza dibujando el píxel  $(x_1, y_1)$  y luego zigzagueando en la dirección de  $(x_2, y_2)$  hasta llegar a ese píxel. Ver [figura 12.7](#).



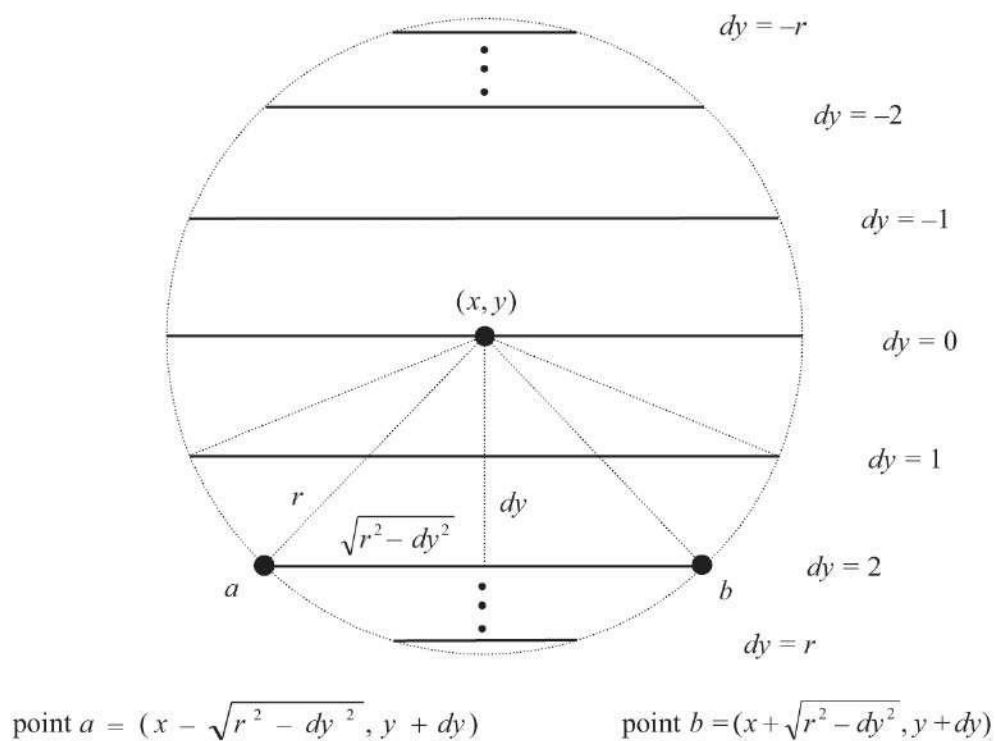
**Figura 12.7** Algoritmo de dibujo lineal: versión básica (abajo, izquierda) y versión mejorada (abajo, derecha).

El uso de dos operaciones de división en cada iteración de bucle hace que este algoritmo no sea eficiente ni preciso. La primera mejora obvia es reemplazar la  $b/a > dy/dx$  condición con el equivalente  $a \cdot dy < b \cdot dx$ , que solo requiere multiplicación de enteros. Una inspección cuidadosa de esta última condición revela que puede verificarse sin ninguna multiplicación. Como se muestra en el algoritmo mejorado de la [figura 12.7](#), esto puede hacerse

De manera eficiente manteniendo una variable que actualiza el valor de  $(a \cdot dy - b \cdot dx)$  cada vez *que se incrementa a o b*.

El tiempo de ejecución de este algoritmo de dibujo de líneas es  $O(n)$ , donde  $n$  es el número de píxeles a lo largo de la línea dibujada. El algoritmo utiliza solo operaciones de suma y resta y se puede implementar de manera eficiente en software o hardware.

**Dibujo circular (drawCircle):** La Figura 12.8 presenta un algoritmo que utiliza tres rutinas que ya hemos implementado: multiplicación, raíz cuadrada y dibujo lineal.



```
// Draws a filled circle of radius  $r$ , centered at  $(x,y)$ .

drawCircle( $x, y, r$ ):
    for each  $dy = -r$  to  $r$  do:
        drawLine( $(x - \sqrt{r^2 - dy^2}, y + dy), (x + \sqrt{r^2 - dy^2}, y + dy)$ )
```

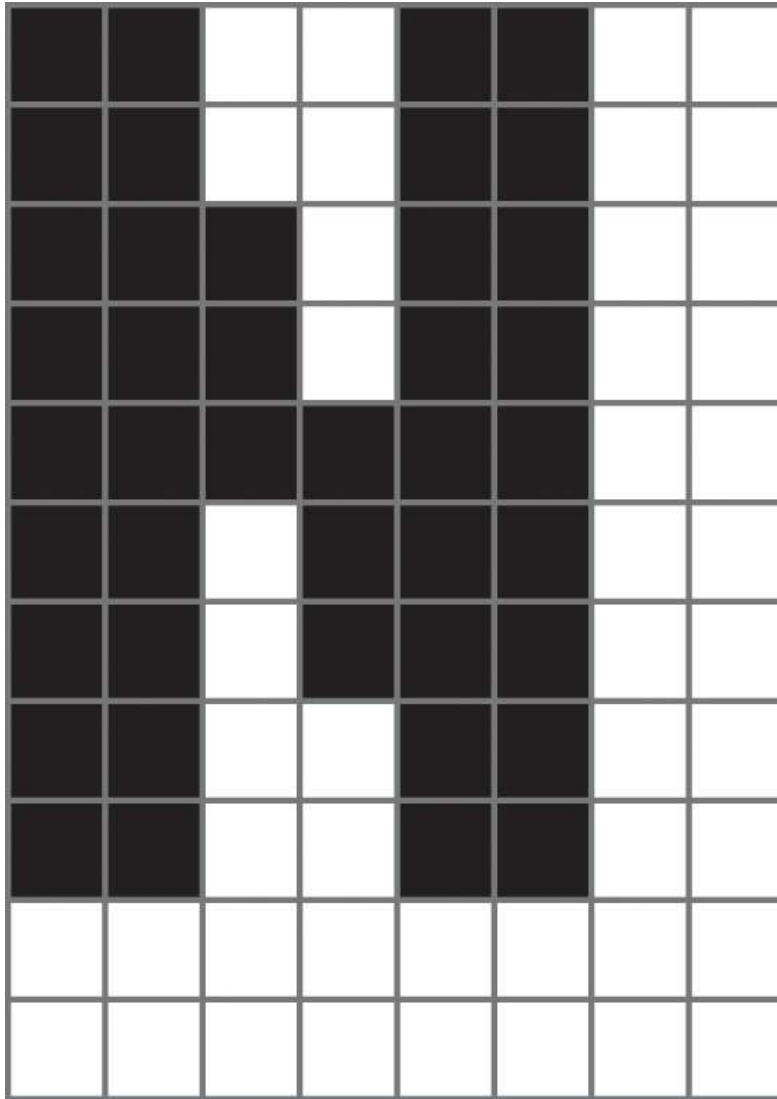
**Figura 12.8** Algoritmo de dibujo de círculos.

El algoritmo se basa en dibujar una secuencia de líneas horizontales (como la típica línea *ab* de la figura), una para cada fila del rango  $y-r$  to  $y+r$ . Dado que  $r$  se especifica en píxeles, el algoritmo termina dibujando una línea en cada fila a lo largo del diámetro norte-sur del círculo, lo que da como resultado un círculo completamente lleno. Un simple ajuste puede hacer que este algoritmo dibuje solo el contorno del círculo, si así lo desea.

### 12.1.5 Salida de caracteres

Para desarrollar una capacidad de visualización de caracteres, primero convertimos nuestra pantalla física orientada a píxeles en una pantalla lógica y orientada a los caracteres adecuada para representar imágenes fijas de mapa de bits que representan personajes. Por ejemplo, considere una pantalla física que presenta 256 filas de 512 píxeles cada una. Si asignamos una cuadrícula de 11 filas por 8 columnas para dibujar un solo carácter, entonces nuestra pantalla puede mostrar 23 líneas de 64 caracteres cada una, con 3 filas adicionales de píxeles sin usar.

**Fuentes:** Los conjuntos de caracteres que utilizan los ordenadores se dividen en *subconjuntos imprimibles* y *no imprimibles*. Para cada carácter imprimible en el conjunto de caracteres Hack (ver apéndice 5), se diseñó una imagen de mapa de bits de 11 filas por 8 columnas, lo mejor que pudimos artísticamente. En conjunto, estas imágenes se denominan *fuentes*. La Figura 12.9 muestra cómo nuestra fuente representa la letra mayúscula N. Para controlar el espaciado de caracteres, cada imagen de carácter incluye al menos un espacio de 1 píxel antes del siguiente carácter de la fila y al menos un espacio de 1 píxel entre filas adyacentes (el espaciado exacto varía según el tamaño y el subrayado ondulado de caracteres individuales). La fuente Hack consta de noventa y cinco imágenes de mapa de bits de este tipo, una para cada carácter imprimible en el conjunto de caracteres Hack.



**Figura 12.9** Ejemplo de un mapa de bits de caracteres.

El diseño de fuentes es un arte antiguo pero vibrante. Las fuentes más antiguas son tan antiguas como el arte de escribir, y las nuevas son introducidas rutinariamente por diseñadores tipográficos que desean hacer una declaración artística o resolver un objetivo técnico o funcional. En nuestro caso, la pequeña pantalla física, por un lado, y el deseo de mostrar un número razonable de caracteres en cada fila, por otro, llevaron a la elección pragmática de un área de imagen frugal de  $11 \times 8$  píxeles. Esta economía nos obligó a diseñar una fuente tosca, que sin embargo cumple bien su propósito.

**Cursor:** Los caracteres generalmente se muestran uno tras otro, de izquierda a derecha, hasta llegar al final de la línea. Por ejemplo, considere un programa en el que la instrucción `print("a")` es seguida (quizás no inmediatamente) por la instrucción `print("b")`. Esto implica que el programa quiere mostrar `ab` en la pantalla. Para lograr esta continuidad, el paquete de escritura de caracteres mantiene un *cursor global* que realiza un seguimiento de la ubicación de la pantalla donde se debe dibujar el siguiente carácter. La información del cursor consta de recuentos de columnas y filas, por ejemplo, `cursor.col` y `cursor.row`. Después de que se ha mostrado un carácter, hacemos `cursor.col++`. Al final de la fila hacemos `cursor.row++` y `cursor.col = 0`. Cuando se llega a la parte inferior de la pantalla, hay una pregunta sobre qué hacer a continuación. Dos acciones posibles son realizar una operación de desplazamiento o borrar la pantalla y comenzar de nuevo colocando el cursor en (0,0).

Para recapitular, describimos un esquema para escribir caracteres individuales en la pantalla. La escritura de otros tipos de datos se deriva naturalmente de esta capacidad básica: las cadenas se escriben carácter por carácter y los números se convierten primero en cadenas y luego se escriben como cadenas.

### 12.1.6 Entrada de teclado

La captura de entradas que provienen del teclado es más compleja de lo que parece. Por ejemplo, considere la instrucción `let name =`

`Keyboard.readLine("ingrese su nombre:");` Por definición, la ejecución de la línea `readLine`

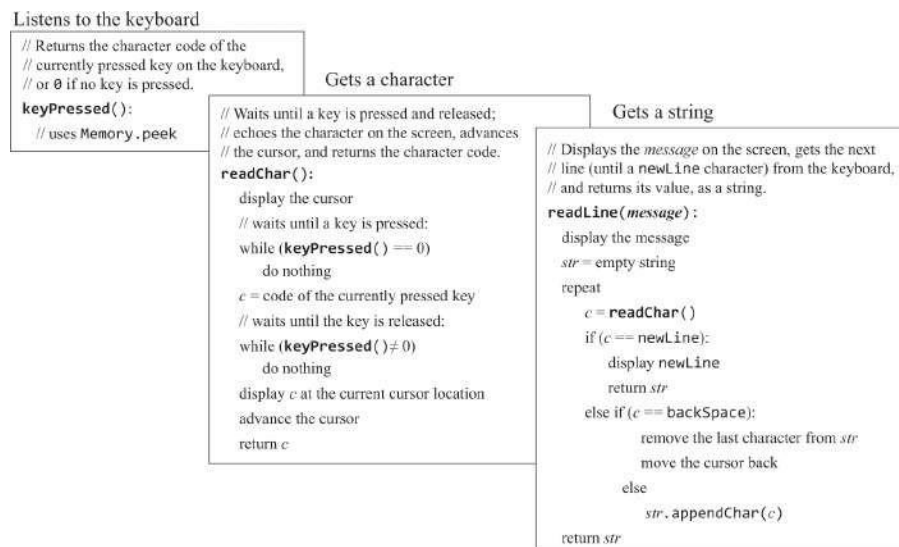
La función depende de la destreza y la colaboración de una entidad impredecible: un usuario humano. La función no finalizará hasta que el usuario haya presionado algunas teclas del teclado, terminando con un botón ENTER. El problema es que los humanos presionan y sueltan las teclas del teclado durante períodos de tiempo variables e impredecibles, y a menudo toman un descanso para tomar un café en el medio. Además, a los humanos les gusta retroceder, eliminar y volver a escribir caracteres. La implementación de la función `readLine` debe controlar todas estas irregularidades.

Esta sección describe cómo se administra la entrada del teclado, en tres niveles de abstracción: (i) detectar qué tecla se presiona actualmente en el teclado,

(ii) capturar una entrada de un solo carácter, y (iii) capturar una entrada de varios caracteres.

**Detección de la entrada del teclado (keyPressed):** la detección de la tecla que se está pulsando actualmente es una operación específica del hardware que depende del teclado

interfaz. En la computadora Hack, el teclado actualiza continuamente un registro de memoria de 16 bits cuya dirección se mantiene en un puntero llamado KBD. El contrato de interacción es el siguiente: si se presiona una tecla en el teclado, esa dirección contiene el código de caracteres de la tecla (el conjunto de caracteres Hack se da en el apéndice 5); de lo contrario, contiene 0. Este contrato se utiliza para implementar la función `keyPressed` que se muestra en la [figura 12.10](#).



**Figura 12.10** Manejo de la entrada desde el teclado.

**Lectura de un solo carácter (`readChar`):** el tiempo transcurrido entre la *tecla presionada* y los *eventos posteriores liberados de la tecla* es impredecible. Por lo tanto, tenemos que escribir código que neutralice esta incertidumbre. Además, cuando los usuarios presionan teclas en el teclado, queremos dar retroalimentación sobre qué teclas se han presionado (algo que probablemente haya llegado a dar por sentado). Por lo general, queremos mostrar algún cursor gráfico en la ubicación de la pantalla donde va la siguiente entrada y, después de presionar alguna tecla, queremos hacer eco del carácter ingresado mostrando su mapa de bits en la pantalla en la ubicación del cursor. Todas estas acciones son implementadas por la función `readChar`.

**Lectura de una cadena (`readLine`):** una entrada de varios caracteres escrita por el usuario se considera definitiva después de presionar la tecla ENTER, lo que produce el carácter `newLine`. Hasta que se presione la tecla ENTER, se debe permitir al usuario

retroceso, eliminar y volver a escribir los caracteres escritos anteriormente. Todas estas acciones son acomodadas por la función `readLine`.

Como de costumbre, nuestras soluciones de control de entrada se basan en una serie de abstracciones en cascada: El programa de alto nivel se basa en la abstracción `readLine`, que se basa en la abstracción `readChar`, que se basa en la abstracción `keyPressed`, que se basa en la abstracción `Memory.peek`, que se basa en el hardware.

---

## 12.2 La especificación de Jack OS

En la sección anterior se presentaron algoritmos que abordan varias tareas clásicas del sistema operativo. En esta sección pasamos a especificar formalmente un sistema operativo en particular: el sistema operativo Jack. El sistema operativo Jack está organizado en ocho clases:

- Matemáticas: proporciona operaciones matemáticas
- String: implementa el tipo String
- Array: implementa el tipo Array
- Memoria: maneja las operaciones de memoria
- Pantalla: maneja la salida de gráficos a la pantalla
- Salida: maneja la salida de caracteres a la pantalla
- Teclado: maneja la entrada del teclado
- Sys: proporciona servicios relacionados con la ejecución

La API completa del sistema operativo se proporciona en el apéndice 6. Esta API se puede ver como la especificación del sistema operativo. En la siguiente sección se describe cómo se puede implementar esta API mediante los algoritmos presentados en la sección anterior.

---

## 12.3 Implementación

Cada clase del sistema operativo es una colección de subrutinas (constructores, funciones y métodos). La mayoría de las subrutinas del sistema operativo son fáciles de implementar y no se analizan aquí. Las subrutinas restantes del sistema operativo se basan en los algoritmos



presentado en la sección 12.2. La implementación de estas subrutinas puede beneficiarse de algunos consejos y pautas, que ahora pasamos a presentar.

**Funciones de inicio:** algunas clases del sistema operativo utilizan estructuras de datos que admiten la implementación de algunas de sus subrutinas. Para cada uno de estos *OSClass*, estas estructuras de datos se pueden declarar estáticamente a nivel de clase e inicializarse mediante una función que, por convención, llamamos *OSClass.init*. Las funciones de inicialización son para fines internos y no están documentadas en la API del sistema operativo.

## Matemática

**multiplicar:** En cada iteración  $i$  del algoritmo de *multiplicación* (ver [figura 12.1](#)), se extrae el  $i$ -ésimo bit del segundo multiplicando. Sugerimos encapsular esta operación en un bit de función auxiliar  $(x, i)$  que devuelva true si el  $i$ -ésimo bit del entero  $x$  es 1, y false en caso contrario. La función  $\text{bit}(x, i)$  se puede implementar fácilmente mediante operaciones de desplazamiento. Por desgracia, Jack no apoya el cambio. En su lugar, puede ser conveniente definir una matriz estática fija de longitud 16, digamos *twoToThe*, y establecer cada elemento  $i$  en 2 elevado a la potencia de  $i$ . La matriz se puede utilizar para admitir la implementación de  $\text{bits}(x, i)$ . La matriz *twoToThe* se puede construir mediante la función *Math.init*.

**dividir:** Los algoritmos de *multiplicación* y *división* presentados en [las figuras 12.1](#) y [12.2](#) están diseñados para operar en números enteros no negativos. Los números con signo se pueden manejar aplicando los algoritmos a los valores absolutos y estableciendo el signo de los valores devueltos de manera adecuada. Para el algoritmo de multiplicación, esto no es necesario: dado que los multiplicandos se dan en el complemento de dos, su producto será correcto sin más preámbulos.

En el algoritmo de *división*,  $y$  se multiplica por un factor de 2, hasta *que*  $y > x$ . Por lo tanto,  $y$  puede desbordarse. El desbordamiento se puede detectar comprobando cuándo  $y$  se vuelve negativo.

**sqrt:** En el algoritmo de *raíz cuadrada* ([figura 12.3](#)), el cálculo de  $(y + 2^j)^2$  puede desbordarse, lo que da como resultado un resultado anormalmente negativo. Este problema se puede abordar cambiando eficientemente la lógica if del algoritmo a: if  $(y + 2^j)^2 \leq x$  y  $(y + 2^j)^2 > 0$  luego  $y = y + 2^j$ .

## Cuerda

Todas las constantes de cadena que aparecen en un programa Jack se realizan como objetos de la clase `String`, cuya API se documenta en el apéndice 6. En concreto, cada cadena se implementa como un objeto que consta de una matriz de valores `char`, una propiedad `maxLength` que contiene la longitud máxima de la cadena y una propiedad `length` que contiene la longitud real de la cadena.

Por ejemplo, considere la instrucción `let str="scooby"`. Cuando el compilador maneja esta declaración, llama al constructor `String`, que crea una matriz de caracteres con `maxLength=6` y `length=6`. Si más tarde llamamos al método `String str.eraseLastChar()`, la longitud de la matriz se convertirá en 5 y la cadena se convertirá efectivamente en "scoob". En general, entonces, los elementos de matriz más allá de la longitud no se consideran parte de la cadena.

¿Qué debería suceder cuando se intenta agregar un carácter a una cadena cuya longitud es igual a `maxLength`? Este problema no está definido por la especificación del sistema operativo: la clase `String` puede actuar correctamente y cambiar el tamaño de la matriz, o no; esto se deja a discreción de las implementaciones individuales del sistema operativo.

**intValue, setInt:** Estas subrutinas se pueden implementar utilizando los algoritmos presentados en la [figura 12.4](#). Tenga en cuenta que ninguno de los algoritmos maneja números negativos, un detalle que debe ser manejado por la implementación.

**newLine, backSpace, doubleQuote:** Como se ve en el apéndice 5, los códigos de estos caracteres son 128, 129 y 34.

Los métodos `String` restantes se pueden implementar directamente operando en la matriz `char` y en el campo `length` que caracteriza a cada objeto `String`.

## Arreglo

**new:** A pesar de su nombre, esta subrutina no es un constructor sino una función. Por lo tanto, la implementación de esta función debe asignar espacio de memoria para la nueva matriz llamando explícitamente a la función OS

`Memoria.alloc.`

**dispose:** Este método `void` es llamado por declaraciones como `do arr.dispose()`. La implementación `dispose` desasigna la matriz llamando a la función del

sistema operativo Memory.deAlloc.

## Memoria

**peek, poke:** Estas funciones proporcionan acceso directo a la memoria del host. ¿Cómo se puede lograr este acceso de bajo nivel utilizando el lenguaje de alto nivel de Jack? Resulta que el lenguaje Jack incluye una trampilla que permite obtener un control completo de la memoria de la computadora host. Esta trampilla se puede explotar para implementar `Memory.peek` y `Memory.poke` usando la programación simple de Jack.

El truco se basa en un uso anómalo de una variable de referencia (puntero). Jack es un lenguaje débilmente tipado; Entre otras peculiaridades, no impide que el programador asigne una constante a una variable de referencia. Esta constante se puede tratar como una dirección de memoria absoluta. Cuando la variable de referencia resulta ser una matriz, este esquema proporciona acceso indexado a cada palabra en la RAM del host. Véase [la figura 12.11](#).

```
// Creates a Jack-level “proxy” of the RAM:
var Array memory;
let memory = 0; // No problem . . .
...
// Gets the value of the RAM at address i:
let x = memory[i];
...
// Sets the value of the RAM at address i:
let memory[i] = 17;
...
```

**Figura 12.11** Una trampilla que permite el control completo de la RAM del host desde Jack.

Después de las dos primeras líneas de código, la base de la matriz de memoria apunta a la primera dirección en la RAM de la computadora (dirección 0). Para obtener o establecer el valor de la ubicación de RAM cuya dirección física es *i*, todo lo que tenemos que hacer es

Manipula el elemento de matriz `memory[i]`. Esto hará que el compilador manipule la ubicación de RAM cuya dirección es  $0 + i$ , la que queremos.

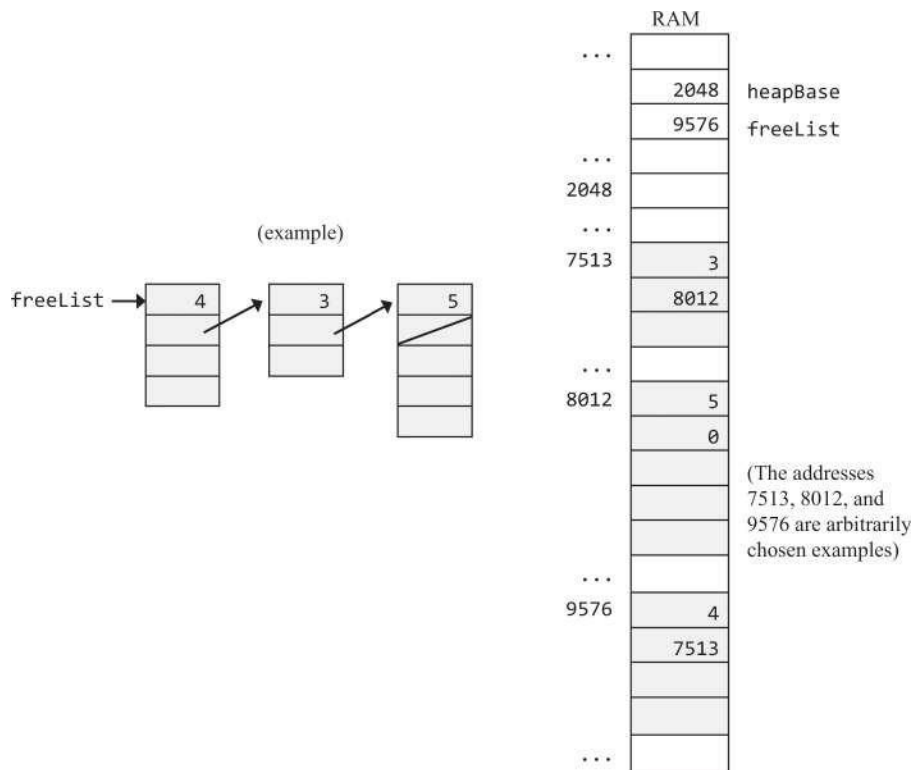
A las matrices Jack no se les asigna espacio en el montón en tiempo de compilación, sino en tiempo de ejecución, siempre y cuando se llame a la nueva función de la matriz . Tenga en cuenta que si `new` fuera un constructor y no una función, el compilador y el sistema operativo habrían asignado la nueva matriz a alguna dirección oscura en la RAM que no podemos controlar. Como muchos trucos clásicos, este truco funciona porque usamos la variable `array` sin inicializarla correctamente, como se hace normalmente cuando se usan matrices.

La matriz de memoria se puede declarar en el nivel de clase e inicializar mediante la función `Memory.init`. Una vez que se realiza este truco, la implementación de `Memory.peek` y `Memory.poke` se vuelve trivial.

**alloc, deAlloc:** Estas funciones pueden ser implementadas por cualquiera de los algoritmos que se muestran en las [figuras 12.5a](#) y [12.5b](#). Se puede usar *el mejor ajuste* o *el primer ajuste* para implementar `Memory.deAlloc`.

La asignación de VM estándar en la plataforma Hack (consulte la sección 7.4.1) especifica que la pila se asigne a las direcciones RAM 256 a 2047. Por lo tanto, el *montón* puede comenzar en la dirección 2048.

Para realizar la lista enlazada `freeList`, la clase `Memory` puede declarar y mantener una variable estática, `freeList`, como se ve en [la figura 12.12](#). Aunque `freeList` se inicializa con el valor de `heapBase` (2048), es posible que después de varias operaciones `alloc` y `deAlloc` `freeList` se convierta en otra dirección en la memoria, como se muestra en la figura.



**Figura 12.12** Vista lógica (izquierda) e implementación física (derecha) de una lista vinculada que admite la asignación dinámica de memoria.

En aras de la eficiencia, se recomienda escribir código Jack que administre la lista enlazada de freeList directamente en la RAM, como se ve en la [figura 12.12](#). La lista vinculada se puede inicializar mediante la función Memory.init.

## Pantalla

La clase Screen mantiene un *color actual* que utilizan todas las funciones de dibujo de la clase. El color actual se puede representar mediante una variable booleana estática.

**drawPixel:** Dibujar un píxel en la pantalla se puede hacer usando Memory.peek y Memory.poke. El mapa de memoria de pantalla de la plataforma Hack especifica que el píxel en la columna *col* y la fila ( $0 \leq col \leq 511, 0 \leq row \leq 255$ ) de fila se asigna al  $col \% 16$  bit de ubicación de memoria  $16384 + row \cdot 32 + col / 16$ . Dibujar un solo píxel requiere cambiar un solo bit en la palabra a la que se accede (y solo ese bit).

**drawLine:** El algoritmo básico de la [figura 12.7](#) puede conducir potencialmente a un desbordamiento. Sin embargo, la versión mejorada del algoritmo elimina el problema.

Algunos aspectos del algoritmo deben generalizarse para dibujar líneas que se extiendan a cuatro direcciones posibles. Recuerde que el origen de la pantalla (coordenadas (0,0)) se encuentra en la esquina superior izquierda. Por lo tanto, algunas de las direcciones y las operaciones más/menos especificadas en el algoritmo deben modificarse mediante la implementación de drawLine .

Los casos especiales pero frecuentes de dibujar líneas rectas, es decir, cuándo  $dx = 0$   $dy = 0$ , no deben ser manejados por este algoritmo. Más bien, deberían beneficiarse de una implementación separada y optimizada.

**drawCircle:** El algoritmo que se muestra en la [figura 12.8](#) puede conducir potencialmente a un desbordamiento. Limitar los radios de los círculos a un máximo de 181 es una solución razonable.

## Salida

La clase Output es una biblioteca de funciones para mostrar caracteres. La clase asume una pantalla orientada a caracteres que consta de 23 filas (indexadas 0 ... 22, de arriba a abajo) de 64 caracteres cada uno (indexado 0 ... 63, de izquierda a derecha). La ubicación del personaje superior izquierdo en la pantalla está indexada (0,0). Un cursor visible, implementado como un pequeño cuadrado relleno, indica dónde se mostrará el siguiente carácter. Cada carácter se muestra representando en la pantalla una imagen rectangular, de 11 píxeles de alto y 8 píxeles de ancho (que incluye márgenes para el espaciado entre caracteres y el interlineado). La colección de todas las imágenes de caracteres se denomina *fuentes*.

**Implementación de fuentes:** El diseño e implementación de una fuente para el conjunto de caracteres Hack (apéndice 5) es un trabajo pesado, que combina el juicio artístico y el trabajo de implementación de memoria. La fuente resultante es una colección de noventa y cinco imágenes rectangulares de mapa de bits, cada una de las cuales representa un carácter imprimible.

Las fuentes normalmente se almacenan en archivos externos que se cargan y utilizan en el paquete de dibujo de caracteres, según sea necesario. En Nand to Tetris, la fuente está incrustada en la clase OS Output. Para cada carácter imprimible, definimos una matriz que contiene el mapa de bits del carácter. La matriz consta de 11 elementos, cada uno correspondiente a una

fila de 8 píxeles. Específicamente, establecemos el valor de



Cada entrada de matriz  $j$  a un valor entero cuya representación binaria (bits) codifica los 8 píxeles que aparecen en la  $j$ -ésima fila del mapa de bits del carácter. También definimos una matriz estática de tamaño 127, cuyos valores de índice 32 ... 126 corresponden a los códigos de los caracteres imprimibles en el conjunto de caracteres Hack (entradas 0 ... 31 no se utilizan). Luego establecemos cada entrada de matriz  $i$  de esa matriz en la matriz de 11 entradas que representa la imagen de mapa de bits del carácter cuyo código de carácter es  $i$  (¿mencionamos la monotonía?).

Los materiales del proyecto 12 incluyen una clase de salida esquelética que contiene código Jack que lleva a cabo todo el trabajo de implementación descrito anteriormente. El código dado implementa la fuente de noventa y cinco caracteres, excepto por un carácter, cuyo diseño e implementación se deja como un ejercicio. Este código se puede activar mediante la función `Output.init`, que también puede inicializar el cursor.

**printChar:** muestra el carácter en la ubicación del cursor y avanza el cursor una columna hacia adelante. Para mostrar un carácter en la ubicación (*fila*, *columna*), donde  $0 \leq row \leq 22$  and  $0 \leq col \leq 63$ , escribimos el mapa de bits del carácter en el cuadro de píxeles que van desde  $11 \cdot row$  to  $11 \cdot row + 10$  y desde  $8 \cdot col$  to  $8 \cdot col + 7$ .

**printString:** se puede implementar mediante una secuencia de llamadas `printChar`.

**printInt:** se puede implementar convirtiendo el entero en una cadena y luego imprimiendo la cadena.

## Teclado

La organización de memoria de computadora Hack (consulte la sección 5.2.6) especifica que el mapa de memoria del *teclado* es un único registro de memoria de 16 bits ubicado en la dirección 24576.

**keyPressed:** Se puede implementar fácilmente usando `Memory.peek()`.

**readChar, readString:** Se puede implementar siguiendo los algoritmos de la [figura 12.10](#).

**readInt:** Se puede implementar leyendo una cadena y convirtiéndola en un `int` con un método `String`.

## Sistema

**wait:** Se supone que esta función espera un número determinado de milisegundos y regresa. Se puede implementar escribiendo un bucle que se ejecuta aproximadamente milisegundos de duración antes de terminar. Tendrá que cronometrar su computadora específica para obtener una espera de un milisegundo, ya que esta constante varía de una CPU a otra. Como resultado, su función `Sys.wait()` no será portátil. La función se puede hacer portátil ejecutando otra función de configuración que establece varias constantes que reflejan las especificaciones de hardware de la plataforma host, pero para Nand a Tetris esto no es necesario.

**halt:** Se puede implementar entrando en un bucle infinito.

**init:** De acuerdo con la especificación del lenguaje Jack (ver sección 9.2.2), un programa Jack es un conjunto de una o más clases. Una clase debe llamarse `Main` y esta clase debe incluir una función llamada `main`. Para comenzar a ejecutar un programa, se debe llamar a la función `Main.main`.

El sistema operativo también es una colección de clases Jack compiladas. Cuando la computadora se inicia, queremos comenzar a ejecutar el sistema operativo y hacer que comience a ejecutar el programa principal. Esta cadena de mando se implementa de la siguiente manera. De acuerdo con el mapeo estándar de VM en la plataforma Hack (sección 8.5.2), el traductor de VM escribe código de arranque (en lenguaje de máquina) que llama a la función del sistema operativo `Sys.init`. Este código de arranque se almacena en la ROM, comenzando en la dirección 0. Cuando reiniciamos la computadora, el contador del programa se establece en 0, el código de arranque comienza a ejecutarse y se llama a la función `Sys.init`.

Con eso en mente, `Sys.init` debería hacer dos cosas: llamar a todas las funciones de inicio de las otras clases del sistema operativo y luego llamar a `Main.main`.

A partir de este momento, el usuario está a merced del programa de aplicación, y el viaje de Nand a Tetris ha llegado a su fin. ¡Esperamos que hayas disfrutado del viaje!

---

## 12.4 Proyecto

**Objetivo:** Implementar el sistema operativo descrito en el capítulo.

**Contrato:** Implemente el sistema operativo en Jack y pruébelo utilizando los programas y escenarios de prueba que se describen a continuación. Cada programa de prueba utiliza un subconjunto de servicios del sistema operativo. Cada una de las clases de sistema operativo se puede implementar y probar unitariamente de forma aislada, en cualquier orden.

**Recursos:** La principal herramienta requerida es Jack, el lenguaje en el que desarrollará el sistema operativo. También necesitará el compilador Jack suministrado para compilar la implementación de su sistema operativo, así como los programas de prueba suministrados, también escritos en Jack. Por último, necesitará el emulador de VM suministrado, que es la plataforma en la que se ejecutarán las pruebas.

La carpeta `projects/12` incluye ocho archivos de clase de sistema operativo esqueléticos llamados

`Math.jack`, `String.jack`, `Array.jack`, `Memory.jack`, `Screen.jack`, `Output.jack`, `Keyboard.jack`, y `Sys.jack`. Cada archivo contiene las firmas de todas las subrutinas de clase. Su tarea es completar las implementaciones que faltan.

**El emulador de VM:** Los desarrolladores de sistemas operativos a menudo se enfrentan al siguiente dilema del huevo y la gallina: ¿Cómo podemos probar una clase de sistema operativo de forma aislada, si la clase usa los servicios de otras clases de sistema operativo que aún no se han desarrollado? Resulta que el emulador de VM está perfectamente posicionado para admitir pruebas unitarias del sistema operativo, una clase a la vez.

Específicamente, el emulador de VM presenta una versión ejecutable del sistema operativo, escrita en Java. Cuando se encuentra una llamada de comando de máquina virtual `foo` en el código de máquina virtual cargado, el emulador procede de la siguiente manera. Si existe una función de máquina virtual denominada `foo` en la base de código cargada, el emulador ejecuta su código de máquina virtual. De lo contrario, el emulador comprueba si `foo` es una de las funciones integradas del sistema operativo. Si es así, ejecuta la implementación incorporada de `foo`. Esta convención es ideal para respaldar la estrategia de prueba que ahora describimos.

---

## Plan de pruebas

La carpeta `projects/12` incluye ocho carpetas de prueba, denominadas `MathTest`, `MemoryTest`, ..., para probar cada una de las ocho clases de SO Matemáticas, Memoria, ....

Cada carpeta contiene un programa Jack, diseñado para probar (usando) los servicios de la clase de sistema operativo correspondiente. Algunas carpetas contienen scripts de prueba y comparan

y algunos contienen solo un archivo o archivos .jack. Para probar la implementación de la clase de sistema operativo Xxx.jack, puede proceder de la siguiente manera:

- Inspeccione el código *XxxTest/\*.jack* suministrado del programa de prueba. Comprenda qué servicios del sistema operativo se prueban y cómo se prueban.
- Coloque la clase de sistema operativo Xxx.jack que desarrolló en la carpeta *XxxTest*.
- Compile la carpeta usando el compilador Jack suministrado. Esto dará como resultado la traducción tanto de su archivo de clase del sistema operativo como el archivo .jack o los archivos del programa de prueba en los archivos .vm correspondientes, almacenados en la misma carpeta.
- Si la carpeta incluye un script de prueba .tst, cargue el script en el emulador de máquina virtual; de lo contrario, cargue la carpeta en el emulador de máquina virtual.
- Siga las pautas de prueba específicas que se detallan a continuación para cada clase de sistema operativo.

**Memoria, matriz, matemáticas:** las tres carpetas que prueban estas clases incluyen scripts de prueba y archivos de comparación. Cada script de prueba comienza con la carga de comandos. Este comando carga todos los archivos .vm de la carpeta actual en el emulador de VM. Los dos comandos siguientes de cada script de prueba crean un archivo de salida y cargan el archivo de comparación proporcionado. A continuación, el script de prueba procede a ejecutar varias pruebas, comparando los resultados de la prueba con los enumerados en el archivo de comparación. Su trabajo es asegurarse de que estas comparaciones terminen con éxito.

Tenga en cuenta que los programas de prueba proporcionados no comprenden una prueba completa de Memory.alloc y Memory.deAlloc. Una prueba completa de estas funciones de administración de memoria requiere inspeccionar los detalles de implementación interna que no son visibles en las pruebas a nivel de usuario. Si desea hacerlo, puede probar estas funciones mediante la depuración paso a paso e inspeccionando el estado de la RAM del host.

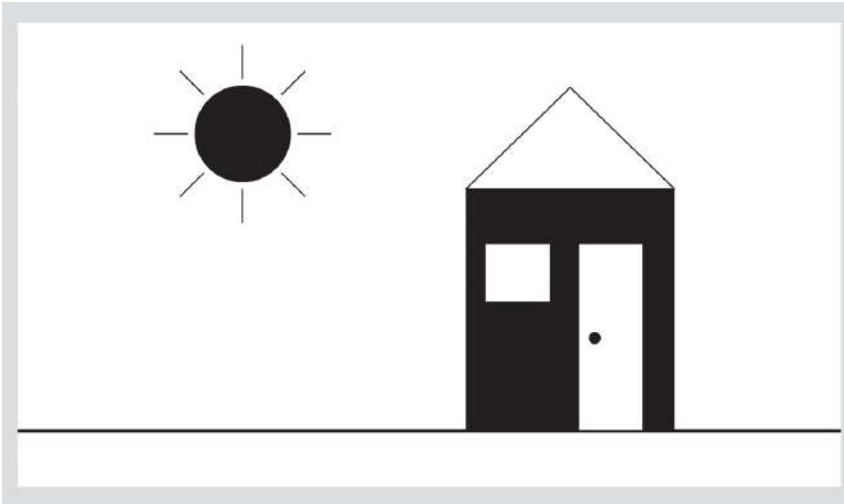
**Cadena:** La ejecución del programa de prueba suministrado debe producir el siguiente resultado:

```
new,appendChar: abcde
setInt: 12345
setInt: -32767
length: 5
charAt(2): 99
setCharAt(2,'-'): ab-de
eraseLastChar: ab-d
intValue: 456
intValue: -32123
backSpace: 129
doubleQuote: 34
newLine: 128
```

**Salida:** La ejecución del programa de prueba suministrado debe producir la siguiente salida:

```
A
0123456789
ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz
!#$%&'()*+,-./:;<=>?@[]^_`{|}~"
-12346789
B
C
D
```

**Pantalla:** La ejecución del programa de prueba suministrado debe producir el siguiente resultado:



**Teclado:** esta clase de sistema operativo se prueba mediante un programa de prueba que afecta la interacción usuario-programa. Para cada función de la clase Keyboard (keyPressed, readChar, readLine, readInt), el programa solicita al usuario que presione algunas teclas. Si la función OS se implementa correctamente y se presionan las teclas solicitadas, el programa imprime correctamente y procede a probar la siguiente función OS. De lo contrario, el programa repite la solicitud. Si todas las solicitudes finalizan correctamente, el programa imprime Prueba finalizada correctamente. En este punto, la pantalla debería mostrar el siguiente resultado:

```
keyPressed test:
Please press the 'Page Down' key
ok
readChar test:
(Verify that the pressed character is echoed to the screen)
Please press the number '3': 3
ok
readLine test:
(Verify echo and usage of 'backspace')
Please type 'JACK' and press enter: JACK
ok
readInt test:
(Verify echo and usage of 'backspace')
Please type '-32123' and press enter: -32123
ok
Test completed successfully
```

**Sistema**

El archivo `jack` proporcionado prueba la función `Sys.wait`. El programa solicita al usuario que presione una tecla (cualquier tecla) y espera dos segundos, usando una llamada a `Sys.wait`. Luego imprime un mensaje en la pantalla. Asegúrese de que el tiempo que transcurre entre la liberación de la clave y la aparición del mensaje impreso sea de unos dos segundos.

La función `Sys.init` no se prueba explícitamente. Sin embargo, recuerde que realiza todas las inicializaciones necesarias del sistema operativo y, a continuación, llama a la función `Main.main` de cada programa de prueba. Por lo tanto, podemos suponer que nada funcionará correctamente a menos que `Sys.init` se implemente correctamente.

---

## Prueba completa

Después de probar con éxito cada clase de sistema operativo de forma aislada, pruebe toda la implementación de su sistema operativo utilizando el juego Pong presentado anteriormente en el libro. El código fuente de Pong está disponible en `projects/11/Pong`. Coloque sus ocho archivos `jack` del sistema operativo en la carpeta Pong y compile la carpeta usando el compilador Jack suministrado. A continuación, cargue la carpeta Pong en el emulador de VM, ejecute el juego y asegúrese de que funcione como se esperaba.

---

## 12.5 Perspectiva

Este capítulo presentó un subconjunto de servicios básicos que se pueden encontrar en la mayoría de los sistemas operativos. Por ejemplo, administrar la memoria, controlar dispositivos de E/S, proporcionar operaciones matemáticas no implementadas en hardware e implementar tipos de datos abstractos como la abstracción de cadenas. Hemos optado por llamar a esta biblioteca de software estándar un *sistema operativo* para reflejar sus dos funciones principales: encapsular los detalles sangrientos del hardware, las omisiones y las idiosincrasias en servicios de software transparentes, y permitir que los compiladores y los programas de aplicación utilicen estos servicios a través de interfaces limpias. Sin embargo, la brecha entre lo que hemos llamado un sistema operativo y los sistemas operativos de potencia industrial sigue siendo amplia.

Para empezar, nuestro sistema operativo carece de algunos de los servicios básicos más estrechamente asociados con los sistemas operativos.



Por ejemplo, nuestro sistema operativo no admite subprocesos múltiples ni multiprocesamiento; En contraste, el núcleo de la mayoría de los

Operating Systems se dedica exactamente a eso. Nuestro sistema operativo no admite dispositivos de almacenamiento masivo; Por el contrario, el almacén de datos principal que manejan los sistemas operativos es una abstracción del sistema de archivos. Nuestro sistema operativo no cuenta con una interfaz de línea de comandos (como en un shell de Unix) ni una interfaz gráfica que consta de ventanas y menús. Por el contrario, esta es la interfaz del sistema operativo que los usuarios esperan ver e interactuar. Muchos otros servicios que se encuentran comúnmente en los sistemas operativos no están presentes en nuestro sistema operativo, como seguridad, comunicación y más.

Otra diferencia notable radica en la forma liberal en que se invocan las operaciones de nuestro sistema operativo. Algunas operaciones del sistema operativo, por ejemplo, mirar y empujar, le dan al programador acceso completo a los recursos de la computadora host. Claramente, el uso inadvertido o malicioso de tales funciones puede causar estragos. Por lo tanto, muchos servicios del sistema operativo se consideran privilegiados y acceder a ellos requiere un mecanismo de seguridad que es más elaborado que una simple llamada a una función. Por el contrario, en la plataforma Hack, no hay diferencia entre el código del sistema operativo y el código de usuario, y los servicios del sistema operativo se ejecutan en el mismo modo de usuario que el de los programas de aplicación.

En términos de eficiencia, los algoritmos que presentamos para la multiplicación y la división eran estándar. Estos algoritmos, o variantes de los mismos, generalmente se implementan en hardware en lugar de en software. El tiempo de ejecución de estos algoritmos es  $O(n)$  operaciones de suma. Dado que la adición de dos números de  $n$  bits requiere operaciones de  $O(n)$  bits (puertas en hardware), estos algoritmos terminan requiriendo operaciones de  $O(n^2)$  bits. Existen algoritmos de multiplicación y división cuyo tiempo de ejecución es asintóticamente significativamente más rápido que  $O(n^2)$ , y para un gran número de bits estos algoritmos son más eficientes. De manera similar, las versiones optimizadas de las operaciones geométricas que presentamos, como el dibujo lineal y el dibujo circular, a menudo se implementan en hardware especial de aceleración de gráficos.

Como todos los sistemas de hardware y software desarrollados en Nand to Tetris, nuestro objetivo no es proporcionar una solución completa que satisfaga todos los deseos y necesidades. Más bien, nos esforzamos por construir una implementación funcional y una comprensión sólida de la base del sistema, y luego proponemos formas de extenderlo aún más. Algunos de estos proyectos de extensión opcionales se mencionan en el

siguiente y último capítulo del libro.

---

## 13 Más diversión para llevar

No dejaremos de explorar, y al final llegaremos a donde empezamos, y conoceremos el lugar por primera vez.

—T. S. Eliot (1888–1965)

¡Felicidades! Hemos terminado la construcción de un sistema informático completo, partiendo de los primeros principios. Esperamos que hayas disfrutado de este viaje. Permítanos, los autores de este libro, compartir un secreto con usted: sospechamos que disfrutamos aún más escribiendo el libro. Después de todo, tenemos que diseñar este sistema informático, y el diseño es a menudo la parte "más divertida" de cada proyecto. Estamos seguros de que a algunos de ustedes, estudiantes aventureros, les gustaría participar en esa acción de diseño. Tal vez le gustaría mejorar la arquitectura; Tal vez tenga ideas para agregar nuevas funciones aquí y allá; Tal vez imagines un sistema más amplio. Y luego, tal vez, quieras estar en el asiento del navegador y decidir a dónde ir, no solo cómo llegar allí.

Casi todos los aspectos del sistema Jack/Hack se pueden mejorar, optimizar o ampliar. Por ejemplo, el lenguaje ensamblador, el lenguaje Jack y el sistema operativo se pueden modificar y ampliar reescribiendo partes de sus respectivas implementaciones de ensamblador, compilador y sistema operativo. Es probable que otros cambios requieran también modificar las herramientas de software suministradas. Por ejemplo, si cambia la especificación de hardware o la especificación de máquina virtual, es probable que desee modificar los emuladores respectivos. O, si desea agregar más dispositivos de entrada o salida a la computadora Hack, probablemente necesitaría modelarlos escribiendo nuevos chips incorporados.

Con el fin de permitir una flexibilidad total de modificación y extensión, pusimos a disposición del público el código fuente de todas las herramientas suministradas. Todos nuestros

el código es 100 por ciento Java, excepto algunos de los archivos por lotes utilizados para iniciar el software en algunas plataformas. El software y su documentación están disponibles en [www.nand2tetris.org](http://www.nand2tetris.org). Le invitamos a modificar y ampliar todas nuestras herramientas como lo considere deseable para su última idea, y luego compartirlas con otros, si lo desea. Esperamos que nuestro código esté escrito y documentado lo suficientemente bien como para que las modificaciones sean una experiencia satisfactoria. En particular, deseamos mencionar que el simulador de hardware suministrado tiene una interfaz simple y bien documentada para agregar nuevos chips incorporados. Esta interfaz se puede utilizar para ampliar la plataforma de hardware simulada con, por ejemplo, almacenamiento masivo o dispositivos de comunicaciones.

Si bien ni siquiera podemos comenzar a imaginar cuáles pueden ser sus mejoras de diseño, podemos esbozar brevemente algunas de las que estábamos pensando.

---

## Realizaciones de hardware

Los módulos de hardware presentados en el libro se implementaron en HDL o como módulos de software ejecutables suministrados. De hecho, así es como se diseña realmente el hardware. Sin embargo, en algún momento, los diseños HDL generalmente se comprometen con el silicio, convirtiéndose en computadoras "reales". ¿No sería bueno hacer que Hack o Jack se ejecuten en una plataforma de hardware real hecha de átomos en lugar de bits?

Se pueden adoptar varios enfoques diferentes para lograr este objetivo. En un extremo, puede intentar implementar la plataforma Hack en una placa FPGA. Esto requeriría reescribir todas las definiciones de chips utilizando un lenguaje de descripción de hardware convencional y luego lidiar con los problemas de implementación relacionados con la realización de la RAM, la ROM y los dispositivos de E / S en la placa host. Uno de estos proyectos opcionales paso a paso, desarrollado por Michael Schröder, se menciona en el [sitio web de www.nand2tetris.org](http://www.nand2tetris.org). Otro enfoque extremo puede ser intentar emular Hack, la VM o incluso la plataforma Jack en un dispositivo de hardware existente como un teléfono celular. Parece que cualquier proyecto de este tipo querría reducir el tamaño de la pantalla Hack para mantener el costo de los recursos de hardware razonable.

---

## **Mejoras de hardware**

Aunque Hack es un *ordenador de programa almacenado*, el programa que ejecuta debe estar prealmacenado en su dispositivo ROM. En la arquitectura actual de Hack, no hay forma de cargar otro programa en la computadora, excepto para simular el reemplazo de todo el chip ROM físico.

Agregar una *capacidad de programa de carga* de manera equilibrada probablemente implicaría cambios en varios niveles de la jerarquía. El hardware de Hack debe modificarse para permitir que los programas cargados residan en la RAM grabable en lugar de en la ROM existente. Probablemente se debería agregar algún tipo de almacenamiento permanente, como un chip de almacenamiento masivo incorporado, al hardware para permitir el almacenamiento de programas. El sistema operativo debe ampliarse para manejar este dispositivo de almacenamiento permanente, así como una nueva lógica para cargar y ejecutar programas. En este punto, un shell de interfaz de usuario del sistema operativo sería útil, proporcionando comandos de administración de archivos y programas.

---

## Idiomas de alto nivel

Como todos los profesionales, los programadores tienen fuertes sentimientos sobre sus herramientas, lenguajes de programación, y les gusta personalizarlas. Y, de hecho, el lenguaje de Jack, que deja mucho que desear, puede mejorarse significativamente. Algunos cambios son simples, otros son más complicados y algunos, como agregar herencia, probablemente también requerirían modificar la especificación de la máquina virtual.

Otra opción es realizar más lenguajes de alto nivel a través de la plataforma Hack. Por ejemplo, ¿qué tal implementar Scheme?

---

## Optimización

Nuestro viaje de Nand a Tetris ha eludido casi por completo los problemas de optimización (excepto por el sistema operativo, que introdujo algunas medidas de eficiencia). La optimización es un gran campo de juego para los piratas informáticos. Puede comenzar con optimizaciones locales en el hardware o en el compilador, pero la mejor inversión vendrá de la optimización del traductor de VM. Por ejemplo, es posible que desee reducir el tamaño del código ensamblador generado y hacer que

más eficiente. Las optimizaciones ambiciosas a una escala más global implicarán cambiar las especificaciones del lenguaje de máquina o el lenguaje de VM.

---

## **Comunicaciones**

¿No sería bueno conectar la computadora Hack a Internet? Esto podría hacerse agregando un chip de comunicación incorporado al hardware y escribiendo una clase de sistema operativo para lidiar con él y para manejar protocolos de comunicación de nivel superior. Algunos otros programas necesitarían hablar con el chip de comunicación incorporado, proporcionando una interfaz a Internet. Por ejemplo, un navegador web que habla HTTP en Jack parece un proyecto factible y digno.

Estos son algunos de nuestros picores de diseño, ¿cuáles son los tuyos?



# Apéndice 1: Síntesis de funciones booleanas

Por la lógica probamos, por la intuición descubrimos.

—Henri Poincaré (1854–1912)

En el capítulo 1 hicimos las siguientes afirmaciones, sin pruebas:

- Dada una verdad mesa representación de un  
Booleano función Nosotros puede sintetizar a  
partir de él una expresión booleana que realiza la función.
- Cualquier función booleana se puede expresar utilizando solo los  
operadores And, Or y Not.
- Cualquier función booleana se puede expresar usando solo operadores  
Nand.

Este apéndice proporciona pruebas de estas afirmaciones y muestra que están interrelacionadas. Además, el apéndice ilustra el proceso mediante el cual las expresiones booleanas se pueden simplificar usando álgebra booleana.

---

## Referencia 1.1 Álgebra booleana

Los operadores booleanos And, Or y Not tienen propiedades algebraicas útiles. Presentamos brevemente algunas de estas propiedades, señalando que sus pruebas se pueden derivar fácilmente de las tablas de verdad relevantes enumeradas en la [figura 1.1](#) del capítulo 1.

Commutative laws:  $x \text{ And } y = y \text{ And } x$

$$x \text{ Or } y = y \text{ Or } x$$

Associative laws:  $x \text{ And } (y \text{ And } z) = (x \text{ And } y) \text{ And } z$

$$x \text{ Or } (y \text{ Or } z) = (x \text{ Or } y) \text{ Or } z$$

Distributive laws:  $x \text{ And } (y \text{ Or } z) = (x \text{ And } y) \text{ Or } (x \text{ And } z)$

$$x \text{ Or } (y \text{ And } z) = (x \text{ Or } y) \text{ And } (x \text{ Or } z)$$

De Morgan laws:  $\text{Not}(x \text{ And } y) = \text{Not}(x) \text{ Or } \text{Not}(y)$

$$\text{Not}(x \text{ Or } y) = \text{Not}(x) \text{ And } \text{Not}(y)$$

Idempotent laws:  $x \text{ And } x = x$

$$x \text{ Or } x = x$$

Estas leyes algebraicas se pueden usar para simplificar funciones booleanas. Por ejemplo, considere la función  $\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y))$ . ¿Podemos reducirlo a una forma más simple? Intentemos ver qué se nos ocurre:

$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y)) =$  // by De Morgan law:

$\text{Not}(\text{Not}(x) \text{ And } (\text{Not}(x) \text{ And } \text{Not}(y))) =$  // by the associative law:

$\text{Not}((\text{Not}(x) \text{ And } \text{Not}(x)) \text{ And } \text{Not}(y)) =$  // by the idempotent law:

$\text{Not}(\text{Not}(x) \text{ And } \text{Not}(y)) =$  // by De Morgan law:

$\text{Not}(\text{Not}(x)) \text{ Or } \text{Not}(\text{Not}(y)) =$  // by double negation:

$x \text{ Or } y$

Las simplificaciones booleanas como la que acabamos de ilustrar tienen importantes implicaciones prácticas. Por ejemplo, la expresión booleana original  $\text{Not}(\text{Not}(x) \text{ And } \text{Not}(x \text{ Or } y))$  se puede implementar en hardware usando cinco puertas lógicas, mientras que la expresión simplificada  $x \text{ Or } y$  se puede implementar usando una sola puerta lógica. Ambas expresiones ofrecen la misma funcionalidad, pero la última es cinco veces más eficiente en términos de costo, energía y velocidad de cálculo.

Reducir una expresión booleana a una más simple es un arte que requiere experiencia y perspicacia. Hay varias herramientas y técnicas de reducción disponibles, pero el problema sigue siendo un desafío. En general, reducir una expresión booleana a su forma más simple es un *problema NP-difícil*.

## A1.2 Sintetizar funciones booleanas

Dada una tabla de verdad de una función booleana, ¿cómo podemos construir o sintetizar una expresión booleana que represente esta función? Y, ahora que lo pienso, ¿tenemos *la garantía* de que cada función booleana representada por una tabla de verdad también puede ser representada por una expresión booleana?

Estas preguntas tienen respuestas muy satisfactorias. Primero, sí: cada función booleana se puede representar mediante una expresión booleana. Además, hay un algoritmo constructivo para hacer precisamente eso. Para ilustrarlo, consulte la [figura A1.1](#) y concéntrese en sus cuatro columnas más a la izquierda. Estas columnas especifican una definición de tabla de verdad de alguna función de tres variables  $f(x,y,z)$ . Nuestro objetivo es sintetizar a partir de estos datos una expresión booleana que represente esta función.

| $x$ | $y$ | $z$ | $f(x,y,z)$ | $f_3(x,y,z)$ | $f_5(x,y,z)$ | $f_7(x,y,z)$ |
|-----|-----|-----|------------|--------------|--------------|--------------|
| 0   | 0   | 0   | 0          | 0            | 0            | 0            |
| 0   | 0   | 1   | 0          | 0            | 0            | 0            |
| 0   | 1   | 0   | 1          | 1            | 0            | 0            |
| 0   | 1   | 1   | 0          | 0            | 0            | 0            |
| 1   | 0   | 0   | 1          | 0            | 1            | 0            |
| 1   | 0   | 1   | 0          | 0            | 0            | 0            |
| 1   | 1   | 0   | 1          | 0            | 0            | 1            |
| 1   | 1   | 1   | 0          | 0            | 0            | 0            |

$$f_3(x,y,z) = \text{Not}(x) \text{ And } y \text{ And Not}(z)$$

$$f_5(x,y,z) = x \text{ And Not}(y) \text{ And Not}(z)$$

$$f_7(x,y,z) = x \text{ And } y \text{ And Not}(z)$$

$$f(x,y,z) = f_3(x,y,z) \text{ Or } f_5(x,y,z) \text{ Or } f_7(x,y,z)$$

**Figura A1.1** Sintetizar una función booleana a partir de una tabla de verdad (ejemplo).

Describiremos el algoritmo de síntesis siguiendo sus pasos en este ejemplo en particular. Comenzamos centrándonos solo en las filas de la tabla de verdad en las que el valor de la función es 1. En la función que se muestra en la [figura A1.1](#), esto sucede en las filas 3, 5 y 7. Para cada una de estas filas  $i$ , definimos un valor booleano

función  $f_i$  que devuelve 0 para todos los valores de la variable excepto para la variable valores en fila  $Y_0$ , para el cual la función devuelve 1. La tabla de verdad en [Figura A1.1](#) produce tres de estas funciones, cuyas definiciones de tabla de verdad se enumeran en las tres columnas más a la derecha de la tabla. Cada una de estas funciones  $f_i$  se puede representar mediante una conjunción (Y-ing) de tres términos, un término para cada variable  $x$ ,  $y$  y  $z$ , cada uno de los cuales es la variable o su negación, dependiendo de si el valor de esta variable es 1 o 0 en fila  $Y_0$ . Esta construcción produce las tres funciones  $f_3$ ,  $f_5$  y  $f_7$ , que aparece en la parte inferior de la tabla. Dado que estas funciones describen los únicos casos en los que la función booleana  $f$  evalúa a 1, concluimos que  $f$  se puede representar mediante la expresión

$$f_{\text{booleana}}(x, y, z) = f_3(x, y, z) \text{ O } f_5(x, y, z) \text{ O } f_7(x, y, z).$$

Deletreándolo:  
 $(\text{No}(x) \text{ y } y \text{ Y no}(z)) \text{ O } (x \text{ Y no}(y) \text{ y no}(z)) \text{ O } (x \text{ Y } y \text{ Y No}(z)).$

Evitando la tediosa formalidad, este ejemplo sugiere que cualquier función booleana puede ser representada sistemáticamente por una expresión booleana que tiene una estructura muy específica: es la disyunción (Or-ing) de todas las funciones conjuntivas (And-ing)  $f_i$  cuya construcción se acaba de describir. Esta expresión, que es la versión booleana de una suma de productos, a veces se denomina *forma normal disyuntiva* (DNF) de la función.

Tenga en cuenta que si la función tiene muchas variables  $y$ , por lo tanto, la tabla de verdad tiene exponencialmente muchas filas, el DNF resultante puede ser largo y engorroso. En este punto, el álgebra booleana y varias técnicas de reducción pueden ayudar a transformar la expresión en una representación más eficiente y viable.

---

### A1.3 El poder expresivo de Nand

Como sugiere nuestro *título de Nand to Tetris*, todas las computadoras se pueden construir usando nada más que puertas Nand. Hay dos formas de respaldar esta afirmación. Una es construir una computadora solo a partir de puertas Nand, que es exactamente lo que hacemos en la parte I del libro. Otra forma es proporcionar una prueba formal, que es lo que haremos a continuación.

**Lema 1:** cualquier función booleana se puede representar mediante una expresión booleana que contenga solo los operadores And, Or y Not.

*Prueba:* Se puede usar cualquier función booleana para generar una tabla de verdad correspondiente. Y, como acabamos de mostrar, cualquier tabla de verdad se puede usar para sintetizar un DNF, que es un Or-ing de Y-ings de variables y sus negaciones. De ello se deduce que cualquier función booleana se puede representar mediante una expresión booleana que contenga solo los operadores And, Or y Y.

Para apreciar la importancia de este resultado, considere el número infinito de funciones que se pueden definir sobre *números enteros* (en lugar de números binarios). Hubiera sido bueno si cada una de estas funciones pudiera representarse mediante una expresión algebraica que involucrara solo suma, multiplicación y negación. Resulta que la gran mayoría de las funciones enteras, por ejemplo,  $f(x) = 2x$  para  $x \neq 7$  y  $f(7) = 312$ , no se pueden expresar usando una forma algebraica cercana. Sin embargo, en el mundo de los *números binarios*, debido al número finito de valores que cada variable puede asumir (0 o 1), tenemos esta atractiva propiedad de que cada función booleana se *puede* expresar usando nada más que los operadores Y, O y No. La implicación práctica es inmensa: cualquier computadora puede construirse a partir de nada más que puertas Y, Or y Not.

Pero, ¿podemos hacerlo mejor que esto?

**Lema 2:** Cualquier función booleana se puede representar mediante una expresión booleana que contenga solo los operadores Not y Y.

*Prueba:* De acuerdo con la ley de De Morgan, el operador Or se puede expresar utilizando los operadores Not y Y. Combinando este resultado con el lema 1, obtenemos la prueba.

Empujando nuestra suerte más allá, ¿podemos hacerlo mejor que esto?

**Teorema:** Cualquier función booleana se puede representar mediante una expresión booleana que contenga solo operadores Nand.

*Demostración:* Una inspección de la tabla de verdad Nand (la penúltima fila en la [figura 1.2](#) en el capítulo 1) revela las siguientes dos propiedades:

- $\text{No}(x) = \text{Nand}(x, x)$

En palabras: Si establece las variables  $x$  e  $y$  de la función Nand en el mismo valor (0 o 1), la función se evalúa como la negación de ese

valor.

- $Y(x, y) = \text{No}(\text{Nand}(x, y))$

Es fácil demostrar que las tablas de verdad de ambos lados de la ecuación son idénticas. Y acabamos de demostrar que Not se puede expresar usando Nand.

Combinando estos dos resultados con el lema 2, obtenemos que cualquier función booleana puede ser representada por una expresión booleana que contenga solo operadores Nand.

Este notable resultado, que bien puede llamarse el teorema fundamental del diseño lógico, estipula que las computadoras se pueden construir a partir de un solo átomo: una puerta lógica que realiza la función Nand. En otras palabras, si tenemos tantas puertas Nand como queramos, podemos conectarlas en patrones de activación que implementen cualquier función booleana dada: todo lo que tenemos que hacer es averiguar el cableado correcto.

De hecho, la mayoría de las computadoras actuales se basan en infraestructuras de hardware que consisten en miles de millones de puertas Nand (o puertas Nor, que tienen propiedades generativas similares). En la práctica, sin embargo, no tenemos que limitarnos solo a las puertas Nand. Si los ingenieros eléctricos y los físicos pueden crear implementaciones físicas eficientes y de bajo costo de otras puertas lógicas elementales, las usaremos felizmente directamente como bloques de construcción primitivos. Esta observación pragmática no le quita nada a la importancia del teorema.

# Apéndice 2: Lenguaje de descripción de hardware

La inteligencia es la facultad de hacer objetos artificiales, especialmente herramientas para hacer herramientas.

—Henry Bergson (1859–1941)

Este apéndice tiene dos partes principales. Las secciones A2.1 a A2.5 describen el lenguaje HDL utilizado en el libro y en los proyectos. La sección A2.6, denominada Guía de supervivencia de HDL, proporciona un conjunto de consejos esenciales para completar los proyectos de hardware con éxito.

Un lenguaje de descripción de hardware (HDL) es un formalismo para definir *chips*: objetos cuyas *interfaces* consisten en pines de entrada y salida que transportan señales binarias y cuyas *implementaciones* son arreglos conectados de otros chips de nivel inferior. Este apéndice describe el HDL que usamos en Nand a Tetris. El capítulo 1 (en particular, la sección 1.3) proporciona antecedentes esenciales que son un requisito previo para este apéndice.

---

## A2.1 Conceptos básicos de HDL

El HDL utilizado en Nand to Tetris es un lenguaje simple, y la mejor manera de aprenderlo es jugar con programas HDL usando el simulador de hardware suministrado. Recomendamos comenzar a experimentar tan pronto como pueda, comenzando con el siguiente ejemplo.

**Ejemplo:** Supongamos que tenemos que comprobar si tres variables de 1 bit  $a$ ,  $b$ ,  $c$  tienen el mismo valor. Una forma de comprobar esta igualdad de tres vías es evaluar la función booleana  $\neg((a \neq b) \vee (b \neq c))$ . Teniendo en cuenta que el binario

operador *no igual* se puede realizar usando una puerta Xor, podemos implementar esta función usando el programa HDL que se muestra en la [figura A2.1](#).

|                |   |                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| interface      | { | <pre> /** If the three given bits are equal, sets out to 1; else sets out to 0. */ CHIP Eq3 {   IN  a, b, c;   OUT out;   PARTS:     Xor(a=a, b=b, out=neq1);           // Xor(a,b) → neq1     Xor(a=b, b=c, out=neq2);           // Xor(b,c) → neq2     Or (a=neq1, b=neq2, out=outOr);     // Or(neq1,neq2) → outOr     Not(in=outOr, out=out);             // Not(outOr) → out   } </pre> |
| implementation | { |                                                                                                                                                                                                                                                                                                                                                                                              |

**Figura A2.1** Ejemplo de programa HDL.

La implementación de Eq3.hdl utiliza cuatro *partes de chip*: dos puertas Xor, una puerta Or y una puerta Not. Para realizar la lógica expresada por  $\neg((a \neq b) \vee (b \neq c))$ , el programador HDL conecta las partes del chip creando y nombrando tres *pines internos*: neq1, neq2 y outOr.

A diferencia de los pines internos, que se pueden crear y nombrar a voluntad, el programador HDL no tiene control sobre los nombres de los pines de entrada y salida. Estos normalmente son suministrados por los arquitectos de los chips y documentados en API dadas. Por ejemplo, en Nand to Tetris, proporcionamos *archivos auxiliares* para todos los chips que tienes que implementar. Cada archivo stub contiene la interfaz del chip, con una implementación faltante. El contrato es el siguiente: Se le permite hacer lo que quiera *según* la declaración PARTS; no se le permite cambiar nada *por encima* de la declaración PARTS.

En el ejemplo de la Ec3, sucede que las dos primeras entradas del chip Eq3 y las dos entradas de las partes del chip Xor y Or tienen los mismos nombres (a y b). Del mismo modo, la salida del chip Eq3 y la de la parte del chip Not tienen el mismo nombre (out). Esto conduce a enlaces como a=a, b=b, y out=out. Tales enlaces pueden parecer peculiares, pero ocurren con frecuencia en los programas HDL, y uno se acostumbra a ellos. Más adelante en el apéndice daremos una regla simple que aclara el significado de estos enlaces.

Es importante destacar que el programador no debe preocuparse por cómo se implementan las piezas del chip. Las partes del chip se utilizan como abstracciones de caja negra, lo que permite al programador concentrarse solo en cómo organizarlas juiciosamente en orden



para realizar la función de chip. Gracias a esta modularidad, los programas HDL pueden ser breves, legibles y susceptibles de pruebas unitarias.

Los chips basados en HDL como `Eq3.hdl` pueden probarse mediante un programa informático llamado *simulador de hardware*. Cuando le indicamos al simulador que evalúe un chip determinado, el simulador evalúa todas las partes del chip especificadas en su sección PARTES. Esto, a su vez, requiere evaluar *sus* piezas de chip de nivel inferior, y así sucesivamente. Este descenso recursivo puede dar lugar a una enorme jerarquía de piezas de chips que se expanden hacia abajo, hasta las puertas terminales Nand a partir de las cuales se fabrican todos los chips. Este costoso desglose se puede evitar utilizando *chips incorporados*, como explicaremos en breve.

**HDL es un lenguaje declarativo:** los programas HDL pueden verse como especificaciones textuales de diagramas de chips. Para cada *chipName* de chip que aparece en el diagrama, el programador escribe un *chipName (...)* en la sección PARTS del programa HDL . Dado que el lenguaje está diseñado para describir *conexiones* en lugar de *procesos*, el orden de las declaraciones PARTS es insignificante: siempre que las partes del chip estén conectadas correctamente, el chip funcionará como se indica. El hecho de que las declaraciones HDL se puedan reordenar sin afectar el comportamiento del chip puede parecer extraño para los lectores que están acostumbrados a la programación convencional. Recuerde: HDL no es un lenguaje de programación; es un lenguaje de especificación.

**Espacios en blanco, comentarios, convenciones de mayúsculas y minúsculas:** HDL distingue entre mayúsculas y minúsculas: `foo` y `Foo` representan dos cosas diferentes. Las palabras clave HDL se escriben en mayúsculas. Se ignoran los caracteres de espacio, los caracteres de nueva línea y los comentarios. Se admiten los siguientes formatos de comentarios:

```
// Comment to end of line
/* Comment until closing */
/** API documentation comment */
```

**Pines:** Los programas HDL cuentan con tres tipos de *pines*: pines de entrada, pines de salida y pines internos. Los últimos pines sirven para conectar las salidas de las piezas del chip a las entradas de otras partes del chip. De forma predeterminada, se supone que los pines son de un solo bit, con valores 0 o 1. Los pines de bus de varios bits también se pueden declarar

y utilizar, como se describe más adelante en este apéndice.

**Los nombres** de chips y pines pueden ser cualquier secuencia de letras y dígitos que no comiencen con un dígito (algunos simuladores de hardware no permiten el uso de guiones). Por convención, los nombres de chip y pin comienzan con una letra mayúscula y una letra minúscula, respectivamente. Para facilitar la lectura, los nombres pueden incluir letras mayúsculas, por ejemplo, `xorResult`. Los programas HDL se almacenan en archivos `.hdl`. El nombre del chip declarado en la instrucción HDL `CHIP Xxx` debe ser idéntico al prefijo del nombre de archivo `Xxx.hdl`.

**Estructura del programa:** Un programa HDL consta de una *interfaz* y una *implementación*. La interfaz consta de la documentación de la API del chip, el nombre del chip y los nombres de sus pines de entrada y salida. La implementación consta de las instrucciones debajo de la palabra clave `PARTS`. La estructura general del programa es la siguiente:

```
/** API documentation: what the chip does. */
CHIP ChipName {
    IN inputPin1, inputPin2, ... ;
    OUT outputPin1, outputPin2, ... ;
PARTS:
    // Here comes the implementation.
}
```

**Partes:** La implementación del chip es una secuencia desordenada de declaraciones chip-part, como se indica a continuación:

```
PARTS:
    chipPart(connection, ... , connection);
    chipPart(connection, ... , connection);
    ...
```

Cada *conexión* se especifica mediante el enlace  $pin1 = pin2$ , donde *pin1* y *pin2* son nombres de pin internos, de entrada, de salida o internos. Estas conexiones se pueden visualizar como "cables" que el programador de HDL crea y nombra, según sea necesario. Para cada "cable" que conecta *chipPart1* y *chipPart2* hay un pin interno que aparece dos veces en el programa HDL: una vez como *sumidero* en algunos

*chipPart1(...)*, y una vez como *fuentes* en algún otro *chipPart2 (...)* declaración. Por ejemplo, considere las siguientes instrucciones:

```
chipPart1(..., out = v,...);           // out of chipPart1 feeds the internal pin v
chipPart2(..., in = v, ...);          // in of chipPart2 is fed from v
chipPart3(..., in1 = v, ..., in2 = v,...); // in1 and in2 of chipPart3 are also fed from v
```

Los pines tienen fan-in 1 y fan-out ilimitado. Esto significa que un pin se puede alimentar desde una sola fuente, pero puede alimentar (a través de múltiples conexiones) uno o más pines en una o más partes del chip. En el ejemplo anterior, el pin *v* interno alimenta simultáneamente tres entradas. Este es el equivalente HDL de *las bifurcaciones en los diagramas* de chips.

**El significado de *a = a*:** Muchos chips en la plataforma Hack usan los mismos nombres de pines. Como se muestra en [la figura A2.1](#), esto lleva a declaraciones como *Xor (a=a, b=b, out=neq1)*. Las dos primeras conexiones alimentan las entradas *a* y *b* del chip implementado (*Eq3*) en las entradas *a* y *b* de la parte del chip *Xor*. La tercera conexión alimenta la salida de la pieza del chip *Xor* al pin interno *neq1*. Aquí hay una regla simple que ayuda a resolver las cosas: en cada declaración de parte de chip, el lado izquierdo de cada enlace = siempre denota un pin de entrada o salida *de la parte de chip*, y el lado derecho siempre denota un pin de entrada, salida o interno *del chip implementado*.

---

## Buses multibit A2.2

Cada entrada, salida o pin interno en un programa HDL puede ser un valor de un solo bit, que es el valor predeterminado, o un valor de varios bits, denominado *bus*.

**Numeración de bits y sintaxis de bus:** Los bits se numeran de derecha a izquierda, comenzando con 0. Por ejemplo, *sel=110*, implica que *sel[2]=1*, *sel[1]=1* y *sel[0]=0*.

**Pines de bus de entrada y salida:** Los anchos de bits de estos pines se especifican cuando se declaran en las instrucciones IN y OUT del chip. La sintaxis es  $x[n]$ , donde  $x$  y  $n$  declaran el nombre del pin y el ancho de bits, respectivamente.

**Pines de bus internos:** Los anchos de bits de los pines internos se deducen implícitamente de los enlaces en los que se declaran, de la siguiente manera,

```
chipPart1(..., x[i] = u, ...);  
chipPart2(..., x[i..j] = v, ...);
```

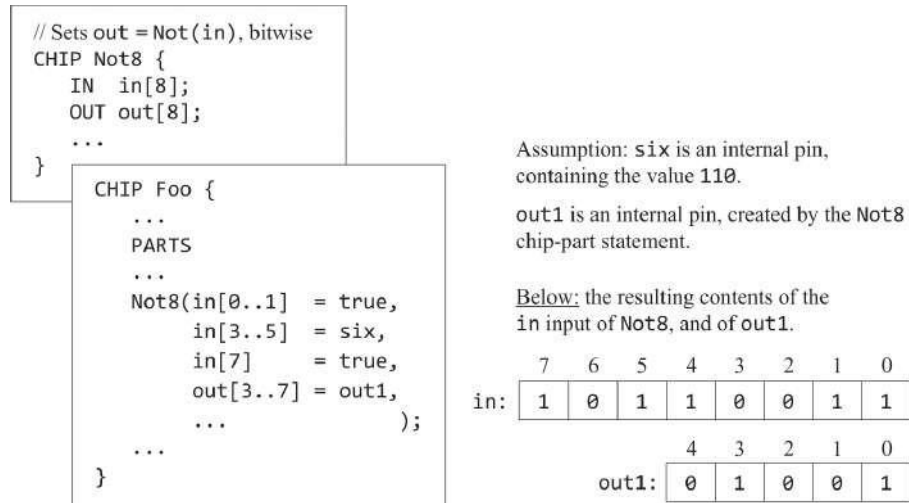
donde  $x$  es un pin de entrada o salida de la parte del chip. El primer enlace define  $u$  como un pin interno de un solo bit y establece su valor en  $x[i]$ . El segundo enlace define  $v$  como un pin de bus interno de bits de ancho  $j - i + 1$  y establece su valor en los bits indexados  $i$  a  $j$  (inclusive) del pin de bus  $x$ .

A diferencia de los pines de entrada y salida, los pines internos (como  $u$  y  $v$ ) no pueden estar subíndices. Por ejemplo,  $u[i]$  no está permitido.

**Buses verdadero/falso:** Las constantes true (1) y false (0) también se pueden usar para definir buses. Por ejemplo, supongamos que  $x$  es un pin de bus de 8 bits y considere esta instrucción:

```
chipPart(..., x[0..2] = true, ..., x[6..7] = true, ...);
```

Esta instrucción establece  $x$  en el valor 11000111. Tenga en cuenta que los bits no afectados se establecen de forma predeterminada en false (0). [La figura A2.2](#) da otro ejemplo.



**Figura A2.2** Buses en acción (ejemplo).

## Chips incorporados A2.3

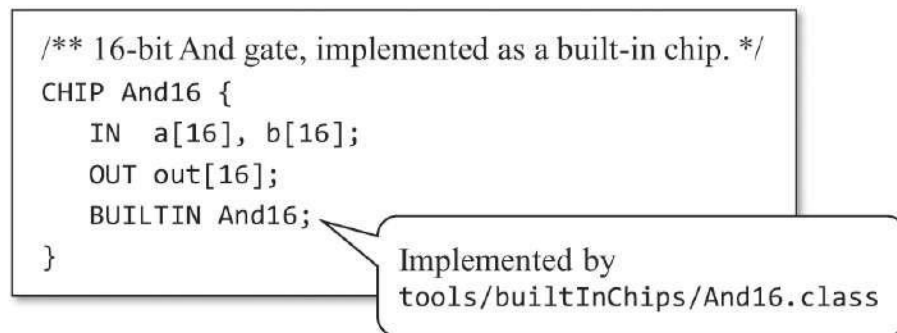
Los chips pueden tener una *implementación nativa*, escrita en HDL, o una implementación *integrada*, proporcionada por un módulo ejecutable escrito en un lenguaje de programación de alto nivel. Dado que el simulador de hardware de Nand a Tetris estaba escrito en Java, era conveniente realizar los chips incorporados como clases de Java. Por lo tanto, antes de construir, digamos, un chip Mux en HDL, el usuario puede cargar un chip Mux incorporado en el simulador de hardware y experimentar con él. El comportamiento del chip Mux incorporado es proporcionado por un archivo de clase Java llamado Mux.class, que forma parte del software del simulador.

La computadora Hack está hecha de unos treinta chips genéricos, enumerados en el apéndice 4. Dos de estos chips, Nand y DFF, se consideran *dados* o *primitivos*, similares a los axiomas en lógica. El simulador de hardware realiza chips dados invocando sus implementaciones integradas. Por lo tanto, en Nand to Tetris, Nand y DFF se pueden usar sin compilarlos en HDL.

Los proyectos 1, 2, 3 y 5 evolucionan en torno a la creación de implementaciones HDL de los chips restantes enumerados en el apéndice 4. Todos estos chips, excepto los chips de CPU y computadora, también tienen implementaciones incorporadas. Esto se hizo para facilitar la simulación del comportamiento, como se explica en el capítulo 1.

Los chips incorporados, una biblioteca de unos treinta *archivos chipName.class*, se suministran en la carpeta nand2tetris/tools/builtInChips de su ordenador. Los chips incorporados tienen interfaces HDL idénticas a las de los chips HDL normales.

Por lo tanto, cada archivo .class va acompañado de un archivo .hdl correspondiente que proporciona la interfaz de chip integrada. La Figura A2.3 muestra una definición típica de HDL de un chip incorporado.



```
/** 16-bit And gate, implemented as a built-in chip. */
CHIP And16 {
    IN  a[16], b[16];
    OUT out[16];
    BUILTIN And16;
}
```

Implemented by  
tools/builtInChips/And16.class

**Figura A2.3** Ejemplo de definición de chip incorporado.

Es importante recordar que el simulador de hardware suministrado es una herramienta de propósito general, mientras que la computadora Hack integrada en Nand to Tetris es una plataforma de hardware específica. El simulador de hardware se puede utilizar para construir puertas, chips y plataformas que no tienen nada que ver con Hack. Por lo tanto, cuando se discute la noción de chips incorporados, ayuda a ampliar nuestra perspectiva y describir su utilidad general para respaldar cualquier posible proyecto de construcción de hardware. En general, entonces, los chips incorporados brindan los siguientes servicios:

**Fundamento:** Los chips incorporados pueden proporcionar implementaciones suministradas de chips que se consideran *dados* o *primitivos*. Por ejemplo, en la computadora Hack, se dan Nand y DFF.

**Eficiencia:** Algunos chips, como las unidades RAM, constan de numerosos chips de nivel inferior. Cuando usamos tales chips como piezas de chips, el simulador de hardware tiene que evaluarlos. Esto se hace evaluando, de forma recursiva, todos los chips de nivel inferior a partir de los cuales están hechos. Esto da como resultado una simulación lenta e ineficiente. El uso de piezas de chip integradas en lugar de chips normales basados en HDL acelera considerablemente la simulación.

**Pruebas unitarias:** los programas HDL utilizan piezas de chip de forma abstracta, sin prestar atención a su implementación. Por lo tanto, al construir un nuevo chip, es

Siempre se recomienda utilizar piezas de chip incorporadas. Esta práctica mejora la eficiencia y minimiza los errores.

**Visualización:** Si el diseñador quiere permitir que los usuarios "vean" cómo funcionan los chips, y tal vez cambiar el estado interno del chip simulado de forma interactiva, puede proporcionar una implementación de chip incorporada que cuente con una interfaz gráfica de usuario. Esta GUI se mostrará cada vez que el chip incorporado se cargue en el simulador o se invoque como una pieza de chip. Excepto por estos efectos secundarios visuales, los chips potenciados por GUI se comportan y se pueden usar como cualquier otro chip. La sección A2.5 proporciona más detalles sobre los chips potenciados por GUI.

**Extensión:** Si desea implementar un nuevo dispositivo de entrada/salida o crear una nueva plataforma de hardware por completo (que no sea Hack), puede admitir estas construcciones con chips incorporados. Para obtener más información sobre el desarrollo de funcionalidades adicionales o nuevas, consulte el capítulo 13.

---

## Chips secuenciales A2.4

Las fichas pueden ser *combinadas* o *secuenciales*. Los chips combinados son independientes del tiempo: responden instantáneamente a los cambios en sus entradas. Los chips secuenciales dependen del tiempo, también llamados *sincronizados*: cuando un usuario o un script de prueba cambia las entradas de un chip secuencial, las salidas del chip pueden cambiar solo al comienzo de la siguiente *unidad de tiempo*, también llamada *ciclo*. El simulador de hardware efectúa la progresión del tiempo utilizando un reloj simulado.

**El reloj:** El reloj bifásico del simulador emite una secuencia infinita de valores denotados  $0, 0+, 1, 1+, 2, 2+, 3, 3+$ , y así sucesivamente. La progresión de esta serie temporal discreta está controlada por dos comandos de simulador llamados tick y tock. Un tic mueve el valor del reloj de  $t$  a  $t+$ , y un toque de  $t+$  a  $t+$  para  $t+1$ , traer la siguiente unidad de tiempo. El *tiempo real* transcurrido durante este período es irrelevante para fines de simulación, ya que el tiempo simulado es controlado por el usuario o por un script de prueba, como se indica a continuación.

Primero, cada vez que se carga un chip secuencial en el simulador, la GUI habilita un botón en forma de reloj (atenuado al simular combinacional



chips). Un clic en este botón (un tic) finaliza la primera fase del ciclo del reloj, y un clic posterior (un tac) finaliza la segunda fase del ciclo, dando lugar a la primera fase del siguiente ciclo, y así sucesivamente.

Alternativamente, se puede ejecutar el reloj desde un script de prueba. Por ejemplo, la secuencia de comandos de secuencias de comandos `repeat n {tick, tock, output}` indica al simulador que avance el reloj  $n$  unidades de tiempo e imprima algunos valores en el proceso. El Apéndice 3 documenta el *Lenguaje de descripción de pruebas* (TDL) que presenta estos comandos.

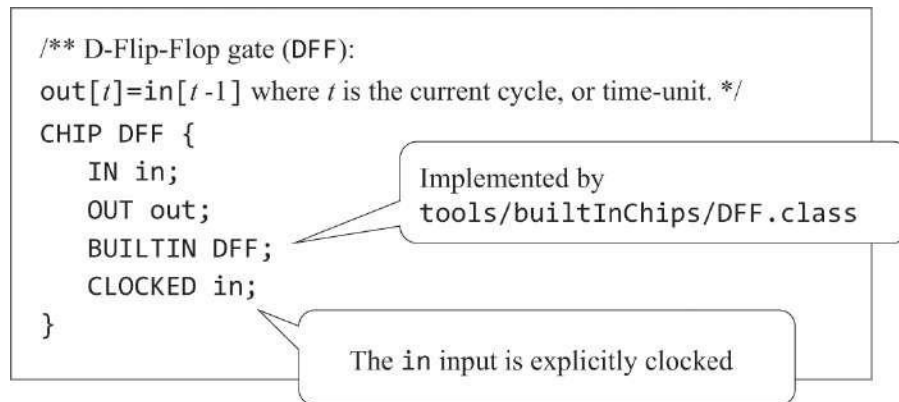
Las unidades de tiempo bifásicas generadas por el reloj regulan las operaciones de todas las partes secuenciales del chip implementado. Durante la primera fase de la unidad de tiempo (tick), las entradas de cada parte secuencial del chip afectan el estado interno del chip, de acuerdo con la lógica del chip. Durante la segunda fase de la unidad de tiempo (tock), las salidas del chip se ajustan a los nuevos valores. Por lo tanto, si miramos un chip secuencial "desde el exterior", vemos que sus pines de salida se estabilizan a nuevos valores solo en el momento, en el punto de transición entre dos unidades de tiempo consecutivas.

Reiteramos que las fichas combinadas son completamente ajenas al reloj. En Nand to Tetris, todas las puertas lógicas y los chips construidos en los capítulos 1 y 2, hasta e incluyendo el ALU, son combinables. Todos los registros y unidades de memoria construidos en el capítulo 3 son secuenciales. De forma predeterminada, las fichas son combinables; Un chip puede volverse *secuencial* explícita o implícitamente, de la siguiente manera.

**Chips secuenciales incorporados:** Un *chip incorporado* puede declarar su dependencia del reloj explícitamente, usando la declaración,

```
CLOCKED pin, pin, ..., pin;
```

donde cada *pin* es uno de los pines de entrada o salida del chip. La inclusión de un pin de entrada  $x$  en la lista CLOCKED estipula que los cambios en  $x$  deben afectar las salidas del chip solo al comienzo de la siguiente unidad de tiempo. La inclusión de un pin de salida  $x$  en la lista CLOCKED estipula que los cambios en cualquiera de las entradas del chip deben afectar a  $x$  solo al comienzo de la siguiente unidad de tiempo. [La Figura A2.4](#) presenta la definición del chip secuencial más básico e integrado en la plataforma Hack: el DFF.



**Figura A2.4** Definición de DFF .

Es posible que solo algunos de los pines de entrada o salida de un chip se declaren como sincronizados. En ese caso, los cambios en los pines de entrada no sincronizados afectan instantáneamente a los pines de salida no sincronizados. Así es como se implementan los pines de dirección en las unidades RAM: la lógica de direccionamiento es combinacional e independiente del reloj.

También es posible declarar la palabra clave `CLOCKED` con una lista vacía de pines. Esta declaración estipula que el chip puede cambiar su estado interno dependiendo del reloj, pero su comportamiento de entrada-salida será combinativo, independientemente del reloj.

**Chips compuestos secuenciales:** la propiedad `CLOCKED` solo se puede definir explícitamente en chips integrados. ¿Cómo, entonces, sabe el simulador que una determinada pieza del chip es secuencial? Si el chip no está incorporado, se dice que se registra cuando se cronometran una o más de sus partes del chip. La propiedad de reloj se verifica de forma recursiva, hasta el final de la jerarquía de chips, donde un chip incorporado puede ser cronometrado explícitamente. Si se encuentra un chip de este tipo, hace que todos los chips que dependen de él (hacia arriba en la jerarquía) estén "sincronizados". Por lo tanto, en la computadora Hack, todos los chips que incluyen una o más partes de chip DFF, ya sea directa o indirectamente, se sincronizan.

Vemos que si un chip no está incorporado, no hay forma de saber a partir de su código HDL si es secuencial o no. *Consejos de mejores prácticas:* El arquitecto del chip debe proporcionar esta información en la documentación de la API del chip.

**Bucles de retroalimentación:** Si la entrada de un chip se alimenta de una de las salidas del chip, ya sea directamente o a través de una ruta

(posiblemente larga) de dependencias, decimos

que el chip contiene un bucle de *retroalimentación*. Por ejemplo, considere las siguientes dos instrucciones de parte de chip:

```
Not (in=loop1, out=loop1) // Invalid feedback loop
DFF (in=loop2, out=loop2) // Valid feedback loop
```

En ambos ejemplos, un pin interno (loop1 o loop2) intenta alimentar la entrada del chip desde su salida, creando un bucle de retroalimentación. La diferencia entre los dos ejemplos es que Not es un chip combinacional, mientras que DFF es secuencial. En el ejemplo No, loop1 crea una dependencia instantánea e incontrolada entre la entrada y la salida, a veces denominada *carrera de datos*. Por el contrario, en el caso de DFF, la dependencia de entrada y salida creada por loop2 se retrasa por el reloj, ya que la entrada de entrada del DFF se declara sincronizada. Por lo tanto,  $out(t)$  no es una función de  $in(t)$  sino más bien de  $in(t - 1)$ .

Cuando el simulador evalúa un chip, comprueba recursivamente si sus diversas conexiones implican bucles de retroalimentación. Para cada bucle, el simulador verifica si el bucle pasa por un pasador cronometrado en algún lugar del camino. Si es así, se permite el bucle. De lo contrario, el simulador detiene el procesamiento y emite un mensaje de error. Esto se hace para evitar carreras de datos incontroladas.

---

## A2.5 Visualización de chips

Los chips incorporados pueden estar *habilitados por GUI*. Estos chips cuentan con efectos secundarios visuales diseñados para animar algunas de las operaciones del chip. Cuando el simulador evalúa una pieza de chip potenciada por GUI, muestra una imagen gráfica en la pantalla. Usando esta imagen, que puede incluir elementos interactivos, el usuario puede inspeccionar el estado actual del chip o cambiarlo. Las acciones permitidas habilitadas por la GUI son determinadas y posibles por el desarrollador de la implementación del chip incorporado.

La versión actual del simulador de hardware cuenta con los siguientes chips integrados con GUI:

**ALU:** Muestra las entradas, la salida y la función calculada actualmente del Hack ALU.

**Registros (ARegister, DRegister, PC):** Muestra el contenido del registro, que puede ser modificado por el usuario.

**Chips RAM:** Muestra una imagen desplazable, similar a una matriz, que muestra el contenido de todas las ubicaciones de memoria, que pueden ser modificadas por el usuario. Si el contenido de una ubicación de memoria cambia durante la simulación, la entrada respectiva en la GUI también cambia.

**Chip ROM (ROM32K):** La misma imagen similar a una matriz que la de los chips RAM, además de un icono que permite cargar un programa de lenguaje de máquina desde un archivo de texto externo. (El chip ROM32K sirve como memoria de instrucciones de la computadora Hack).

**Chip de pantalla:** muestra una ventana de 256 filas por 512 columnas que simula la pantalla física. Si, durante una simulación, uno o más bits en el mapa de memoria de la pantalla residente en la RAM cambian, los píxeles respectivos en la GUI de la pantalla también cambian. Este bucle de actualización continua está integrado en la implementación del simulador.

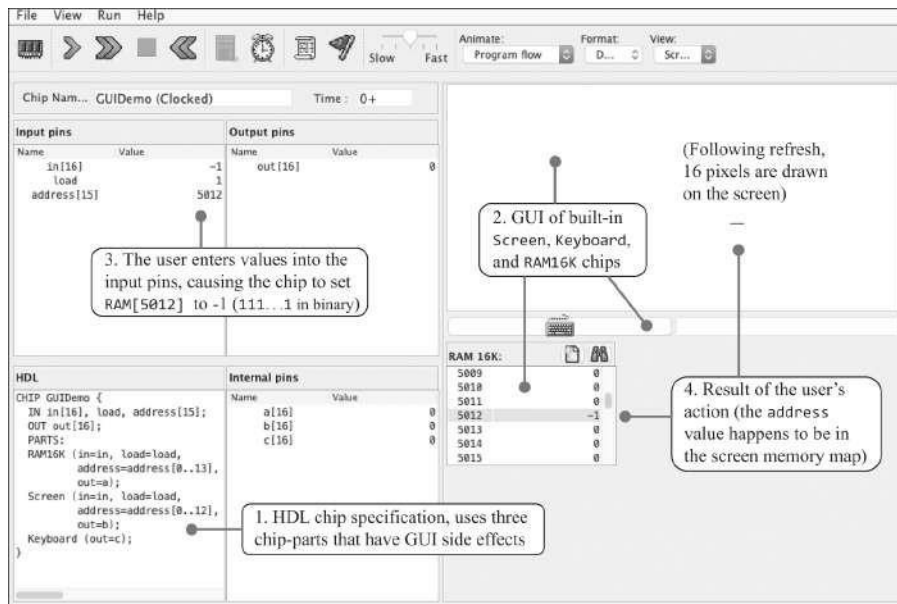
**Chip de teclado:** Muestra un icono de teclado. Al hacer clic en este icono, se conecta el teclado real de su computadora al chip simulado. A partir de este momento, cada tecla presionada en el teclado real es interceptada por el chip simulado y su código binario aparece en el mapa de memoria del teclado residente en la RAM. Si el usuario mueve el foco del mouse a otra área en la GUI del simulador, el control del teclado se restaura en la computadora real.

La Figura A2.5 presenta un chip que utiliza tres partes de chip potenciadas por GUI. La Figura A2.6 muestra cómo el simulador maneja este chip. La lógica del chip GUIDemo alimenta su entrada en dos destinos: registrar la dirección del número en la parte del chip RAM16K y registrar la dirección del número en la parte del chip de la pantalla. Además, la lógica del chip alimenta los valores de salida de sus tres partes del chip a los pines internos "sin salida" a, b y c. Estas conexiones sin sentido están diseñadas para un solo propósito: ilustrar cómo el simulador trata con piezas de chip integradas y potenciadas por GUI.

```
// Demo of GUI-empowered chips.
// The logic of this chip is meaningless, and is used merely to force
// the simulator to display the GUI effects of its built-in chip-parts.

CHIP GUIDemo {
    IN in[16], load, address[15];
    OUT out[16];
    PARTS:
    RAM16K(in=in, load=load, address=address[0..13], out=a);
    Screen(in=in, load=load, address=address[0..12], out=b);
    Keyboard(out=c);
}
```

**Figura A2.5** Un chip que activa piezas de chip potenciadas por GUI.



**Figura A2.6** Demostración de chips potenciados por GUI. Dado que el programa HDL cargado utiliza piezas de chip habilitadas para GUI (paso 1), el simulador renderiza sus respectivas imágenes de GUI (paso 2). Cuando el usuario cambia los valores de los pines de entrada del chip (paso 3), el simulador refleja estos cambios en las respectivas GUI (paso 4).

Observe cómo los cambios efectuados por el usuario (paso 3) afectan a la pantalla (paso 4). La línea horizontal encerrada en un círculo que se muestra en la pantalla es el efecto secundario visual de almacenar -1 en la ubicación de memoria 5012. Dado que el código binario complementario de dos bits de 16 bits de -1 es 1111111111111111, la computadora dibuja 16 píxeles a partir de la columna 320 de la fila 156, que resulta ser la pantalla

coordinadas asociadas con la dirección RAM 5012. La asignación de direcciones de memoria *en coordenadas de pantalla* (fila, columna) se especifica en el capítulo 4 (sección 4.2.5).

---

## Guía de supervivencia de HDL A2.6

En esta sección se proporcionan consejos prácticos sobre cómo desarrollar chips en HDL mediante el simulador de hardware suministrado. Los consejos se enumeran sin ningún orden en particular. Recomendamos leer esta sección una vez, de principio a fin, y luego consultarla según sea necesario.

**Chip:** Su carpeta `nand2tetris/projects` incluye trece subcarpetas, denominadas 01, 02, ..., 13 (correspondientes a los números de capítulo correspondientes). Las carpetas de proyectos de hardware son 01, 02, 03 y 05. Cada carpeta de proyecto de hardware contiene un conjunto de archivos auxiliares HDL suministrados, uno para cada chip que tenga que construir. Los archivos HDL suministrados no contienen implementaciones; Construir estas implementaciones es de lo que se trata el proyecto. Si no construye estas fichas en el orden en que se describen en el libro, puede tener dificultades. Por ejemplo, supongamos que comienza el proyecto 1 construyendo el chip Xor. Si su implementación de `Xor.hdl` incluye, por ejemplo, piezas de chip `And` y `Or`, y aún no ha implementado `And.hdl` y `Or.hdl`, su programa `Xor.hdl` no funcionará incluso si su implementación es perfectamente correcta.

Sin embargo, tenga en cuenta que si la carpeta del proyecto no incluye archivos `And.hdl` y `Or.hdl`, el programa `Xor.hdl` funcionará correctamente. El simulador de hardware, que es un programa Java, presenta implementaciones integradas de todos los chips necesarios para construir la computadora Hack (con la excepción de la CPU y los chips de computadora). Cuando el simulador evalúa una pieza de chip, digamos `And`, busca un archivo `And.hdl` en la carpeta actual. En este punto hay tres posibilidades:

- No se encuentra ningún archivo HDL. En este caso, la implementación incorporada del chip se activa, cubriendo la implementación HDL faltante.
- Se encuentra un archivo HDL de código auxiliar. El simulador intenta ejecutarlo. Si no se encuentra una implementación, se produce un error en la ejecución.

- Se encuentra un archivo HDL, con una implementación HDL. El simulador lo ejecuta, informando de los errores, si los hay, lo mejor que puede.

*Consejos de mejores prácticas:* puede hacer una de dos cosas. Intente implementar los chips en el orden presentado en el libro y en las descripciones de los proyectos. Dado que los chips se discuten de abajo hacia arriba, desde los chips básicos hasta los más complejos, no encontrará problemas de implementación de pedidos de chips, siempre que, por supuesto, complete cada implementación de chip correctamente antes de pasar a implementar la siguiente.

Una alternativa recomendada es crear una subcarpeta llamada, por ejemplo, stubs, y mover todos los archivos .hdl stub suministrados a ella. A continuación, puede mover cada archivo auxiliar en el que desee trabajar a su carpeta de trabajo, uno por uno. Cuando haya terminado de implementar un chip con éxito, muévelo a, digamos, una subcarpeta completa. Esta práctica obliga al simulador a usar siempre chips integrados, ya que la carpeta de trabajo incluye solo el archivo .hdl en el que está trabajando (así como los archivos .tst y .cmp suministrados).

**Archivos HDL y scripts de prueba:** el archivo .hdl en el que está trabajando y su archivo de script de prueba .tst asociado deben estar ubicados en la misma carpeta. Cada script de prueba proporcionado comienza con un comando de carga que carga el archivo .hdl que se supone que debe probar. El simulador siempre busca este archivo en la carpeta actual.

En principio, el menú Archivo del simulador permite al usuario cargar, de forma interactiva, tanto un archivo .hdl como un archivo de script .tst. Esto puede crear problemas potenciales. Por ejemplo, puede cargar el archivo .hdl en el que está trabajando en el simulador y, a continuación, cargar un script de prueba desde otra carpeta. Cuando ejecuta el script de prueba, es posible que cargue una versión diferente del programa HDL en el simulador (posiblemente, un archivo stub). En caso de duda, inspeccione el panel denominado HDL en la GUI del simulador para comprobar qué código HDL está cargado actualmente. *Consejos de prácticas recomendadas:* Utilice el menú Archivo del simulador para cargar un archivo .hdl o un archivo .tst, pero no ambos.

**Probar chips de forma aislada:** En algún momento puede convencerse de que su chip es correcto, aunque todavía no pase la prueba. De hecho, es posible que el chip esté perfectamente implementado, pero una de sus partes del chip no lo está. Además, un chip que pasó su prueba con éxito puede fallar cuando se usa como



astilla por otra astilla. Una de las mayores limitaciones inherentes del diseño de hardware es que los scripts de prueba, especialmente aquellos que prueban chips complejos, no pueden garantizar que el chip probado funcione perfectamente en todas las circunstancias.

La buena noticia es que siempre puede diagnosticar qué parte del chip está causando el problema. Cree una subcarpeta de prueba y copie en ella solo las tres

.hdl, .tst y .out relacionados con el chip que está construyendo actualmente. Si la implementación del chip pasa su prueba en esta subcarpeta tal cual (permitiendo que el simulador use las partes de chip integradas predeterminadas), debe haber un problema con una de las implementaciones de partes de chip, es decir, con uno de los chips que construyó anteriormente en este proyecto. Copie los otros chips en esta carpeta de prueba, uno por uno, y repita la prueba hasta que encuentre el chip problemático.

**Errores de sintaxis HDL:** el simulador de hardware muestra errores en la barra de estado inferior. En computadoras con pantallas pequeñas, estos mensajes a veces están en la parte inferior de la pantalla, no visibles. Si carga un programa HDL y no aparece nada en el panel HDL, pero no se ve ningún mensaje de error, este puede ser el problema. Su computadora debe tener una forma de mover la ventana, usando el teclado. Por ejemplo, en Windows, use Alt+Espacio, M y las teclas de dirección.

**Pines desconectados:** el simulador de hardware no considera que los pines desconectados sean errores. De forma predeterminada, establece cualquier pin de entrada o salida no conectado en false (valor binario 0). Esto puede causar errores misteriosos en las implementaciones de su chip.

Si un pin de salida de su chip es siempre 0, asegúrese de que esté conectado correctamente a otro pin en su programa. En particular, verifique los nombres de los pines internos ("cables") que alimentan este pin, ya sea directa o indirectamente. Los errores tipográficos son particularmente peligrosos aquí, ya que el simulador no arroja errores en los cables desconectados. Por ejemplo, considere la instrucción `Foo(..., suma = sol)`, donde se supone que la salida de `suma` de `Foo` canaliza su valor a un pin interno. De hecho, el simulador creará felizmente un pin interno llamado `sol`. Ahora, si se suponía que el valor de `sum` alimentaba el pin de salida del chip implementado, o el pin de entrada de otra parte del chip, este pin será de hecho 0, *siempre*, ya que nada se canalizará desde `Foo` en adelante.

En resumen, si un pin de salida es siempre 0, o si una de las partes del chip no parece estar funcionando correctamente, verifique la ortografía de todos los nombres de pines relevantes y verifique que todos los pines de entrada de la parte del chip estén conectados.

**Pruebas personalizadas:** para cada archivo *chip.hdl* que tenga que completar, la carpeta del proyecto también incluye un script de prueba proporcionado, denominado *chip.tst*, y un archivo de comparación, denominado *chip.cmp*. Una vez que su chip comience a generar salidas, su carpeta también incluirá un archivo de salida llamado *chip.out*. Si su chip falla en el script de prueba, no olvide consultar el archivo *.out*. Inspeccione los valores de salida enumerados y busque pistas sobre el error. Si por alguna razón no puede ver el archivo de salida en la GUI del simulador, siempre puede inspeccionarlo usando un editor de texto.

Si lo desea, puede realizar sus propias pruebas. Copie el script de prueba proporcionado en, por ejemplo, *MyTestChip.tst* y modifique los comandos de script para obtener más información sobre el comportamiento de su chip. Comience cambiando el nombre del archivo de salida en la línea del archivo de salida y eliminando la línea de comparación. Esto hará que la prueba siempre se ejecute hasta su finalización (de forma predeterminada, la simulación se detiene cuando una línea de salida no está de acuerdo con la línea correspondiente en el archivo de comparación). Considere modificar la línea de lista de salida para mostrar las salidas de sus pines internos.

El Apéndice 3 documenta el Lenguaje de Descripción de Pruebas (TDL) que presenta todos estos comandos.

**Pines internos de subtransporte (indexación):** Esto no está permitido. Los únicos pines de bus que se pueden indexar son los pines de entrada y salida del chip implementado o los pines de entrada y salida de sus partes de chip. Sin embargo, existe una solución para los pines de bus internos de sub-buse. Para motivar la solución alternativa, aquí hay un ejemplo que no funciona:

```
CHIP Foo {  
  IN in[16];  
  OUT out;  
  PARTS:  
    Not16 (in=in, out=notIn);  
    Or8Way (in=notIn[4..11], out=out); // Error: internal bus cannot be indexed.  
}
```

Posible solución, usando la solución alternativa:

```
Not16 (in=in, out[4..11]=notIn);  
Or8Way (in=notIn, out=out); // Works!
```

**Salidas múltiples:** A veces es necesario dividir el valor de varios bits de un pin de bus en dos buses. Esto se puede hacer mediante el uso de múltiples enlaces out=.

Por ejemplo:

```
CHIP Foo {  
    IN in[16];  
    OUT out[8];  
    PARTS:  
        Not16 (in=in, out[0..7]=low8, out[8..15]=high8); // Splitting the out value  
        Bar8Bit (a=low8, b=high8, out=out);  
}
```

A veces, es posible que desee generar un valor y también usarlo para cálculos adicionales. Esto se puede hacer de la siguiente manera:

```
CHIP Foo {  
    IN a, b, c;  
    OUT out1, out2;  
    PARTS:  
        Bar (a=a, b=b, out=x, out=out1); // Bar's output feeds the out1 output of Foo  
        Baz (a=x, b=c, out=out2); // A copy of Bar's output also feeds Baz's a input  
}
```

**Piezas de chips "autocompletadas" (más o menos):** Las firmas de todos los chips mencionados en este libro se enumeran en el apéndice 4, que también tiene una versión basada en la web (en [www.nand2tetris.org](http://www.nand2tetris.org)). Para usar una pieza de chip en una implementación de chip, copie y pegue la firma del chip del documento en línea en su programa HDL y, a continuación, rellene los enlaces que faltan. Esta práctica ahorra tiempo y minimiza los errores de escritura.

# Apéndice 3: Lenguaje de descripción de pruebas

Los errores son los portales del descubrimiento.

—James Joyce (1882–1941)

Las pruebas son un elemento de importancia crítica del desarrollo de sistemas, y uno que generalmente recibe poca atención en la educación en ciencias de la computación. En Nand to Tetris nos tomamos muy en serio las pruebas. Creemos que antes de comenzar a desarrollar un nuevo módulo *P* de hardware o software, primero se debe desarrollar un módulo *T* diseñado para probarlo. Además, *T* debería ser parte del contrato de desarrollo de *P*. Por lo tanto, para cada chip o sistema de software especificado en este libro, suministramos programas de prueba oficiales, escritos por nosotros. Aunque puede probar su trabajo de la forma que mejor le parezca, el contrato es tal que, en última instancia, su implementación debe pasar *nuestras* pruebas.

Con el fin de agilizar la definición y ejecución de las numerosas pruebas dispersas por todos los proyectos del libro, diseñamos un lenguaje uniforme de descripción de pruebas. Este lenguaje funciona casi igual en todas las herramientas relevantes suministradas en Nand to Tetris: el *simulador de hardware* utilizado para simular y probar chips escritos en HDL, el emulador de *CPU* utilizado para simular y probar programas de lenguaje de máquina, y el *emulador de VM* utilizado para simular y probar programas escritos en el lenguaje VM, que suelen ser programas Jack compilados.

Cada uno de estos simuladores cuenta con una interfaz gráfica de usuario que permite probar el chip o programa cargado de forma interactiva, o por lotes, utilizando un script de prueba. Un script de prueba es una secuencia de comandos que cargan un módulo de hardware o software en el simulador correspondiente y someten el módulo a una serie de escenarios de prueba planificados previamente. Además, los scripts de prueba cuentan con comandos

para imprimir los resultados de la prueba y compararlos con los resultados deseados, tal como se define en los archivos de comparación suministrados. En resumen, un script de prueba permite una prueba sistemática, replicable y documentada del código subyacente, un requisito invaluable en cualquier proyecto de desarrollo de hardware o software.

En Nand to Tetris, no esperamos que los alumnos escriban scripts de prueba. Todos los scripts de prueba necesarios para probar todos los módulos de hardware y software mencionados en el libro se suministran con los materiales del proyecto. Por lo tanto, el propósito de este apéndice no es enseñarle cómo escribir scripts de prueba, sino ayudarlo a comprender la sintaxis y la lógica de los scripts de prueba proporcionados. Por supuesto, puede personalizar los scripts suministrados y crear otros nuevos, como desee.

---

### A3.1 Directrices generales

Las siguientes pautas de uso se aplican a todas las herramientas de software y scripts de prueba.

**Formato y uso del archivo:** El acto de probar un módulo de hardware o software implica cuatro tipos de archivos. Aunque no es obligatorio, recomendamos que los cuatro archivos tengan el mismo prefijo (nombre de archivo):

*Xxx.yyy*: donde *Xxx* es el nombre del módulo probado y *yyy* es hdl, hack, asm o vm, que representan, respectivamente, una definición de chip escrita en HDL, un programa escrito en el lenguaje de máquina Hack, un programa escrito en el lenguaje ensamblador Hack o un programa escrito en el lenguaje de máquina virtual VM

*Xxx.tst*: un script de prueba que guía al simulador a través de una serie de pasos diseñados para probar el código almacenado en *Xxx*

*Xxx.out*: un archivo de salida opcional en el que los comandos de script pueden escribir los valores actuales de las variables seleccionadas durante la simulación

*Xxx.cmp*: un archivo de comparación opcional que contiene los *valores deseados* de las variables seleccionadas, es decir, los valores que *debe* generar la simulación si el módulo se implementa correctamente

Todos estos archivos deben guardarse en la misma carpeta, que puede llamarse convenientemente *Xxx*. En la documentación y las descripciones de todos los simuladores, el término "carpeta actual" se refiere a la carpeta desde la que se ha abierto el último archivo en el entorno del simulador.

**Espacio en blanco:** se omiten los caracteres de espacio, los caracteres de nueva línea y los comentarios de los scripts de prueba (*archivos Xxx.tst*). Los siguientes formatos de comentario pueden aparecer en scripts de prueba:

```
// Comment to end of line
/* Comment until closing */
/** API documentation comment */
```

Los scripts de prueba no distinguen entre mayúsculas y minúsculas, excepto los nombres de archivos y carpetas.

**Uso:** Para cada módulo de hardware o software *Xxx* en Nand to Tetris suministramos un archivo de script *Xxx.tst* y un archivo de comparación *Xxx.cmp*. Estos archivos están diseñados para probar la implementación de *Xxx*. En algunos casos, también suministramos una versión esquelética de *Xxx*, por ejemplo, una interfaz HDL a la que le falta una implementación. Todos los archivos de todos los proyectos son archivos de texto sin formato que deben verse y editarse con editores de texto sin formato.

Normalmente, iniciará una sesión de simulación cargando el archivo de script *Xxx.tst* proporcionado en el simulador correspondiente. El primer comando del script normalmente carga el código almacenado en el módulo probado *Xxx*. A continuación, opcionalmente, vienen los comandos que inicializan un archivo de salida y especifican un archivo de comparación. Los comandos restantes del script ejecutan las pruebas reales.

**Controles de simulación:** Cada uno de los simuladores suministrados cuenta con un conjunto de menús e iconos para controlar la simulación.

**Menú Archivo:** Permite cargar en el simulador un programa relevante (archivo .hdl, archivo .hack, archivo .asm, archivo .vm o un nombre de carpeta) o un script de prueba (archivo .tst). Si el usuario no carga un script de prueba, el simulador carga un script de prueba predeterminado (que se describe a continuación).

**Icono de reproducción :** indica al simulador que ejecute el siguiente paso de simulación, como se especifica en el script de prueba cargado

actualmente.

Icono de pausa : indica al simulador que detenga la ejecución del script de prueba cargado actualmente. Útil para inspeccionar varios elementos del entorno simulado.

Icono de avance rápido : indica al simulador que ejecute todos los comandos del script de prueba cargado actualmente.

Icono de parada : indica al simulador que detenga la ejecución del script de prueba cargado actualmente.

Icono de rebobinado : Indica al simulador que restablezca la ejecución del script de prueba cargado actualmente, es decir, que esté listo para comenzar a ejecutar el script de prueba desde su primer comando en adelante.

Tenga en cuenta que los iconos del simulador enumerados anteriormente no "ejecutan el código". En su lugar, ejecutan el script de prueba, que ejecuta el código.

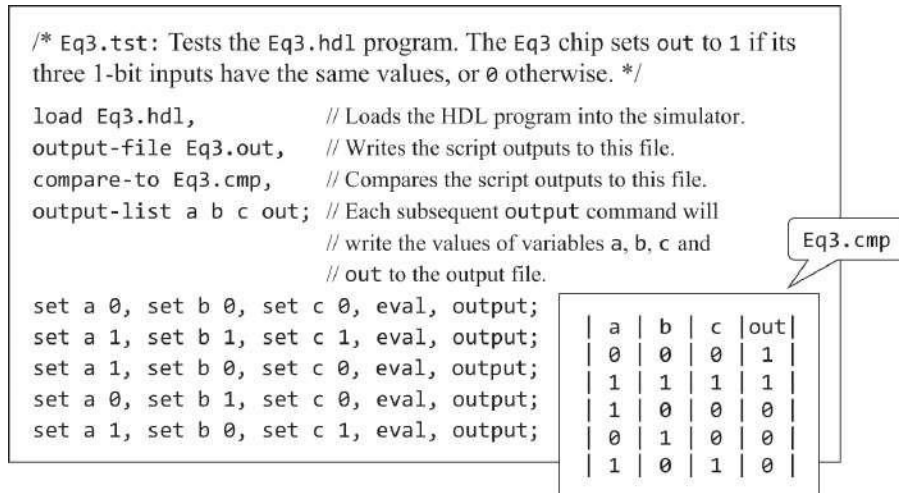
---

### A3.2 Prueba de chips en el simulador de hardware

El simulador de hardware suministrado está diseñado para probar y simular definiciones de chip escritas en el lenguaje de descripción de hardware (HDL) descrito en el apéndice 2. El capítulo 1 proporciona antecedentes esenciales sobre el desarrollo y las pruebas de chips; por lo tanto, recomendamos leerlo primero.

**Ejemplo:** [La Figura A2.1](#) en el apéndice 2 presenta un chip Eq3, diseñado para verificar si tres entradas de 1 bit son iguales. [La Figura A3.1](#) presenta Eq3.tst, un script diseñado para probar el chip, y Eq3.cmp, un archivo de comparación que contiene el resultado esperado de esta prueba.





**Figura A3.1** Script de prueba y archivo de comparación (ejemplo).

Un script de prueba normalmente comienza con algunos comandos de configuración, seguidos de una serie de pasos de simulación, cada uno de los cuales termina con un punto y coma. Un paso de simulación generalmente le indica al simulador que vincule los pines de entrada del chip a los valores de prueba, evalúe la lógica del chip y escriba los valores de las variables seleccionadas en un archivo de salida designado.

El chip Eq3 tiene tres entradas de 1 bit; por lo tanto, una prueba exhaustiva requeriría ocho escenarios de prueba. El tamaño de una prueba exhaustiva crece exponencialmente con el tamaño de entrada. Por lo tanto, la mayoría de los scripts de prueba prueban solo un subconjunto de valores de entrada representativos, como se muestra en la figura.

**Tipos de datos y variables:** los scripts de prueba admiten dos tipos de datos: *enteros* y *cadenas*. Las constantes enteras se pueden expresar en formato decimal (prefijo %D), que es el formato predeterminado, formato binario (prefijo %B) o hexadecimal (prefijo %X). Estos valores siempre se traducen en sus valores binarios equivalentes de complemento de dos. Por ejemplo, considere los siguientes comandos:

```
set a1 %B1111111111111111
set a2 %XFFFF
set a3 %D-1
set a4 -1
```

Las cuatro variables se establecen en el mismo valor: 1111111111111111 en binario, que resulta ser el binario, la representación del complemento de dos -1 en decimal.

Los valores de cadena se especifican mediante un prefijo %S y deben ir entre comillas dobles. Las cadenas se utilizan estrictamente con fines de impresión y no se pueden asignar a variables.

El reloj bifásico del simulador de hardware (utilizado solo en las pruebas de chips secuenciales) emite una serie de valores denotados 0, 0+, 1, 1+, 2, 2+, 3, 3+, y así sucesivamente. La progresión de estos ciclos de *reloj* (también llamados *unidades de tiempo*) se puede controlar mediante dos comandos de script llamados tick y tock. Un tic mueve el valor del reloj de  $t$  a  $t+$ , y un toque de  $t+to$  para  $t+1$ , traer la siguiente unidad de tiempo. La unidad de tiempo actual se almacena en una variable de sistema denominada time, que es de solo lectura.

Los comandos de script pueden acceder a tres tipos de variables: pines, variables de chips incorporados y la variable del sistema time.

*Pines*: pines de entrada, salida e internos del chip simulado (por ejemplo, el comando establecido en 0 establece el valor del pin cuyo nombre está en 0)

*Variables de los chips incorporados*: expuestas por la implementación externa del chip (ver [figura A3.2](#).)

| Chip name          | Exposed variables | Data type / range       | Methods                 |
|--------------------|-------------------|-------------------------|-------------------------|
| Register           | Register[ ]       | 16-bit (-32768...32767) | load Xxx.hack / Xxx.asm |
| ARegister          | ARegister[ ]      | 16-bit                  |                         |
| DRegister          | DRegister[ ]      | 16-bit                  |                         |
| PC (prog. counter) | PC[ ]             | 15-bit (0...32767)      |                         |
| RAM8               | RAM8[0...7]       | each entry is 16-bit    |                         |
| RAM64              | RAM64[0...63]     | each entry is 16-bit    |                         |
| RAM512             | RAM512[0...511]   | each entry is 16-bit    |                         |
| RAM4K              | RAM4K[0...4095]   | each entry is 16-bit    |                         |
| RAM16K             | RAM16K[0...16383] | each entry is 16-bit    |                         |
| ROM32K             | ROM32K[0...32767] | each entry is 16-bit    |                         |
| Screen             | Screen[0...16383] | each entry is 16-bit    |                         |
| Keyboard           | Keyboard[ ]       | 16-bit, read-only       |                         |

**Figura A3.2** Variables y métodos de chips clave incorporados en Nand a Tetris.

time: el número de unidades de tiempo que transcurrieron desde que se inició la simulación (una variable de solo lectura)

**Comandos de script:** Un script es una secuencia de comandos. Cada comando termina con una coma, un punto y coma o un signo de exclamación. Estos terminadores tienen la siguiente semántica:

*Coma (,):* Termina un comando de script.

*Punto y coma (;):* Finaliza un comando de guión y un paso de simulación.

Un *paso de simulación* consta de uno o varios comandos de guión. Cuando el usuario indica al simulador que *realice un solo paso* mediante el menú del simulador o el icono de reproducción, el simulador ejecuta el script desde el comando actual hasta que se alcanza un punto y coma, momento en el que se pausa la simulación.

*Signo de exclamación (!):* Finaliza un comando de script y detiene la ejecución del script. El usuario puede reanudar posteriormente la ejecución del script a partir de ese momento. Normalmente se usa con fines de depuración.

A continuación, agrupamos los comandos de script en dos secciones conceptuales: *comandos de configuración*, utilizados para cargar archivos e inicializar configuraciones, y *comandos de simulación*, utilizados para guiar el simulador a través de las pruebas reales.

## Comandos de configuración

**load Xxx.hdl:** Carga el programa HDL almacenado en Xxx.hdl en el simulador. El nombre de archivo debe incluir la extensión .hdl y no debe incluir una especificación de ruta de acceso. El simulador intentará cargar el archivo desde la carpeta actual y, en su defecto, desde la carpeta tools/builtInChips.

**output-file Xxx.out:** Indica al simulador que escriba los resultados de la salida comandos en el archivo con nombre, que debe incluir una extensión .out. El archivo de salida se creará en la carpeta actual.

**output-list v1, v2, ...:** Especifica qué escribir en el archivo de salida cuando el archivo de salida

output en el script (hasta la siguiente lista de salida comando, si lo hubiera). Cada valor de la lista es un nombre de variable seguido de una especificación de formato. El comando también produce una sola línea de encabezado, que consta de los nombres de las variables, que se escribe en el archivo de salida. Cada elemento *v* en la lista de salida tiene la sintaxis *varName format padL.len.padR* (sin espacios). Esta directiva indica al simulador que escriba los caracteres de espacio padL y, a continuación,

el valor actual de la variable *varName*, utilizando el método

formato especificado y *columnas len* , luego *espacios padR* y, finalmente, el símbolo divisor |. El *formato* puede ser %B (binario), %X (hexa), %D (decimal) o

%S (cadena). El formato predeterminado es %B1.1.1.

Por ejemplo, el chip CPU.hdl de la plataforma Hack tiene un pin de entrada llamado reset, un pin de salida llamado pc (entre otros) y una parte del chip llamada DRegister (entre otros). Si queremos rastrear los valores de estas entidades durante la simulación, podemos usar algo como el siguiente comando:

```
Output-list time%S1.5.1 // The system variable time
            reset%B2.1.2 // One of the chip's input pins
            pc%D2.3.1    // One of the chip's output pins
            DRegister[]%X3.4.4 // The internal state of this chip-part
```

(Las variables de estado de los chips incorporados se explican a continuación). Este comando output-list puede terminar produciendo el siguiente resultado, después de dos comandos de salida posteriores:

| time | reset | pc | DRegister[] |  |
|------|-------|----|-------------|--|
| 20+  | 0     | 21 | FFFF        |  |
| 21   | 0     | 22 | FFFF        |  |

**compare-to Xxx.cmp:** especifica que la línea de salida generada por cada comando de salida posterior debe compararse con su línea correspondiente en el archivo de comparación especificado (que debe incluir la extensión .cmp). Si dos líneas no son iguales, el simulador muestra un mensaje de error y detiene la ejecución del script. Se supone que el archivo de comparación está presente en la carpeta actual.

## Comandos de simulación

**set varName value:** Asigna el valor a la variable. La variable es un pin o una variable interna del chip simulado o una de sus partes de chip. Los anchos de bits del valor y la variable deben ser compatibles.

**eval:** Indica al simulador que aplique la lógica del chip a los valores actuales de los pines de entrada y calcule los valores de salida resultantes.

**output:** hace que el simulador siga la siguiente lógica:

1. Obtener los valores actuales de todas las variables enumeradas en la última lista de salida mandar.
2. Cree una línea de salida utilizando el formato especificado en la última lista de salida mandar.
3. Escriba la línea de salida en el archivo de salida.
4. (Si se ha declarado previamente un archivo de comparación mediante un comando `compare-to`): Si la línea de salida difiere de la línea actual del archivo de comparación, muestre un mensaje de error y detenga la ejecución del script.
5. Avance los cursores de línea del archivo de salida y el archivo de comparación.

**tick:** finaliza la primera fase de la unidad de tiempo actual (ciclo de reloj).

**tock:** Finaliza la segunda fase de la unidad de tiempo actual y se embarca en la primera fase de la siguiente unidad de tiempo.

**repeat *n* {commands}:** Indica al simulador que repita los comandos entre llaves *n* veces. Si se omite *n*, el simulador repite los comandos hasta que la simulación se haya detenido por algún motivo (por ejemplo, cuando el usuario hace clic en el icono Detener).

**while *booleanCondition* {commands}:** Indica al simulador que repita los comandos entre llaves siempre que *booleanCondition* sea true. La condición es de la forma *x op y* donde *x* e *y* son constantes o nombres de variables y *op* es =, >, <, >=, <= o <>. Si *x* e *y* son cadenas, *op* puede ser = o <>.

**echo *text*:** muestra el *texto* en la línea de estado del simulador. El texto debe ir entre comillas dobles.

**clear-echo:** borra la línea de estado del simulador.

**valor varName del *punto de interrupción*:** comienza a comparar el valor actual de la variable especificada con el *valor* especificado después de la ejecución de cada comando de script posterior. Si la variable contiene el *valor especificado*, la ejecución se detiene y se muestra un mensaje. De lo contrario, la ejecución continúa normalmente. Útil para fines de depuración.

**clear-breakpoints:** borra todos los puntos de interrupción definidos anteriormente.

**Argumento (s) del método *builtInChipName* (s):** ejecuta el método especificado de la pieza de chip incorporada especificada utilizando los argumentos proporcionados. El diseñador de un chip incorporado puede proporcionar métodos que permitan al usuario (o un script de prueba) manipular el chip simulado. Ver [figura A3.2](#).

**Variables de los chips incorporados:** Los chips pueden ser implementados por programas HDL o por módulos ejecutables suministrados externamente. En este último caso, se dice que el chip está *incorporado*. Los chips incorporados pueden facilitar el acceso al estado del chip utilizando la sintaxis *chipName[varName]*, donde *varName* es una variable específica de la implementación que debe documentarse en la API del chip. Ver [figura A3.2](#).

Por ejemplo, considere el conjunto de comandos de script `RAM16K[1017]` 15. Si `RAM16K` es el chip simulado actualmente, o una parte del chip simulado actualmente, este comando establece su número de ubicación de memoria 1017 en 15. Y, dado que el chip `RAM16K` incorporado tiene efectos secundarios de GUI, el nuevo valor también se reflejará en la imagen visual del chip.

Si un chip incorporado mantiene un estado interno de un solo valor, se puede acceder al valor actual del estado a través de la notación *chipName[]*. Si el estado interno es un vector, se usa la notación *chipName[i]*. Por ejemplo, al simular el chip `Register` incorporado, se pueden escribir comandos de script como `set Register[] 135`. Este comando establece el estado interno del chip en 135; en la siguiente unidad de tiempo, el chip `Register` se comprometerá con este valor y su pin de salida comenzará a emitirlo.

**Métodos de chips integrados:** Los chips integrados también pueden exponer *métodos* que pueden ser utilizados por comandos de scripting. Por ejemplo, en la computadora Hack, los programas residen en una unidad de memoria de instrucciones implementada por el chip incorporado `ROM32K`. Antes de ejecutar un programa de lenguaje de máquina en el Hack

computadora, el programa debe cargarse en este chip. Para facilitar este servicio, la implementación incorporada de ROM32K presenta un método de carga que permite cargar un archivo de texto que contiene instrucciones de lenguaje de máquina. Se puede acceder a este método usando un comando de script como ROM32K load *fileName.hack*.

**Ejemplo final:** Terminamos esta sección con un script de prueba relativamente complejo diseñado para probar el chip de computadora superior de la computadora Hack.

Una forma de probar el chip de computadora es cargar un programa de lenguaje de máquina en él y monitorear los valores seleccionados a medida que la computadora ejecuta el programa, una instrucción a la vez. Por ejemplo, escribimos un programa de lenguaje de máquina que calcula el máximo de RAM[0] y RAM[1] y escribe el resultado en RAM[2]. El programa se almacena en un archivo llamado Max.hack.

Tenga en cuenta que en el nivel bajo en el que estamos operando, si dicho programa no se ejecuta correctamente, puede deberse a que el programa tiene errores o porque el hardware tiene errores (o, tal vez, el script de prueba tiene errores o el simulador de hardware tiene errores). Para simplificar, supongamos que todo está libre de errores, excepto, posiblemente, el chip de computadora simulado .

Para probar el chip de computadora usando el programa Max.hack, escribimos un script de prueba llamado ComputerMax.tst. Este script carga Computer.hdl en el simulador de hardware y luego carga el programa Max.hack en su parte de chip ROM32K. Una forma razonable de verificar si el chip funciona correctamente es la siguiente: coloque algunos valores en RAM [0] y RAM [1], reinicie la computadora, ejecute el reloj suficientes ciclos e inspeccione la RAM [2]. Esto, en pocas palabras, es para lo que está diseñado el guión de la [figura A3.3](#).



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> /* ComputerMax.tst script. Uses a Max.hack program that sets RAM[2] to max(RAM[0], RAM[1]). */  // Loads Computer and sets up for the simulation: load Computer.hdl, output-file ComputerMax.out, compare-to ComputerMax.cmp, output-list RAM16K[0] RAM16K[1] RAM16K[2];  // Loads Max.hack into the ROM32K chip-part: ROM32K load Max.hack,  // Sets the first 2 cells of the RAM16K chip-part // to test values: set RAM16K[0] 3, set RAM16K[1] 5, output;  // Runs enough clock cycles to complete the // program's execution: repeat 14 {     tick, tock,     output; }  // (Script continues on the right) </pre> | <pre> // Sets up for another test, using other values.  // Resets the Computer: Done by setting // reset to 1, and running the clock // in order to commit the Program Counter // (PC, a sequential chip) to the new reset value: set reset 1, tick, tock, output;  // Sets reset to 0, loads new test values, and // runs enough clock cycles to complete the // program's execution: set reset 0, set RAM16K[0] 23456, set RAM16K[1] 12345, output; repeat 14 {     tick, tock,     output; } </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Figura A3.3** Prueba del chip de computadora superior.

¿Cómo podemos saber que catorce ciclos de reloj son suficientes para ejecutar este programa? Esto se puede encontrar por prueba y error, comenzando con un valor grande y observando que las salidas de la computadora se estabilizan después de un tiempo, o analizando el comportamiento en tiempo de ejecución del programa cargado.

**Script de prueba predeterminado:** cada simulador de Nand a Tetris presenta un script de prueba predeterminado. Si el usuario no carga un script de prueba en el simulador, se usa el script de prueba predeterminado. El script de prueba predeterminado del simulador de hardware se define de la siguiente manera:

```

// Default test script of the hardware simulator:
repeat {
    tick,
    tock;
}

```

---

## Referencia A3.3 Probar programas de lenguaje de máquina en el emulador de CPU

A diferencia de la *Simulador de hardware* cuál es un de uso general diseñado para soportar la construcción de cualquier plataforma de hardware, el

*El emulador de CPU* es una herramienta de un solo propósito, diseñada para simular la ejecución de programas de lenguaje de máquina en una plataforma específica: la computadora Hack. Los programas se pueden escribir en el lenguaje de máquina simbólico o binario Hack descrito en el capítulo 4.

Como de costumbre, la simulación involucra cuatro archivos: el programa probado (*Xxx.asm* o *Xxx.hack*), un script de prueba (*Xxx.tst*), un archivo de salida opcional (*Xxx.out*) y un archivo de comparación opcional (*Xxx.cmp*). Todos estos archivos residen en la misma carpeta, normalmente llamada *Xxx*.

**Ejemplo:** Considere el programa de multiplicación *Mult.hack*, diseñado para efectuar

$RAM[2] = RAM[0] * RAM[1]$ . Supongamos que queremos probar este programa en el emulador de CPU. Una forma razonable de hacerlo es poner algunos valores en  $RAM[0]$  y  $RAM[1]$ , ejecutar el programa e inspeccionar  $RAM[2]$ . Esta lógica se lleva a cabo mediante el script de prueba que se muestra en la [figura A3.4](#).

```

// Loads the program and sets up for the simulation:
load Mult.hack,
output-file Mult.out,
compare-to Mult.cmp,
output-list RAM[2]%D2.6.2;

// Sets the first 2 RAM cells to test values:
set RAM[0] 2,
set RAM[1] 5;

// Runs enough clock cycles to complete the program's execution:
repeat 20 {
    ticktock;
}
output;

// Re-runs the program, with different test values:
set PC 0,
set RAM[0] 8,
set RAM[1] 7;

// Mult.hack is based on a naïve repetitive addition algorithm,
// so greater multiplicands require more clock cycles:
repeat 50 {
    ticktock;
}
output;

```

**Figura A3.4** Probar un programa de lenguaje de máquina en el emulador de CPU.

**Variables:** Los comandos de secuencias de comandos que se ejecutan en el emulador de CPU pueden acceder a los siguientes elementos de la computadora Hack:

|         |                                                                                                                                                              |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A:      | Current value of the address register (unsigned 15-bit)                                                                                                      |
| D:      | Current value of the data register (16-bit)                                                                                                                  |
| PC:     | Current value of the Program Counter (unsigned 15-bit)                                                                                                       |
| RAM[i]: | Current value of RAM location <i>i</i> (16-bit)                                                                                                              |
| time:   | Number of <i>time units</i> (also called <i>clock cycles</i> , or <i>ticktocks</i> ) that elapsed since the simulation started (a read-only system variable) |

**Comandos:** El emulador de CPU admite todos los comandos descritos en la sección A3.2, excepto los siguientes cambios:

*load progName:* Donde *progName* es *Xxx.asm* o *Xxx.hack*. Este comando carga un programa de lenguaje de máquina (para ser probado) en la memoria de instrucciones simulada. Si el programa está escrito en ensamblador, el simulador lo traduce a binario, sobre la marcha, como parte de la ejecución del comando *load progName*.

*eval:* No aplicable en el emulador de CPU.

*Argumento (s) del método builtInChipName (s):* no se aplica en el emulador de CPU.

*tickTock:* Este comando se usa en lugar de *tick* y *tock*. Cada *ticktock* avanza el reloj una unidad de tiempo (ciclo).

### Script de prueba predeterminado

```
// Default test script of the CPU emulator:
repeat {
    ticktock;
}
```

---

## A3.4 Prueba de programas de VM en el emulador de VM

El emulador de VM *suministrado* es una implementación Java de la máquina virtual especificada en los capítulos 7 y 8. Se puede utilizar para simular la ejecución de programas de VM, visualizar sus operaciones y mostrar los estados de los segmentos de memoria virtual afectados.

Un programa de máquina virtual consta de uno o varios archivos *.vm*. Por lo tanto, la simulación de un programa VM involucra el programa probado (un solo *archivo Xxx.vm* o una *carpeta Xxx que contiene uno o más archivos .vm*) y, opcionalmente, un script de prueba (*Xxx.tst*), un archivo de comparación (*Xxx.cmp*) y un archivo de salida (*Xxx.out*). Todos estos archivos residen en la misma carpeta, normalmente llamada *Xxx*.

**Segmentos de memoria virtual:** los comandos de VM *push* y *pop* están diseñados para manipular *segmentos de memoria virtual* (argumento, local, etc.). Estos

segmentos deben asignarse a la RAM del host, una tarea que el emulador de VM lleva a cabo como efecto secundario de la simulación de la ejecución de los comandos de VM

call, function y return.

**Código de inicio:** cuando el traductor de VM traduce un programa de VM, genera código de lenguaje de máquina que establece el puntero de pila en 256 y, a continuación, llama a la función Sys.init, que inicializa las clases del sistema operativo y llama a Main.main. De manera similar, cuando se indica al emulador de VM que ejecute un programa de VM (una colección de una o más funciones de VM), se programa para comenzar a ejecutar la función Sys.init. Si no se encuentra una función de este tipo en el código de máquina virtual cargado, el emulador está programado para comenzar a ejecutar el primer comando en el código de máquina virtual cargado.

Esta última convención se agregó al emulador de VM para admitir pruebas unitarias del traductor de VM, que abarca dos capítulos de libros y proyectos. En el proyecto 7, creamos un traductor de VM básico que solo controla los comandos push, pop y aritméticos sin controlar los comandos de llamada a funciones. Si queremos ejecutar tales programas, debemos anclar de alguna manera los segmentos de memoria virtual en la RAM del host, al menos los segmentos mencionados en el código de VM simulado. Convenientemente, esta inicialización se puede lograr mediante comandos de script que manipulan los punteros que controlan las direcciones RAM base de los segmentos virtuales. Usando estos comandos de script, podemos anclar los segmentos virtuales en cualquier lugar que queramos en la RAM del host.

**Ejemplo:** El archivo FibonacciSeries.vm contiene una secuencia de comandos de VM que calculan los primeros  $n$  elementos de la serie de Fibonacci. El código está diseñado para operar en dos argumentos:  $n$  y la dirección de memoria inicial en la que se deben almacenar los elementos calculados. El script de prueba que se muestra en [la figura A3.5](#) prueba este programa usando los argumentos 6 y 4000.

```

/* The FibonacciSeries.vm program computes the first  $n$  Fibonacci numbers.
In this test  $n = 6$ , and the numbers will be written to RAM addresses 4000 to 4005. */

load FibonacciSeries.vm,
output-file FibonacciSeries.out,
compare-to FibonacciSeries.cmp,
output-list RAM[4000]%D1.6.2 RAM[4001]%D1.6.2 RAM[4002]%D1.6.2
          RAM[4003]%D1.6.2 RAM[4004]%D1.6.2 RAM[4005]%D1.6.2;

// The program's code contains no function/call/return commands.
// Therefore, the script initializes the stack, local and argument segments explicitly:

set SP 256,
set local 300,
set argument 400;

// Sets the first argument to  $n = 6$ , the second argument to the address where the series
// will be written, and runs enough VM steps for completing the program's execution:
set argument[0] 6,
set argument[1] 4000;
repeat 140 {
    vmstep;
}
output;

```

**Figura A3.5** Prueba de un programa de VM en el emulador de VM.

**Variables:** los comandos de scripting que se ejecutan en el emulador de VM pueden acceder a los siguientes elementos de la máquina virtual:

### Contenido de los segmentos de VM:

|                  |                                                      |
|------------------|------------------------------------------------------|
| local[ $i$ ]:    | Value of the $i$ -th element of the local segment    |
| argument[ $i$ ]: | Value of the $i$ -th element of the argument segment |
| this[ $i$ ]:     | Value of the $i$ -th element of the this segment     |
| that[ $i$ ]:     | Value of the $i$ -th element of the that segment     |
| temp[ $i$ ]:     | Value of the $i$ -th element of the temp segment     |

### Punteros de segmentos de máquina virtual:

|           |                                                 |
|-----------|-------------------------------------------------|
| local:    | Base address of the local segment in the RAM    |
| argument: | Base address of the argument segment in the RAM |
| this:     | Base address of the this segment in the RAM     |
| that:     | Base address of the that segment in the RAM     |

## Variables específicas de la implementación:

|                  |                                                                                                                                                                                                                                                                                                                                                            |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RAM[i]:          | Value of the $i$ -th location of the host RAM                                                                                                                                                                                                                                                                                                              |
| SP:              | Value of the stack pointer                                                                                                                                                                                                                                                                                                                                 |
| currentFunction: | Name of the currently executing function (read-only)                                                                                                                                                                                                                                                                                                       |
| line:            | Contains a string of the form<br><i>currentFunctionName.lineIndexInFunction</i> (read-only).<br>For example, when execution reaches the third line of the function <code>Sys.init</code> , the <code>line</code> variable contains the value <code>Sys.init.3</code> . Can be used for setting breakpoints in selected locations in the loaded VM program. |

**Comandos:** el emulador de VM admite todos los comandos descritos en la Sección A3.2, excepto los siguientes cambios:

`load source`: donde el *parámetro de origen* opcional es *Xxx.vm*, un archivo que contiene código de máquina virtual, o *Xxx*, *el nombre de una carpeta que contiene uno o más*

*.vm* (en cuyo caso se cargan todos, uno tras otro). Si el archivo *.vm* Los archivos se encuentran en la carpeta actual, se puede omitir el argumento `source`.

`tick / tock`: No aplicable.

`vmstep`: Simula la ejecución de un único comando de VM y avanza al siguiente comando del código.

## **Script predeterminado**

```
// Default script of the VM emulator:  
repeat {  
    vmStep;  
}
```

## Apéndice 4: El conjunto de chips de hackeo

Las fichas están ordenadas alfabéticamente por nombre. En la versión en línea de este documento, disponible en [www.nand2tetris.org](http://www.nand2tetris.org), este formato API es útil: para usar una pieza de chip, copie y pegue la firma del chip en su programa HDL, luego complete los enlaces que faltan (también llamados *conexiones*).



```

Add16(a= ,b= ,out= ) /* Adds up two 16-bit two's complement values */
ALU(x= ,y= ,zx= ,nx= ,zy= ,ny= ,f= ,no= ,out= ,zr= ,ng= ) /* Hack ALU */
And(a= ,b= ,out= ) /* And gate */
And16(a= ,b= ,out= ) /* 16-bit And */
ARegister(in= ,load= ,out= ) /* Address register (built-in) */
Bit(in= ,load= ,out= ) /* 1-bit register */
CPU(inM= ,instruction= ,reset= ,outM= ,writeM= ,addressM= ,pc= ) /* Hack CPU */
DFF(in= ,out= ) /* Data flip-flop gate (built-in) */
DMux(in= ,sel= ,a= ,b= ) /* Routes the input to one out of two outputs */
DMux4Way(in= ,sel= ,a= ,b= ,c= ,d= ) /* Routes the input to one out of four outputs */
DMux8Way(in= ,sel= ,a= ,b= ,c= ,d= ,e= ,f= ,g= ,h= ) /* Routes the input to one out of 8 outputs */
DRegister(in= ,load= ,out= ) /* Data register (built-in) */
HalfAdder(a= ,b= ,sum= , carry= ) /* Adds up two bits */
FullAdder(a= ,b= ,c= ,sum= ,carry= ) /* Adds up three bits */
Inc16(in= ,out= ) /* Sets out to in + 1 */
Keyboard(out= ) /* Keyboard memory map (built-in) */
Memory(in= ,load= ,address= ,out= ) /* Data memory of the Hack platform (RAM) */
Mux(a= ,b= ,sel= ,out= ) /* Selects between two inputs */
Mux16(a= ,b= ,sel= ,out= ) /* Selects between two 16-bit inputs */
Mux4Way16(a= ,b= ,c= ,d= ,sel= ,out= ) /* Selects between four 16-bit inputs */
Mux8Way16(a= ,b= ,c= ,d= ,e= ,f= ,g= ,h= ,sel= ,out= ) /* Selects between eight 16-bit inputs */
Nand(a= ,b= ,out= ) /* Nand gate (built-in) */
Not(in= ,out= ) /* Not gate */
Not16(in= ,out= ) /* 16-bit Not */
Or(a= ,b= ,out= ) /* Or gate */
Or16(a= ,b= ,out= ) /* 16-bit Or */
Or8Way(in= ,out= ) /* 8-way Or */
PC(in= ,load= ,inc= ,reset= ,out= ) /* Program Counter */
RAM8(in= ,load= ,address= ,out= ) /* 8-word RAM */
RAM64(in= ,load= ,address= ,out= ) /* 64-word RAM */
RAM512(in= ,load= ,address= ,out= ) /* 512-word RAM */
RAM4K(in= ,load= ,address= ,out= ) /* 4K RAM */
RAM16K(in= ,load= ,address= ,out= ) /* 16K RAM */
Register(in= ,load= ,out= ) /* 16-bit register */
ROM32K(address= ,out= ) /* Instruction memory of the Hack platform (ROM, built-in) */
Screen(in= ,load= ,address= ,out= ) /* Screen memory map (built-in) */
Xor(a= ,b= ,out= ) /* Xor gate */

```

## **Apéndice 5: El conjunto de personajes de Hack**

|           |       |        |        |                 |
|-----------|-------|--------|--------|-----------------|
| 32: space | 56: 8 | 80: P  | 104: h | 127: DEL        |
| 33: !     | 57: 9 | 81: Q  | 105: i | 128: newLine    |
| 34: "     | 58: : | 82: R  | 106: j | 129: backSpace  |
| 35: #     | 59: ; | 83: S  | 107: k | 130: leftArrow  |
| 36: \$    | 60: < | 84: T  | 108: l | 131: upArrow    |
| 37: %     | 61: = | 85: U  | 109: m | 132: rightArrow |
| 38: &     | 62: > | 86: V  | 110: n | 133: downArrow  |
| 39: '     | 63: ? | 87: W  | 111: o | 134: home       |
| 40: (     | 64: @ | 88: X  | 112: p | 135: end        |
| 41: )     | 65: A | 89: Y  | 113: q | 136: pageUp     |
| 42: *     | 66: B | 90: Z  | 114: r | 137: pageDown   |
| 43: +     | 67: C | 91: [  | 115: s | 138: insert     |
| 44: ,     | 68: D | 92: /  | 116: t | 139: delete     |
| 45: -     | 69: E | 93: ]  | 117: u | 140: esc        |
| 46: .     | 70: F | 94: ^  | 118: v | 141: f1         |
| 47: /     | 71: G | 95: _  | 119: w | 142: f2         |
| 48: 0     | 72: H | 96: `  | 120: x | 143: f3         |
| 49: 1     | 73: I | 97: a  | 121: y | 144: f4         |
| 50: 2     | 74: J | 98: b  | 122: z | 145: f5         |
| 51: 3     | 75: K | 99: c  | 123: { | 146: f6         |
| 52: 4     | 76: L | 100: d | 124:   | 147: f7         |
| 53: 5     | 77: M | 101: e | 125: } | 148: f8         |
| 54: 6     | 78: N | 102: f | 126: ~ | 149: f9         |
| 55: 7     | 79: O | 103: g |        | 150: f10        |
|           |       |        |        | 151: f11        |
|           |       |        |        | 152: f12        |

# Apéndice 6: La API de Jack OS

El lenguaje Jack es compatible con ocho clases estándar que proporcionan servicios básicos del sistema operativo como asignación de memoria, funciones matemáticas, captura de entrada y renderizado de salida. Este apéndice documenta la API de estas clases.

---

## Matemática

Esta clase proporciona funciones matemáticas comúnmente necesarias.

`function int multiply(int x, int y):` Devuelve el producto de  $x$  e  $y$ . Cuando un compilador Jack detecta el operador de multiplicación `*` en el código del programa, lo maneja invocando esta función. Por lo tanto, las expresiones de Jack  $x * y$  y la llamada a la función `Math.multiply(x,y)` devuelven el mismo valor.

`function int divide(int x, int y):` Devuelve la parte entera de  $x / y$ . Cuando un compilador Jack detecta el operador de división `/` en el código del programa, lo maneja invocando esta función. Por lo tanto, las expresiones de Jack  $x / y$  y la llamada de función `Math.divide(x,y)` devuelven el mismo valor.

`function int min(int x, int y):` Devuelve el mínimo de  $x$  e  $y$ . `function`

`int max(int x, int y):` Devuelve el máximo de  $x$  e  $y$ .

`function int sqrt(int x):` Devuelve la parte entera de la raíz cuadrada de  $x$ .

---

## Cuerda

Esta clase representa cadenas de valores char y proporciona servicios de procesamiento de cadenas comúnmente necesarios.

constructor String new(int maxLength): Construye una nueva cadena vacía con una longitud máxima de maxLength y una longitud inicial de 0.

method void dispose(): Elimina esta cadena.

method int length(): Devuelve el número de caracteres de esta cadena.

method char charAt(int i): Devuelve el carácter en la i-ésima ubicación de esta cadena.

method void setCharAt(int i, char c): Establece el carácter en la i-ésima ubicación de esta cadena en c.

method String appendChar(char c): Agrega c al final de esta cadena y devuelve esta cadena.

method void eraseLastChar(): Borra el último carácter de esta cadena.

method int intValue(): Devuelve el valor entero de esta cadena hasta que se detecta un carácter que no sea un dígito.

method void setInt(int val): Establece esta cadena para que contenga una representación del valor dado.

function char backSpace(): Devuelve el carácter de retroceso.

function char doubleQuote(): Devuelve el carácter de comillas dobles. function char newLine(): Devuelve el carácter de nueva línea.

---

## Arreglo

En el lenguaje Jack, las matrices se implementan como instancias de la clase OS Array. Una vez declarados, se puede acceder a los elementos de la matriz usando la sintaxis arr[i]. Las matrices Jack no se escriben: cada elemento de la matriz puede contener un tipo de datos primitivo o un tipo de objeto, y diferentes elementos en la misma matriz pueden tener diferentes tipos.

function Array new(int size): Construye una nueva matriz del tamaño dado.

method void dispose(): Elimina esta matriz.

---

## Salida

Esta clase proporciona funciones para mostrar caracteres. Asume una pantalla orientada a caracteres que consta de 23 filas (indexadas 0... 22, de arriba a abajo) de 64 caracteres cada uno (indexado 0... 63, de izquierda a derecha). La ubicación del personaje superior izquierdo en la pantalla está indexada (0,0). Cada carácter se muestra representando en la pantalla una imagen rectangular de 11 píxeles de altura y 8 píxeles de ancho (que incluye márgenes para el espaciado entre caracteres y el interlineado). Si es necesario, las imágenes de mapa de bits ("fuente") de todos los caracteres se pueden encontrar inspeccionando el código dado de la clase Output. Un cursor visible, implementado como un pequeño cuadrado relleno, indica dónde se mostrará el siguiente carácter.

function void moveCursor(int i, int j): Mueve el cursor a la j-ésima columna de la columna i-

th y anula el carácter que se muestra allí.

function void printChar(char c): Muestra el carácter en la ubicación del cursor y avanza el cursor una columna hacia adelante.

function void printString(String s): Muestra la cadena que comienza en la ubicación del cursor y avanza el cursor adecuadamente.

function void printInt(int i): Muestra el entero que comienza en la ubicación del cursor y avanza el cursor adecuadamente.

function void println(): Avanza el cursor hasta el principio de la siguiente línea.

function void backSpace(): Mueve el cursor una columna hacia atrás.

---

## Pantalla

Esta clase proporciona funciones para mostrar formas gráficas en la pantalla. La pantalla física de Hack consta de 256 filas (indexadas 0... 255, de arriba a abajo) de 512 píxeles cada uno (indexado 0... 511, de izquierda a derecha). El píxel superior izquierdo de la pantalla está indexado (0,0).

function void clearScreen(): Borra toda la pantalla.

function void setColor(boolean b): Establece el color actual. Este color se utilizará en todas las llamadas posteriores a la función drawXxx. El negro está representado por verdadero,

blanco por falso.

function void drawPixel(int x, int y): Dibuja el píxel (x,y) usando el color actual.

function void drawLine(int x1, int y1, int x2, int y2): Dibuja una línea de píxel (x1,y1) a píxel (x2,y2) usando el color actual.

function void drawRectangle(int x1, int y1, int x2, int y2): Dibuja un rectángulo relleno cuya esquina superior izquierda es (x1,y1) y la esquina inferior derecha es (x2,y2) usando el color actual.

function void drawCircle(int x, int y, int r): Dibuja un círculo relleno de radio  $r \leq 181$  alrededor de (x,y) usando el color actual.

---

## Teclado

Esta clase proporciona funciones para leer entradas de un teclado estándar.

function char keyPressed(): Devuelve el carácter de la tecla presionada actualmente en el teclado; si no se presiona ninguna tecla, devuelve 0. Reconoce todos los valores del juego de caracteres Hack (consulte el apéndice 5). Estos incluyen los caracteres newLine (128, valor devuelto de String.newLine()), backSpace (129, valor devuelto de String.backSpace()), leftArrow (130), upArrow (131), rightArrow (132), downArrow (133), home (134), end (135), pageUp (136), pageDown (137), insertar (138), suprimir (139), esc (140) y f1–f12 (141–152).

function char readChar(): Espera hasta que se presiona y suelta una tecla del teclado, luego muestra el carácter correspondiente en la pantalla y devuelve el carácter.

function String readLine(String message): Muestra el mensaje, lee desde el teclado la cadena de caracteres introducida hasta que se detecta un carácter newLine, muestra la cadena y devuelve la cadena. También maneja los retrocesos del usuario.

function int readInt(String message): Muestra el mensaje, lee desde el teclado la cadena de caracteres introducida hasta que se detecta un carácter newLine, muestra la cadena en la pantalla y devuelve su valor entero hasta que se detecta el primer carácter no dígito de la cadena introducida. También maneja los retrocesos del usuario.

---

## Memoria

Esta clase proporciona servicios de administración de memoria. La Hack RAM consta de 32.768 palabras, cada una con un número binario de 16 bits.

function int peek(int address): Devuelve el valor de RAM[address].

function void poke(int address, int value): Establece RAM[address] en el valor dado.

function Array alloc(int size): Encuentra un bloque de RAM disponible del tamaño dado y devuelve su dirección base.

function void deAlloc(Array o): Desasigna el objeto dado, que se convierte en una matriz. En otras palabras, hace que el bloque de RAM que se inicia en esta dirección esté disponible para futuras asignaciones de memoria.

---

## Sistema

Esta clase proporciona servicios básicos de ejecución de programas.

function void halt(): Detiene la ejecución del programa.

function void error(int errorCode): Muestra el código de error, utilizando el formato ERR<errorCode> y detiene la ejecución del programa.

function void wait(int duration): Espera aproximadamente milisegundos de duración y devuelve.



# Índice

Ada. Véase [King-Noel, Augusta](#)  
[Ada](#) Adder  
    abstracción, [35–36](#)  
    Aplicación, [41](#)  
Dirección, [46](#), [85](#)  
Registro de direcciones, [63](#), [64](#), [87](#), [89](#)  
Espacio de direcciones, [82](#), [92–93](#)  
ALU. Véase [Unidad Lógica Aritmética \(ALU\)](#) Y puerta, [9](#), [14](#)  
Comandos aritmético-lógicos, [147](#), [164](#)  
Unidad lógica aritmética (ALU), [3](#), [4](#), [36](#), [86](#), [94](#)  
Ensamblador, [7](#), [103–11](#)  
    API, [110–112](#)  
    Antecedentes, [103–105](#)  
    implementación, [109–110](#)  
    comandos de macro (ver [Instrucción de macro](#))  
    mnemotécnico, [104](#)  
    pseudoinstrucciones, [105–106](#)  
    Tabla de símbolos, [108–109](#)  
    traducción, [108–109](#)  
    ensamblador de dos pasadas, [108](#)  
Lenguajes ensambladores, [63](#)  
Especificación del lenguaje ensamblador (Hack), [106–107](#)  
  
Simulación de comportamiento, [24–25](#)  
Números binarios, [32–33](#)  
Editor de mapas de bits, [188](#)  
Álgebra de Boole, [9–12](#), [277–278](#)  
Aritmética booleana, [31–43](#)  
    operaciones aritméticas, [31–32](#)  
    Suma binaria, [33–34](#)  
    números binarios, [32–33](#)  
    Operadores booleanos, [10](#)  
    Llevar lookahead, [43](#)  
    tamaño de palabra  
    fijo, [33](#)

- implementación, [41–42](#)
- números negativos, [34–35](#)
- Desbordamiento, [34](#)
- Complemento de base, [34](#)
- números binarios con signo, [34–35](#)
- método de complemento de dos, [34](#)
- Simplificación de funciones booleanas, [278](#)
- Síntesis de funciones booleanas, [277–281](#)
  - Forma normal disyuntiva (DNF), [279](#)
  - Nand, poder expresivo de, [279–281](#)
  - sintetizar funciones booleanas, [278–279](#)
- Bootstrapping, [118](#), [167–168](#)
- Ramificación, [62](#), [149–151](#)
  - condicional, [68](#)
  - lenguaje de máquina, [62](#), [67](#)
  - incondicional, [67](#)
  - Control de programas de VM, [149–151](#), [158](#)
- Chips integrados
  - Resumen, [25–27](#)
  - HDL, [287–289](#)
  - métodos de, [306](#)
  - variables de, [305–306](#)
- Autobús, [286–289](#), [296](#)
- Cadena de llamadas, [152](#)
- Llevar lookahead, [43](#)
- Unidad central de procesamiento (CPU), [86–87](#)
  - abstracción, [89–90](#)
  - Aritmética booleana, [31](#)
  - Hackear computadora, [89–90](#)
  - Implementación, [94–96](#)
  - lenguaje de máquina, [62](#)
  - memoria, [59](#)
  - von Neumann arquitectura y, [84](#)
- Generación de código. *Consulte* [Compilador](#),  
emulador de CPU [de generación de código](#), [80](#)
  - prueba de programas de lenguaje de máquina en, [chips 308–309](#)
  - Chips integrados, [25–27](#), [287–291](#)
  - Combinatorial, [45](#), [50](#), [289–290](#)
  - Potenciado por GUI, [291–295](#)
  - orden de ejecución, [294](#)
  - secuencial, [45](#), [50](#), [290–293](#)
  - pruebas en simulador de hardware, [301–308](#) dependientes del tiempo (*consulte* [Chips secuenciales](#))
  - visualización independiente del tiempo (*ver* [Chips combinados](#)), [291–293](#)

Conjetura de Church-Turing, 3  
Dibujo circular, 257  
Ciclos de reloj, 302  
Chips combinados, 45, 50, 289–290  
Common Language Runtime (CLR), 146  
Compilador  
    Generación de código, 212–230  
    Manejo de objetos actuales, 221, 223  
    lenguajes orientados a objetos, 121  
    Arquitectura de software, 235–236  
    Especificación, 230  
    tablas de símbolos, 212, 214  
    Análisis de sintaxis, 191–210  
    pruebas, 240–241  
    uso del sistema operativo, 219,  
234  
Compilación (generación de código)  
    matrices, 228–230  
    constructores, 222, 223, 234  
    lógica de flujo de control, 220  
    expresiones, 216–218  
    Métodos, 225–228  
    objetos, 221–228  
    Declaraciones, 219–221  
    cuerdas, 218–219  
    subrutinas, 233  
    variables, 213–216  
Computación de conjuntos de instrucciones complejas (CISC), 101  
Ramificación condicional, 68  
Convertidor, 19

Fecha de cambio (DFF), 46, 49, 52  
Memoria de datos, 66, 85, 88, 92  
Carrera de datos, 55, 291  
Registros de datos, 63, 87  
Lenguaje declarativo, HDL as, 284–285  
Demultiplexor, 9, 20, 23  
Árbol de derivación, 196  
Forma normal disyuntiva (DNF), 279

Ciclo de búsqueda-ejecución, 87  
Flip-flop.  
*Véase Fuentes de cambio de datos*, 247, 258

Puerta. *Véase también tipos específicos*  
    abstracción, 12–13  
    implementación, 14–17  
    primitiva y compuesta, 13–14

Pila global, [154](#)  
Instrucciones de  
Goto,  
    en lenguaje ensamblador,  
    [65](#) en lenguaje VM, [149–151](#)  
Gramática, [193](#), [194](#), [200](#)  
Unidad de procesamiento de gráficos (GPU), [101](#), [255](#)  
Chips potenciados por GUI, [289–295](#)

Hackear la arquitectura de  
    computadoras, [93-94](#), [97](#)  
    CPU, [89-90](#), [94-96](#)  
    Memoria de datos, [92](#), [96-97](#)  
    Dispositivos de entrada/salida, [91](#)  
    Memoria de instrucciones, [90](#)  
    Resumen, [88-89](#)  
Especificación del lenguaje de hackeo. *Consulte* [Especificación de lenguaje ensamblador \(Hack\)](#) Lenguaje de descripción de hardware (HDL),  
    Conceptos básicos, [283–286](#)  
    numeración de bits y sintaxis de bus, [296](#) chips incorporados, [287–289](#)  
    bucles de retroalimentación, [291](#)  
    archivos y scripts de prueba, [295](#)  
    Guía de supervivencia HDL, [294–297](#) buses multibit, [286–287](#)  
    salidas múltiples, [296–297](#)  
    Ejemplo de programa, [284](#)  
    Estructura del programa, [285](#)  
    Chips secuenciales, [291–293](#)  
    Pines internos de sub-busing (indexación), [296–297](#) Errores de sintaxis, [295](#)  
    unidad de tiempo, [289](#)  
    pines desconectados, [295–296](#)  
    Visualización de fichas, [291–293](#)  
Simulador de hardware, [15–18](#), [25–26](#)  
Arquitectura de Harvard, [100](#)  
Tabla hash, [112](#)  
Tipos de datos de lenguaje de programación  
    de alto nivel (Jack), [179–181](#)  
    expresiones, [182–183](#)  
    Construcción/eliminación de objetos, [185](#) Uso del sistema operativo, [175–176](#) Prioridad del operador, [183](#)  
    ejemplos de programas (*ver* [ejemplos de programas de Jack](#)) estructura del programa, [176–178](#)

Biblioteca de clases estándar, [173](#)  
declaraciones, [182–183](#)  
cuerdas, [180](#)

llamadas de subrutina, [184](#)  
variables, [181–182](#)  
escribir solicitudes de Jack, [185–187](#)

Dispositivos de  
  entrada/salida (E/S),  
    [74–75](#), [87](#)  
  mapeado en memoria,  
[87](#) Instrucción  
  decodificación, [95](#)  
  ejecución, [95](#)  
  Buscar, [96](#)  
  memoria, [85](#), [86](#)  
  registro, [87](#)  
Código intermedio, [125](#)  
Pasadores internos, [17](#)

API del sistema operativo  
  (SO) Jack, [317–320](#)  
  Clase de matriz, [318](#)  
  implementación, [261–267](#)  
  Clase de teclado, [319–320](#)  
  Clase de matemáticas, [317](#)  
  Clase de memoria, [320](#)  
  Clase de salida, [318–319](#)  
  Clase de pantalla, [319](#)  
  Especificación, [261](#)  
  Clase de cuerda, [317–318](#)  
  Clase Sys, [320](#)  
Ejemplos de programas Jack, [172–](#)  
  [175](#) Procesamiento de matrices,  
    [173](#)  
  Hola mundo, [172](#)  
  Procesamiento iterativo,  
    implementación [de 173](#)  
  listas enlazadas  
  Orientado a objetos, ejemplo multiclase, [175–](#)  
    [176](#) Orientado a objetos, ejemplo de una sola  
    clase, [173–175](#) capturas de pantalla, [186](#)  
  juego de computadora simple (Square), [186](#), [188](#)  
Lenguaje de programación Jack. Véase [Lenguaje de programación de alto](#)  
[nivel \(Jack\)](#) Java Runtime Environment (JRE), [128](#), [145](#)  
Máquina virtual Java (JVM), [125](#), [132](#), [145](#)

Entrada de teclado, [259–260](#)  
  detección de una pulsación de  
  tecla, [259](#) lectura de un solo  
  carácter, [260](#) lectura de una  
  cadena, [260](#)  
Mapa de memoria del teclado, [74](#),  
[91](#), [292](#) King-Noel, Augusta Ada, [78](#)

Símbolos de etiquetas, [74](#), [106](#)  
Último en entrar, primero en salir  
(LIFO), [129](#), [154](#) Bits menos  
significativos (LSB), [33](#) Léxico, [193](#)  
Dibujo lineal, [256](#)  
Variable local, [132](#), [152](#), [214](#)  
Puertas lógicas. *Ver* [Puerta](#)  
División larga, [250](#)

Direccionamiento en  
  lenguaje de máquina,  
    [67](#)  
  ramificación, [67–68](#)  
  entrada/salida: [74–75](#)  
  memoria, [66](#)  
  Resumen, [61–66](#)  
  Ejemplo de programa, [69–70](#)  
  registros, [67](#)  
  Especificación, [71–73](#)  
  símbolos, [73–74](#)  
  sintaxis y convenciones de  
  archivos, [76](#) variables, [68](#)  
Ejemplos de programas de lenguaje de máquina,  
  [61–82](#) adición, [77](#)  
  direccionamiento (punteros), [78](#)  
  iteración, [77–78](#)  
Instrucción macro, [81](#), [115](#)  
Variables miembro,  
[175](#) Memoria  
  reloj, [46–49](#)  
  Puertas de chancas, [49–50](#), [52](#)  
  implementación, [54–57](#)  
  Resumen, [45–46](#)  
  Memoria de acceso aleatorio (RAM), [53–54](#)  
  registros, [52](#)  
  lógica secuencial, [46–49](#), [50–51](#)  
Algoritmos de asignación de  
memoria, [253](#) Mapa de memoria  
  concepto, [74](#), [87–88](#)  
  teclado, [75](#), [92](#)  
  pantalla, [75](#), [91](#), [188](#)  
Metalenguaje, [194](#)  
Mnemotécnico, [104](#)  
Diseño modular, [5–6](#)  
Bits más significativos (MSB), [33](#)  
Multiplexor, [5](#), [9](#), [20](#)

Puertas Nand, [2](#), [3](#), [279–281](#)

- Función booleana, [19](#)
  - en implementaciones de DFF, [55](#)
  - hardware basado en, [25](#)
  - especificación, [19](#)
- Nor puerta, [28](#), [281](#)
- No puerta, [14](#), [21](#)

- Construcción y eliminación de objetos (Jack), [185](#)
- Lenguajes orientados a objetos, [188](#), [221](#)
- Object variables, [121](#), [221](#)
- Sistema operativo, [245](#)
  - Algoritmos algebraicos, [248–251](#)
  - Salida de caracteres, [258–259](#)
  - eficiencia, [248](#)
  - Algoritmos geométricos, [255–258](#)
  - Gestión del teclado (*consulte* Sistema operativo, gestión de memoria), [259–260](#)
  - Administración de memoria, [252–254](#)
  - Resumen, [245–247](#)
  - Peek / Poke, [254](#)
    - manejo de cuerdas, [251–252](#)
- Optimización, [275](#)
- O puerta, [9](#), [14](#)

- Variables de parámetros, [181](#)
- Analizador, [110](#), [140](#)
- Árbol de análisis, [196](#)
- Análisis sintáctico. *Véase* [Análisis de sintaxis](#)
- Peek and poke, [254](#), [272](#)
- Dibujo de píxeles, [255](#)
- Puntero, [78–79](#)
- Símbolos predefinidos, [73–74](#), [106](#)
- Lenguajes de procedimiento, [225](#)
- Control del programa. *Consulte* [Proyectos, máquina virtual, control de programas](#)
- Contador de programas, [87](#), [89](#), [96](#)
- Proyectos
  - Ensamblador (Proyecto 6), [113–114](#)
  - Aritmética booleana (proyecto 2), [42](#)
  - Lógica booleana (proyecto 1), [27–28](#)
  - Compilador, generación de código (Project 11), [237–241](#)
  - Compilador, Análisis de sintaxis, (Proyecto 10), [206–209](#)
  - Arquitectura de computadoras (Proyecto 5), [98–99](#)
  - Lenguaje de alto nivel (Proyecto 9), [187–188](#)
  - Lenguaje de máquina (Proyecto 4), [78–80](#)
  - memoria (proyecto 3), [58](#)
  - Sistema operativo (Proyecto 12), [267](#)
  - Máquina virtual, control de programas (Proyecto 7), [165–168](#)



Traductor de VM, procesamiento de pila (proyecto 8), [142–144](#) Pseudoinstrucciones, [76](#), [105](#)  
Comandos push/pop, [133](#), [164](#)  
Máquina virtual Python (PVM), [146](#)

Complemento de Radix, [34](#)

Memoria de acceso aleatorio (RAM), [3](#), [7](#), [53](#), [56](#)

fichas, [4](#)

derivación del término, [85](#)

dispositivo, descripción

de, [5](#) Hackear

computadora, [320](#)

Sistema operativo, [263](#)

Memoria de solo lectura (ROM), [88](#), [100](#)

Análisis de descenso recursivo, [198](#)

Computación de conjunto de instrucciones reducido (RISC), [101](#) Variable de referencia, [221](#)

Registros, [5](#)

dirección del remitente, [149](#), [154–155](#), [161–163](#)

Sistema de tiempo de ejecución, [147–148](#)

Mapa de memoria de pantalla, [74](#), [91](#)

Chips secuenciales, [45](#), [50](#), [289–291](#)

Lógica secuencial, [46–51](#)

Shannon, Claude, [13 años](#)

Máquina de apilamiento, [128–132](#)

Desbordamiento de pila, [157](#)

Procesamiento de pilas. Consulte [Proyectos](#), traductor de máquinas virtuales, biblioteca de clases estándar de procesamiento de pila, [118](#), [122](#), [245](#). Véase también [Sistema operativo](#) Hambruna [86](#)

Variables estáticas, [121](#), [181](#), [214](#)

Computadora de programa almacenado,

Hack as, [274](#) Concepto de programa

almacenado, [84](#)

Dispositivos de conmutación, [2](#)

Símbolos (lenguaje de máquina), [73–74](#)

Tabla de símbolos

ensamblador, [103](#), [108](#), [112](#)

Generación de código, [212](#), [214](#)

compilador, [202](#)

Análisis de sintaxis, [191–210](#)

árbol de derivación, [195](#)

gramática, [193](#), [194–196](#), [200](#)

Implementación, [202–206](#)

Tokenizador de Jack, [203](#), [207](#)

Análisis léxico, [193–194](#)

metalenguaje, [194](#)

analizador, [198–199](#), [209](#)

Árbol de análisis, [195](#)

análisis, [196](#)

Análisis de descenso recursivo,

[198](#) tabla de símbolos, [202](#)

Analizador de sintaxis, [200](#)

Lenguaje de descripción de pruebas,

[299–312](#) Descripción general,

[299–301](#)

Chips de prueba, [301–308](#)

prueba de programas de lenguaje de máquina,

[308–309](#) prueba de programas de VM, [310–](#)

[312](#)

Guiones de prueba, [17](#), [295](#), [308](#)

Chips independientes del tiempo. Véase [Fichas](#)

[combinadas](#) Unidades de tiempo, [289](#), [302](#)

Tokenizador, [193](#)

Tabla de la verdad, [10](#), [11](#), [12](#)

Turing, Alan, [126](#)

Máquina de Turing, [84](#)

Método del complemento de dos,

[34](#) Ramificación incondicional, [67](#)

Máquina virtual

API, [139–142](#), [164–165](#)

Antecedentes, [125–128](#), [147–149](#)

ramificación, [149–151](#)

Llamada y devolución de funciones, [151–](#)

[157](#) Implementación, [134–139](#), [159–164](#)

Soporte del sistema operativo, [151](#)

empuje / pop, [especificación](#)

[129–130](#), [149–150](#), [157–159](#)

pila, [128](#)

aritmética de pila, [130–132](#)

segmentos de memoria virtual,

[132](#) emulador de VM, [138–139](#)

Segmentos de memoria virtual, [132](#)

Registros virtuales, [68](#), [74](#), [77](#)

Visualización de chips,

[291–293](#) von Neumann,

John, [66](#)

von Neumann arquitectura, [66](#), [83](#), [84](#)

Tamaño de la palabra, [33](#)

XML, [191–192](#), [206](#), [210](#)

Puerta de Sar, [9](#), [16–17](#)